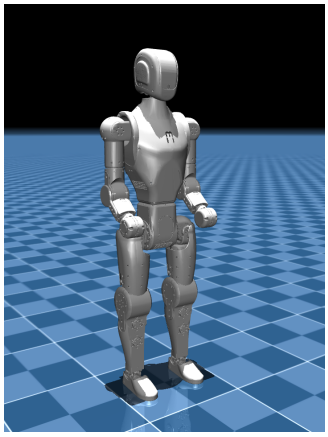
A HANDS-ON ROBOTICS LABORATORY

Engineering an AI Humanoid Robot

From first simulation to a safely commissioned physical research platform

Menlo Research Asimov 1

17 project lessons + 70 in-depth labs
420 executed worked cases
Complete standalone code
Mechanics, control and estimation
Learning, perception and interfaces
Staged physical engineering



Complete textbook edition / 25 October 2026

Prepared for an experienced software engineer beginning robotics. Platform facts are source-linked; exercise assumptions and physical deployment gates are explicit.

Contents

Reader's guide - From software experience to physical engineering 9

The scope of the complete edition . . . 9

A practical study sequence . . . 11

How to read a worked numerical case . . . 12

Running the extended code . . . 13

What numerical validation can establish . . . 15

Managing evidence, changes and scope . . . 15

The EUR 6,000 boundary . . . 16

An honest final capability statement . . . 17

Part I - Engineering the project 18

1. Project vision and how to use this book . . . 18

2. Feasibility assessment and official-source audit . . . 20

3. Complete system architecture . . . 22

4. Dependency-aware roadmap . . . 25

5. Recommended stack and reproducibility policy . . . 27

6. Repository layout and ownership . . . 28

Part II - The simulation laboratory 30

Lesson 0 - Establish a reproducible environment . . . 30

Lesson 1 - My first observable humanoid simulation 33

Lesson 2 - Joint control as a measurable feedback loop 39

Lesson 3 - Sensors, telemetry and state estimation 42

Lesson 4 - Frames, forward kinematics and inverse kinematics 46

Part III - Middleware, balance, learning and perception 51

Lesson 5 - A modular ROS 2 humanoid workspace 51

Lesson 6 - Static balance and the limits of a supported pose 59

Lesson 7 - Reinforcement learning without hidden assumptions 63

Lesson 8 - Standing, walking and turning as research milestones 69

Lesson 9 - Perception and embodied behavior 72

Part IV - From simulation evidence to physical hardware 76

Lesson 10 - Sim-to-real contracts, identification and rollback 76

Lesson 11 - Build a defensible EUR 6,000 budget 81

Lesson 12 - Mechanical design, fabrication and incremental assembly 88

Lesson 13 - Electrical and embedded engineering on a single joint 91

Lesson 14 - Assembly, calibration and supported commissioning 95

Part V - The end-to-end research demonstration 100

Lesson 15 - Recognize an object and execute a validated skill 100

Lesson 16 - Research quality, maintenance and the next build 103

Part VI - Mathematical foundations in depth 107

Laboratory 1 - Units, dimensional reasoning and motor-speed conversion 107

Laboratory 2 - Vectors, projections and joint-axis torque 117

Laboratory 3 - Rotation matrices and the Rodrigues formula 127

Laboratory 4 - Unit quaternions, sign equivalence and incremental attitude 136

Laboratory 5 - Homogeneous transforms and frame-chain round trips 147

Laboratory 6 - Finite-difference Jacobians and derivative verification 156

Laboratory 7 - Least-squares identification and residual diagnosis 166

Laboratory 8 - Discrete integration and numerical stability 176

Laboratory 9 - Monte Carlo uncertainty propagation and repeatability 185

Laboratory 10 - Sampling, aliasing and the difference between signal and display rate . . 195

Part VII - Mechanics, kinematics and contact 205

Laboratory 11 - Link masses, payload shifts and center of mass 205

Laboratory 12 - Inertia tensors and the parallel-axis theorem 215

Laboratory 13 - Angular momentum, kinetic energy and off-axis motion 224

Laboratory 14 - Gravity, acceleration and joint-load budgeting 233

Laboratory 15 - Gear ratio, reflected inertia and backlash dead zones 243

Laboratory 16 - Closed-form two-link inverse kinematics and branch choice 252

Laboratory 17 - Singular values and damped least-squares motion 261

Laboratory 18 - Support polygons, center of pressure and load transfer 271

Laboratory 19 - Linear inverted pendulum prediction and capture-point intuition 281

Laboratory 20 - Friction cones, slip feasibility and contact margins 291

Part VIII - Feedback, trajectories and actuator constraints 301

Laboratory 21 - Second-order joint dynamics and PD tuning 301

Laboratory 22 - Integral action, saturation and anti-windup 310

Laboratory 23 - Minimum-jerk trajectories and duration constraints 321

Laboratory 24 - Position envelopes, slew limits and stopping distance 330

Laboratory 25 - Gravity compensation and model-mismatch torque 340

Laboratory 26 - Torque-speed envelopes and trajectory feasibility 350

Laboratory 27 - Feedback delay, sample-and-hold and tracking degradation 359

Laboratory 28 - First-order thermal models and duty-cycle limits 368

Laboratory 29 - Mechanical energy, electrical demand and regeneration 378

Laboratory 30 - Control deadlines, jitter and tail-latency budgets 388

Part IX - Sensors, geometry and state estimation 398

Laboratory 31 - Encoder counts, quantization, wrapping and velocity estimates 398

Laboratory 32 - Gyroscope bias calibration and attitude drift 407

Laboratory 33 - Complementary attitude filtering and acceleration contamination 416

Laboratory 34 - Scalar Kalman filtering and covariance interpretation 425

Laboratory 35 - Position and velocity estimation with intermittent measurements 436

Laboratory 36 - Camera intrinsics, projection and pixel-error sensitivity 447

Laboratory 37 - Stereo depth, disparity uncertainty and range limits 456

Laboratory 38 - Timestamps, interpolation and asynchronous sensor streams 466

Laboratory 39 - Contact detection, thresholds and hysteresis 475

Laboratory 40 - Zero-velocity updates, foot constraints and slip bias 484

Part X - Software interfaces, timing and supervisory control 494

Laboratory 41 - Typed joint-command contracts and rejection accounting 494

Laboratory 42 - Message queues, delivery policy and stale-command risk 504

Laboratory 43 - Lifecycle activation, preconditions and fault latching 514

Laboratory 44 - Multi-rate controllers, policy decimation and sample age 524

Laboratory 45 - CAN bandwidth screening without invented motor commands 534

Laboratory 46 - A synthetic packet codec and corruption tests 544

Laboratory 47 - Observation ordering, normalization and schema hashes 553

Laboratory 48 - Offline replay, action parity and fault localization 562

Laboratory 49 - Ring buffers, bounded memory and time-window diagnostics 572

Laboratory 50 - Fault persistence, debounce and explicit re-arming 581

Part XI - Learning algorithms, evaluation and embodied skills 591

Laboratory 51 - Markov decision processes and discounted reward 591

Laboratory 52 - Temporal-difference residuals, GAE and episode boundaries 600

Laboratory 53 - PPO clipping, advantage signs and surrogate objectives 610

Laboratory 54 - Gaussian policies, action limits and saturation probability 619

Laboratory 55 - Streaming normalization, numerical stability and evaluation leakage . . . 628

Laboratory 56 - Domain randomization, correlations and feasible parameter sets 638

Laboratory 57 - Fall rates, confidence intervals and finite evidence 648

Laboratory 58 - Gait phase metrics, contact slip and energy accounting 658

Laboratory 59 - Color segmentation, false positives and image-space evidence 668

Laboratory 60 - Allowlisted skills, bounded parameters and behavior outcomes 678

Part XII - Physical engineering, evidence and integration 689

Laboratory 61 - Landed cost, VAT, contingency and scope decisions 689

Laboratory 62 - Dimensional tolerances, stack-ups and manufacturing variation 698

Laboratory 63 - Load paths, bolt-group screening and nominal stress 708

Laboratory 64 - Cable resistance, voltage drop and heat screening 717

Laboratory 65 - Capacitor charging, stored energy and inrush screening 727

Laboratory 66 - Bidirectional calibration and hidden hysteresis 737

Laboratory 67 - Commissioning evidence as a dependency graph 746

Laboratory 68 - Immutable experiment manifests and configuration identity 757

Laboratory 69 - Fault injection, diagnostic coverage and remaining blind spots 766

Laboratory 70 - An integrated synthetic joint with perception, gating and fault recovery . 776

Appendices - Reproducibility and evidence 787

Appendix A. Complete numerical and fault tests 787

Appendix B. Measured preparation results 789

Appendix C. Live telemetry and model-rendering utilities 792

Appendix D. Mermaid architecture sources 794

Appendix E. Units, equations and glossary 796

Appendix F. Source register and update policy 797

Appendix G. Expanded-edition numerical validation 799

Appendix H. Additional primary learning resources 802

Reader's guide - From software experience to physical engineering

The scope of the complete edition

This edition expands the original project manual into a textbook and laboratory reference. The first five parts establish the Asimov project, its source audit, the runnable simulation and the staged physical-build process. Parts VI-XII then develop the mathematics, mechanics, control, estimation, software interfaces, learning and physical engineering in 70 detailed numerical laboratories. Each laboratory has six executed worked cases, a complete standalone program, a controlled parameter study, a graph, validation reasoning and extension exercises.

The 420 worked cases are not 420 successful robot behaviors. They are controlled calculations and synthetic experiments used to understand individual engineering relationships. Some deliberately violate assumptions, miss targets, saturate a controller, expose stale data or produce a negative feasibility margin. Those are useful results when clearly identified. A complete book should teach you how to recognize and explain failure rather than report every calculation as a success.

Read the original practical lessons as the project spine. Use the extended laboratories before implementing the corresponding idea in the humanoid. For example, complete the rotation and transform studies before editing an IMU mounting transform, the feedback and delay studies before changing controller gains, and the observation-schema/replay studies before exporting a policy. The physical-engineering studies help you assess procurement and test plans; their invented numerical values are not specifications for a real robot.

What the additional depth changes

The earlier compact guide told you where a Jacobian belongs in the architecture. This edition also asks you to derive a two-link Jacobian, compare analytic and finite-difference columns, inspect singular values, quantify regularization tradeoffs and reject a misleading inverse solution. The same pattern applies across the curriculum: define the quantity, build an independent check, vary one assumption, measure its effect and decide where the model stops being credible.

The publication uses a conventional 7-by-10-inch technical-book page with readable body typography and separate code typography. Chapters and figures are arranged for reading and reference. Page count is verified after compilation; blank leaves, repeated prompt text and enlarged spacing are not used to meet the length requirement. Repeated equations appear only where a worked calculation needs them, while common tooling is introduced once.

A practical study sequence

Stage	Read and run	Evidence to retain
Orientation	Project architecture, source audit, environment setup	Workstation inventory and pinned model identity
Mathematical foundations	Laboratories 1-10	Units, transform round trips, derivative residuals, sampling and uncertainty studies
Mechanical reasoning	Laboratories 11-20	Mass/inertia checks, load budgets, IK reachability and contact-feasibility results
First robot motion	Practical simulation and joint-control lessons	Headless test results, viewer inspection, named-joint telemetry
Feedback and constraints	Laboratories 21-30	Tracking, saturation, delay, thermal and power analyses
Sensing and estimation	Laboratories 31-40	Calibration, time alignment, filter error and assumption-failure reports
Modular software	ROS practical lesson and Laboratories 41-50	Message rejection, activation, age, replay and fault tests
Learning and perception	RL/perception practical lessons and Laboratories 51-60	Environment checks, held-out results, policy contract and skill approval tests
Physical planning	Procurement/commissioning practical lessons and Laboratories 61-70	Complete quotes, design reviews, measured calibration and actual commissioning records
Integration	Supported simulation demo, then commissioned physical skill	Traceable observation-to-command evidence and tested rollback

The ordering is deliberately revisitable. You do not need to finish every mathematical extension before seeing the first simulator window. You do need to understand a quantity before relying on it to authorize physical motion. Keep simulation experiments, offline numerical studies and actual hardware tests in distinct result folders.

Suggested effort and pacing

A small numerical laboratory can take one to three focused sessions if you read the equation, predict two cases by hand, execute all six, inspect the code and complete one extension. Some topics, such as covariance calibration or contact-constrained estimation, may take longer. These estimates are study planning assumptions; your actual pace depends on mathematics background and the depth of the extensions you choose.

The project is better measured by completed evidence gates than by pages read. A week spent finding a frame-sign error can be more productive than a week of copying several advanced algorithms that remain untested. Maintain a short experiment journal with one hypothesis per entry. Explain what result would disprove the hypothesis before running the test.

How to read a worked numerical case

Start with the parameter contract. Identify which values are physical quantities, which are numerical settings and which are artificial assumptions. Check units before evaluating the equation. Then inspect the output metrics, including unfavorable ones. A large positive power peak can matter even when net energy is small; a small fit residual can coexist with biased calibration; a zero fall rate is meaningless if the base is fixed.

Next follow the numbered derivation. It reports intermediate values produced by the same explicit model so you can locate the first disagreement. Independently reproduce at least one intermediate quantity with a calculator or a short separate script. Independence matters: copying the same wrong formula into both implementation and test does not create strong verification.

Finally ask what would change on the physical robot. Would the input become a noisy sensor estimate? Would a timestamp come from another clock? Would a motor map through a parallel linkage? Would a force be limited by friction, current or temperature? The connection section names the missing evidence rather than silently assuming the numerical model is a physical specification.

Predict, execute, explain

For the first and last cases in each sweep, predict the direction of change before running the program. State the reason using the equation. For example, gear-reflected inertia grows with the square of the ratio, cable heating grows with the square of current in a fixed-resistance model, and first-order stereo depth uncertainty grows with the square of range under fixed focal length and disparity noise.

A prediction can be wrong for a useful reason. Perhaps a nonlinear clipping boundary was crossed, an estimator changed its gain, a discrete timestep rounded a requested delay, or a previously active constraint became inactive. Record the reason. Do not smooth away these boundaries to make a graph look cleaner; they often define the engineering problem.

Running the extended code

All extended modules are saved under /data/Robotics_Project/expanded_lab/modules/. They use NumPy and the Python standard library. The standalone programs accept a case index from zero to five and print parameters, metrics, derivation and interpretive notes as JSON. The catalog runner offers the same experiments by a stable short key and can execute all published cases in one command.

```
cd /data/Robotics_Project
.venv-book/bin/python expanded_lab/run.py list
.venv-book/bin/python expanded_lab/run.py units --case 0
.venv-book/bin/python expanded_lab/run.py test_all --output
    expanded_lab/results/recheck.json
```

The .venv-book path identifies the environment used on this task's host. A virtual environment is not a portable deliverable across operating systems, Python installations or machines. On another machine, create the Lesson 0 environment and substitute its python command. The extended numerical laboratories need only the pinned NumPy baseline; simulator, ROS and learning dependencies are needed for their corresponding practical projects.

The results folder contains the executed JSON cases and the graphs included in the book. Those files are preparation evidence and regression fixtures. Recompute them after editing equations or assumptions. Keep the original results so a changed answer can be traced to a changed source rather than mistaken for an unexplained regression.

Shared numerical runner

The runner is printed here once. Every individual laboratory also includes a standalone module, so you can inspect or run it without relying on dynamic catalog execution.

/data/Robotics_Project/expanded_lab/run.py

```
"""Run one original numerical textbook experiment or validate the full catalog."""
import argparse
import json
from pathlib import Path
from catalog import load_catalog,evaluate

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('experiment',help='catalog key, list, or test_all')
    parser.add_argument('--case',type=int,default=-1)
    parser.add_argument('--output',type=Path)
    args=parser.parse_args();specs=load_catalog()
    if args.experiment=='list':
        for i,s in enumerate(specs,1):print(i,s['key'],s['title'])
        return
    selected=specs if args.experiment=='test_all' else [s for s in specs if
    s['key']==args.experiment]
    if not selected:parser.error('Unknown experiment; use list')
    results=[]
    for spec in selected:
        values=spec['values'] if args.experiment=='test_all' else [
            spec['values'][args.case] if args.case>=0 else spec['params'][spec['vary']]]
        for value in values:
            params=dict(spec['params']);params[spec['vary']]=value
            result=evaluate(spec,params)
            results.append(dict(key=spec['key'],parameters=params,**result))
    if args.output:
        args.output.parent.mkdir(parents=True,exist_ok=True)
        args.output.write_text(json.dumps(results,indent=2))
    if args.experiment=='test_all':
        print(f'Executed {len(results)} finite-result cases across {len(selected)}
    experiments')
    else:print(json.dumps(results,indent=2))

if __name__=='__main__':main()
```

What numerical validation can establish

A transform round trip can establish consistency of a mathematical implementation. A finite-difference check can detect an analytic derivative error at tested configurations. A seeded simulation can establish repeatability of a stated model. None of these proves that a fabricated link has the modeled mass, that an IMU has the assumed timing or that a drive stops in the required time.

Use several kinds of evidence. Algebraic identities check internal consistency. Independent reference calculations check formulas. Synthetic ground truth checks estimation and metric definitions. Model-based simulation tests interaction. Bench measurements check actual interfaces and parameters. Protected physical trials check behavior within a reviewed envelope. Each level answers a different question and has a different scope.

The extended suite executes 420 published parameter cases and checks finite output. Many modules also contain explicit identities or invariant assertions. The accompanying semantic test suite checks cross-case behavior and known reference properties. These are software and numerical checks. They do not imply that every arbitrary edited input is valid or that any physical system has been commissioned.

Managing evidence, changes and scope

Name each result with experiment, source identity, parameter configuration, seed and timestamp. Keep raw data separate from derived plots. If an explanation changes after looking at the result, preserve the original hypothesis and label the new one. This simple practice reduces the temptation to retrofit a story around a favorable number.

When a laboratory introduces a threshold, inspect its provenance. Some thresholds are mathematical boundaries, such as a raw inverse-kinematics cosine outside [-1,1]. Some are proposed exercise criteria, such as an error target for a supported simulated elbow. Others must come from approved physical data, such as voltage, current, temperature, connector and structural limits. Do not transfer a threshold from one category to another.

A change to joint ordering, observation normalization, camera extrinsics, a physical ankle mechanism or motor firmware can invalidate several downstream tests. Use the dependency/evidence studies to determine what must be repeated. Retain a known-good software bundle and a physically appropriate shutdown/rollback procedure. A robot under load must remain supported while a software release is reverted.

The EUR 6,000 boundary

The robot budget remains EUR 6,000. You also mentioned a possible EUR 2,000-3,000 laptop purchase; it remains a separate optional budget decision unless you explicitly include it in the robot total. The existing Intel Core i7 and GeForce RTX system should be inventoried and benchmarked before buying another machine. No laptop model or future GPU capability is invented in this edition.

The book does not establish a complete self-source Asimov 1 within EUR 6,000. The source audit and procurement ledger explain the unresolved quotations and release dependencies. The later budget laboratory uses fictional allowances to teach arithmetic; those allowances do not replace the ledger or become supplier prices. A small supported module remains the practical staged physical scope until a compatible, complete landed-cost estimate is available.

An honest final capability statement

At the end of the software work you should be able to load and inspect the actual pinned Asimov model, run bounded supported-joint trajectories, validate nearby position IK, analyze telemetry, train and evaluate a small supported learning task, export a policy artifact, test perception and execute a bounded simulated skill. Standing and walking remain research capabilities to earn through their own declared tests, not promises generated by including the relevant chapter.

For physical work, your first credible milestone is a release-matched, inspected, guarded and commissioned joint or small supported module. Its stopping behavior, feedback mapping, limits, calibration and power handling must be demonstrated on actual hardware. A full humanoid and unsupported locomotion require further verified design, funding and evidence. The point of the complete book is to equip you to make those engineering decisions independently.

Part I - Engineering the project

1. Project vision and how to use this book

Build the robot in layers. First make a model observable. Then move a joint under feedback, recover known geometric relationships, measure behavior, and build software boundaries that reject bad commands. Only after those skills are repeatable should money and force enter the project. The physical outcome depends on verified parts, fabrication, and commissioning evidence.

The simulation target is the Menlo Research Asimov 1. The initial physical target is a guarded, supported module that can sense an object and execute a small, repeatable gesture. A complete walking humanoid is a later research objective with separate feasibility and safety gates. Success means you can explain a failure using measurements, reproduce a result from a pinned configuration, and decide when the evidence is insufficient to proceed.

This book is written for an experienced software engineer beginning robotics. You already understand interfaces and distributed systems. The new challenges are physical units, geometric frames, forces, contact, uncertainty, deadlines, and faults that can release stored energy. A software rollback cannot restore a stripped gearbox.

Each lesson follows A-O: purpose, concepts, physical relevance, architecture, prerequisites, implementation, complete code, walkthrough, expected result, validation, troubleshooting, exercise, hardware connection, completion checklist, and preview. Read one lesson, run its tests, and record the result before continuing. All original teaching code is printed in the book and also saved beside it under humanoid_lab/. External CAD and meshes remain in the upstream checkout because copying them into code listings would not make them reproducible or useful.

What has and has not been demonstrated

The included lab is an educational simulation stack. Headless physics, model contracts, small IK, synthetic perception, command-gate tests, and the supported demonstration were exercised during preparation. Exact results appear in the validation appendix. ROS 2 package files are provided but require a Jazzy installation for integration testing. A short PPO run validates the training plumbing; it does not demonstrate learned standing or walking. No physical actuator was connected, no harness was load tested, and no completed robot is claimed.

Verified means supported by a named primary source or directly inspected asset. Engineering assumption means a proposed design value that must be tested. Unverified means a claim not independently established here. Missing means unavailable in the inspected public release, not proof that the manufacturer does not possess it. Every acceptance threshold proposed by this book is an engineering exercise, not a manufacturer rating.

Your workstation and budget

You reported an Intel Core i7 and NVIDIA GeForce RTX, with an optional EUR 2,000-3,000 laptop purchase. The exact CPU generation, RAM, GPU, VRAM, operating system, and driver remain unknown. Start with CPU MuJoCo. Do not buy a laptop based on the RTX label alone: mobile GPUs differ in memory, power limits, cooling, and sustained throughput. Treat the possible laptop allowance separately from the EUR 6,000 robot budget until you decide otherwise.

Run the following inventory commands on Linux; they read information without changing drivers. On Windows, record the same facts from System Information and nvidia-smi. Save output in your experiment log, including OS version and free SSD space.

```
lscpu
free -h
cat /etc/os-release
nvidia-smi --query-gpu=name,memory.total,driver_version --format=csv
df -h .
```

For the early single-model lessons, 16 GB RAM and a modern CPU are a practical planning target, not a measured minimum. Rendering needs a functioning graphics stack; physics can run headlessly. Measure simulated seconds per wall second before planning a large training job. A new laptop is

unnecessary for learning coordinates, controllers, ROS messages, or small PPO tasks.

2. Feasibility assessment and official-source audit

The official README describes a roughly 1.2 m, 35 kg machine with 25 powered joints and advertises a USD 20,000 unassembled kit. That public asking price is not an EU landed quotation. It is enough to reject the claim that buying the published kit fits EUR 6,000. It does not determine the self-source cost. [S1]

The checked-out main revision is 96a6807c163e4e6cabdc124eefcc45b0ae2cb57a. The GitHub tree was inspected directly on 25 October 2026. Some search-indexed pages described an older 23-joint simulation and different paths. The actual pinned checkout contains 25-DOF and 27-DOF model variants. Use the 25-DOF version throughout this book. Its upstream simulation README names MuJoCo 3.4.0 as validated; the lab uses that version. [S2]

Subject	Evidence in this edition	Status and consequence
Mechanical source	mechanical/ASV1 contains 168 STEP files and 25 PDFs in the inspected tree	Verified file inventory; not assembly quantities
Electrical source	electrical/ contains board sources and wiring CSVs	Verified design assets; release reconciliation required
Simulation	sim-model/asimov_1_25dof/xml /asimov_1_25dof.xml and matching URDF	Verified files; loaded in the lab
27-DOF variant	Separate model with gripper structure	Verified variant; excluded from this curriculum
Actuation in 25-DOF MJCF	No actuator definitions in inspected XML	Adapter explicitly adds teaching torque motors
Firmware and SDK	Public manual describes operation while SDK is in development	Availability must be confirmed for purchased release

Subject	Evidence in this edition	Status and consequence
ROS 2 compatibility	No tested upstream ROS 2 package established in this checkout	Book supplies an educational bridge
Complete procurement BOM	Manual requests email for BOM; referenced fabrication manifest absent here	No complete priced BOM established
Structural hardware revision	Commit identifies assets, not an approved physical serial-number release	Match fabrication, electronics and firmware before ordering
Licenses	HARDWARE-LICENSE.txt and SOFTWARE-LICENSE.txt	Preserve notices and review applicable redistribution terms

The repository's hardware/software licenses are CERN-OHL-S-2.0 and GPL-2.0 as published. Original teaching code is separate from those assets; do not assume packaging it together changes upstream obligations. This book provides engineering guidance, not a legal interpretation of redistribution terms. [S1]

Joint topology and revision mismatches

The documented powered layout is two six-axis legs, two five-axis arms, waist yaw, and two neck axes. Shoulder pitch/roll/yaw, elbow, and wrist yaw make five per arm. Hip pitch/roll/yaw, knee, ankle pitch, and ankle roll make six per leg. The hardware ankle is a parallel linkage: two motor coordinates map to pitch and roll. Simulator joints are not automatically motor shaft coordinates. [S3]

A concrete mismatch matters more than the label 'digital twin': the inspected XML gives the left elbow an upper range near 2.09443 rad, whereas the mechanical-tour table gives a different bound. Tutorial code reads its bounds from its pinned XML. Physical bounds must come from the accepted hardware release and measured mechanism. Never copy either table blindly into the motor driver.

The mechanical-tour page itself flags some illustrations as outdated. Documentation also describes a 23-DOF locomotion configuration with locked neck axes. These statements can coexist with a 25-axis physical robot. Model size, controlled action count, and physical motor count are different quantities. Record all three in a deployment manifest. [S3, S4]

Development status and buildability

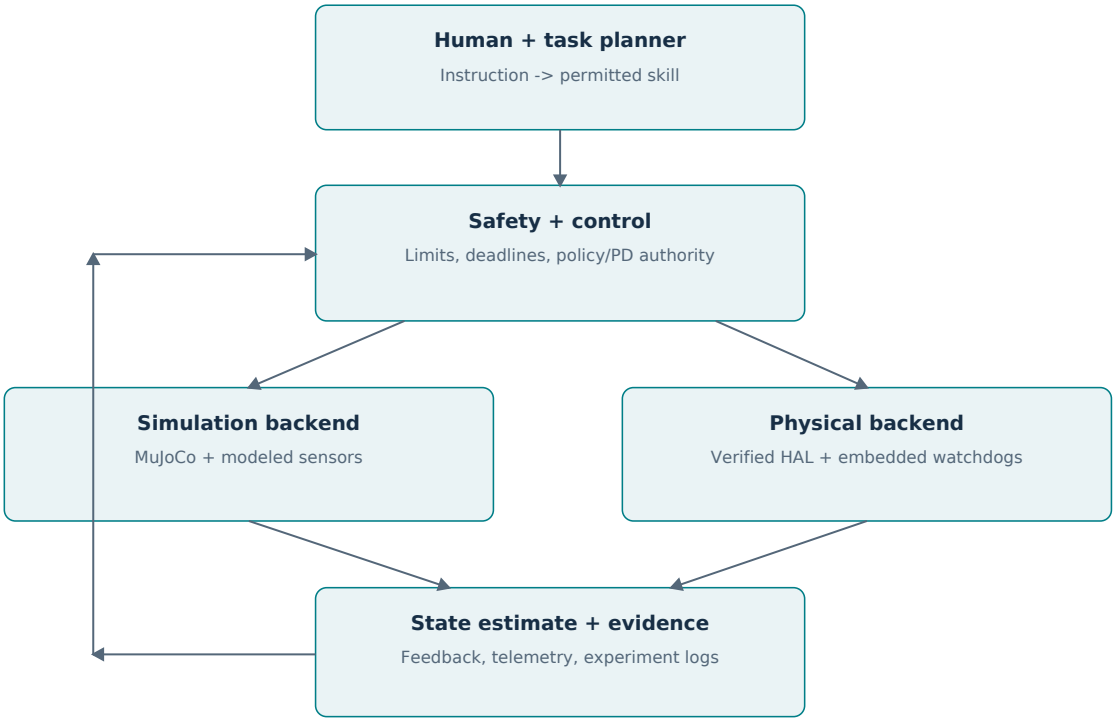
The public manual describes firmware modes and a supplied locomotion policy, while the repository roadmap lists software work still pending. Treat these as release-specific claims until the matching image, API schema, policy, checksums, calibration workflow, and support procedure are obtained. A manual's claim of zero-shot transfer is not evidence for your assembled unit. [S1, S4]

The platform is useful for simulation and inspecting real design tradeoffs. A full self-source build is not a beginner turnkey exercise: it combines machined parts, process-specific polymers, high-energy power, parallel ankle calibration, custom boards, and release-dependent software. Advance with one guarded module and engineering review of the support and power design.

The branch listing included main and several electrical, mechanical, gripper and simulation branches at inspection. Feature branches indicate work streams, not compatibility guarantees. Do not mix a board from one branch, an ankle from another, and firmware from a third without a documented integration decision.

3. Complete system architecture

Think of the system as a company with explicit authority. The planner proposes a job. A skills library translates it into a known procedure. A supervisor authorizes bounded execution. A controller makes small repeated corrections. Drivers deliver power and report measurements. No higher layer gets to bypass the lower layer's limits.



Subsystem	Analogy	Engineering function and interface
Structure and transmissions	Skeleton and tendons	Carry loads and define axis geometry, stiffness, backlash and travel
Actuators and drivers	Muscles and local reflexes	Produce limited torque; report encoder, current and thermal state
Encoders, IMU and camera	Senses	Measure different quantities in named frames with timestamps
Power and protection	Energy supply and circuit breakers	Distribute energy, limit faults and isolate hazardous motion
Embedded controller	Local reflex coordinator	Enforce deadlines, watchdogs and release-specific bus rules
Motion controller	Skilled operator	Track joint or whole-body targets within feasible constraints
State estimator	Shared situational picture	Fuse delayed, noisy observations with a model

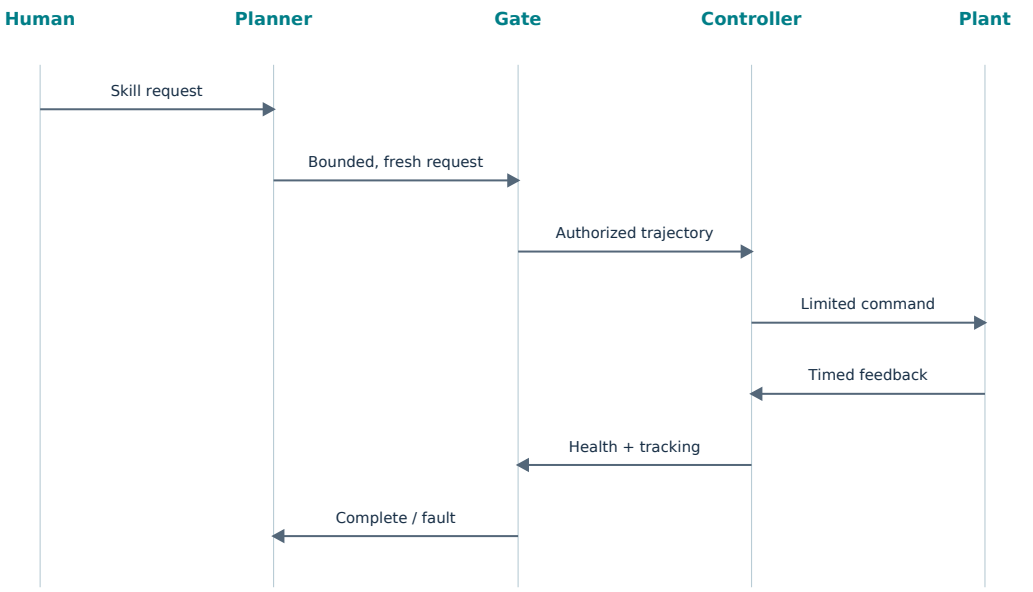
Subsystem	Analogy	Engineering function and interface
ROS 2	Typed internal messaging	Connect modular components; transport alone guarantees no control deadline
Simulator	Rehearsal space	Approximate dynamics, contacts, sensors and failure cases
Policy inference	Learned reflex	Map a defined observation to a bounded action at a defined rate
Perception and task planner	Eyes and foreman	Locate objects and select validated skills
Safety supervisor	Independent authority	Veto unsafe transitions, detect stale data and trigger a tested stop
Logging and deployment	Flight recorder and release management	Preserve evidence, hashes, configurations and rollback artifacts

The official electrical design divides media/networking and motion-control responsibilities across two compute stacks. Its documented CAN-interface specification and physical branch mapping are not fully reconciled publicly; the manual explicitly calls that out. Do not infer which baud rate belongs to a limb from the number of connectors. [S5]

Control, policy and high-level AI

Scripted control follows a prescribed time trajectory. Classical feedback uses error and a model, for example a proportional-derivative joint controller. A learned policy selects actions based on training experience. High-level AI selects a task or skill. Each can exist without the others. A wave following a sine function is not reinforcement learning, and a language interface does not make a motor controller intelligent.

Use simulation time for physical integration and monotonic wall time for watchdog deadlines. Keep these clocks explicit in messages. A playback run may execute ten simulated seconds in one wall second; a physical watchdog must still expire in actual elapsed time. A camera frame's exposure time is not its arrival time.



The physical power-isolation path must remain effective if ROS, the policy process, the planner, or the network hangs. Cutting motor power may cause a heavy robot to collapse; the support rig is part of the stop strategy. Software DAMP mode is not an emergency stop. The official software guide makes this distinction explicit. [S4]

4. Dependency-aware roadmap

Effort estimates below are planning ranges for an experienced software engineer, in focused hours. They include learning and debugging but exclude supplier lead times, fabrication queues, and recovery from hardware damage. Hardware costs remain unquoted unless specifically identified.

Phase	Deliverable and dependency	Effort	Acceptance gate
0	Reproducible environment; none	8-16 h	Import versions, save lock and hardware inventory
1	Asimov view and model inspection; 0	6-12 h	Correct joint count and finite state
2	Bounded joint trajectory; 1	12-24 h	Limits, error and saturation measured

Phase	Deliverable and dependency	Effort	Acceptance gate
3	Telemetry and sensor semantics; 2	12-24 h	Units, frames and time recorded
4	Forward/inverse kinematics; 2-3	20-40 h	Reachable target residual under 2 mm
5	ROS 2 workspace; 2-4	20-40 h	Two nodes communicate and stale command rejected
6	Balance experiments; 3-5	24-60 h	Repeatable unsupported evaluation and honest fall report
7	PPO task and export; 2-6	30-80 h	Held-out evaluation beats chosen baseline
8	Locomotion research; 6-7	80-200+ h	Tracking, falls, energy and slip evaluated
9	Perception and bounded skills; 4-5	20-50 h	Detect/no-detect cases and unknown-depth behavior pass
10	Sim-to-real contracts; 3-9	30-80 h	Observation/action/timing replay and rollback pass
11	Procurement and fabrication; 10 plus quotes	20-60 h	Complete release-matched landed BOM within budget
12	Single-actuator electrical bench; 11	30-80 h	Isolation, watchdog, feedback and limits measured
13	Incremental assembly; 12	40-120 h	One module at a time passes inspection
14	Supported commissioning; 13	40-100+ h	Signed records and repeatable stop tests
15	Physical skill deployment; 10 and 14	40-120+ h	Supported behavior repeatable without envelope violations
16	Integrated research demonstration; 9 and 15	20-60 h	Object-dependent skill, logs, faults and rollback demonstrated

Keep these dependencies visible. Procurement discovery can run while learning simulation, but purchasing high-value actuators before a compatible release is identified is premature. Locomotion is a research milestone, not a

guaranteed outcome at a fixed training step count. Hardware phases may stop at a modular platform while simulation continues to full-body locomotion.

Use a milestone record with fields: gate, date, repository hash, environment lock hash, model hash, configuration, seed, test command, observed metric, threshold, outcome, reviewer, and artifact path. A failed result is useful evidence. Do not overwrite it with a later success using the same filename.

5. Recommended stack and reproducibility policy

The baseline is Ubuntu 24.04 LTS, Python 3.12, MuJoCo 3.4.0, and ROS 2 Jazzy. Jazzy remains supported through May 2029 according to the ROS release table. Lyrical is newer and listed through May 2031; it is not required for this curriculum. We choose Jazzy for its established Ubuntu 24.04 package set and Gazebo Harmonic pairing. Iron is already end-of-life. Recheck the release table when updating this book's environment. [S6, S7, S8]

Tool	Best role here	Hardware and integration tradeoff
MuJoCo 3.4.0	First simulation, dynamics, IK, CPU RL	Fast headless CPU physics; Python bridge needed for ROS; native MJCF
Gazebo Harmonic	Optional ROS-centric sensor and systems work	ROS Jazzy pairing documented; URDF/SDF conversion and plugin work; GPU rendering varies
Isaac Sim	Optional rich rendering and RTX sensor workflows	Significant RTX/VRAM and driver requirements; asset import must be revalidated
Isaac Lab	Optional batched training framework	A framework built around compatible simulation backends, not a drop-in MJCF viewer

No simulator is 'the most realistic' for every contact, actuator, camera, or deadline. Validate the phenomenon that matters: a soft toe, backlash, thermal derating, or latency may dominate your transfer error more than the solver brand. URDF captures a tree of links and joints; MJCF adds MuJoCo-specific

world, contact, sensor and actuator settings. Conversion can lose semantics and requires regression tests.

Current Isaac Sim requirements list 32 GB RAM and 16 GB VRAM at the minimum tier, with an RTX 4080-class GPU and release-specific drivers. These are product requirements, not a promise that a similarly named laptop GPU performs like its desktop counterpart. Use the Compatibility Checker for the chosen release. Keep optional Isaac environments isolated from this book's CPU virtual environment; select an exact Isaac Lab/Sim/Python pair from their cross-product compatibility page after the hardware inventory is known. No unverified pair is installed by the tutorial. [S9]

Host installation gives direct device access and straightforward desktop rendering. A virtual environment isolates Python packages but shares the host driver and kernel. A container adds userspace reproducibility but still depends on host graphics, USB/CAN permissions and kernel timing. A virtual machine isolates the OS but may complicate graphics and device passthrough. Use native Ubuntu for the initial lab when available; a headless container is useful for CI after the baseline works.

The pinned Python packages are a tested teaching baseline rather than claims about the newest versions: MuJoCo 3.4.0, NumPy 2.2.6, Matplotlib 3.10.3, pytest 8.4.1, Gymnasium 1.2.1, Stable-Baselines3 2.7.0, OpenCV 4.12.0.88, and CPU PyTorch 2.8.0. A full resolved freeze is supplied beside the code. Save apt package versions separately because pip cannot record ROS Debian packages.

6. Repository layout and ownership

All tutorial commands assume the following location. If you use another location, update the shell root and HUMANOID_LAB_ROOT together. The actual book and code are saved at these paths.

```
/data/Robotics_Project/
  AI_Humanoid_Robotics_Engineering_Book.pdf
  book_source/                 editable teaching manuscript
  research/asimov-1/           pinned upstream CAD and models
  humanoid_lab/
    requirements.txt          direct teaching dependencies
    requirements-lock.txt     resolved environment snapshot
    model.py                  simulator adapter and bounded PD
    simulate.py               joint inspection and viewer
    telemetry.py              recorded plots and sensor inventory
    kinematics.py             planar FK and Asimov position IK
```

```
balance.py                static geometry and passive fall baseline
learning.py               reach/stand/walk Gymnasium tasks and PPO
perception.py             image-space color detector
safety.py                 offline command-gate exercise
demo.py                   supported perception-to-gesture demo
audit.py                  procurement and commissioning records
config/                   BOM and commissioning records
tests/                    meaningful numerical and fault tests
ros2_ws/src/humanoid_bridge/  ROS node package and launch file
logs/                     experiment outputs; do not version large runs
```

The simulator adapter deliberately does not import a physical CAN library. That is an architectural boundary: the tests cannot accidentally send commands to a real actuator. Create a separate hardware transport only after the release-specific protocol is verified, with its own reviewed configuration and test fixtures.

Part II - The simulation laboratory

Lesson 0 - Establish a reproducible environment

A. Why this matters

You need to distinguish a model bug from a package mismatch. An environment you cannot recreate makes every later result weaker. Begin with a small, explicit dependency set and a commit-addressed robot model.

B. Concepts explained simply

A physics engine numerically advances a state. The state includes positions and velocities. A model contains geometry, mass, joints, and equations that govern those changes. Rendering only draws a state. Therefore a successful headless test proves something useful even when no window can open, but does not prove the graphics stack works.

C. Real-world relevance

A released robot controller needs traceable software, calibration and model versions. The same habit begins here with a Python environment and a git hash.

D. Architecture

Operating system -> Python virtual environment -> MuJoCo binding -> pinned robot model -> tests and logs. ROS is added later so the first failure has fewer possible causes.

E. Prerequisites

Ubuntu 24.04, internet access and adequate disk space. The commands below are for the reader's machine, not instructions to change an existing host blindly. Keep the supplied files at /data/Robotics_Project/humanoid_lab. If /data is not yours to write, choose a user-owned project directory and adapt paths consistently.

F. Step-by-step implementation

Install standard development and desktop OpenGL support, create a venv, then install CPU PyTorch before the other dependencies so pip does not select a large CUDA stack. sudo changes OS packages; understand and review those commands on your own machine.

```
sudo apt update
sudo apt install python3-venv python3-dev git build-essential libgl1 libglfw3
cd /data/Robotics_Project
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install torch==2.8.0 --index-url https://download.pytorch.org/whl/cpu
python -m pip install -r humanoid_lab/requirements.txt
python -m pip check
python -m pip freeze > humanoid_lab/requirements-lock.txt
```

The supplied research checkout is already present. To recreate it on another machine where the directory does not exist:

```
cd /data/Robotics_Project
mkdir -p research
git clone https://github.com/menloresearch/asimov-1.git research/asimov-1
git -C research/asimov-1 checkout 96a6807c163e4e6cabdc124eefcc45b0ae2cb57a
git -C research/asimov-1 rev-parse HEAD
```

Set VS Code's interpreter to .venv/bin/python. Open the project root, run tests from a terminal first, and use a breakpoint before mj_step to inspect arrays. Do not combine a Conda interpreter, a ROS system interpreter and a pip venv accidentally.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/requirements.txt

/data/Robotics_Project/humanoid_lab/requirements.txt

```
mujoco==3.4.0
numpy==2.2.6
matplotlib==3.10.3
pytest==8.4.1
gymnasium==1.2.1
stable-baselines3==2.7.0
opencv-python-headless==4.12.0.88
```

H. Code walkthrough

Direct dependency pins make the intended API set explicit. The resolved lock records transitive versions as well. The upstream commit fixes asset paths and model semantics. A lock captured on x86 Linux is not necessarily portable to a different operating system or architecture.

I. Expected result

mujoco imports and prints 3.4.0. pip check reports no dependency conflicts. The checkout hash matches the printed revision. No robot hardware is involved.

J. Validation

```
python -c "import mujoco, numpy, torch; print(mujoco.__version__, numpy.__version__,
    torch.__version__)"
cd /data/Robotics_Project/humanoid_lab
python -m pytest -q
```

The test files are printed in the validation appendix. The prepared edition used Python 3.12.14; exact transitive versions are in requirements-lock.txt.

K. Troubleshooting

If wheels are unavailable, verify Python 3.12 and CPU architecture before compiling packages. If OpenGL fails, run headless physics first; a DISPLAY or driver error is distinct from an XML compilation failure. On macOS, passive-viewer scripts require mjpython; this book's ROS setup remains Ubuntu-specific. [S10]

L. Practical exercise

Create an environment manifest with OS, Python, model hash, package freeze and workstation inventory. Recreate a second venv from the freeze and compare imports.

M. Connection to the physical Asimov 1

The release-manifest habit transfers directly. These Python package pins do not select motor firmware or electrical revisions.

N. Completion checklist

You can explain the difference between a model, state and renderer; reproduce imports; identify your active Python executable; and locate the exact upstream model. Budget consumed by these software steps: EUR 0 excluding your existing computer and internet.

O. Next lesson preview

Load the humanoid, inspect its joint coordinates and observe a supported motion.

Lesson 1 - My first observable humanoid simulation

A. Why this matters

Seeing the robot is the first milestone, but observation must include names, coordinate addresses, limits and time. A picture alone cannot establish that commands reach the intended joint.

B. Concepts explained simply

A degree of freedom is an independent coordinate. A hinge joint rotates; a slide joint translates. A floating base adds three translations and a quaternion representation of orientation: its position array has seven entries while its velocity has six. Thus nq and nv can differ. Joint index, position-array address, velocity-array address and actuator index are different identifiers.

C. Real-world relevance

A wrong mapping can send a knee target to a shoulder. Map by verified names, then check dimensions and ordering before every deployment.

D. Architecture

Pinned MJCF -> teaching adapter -> MjModel -> MjData -> bounded controller -> mj_step -> viewer and CSV. The adapter fixes the pelvis, adds torque motors and retains the official mesh and inertial model.

E. Prerequisites

Lesson 0, the upstream meshes, model.py and simulate.py below. Your first run uses a fixed pelvis fixture. It is intentionally incapable of demonstrating free standing.

F. Step-by-step implementation

```
cd /data/Robotics_Project/humanoid_lab
python simulate.py --headless --seconds 10
python simulate.py --seconds 30
```

The first command tests physics without opening a window. The second opens a viewer with a useful camera distance. Rotate and zoom with the viewer controls and watch the left elbow. Head, torso and legs are visible even though this exercise moves one arm joint.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/model.py

/data/Robotics_Project/humanoid_lab/model.py

```
"""Educational Asimov model adapters; simulation only."""
from pathlib import Path
import xml.etree.ElementTree as ET
import mujoco as mj
import numpy as np

ROOT = Path(__file__).resolve().parent
ASSET = (ROOT.parent / 'research/asimov-1/sim-model/'
         'asimov_1_25dof/xml/asimov_1_25dof.xml')

def load(supported: bool = True):
    tree = ET.parse(ASSET)
    xml = tree.getroot()
    xml.find('compiler').set('meshdir', str(ASSET.parent.parent / 'meshes'))
    xml.find('option').set('timestep', '0.002')
    # Fixed pelvis in the teaching fixture. This is not balancing.
    if supported:
        base = xml.find('.//freejoint')
        xml.find('.//body[@name="pelvis_link"]').remove(base)
    for actuator in list(xml.findall('actuator')):
        xml.remove(actuator)
    actuators = ET.SubElement(xml, 'actuator')
    joints = xml.findall('.//worldbody//joint')
    for j in joints:
        # Remove passive springs so they do not hide controller behavior.
        for attr in ('stiffness', 'damping', 'springref'):
            j.set(attr, '0')
        limits = j.get('actuatorfrcrange', '-8 8')
        ET.SubElement(actuators, 'motor', name=j.get('name')+'_motor',
                      joint=j.get('name'), gear='1',
                      ctrllimited='true', ctrlrange=limits)
    model = mj.MjModel.from_xml_string(ET.tostring(xml, encoding='unicode'))
    model.opt.gravity[:] = [0, 0, -9.81]
    data = mj.MjData(model)
    mj.mj_forward(model, data)
    return model, data

def addresses(model):
    ids = model.actuator_trnid[:, 0].astype(int)
    return ids, model.jnt_qposadr[ids], model.jnt_dofadr[ids]

def names(model):
    ids, _, _ = addresses(model)
    return [mj.mj_id2name(model, mj.mjtObj.mjOBJ_JOINT, int(j)) for j in ids]

def command(model, data, target, kp=35., kd=4., fraction=0.25):
    ids, qi, vi = addresses(model)
    target = np.asarray(target, dtype=float)
    if target.shape != (model.nu,) or not np.isfinite(target).all():
        raise ValueError('Expected one finite target per actuator')
    lo, hi = model.jnt_range[ids].T
```

```
    target = np.clip(target, lo, hi)
    # qfrc_bias is ideal model feedforward, not measured motor torque.
    torque = kp*(target-data.qpos[qi])-kd*data.qvel[vi]+data.qfrc_bias[vi]
    cap = np.max(np.abs(model.actuator_ctrlrange), axis=1)*fraction
    data.ctrl[:] = np.clip(torque, -cap, cap)
    return target.copy()
```

File: /data/Robotics_Project/humanoid_lab/simulate.py

/data/Robotics_Project/humanoid_lab/simulate.py

```
"""Inspect, view and log a supported Asimov joint trajectory."""
import argparse
import csv
import time
import numpy as np
import mujoco as mj
from model import ROOT, load, addresses, names, command

def main():
    p = argparse.ArgumentParser()
    p.add_argument('--headless', action='store_true')
    p.add_argument('--seconds', type=float, default=10.)
    args = p.parse_args()
    if not 0 < args.seconds <= 600:
        p.error('seconds must be in (0, 600]')
    m, d = load()
    ids, qi, vi = addresses(m)
    joint_names = names(m)
    print('nq, nv, nu:', m.nq, m.nv, m.nu)
    for name, j in zip(joint_names, ids):
        print(name, 'range=', m.jnt_range[j])
    j = joint_names.index('left_elbow_joint')
    home = d.qpos[qi].copy()
    path = ROOT/'logs/joints.csv'
    path.parent.mkdir(exist_ok=True)
    with path.open('w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['t', 'target', 'q', 'dq', 'command_Nm'])
        def tick():
            target = home.copy()
            target[j] += 0.1*np.sin(2*np.pi*0.25*d.time)
            actual = command(m, d, target)
            mj.mj_step(m, d)
            if not np.isfinite(d.qpos).all():
                raise RuntimeError('Nonfinite simulation state')
            writer.writerow([d.time, actual[j], d.qpos[qi[j]],
                             d.qvel[vi[j]], d.ctrl[j]])
            if round(d.time/m.opt.timestep) % 100 == 0:
                f.flush()
        if args.headless:
            for _ in range(round(args.seconds/m.opt.timestep)):
                tick()
        else:
            import mujoco.viewer
            with mujoco.viewer.launch_passive(m, d) as v:
                v.cam.distance = 2.2
                v.cam.lookat[:] = [0, 0, .65]
                v.cam.elevation = -15
                while v.is_running() and d.time < args.seconds:
                    start = time.monotonic()
                    tick()
                    v.sync()
                    time.sleep(max(0., m.opt.timestep-(time.monotonic()-start)))
    print('Saved', path)
```

```
if __name__ == '__main__':
    main()
```

H. Code walkthrough

load reads the verified XML rather than reconstructing a humanoid from guessed geometry. An absolute mesh directory makes the in-memory XML resolve assets correctly. Removing the floating joint attaches the pelvis to the world for this teaching fixture. The adapter replaces actuator definitions with gear-one torque motors, so ctrl is a simulated torque command in Nm. This is an educational actuator model, not an identified physical drive.

addresses obtains each actuator's joint and its qpos/qvel addresses. command checks the entire target vector, clips positions to model bounds, computes feedback plus ideal bias compensation, and clips torque to 25% of the XML actuator control envelope. The 25% value is a simulation teaching choice. mj_step integrates a 2 ms timestep; the CSV stores each step's feedback. The viewer's sync updates graphics; the sleep aims at readable wall-clock speed but provides no hard deadline.

I. Expected result

The supported model has nq=25, nv=25 and nu=25. The terminal lists joint ranges, a small elbow motion appears in the viewer, and logs/joints.csv contains about 5,000 data rows for ten simulated seconds.

J. Validation

Run test_asimov_contract and test_bounded_feedback_and_rejection from the appendix. Check finite state, model gravity, time increment and the printed actuator count. Manually verify that the left elbow moves while the fixed pelvis stays fixed. Share the test output before proceeding to Lesson 2 if following this as a coached course.

K. Troubleshooting

A missing mesh often means the checkout is incomplete or paths belong to an older release. An absent actuator means the original model was opened without the adapter. A falling robot usually means you loaded the free-base model directly; that is not this fixture. If the window does not open, preserve the successful headless result and diagnose graphics separately.

L. Practical exercise

Print joint type, qpos address and qvel address in a four-column inventory. Compare the supported and unsupported model dimensions without applying a motor command.

M. Connection to the physical Asimov 1

Names and geometric inspection are reusable. Fixing the pelvis, ideal bias compensation, low-delay observations and gear-one torque motors are simulator conveniences that must be replaced or calibrated.

N. Completion checklist

You can launch both modes, identify all 25 actuators, explain nq versus nv, find the CSV, and state why this demonstration proves no balance capability. No hardware expenditure is required.

O. Next lesson preview

Measure tracking and introduce feedback gains, saturation and safe trajectory shapes.

Lesson 2 - Joint control as a measurable feedback loop

A. Why this matters

An angle command is a request. Physical motion depends on torque, inertia, gravity, friction, limits and time. Tracking error tells you whether the request is achieved.

B. Concepts explained simply

Position q is an angle in radians, velocity dq is its rate in rad/s, and torque is a turning effort in Nm. One radian is about 57.3 degrees. A revolute joint changes orientation; a prismatic joint changes distance. The same controller structure can apply, but its units change.

Our controller uses tau = Kp*(q_target-q) - Kd*dq + bias(q,dq). Kp has units Nm/rad and Kd has units Nm*s/rad. With error 0.1 rad, Kp=35 and velocity 0.2 rad/s, Kd=4, the feedback portion is 3.5-0.8=2.7 Nm. Gravity and other bias terms are additional. A torque cap may reduce the result.

PID adds Ki times the accumulated error. Integral action can remove steady offsets but can wind up when the motor saturates. Begin with PD and an identified bias before adding an integral. If using integral control, freeze or back-calculate the integral during saturation and reset it on state transitions.

C. Real-world relevance

Actual current, temperature, voltage and tracking-error measurements constrain usable gains. A high simulated gain does not authorize the same gain on an assembled limb.

D. Architecture

Reference trajectory -> position/velocity limits -> PD and bias -> torque saturation -> actuator and joint -> encoder feedback. Log both requested and applied values; hiding clipping prevents diagnosis.

E. Prerequisites

Lesson 1 and basic differentiation. Keep the fixed-pelvis fixture. The code for this lesson is the complete command function and simulator from Lesson 1; no unprinted source file is required.

F. Step-by-step implementation

Run the ten-second trajectory, then render its telemetry. Change only one simulation parameter at a time. Try Kp=20 with Kd=4, then Kp=35 with Kd=6. Preserve separate CSVs before rerunning.

```
python simulate.py --headless --seconds 10
cp logs/joints.csv logs/joints_baseline.csv
python telemetry.py
```

For both arms, create a second index by joint name and use a mirrored bounded elbow excursion. For a knee, read its asymmetric sign and limits first. Test a 0.03 rad knee excursion in the supported fixture, never by assuming left and right signs match. A minimum-jerk transition uses s(t)=10*r^3-15*r^4+6*r^5, r clipped to [0,1], with q(t)=q_start+s*(q_end-q_start).

G. Complete code

The executable files are model.py and simulate.py printed in Lesson 1, plus telemetry.py in Lesson 3. The additional numerical analysis is this complete terminal program:

```
python - <<'PY'
import numpy as np
x = np.genfromtxt('logs/joints.csv', delimiter=',', names=True)
e = x['target']-x['q']
print('RMSE rad:', np.sqrt(np.mean(e*e)))
print('Peak speed rad/s:', np.max(np.abs(x['dq'])))
print('Peak command Nm:', np.max(np.abs(x['command_Nm'])))
assert np.isfinite(e).all()
PY
```

H. Code walkthrough

The sine target is 0.1 rad at 0.25 Hz around the XML reference pose, with model-limit clipping. The callback executes before integration so torque is computed from the current state. qfrc_bias is ideal model information; physical software would need a trusted dynamics model or another strategy. Position and torque clipping protect different quantities. Neither ensures collision avoidance.

I. Expected result

The elbow follows a smooth curve with nonzero error. Torque may plateau where the simulation cap is active. The supported pelvis prevents whole-body falling but other joints can still oscillate or contact.

J. Validation

Require finite state and applied commands within their configured caps. Proposed exercise target: elbow RMSE below 0.1 rad for the baseline run; record the measured value rather than assuming success. Inject NaN and wrong-length targets and verify rejection. Compare a smooth trajectory against a step command in simulation and inspect peak speed.

K. Troubleshooting

Oscillation suggests excessive gain, inadequate damping or an integration issue. Persistent error with a flat-topped torque trace suggests saturation or load mismatch. A joint moving opposite to expectation suggests a sign/frame problem, not a need to negate every command globally.

L. Practical exercise

Implement minimum-jerk motion as a new simulation-only trajectory, compute peak speed, and reject duration choices that exceed a chosen speed envelope.

M. Connection to the physical Asimov 1

The feedback structure and logging transfer. Gains, friction, current-to-torque conversion, backlash, position zeros, thermal limits and ankle mapping require measured data. Simulator limits are not physical protection settings.

N. Completion checklist

You can distinguish angle, speed and torque; reproduce tracking metrics; explain saturation and integral windup; and reject invalid commands. Continue only after a repeatable joint experiment.

O. Next lesson preview

Read sensors with explicit units, frames and timestamps, and turn logs into a useful diagnostic view.

Lesson 3 - Sensors, telemetry and state estimation

A. Why this matters

A controller needs a trustworthy estimate of what the robot is doing now. Sensor messages are partial, noisy, delayed observations, not the entire physical state.

B. Concepts explained simply

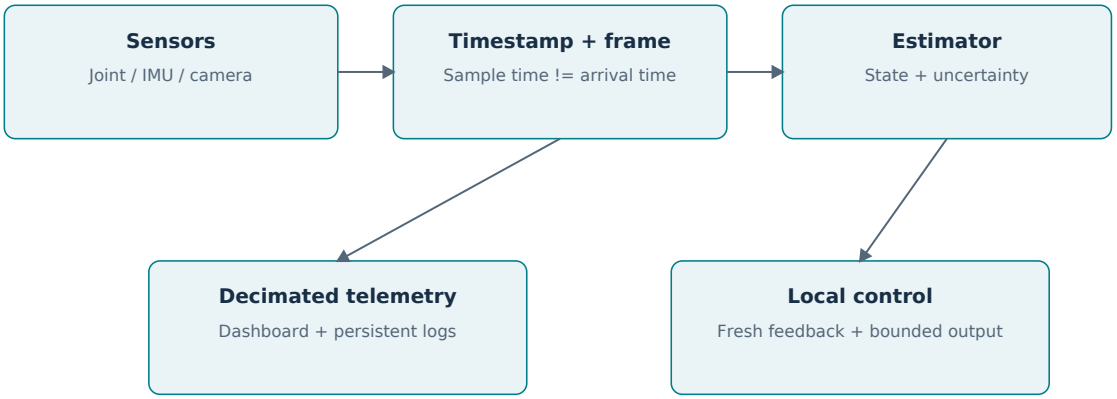
An encoder measures a shaft or output angle. A gyroscope measures angular velocity. An accelerometer measures specific force rather than simply the second derivative of world position; its stationary reading includes the support reaction against gravity. A camera measures light projected into pixels. Foot contact is an interaction between surfaces, not a boolean always available from the real hardware.

Latency is age, jitter is variation in age, and sample frequency is how often a measurement is produced. A 10 Hz telemetry feed cannot be silently substituted for a 500 Hz feedback loop. Store exposure/sample time, receive time, sequence, frame and units. Your IoT experience helps with delivery and observability, while actuator deadlines require a different timing contract.

C. Real-world relevance

The physical robot's encoder offsets and IMU orientation affect every estimate. Gravity projected into the wrong frame can look like a sustained acceleration. Motor current is not a calibrated contact-force sensor.

D. Architecture



E. Prerequisites

A successful joint log from Lesson 2, NumPy and Matplotlib. No physical IMU or camera is needed for this lesson.

F. Step-by-step implementation

```
python telemetry.py
```

Open logs/telemetry.png. The terminal prints sensor names and dimensions from the actual model, then its subtree center of mass in world coordinates. MuJoCo gyro and accelerometer channels are modeled sensors; frame quaternion and linear velocity channels are privileged simulation signals unless corresponding estimators exist on hardware.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/telemetry.py

/data/Robotics_Project/humanoid_lab/telemetry.py

```
"""Plot recorded feedback; no actuator network connection."""
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from model import ROOT, load

if __name__ == '__main__':
    data = np.genfromtxt(ROOT/'logs/joints.csv', delimiter=',', names=True)
    fig, axes = plt.subplots(3, 1, figsize=(8, 7), sharex=True)
    axes[0].plot(data['t'], data['target'], label='target')
    axes[0].plot(data['t'], data['q'], label='encoder truth')
    axes[0].set_ylabel('Angle [rad]'); axes[0].legend()
    axes[1].plot(data['t'], data['dq']); axes[1].set_ylabel('Speed [rad/s]')
    axes[2].plot(data['t'], data['command_Nm']); axes[2].set_ylabel('Command [Nm]')
    axes[2].set_xlabel('Simulation time [s]')
    fig.tight_layout(); fig.savefig(ROOT/'logs/telemetry.png', dpi=160)
    m, d = load()
    for i in range(m.nsensor):
        a, n = m.sensor_adr[i], m.sensor_dim[i]
        print(m.sensor(i).name, d.sensordata[a:a+n].copy())
    print('COM world [m]:', d.subtree_com[m.body('pelvis_link').id])
```

H. Code walkthrough

genfromtxt reads named CSV columns. Three aligned plots share time so you can relate error, velocity and saturation. Sensor addresses and dimensions select exact slices; copying samples avoids retaining mutable simulator views. subtree_com is mass-weighted model truth and is used later to check an estimator, not as a sensor you can plug into a real robot.

For a basic attitude estimator, integrate gyro orientation over small dt, then correct roll/pitch slowly toward an accelerometer-derived gravity direction only when linear acceleration is plausibly low. A complementary filter combines fast gyro changes with slow gravity reference. Accelerometers alone cannot determine yaw, and a six-axis IMU without external references drifts in yaw. A full estimator also uses kinematic contact constraints and rejects slipping-foot assumptions.

I. Expected result

Readable angle, speed and torque-command plots. The initial supported fixture's sensor values can differ from expectations until you identify local axes and specific-force conventions. Investigate a discrepancy rather than relabeling the plotted units.

J. Validation

Check CSV time is monotonic with approximately 0.002 s increments. Verify the encoder channel's unit by changing a joint by 0.1 rad. For an added noise model, use a fixed random seed and show both ground truth and noisy measurement. Verify quaternion ordering before comparing to ROS: MuJoCo uses w,x,y,z while ROS messages use x,y,z,w.

K. Troubleshooting

A constant copied history often comes from storing array views. A rotated gravity vector may indicate world versus sensor-frame confusion. A slow plot may indicate every physics step is being rendered. Decimate visualization while preserving controller sampling.

L. Practical exercise

Add synthetic gyro bias and Gaussian encoder noise to copied data only. Estimate a zero-rate gyro bias from a stationary segment, subtract it, and compare drift over a held-out segment.

M. Connection to the physical Asimov 1

Telemetry schemas and diagnostic plots are reusable. Sensor timing, calibration, transport and noise must come from real recordings. Do not assume foot force sensing is present merely because the simulator reports contacts.

N. Completion checklist

Every stream has known units and frame; every time variable has a clock; plotted values are copies; and ground truth is distinguished from measurements and estimates.

O. Next lesson preview

Use geometry to predict and command limb endpoints.

Lesson 4 - Frames, forward kinematics and inverse kinematics

A. Why this matters

A hand target is usually expressed in space, while motors accept joint coordinates. Kinematics translates between those descriptions before dynamics determines the forces required.

B. Concepts explained simply

A coordinate frame is an origin and three axes. A rotation changes the basis used to express a vector; a translation changes its origin. A 90-degree rotation about z maps (1,0,0) to (0,1,0). With a translation (1,2,0), that same point becomes (1,3,0). The order matters.

A homogeneous transform T has a 3x3 rotation R and translation p in its upper rows, and [0,0,0,1] below. It maps a homogeneous point using x_A=T_AB*x_B. Chaining transforms means multiplication in frame order. Inverting a rigid transform uses R transpose and -R transpose times p.

Euler angles are three ordered rotations and have singular configurations. A unit quaternion represents rotation with four numbers and a normalization constraint; q and -q represent the same orientation. Use matrices or quaternions for composition and reserve Euler angles for labeled displays.

For a two-link planar arm, x=l1*cos(q1)+l2*cos(q1+q2), y=l1*sin(q1)+l2*sin(q1+q2). With l1=.3 m, l2=.2 m and zero angles, the endpoint is (.5,0) m. At q1=pi/2 and q2=0 it is (0,.5) m. This is forward kinematics.

The Jacobian J is the local sensitivity of endpoint position to joint changes. For example, at zero angles the second row is [.5,.2], so a small elbow motion of .01 rad changes y by about .002 m. Fully stretched arms lose some directions of instantaneous motion: this is a singularity.

C. Real-world relevance

A camera-to-base transform and encoder zero error can dominate reaching accuracy. Kinematics must use the same joint conventions as the assembled mechanism.

D. Architecture

Target in world frame -> FK and Jacobian -> damped least-squares update -> joint constraints -> recompute FK -> residual. This lesson solves endpoint position only; it does not plan collision-free dynamic motion.

E. Prerequisites

Lessons 1-3, NumPy matrix multiplication and the fixed-pelvis fixture. No new kinematics library is needed for the first experiment. Pinocchio becomes useful for larger model-based dynamics and optimization workflows once your conventions are tested.

F. Step-by-step implementation

```
python kinematics.py
```

The target is a small displacement from the model's current left wrist-link origin. That choice makes reachability plausible and avoids inventing a hand geometry. To visualize, call solve before launching a MuJoCo passive viewer and inspect the resulting pose without dynamics. Kinematic pose inspection must not be mistaken for a physically executed trajectory.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/kinematics.py

/data/Robotics_Project/humanoid_lab/kinematics.py

```
"""Position-only damped least-squares IK in a fixed pelvis fixture."""
import numpy as np
import mujoco as mj
from model import load, addresses, names

def planar(q, lengths=(.3, .2)):
    a, b = q; l1, l2 = lengths
    return np.array([l1*np.cos(a)+l2*np.cos(a+b),
                     l1*np.sin(a)+l2*np.sin(a+b)])

def solve(m, d, target, iterations=150, tolerance=.002):
    joint_names = names(m)
    arm = [i for i, name in enumerate(joint_names) if name.startswith('left_')
           and any(x in name for x in ('shoulder', 'elbow', 'wrist'))]
    ids, qi, vi = addresses(m)
    bid = m.body('left_wrist_yaw_link').id
    jp, jr = np.zeros((3, m.nv)), np.zeros((3, m.nv))
    for _ in range(iterations):
        mj.mj_forward(m, d)
        error = target-d.xpos[bid]
        if np.linalg.norm(error) < tolerance:
            return True, float(np.linalg.norm(error))
        mj.mj_jacBody(m, d, jp, jr, bid)
        j = jp[:, vi[arm]]
        dq = j.T @ np.linalg.solve(j@j.T+.01**2*np.eye(3), error)
        d.qpos[qi[arm]] += np.clip(dq, -.04, .04)
        d.qpos[qi[arm]] = np.clip(d.qpos[qi[arm]],
            m.jnt_range[ids[arm], 0], m.jnt_range[ids[arm], 1])
    mj.mj_forward(m, d)
    return False, float(np.linalg.norm(target-d.xpos[bid]))

if __name__ == '__main__':
    print('Two-link at zero:', planar([0., 0.]))
    m, d = load()
    target = d.xpos[m.body('left_wrist_yaw_link').id].copy()
    target += [.02, 0., .01]
    ok, error = solve(m, d, target)
    print('IK converged:', ok, 'residual [m]:', error)
    if not ok:
        raise SystemExit('Target not reached; retain last validated pose')
```

The following complete numerical example verifies the frame convention:

```
python - <<'PY'
import numpy as np
R = np.array([[0., -1., 0.], [1., 0., 0.], [0., 0., 1.]])
p = np.array([1., 2., 0.])
point = R @ np.array([1., 0., 0.])+p
print(point)
assert np.allclose(point, [1., 3., 0.])
assert np.allclose(R.T @ R, np.eye(3))
```

PY

H. Code walkthrough

mj_forward updates poses for the current coordinates. mj_jacBody calculates the wrist body-origin Jacobian. The solver selects only left-arm columns, so leg and base coordinates cannot satisfy the hand request accidentally. It uses dq=J.T*(J*J.T+lambda^2*I)^-1*error with lambda=.01, caps each iteration at .04 rad, then clips joint coordinates. Damping reduces ill-conditioning but does not make unreachable targets reachable.

The result reports a boolean and residual, so a failed solve is not passed downstream as a completed task. This implementation mutates the candidate data even on failure; keep a separate validated state and copy it back on a physical planning failure. Add orientation residuals, collision checks and a trajectory stage before any dynamic reaching deployment.

I. Expected result

The planar example prints [.5,0]. The Asimov candidate reaches the nearby position within a 2 mm numerical tolerance on the fixed base. Numerical tolerance is not a claim of physical accuracy.

J. Validation

Use test_planar_geometry_and_ik. Add a finite-difference Jacobian check at a nonsingular pose: perturb one angle by epsilon, recompute the wrist position, divide its displacement by epsilon, and compare to the corresponding Jacobian column. For a target several meters away, require failure reporting rather than apparent success.

K. Troubleshooting

Wrong Jacobian columns create unexpected body motion. A stubborn residual near a limit may be reachability, not numerical failure. A body-origin target differs from a hand-tool tip; add a named site and include its offset before calling that target a grasp point.

L. Practical exercise

Implement a transform inversion test and a small Cartesian path of five reachable waypoints, each solved from the previous successful pose. Stop on the first failure and preserve its residual.

M. Connection to the physical Asimov 1

FK and frame tests are reusable after calibration. Wrist tool offsets, base attitude, link dimensions and the parallel ankle mapping require physical identification.

N. Completion checklist

You can transform a point, explain rotation order and quaternion ordering, validate a Jacobian column, distinguish reachability from convergence, and reject failed IK.

O. Next lesson preview

Place simulation and commands behind ROS 2 interfaces without changing the numerical controller.

Part III – Middleware, balance, learning and perception

Lesson 5 – A modular ROS 2 humanoid workspace

A. Why this matters

A single script is useful for proving a numerical idea. Multiple reusable robot capabilities need discoverable messages, explicit lifecycles and diagnostics. Keep the controller independent from the messaging layer so communication changes do not rewrite the physics.

B. Concepts explained simply

A node is a process participant. A topic carries a stream; a service is a request and response; an action is a long-running goal with feedback and cancellation. Parameters configure nodes. Launch files start a repeatable set of processes. QoS expresses delivery and buffering choices; reliable delivery can still deliver stale data, while best-effort delivery can lose a fresh packet.

TF2 maintains timestamped frame relationships. URDF describes the link tree consumed by robot_state_publisher and RViz2. JointState supplies scalar joint feedback but does not define frame geometry. ros2_control standardizes controller and hardware interfaces; it is not automatically integrated merely because a topic is named joint_states. Lifecycle nodes add explicit configuration and activation transitions, useful when enabling hardware only after prerequisites pass.

C. Real-world relevance

Separate simulation bridge, state estimation, kinematics, perception, behavior, safety and diagnostics nodes. Physical drivers should export verified state/command interfaces and preserve watchdog authority locally. The sample below provides a small two-node simulation system rather than claiming an entire hardware-control framework.

D. Architecture

wave node -> /joint_targets -> simulation bridge -> bounded PD and physics -> /joint_states -> wave initialization and telemetry. A future estimator consumes IMU/joint/contact streams, a planner produces validated paths, and a safety node gates activation. No planner bypasses the local limits.

E. Prerequisites

Ubuntu 24.04 and ROS 2 Jazzy. The package uses Python 3.12 to match Jazzy's system bindings. A separate --system-site-packages venv allows apt-installed rclpy to be imported. Do not activate an unrelated Conda environment.

F. Step-by-step implementation

The complete installer below follows the official apt-source bootstrap in [S7] and refuses other OS versions. It downloads the current official Noble repository-configuration package; this is a changing bootstrap dependency, so preserve its version and checksum with your environment record. Review the script before running it on your machine. It was not executed during book preparation.

/data/Robotics_Project/humanoid_lab/setup_ros.sh

```
#!/usr/bin/env bash
set -euo pipefail
. /etc/os-release
if [ "${ID:-}" != "ubuntu" ] || [ "${VERSION_ID:-}" != "24.04" ]; then
  echo 'This installer is for Ubuntu 24.04 only.' >&2
  exit 1
fi
sudo apt update
sudo apt install -y software-properties-common curl python3-venv
sudo add-apt-repository -y universe
python3 - <<'PY'
import json
import urllib.request
api = 'https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest'
release = json.load(urllib.request.urlopen(api))
assets = [x for x in release['assets'] if x['name'].startswith('ros2-apt-source_')
          and x['name'].endswith('.noble_all.deb')]
if len(assets) != 1:
    raise SystemExit('No unique Noble ROS 2 apt source asset; inspect official releases')
url = assets[0]['browser_download_url']
if not url.startswith('https://github.com/ros-infrastructure/ros-apt-source/releases/'):
    raise SystemExit('Unexpected download origin')
print('Official asset:', url)
urllib.request.urlretrieve(url, '/tmp/robotics-ros2-apt-source.deb')
PY
sudo dpkg -i /tmp/robotics-ros2-apt-source.deb
sudo apt update
sudo apt install -y ros-jazzy-desktop ros-dev-tools python3-colcon-common-extensions
echo 'Next: source /opt/ros/jazzy/setup.bash'

bash /data/Robotics_Project/humanoid_lab/setup_ros.sh
```

After installation, prepare the package environment:

```
sudo apt update
sudo apt install ros-jazzy-desktop ros-dev-tools python3-colcon-common-extensions
source /opt/ros/jazzy/setup.bash
/usr/bin/python3 -m venv --system-site-packages /data/Robotics_Project/.venv-ros
source /data/Robotics_Project/.venv-ros/bin/activate
python -m pip install mujoco==3.4.0 numpy==2.2.6
export HUMANOID_LAB_ROOT=/data/Robotics_Project/humanoid_lab
cd /data/Robotics_Project/humanoid_lab/ros2_ws
colcon build --symlink-install --packages-select humanoid_bridge
source install/setup.bash
ros2 launch humanoid_bridge lab.launch.py
```

In a second terminal, source ROS and the workspace, then run ros2 topic echo /joint_states and ros2 topic hz /joint_states. Stop the wave node and observe the simulation fallback. The launch defaults to a GUI; for a headless integration check run the bridge executable directly without enabling its viewer parameter.

G. Complete code

All files below are in /data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/. Create resource/humanoid_bridge and humanoid_bridge/__init__.py as empty files. Names here are tutorial package names, not purported upstream Asimov packages.

/data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/package.xml

```
<?xml version="1.0"?>
<package format="3">
  <name>humanoid_bridge</name><version>0.1.0</version>
  <description>Supported Asimov teaching bridge; simulation only.</description>
  <maintainer email="student@example.org">Robotics Lab</maintainer>
  <license>MIT</license>
  <buildtool_depend>ament_python</buildtool_depend>
  <exec_depend>rclpy</exec_depend>
  <exec_depend>sensor_msgs</exec_depend>
  <exec_depend>launch</exec_depend>
  <exec_depend>launch_ros</exec_depend>
  <export><build_type>ament_python</build_type></export>
</package>
```

/data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/setup.py

```
from setuptools import setup
setup(name='humanoid_bridge', version='0.1.0',
      packages=['humanoid_bridge'],
      data_files=[('share/ament_index/resource_index/packages',
    ['resource/humanoid_bridge']),
                  ('share/humanoid_bridge', ['package.xml']),
                  ('share/humanoid_bridge/launch', ['launch/lab.launch.py'])],
      install_requires=['setuptools'], zip_safe=True,
      maintainer='Robotics Lab', maintainer_email='student@example.org',
      description='Simulation teaching bridge', license='MIT',
      entry_points={'console_scripts': [
          'bridge = humanoid_bridge.bridge:main',
          'wave = humanoid_bridge.wave:main']})
```

/data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/setup.cfg

```
[develop]
script_dir=$base/lib/humanoid_bridge
[install]
install_scripts=$base/lib/humanoid_bridge
```

/data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/humanoid_bridge/bridge.py

```
import os
import sys
import time
sys.path.insert(0, os.environ['HUMANOID_LAB_ROOT'])
import mujoco as mj
import numpy as np
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState
from model import load, addresses, names, command

class Bridge(Node):
    def __init__(self):
        super().__init__('asimov_simulation')
        self.m, self.d = load()
        _, self.qi, self.vi = addresses(self.m)
        self.joints = names(self.m)
        self.home = self.d.qpos[self.qi].copy()
        self.target = self.home.copy()
        self.last = -np.inf
        self.pub = self.create_publisher(JointState, '/joint_states', 10)
        self.create_subscription(JointState, '/joint_targets', self.receive, 10)
        self.viewer = None
        if self.declare_parameter('viewer', False).value:
            import mujoco.viewer
            self.viewer = mujoco.viewer.launch_passive(self.m, self.d)
            self.viewer.cam.distance = 2.2
            self.viewer.cam.lookat[:] = [0, 0, .65]
        self.create_timer(.01, self.tick)

    def receive(self, msg):
        stamp = msg.header.stamp.sec+msg.header.stamp.nanosec*1e-9
        age = self.get_clock().now().nanoseconds*1e-9-stamp
        if msg.name != self.joints or len(msg.position) != self.m.nu:
            self.get_logger().warning('Rejected names or shape'); return
        if not 0 <= age <= .1 or not np.isfinite(msg.position).all():
            self.get_logger().warning('Rejected stale/nonfinite command'); return
        self.target = np.array(msg.position)
        self.last = time.monotonic()

    def tick(self):
        # Simulation fallback only. A physical machine needs a rig-specific stop.
        target = self.target if time.monotonic()-self.last < .1 else self.home
        for _ in range(5):
            command(self.m, self.d, target); mj.mj_step(self.m, self.d)
        msg = JointState(); msg.header.stamp = self.get_clock().now().to_msg()
        msg.name = self.joints
        msg.position = self.d.qpos[self.qi].tolist()
        msg.velocity = self.d.qvel[self.vi].tolist()
        msg.effort = self.d.actuator_force.tolist()
        self.pub.publish(msg)
        if self.viewer:
            self.viewer.sync()
```

```
def main():
    rclpy.init(); node = Bridge()
    try:
        rclpy.spin(node)
    finally:
        if node.viewer: node.viewer.close()
        node.destroy_node(); rclpy.shutdown()
```

/data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/humanoid_bridge/wave.py

```
import math
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState

class Wave(Node):
    def __init__(self):
        super().__init__('supported_elbow_wave')
        self.pub = self.create_publisher(JointState, '/joint_targets', 1)
        self.names = None; self.home = None; self.start = None
        self.create_subscription(JointState, '/joint_states', self.receive, 10)
        self.create_timer(.02, self.tick)

    def receive(self, msg):
        if self.home is None:
            self.names = list(msg.name); self.home = list(msg.position)
            self.start = self.get_clock().now().nanoseconds*1e-9

    def tick(self):
        if self.home is None: return
        now = self.get_clock().now()
        t = now.nanoseconds*1e-9-self.start
        msg = JointState(); msg.header.stamp = now.to_msg()
        msg.name = self.names; msg.position = self.home.copy()
        j = self.names.index('left_elbow_joint')
        msg.position[j] += .05*(1-math.cos(2*math.pi*.25*t))
        self.pub.publish(msg)

def main():
    rclpy.init(); node = Wave()
    try: rclpy.spin(node)
    finally:
        node.destroy_node(); rclpy.shutdown()
```

/data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/launch/lab.launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package='humanoid_bridge', executable='bridge',
             parameters=[{'viewer': True}]),
        Node(package='humanoid_bridge', executable='wave')])
```

H. Code walkthrough

The bridge creates a supported model once. It requires exact joint-name ordering and a complete finite vector. Incoming header timestamps are checked against the ROS system clock; the watchdog uses a monotonic clock. This minimal example assumes same-host clocks and does not enable simulated ROS time. Do not set use_sim_time without adding a /clock publisher and revising the timestamp contract.

A 10 ms timer advances five 2 ms physics steps and publishes feedback. The nominal wall rate is 100 Hz, not a guaranteed physical deadline. The wave node captures the first feedback pose, then publishes a smooth 50 Hz elbow target. Stopping publication causes the bridge to request its home pose after 100 ms. That is a simulation teaching fallback, not a physical emergency stop.

JointState effort here is simulated actuator force for gear-one hinge motors. A physical driver must document whether its reported value is command torque, current-derived torque, or independently measured output effort. Use SensorDataQoS for appropriate high-rate lossy telemetry only after checking compatibility; use short queues and expiration for commands even if reliable QoS is chosen.

I. Expected result

Two nodes appear, the left elbow moves in the simulation viewer, and /joint_states updates near the nominal 100 Hz wall timer. There is no TF publisher in this minimal bridge; RViz requires robot_state_publisher with the pinned URDF and matching mesh resolution as an extension.

J. Validation

This edition's ROS package was syntax-checked but not executed against a local Jazzy runtime. On your machine verify build, discovery, topic counts, actual viewer movement, stale-target rejection and fallback after the command node stops. Record the measured topic rate; a timer setting alone is not evidence. Then test a zero or old header stamp and a reordered name array.

K. Troubleshooting

No rclpy import usually means the wrong Python/venv. No package discovery suggests the workspace was not built or sourced. Invisible topics across terminals may indicate ROS_DOMAIN_ID or middleware configuration differences. Mesh failures in RViz are a separate URDF path issue. An idle bridge can be correct if command timestamps are rejected.

L. Practical exercise

Add a diagnostic topic reporting command age, rejected-message count and simulation real-time factor. Then place the bridge behind a lifecycle activation state so it refuses targets until configured.

M. Connection to the physical Asimov 1

Node boundaries and message contracts transfer. The physical backend needs a verified hardware interface, deterministic local execution, release-specific safety state transitions and tested power isolation. Ordinary ROS Python timers do not establish real-time performance.

N. Completion checklist

The package builds on Jazzy, two nodes exchange named data, stale targets are rejected, and stopping the publisher produces the documented simulation fallback. ROS integration remains a reader-side gate until these results are recorded.

O. Next lesson preview

Investigate support geometry and unsupported dynamics before training a policy.

Lesson 6 - Static balance and the limits of a supported pose

A. Why this matters

A robot can look upright because it is fixed to the world, balanced quasi-statically, or dynamically stabilized. Those are three different engineering situations. Begin by identifying which constraints supply the forces.

B. Concepts explained simply

Center of mass (COM) is the mass-weighted average of link centers: c=sum(m_i*c_i)/sum(m_i). Two masses, 2 kg at x=0 and 1 kg at x=.3 m, have COM x=.1 m. The support polygon is the convex hull of active contact patches. For slow, level-ground conditions, the COM's vertical projection inside that polygon is a useful static check, not a general dynamic guarantee.

Center of pressure (COP) locates the resultant normal contact force on a support plane. With two vertical loads of 100 N at x=-.1 m and 200 N at x=.1 m, COP x=.0333 m. Zero Moment Point (ZMP) is a related dynamic criterion under particular planar-contact assumptions; acceleration and angular momentum matter. COP, COM and ZMP are not interchangeable.

A linear inverted pendulum approximation uses x_ddot=(g/h)*(x-zmp) at approximately fixed COM height. At h=.6 m and x-zmp=.01 m, acceleration is about .1635 m/s^2. Small position errors can grow; being centered once is insufficient.

C. Real-world relevance

Feet may slip, lift or deform. A harness can unload the robot and change the experiment. Report support forces and constraints when claiming standing performance.

D. Architecture

Model mass and contacts -> support estimate -> COM and orientation estimate -> posture correction -> limited joint torques -> contact feedback. Static geometry informs planning; feedback and contact constraints determine dynamic recovery.

E. Prerequisites

Joint control, sensor semantics and kinematics. No physical standing tests are authorized by a simulator result.

F. Step-by-step implementation

Run the deliberately passive free-base baseline and read the JSON report. It applies no stabilizing motor commands. Its purpose is to demonstrate the failure a standing controller must improve upon.

```
python balance.py
```

Next implement supported posture tracking with the existing controller and compare to that baseline, labeling the fixture clearly. For actual unsupported standing research, use the stand task in Lesson 7 and measure it; do not silently treat a training run as solved.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/balance.py

/data/Robotics_Project/humanoid_lab/balance.py

```
"""Static support geometry and a deliberately passive free-base fall test."""
import json
import numpy as np
import mujoco as mj
from model import ROOT, load

def margin_rectangle(point, xmin, xmax, ymin, ymax):
    x, y = point
    return float(min(x-xmin, xmax-x, y-ymin, ymax-y))

def passive_trial(seed):
    rng = np.random.default_rng(seed)
    m, d = load(supported=False)
    d.qvel[:3] = rng.normal(0, .01, 3)
    start = d.qpos[:3].copy()
    fallen = False
    peak_tilt = 0.
    for _ in range(1000):
        mj.mj_step(m, d)
        rot = d.xmat[m.body('pelvis_link').id].reshape(3, 3)
        tilt = np.arccos(np.clip(rot[2, 2], -1, 1))
        peak_tilt = max(peak_tilt, float(tilt))
        if d.qpos[2] < .35 or tilt > .6:
            fallen = True; break
    return dict(seed=seed, fallen=fallen, time=float(d.time),
                displacement=float(np.linalg.norm(d.qpos[:2]-start[:2])),
                peak_tilt_rad=peak_tilt)

if __name__ == '__main__':
    # An illustrative polygon, not a measured Asimov foot polygon.
    print('Illustrative COM margin:', margin_rectangle([.01, 0], -.1, .1, -.08, .08))
    trials = [passive_trial(i) for i in range(5)]
    (ROOT/'logs/passive_trials.json').write_text(json.dumps(trials, indent=2))
    print(trials)
```

H. Code walkthrough

margin_rectangle is a signed margin to four rectangle edges for a point. Its sample rectangle is illustrative, not measured Asimov geometry. A general foot-support hull needs actual contact points and convex-polygon edge distances. passive_trial loads a floating base, introduces a small seeded initial velocity, advances up to two seconds, and labels a fall when pelvis height or tilt crosses an explicitly chosen threshold.

The world-up test uses the pelvis rotation matrix's z-axis component. Quaternion components themselves should not be thresholded as if they were

tilt angles. Five seeds expose repeatability; they do not establish robust control.

I. Expected result

The prepared passive trials all fell in roughly half a simulated second. This is a useful negative baseline. A fixed-pelvis trajectory can still look stable because the fixture supplies missing support.

J. Validation

Check positive static margin inside a rectangle and negative margin outside. Evaluate any standing controller over at least 20 held-out seeds and varied initial state before widening disturbances. Proposed research gate: 60 s per run, no falls in the declared test set, bounded tracking error and no unexplained sustained torque saturation. This is a simulation gate, not a physical safety certification.

K. Troubleshooting

A zero fall rate from a fixed root is invalid evidence. Rising energy or large contact impulses may indicate poor initialization or gains. Torque saturation can make an otherwise good geometry infeasible. More friction can hide foot-slip failures instead of solving them.

L. Practical exercise

Add a small known horizontal force for a short interval in simulation. Record force magnitude, duration, impulse and exact application point, then compare recovery with an unperturbed run.

M. Connection to the physical Asimov 1

Support estimates need measured mass, contact properties and sensor calibration. A physical rig must handle the robot and a stopped or failed joint without relying on software torque.

N. Completion checklist

You can separate supported motion from static and dynamic balance, compute a simple COM/COP example, and report both successes and falls under declared conditions.

O. Next lesson preview

Train a small supported task, then extend the same environment contract to standing research.

Lesson 7 - Reinforcement learning without hidden assumptions

A. Why this matters

Learning can discover feedback strategies, but a reward is only a surrogate objective. You need enough control knowledge to recognize an unrealistic policy before deploying it.

B. Concepts explained simply

The environment advances the simulated world. The agent chooses an action from an observation. An episode begins with reset and ends with a task failure or time limit. A policy maps observations to actions; a value function estimates future reward. Training changes policy parameters; evaluation freezes them.

PPO, Proximal Policy Optimization, constrains policy updates using a clipped probability-ratio objective so updates do not move too far from the data-generating policy at once. It still needs good observations, action scales, rewards and evaluation. Stable-Baselines3 implements PPO using PyTorch; the environment implements Gymnasium's reset/step contract. [S11, S12]

For the first task the pelvis is fixed and the policy controls one elbow target offset. It observes joint errors, velocities, gravity direction, velocity channels, its previous action and the requested elbow goal. The reward penalizes squared goal error, action changes and effort. This teaches an environment contract before introducing difficult contacts.

C. Real-world relevance

A deployable policy needs a documented observation order, normalization, action scaling, control period, actuator model and failure behavior. A neural network file alone does not specify a robot controller.

D. Architecture

Reset and randomization -> observation -> PPO actor -> bounded position target -> PD inner loop -> ten physics steps -> observation, reward, termination and metrics. Training and inference live in separate processes from hardware transport.

E. Prerequisites

Lessons 1-6, CPU PyTorch and the pinned Gymnasium/SB3 packages. Begin on CPU. A large GPU does not fix an erroneous joint order or reward.

F. Step-by-step implementation

```
python learning.py check --task reach
python learning.py train --task reach --steps 100000
python learning.py evaluate --task reach
python learning.py export --task reach
```

The environment checker verifies interface compatibility; it cannot verify that the reward represents a useful physical task. Training saves checkpoints and a final policy. Evaluation uses seeds 100-119, separate from training seed 7. Export writes a traced actor and metadata that must travel together.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/learning.py

/data/Robotics_Project/humanoid_lab/learning.py

```
"""CPU Gymnasium tasks: supported reach, free-base stand, free-base walk."""
import argparse
import json
import gymnasium as gym
import mujoco as mj
import numpy as np
from gymnasium import spaces
from model import ROOT, load, addresses, names, command

class HumanoidTask(gym.Env):
    metadata = {'render_modes': []}
    def __init__(self, task='reach'):
        super().__init__()
        if task not in ('reach', 'stand', 'walk'):
            raise ValueError(task)
        self.task = task
        self.m, self.d = load(supported=task == 'reach')
        self.ids, self.qi, self.vi = addresses(self.m)
        self.home = self.m.qpos0[self.qi].copy()
        self.elbow = names(self.m).index('left_elbow_joint')
        self.n = 1 if task == 'reach' else self.m.nu
        self.action_space = spaces.Box(-1., 1., (self.n,), np.float32)
        self.observation_space = spaces.Box(-np.inf, np.inf,
            (2*self.m.nu+9+self.n+1,), np.float32)
        self.nominal_mass = self.m.body_mass.copy()
        self.nominal_inertia = self.m.body_inertia.copy()
        self.previous = np.zeros(self.n)
        self.steps = 0
        self.goal = self.home[self.elbow]+.1

    def observation(self):
        rot = self.d.xmat[self.m.body('pelvis_link').id].reshape(3, 3)
        if self.task == 'reach':
            velocity, omega = np.zeros(3), np.zeros(3)
        else:
            # Free-joint translation velocity is world-frame; rotate to body.
            velocity = rot.T @ self.d.qvel[:3]
            omega = self.d.qvel[3:6].copy()
        return np.concatenate([self.d.qpos[self.qi]-self.home,
            self.d.qvel[self.vi], rot.T @ np.array([0., 0., -1.]),
            velocity, omega, self.previous,
            [self.goal-self.home[self.elbow]]]).astype(np.float32)

    def reset(self, *, seed=None, options=None):
        super().reset(seed=seed)
        mj.mj_resetData(self.m, self.d)
        # Correlated mass/inertia scaling keeps the randomized inertias physical.
        scale = self.np_random.uniform(.95, 1.05)
        self.m.body_mass[:] = self.nominal_mass*scale
        self.m.body_inertia[:] = self.nominal_inertia*scale
        mj.mj_setConst(self.m, self.d)
```

```
        self.steps = 0; self.previous = np.zeros(self.n)
        self.goal = self.home[self.elbow]+self.np_random.uniform(.05, .15)
        mj.mj_forward(self.m, self.d)
        return self.observation(), {}

    def step(self, action):
        action = np.asarray(action, dtype=float)
        if action.shape != (self.n,) or not np.isfinite(action).all():
            raise ValueError('Invalid action')
        action = np.clip(action, -1, 1)
        target = self.home.copy()
        if self.task == 'reach':
            target[self.elbow] += .2*action[0]
        else:
            target += .15*action
        for _ in range(10):
            command(self.m, self.d, target)
            mj.mj_step(self.m, self.d)
        self.steps += 1
        rot = self.d.xmat[self.m.body('pelvis_link').id].reshape(3, 3)
        upright = float(rot[2, 2])
        error = float(self.d.qpos[self.qi[self.elbow]]-self.goal)
        smooth = float(np.sum((action-self.previous)**2))
        effort = float(np.mean(self.d.ctrl**2))
        fallen = self.task != 'reach' and (self.d.qpos[2] < .35 or upright < .8)
        bad = not np.isfinite(self.d.qpos).all()
        if self.task == 'reach':
            reward = -100*error**2-.005*smooth-.0001*effort
        else:
            vx = float(self.d.qvel[0])
            desired = .15 if self.task == 'walk' else 0.
            reward = 1.+upright-np.square(vx-desired)*3-.01*smooth-.0001*effort
            if fallen: reward -= 10.
        self.previous = action.copy()
        return self.observation(), float(reward), bool(fallen or bad), self.steps >= 250, {
            'error_rad': abs(error), 'upright': upright, 'fallen': bool(fallen)}

def main():
    p = argparse.ArgumentParser()
    p.add_argument('mode', choices=['check', 'train', 'evaluate', 'export'])
    p.add_argument('--task', default='reach', choices=['reach', 'stand', 'walk'])
    p.add_argument('--steps', type=int, default=100000)
    args = p.parse_args()
    from stable_baselines3 import PPO
    from stable_baselines3.common.env_checker import check_env
    from stable_baselines3.common.monitor import Monitor
    from stable_baselines3.common.callbacks import CheckpointCallback
    folder = ROOT/'logs'/args.task; folder.mkdir(exist_ok=True)
    env = HumanoidTask(args.task)
    if args.mode == 'check':
        check_env(env); print('Environment contract passed'); return
    if args.mode == 'train':
        model = PPO('MlpPolicy', Monitor(env, str(folder/'monitor')), seed=7,
                    device='cpu', n_steps=512, batch_size=64, verbose=1)
        model.learn(total_timesteps=args.steps, callback=CheckpointCallback(
            save_freq=10000, save_path=str(folder), name_prefix='checkpoint'))
        model.save(folder/'policy')
```

```
        (folder/'metadata.json').write_text(json.dumps(dict(
            task=args.task, seed=7, action_dim=env.n,
            observation_dim=env.observation_space.shape[0],
            joints=names(env.m), dt_control=.02, mujoco=mj.__version__), indent=2))
    elif args.mode == 'evaluate':
        model = PPO.load(folder/'policy', device='cpu')
        results = []
        for seed in range(100, 120):
            obs, _ = env.reset(seed=seed); total = 0.; errors=[]
            for _ in range(250):
                action, _ = model.predict(obs, deterministic=True)
                obs, r, end, cutoff, info = env.step(action)
                total += r; errors.append(info['error_rad'])
                if end or cutoff: break
            results.append(dict(seed=seed, total=total,
                final_error=float(np.mean(errors[-50:])), **info))
        (folder/'evaluation.json').write_text(json.dumps(results, indent=2))
        print('Mean final error:', np.mean([x['final_error'] for x in results]))
        print('Fall fraction:', np.mean([x['fallen'] for x in results]))
    else:
        import torch
        model = PPO.load(folder/'policy', device='cpu')
        class Actor(torch.nn.Module):
            def __init__(self, policy):
                super().__init__(); self.policy = policy
            def forward(self, observation):
                return self.policy(observation, deterministic=True)[0]
        sample = torch.zeros((1, env.observation_space.shape[0]))
        actor = Actor(model.policy).eval()
        traced = torch.jit.trace(actor, sample)
        assert torch.allclose(actor(sample), traced(sample))
        traced.save(str(folder/'actor.pt'))
        print('Exported inference artifact; physical deployment remains gated')
    env.close()

if __name__ == '__main__':
    main()
```

H. Code walkthrough

Observation length is 2*nu+9+action_dim+1. The gravity direction comes from the pelvis rotation. Supported reach uses zero base velocity channels. The initial elbow goal is randomized over a small range. Action [-1,1] becomes a bounded .2 rad elbow offset; stand/walk use .15 rad offsets for all 25 joints. These are teaching scales, not a tuned locomotion configuration.

reset seeds the environment generator, restores state, and applies a small common mass/inertia factor. This preserves physical inertia consistency while testing a narrow kind of mismatch. It is a first domain-randomization example, not comprehensive sim-to-real randomization. step returns terminated for falls or invalid physics and truncated at 250 control steps. Treating a time limit as a fall changes value targets incorrectly.

The loop uses 2 ms physics and 20 ms policy updates. The command function supplies an ideal PD inner loop. The network learns targets, not raw motor bus messages. Export checks the traced actor against its eager output on a sample. The preparation audit also compared eager and exported actions on 100 rollout observations. SB3 predict clips Box actions; an exported actor must receive the same external action clipping. Observation-schema hash checks and replay on hardware recordings are still required before deployment.

I. Expected result

A valid environment, readable training metrics, checkpoints, metadata, a deterministic evaluation report and actor.pt. A 2,048-step smoke run only shows that the plumbing executes; it may not solve the reaching objective.

J. Validation

Measure mean absolute elbow error over the final 50 steps per held-out episode. Compare against a no-motion baseline and the classical controller with direct goal access. Proposed reach gate: mean final error below .03 rad with no nonfinite states across 20 held-out seeds. Reject a policy that wins through a sensor-order bug or simulator exploit. Rerun with multiple training seeds before claiming reproducible learning.

K. Troubleshooting

Reward improves but error does not: inspect individual reward terms and action scaling. Exploding values: verify finite observations and reward magnitude. Policy works only at reset: check whether goal information or time leaks into evaluation. Poor standing after reach success is expected; the dynamics and action dimension changed.

L. Practical exercise

Log each reward component separately. Remove the effort penalty in a simulation experiment and observe whether the policy uses more saturation to improve error. Add observation noise on copies and quantify performance loss.

M. Connection to the physical Asimov 1

The exported actor remains offline until hardware observation, timing, actuator and safety contracts pass. Randomize measured delays, friction, backlash, motor strength, noise and contact properties using defensible ranges from identification, not arbitrary values chosen to make a policy look robust.

N. Completion checklist

You can explain every observation/action element, reproduce a seeded evaluation, compare baselines, separate termination from truncation and reject reward hacking. Training throughput and wall-time estimates come from your own benchmark.

O. Next lesson preview

Extend evaluation to locomotion and learn what evidence a walking claim requires.

Lesson 8 - Standing, walking and turning as research milestones

A. Why this matters

Locomotion coordinates contact transitions while preserving feasible torques and momentum. A robot moving forward while falling is not walking, and a large reward is not a gait evaluation.

B. Concepts explained simply

A gait alternates stance, when a foot supports load, and swing, when it moves toward a future contact. A foot trajectory needs position, velocity and clearance conditions. Velocity commands specify intent, while balance and contact controllers determine whether that intent is feasible.

Classical gait generation makes phases and constraints explicit and is easier to inspect, but model error can be costly. Whole-body optimization can coordinate tasks while enforcing constraints, but requires accurate models and careful numerical design. Learned policies can handle complex contact behavior, but can exploit simulation and require broad testing. Combine approaches only with a clear authority boundary.

C. Real-world relevance

Foot slip, terrain compliance, ankle mapping and timing delays can destroy an otherwise good simulated gait. Turning adds yaw dynamics and changes support geometry; it is a separate test axis.

D. Architecture

Velocity command -> gait or learned policy -> whole-body/joint targets -> constrained feedback -> contacts -> state estimator -> gait/policy feedback. A fall detector and command watchdog remain outside the reward.

E. Prerequisites

Repeatable unsupported standing evaluation and the learned-task contract. The included stand and walk variants are a runnable research starting point, not validated locomotion solutions.

F. Step-by-step implementation

```
python learning.py check --task stand
python learning.py train --task stand --steps 100000
python learning.py evaluate --task stand
```

Do not start walking deployment until standing meets your declared gate. The walk variant rewards forward velocity near .15 m/s. It intentionally does not supply a claimed solved policy. Run it as a training experiment after standing, then inspect videos, contacts and metrics. It has no turning command in the current observation; add an explicit yaw-rate command, observation and tracking term before attempting turning training.

G. Complete code

The complete environment, training and evaluation code is learning.py in Lesson 7, with --task stand or --task walk. The following complete evaluator reads a walking trajectory CSV with columns t,x,y,yaw,vx,vy,wz,cmd_vx,cmd_vy,cmd_wz,power_w,slip_mps,fallen. Produce those channels from your simulator/controller before claiming a gait result.

```
# /data/Robotics_Project/humanoid_lab/gait_metrics.py
import argparse
import numpy as np

def summarize(path):
    x = np.genfromtxt(path, delimiter=',', names=True)
    if len(x) < 2 or not np.all(np.diff(x['t']) > 0):
        raise ValueError('Need increasing timestamped samples')
    distance = float(np.sum(np.hypot(np.diff(x['x']), np.diff(x['y']))))
```

```
    tracking = np.sqrt(np.mean((x['vx']-x['cmd_vx'])**2+
                               (x['vy']-x['cmd_vy'])**2))
    yaw_tracking = np.sqrt(np.mean((x['wz']-x['cmd_wz'])**2))
    energy = np.trapezoid(np.maximum(x['power_w'], 0), x['t'])
    return dict(distance_m=distance, velocity_rmse_mps=float(tracking),
                yaw_rate_rmse_radps=float(yaw_tracking),
                positive_energy_j=float(energy),
                slip_rms_mps=float(np.sqrt(np.mean(x['slip_mps']**2))),
                fell=bool(np.any(x['fallen'] > 0)))

if __name__ == '__main__':
    p = argparse.ArgumentParser(); p.add_argument('csv')
    print(summarize(p.parse_args().csv))
```

H. Code walkthrough

The basic stand/walk reward is intentionally small: uprightness, velocity error, action smoothness and effort. It is insufficient by itself to exclude all unwanted gaits. Add lateral velocity, yaw-rate, foot-clearance, joint-speed, contact-slip and posture constraints only after inspecting failure cases. Reward design should follow a failure explanation, not random coefficient tuning.

The metric program integrates positive power over actual time. Mechanical power tau*dq differs from battery electrical power; report which one you logged. Slip should be measured only for a declared loaded contact, not a swinging foot. A zero slip measurement with no foot load is meaningless.

I. Expected result

You obtain performance data and possibly failures. This book does not claim that the supplied reward and 100,000 steps produce standing, walking or turning. Those are gated research deliverables requiring iteration.

J. Validation

For each final policy, evaluate multiple training seeds and at least 20 held-out episodes per condition. Include still commands, starts/stops, forward/backward/lateral commands, turn commands after implemented, friction variation, modest slopes and declared push impulses. Proposed simulation gate: zero falls in the published test matrix, velocity RMSE under .1 m/s at .15 m/s command, and bounded slip and torque saturation with thresholds chosen before evaluation. Report confidence limits; zero observed falls is not zero real-world risk.

K. Troubleshooting

A policy may maximize reward by shuffling, hopping or exploiting favorable collisions. A baseline can fail because the default pose starts in penetration. Contact jitter may require a smaller timestep or better collision geometry rather than more RL steps. Do not disable self-collision solely to improve scores without documenting it.

L. Practical exercise

Add command-conditioned yaw-rate tracking and create a held-out turn test. Preserve a no-turn policy as a baseline and compare energy and fall fraction.

M. Connection to the physical Asimov 1

No simulated gait authorizes physical walking. Release-matched policy interfaces, calibrated ankles, measured timing and protected commissioning come first. Initial real tests remain supported and slow.

N. Completion checklist

Your gait claim has a defined test matrix, raw traces, videos, tracking/slip/energy metrics, fall counts and an explicit statement of model assumptions. Failed gates block progression.

O. Next lesson preview

Add perception and high-level skills without giving an AI planner unrestricted motor access.

Lesson 9 - Perception and embodied behavior

A. Why this matters

Perception should change a robot's choice of validated behavior. A detector alone is not embodied AI, while a motion script that ignores feedback is not a robust interaction.

B. Concepts explained simply

A pinhole camera maps a 3-D point to pixels: u=fx*X/Z+cx and v=fy*Y/Z+cy. With fx=400 px, cx=160 px and X/Z=.1, u=200 px. One pixel coordinate does not determine Z. A monocular colored blob supplies an image-space direction, not a verified metric reach target.

HSV separates hue from brightness and saturation, making simple color segmentation convenient under controlled lighting. Connected components group adjacent mask pixels. This is a classical vision algorithm; a trained object detector is a different technique requiring its own model license, accuracy checks and compute budget.

C. Real-world relevance

Camera intrinsics, lens distortion, exposure, lighting and camera-to-body calibration affect object estimates. Depth requires an actual depth source, known geometry or a calibrated inference method with uncertainty.

D. Architecture

Camera or test image -> calibrated preprocessing -> detector -> timestamped observation -> behavior state machine -> prevalidated skill -> command gate -> controller -> feedback. A language model may select a permitted skill name and bounded parameters, never arbitrary control code.

E. Prerequisites

Lessons 4-5, OpenCV, and a supported simulated motion. Start with synthetic images so detector failures are separable from camera transport.

F. Step-by-step implementation

```
python perception.py
```

Inspect logs/object.png and logs/detection.json. The green circle should be detected near pixel (210,110). The JSON explicitly leaves depth_m null. For a real camera extension, log its calibration ID, capture timestamp, image dimensions and detector result before connecting behavior.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/perception.py

/data/Robotics_Project/humanoid_lab/perception.py

```
"""A reproducible color detector with explicit unknown depth."""
import json
import cv2
import numpy as np
from model import ROOT

def detect(bgr):
    hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, np.array([40, 80, 70]), np.array([85, 255, 255]))
    count, labels, stats, centers = cv2.connectedComponentsWithStats(mask)
    if count <= 1:
        return None
    k = 1+int(np.argmax(stats[1:, cv2.CC_STAT_AREA]))
    if stats[k, cv2.CC_STAT_AREA] < 100:
        return None
    u, v = centers[k]
    return dict(u=float(u), v=float(v), depth_m=None,
                area_px=int(stats[k, cv2.CC_STAT_AREA]))

if __name__ == '__main__':
    image = np.zeros((240, 320, 3), dtype=np.uint8)
    cv2.circle(image, (210, 110), 20, (0, 255, 0), -1)
    result = detect(image)
    assert result is not None and abs(result['u']-210) < 1
    assert detect(np.zeros_like(image)) is None
    cv2.imwrite(str(ROOT/'logs/object.png'), image)
    (ROOT/'logs/detection.json').write_text(json.dumps(result, indent=2))
    print(result)
```

H. Code walkthrough

BGR-to-HSV conversion uses OpenCV's channel convention. inRange makes a binary green mask. connectedComponentsWithStats excludes background and selects the largest component above a minimum area. Returning None is an ordinary 'not observed' state. It must not be converted to a zero coordinate that looks like a valid target.

A skill library should expose small interfaces such as observe_object, supported_wave and return_to_fixture_home. Validate object identity, observation age, preconditions, amplitude and timeout before execution. The planner's output needs a strict schema and an allowlist. Unknown skills and out-of-range values are rejected. Speech recognition and natural-language planning can be added after the deterministic path passes.

I. Expected result

The synthetic positive image yields a centroid and area; the empty image yields no object; no unearned depth estimate appears. No model training or external AI service is required.

J. Validation

Test no-object, correct color, distractor color, too-small blob and multiple components. Evaluate real-image precision/recall under a declared lighting setup before enabling physical gestures. Stale detections must trigger observation refresh or no motion.

K. Troubleshooting

Wrong colors often mean RGB/BGR confusion. Hue ranges depend on OpenCV's representation. A green wall can be a larger component than the target; object identity is not guaranteed by color. Unstable detections require temporal validation and uncertainty handling, not merely larger gestures.

L. Practical exercise

Require three consistent detections over a short time window before selecting a skill. Add a disappearance test that cancels an unstarted action without inventing a new target.

M. Connection to the physical Asimov 1

The documented monocular camera is useful for direction and appearance tasks. Metric reaching requires calibrated geometry and additional constraints. Use a small supported gesture as the first demonstration rather than depth-dependent approaching.

N. Completion checklist

You can distinguish image-space detection from 3-D localization, handle missing observations, explain calibration requirements and constrain the planner to an allowlisted skill.

O. Next lesson preview

Define the contracts and rollback required to move from simulated actions to a physical stack.

Part IV - From simulation evidence to physical hardware

Lesson 10 - Sim-to-real contracts, identification and rollback

A. Why this matters

Simulation supplies perfect state, immediate commands and simplified actuators. Physical systems have stale packets, flexible structures, thermal limits and changing batteries. Deployment begins by measuring these differences and making them explicit.

B. Concepts explained simply

System identification estimates model parameters from experiments. Calibration aligns a measurement to a reference, such as encoder zero or camera geometry. Validation tests whether the calibrated model predicts new data. Domain randomization trains over a range of plausible parameters; it cannot excuse a completely wrong mechanism.

Compare a low-amplitude measured joint response against the simulator using the same sampled command. Fit only parameters the experiment can identify. For example, a static gravity test helps estimate load torque and offset but poorly identifies inertia. A slow reversal can reveal backlash or stiction; it is not a thermal test.

C. Real-world relevance

Mass distribution, friction, cable forces, motor saturation, encoder quantization, transport delay, compliance, battery voltage and foot-ground contact all change performance. A policy relying on true base velocity cannot deploy until a usable estimator supplies that signal.

D. Architecture

Simulation artifact -> schema and hash checks -> offline replay -> timing test -> hardware-in-the-loop with output inhibited -> single-joint fixture -> supported module -> protected whole body -> limited autonomous skill. Each stage has a separate go/no-go record.

E. Prerequisites

Successful numerical tests, held-out policy evaluation if a learned policy is used, release-specific hardware documentation and a reviewed physical support/stop strategy. You can complete offline work before buying actuators.

F. Step-by-step implementation

Create a deployment manifest with model/firmware/policy hashes, joint and motor orders, SI units, signed zeros, observation names, normalization constants, action scaling, controller frequency, maximum command age and fault response. Replay recorded observations through exported inference and compare with the training implementation.

Then test safety.py entirely offline. Its timestamps are supplied from one assumed monotonic clock; across hosts, redesign the protocol with measured clock synchronization, sequence handling and arrival-time constraints. Do not subtract unrelated host clocks and call the result latency.

```
python safety.py
python -m pytest -q tests/test_contracts.py
```

G. Complete code

File: /data/Robotics_Project/humanoid_lab/safety.py

/data/Robotics_Project/humanoid_lab/safety.py

```
"""Offline command-gate exercise. Not a certified hardware safety system."""
from dataclasses import dataclass
import numpy as np

@dataclass
class Gate:
    lower: np.ndarray
    upper: np.ndarray
    max_speed: float = .2
    timeout: float = .1
    last_sequence: int = -1
    fault: str = ''

    def validate(self, now, sent, sequence, target, measured, dt):
        target, measured = np.asarray(target), np.asarray(measured)
        checks = [
            (not np.isfinite([now, sent, dt]).all() or dt <= 0, 'invalid time'),
            (now-sent < 0 or now-sent > self.timeout, 'stale or future command'),
            (sequence <= self.last_sequence, 'replayed sequence'),
            (target.shape != self.lower.shape or measured.shape != target.shape,
             'shape mismatch')]
        for invalid, reason in checks:
            if invalid: self.fault = reason
        if not self.fault:
            if not np.isfinite(target).all() or not np.isfinite(measured).all():
                self.fault = 'nonfinite state'
            elif np.any(target < self.lower) or np.any(target > self.upper):
                self.fault = 'position limit'
            elif np.any(np.abs(target-measured) > self.max_speed*dt):
                self.fault = 'step envelope'
        if self.fault:
            raise RuntimeError(self.fault)
        self.last_sequence = sequence
        return target.copy()

if __name__ == '__main__':
    g = Gate(np.array([-1.]), np.array([1.]))
    print(g.validate(1., .99, 1, [.001], [0.], .01))
    try:
        g.validate(2., .99, 2, [.001], [0.], .01)
    except RuntimeError as e:
        print('Latched expected fault:', e)
```

H. Code walkthrough

Gate validates timing, sequence, shape, finiteness, range and per-command step envelope. It latches a fault, so a subsequent healthy packet cannot silently resume motion. A deliberate, reviewed reset procedure is required outside this teaching class. The class is an offline exercise, not a complete protective control system.

The step-envelope comparison does not by itself measure actual speed, and a valid position target can still demand unsafe torque. Production supervision also needs measured speed/current/temperature/voltage, travel margin, rate and acceleration limits, estimator health, missed-deadline counters and independent stop authority. Failure handling must be appropriate for a loaded robot: blindly commanding zero torque can release a limb or collapse the machine.

I. Expected result

A small valid target is accepted. An old packet raises a latched fault. No motor library is imported and no hardware output is possible.

J. Validation

Stage	Proposed go criteria	Rollback
Offline replay	Exact schema; eager/exported outputs agree on recorded dataset; no NaNs	Restore previous policy/config bundle
Timing under load	Measured worst-case inference plus transport fits reviewed deadline with margin	Reduce load or restore prior controller
HIL inhibited	Correct decoding, joint mapping, timeouts and fault state; output verified disabled	Disconnect transport and inspect logs
Single joint	Approved low-energy envelope, bounded tracking and stop response	Isolate power with load mechanically supported
Supported module	Repeatable low-speed trajectory; no thermal or deadline violations	Return to single joint and previous calibration
Whole-body support	Rig and stop tests passed; all local interfaces verified	Maintain support; isolate hazardous power
Physical skill	Repeated supervised trials within declared envelope	Disable skill; restore validated release

Record the method used to prove output inhibition and the instrument used to measure timing. A software flag saying 'disabled' is not sufficient evidence of electrical isolation.

K. Troubleshooting

Export mismatch often means normalization, action squashing, joint ordering or dtype changed. Good nominal timing with rare overruns requires stress tests and worst-case analysis. A policy that fails only after several minutes may be responding to thermal derating or voltage droop rather than perception.

L. Practical exercise

Replay duplicated, delayed, out-of-order and future-stamped commands through Gate. Confirm all expected faults are recorded and remain latched. Add a schema version mismatch case.

M. Connection to the physical Asimov 1

Ankle joint coordinates and motor A/B coordinates need a validated mechanism transform. Preserve power consistency: if joint rates equal J times motor rates, motor torques equal J transpose times joint torques under the ideal mapping. Do not reuse a small-angle approximation outside its verified range.

N. Completion checklist

You have a manifest, reproducible replay, measured timing, explicit stops and rollback procedures, and no dependency on unavailable ground-truth channels. Hardware deployment remains no-go until its own gates pass.

O. Next lesson preview

Convert an attractive platform into a complete, release-matched procurement decision.

Lesson 11 - Build a defensible EUR 6,000 budget

A. Why this matters

Actuator cost is only one part of a humanoid build. Fabrication, interfaces, tooling, protection, support, shipping and VAT can dominate the remaining budget. An unquoted line is an unknown liability, not zero euros.

B. Concepts explained simply

A bill of materials is a quantity-controlled list tied to a release. A quotation states a supplier, item revision, price, tax basis, availability, shipping and validity period. A budget allowance is an internal spending cap. These are different documents.

Landed total = parts + fabrication + shipping + applicable taxes/duties + tools/protection + spares + contingency. Avoid adding VAT twice to VAT-inclusive EU prices. Treat optional computer upgrades separately until the budget owner confirms inclusion.

C. Real-world relevance

Replacing a motor with a cheaper torque-equivalent part can change mounting, inertia, encoder semantics, protocol and thermal behavior. Compatibility is an engineering decision, not a price-filter checkbox.

D. Architecture

Select hardware release -> extract part quantities -> obtain supplier quotes -> verify compatibility -> reconcile landed costs -> reserve contingency -> purchase one stage -> reforecast remaining stages.

E. Prerequisites

Official BOM access, CAD revision, motor datasheets/protocols, firmware availability, existing tool inventory and a location suitable for a guarded fixture. The public material inspected here did not establish a complete purchasable EU BOM.

F. Step-by-step implementation

Read the itemized CSV below, keep missing prices blank, and run the audit. It deliberately reports approved=false because a known subtotal is not a feasibility total.

```
python audit.py budget config/bom.csv
```

The actuator grouping is derived from the official module checklist: three EC-A6416-P2-25, four EC-A5013-H17-100, four EC-A3814-H14-107, four EC-A4315-P2-36 and ten EC-A4310-P2-36, totaling 25. This is a procurement grouping, not a protocol specification. [S13]

A directly opened BerryBase listing showed EUR 189.00 including VAT, excluding shipping, for a Raspberry Pi 5 8 GB on 25 October 2026. It is a dated price observation, not a reserved quote. Cooling, storage, power accessories and carrier boards are additional. The supplied CSV records this single verified price; all other unresolved costs remain explicit. [S14]

G. Complete code and procurement ledger

File: /data/Robotics_Project/humanoid_lab/audit.py

/data/Robotics_Project/humanoid_lab/audit.py

```
"""Procurement completeness and recorded commissioning gate checker."""
import argparse
import csv
import json
from pathlib import Path
from model import ROOT

def budget(path):
    rows = list(csv.DictReader(Path(path).open()))
    verified = 0.; unknown=[]
    for row in rows:
        if row['verified_eur']:
            verified += float(row['quantity'])*float(row['verified_eur'])
        else:
            unknown.append(row['component'])
    return dict(known_subtotal_eur=round(verified, 2),
                unquoted=unknown, approved=False,
                reason='Incomplete quotes' if unknown else 'Requires compatibility review')

def commissioning(path):
    report = json.loads(Path(path).read_text())
    required = ['mechanical_inspection', 'polarity', 'protective_devices',
                'support_rig', 'emergency_isolation', 'watchdog',
                'encoder_direction', 'limits', 'thermal', 'rollback']
    failed = [x for x in required if report.get(x) != 'PASS']
    return dict(go=not failed, failed=failed,
                note='Record review only; not independent physical verification')

if __name__ == '__main__':
    p = argparse.ArgumentParser(); p.add_argument('mode', choices=['budget', 'commission'])
    p.add_argument('path'); args = p.parse_args()
    print(json.dumps(budget(args.path) if args.mode == 'budget'
                     else commissioning(args.path), indent=2))
```

File: /data/Robotics_Project/humanoid_lab/config/bom.csv

Item / quantity	Specification	Price evidence / supplier	Compatibility / priority
EC-A6416-P2-25 / 3	Exact released actuator	Unquoted / Manufacturer quote required	Revision and firmware confirm / P0
EC-A5013-H17-100 / 4	Exact released actuator	Unquoted / Manufacturer quote required	Revision and firmware confirm / P0
EC-A3814-H14-107 / 4	Exact released actuator	Unquoted / Manufacturer quote required	Revision and firmware confirm / P0
EC-A4315-P2-36 / 4	Exact released actuator	Unquoted / Manufacturer quote required	Revision and firmware confirm / P0
EC-A4310-P2-36 / 10	Exact released actuator	Unquoted / Manufacturer quote required	Ankle mapping required / P0
Frames and machined parts / 1	Release-specific set	Unquoted / EU fabrication quote required	CAD revision confirm / P0
Bearings fasteners shafts couplers / 1	Release-specific set	Unquoted / BOM quantities required	No substitution approved / P0
MJF printed parts / 1	PA12 set	Unquoted / EU bureau quote required	Material process match / P0
Motor drivers / 1	Confirm integrated versus additional	Unquoted / Actuator vendor	Do not double count / P0
Raspberry Pi 5 8GB / 1	Board only	EUR 189.00 / BerryBase snapshot 2026-10-25	Pi5 role verified accessories extra / P1
Radxa CM5 and custom PCBs / 1	MCB PDB head board media HAT	Unquoted / Fabrication quote required	Matched release / P0
Bench supply and distribution / 1	Voltage and regen approval pending	Unquoted / EU supplier quote required	Engineering review / P0
Battery charger protection / 1	Released matched set	Unquoted / Vendor quote required	Defer for bench platform / P2

Item / quantity	Specification	Price evidence / supplier	Compatibility / priority
Fuses contactor emergency stop / 1	DC-rated engineered set	Unquoted / EU supplier quote required	Rating approval pending / P0
Microcontroller CAN interface / 1	Release-specific interfaces	Unquoted / EU supplier quote required	Protocol confirmation / P0
IMU camera encoders / 1	Confirm integrated sensors first	Unquoted / EU supplier quote required	No double counting / P1
Cooling cabling connectors / 1	Release-specific set	Unquoted / EU supplier quote required	Crimp and current review / P0
Tools and metrology / 1	Required set	Unquoted / EU supplier quote required	Existing inventory credit / P0
Support rig and barriers / 1	Load-rated reviewed system	Unquoted / EU fabricator quote required	Fall arrest and access review / P0
Spare components / 1	Critical spares	Unquoted / Quote required	Same revisions / P1
Shipping VAT import charges / 1	Landed cost reconciliation	Unquoted / Supplier quote required	Avoid VAT double count / P0
Contingency / 1	Uncommitted reserve	Unquoted / Budget allowance	Not a supplier price / P0

The CSV quantity '1' for an assembly set is a pending quote scope, not an assertion that the set contains one fastener or one bearing. Replace set rows with actual part-number rows before ordering. Motor drivers, encoders and IMUs may be integrated: reconcile them to prevent double counting.

H. Code walkthrough

budget totals only explicitly priced quantities and reports every unquoted category. It never concludes that a robot fits a budget from a partial subtotal. commissioning uses a different path to evaluate recorded readiness; it does not verify physical tests independently.

Use priority P0 for required motion/protection dependencies, P1 for the selected research capability, and P2 for deferred expansion. Remove an

optional battery from a bench stage only after choosing a suitable power/regen design; do not remove emergency isolation or support to create apparent savings.

I. Expected result

The priced board subtotal is EUR 189.00, with many unquoted categories and no purchase approval. That honest result is more useful than a fabricated EUR 5,999 full-robot estimate.

J. Validation and three scenarios

Scenario	Technical scope	Financial conclusion
A: complete Asimov 1	25-actuator machine, structure, custom boards, matched firmware, rig and tools	Not established within EUR 6,000; published USD 20,000 kit does not fit
B: lower-body research	12 actuators, pelvis, full safety/support, power and estimator	Unquoted; fewer joints do not prove affordability or safe locomotion
C: modular bench platform	One joint first, then a small supported limb or head/arm subset	Preferred staged scope; buy only after exact module quote and rig review

For Scenario C, an illustrative allocation ceiling is: EUR 1,500 actuators/drives; EUR 700 mechanical work; EUR 600 bench power/protection; EUR 400 compute/CAN/sensing; EUR 700 fixture and barriers; EUR 450 tools; EUR 300 spares; EUR 450 shipping/tax adjustments; EUR 900 contingency. Total EUR 6,000. These are planning allowances, not supplier prices or proof that the selected Asimov module fits. If the first compatible actuator exceeds its allocation, reduce scope or revise the funding decision before ordering.

A lower-body pair uses two hip-pitch, two hip-roll, two hip-yaw, two knee and four ankle actuators. Obtain twelve release-matched motor quotes and fabrication estimates before considering Scenario B. A seated or bench arm can support perception/control research without claiming biped locomotion.

K. Troubleshooting

A cheap listing may exclude the drive, encoder, shipping or tax. A STEP inventory does not encode all purchasing quantities. An available model number may ship with a different firmware. Quote validity and lead time matter as much as the price.

L. Practical exercise

Obtain quotes for one exact motor, its required interface, compatible power/protection and one mounting fixture. Record manufacturer confirmation of firmware/protocol and mounting compatibility before comparing prices.

M. Connection to the physical Asimov 1

Use the official release BOM and manufacturer confirmation to choose exact parts. EU electronics suppliers and fabrication bureaus are candidates for quotations; this edition does not assert stock or compatibility for unquoted items.

N. Completion checklist

All required rows have quantities, exact specifications, tax basis, landed cost and compatibility evidence; contingency remains funded; and the selected stage stays below its approved ceiling. Until then, stay with simulation and procurement research.

O. Next lesson preview

Inspect and fabricate mechanical parts using load paths and process constraints.

Lesson 12 - Mechanical design, fabrication and incremental assembly

A. Why this matters

The model assumes rigid links and known axes. Poor fit, weak layer orientation or a misaligned bearing changes those assumptions and may create a failure under load.

B. Concepts explained simply

STEP describes solid geometry suitable for engineering exchange. STL describes a triangle surface and normally carries neither a trustworthy unit declaration nor manufacturing tolerances. A pretty mesh is not a complete manufacturing drawing.

A load path traces force through joints, bearings, fasteners and structure into the support. A 2 kg mass at a 0.2 m lever arm produces about 3.92 Nm of static gravity torque before acceleration or impacts. An illustrative 10 Nm torque applied through a 0.05 m lever corresponds to 200 N. Those simple calculations help identify where detailed analysis is needed, but do not size a human-safe support rig.

C. Real-world relevance

Fastener preload, bearing fits, shaft alignment and cyclic fatigue affect service life. A cable crossing a joint can apply unmodeled torque or be cut by a moving edge.

D. Architecture

Released CAD and drawings -> process/material specification -> load and fit review -> first article -> dimensional inspection -> unpowered assembly -> guarded joint test -> limb -> torso integration.

E. Prerequisites

Exact release documents, competent mechanical review for load-bearing parts and rig, suitable metrology, a parts traveler and a de-energized work area. Never infer a tightening torque from the bolt diameter alone; material, grade, lubrication and joint design matter.

F. Step-by-step implementation

1. Inventory the selected module's released part numbers, drawing revisions and required quantities.
2. Trace gravity, acceleration and stop loads through the module and its fixture. Review bending, bearing and fastener loads.
3. Identify every fit and datum from drawings. Confirm units with a known dimension before exporting STL or ordering machining.
4. Fabricate one first article. Measure critical holes, bearing seats, flatness and mating distances before ordering the rest.
5. Dry-assemble unpowered. Confirm free movement, alignment, cable clearance, end stops and access for service.
6. Apply only documented assembly practices, lubrication and fastener treatments. Record torque tool identity and inspection results.
7. Reinspect after the first guarded motion and thermal cycle before expanding to a limb.

G. Complete inspection program

This lesson's executable artifact checks a simple dimensional traveler. Numerical tolerances below are examples of a file format, not approved Asimov tolerances.

```
# /data/Robotics_Project/humanoid_lab/inspect_parts.py
import csv
import sys

def inspect(path):
    failures=[]
    for row in csv.DictReader(open(path, newline='')):
        value=float(row['measured_mm'])
        lo, hi=float(row['min_mm']), float(row['max_mm'])
        if not lo <= value <= hi:
            failures.append(row['part']+':'+row['feature'])
    return failures

if __name__ == '__main__':
    failed=inspect(sys.argv[1])
    print('FAILED FEATURES:', failed)
    raise SystemExit(bool(failed))
```

CSV header: part,feature,min_mm,max_mm,measured_mm. Populate its bounds from the approved drawing, never from an illustrative tutorial number.

H. Walkthrough and material choices

The program compares measured dimensions to released bounds and returns a failing exit code for a nonconforming feature. It does not inspect material strength, surface condition or preload. Keep instrument uncertainty and calibration in the traveler.

PETG is useful for noncritical fixtures and covers when heat/creep constraints permit. ABS/ASA may suit warmer noncritical housings with appropriate printing practice. Nylon performance depends on moisture, formulation and process. MJF PA12 has different process properties from FDM nylon. None is an automatic substitute for a released load-bearing part. Machined aluminum or steel features require the specified alloy, heat treatment, fit and finish.

FDM strength is anisotropic: a load across layer interfaces can fail differently from a load along continuous extrusion. Orientation, inserts, wall geometry and temperature affect the result. Test coupons and representative joints under the relevant load cases. Do not replace a specified 7075 component with a printed replica merely because the shape fits.

I. Expected result

A dimensionally inspected, unpowered module that moves freely through its reviewed travel without cable strain or interference. Any critical nonconformance blocks powered testing.

J. Validation

Check identification, material/process certificates where needed, drawing dimensions, bearing fit, fastener engagement, end-stop clearance, cable bend radius and fixture attachment. Measure backlash and axis alignment. Have a qualified designer determine proof-load procedures for the support; do not improvise a drop test with a robot attached.

K. Troubleshooting

Binding often indicates misalignment, warped prints or an overconstrained bearing arrangement. A loose fit should not be hidden with adhesive unless approved. Threads that strip during assembly require a design/process review, not more tightening.

L. Practical exercise

Create a serviceability drawing showing how to remove one actuator without cutting cables or unloading another unsupported structure.

M. Connection to the physical Asimov 1

The repository includes process-labeled fabrication directories. Preserve their material/process intent and reconcile part numbers to the exact assembly manual revision before manufacture.

N. Completion checklist

Critical dimensions pass; material/process choices are justified; all loads have a continuous path into a reviewed fixture; and a second person can follow the traveler.

O. Next lesson preview

Design the power and communication experiment before energizing a single actuator.

Lesson 13 - Electrical and embedded engineering on a single joint

A. Why this matters

A motor can generate torque unexpectedly, and a stopped motor can still draw substantial current. Electrical protection and mechanical support must work before software is trusted.

B. Concepts explained simply

Voltage sets an electrical potential; current produces heating and magnetic force; power is V*I. A 48 V, 2 A example consumes 96 W electrically. This is an arithmetic example, not a recommended supply setting. Motor phase current and battery bus current are different quantities. Peak currents, acceleration, regeneration and duty cycle matter.

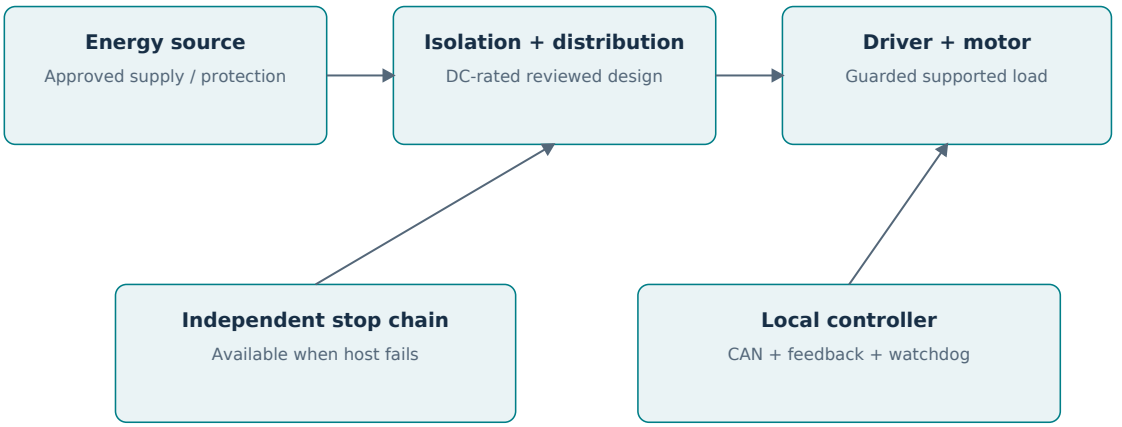
A fuse protects a circuit against specified fault conditions; it does not necessarily react fast enough to protect a person or motor semiconductor. A contactor must be rated to interrupt the actual DC circuit. An emergency-stop button is only one component of a reviewed isolation system. Regeneration

can raise bus voltage when a motor is driven by a falling load; some bench supplies cannot absorb it.

C. Real-world relevance

Connector shell names do not establish pin order, polarity or current rating. The public design contains board files and wiring inventories, but a release-specific schematic and manufacturer protocol are still required for your exact assembly.

D. Architecture



The figure is a functional energy/control architecture, not a wiring schematic. Wire gauges, fuse values, contactor ratings, pinouts, precharge and regen provisions must be selected from approved hardware data and load analysis.

E. Prerequisites

Exact actuator datasheet/protocol revision, drive configuration tool, verified harness/schematic, current-limited supply approved for the actuator and regen case, suitable instruments, guarded fixture, independent isolation and competent electrical review.

F. Step-by-step implementation

1. De-energize and disconnect all energy sources. Verify isolation with a suitable instrument and protect stored-energy outputs.
2. Inspect conductor identification, polarity, insulation, strain relief, terminal retention and continuity against the approved schematic.

3. Verify supply range, current limiting, short-circuit protection, inrush/precharge and regeneration handling before attaching the actuator.
4. Support the output mechanically and clear the swept volume. Set released low-energy commissioning limits using the vendor tool before motion is possible.
5. Connect one actuator only. Read identity, firmware and feedback with torque disabled using documented commands.
6. Assign a unique ID using the official procedure. Record the physical label and bus mapping.
7. Test the independent isolation and watchdog with the mechanical load secured. Record whether the actuator actually stops producing hazardous motion.
8. Enable a small bounded motion only after direction, zero, limits and fault behavior are verified. Stop and inspect before adding another joint.

No guessed CAN payloads, voltage settings or connector pinouts are supplied. The public materials inspected do not establish a complete validated transport for your purchased release. A fabricated runnable motor command would be worse than an explicit gate.

G. Complete offline verification code

Use safety.py from Lesson 10 and audit.py from Lesson 11 to validate command envelopes and recorded readiness before any transport exists. A physical driver must be implemented from the confirmed protocol and tested on an inhibited-output fixture. The complete commissioning record template is:

/data/Robotics_Project/humanoid_lab/config/commissioning.json

```
{
  "mechanical_inspection": "NOT_TESTED",
  "polarity": "NOT_TESTED",
  "protective_devices": "NOT_TESTED",
  "support_rig": "NOT_TESTED",
  "emergency_isolation": "NOT_TESTED",
  "watchdog": "NOT_TESTED",
  "encoder_direction": "NOT_TESTED",
  "limits": "NOT_TESTED",
  "thermal": "NOT_TESTED",
  "rollback": "NOT_TESTED"
}
```

H. Walkthrough and CAN fundamentals

CAN uses a differential pair on a terminated bus. A conventional high-speed CAN segment normally has a 120-ohm termination at each physical end; two in parallel produce about 60 ohms in a de-energized resistance check when the design permits that measurement. This is a general diagnostic, not permission to add resistors without checking existing integrated termination and the release topology. Avoid arbitrary star wiring and long stubs.

Confirm bit rate, transceiver type, termination location, common-mode/ground strategy and ID uniqueness. A USB-to-CAN adapter's connector or software API does not prove protocol compatibility. A custom embedded loop should verify command age/sequence and sensor freshness locally, reject malformed data, and transition to a mechanically safe reviewed state if the host disappears.

I. Expected result

One identified actuator reports credible telemetry with its output initially disabled. The isolation and communication-failure experiments are documented before low-amplitude motion is attempted.

J. Validation

Require signed polarity/continuity inspection, supply behavior under a dummy load where appropriate, documented current limiting, unique ID, encoder-direction agreement, configuration readback, watchdog timing and emergency-isolation effect. Compare measured stop behavior against the reviewed risk envelope. A 'PASS' typed into JSON without evidence is not a test.

K. Troubleshooting

Intermittent CAN errors may indicate termination, ground, bit-rate or signal-integrity issues. Brownouts may be inrush or shared return impedance. A bus voltage spike during deceleration suggests regen handling. Stop to diagnose rather than repeatedly re-enabling a faulting drive.

L. Practical exercise

Capture an inhibited-output communication log and annotate request, response, sequence, timestamp, error flags and timeout. Confirm the same log decodes identically offline.

M. Connection to the physical Asimov 1

The manual requires assigning IDs individually before assembly. Follow its release-specific procedure; do not send a broadcast assignment on a populated robot bus. [S15]

N. Completion checklist

One joint passes identity, power, communication, feedback, limits and independent stop tests. No unresolved pinout, voltage, protocol or support assumption remains. Only then duplicate the validated module.

O. Next lesson preview

Commission a supported system while preserving local timing and safety authority.

Lesson 14 - Assembly, calibration and supported commissioning

A. Why this matters

Integration creates faults that single-component tests miss: reversed encoders, overloaded supplies, cable interference, mixed firmware and incorrect model zeros. Commissioning should isolate these faults in a controlled sequence.

B. Concepts explained simply

A zero offset aligns the encoder coordinate to the model reference pose. A direction sign aligns positive rotation. Gear ratio and parallel linkages may require a nonlinear mapping. Calibration is incomplete until a separate measurement validates the result over the intended range.

C. Real-world relevance

A biped with motor power removed may collapse. A supported fixture can reduce consequences, but it must be designed for the machine, test envelope and foreseeable failures. A tether that merely keeps the robot nearby is not necessarily a fall-arrest system.

D. Architecture

Mechanical support and isolation -> embedded protection -> real-time joint loop -> state estimator -> ROS supervision -> allowed skill. Activation requires evidence from every lower layer.

E. Prerequisites

Release-matched hardware, completed single-joint tests, load-reviewed rig, clear exclusion zone, accessible isolation, trained observer and documented shutdown/rollback procedures.

F. Step-by-step implementation

Step	Measurement and pass criterion	If it fails
Inspection	Correct parts, retained cables, fasteners and support attachment	Repair unpowered and reinspect
Power	Polarity, current limiting, protection and bus behavior match reviewed design	Disconnect and correct power design
Isolation	Each E-stop/disconnect achieves reviewed physical outcome under support	Remain no-go; repair isolation chain
Identity	Every actuator ID and firmware matches the release inventory	Test one actuator at a time
Direction	Small commanded positive change matches independent observation	Disable; correct mapping and retest
Zero	Measured reference pose matches model within approved tolerance	Recalibrate with support
Limits	Conservative position/current/speed limits read back correctly	Reconfigure while inhibited
IMU	Static orientations and zero-rate bias agree with references	Correct mounting transform/calibration
Limb motion	Small slow trajectory tracks without binding or unexplained heating	Return to joint-level investigation
Supported body	No deadline, power or mapping violations across repeated trials	Disable higher-level skill
Contact tests	Rig, foot load and contact observations match declared setup	Reassess support and model
Progression	Previous gates and reviewer approval recorded	Remain at supported stage

Set numerical tolerances from measurement uncertainty, the manufacturer and the reviewed experiment envelope. Do not use a generic 1-degree or temperature limit merely because it looks reasonable. Store the threshold and rationale with each test.

G. Complete readiness-check command

```
python audit.py commission config/commissioning.json
```

The complete program is printed in Lesson 11. Its default record is NOT_TESTED for every item, so the supplied result is no-go. Add evidence identifiers and reviewer names to a controlled record; do not replace real tests with automated approval.

H. Walkthrough and physical software boundaries

High-level compute handles ROS, perception, planning, orchestration and telemetry. The timing-critical controller validates feedback age, limits, setpoints and inference deadlines. Embedded drivers enforce local watchdogs, protection and bus semantics. Record measured cycle-time histograms and missed deadlines under realistic CPU/network load; ordinary WebSockets or MQTT are suitable for telemetry and high-level requests, not the direct stabilization loop.

Calibrate the parallel ankle in motor coordinates and joint coordinates using a verified geometric mapping and independent measurement. Validate its position and torque transforms together. A left/right sign fix in the UI must not hide an inconsistent lower-level convention.

I. Expected result

A calibrated supported module or body executes a small reviewed trajectory while feedback, temperatures, current, timing and faults are logged. You can trigger the documented stop and explain how loads remain supported.

J. Validation

Repeat each trial after a cold start, a software restart and a fault reset. Disconnect host communication intentionally with output support in place. Confirm no automatic motion resumes when packets return. Require manual re-arming through the reviewed procedure. Initial commissioning excludes unsupported dynamic walking.

K. Troubleshooting

A growing tracking error can be binding, saturation or bad zeroing. A different pose after reboot can indicate unsaved offsets or absolute-encoder interpretation. Thermal drift needs measured duty-cycle and cooling analysis. Do not hide intermittent faults by increasing watchdog timeouts without a timing justification.

L. Practical exercise

Perform an unpowered shutdown rehearsal with two people: name who isolates energy, who controls the rig, where stored energy remains, and how the faulted configuration is preserved for diagnosis.

M. Connection to the physical Asimov 1

Use the manufacturer's matched firmware/images and released procedures. A public API describing STAND/MOVE modes does not establish readiness of your assembly. Keep software compliance modes distinct from emergency isolation.

N. Completion checklist

Inspection, calibration, limits, timing, rig, isolation, watchdog and rollback all have dated evidence. A physical policy is disabled until those records pass.

O. Next lesson preview

Integrate perception with a small validated behavior and publish its evidence.

Part V - The end-to-end research demonstration

Lesson 15 - Recognize an object and execute a validated skill

A. Why this matters

An end-to-end demonstration should prove a traceable chain from observation to bounded motion and feedback. Limit the first task to the physical platform you actually have.

B. Concepts explained simply

A behavior state machine separates OBSERVE, VALIDATE, EXECUTE, VERIFY, COMPLETE and FAULT. Each transition has a condition. A skill has preconditions, bounded parameters, timeout, success criteria and cancellation behavior. 'Object seen' is not the same event as 'task complete'.

C. Real-world relevance

A one-joint supported platform can detect a colored object and make a gesture. A full-body platform with unresolved walking cannot safely promise approach or grasp. Match the demonstration to verified capability.

D. Architecture

Synthetic or camera frame -> classical detector -> permitted supported gesture -> command-envelope check -> bounded controller -> feedback log -> completion report. A future language planner may select the same gesture through a strict allowlisted schema.

E. Prerequisites

Perception tests, supported control, offline command-gate tests and logging. A physical run additionally requires every hardware commissioning gate and a verified transport implementation.

F. Step-by-step implementation

```
python perception.py
python demo.py
cat logs/demo.json
```

The included run uses a synthetic image and the fixed-pelvis Asimov model. It moves the left elbow through a smooth three-second out-and-back gesture. This completes the software chain without claiming a camera is mounted on actual hardware.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/demo.py

/data/Robotics_Project/humanoid_lab/demo.py

```
"""Detect an object and execute a bounded supported elbow gesture."""
import json
import numpy as np
import mujoco as mj
from model import ROOT, load, addresses, names, command
from perception import detect
from safety import Gate

def run():
    frame = np.zeros((240, 320, 3), np.uint8)
    frame[90:130, 190:230, 1] = 255
    detection = detect(frame)
    m, d = load(); ids, qi, vi = addresses(m)
    home = d.qpos[qi].copy(); joint = names(m).index('left_elbow_joint')
    gate = Gate(m.jnt_range[ids, 0], m.jnt_range[ids, 1], max_speed=1.)
    records=[]; previous=home.copy()
    if detection is None:
        return {'state': 'NO_OBJECT', 'motion': False}
    # Prevalidated gesture only. No depth-dependent reach is attempted.
    for k in range(1500):
        t = k*m.opt.timestep
        goal = home.copy()
        goal[joint] += .05*(1-np.cos(2*np.pi*t/3.))
        approved = gate.validate(t, t, k, goal, previous, m.opt.timestep)
        command(m, d, approved); previous=approved.copy()
        mj.mj_step(m, d)
        records.append([float(d.time), float(d.qpos[qi[joint]])])
    report = {'state': 'COMPLETE_SIMULATION', 'motion': True,
              'object': detection, 'steps': len(records),
              'max_elbow_rad': max(x[1] for x in records)}
    (ROOT/'logs/demo.json').write_text(json.dumps(report, indent=2))
    np.savetxt(ROOT/'logs/demo.csv', records, delimiter=',', header='t,elbow')
    return report

if __name__ == '__main__':
    print(run())
```

H. Code walkthrough

The detector produces a centroid with unknown depth. The behavior chooses a predefined gesture, avoiding an unjustified metric reach. Each setpoint is checked against the command envelope before PD execution. The gate checks consecutive command steps in this demonstration; a separate tracking monitor must compare actual joint motion and measured speed before physical use.

The simulator logs feedback and writes a final JSON state. A physical skill implementation needs measured success criteria and an explicit FAULT/cancel path; a completed for-loop alone is not physical task verification. Extend the test with no-object input and fault injection before extending its motion.

I. Expected result

logs/demo.json reports COMPLETE_SIMULATION, 1,500 physics steps and an object observation. logs/demo.csv records elbow feedback. The prepared run's peak elbow angle was about .8843 rad; it includes the model's initial reference angle, so it is not a .8843 rad excursion.

J. Validation

Require detector positive and negative cases, finite feedback, bounded commands, final tracking-error measurement and cancellation tests. Demonstrate rejection of a stale command, invalid target and unsupported skill request. A physical first demo should pass repeated supervised trials without protection trips, unusual heating or missed-deadline violations under the same declared setup.

K. Troubleshooting

If motion occurs without a valid observation, the behavior transition is wrong. If detection succeeds but no command is sent, inspect preconditions and gate reasons. If commands are sent but tracking fails, diagnose the controller/physical layer separately from vision.

L. Practical exercise

Replace the synthetic image with a recorded, calibrated camera frame and keep the skill amplitude unchanged. Add three-frame observation consistency and a skill timeout before using a live camera stream.

M. Connection to the physical Asimov 1

Reuse the detector, state-machine design, schemas and logs. Replace only the verified simulation backend with a commissioned hardware interface; then revalidate timing, tracking, thermal behavior and stop handling. The full Asimov and a smaller module need different preconditions.

N. Completion checklist

You can trace a command to its observation, identify every gate, measure actual feedback and demonstrate a no-motion result on invalid input. Label the result as simulation, supported physical module or protected whole-body test accurately.

O. Next lesson preview

Turn a demonstration into a reproducible research result that another engineer can evaluate.

Lesson 16 - Research quality, maintenance and the next build

A. Why this matters

A working demonstration can still be fragile. Research value comes from repeatability, documented limits, baseline comparisons and an honest account of failures.

B. Concepts explained simply

Unit tests verify small properties. Integration tests verify components together. Simulation regressions detect changes in behavior under fixed conditions. Bench tests check actual interfaces. Validation measures the capability claimed; verification checks implementation against specified requirements. Both are necessary.

C. Real-world relevance

A version update, replacement actuator or changed cable routing can invalidate previous evidence. Configuration changes need explicit impact review and a path back to a known-good release.

D. Architecture

Commit -> pinned environment -> model/config hashes -> numerical tests -> simulation matrix -> release artifact -> commissioning record -> monitored deployment -> logged evidence -> issue and revision.

E. Prerequisites

All relevant earlier gates, a clear capability statement and retained raw data. Failed experiments belong in the record.

F. Step-by-step implementation

Create a release folder containing source revision, environment freeze, model hash, policy metadata, calibration IDs, test commands, numeric outputs, evaluation matrix and rollback instructions. Keep personal credentials and large raw training files out of git. Make logs immutable or uniquely named per experiment.

```
cd /data/Robotics_Project/humanoid_lab
python -m pytest -q
python learning.py check --task reach
python simulate.py --headless --seconds 10
python demo.py
python -m pip freeze > requirements-lock.txt
```

Use headless CI for numerical and fault tests. Add a separate manually controlled hardware test stage so an automatic software push cannot unexpectedly energize a mechanism. Pin CI runner images by digest for a release; do not assume a floating ubuntu-latest runner is your Ubuntu 24.04 environment.

G. Complete code

The complete numerical and contract tests are printed in Appendix A. Use them unchanged as the first regression suite. The commands above define the minimal release check; physical tests remain separately gated.

H. Walkthrough

A test asserting only that a constant equals itself adds no engineering evidence. Useful tests here check model dimensions, bounded torque, nonfinite-target rejection, IK residual, fault latching, missing depth and seeded resets. Broader tests should target changed risks: frame conversions, policy parity, transport decoding, observation age and physical limit readback.

Benchmark simulated seconds per wall second, inference latency percentiles and worst observed latency, deadline misses, memory use and log throughput. Report hardware, sample count, load, seeds and warm-up. An average latency alone hides rare overruns.

I. Expected result

Another engineer can reproduce the supported simulation demo and understand exactly which ROS, learning and physical milestones still require evidence. The completed book remains a working laboratory guide rather than a promise of a finished walking machine.

J. Validation

Perform a clean-environment rebuild, rerun the suite, reproduce one positive and one negative demonstration, and verify the rollback bundle restores the previous measured result. For every claimed capability, identify the raw evidence and the conditions where it has not been tested.

K. Troubleshooting

A result that cannot be reproduced may depend on an untracked model, seed, environment variable or calibration. A regression hidden by wider tolerances needs explanation. Restore the last good release, isolate the change and rerun the narrow relevant experiment first.

L. Practical exercise

Write a one-page research report: hypothesis, baseline, intervention, test matrix, quantitative results, failures, limits and next experiment. Make the next experiment answer one concrete uncertainty.

M. Connection to the physical Asimov 1

The same release discipline supports future limb additions and whole-body work. Re-run impact-relevant commissioning after any structural, power, firmware, estimator or policy change.

N. Completion checklist

Your demo has a precise capability statement, reproducible software, preserved logs, known limitations and tested rollback. The next build decision is grounded in measured gaps and a complete procurement forecast.

O. Next lesson preview

Choose the next research question from evidence: better state estimation, a calibrated physical joint, a more reliable skill, or contact-rich locomotion. Keep the same gates as scope increases.

Part VI - Mathematical foundations in depth

The following ten laboratories develop the language used by every later robot component: units, vectors, rotations, transforms, derivatives, fitting, numerical integration, uncertainty and sampling. Work through the derivations before running the code. A number should carry its unit and frame all the way from a sensor through an algorithm into a log. The examples are intentionally small enough to calculate by hand and large enough to expose errors that disappear in a trivial zero-pose test.

Laboratory 1 - Units, dimensional reasoning and motor-speed conversion

A. The engineering question

A number without a unit is an incomplete interface.

B. Concepts and mathematical model

Radians are dimensionless in dimensional analysis, but treating an angle as interchangeable with a count or degree is a software error. Start from the quantity you want, write the conversion chain, and cancel the units before calculating. A gearbox changes both angular speed and available torque; it does not create power. Efficiency represents loss, while a gear ratio represents an ideal geometric relationship. This experiment keeps input and output shaft quantities separate and exposes why an RPM value cannot be sent straight to a radians-per-second controller.

omega_motor = RPM*2*pi/60; omega_joint = omega_motor/G; tau_joint = eta*G*tau_motor; P = tau*omega.

C. Real-world meaning and limits

Confirm whether a vendor encoder reports motor-shaft or output-shaft position before applying a ratio. Actual Asimov actuator settings must come from its confirmed release.

D. Architecture and data flow

Configuration (rpm, ratio, motor_torque, efficiency) -> numerical model -> joint_speed_rad_s, joint_torque_Nm, input_power_W -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

RPM is motor-shaft speed; ratio is motor revolutions per output revolution; motor_torque is shaft Nm; efficiency is a fraction.

Parameter	Nominal value	Interpretation
rpm	1200	motor revolutions per minute
ratio	36	dimensionless motor/output ratio
motor_torque	0.3	Nm
efficiency	0.8	fraction

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/01_units.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only ratio changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project
```

```
.venv-book/bin/python expanded_lab/modules/01_units.py --case 0
```

```
.venv-book/bin/python expanded_lab/modules/01_units.py --case 3

.venv-book/bin/python expanded_lab/run.py units --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/01_units.py

```
"""Units, dimensional reasoning and motor-speed conversion. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rpm, ratio = p["rpm"], p["ratio"]
    torque, efficiency = p["motor_torque"], p["efficiency"]
    assert rpm > 0 and ratio > 0 and 0 < efficiency <= 1
    motor_speed = rpm * 2 * np.pi / 60
    joint_speed = motor_speed / ratio
    joint_torque = torque * ratio * efficiency
    input_power = torque * motor_speed
    output_power = joint_torque * joint_speed
    recovered_rpm = joint_speed * ratio * 60 / (2 * np.pi)
    assert abs(recovered_rpm - rpm) < 1e-9
    metrics = dict(
        joint_speed_rad_s=joint_speed,
        joint_torque_Nm=joint_torque,
        input_power_W=input_power,
        output_power_W=output_power,
        loss_W=input_power - output_power,
    )
    derivation = [
        f"Motor speed: {rpm:g} rev/min times 2*pi/60 = {motor_speed:.6f} rad/s.",
        f"Divide by ratio {ratio:g}: output speed = {joint_speed:.6f} rad/s.",
        f"Multiply input torque {torque:g} Nm by ratio and efficiency: {joint_torque:.6f}
    Nm.",
        f"Input mechanical power = {input_power:.6f} W; output = {output_power:.6f} W.",
        f"The difference, {input_power-output_power:.6f} W, is modeled mechanical loss.",
    ]
    notes = [
        f"The ideal power ratio is reduced to {efficiency:.3f}; a torque increase does not
    imply a power increase.",
        "This constant-efficiency model excludes speed-dependent loss, driver current and
    thermal limits.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {
        "rpm": 1200.0,
        "ratio": 36.0,
        "motor_torque": 0.3,
        "efficiency": 0.8,
```

```
}
parameters["ratio"] = [6.0, 12.0, 25.0, 36.0, 50.0, 100.0][args.case]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Trace the speed conversion first, then the torque relation, then check power. The recovered RPM assertion is a round-trip invariant; it catches a conversion direction error that a plausible-looking output torque would not.

I. Expected behavior and six worked cases

The controlled variable is ratio. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

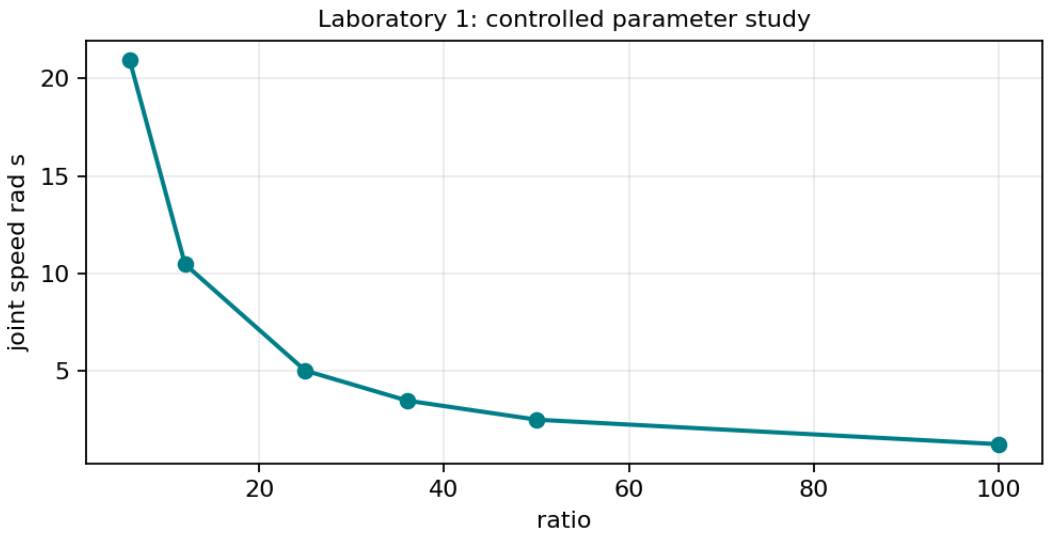


Figure L1. joint speed rad s as ratio changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	ratio	joint speed rad s
0	6	20.94395102
1	12	10.47197551
2	25	5.02654825
3	36	3.4906585
4	50	2.51327412
5	100	1.25663706

Worked case 0: ratio = 6

This case keeps the nominal configuration except for ratio=6. The full input is {"efficiency": 0.8, "motor_torque": 0.3, "ratio": 6.0, "rpm": 1200.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
joint speed rad s	20.94395102
joint torque Nm	1.44
input power W	37.69911184
output power W	30.15928947
loss W	7.53982237

1. Motor speed: 1200 rev/min times 2*pi/60 = 125.663706 rad/s.
2. Divide by ratio 6: output speed = 20.943951 rad/s.
3. Multiply input torque 0.3 Nm by ratio and efficiency: 1.440000 Nm.
4. Input mechanical power = 37.699112 W; output = 30.159289 W.
5. The difference, 7.539822 W, is modeled mechanical loss.

Worked case 1: ratio = 12

This case keeps the nominal configuration except for ratio=12. The full input is {"efficiency": 0.8, "motor_torque": 0.3, "ratio": 12.0, "rpm": 1200.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
joint speed rad s	10.47197551
joint torque Nm	2.88
input power W	37.69911184
output power W	30.15928947
loss W	7.53982237

1. Motor speed: 1200 rev/min times 2*pi/60 = 125.663706 rad/s.
2. Divide by ratio 12: output speed = 10.471976 rad/s.
3. Multiply input torque 0.3 Nm by ratio and efficiency: 2.880000 Nm.
4. Input mechanical power = 37.699112 W; output = 30.159289 W.
5. The difference, 7.539822 W, is modeled mechanical loss.

Worked case 2: ratio = 25

This case keeps the nominal configuration except for ratio=25. The full input is {"efficiency": 0.8, "motor_torque": 0.3, "ratio": 25.0, "rpm": 1200.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
joint speed rad s	5.02654825
joint torque Nm	6
input power W	37.69911184
output power W	30.15928947

Computed quantity	Numerical result
loss W	7.53982237

1. Motor speed: 1200 rev/min times 2*pi/60 = 125.663706 rad/s.
2. Divide by ratio 25: output speed = 5.026548 rad/s.
3. Multiply input torque 0.3 Nm by ratio and efficiency: 6.000000 Nm.
4. Input mechanical power = 37.699112 W; output = 30.159289 W.
5. The difference, 7.539822 W, is modeled mechanical loss.

Worked case 3: ratio = 36

This case keeps the nominal configuration except for ratio=36. The full input is {"efficiency": 0.8, "motor_torque": 0.3, "ratio": 36.0, "rpm": 1200.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
joint speed rad s	3.4906585
joint torque Nm	8.64
input power W	37.69911184
output power W	30.15928947
loss W	7.53982237

1. Motor speed: 1200 rev/min times 2*pi/60 = 125.663706 rad/s.
2. Divide by ratio 36: output speed = 3.490659 rad/s.
3. Multiply input torque 0.3 Nm by ratio and efficiency: 8.640000 Nm.
4. Input mechanical power = 37.699112 W; output = 30.159289 W.
5. The difference, 7.539822 W, is modeled mechanical loss.

Worked case 4: ratio = 50

This case keeps the nominal configuration except for ratio=50. The full input is {"efficiency": 0.8, "motor_torque": 0.3, "ratio": 50.0, "rpm": 1200.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
joint speed rad s	2.51327412
joint torque Nm	12
input power W	37.69911184
output power W	30.15928947
loss W	7.53982237

1. Motor speed: 1200 rev/min times 2*pi/60 = 125.663706 rad/s.
2. Divide by ratio 50: output speed = 2.513274 rad/s.
3. Multiply input torque 0.3 Nm by ratio and efficiency: 12.000000 Nm.
4. Input mechanical power = 37.699112 W; output = 30.159289 W.
5. The difference, 7.539822 W, is modeled mechanical loss.

Worked case 5: ratio = 100

This case keeps the nominal configuration except for ratio=100. The full input is {"efficiency": 0.8, "motor_torque": 0.3, "ratio": 100.0, "rpm": 1200.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
joint speed rad s	1.25663706
joint torque Nm	24
input power W	37.69911184
output power W	30.15928947

Computed quantity	Numerical result
loss W	7.53982237

1. Motor speed: 1200 rev/min times 2*pi/60 = 125.663706 rad/s.
2. Divide by ratio 100: output speed = 1.256637 rad/s.
3. Multiply input torque 0.3 Nm by ratio and efficiency: 24.000000 Nm.
4. Input mechanical power = 37.699112 W; output = 30.159289 W.
5. The difference, 7.539822 W, is modeled mechanical loss.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The ideal power ratio is reduced to 0.800; a torque increase does not imply a power increase.

This constant-efficiency model excludes speed-dependent loss, driver current and thermal limits.

K. Failure modes and troubleshooting

Confusing motor current with output torque; forgetting 60 seconds per minute; using peak torque as a continuous operating point.

L. Practical exercises and extension tasks

1. Hold output speed constant while increasing ratio: calculate the required motor speed.
2. Set efficiency to one and prove input/output mechanical power agree.
3. Introduce a 5% torque-constant error and propagate it to output torque.

M. Connection to the Asimov research platform

Confirm whether a vendor encoder reports motor-shaft or output-shaft position before applying a ratio. Actual Asimov actuator settings must come from its confirmed release.

N. Completion record

Record your chosen case, predicted trend, measured joint_speed_rad_s, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 2, Vectors, projections and joint-axis torque. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 2 - Vectors, projections and joint-axis torque

A. The engineering question

A force vector contains both magnitude and direction.

B. Concepts and mathematical model

The moment about an origin is r cross F, where r points from that origin to the point of application. A joint only responds to the component of that moment along its rotation axis. This explains why the same external force can produce different joint torques at different poses. A projection is meaningful only when both vectors are expressed in the same coordinate frame and the axis is normalized. In this laboratory the changing lever length is geometric; it is not an actuator specification.

moment = r cross F; tau_axis = axis dot moment; F_parallel = axis*(axis dot F).

C. Real-world meaning and limits

A limb load must be transformed into each joint frame along the chain; this is the foundation of Jacobian-transpose force mapping.

D. Architecture and data flow

Configuration (length, force, angle_deg) -> numerical model -> axis_torque_Nm, force_perpendicular_N, force_magnitude_N -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

length is the lever in meters; force is Newtons; angle_deg is the angle from the lever toward the applied force.

Parameter	Nominal value	Interpretation
length	0.2	meters
force	50	Newtons
angle_deg	60	degrees

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/02_vectors.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only angle_deg changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/02_vectors.py --case 0

.venv-book/bin/python expanded_lab/modules/02_vectors.py --case 3
```

```
.venv-book/bin/python expanded_lab/run.py vectors --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/02_vectors.py

```
"""Vectors, projections and joint-axis torque. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    length, force = p["length"], p["force"]
    angle = np.deg2rad(p["angle_deg"])
    r = np.array([length, 0.0, 0.0])
    F = force * np.array([np.cos(angle), np.sin(angle), 0.0])
    axis = np.array([0.0, 0.0, 1.0])
    moment = np.cross(r, F)
    torque = float(axis @ moment)
    perpendicular = force * np.sin(angle)
    assert np.isclose(torque, length * perpendicular)
    metrics = dict(
        axis_torque_Nm=torque,
        force_perpendicular_N=perpendicular,
        force_magnitude_N=float(np.linalg.norm(F)),
        lever_m=length,
    )
    derivation = [
        f"Force components in the chosen frame: Fx={F[0]:.6f} N, Fy={F[1]:.6f} N.",
        f"The lever is ({length:.6f}, 0, 0) m, so only Fy contributes to the z moment.",
        f"r cross F gives Mz={moment[2]:.6f} Nm.",
        f"Projection onto the unit z axis gives torque {torque:.6f} Nm.",
    ]
    notes = [
        f"The effective perpendicular force is {perpendicular:.6f} N, rather than the full
    {force:g} N.",
        "Changing origin changes moment; changing coordinates consistently does not change
    its physical meaning.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"length": 0.2, "force": 50.0, "angle_deg": 60.0}
    parameters["angle_deg"] = [0.0, 15.0, 30.0, 45.0, 60.0, 90.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Cross-product order determines sign. The scalar check against length times perpendicular force gives a separate geometric verification. Negative angles are a useful extension because the sign should reverse.

I. Expected behavior and six worked cases

The controlled variable is angle_deg. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

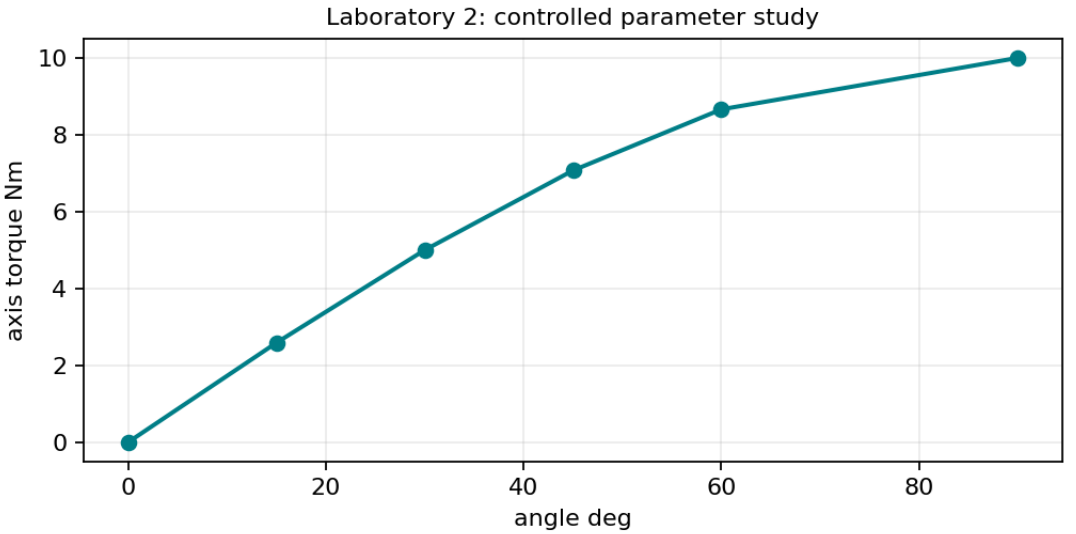


Figure L2. axis torque Nm as angle_deg changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	angle_deg	axis torque Nm
0	0	0
1	15	2.58819045
2	30	5
3	45	7.07106781
4	60	8.66025404

Case	angle_deg	axis torque Nm
5	90	10

Worked case 0: angle_deg = 0

This case keeps the nominal configuration except for angle_deg=0. The full input is {"angle_deg": 0.0, "force": 50.0, "length": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
axis torque Nm	0
force perpendicular N	0
force magnitude N	50
lever m	0.2

1. Force components in the chosen frame: Fx=50.000000 N, Fy=0.000000 N.
2. The lever is (0.200000, 0, 0) m, so only Fy contributes to the z moment.
3. r cross F gives Mz=0.000000 Nm.
4. Projection onto the unit z axis gives torque 0.000000 Nm.

Worked case 1: angle_deg = 15

This case keeps the nominal configuration except for angle_deg=15. The full input is {"angle_deg": 15.0, "force": 50.0, "length": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
axis torque Nm	2.58819045
force perpendicular N	12.94095226
force magnitude N	50

Computed quantity	Numerical result
lever m	0.2

1. Force components in the chosen frame: Fx=48.296291 N, Fy=12.940952 N.
2. The lever is (0.200000, 0, 0) m, so only Fy contributes to the z moment.
3. r cross F gives Mz=2.588190 Nm.
4. Projection onto the unit z axis gives torque 2.588190 Nm.

The effective perpendicular force is 12.940952 N, rather than the full 50 N.

Worked case 2: angle_deg = 30

This case keeps the nominal configuration except for angle_deg=30. The full input is {"angle_deg": 30.0, "force": 50.0, "length": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
axis torque Nm	5
force perpendicular N	25
force magnitude N	50
lever m	0.2

1. Force components in the chosen frame: Fx=43.301270 N, Fy=25.000000 N.
2. The lever is (0.200000, 0, 0) m, so only Fy contributes to the z moment.
3. r cross F gives Mz=5.000000 Nm.
4. Projection onto the unit z axis gives torque 5.000000 Nm.

The effective perpendicular force is 25.000000 N, rather than the full 50 N.

Worked case 3: angle_deg = 45

This case keeps the nominal configuration except for angle_deg=45. The full input is {"angle_deg": 45.0, "force": 50.0, "length": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
axis torque Nm	7.07106781
force perpendicular N	35.35533906
force magnitude N	50
lever m	0.2

1. Force components in the chosen frame: Fx=35.355339 N, Fy=35.355339 N.
2. The lever is (0.200000, 0, 0) m, so only Fy contributes to the z moment.
3. r cross F gives Mz=7.071068 Nm.
4. Projection onto the unit z axis gives torque 7.071068 Nm.

The effective perpendicular force is 35.355339 N, rather than the full 50 N.

Worked case 4: angle_deg = 60

This case keeps the nominal configuration except for angle_deg=60. The full input is {"angle_deg": 60.0, "force": 50.0, "length": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
axis torque Nm	8.66025404
force perpendicular N	43.30127019
force magnitude N	50
lever m	0.2

1. Force components in the chosen frame: Fx=25.000000 N, Fy=43.301270 N.
2. The lever is (0.200000, 0, 0) m, so only Fy contributes to the z moment.
3. r cross F gives Mz=8.660254 Nm.
4. Projection onto the unit z axis gives torque 8.660254 Nm.

The effective perpendicular force is 43.301270 N, rather than the full 50 N.

Worked case 5: angle_deg = 90

This case keeps the nominal configuration except for angle_deg=90. The full input is {"angle_deg": 90.0, "force": 50.0, "length": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
axis torque Nm	10
force perpendicular N	50
force magnitude N	50
lever m	0.2

1. Force components in the chosen frame: Fx=0.000000 N, Fy=50.000000 N.
2. The lever is (0.200000, 0, 0) m, so only Fy contributes to the z moment.
3. r cross F gives Mz=10.000000 Nm.
4. Projection onto the unit z axis gives torque 10.000000 Nm.

The effective perpendicular force is 50.000000 N, rather than the full 50 N.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The effective perpendicular force is 0.000000 N, rather than the full 50 N.

Changing origin changes moment; changing coordinates consistently does not change its physical meaning.

K. Failure modes and troubleshooting

Using an unnormalized axis; swapping r cross F with F cross r; adding vectors from world and body coordinates.

L. Practical exercises and extension tasks

1. Rotate r and F by the same rotation and verify the transformed moment.
2. Reverse the force and verify torque sign reversal.
3. Find the force angle that produces half the maximum joint moment.

M. Connection to the Asimov research platform

A limb load must be transformed into each joint frame along the chain; this is the foundation of Jacobian-transpose force mapping.

N. Completion record

Record your chosen case, predicted trend, measured axis_torque_Nm, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 3, Rotation matrices and the Rodrigues formula. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 3 - Rotation matrices and the Rodrigues formula

A. The engineering question

A valid rotation preserves lengths and angles.

B. Concepts and mathematical model

Its columns form an orthonormal right-handed basis. The Rodrigues formula constructs a rotation from a unit axis and an angle using the skew-symmetric cross-product matrix. This is more informative than memorizing three unrelated Euler-angle matrices because it makes the axis explicit. Rotation composition is generally noncommutative: rotating about x and then z differs from rotating about z and then x. Numerical checks should test orthogonality and determinant, not only whether a transformed vector looks plausible.

R = I + sin(theta)*K + (1-cos(theta))*K*K, where K*v = axis cross v.

C. Real-world meaning and limits

IMU-to-pelvis and camera-to-head rotations must be calibrated and tested with physical reference orientations.

D. Architecture and data flow

Configuration (angle_deg) -> numerical model -> rotated_x_m, rotated_y_m, rotated_z_m -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

angle_deg is a right-handed active rotation about the normalized axis (1,2,3). Input and output vectors use one fixed basis.

Parameter	Nominal value	Interpretation
angle_deg	45	degrees

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/03_rotations.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only angle_deg changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/03_rotations.py --case 0

.venv-book/bin/python expanded_lab/modules/03_rotations.py --case 3

.venv-book/bin/python expanded_lab/run.py rotations --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/03_rotations.py

```
"""Rotation matrices and the Rodrigues formula. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    theta = np.deg2rad(p["angle_deg"])
    axis = np.array([1.0, 2.0, 3.0])
    axis /= np.linalg.norm(axis)
    x, y, z = axis
    K = np.array([[0.0, -z, y], [z, 0.0, -x], [-y, x, 0.0]])
    R = np.eye(3) + np.sin(theta) * K + (1 - np.cos(theta)) * (K @ K)
    v = np.array([0.3, -0.2, 0.5])
    rotated = R @ v
    ortho = np.linalg.norm(R.T @ R - np.eye(3))
    det = np.linalg.det(R)
    roundtrip = np.linalg.norm(R.T @ rotated - v)
    assert ortho < 1e-12 and abs(det - 1) < 1e-12
    metrics = dict(
        rotated_x_m=rotated[0],
        rotated_y_m=rotated[1],
        rotated_z_m=rotated[2],
        orthogonality_error=ortho,
        roundtrip_error_m=roundtrip,
    )
    derivation = [
        f"Normalize axis (1,2,3) to {axis.tolist()}.",
        f'Convert {p["angle_deg"]:g} degrees to {theta:.6f} radians.',
        f"Assemble the skew matrix and evaluate Rodrigues; determinant is {det:.12f}.",
        f"R times (0.3,-0.2,0.5) gives {rotated.tolist()} meters.",
        f"Transpose the rotation to recover the original vector; residual is {roundtrip:.3e}
     m.",
    ]
    notes = [
        "A determinant near one does not alone guarantee orthogonality; both tests are
    retained.",
        "The chosen axis and vector are teaching geometry, unrelated to an unverified motor
    mounting.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"angle_deg": 45.0}
    parameters["angle_deg"] = [0.0, 15.0, 45.0, 90.0, 135.0, 180.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
```

```
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The skew matrix encodes the cross product. The quadratic term leaves the component along the axis unchanged while rotating the perpendicular component. Transpose is the inverse only after the rotation validity checks pass.

I. Expected behavior and six worked cases

The controlled variable is angle_deg. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

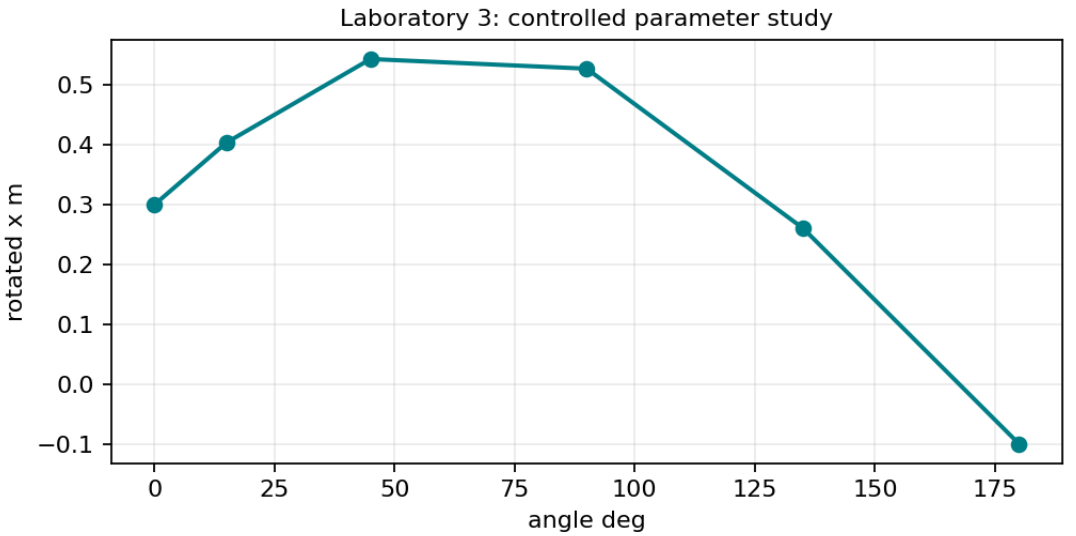


Figure L3. rotated x m as angle_deg changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	angle_deg	rotated x m
0	0	0.3
1	15	0.40386084

Case	angle_deg	rotated x m
2	45	0.54379293
3	90	0.52761799
4	135	0.26095022
5	180	-0.1

Worked case 0: angle_deg = 0

This case keeps the nominal configuration except for angle_deg=0. The full input is {"angle_deg": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
rotated x m	0.3
rotated y m	-0.2
rotated z m	0.5
orthogonality error	0
roundtrip error m	0

1. Normalize axis (1,2,3) to [0.2672612419124244, 0.5345224838248488, 0.8017837257372732].
2. Convert 0 degrees to 0.000000 radians.
3. Assemble the skew matrix and evaluate Rodrigues; determinant is 1.000000000000.
4. R times (0.3,-0.2,0.5) gives [0.3, -0.2, 0.5] meters.
5. Transpose the rotation to recover the original vector; residual is 0.000e+00 m.

Worked case 1: angle_deg = 15

This case keeps the nominal configuration except for angle_deg=15. The full input is {"angle_deg": 15.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
rotated x m	0.40386084
rotated y m	-0.15870141
rotated z m	0.43784733
orthogonality error	1.2244318e-16
roundtrip error m	5.5511151e-17

1. Normalize axis (1,2,3) to [0.2672612419124244, 0.5345224838248488, 0.8017837257372732].
2. Convert 15 degrees to 0.261799 radians.
3. Assemble the skew matrix and evaluate Rodrigues; determinant is 1.000000000000.
4. R times (0.3,-0.2,0.5) gives [0.4038608443373136, -0.15870141074575236, 0.4378473257180637] meters.
5. Transpose the rotation to recover the original vector; residual is 5.551e-17 m.

Worked case 2: angle_deg = 45

This case keeps the nominal configuration except for angle_deg=45. The full input is {"angle_deg": 45.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
rotated x m	0.54379293
rotated y m	-0.00724982
rotated z m	0.29023557

Computed quantity	Numerical result
orthogonality error	2.4194837e-16
roundtrip error m	6.2063354e-17

1. Normalize axis (1,2,3) to [0.2672612419124244, 0.5345224838248488, 0.8017837257372732].
2. Convert 45 degrees to 0.785398 radians.
3. Assemble the skew matrix and evaluate Rodrigues; determinant is 1.000000000000.
4. R times (0.3,-0.2,0.5) gives [0.5437929346446913, -0.007249817872773581, 0.2902355670336 1864] meters.
5. Transpose the rotation to recover the original vector; residual is 6.206e-17 m.

Worked case 3: angle_deg = 90

This case keeps the nominal configuration except for angle_deg=90. The full input is {"angle_deg": 90.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
rotated x m	0.52761799
rotated y m	0.3069045
rotated z m	0.08619101
orthogonality error	3.3322248e-16
roundtrip error m	7.8504623e-17

1. Normalize axis (1,2,3) to [0.2672612419124244, 0.5345224838248488, 0.8017837257372732].
2. Convert 90 degrees to 1.570796 radians.
3. Assemble the skew matrix and evaluate Rodrigues; determinant is 1.000000000000.

4. R times (0.3,-0.2,0.5) gives [0.5276179870598791, 0.3069044967649697, 0.08619100647006048] meters.
5. Transpose the rotation to recover the original vector; residual is 7.850e-17 m.

Worked case 4: angle_deg = 135

This case keeps the nominal configuration except for angle_deg=135. The full input is {"angle_deg": 135.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
rotated x m	0.26095022
rotated y m	0.55843561
rotated z m	0.00739285
orthogonality error	3.3907847e-16
roundtrip error m	1.3877788e-16

1. Normalize axis (1,2,3) to [0.2672612419124244, 0.5345224838248488, 0.8017837257372732].
2. Convert 135 degrees to 2.356194 radians.
3. Assemble the skew matrix and evaluate Rodrigues; determinant is 1.000000000000.
4. R times (0.3,-0.2,0.5) gives [0.26095022217007235, 0.5584356070764644, 0.00739285455899949] meters.
5. Transpose the rotation to recover the original vector; residual is 1.388e-16 m.

Worked case 5: angle_deg = 180

This case keeps the nominal configuration except for angle_deg=180. The full input is {"angle_deg": 180.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
rotated x m	-0.1
rotated y m	0.6
rotated z m	0.1
orthogonality error	3.7233036e-16
roundtrip error m	1.3597400e-16

1. Normalize axis (1,2,3) to [0.2672612419124244, 0.5345224838248488, 0.8017837257372732].
2. Convert 180 degrees to 3.141593 radians.
3. Assemble the skew matrix and evaluate Rodrigues; determinant is 1.000000000000.
4. R times (0.3,-0.2,0.5) gives [-0.09999999999999995, 0.6000000000000001, 0.0999999999999999] meters.
5. Transpose the rotation to recover the original vector; residual is 1.360e-16 m.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

A determinant near one does not alone guarantee orthogonality; both tests are retained.

The chosen axis and vector are teaching geometry, unrelated to an unverified motor mounting.

K. Failure modes and troubleshooting

Treating a reflection with determinant -1 as a rotation; transposing only one factor in a composed transformation.

L. Practical exercises and extension tasks

1. Verify the axis vector is unchanged by its own rotation.

2. Compare x-then-z and z-then-x rotations.
3. Add a small matrix perturbation and observe which validity test detects it.

M. Connection to the Asimov research platform

IMU-to-pelvis and camera-to-head rotations must be calibrated and tested with physical reference orientations.

N. Completion record

Record your chosen case, predicted trend, measured rotated_x_m, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 4, Unit quaternions, sign equivalence and incremental attitude. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 4 - Unit quaternions, sign equivalence and incremental attitude

A. The engineering question

Quaternions avoid the coordinate singularities of a three-angle parameterization, but they introduce a unit constraint and a component-order convention.

B. Concepts and mathematical model

A quaternion and its negative represent the same physical rotation. Comparing their components with ordinary Euclidean distance can therefore report a large difference for identical attitudes. For constant angular velocity, an exact axis-angle increment can be composed each timestep. This is a simple reference against which to test a differential integrator and normalization policy.

q_increment = [cos(|omega|*dt/2), axis*sin(|omega|*dt/2)]; q_next = q*q_increment; angular error = 2*acos(|q1 dot q2|).

C. Real-world meaning and limits

Real gyro integration needs calibrated axes, bias estimation, timestamp handling and external correction for drift.

D. Architecture and data flow

Configuration (dt, duration, rate) -> numerical model -> final_w, final_z, angle_error_rad -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

dt and duration are seconds; rate is constant body-z angular speed in rad/s; quaternion order is w,x,y,z.

Parameter	Nominal value	Interpretation
dt	0.01	seconds
duration	2	seconds
rate	0.7	rad/s

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/04_quaternions.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only dt changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project
```

```
.venv-book/bin/python expanded_lab/modules/04_quaternions.py --case 0

.venv-book/bin/python expanded_lab/modules/04_quaternions.py --case 3

.venv-book/bin/python expanded_lab/run.py quaternions --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/04_quaternions.py

```
"""Unit quaternions, sign equivalence and incremental attitude. Numerical teaching
    experiment only."""

import math
import json
import argparse
import numpy as np


def multiply(a, b):
    aw, ax, ay, az = a
    bw, bx, by, bz = b
    return np.array(
        [
            aw * bw - ax * bx - ay * by - az * bz,
            aw * bx + ax * bw + ay * bz - az * by,
            aw * by - ax * bz + ay * bw + az * bx,
            aw * bz + ax * by - ay * bx + az * bw,
        ]
    )


def experiment(p):
    dt, duration, rate = p["dt"], p["duration"], p["rate"]
    n = round(duration / dt)
    actual = n * dt
    inc = np.array([np.cos(rate * dt / 2), 0.0, 0.0, np.sin(rate * dt / 2)])
    q = np.array([1.0, 0.0, 0.0, 0.0])
    for _ in range(n):
        q = multiply(q, inc)
        q /= np.linalg.norm(q)
    exact = np.array(
        [np.cos(rate * actual / 2), 0.0, 0.0, np.sin(rate * actual / 2)]
    )
    error = 2 * np.arccos(np.clip(abs(q @ exact), 0, 1))
    same = 2 * np.arccos(np.clip(abs(q @ (-q)), 0, 1))
    assert np.isclose(np.linalg.norm(q), 1)
    metrics = dict(
        final_w=q[0],
        final_z=q[3],
        angle_error_rad=error,
        sign_equivalence_error_rad=same,
        actual_duration_s=actual,
    )
    derivation = [
        f"One increment rotates {rate*dt:.6f} rad about local z.",
        f"Apply {n} increments, giving actual duration {actual:.6f} s.",
        f"The final scalar-first quaternion is {q.tolist()}.",
        f"Use the absolute dot product for sign-invariant error: {error:.9f} rad.",
        f"Compare q with -q using the same metric: {same:.9f} rad.",
    ]
    notes = [
        "Constant-axis exact increments are a reference case, not a full noisy IMU
```

```
    estimator.",
        "Changing the multiplication side changes whether increments are interpreted in body
     or world coordinates.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"dt": 0.01, "duration": 2.0, "rate": 0.7}
    parameters["dt"] = [0.001, 0.002, 0.005, 0.01, 0.02, 0.05][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The Hamilton product is written explicitly so component order and multiplication side are inspectable. Normalizing after composition controls floating-point drift, while the exact solution supplies an independent check.

I. Expected behavior and six worked cases

The controlled variable is dt. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

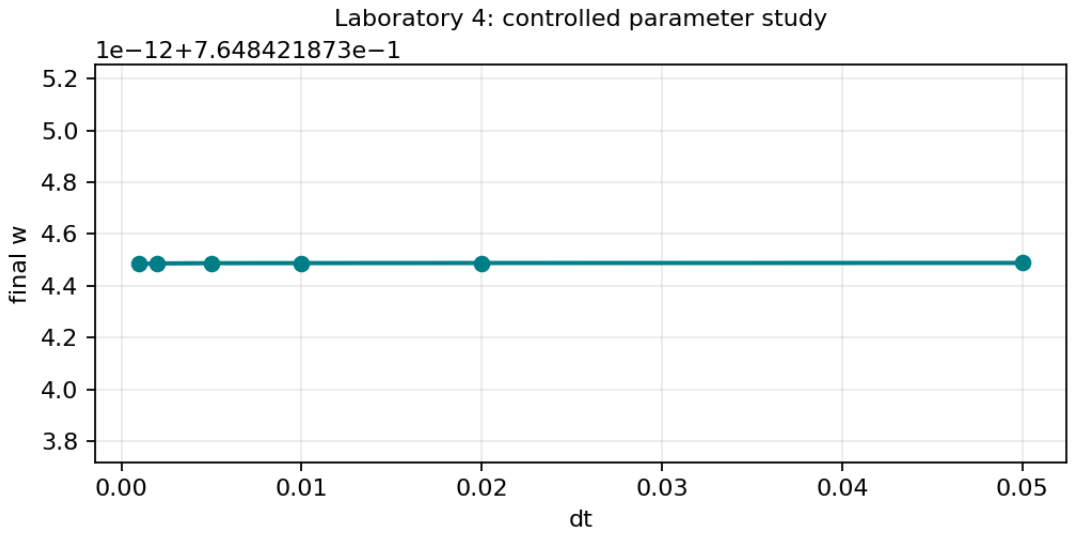


Figure L4. final w as dt changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	dt	final w
0	0.001	0.76484219
1	0.002	0.76484219
2	0.005	0.76484219
3	0.01	0.76484219
4	0.02	0.76484219
5	0.05	0.76484219

Worked case 0: dt = 0.001

This case keeps the nominal configuration except for dt=0.001. The full input is {"dt": 0.001, "duration": 2.0, "rate": 0.7}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final w	0.76484219

Computed quantity	Numerical result
final z	0.64421769
angle error rad	0
sign equivalence error rad	0
actual duration s	2

1. One increment rotates 0.000700 rad about local z.
2. Apply 2000 increments, giving actual duration 2.000000 s.
3. The final scalar-first quaternion is [0.7648421872844843, 0.0, 0.0, 0.644217687237696].
4. Use the absolute dot product for sign-invariant error: 0.000000000 rad.
5. Compare q with -q using the same metric: 0.000000000 rad.

Worked case 1: dt = 0.002

This case keeps the nominal configuration except for dt=0.002. The full input is {"dt": 0.002, "duration": 2.0, "rate": 0.7}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final w	0.76484219
final z	0.64421769
angle error rad	0
sign equivalence error rad	0
actual duration s	2

1. One increment rotates 0.001400 rad about local z.
2. Apply 1000 increments, giving actual duration 2.000000 s.
3. The final scalar-first quaternion is [0.7648421872844865, 0.0, 0.0, 0.6442176872376935].

4. Use the absolute dot product for sign-invariant error: 0.000000000 rad.
5. Compare q with -q using the same metric: 0.000000000 rad.

Worked case 2: dt = 0.005

This case keeps the nominal configuration except for dt=0.005. The full input is {"dt": 0.005, "duration": 2.0, "rate": 0.7}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final w	0.76484219
final z	0.64421769
angle error rad	0
sign equivalence error rad	0
actual duration s	2

1. One increment rotates 0.003500 rad about local z.
2. Apply 400 increments, giving actual duration 2.000000 s.
3. The final scalar-first quaternion is [0.7648421872844876, 0.0, 0.0, 0.644217687237692].
4. Use the absolute dot product for sign-invariant error: 0.000000000 rad.
5. Compare q with -q using the same metric: 0.000000000 rad.

Worked case 3: dt = 0.01

This case keeps the nominal configuration except for dt=0.01. The full input is {"dt": 0.01, "duration": 2.0, "rate": 0.7}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final w	0.76484219
final z	0.64421769

Computed quantity	Numerical result
angle error rad	0
sign equivalence error rad	0
actual duration s	2

1. One increment rotates 0.007000 rad about local z.
2. Apply 200 increments, giving actual duration 2.000000 s.
3. The final scalar-first quaternion is [0.7648421872844878, 0.0, 0.0, 0.6442176872376917].
4. Use the absolute dot product for sign-invariant error: 0.000000000 rad.
5. Compare q with -q using the same metric: 0.000000000 rad.

Worked case 4: dt = 0.02

This case keeps the nominal configuration except for dt=0.02. The full input is {"dt": 0.02, "duration": 2.0, "rate": 0.7}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final w	0.76484219
final z	0.64421769
angle error rad	0
sign equivalence error rad	0
actual duration s	2

1. One increment rotates 0.014000 rad about local z.
2. Apply 100 increments, giving actual duration 2.000000 s.
3. The final scalar-first quaternion is [0.7648421872844882, 0.0, 0.0, 0.6442176872376915].
4. Use the absolute dot product for sign-invariant error: 0.000000000 rad.

5. Compare q with -q using the same metric: 0.000000000 rad.

Worked case 5: dt = 0.05

This case keeps the nominal configuration except for dt=0.05. The full input is {"dt": 0.05, "duration": 2.0, "rate": 0.7}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final w	0.76484219
final z	0.64421769
angle error rad	0
sign equivalence error rad	0
actual duration s	2

1. One increment rotates 0.035000 rad about local z.
2. Apply 40 increments, giving actual duration 2.000000 s.
3. The final scalar-first quaternion is [0.7648421872844885, 0.0, 0.0, 0.6442176872376909].
4. Use the absolute dot product for sign-invariant error: 0.000000000 rad.
5. Compare q with -q using the same metric: 0.000000000 rad.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Constant-axis exact increments are a reference case, not a full noisy IMU estimator.

Changing the multiplication side changes whether increments are interpreted in body or world coordinates.

K. Failure modes and troubleshooting

Comparing q and -q directly; passing scalar-first values into an API expecting scalar-last; integrating degrees per second as radians.

L. Practical exercises and extension tasks

1. Replace exact increments by first-order quaternion integration and compare timestep sensitivity.
2. Alternate x and y angular increments and examine composition order.
3. Test conversion to ROS scalar-last message ordering and back.

M. Connection to the Asimov research platform

Real gyro integration needs calibrated axes, bias estimation, timestamp handling and external correction for drift.

N. Completion record

Record your chosen case, predicted trend, measured final_w, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 5, Homogeneous transforms and frame-chain round trips. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 5 - Homogeneous transforms and frame-chain round trips

A. The engineering question

A rigid transform combines a rotation and translation without losing the frame relationship between them.

B. Concepts and mathematical model

The notation T_AB means coordinates expressed in B are converted to A. Adjacent frame labels cancel during composition: T_AB*T_BC gives T_AC. This mnemonic is safer than deciding matrix multiplication order from visual intuition. Points carry translation; direction vectors do not. A transform inverse uses the transposed rotation and a rotated negative translation, not simply negating the position column.

T_AB = [[R_AB,p_AB],[0,1]]; T_AC = T_AB*T_BC; T_BA = [[R_AB.T,-R_AB.T*p_AB],[0,1]].

C. Real-world meaning and limits

Camera-to-base transforms are reusable only after matching link definitions, mounting offsets and timestamps to the assembled robot.

D. Architecture and data flow

Configuration (angle_deg, offset) -> numerical model -> point_x_m, point_y_m, point_z_m -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

angle_deg is the A-from-B active z rotation; offset is B-to-C origin displacement in meters.

Parameter	Nominal value	Interpretation
angle_deg	45	degrees
offset	0.15	meters

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/05_transforms.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only angle_deg changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/05_transforms.py --case 0

.venv-book/bin/python expanded_lab/modules/05_transforms.py --case 3

.venv-book/bin/python expanded_lab/run.py transforms --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/05_transforms.py

```
"""Homogeneous transforms and frame-chain round trips. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def transform(angle, translation):
    c, s = np.cos(angle), np.sin(angle)
    T = np.eye(4)
    T[:3, :3] = [[c, -s, 0], [s, c, 0], [0, 0, 1]]
    T[:3, 3] = translation
    return T


def experiment(p):
    a = np.deg2rad(p["angle_deg"])
    T_ab = transform(a, [0.1, 0.2, 0.3])
    T_bc = transform(-0.3, [p["offset"], 0.0, 0.05])
    T_ac = T_ab @ T_bc
    point = np.array([0.2, 0.1, 0.0, 1.0])
    world = T_ac @ point
    R, t = T_ac[:3, :3], T_ac[:3, 3]
    inverse = np.eye(4)
    inverse[:3, :3] = R.T
    inverse[:3, 3] = -R.T @ t
    recovered = inverse @ world
    error = np.linalg.norm(recovered - point)
    direct = T_ab @ (T_bc @ point)
    assert np.allclose(world, direct) and error < 1e-12
    metrics = dict(
        point_x_m=world[0],
        point_y_m=world[1],
        point_z_m=world[2],
        roundtrip_error_m=error,
        translation_norm_m=np.linalg.norm(t),
    )
    derivation = [
        f'T_BC translation is ({p["offset"]:.6f},0,0.05) m.',
        f"T_AB rotates by {a:.6f} rad before adding its translation.",
        f"Compose T_AC; the resulting translation is {t.tolist()} m.",
        f"Point (0.2,0.1,0) in C becomes {world[:3].tolist()} in A.",
        f"Apply the analytic inverse; recovery error is {error:.3e} m.",
    ]
    notes = [
        "The point result includes both translations in the correct rotated directions.",
        "A direction encoded with homogeneous last element zero would ignore translation.",
    ]
    return metrics, derivation, notes

if __name__ == "__main__":
```

```
parser = argparse.ArgumentParser()
parser.add_argument("--case", type=int, default=0, choices=range(6))
args = parser.parse_args()
parameters = {"angle_deg": 45.0, "offset": 0.15}
parameters["angle_deg"] = [0.0, 15.0, 30.0, 45.0, 60.0, 90.0][args.case]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Build each transform independently, compose in the frame-label order, and check both nested multiplication and inverse recovery. These two checks expose different classes of implementation error.

I. Expected behavior and six worked cases

The controlled variable is angle_deg. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

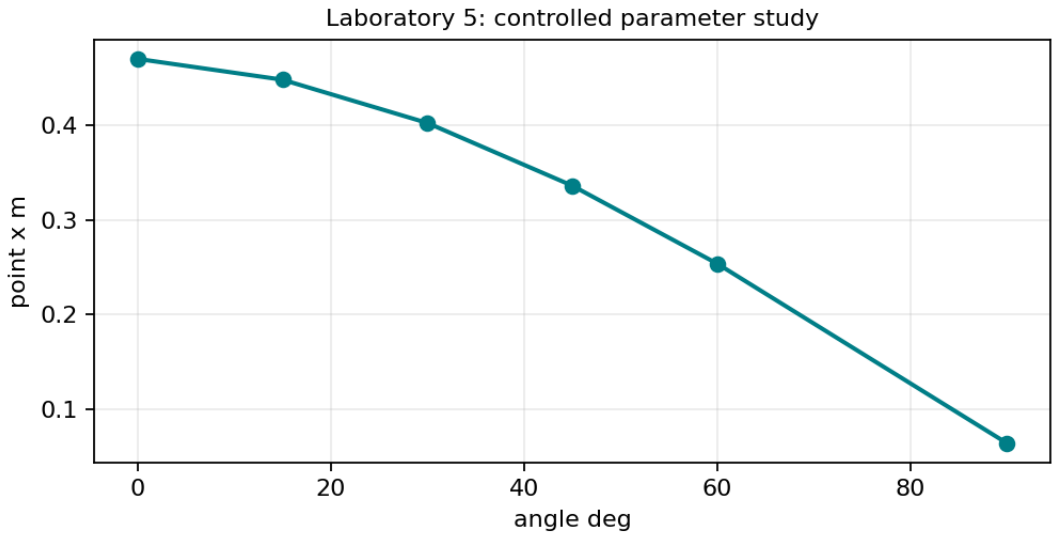


Figure L5. point x m as angle_deg changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	angle_deg	point x m
0	0	0.47061932
1	15	0.4485621
2	30	0.40275094
3	45	0.33630781
4	60	0.25376069
5	90	0.06357039

Worked case 0: angle_deg = 0

This case keeps the nominal configuration except for angle_deg=0. The full input is {"angle_deg": 0.0, "offset": 0.15}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
point x m	0.47061932
point y m	0.23642961
point z m	0.35
roundtrip error m	6.2063354e-17
translation norm m	0.47434165

1. T_BC translation is (0.150000,0,0.05) m.
2. T_AB rotates by 0.000000 rad before adding its translation.
3. Compose T_AC; the resulting translation is [0.25, 0.2, 0.35] m.
4. Point (0.2,0.1,0) in C becomes [0.47061931849125516, 0.23642960758029272, 0.35] in A.
5. Apply the analytic inverse; recovery error is 6.206e-17 m.

Worked case 1: angle_deg = 15

This case keeps the nominal configuration except for angle_deg=15. The full input is {"angle_deg": 15.0, "offset": 0.15}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
point x m	0.4485621
point y m	0.33111164
point z m	0.35
roundtrip error m	7.8504623e-17
translation norm m	0.48939444

1. T_BC translation is (0.150000,0,0.05) m.
2. T_AB rotates by 0.261799 rad before adding its translation.
3. Compose T_AC; the resulting translation is [0.24488887394336026, 0.23882285676537812, 0.35] m.
4. Point (0.2,0.1,0) in C becomes [0.44856209520496615, 0.3311116369118344, 0.35] in A.
5. Apply the analytic inverse; recovery error is 7.850e-17 m.

Worked case 2: angle_deg = 30

This case keeps the nominal configuration except for angle_deg=30. The full input is {"angle_deg": 30.0, "offset": 0.15}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
point x m	0.40275094
point y m	0.41685862
point z m	0.35

Computed quantity	Numerical result
roundtrip error m	6.2063354e-17
translation norm m	0.5009798

1. T_BC translation is (0.150000,0,0.05) m.
2. T_AB rotates by 0.523599 rad before adding its translation.
3. Compose T_AC; the resulting translation is [0.22990381056766582, 0.275, 0.35] m.
4. Point (0.2,0.1,0) in C becomes [0.4027509411565564, 0.4168586248600592, 0.35] in A.
5. Apply the analytic inverse; recovery error is 6.206e-17 m.

Worked case 3: angle_deg = 45

This case keeps the nominal configuration except for angle_deg=45. The full input is {"angle_deg": 45.0, "offset": 0.15}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
point x m	0.33630781
point y m	0.48782706
point z m	0.35
roundtrip error m	2.7755576e-17
translation norm m	0.50856623

1. T_BC translation is (0.150000,0,0.05) m.
2. T_AB rotates by 0.785398 rad before adding its translation.
3. Compose T_AC; the resulting translation is [0.20606601717798212, 0.3060660171779821, 0.35] m.
4. Point (0.2,0.1,0) in C becomes [0.3363078107879135, 0.48782705589989317, 0.35] in A.

5. Apply the analytic inverse; recovery error is 2.776e-17 m.

Worked case 4: angle_deg = 60

This case keeps the nominal configuration except for angle_deg=60. The full input is {"angle_deg": 60.0, "offset": 0.15}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
point x m	0.25376069
point y m	0.53918055
point z m	0.35
roundtrip error m	3.9252311e-17
translation norm m	0.51182177

1. T_BC translation is (0.150000,0,0.05) m.
2. T_AB rotates by 1.047198 rad before adding its translation.
3. Compose T_AC; the resulting translation is [0.17500000000000002, 0.3299038105676658, 0.35] m.
4. Point (0.2,0.1,0) in C becomes [0.25376069363119597, 0.5391805487368491, 0.35] in A.
5. Apply the analytic inverse; recovery error is 3.925e-17 m.

Worked case 5: angle_deg = 90

This case keeps the nominal configuration except for angle_deg=90. The full input is {"angle_deg": 90.0, "offset": 0.15}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
point x m	0.06357039
point y m	0.57061932

Computed quantity	Numerical result
point z m	0.35
roundtrip error m	2.7755576e-17
translation norm m	0.50497525

1. T_BC translation is (0.150000,0,0.05) m.
2. T_AB rotates by 1.570796 rad before adding its translation.
3. Compose T_AC; the resulting translation is [0.10000000000000002, 0.35, 0.35] m.
4. Point (0.2,0.1,0) in C becomes [0.06357039241970733, 0.5706193184912551, 0.35] in A.
5. Apply the analytic inverse; recovery error is 2.776e-17 m.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The point result includes both translations in the correct rotated directions.

A direction encoded with homogeneous last element zero would ignore translation.

K. Failure modes and troubleshooting

Using -translation directly for the inverse; multiplying a three-vector by a 4x4 matrix; confusing a point with a direction.

L. Practical exercises and extension tasks

1. Replace the point last coordinate with zero and explain the changed result.
2. Insert a head-to-camera transform and verify a four-frame chain.
3. Perturb camera translation by 5 mm and calculate world-position error.

M. Connection to the Asimov research platform

Camera-to-base transforms are reusable only after matching link definitions, mounting offsets and timestamps to the assembled robot.

N. Completion record

Record your chosen case, predicted trend, measured point_x_m, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 6, Finite-difference Jacobians and derivative verification. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 6 - Finite-difference Jacobians and derivative verification

A. The engineering question

The Jacobian answers a local question: how much does an endpoint move for a small joint displacement? Analytic derivatives are efficient, but a sign or index error can hide inside plausible motion.

B. Concepts and mathematical model

Central finite differences provide a separate numerical check. The perturbation must be small enough to approximate a derivative but not so small that subtraction roundoff dominates. This laboratory compares the two methods at a nonsingular two-link pose and makes epsilon an experimental variable.

J_ij = partial f_i/partial q_j; central difference = [f(q+epsilon*e_j)-f(q-epsilon*e_j)]/(2*epsilon).

C. Real-world meaning and limits

The same column test should validate the selected MuJoCo or hardware-model Jacobian before Cartesian controllers depend on it.

D. Architecture and data flow

Configuration (epsilon) -> numerical model -> derivative_error, velocity_x_m_s, velocity_y_m_s -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

epsilon is a central-difference joint perturbation in radians; lengths are 0.3 and 0.2 meters.

Parameter	Nominal value	Interpretation
epsilon	0.00001	radian perturbation

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/06_jacobian.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only epsilon changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/06_jacobian.py --case 0

.venv-book/bin/python expanded_lab/modules/06_jacobian.py --case 3

.venv-book/bin/python expanded_lab/run.py jacobian --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/06_jacobian.py

```
"""Finite-difference Jacobians and derivative verification. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def fk(q):
    a, b = q
    return np.array(
        [
            0.3 * np.cos(a) + 0.2 * np.cos(a + b),
            0.3 * np.sin(a) + 0.2 * np.sin(a + b),
        ]
    )


def experiment(p):
    q = np.array([0.4, -0.7])
    eps = p["epsilon"]
    a, b = q
    J = np.array(
        [
            [-0.3 * np.sin(a) - 0.2 * np.sin(a + b), -0.2 * np.sin(a + b)],
            [0.3 * np.cos(a) + 0.2 * np.cos(a + b), 0.2 * np.cos(a + b)],
        ]
    )
    estimated = np.zeros((2, 2))
    for j in range(2):
        delta = np.zeros(2)
        delta[j] = eps
        estimated[:, j] = (fk(q + delta) - fk(q - delta)) / (2 * eps)
    error = np.linalg.norm(estimated - J)
    velocity = J @ np.array([0.1, -0.2])
    metrics = dict(
        derivative_error=error,
        velocity_x_m_s=velocity[0],
        velocity_y_m_s=velocity[1],
        jacobian_condition=np.linalg.cond(J),
    )
    derivation = [
        f"At joint pose {q.tolist()} rad, endpoint is {fk(q).tolist()} m.",
        f"Use symmetric perturbations of {eps:.3e} rad for each joint.",
        f"The analytic first column is {J[:,0].tolist()} m/rad.",
        f"The estimated first column is {estimated[:,0].tolist()} m/rad.",
        f"Full matrix residual norm is {error:.3e}; predicted endpoint velocity is
    {velocity.tolist()} m/s.",
    ]
    notes = [
        "Reducing epsilon eventually makes floating-point cancellation important.",
        "Compare columns in the same frame and at the same configuration; otherwise the
```

```
    residual is not a derivative test.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"epsilon": 1e-05}
    parameters["epsilon"] = [0.1, 0.01, 0.001, 1e-05, 1e-07, 1e-09][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The two columns correspond to the two independent joint perturbations. Multiplying by a joint-rate vector produces a local Cartesian velocity, whose units provide another check on the derivative.

I. Expected behavior and six worked cases

The controlled variable is epsilon. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

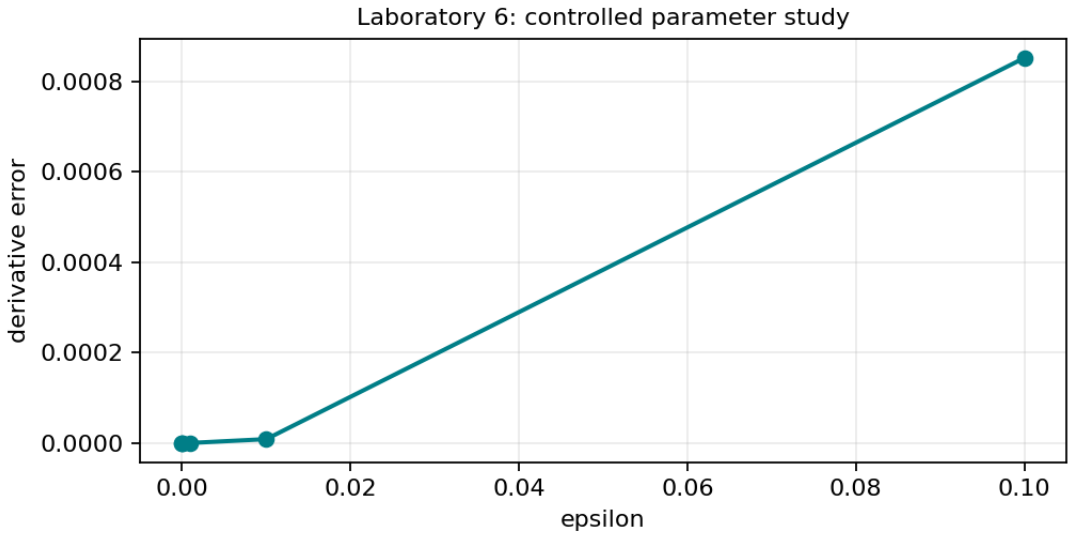


Figure L6. derivative error as epsilon changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	epsilon	derivative error
0	0.1	0.00085232
1	0.01	8.5273815e-06
2	0.001	8.5274238e-08
3	0.00001	1.0136602e-11
4	1.0000000e-07	4.8850740e-10
5	1.0000000e-09	2.8168134e-08

Worked case 0: epsilon = 0.1

This case keeps the nominal configuration except for epsilon=0.1. The full input is {"epsilon": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
derivative error	0.00085232

Computed quantity	Numerical result
velocity x m s	-0.01759295
velocity y m s	0.0085251
jacobian condition	6.62156114

1. At joint pose [0.4, -0.7] rad, endpoint is [0.4673855960259867, 0.05772146136032724] m.
2. Use symmetric perturbations of 1.000e-01 rad for each joint.
3. The analytic first column is [-0.05772146136032724, 0.4673855960259867] m/rad.
4. The estimated first column is [-0.05762530701449331, 0.46660700942788436] m/rad.
5. Full matrix residual norm is 8.523e-04; predicted endpoint velocity is [-0.017592954402486306, 0.008525100037574427] m/s.

Worked case 1: epsilon = 0.01

This case keeps the nominal configuration except for epsilon=0.01. The full input is {"epsilon": 0.01}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
derivative error	8.5273815e-06
velocity x m s	-0.01759295
velocity y m s	0.0085251
jacobian condition	6.62156114

1. At joint pose [0.4, -0.7] rad, endpoint is [0.4673855960259867, 0.05772146136032724] m.
2. Use symmetric perturbations of 1.000e-02 rad for each joint.
3. The analytic first column is [-0.05772146136032724, 0.4673855960259867] m/rad.

4. The estimated first column is [-0.05772049934078538, 0.46737780630500236] m/rad.
5. Full matrix residual norm is 8.527e-06; predicted endpoint velocity is [-0.017592954402486306, 0.008525100037574427] m/s.

Worked case 2: epsilon = 0.001

This case keeps the nominal configuration except for epsilon=0.001. The full input is {"epsilon": 0.001}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
derivative error	8.5274238e-08
velocity x m s	-0.01759295
velocity y m s	0.0085251
jacobian condition	6.62156114

1. At joint pose [0.4, -0.7] rad, endpoint is [0.4673855960259867, 0.05772146136032724] m.
2. Use symmetric perturbations of 1.000e-03 rad for each joint.
3. The analytic first column is [-0.05772146136032724, 0.4673855960259867] m/rad.
4. The estimated first column is [-0.05772145174004484, 0.4673855181283916] m/rad.
5. Full matrix residual norm is 8.527e-08; predicted endpoint velocity is [-0.017592954402486306, 0.008525100037574427] m/s.

Worked case 3: epsilon = 0.00001

This case keeps the nominal configuration except for epsilon=0.00001. The full input is {"epsilon": 1e-05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
derivative error	1.0136602e-11

Computed quantity	Numerical result
velocity x m s	-0.01759295
velocity y m s	0.0085251
jacobian condition	6.62156114

1. At joint pose [0.4, -0.7] rad, endpoint is [0.4673855960259867, 0.05772146136032724] m.
2. Use symmetric perturbations of 1.000e-05 rad for each joint.
3. The analytic first column is [-0.05772146136032724, 0.4673855960259867] m/rad.
4. The estimated first column is [-0.05772146136062694, 0.4673855960181383] m/rad.
5. Full matrix residual norm is 1.014e-11; predicted endpoint velocity is [-0.017592954402486306, 0.008525100037574427] m/s.

Worked case 4: epsilon = 1.0000000e-07

This case keeps the nominal configuration except for epsilon=1.0000000e-07. The full input is {"epsilon": 1e-07}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
derivative error	4.8850740e-10
velocity x m s	-0.01759295
velocity y m s	0.0085251
jacobian condition	6.62156114

1. At joint pose [0.4, -0.7] rad, endpoint is [0.4673855960259867, 0.05772146136032724] m.
2. Use symmetric perturbations of 1.000e-07 rad for each joint.
3. The analytic first column is [-0.05772146136032724, 0.4673855960259867] m/rad.

4. The estimated first column is [-0.05772146177696058, 0.4673855960590778] m/rad.
5. Full matrix residual norm is 4.885e-10; predicted endpoint velocity is [-0.017592954402486306, 0.008525100037574427] m/s.

Worked case 5: epsilon = 1.0000000e-09

This case keeps the nominal configuration except for epsilon=1.0000000e-09. The full input is {"epsilon": 1e-09}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
derivative error	2.8168134e-08
velocity x m s	-0.01759295
velocity y m s	0.0085251
jacobian condition	6.62156114

1. At joint pose [0.4, -0.7] rad, endpoint is [0.4673855960259867, 0.05772146136032724] m.
2. Use symmetric perturbations of 1.000e-09 rad for each joint.
3. The analytic first column is [-0.05772146136032724, 0.4673855960259867] m/rad.
4. The estimated first column is [-0.05772143873983281, 0.46738560244286015] m/rad.
5. Full matrix residual norm is 2.817e-08; predicted endpoint velocity is [-0.017592954402486306, 0.008525100037574427] m/s.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Reducing epsilon eventually makes floating-point cancellation important.

Compare columns in the same frame and at the same configuration; otherwise the residual is not a derivative test.

K. Failure modes and troubleshooting

Reusing a mutable q array without restoring it; comparing degrees-based perturbations to radian derivatives; testing only at zero pose.

L. Practical exercises and extension tasks

1. Repeat with forward differences and compare convergence order.
2. Check every column at ten seeded nonsingular poses.
3. Move near a stretched configuration and track conditioning separately from derivative accuracy.

M. Connection to the Asimov research platform

The same column test should validate the selected MuJoCo or hardware-model Jacobian before Cartesian controllers depend on it.

N. Completion record

Record your chosen case, predicted trend, measured derivative_error, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 7, Least-squares identification and residual diagnosis. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 7 - Least-squares identification and residual diagnosis

A. The engineering question

Many calibration tasks are linear in their unknown parameters even when the robot is complicated.

B. Concepts and mathematical model

A simple example estimates a sensor scale and offset from known references. Least squares chooses parameters minimizing squared residuals, but a small residual does not guarantee the experiment identified both parameters well. Excitation range determines conditioning. Noise, repeated references and validation data must be considered before accepting a fit.

y = a*x+b+noise; theta_hat = argmin ||X*theta-y||^2; X=[x,1].

C. Real-world meaning and limits

Encoder zero, current-to-torque scale and camera calibration require trusted independent references and validation outside the fit data.

D. Architecture and data flow

Configuration (samples, noise, span) -> numerical model -> scale_error, offset_error_rad, residual_rmse_rad -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

samples is the number of references; noise is measurement standard deviation in radians; span is the symmetric excitation half-range.

Parameter	Nominal value	Interpretation
samples	40	count
noise	0.005	measurement standard deviation

Parameter	Nominal value	Interpretation
span	0.5	radians

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/07_least_squares.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only span changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/07_least_squares.py --case 0

.venv-book/bin/python expanded_lab/modules/07_least_squares.py --case 3

.venv-book/bin/python expanded_lab/run.py least_squares --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/07_least_squares.py

```
"""Least-squares identification and residual diagnosis. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    n = int(p["samples"])
    sigma = p["noise"]
    span = p["span"]
    rng = np.random.default_rng(11)
    x = np.linspace(-span, span, n)
    X = np.column_stack([x, np.ones(n)])
    truth = np.array([1.04, 0.025])
    y = X @ truth + rng.normal(0, sigma, n)
    estimate, _, _, _ = np.linalg.lstsq(X, y, rcond=None)
    residual = y - X @ estimate
    variance = float(residual @ residual / (n - 2))
    covariance = variance * np.linalg.inv(X.T @ X)
    metrics = dict(
        scale_error=estimate[0] - truth[0],
        offset_error_rad=estimate[1] - truth[1],
        residual_rmse_rad=np.sqrt(np.mean(residual**2)),
        design_condition=np.linalg.cond(X),
        scale_std_estimate=np.sqrt(covariance[0, 0]),
    )
    derivation = [
        f"Collect {n} reference positions spanning [-{span:g},{span:g}] rad.",
        f"True scale and offset are {truth.tolist()}; inject noise sigma={sigma:g} rad.",
        f"Least-squares estimate is {estimate.tolist()}.",
        f"Residual degrees of freedom are {n-2}; variance estimate is {variance:.6e}.",
        f"The design matrix condition is {np.linalg.cond(X):.6f}; estimated scale
    uncertainty is {np.sqrt(covariance[0,0]):.6f}.",
    ]
    notes = [
        f"The identification range is {2*span:g} rad; narrowing it weakens scale estimation
    even if the offset remains observable.",
        "Covariance assumes the linear noise model and does not account for backlash,
    systematic reference error or temperature drift.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"samples": 40, "noise": 0.005, "span": 0.5}
    parameters["span"] = [0.01, 0.025, 0.05, 0.1, 0.5, 1.0][args.case]
    metrics, derivation, notes = experiment(parameters)
```

```
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Use lstsq rather than explicitly inverting the normal equations for the estimate. The covariance calculation is kept separate to inspect its assumptions. Compare parameter errors to the injected truth and residual magnitude.

I. Expected behavior and six worked cases

The controlled variable is span. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

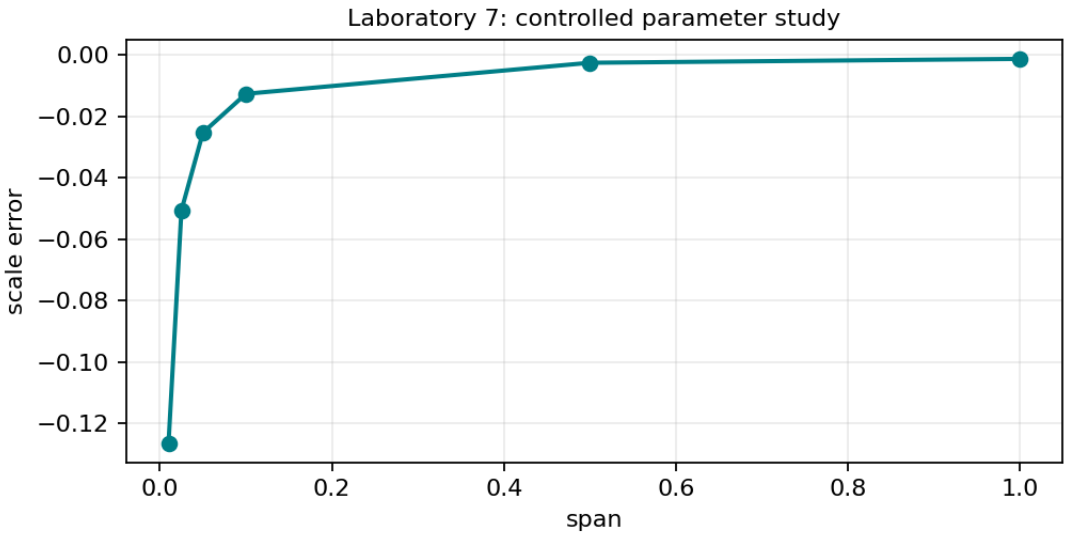


Figure L7. scale error as span changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	span	scale error
0	0.01	-0.12639436

Case	span	scale error
1	0.025	-0.05055774
2	0.05	-0.02527887
3	0.1	-0.01263944
4	0.5	-0.00252789
5	1	-0.00126394

Worked case 0: span = 0.01

This case keeps the nominal configuration except for span=0.01. The full input is {"noise": 0.005, "samples": 40, "span": 0.01}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
scale error	-0.12639436
offset error rad	-0.00040906
residual rmse rad	0.00416545
design condition	168.92775191
scale std estimate	0.11414873

1. Collect 40 reference positions spanning [-0.01,0.01] rad.
2. True scale and offset are [1.04, 0.025]; inject noise sigma=0.005 rad.
3. Least-squares estimate is [0.9136056388714553, 0.024590942846796585].
4. Residual degrees of freedom are 38; variance estimate is 1.826418e-05.
5. The design matrix condition is 168.927752; estimated scale uncertainty is 0.114149.

Worked case 1: span = 0.025

This case keeps the nominal configuration except for span=0.025. The full input is {"noise": 0.005, "samples": 40, "span": 0.025}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
scale error	-0.05055774
offset error rad	-0.00040906
residual rmse rad	0.00416545
design condition	67.57110076
scale std estimate	0.04565949

1. Collect 40 reference positions spanning [-0.025,0.025] rad.
2. True scale and offset are [1.04, 0.025]; inject noise sigma=0.005 rad.
3. Least-squares estimate is [0.9894422555485823, 0.024590942846796585].
4. Residual degrees of freedom are 38; variance estimate is 1.826418e-05.
5. The design matrix condition is 67.571101; estimated scale uncertainty is 0.045659.

The identification range is 0.05 rad; narrowing it weakens scale estimation even if the offset remains observable.

Worked case 2: span = 0.05

This case keeps the nominal configuration except for span=0.05. The full input is {"noise": 0.005, "samples": 40, "span": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
scale error	-0.02527887
offset error rad	-0.00040906

Computed quantity	Numerical result
residual rmse rad	0.00416545
design condition	33.78555038
scale std estimate	0.02282975

1. Collect 40 reference positions spanning [-0.05,0.05] rad.
2. True scale and offset are [1.04, 0.025]; inject noise sigma=0.005 rad.
3. Least-squares estimate is [1.0147211277742911, 0.024590942846796585].
4. Residual degrees of freedom are 38; variance estimate is 1.826418e-05.
5. The design matrix condition is 33.785550; estimated scale uncertainty is 0.022830.

The identification range is 0.1 rad; narrowing it weakens scale estimation even if the offset remains observable.

Worked case 3: span = 0.1

This case keeps the nominal configuration except for span=0.1. The full input is {"noise": 0.005, "samples": 40, "span": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
scale error	-0.01263944
offset error rad	-0.00040906
residual rmse rad	0.00416545
design condition	16.89277519
scale std estimate	0.01141487

1. Collect 40 reference positions spanning [-0.1,0.1] rad.
2. True scale and offset are [1.04, 0.025]; inject noise sigma=0.005 rad.

3. Least-squares estimate is [1.0273605638871455, 0.024590942846796585].
4. Residual degrees of freedom are 38; variance estimate is 1.826418e-05.
5. The design matrix condition is 16.892775; estimated scale uncertainty is 0.011415.

The identification range is 0.2 rad; narrowing it weakens scale estimation even if the offset remains observable.

Worked case 4: span = 0.5

This case keeps the nominal configuration except for span=0.5. The full input is {"noise": 0.005, "samples": 40, "span": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
scale error	-0.00252789
offset error rad	-0.00040906
residual rmse rad	0.00416545
design condition	3.37855504
scale std estimate	0.00228297

1. Collect 40 reference positions spanning [-0.5,0.5] rad.
2. True scale and offset are [1.04, 0.025]; inject noise sigma=0.005 rad.
3. Least-squares estimate is [1.0374721127774293, 0.024590942846796582].
4. Residual degrees of freedom are 38; variance estimate is 1.826418e-05.
5. The design matrix condition is 3.378555; estimated scale uncertainty is 0.002283.

The identification range is 1 rad; narrowing it weakens scale estimation even if the offset remains observable.

Worked case 5: span = 1

This case keeps the nominal configuration except for span=1. The full input is {"noise": 0.005, "samples": 40, "span": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
scale error	-0.00126394
offset error rad	-0.00040906
residual rmse rad	0.00416545
design condition	1.68927752
scale std estimate	0.00114149

1. Collect 40 reference positions spanning [-1,1] rad.
2. True scale and offset are [1.04, 0.025]; inject noise sigma=0.005 rad.
3. Least-squares estimate is [1.0387360563887147, 0.024590942846796582].
4. Residual degrees of freedom are 38; variance estimate is 1.826418e-05.
5. The design matrix condition is 1.689278; estimated scale uncertainty is 0.001141.

The identification range is 2 rad; narrowing it weakens scale estimation even if the offset remains observable.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The identification range is 0.02 rad; narrowing it weakens scale estimation even if the offset remains observable.

Covariance assumes the linear noise model and does not account for backlash, systematic reference error or temperature drift.

K. Failure modes and troubleshooting

Judging calibration only on training residuals; confusing standard deviation with a hard error bound; assuming repeated identical inputs identify scale.

L. Practical exercises and extension tasks

1. Fit a separate held-out reference set and report prediction error.
2. Add a quadratic distortion and inspect residual versus position.
3. Change sample count and compare uncertainty to excitation range.

M. Connection to the Asimov research platform

Encoder zero, current-to-torque scale and camera calibration require trusted independent references and validation outside the fit data.

N. Completion record

Record your chosen case, predicted trend, measured scale_error, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 8, Discrete integration and numerical stability. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 8 - Discrete integration and numerical stability

A. The engineering question

A stable continuous equation can become unstable after a poor discretization.

B. Concepts and mathematical model

The scalar decay equation is simple enough to solve exactly, making it a useful laboratory for numerical reasoning. Explicit Euler predicts the next state from the current derivative. Implicit Euler evaluates the derivative at the next state and solves the resulting equation. Smaller timesteps reduce error but increase computation; choosing them blindly can conceal a model or controller problem.

x_dot=-k*x; exact x(t)=x0*exp(-k*t); explicit x_next=(1-k*dt)*x; implicit x_next=x/(1+k*dt).

C. Real-world meaning and limits

A MuJoCo timestep, controller period and policy period are separate choices; document their relationship and validate contact/controller behavior together.

D. Architecture and data flow

Configuration (dt, decay_rate, duration) -> numerical model -> explicit_final, implicit_final, exact_final -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

dt is seconds; decay_rate is inverse seconds; duration is desired integration horizon. Actual duration is an integer number of steps.

Parameter	Nominal value	Interpretation
dt	0.02	seconds
decay_rate	10	inverse seconds
duration	1	seconds

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/08_integration.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only dt changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/08_integration.py --case 0

.venv-book/bin/python expanded_lab/modules/08_integration.py --case 3

.venv-book/bin/python expanded_lab/run.py integration --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/08_integration.py

```
"""Discrete integration and numerical stability. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    dt, k, T = p["dt"], p["decay_rate"], p["duration"]
    n = max(1, round(T / dt))
    actual = n * dt
    explicit = 1.0
    implicit = 1.0
    for _ in range(n):
        explicit *= 1 - k * dt
        implicit /= 1 + k * dt
    exact = np.exp(-k * actual)
    factor = abs(1 - k * dt)
    metrics = dict(
        explicit_final=explicit,
        implicit_final=implicit,
        exact_final=exact,
        explicit_abs_error=abs(explicit - exact),
        implicit_abs_error=abs(implicit - exact),
        explicit_amplification=factor,
    )
    derivation = [
        f"Choose dt={dt:g} s and decay rate k={k:g} per second.",
        f"Advance {n} steps to actual time {actual:.6f} s.",
        f"Explicit one-step multiplier is {1-k*dt:.6f}; its magnitude is {factor:.6f}.",
        f"Exact final value is {exact:.8f}; explicit gives {explicit:.8f}.",
        f"Implicit final value is {implicit:.8f}; compare absolute errors in the table.",
    ]
    notes = [
        (
            "The explicit recurrence is stable for this scalar decay."
            if factor < 1
            else "The explicit recurrence is on or outside its stability boundary; reducing
    dt changes the numerical problem."
        ),
        "Implicit stability does not mean high accuracy; a heavily damped numerical result
    can still be wrong.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"dt": 0.02, "decay_rate": 10.0, "duration": 1.0}
    parameters["dt"] = [0.005, 0.01, 0.02, 0.05, 0.1, 0.25][args.case]
```

```
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Track the recurrence multiplier before the final state. The exact exponential allows an accuracy comparison independent of either integrator. A negative explicit multiplier alternates sign even when the continuous solution does not.

I. Expected behavior and six worked cases

The controlled variable is dt. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

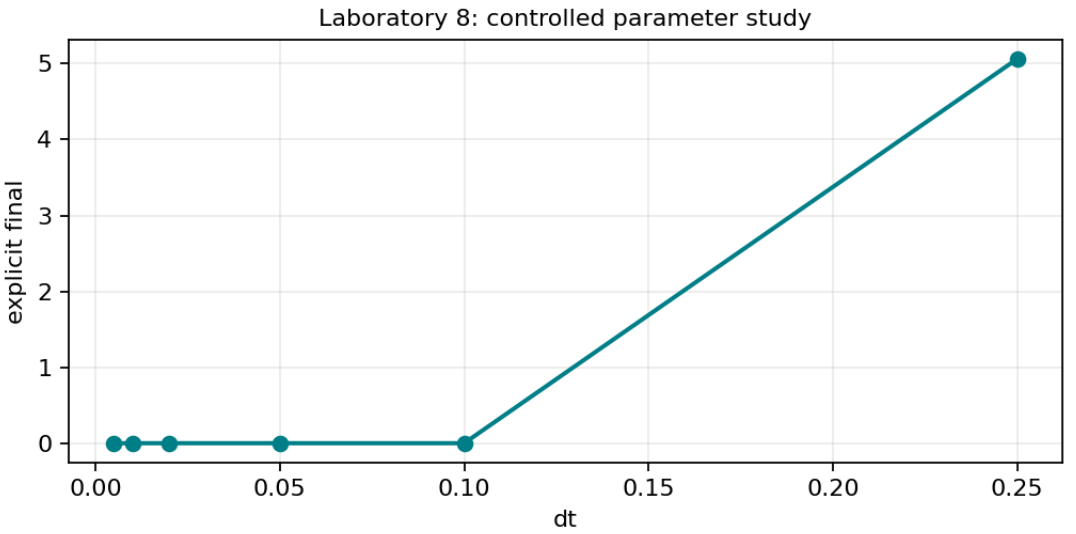


Figure L8. explicit final as dt changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	dt	explicit final
0	0.005	0.00003505

Case	dt	explicit final
1	0.01	0.00002656
2	0.02	0.00001427
3	0.05	9.5367432e-07
4	0.1	0
5	0.25	5.0625

Worked case 0: dt = 0.005

This case keeps the nominal configuration except for dt=0.005. The full input is {"decay_rate": 10.0, "dt": 0.005, "duration": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
explicit final	0.00003505
implicit final	0.00005783
exact final	0.0000454
explicit abs error	0.00001035
implicit abs error	0.00001243
explicit amplification	0.95

1. Choose dt=0.005 s and decay rate k=10 per second.
2. Advance 200 steps to actual time 1.000000 s.
3. Explicit one-step multiplier is 0.950000; its magnitude is 0.950000.
4. Exact final value is 0.00004540; explicit gives 0.00003505.
5. Implicit final value is 0.00005783; compare absolute errors in the table.

Worked case 1: dt = 0.01

This case keeps the nominal configuration except for dt=0.01. The full input is {"decay_rate": 10.0, "dt": 0.01, "duration": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
explicit final	0.00002656
implicit final	0.00007257
exact final	0.0000454
explicit abs error	0.00001884
implicit abs error	0.00002717
explicit amplification	0.9

1. Choose dt=0.01 s and decay rate k=10 per second.
2. Advance 100 steps to actual time 1.000000 s.
3. Explicit one-step multiplier is 0.900000; its magnitude is 0.900000.
4. Exact final value is 0.00004540; explicit gives 0.00002656.
5. Implicit final value is 0.00007257; compare absolute errors in the table.

Worked case 2: dt = 0.02

This case keeps the nominal configuration except for dt=0.02. The full input is {"decay_rate": 10.0, "dt": 0.02, "duration": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
explicit final	0.00001427
implicit final	0.00010988
exact final	0.0000454

Computed quantity	Numerical result
explicit abs error	0.00003113
implicit abs error	0.00006448
explicit amplification	0.8

1. Choose dt=0.02 s and decay rate k=10 per second.
2. Advance 50 steps to actual time 1.000000 s.
3. Explicit one-step multiplier is 0.800000; its magnitude is 0.800000.
4. Exact final value is 0.00004540; explicit gives 0.00001427.
5. Implicit final value is 0.00010988; compare absolute errors in the table.

Worked case 3: dt = 0.05

This case keeps the nominal configuration except for dt=0.05. The full input is {"decay_rate": 10.0, "dt": 0.05, "duration": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
explicit final	9.5367432e-07
implicit final	0.00030073
exact final	0.0000454
explicit abs error	0.00004445
implicit abs error	0.00025533
explicit amplification	0.5

1. Choose dt=0.05 s and decay rate k=10 per second.
2. Advance 20 steps to actual time 1.000000 s.
3. Explicit one-step multiplier is 0.500000; its magnitude is 0.500000.
4. Exact final value is 0.00004540; explicit gives 0.00000095.
5. Implicit final value is 0.00030073; compare absolute errors in the table.

Worked case 4: dt = 0.1

This case keeps the nominal configuration except for dt=0.1. The full input is {"decay_rate": 10.0, "dt": 0.1, "duration": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
explicit final	0
implicit final	0.00097656
exact final	0.0000454
explicit abs error	0.0000454
implicit abs error	0.00093116
explicit amplification	0

1. Choose dt=0.1 s and decay rate k=10 per second.
2. Advance 10 steps to actual time 1.000000 s.
3. Explicit one-step multiplier is 0.000000; its magnitude is 0.000000.
4. Exact final value is 0.00004540; explicit gives 0.00000000.
5. Implicit final value is 0.00097656; compare absolute errors in the table.

Worked case 5: dt = 0.25

This case keeps the nominal configuration except for dt=0.25. The full input is {"decay_rate": 10.0, "dt": 0.25, "duration": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
explicit final	5.0625
implicit final	0.00666389
exact final	0.0000454

Computed quantity	Numerical result
explicit abs error	5.0624546
implicit abs error	0.00661849
explicit amplification	1.5

1. Choose dt=0.25 s and decay rate k=10 per second.
2. Advance 4 steps to actual time 1.000000 s.
3. Explicit one-step multiplier is -1.500000; its magnitude is 1.500000.
4. Exact final value is 0.00004540; explicit gives 5.06250000.
5. Implicit final value is 0.00666389; compare absolute errors in the table.

The explicit recurrence is on or outside its stability boundary; reducing dt changes the numerical problem.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The explicit recurrence is stable for this scalar decay.

Implicit stability does not mean high accuracy; a heavily damped numerical result can still be wrong.

K. Failure modes and troubleshooting

Equating absence of NaNs with accuracy; changing dt without scaling controller updates; comparing runs with different actual horizons.

L. Practical exercises and extension tasks

1. Find the explicit stability boundary analytically and confirm it numerically.
2. Compare equal actual horizons for non-dividing timesteps.
3. Apply the same methodology to a mass-spring system.

M. Connection to the Asimov research platform

A MuJoCo timestep, controller period and policy period are separate choices; document their relationship and validate contact/controller behavior together.

N. Completion record

Record your chosen case, predicted trend, measured explicit_final, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 9, Monte Carlo uncertainty propagation and repeatability. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 9 - Monte Carlo uncertainty propagation and repeatability

A. The engineering question

A nominal calculation hides variability.

B. Concepts and mathematical model

Monte Carlo propagation samples uncertain inputs, evaluates the same physical relationship repeatedly, and summarizes the output distribution. It is useful when several uncertainties combine nonlinearly, but its answer is only as credible as the input distributions and correlations. A fixed seed makes an experiment repeatable; multiple independent seeds are needed to assess sampling variability rather than overfit a convenient random realization.

tau = m*g*l*cos(theta); sample uncertain m, l and theta; summarize mean, standard deviation and quantiles.

C. Real-world meaning and limits

Use measured calibration and manufacturing variation to justify randomization ranges rather than arbitrary broad noise.

D. Architecture and data flow

Configuration (seed, samples, angle_noise) -> numerical model -> mean_torque_Nm, std_torque_Nm, lower_quantile_Nm -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

seed identifies a pseudorandom stream; samples controls numerical sample count; angle_noise is standard deviation in radians.

Parameter	Nominal value	Interpretation
seed	17	integer random seed
samples	5000	count
angle_noise	0.02	radians

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/09_uncertainty.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only angle_noise changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/09_uncertainty.py --case 0

.venv-book/bin/python expanded_lab/modules/09_uncertainty.py --case 3
```

```
.venv-book/bin/python expanded_lab/run.py uncertainty --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/09_uncertainty.py

```
"""Monte Carlo uncertainty propagation and repeatability. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(int(p["seed"]))
    n = int(p["samples"])
    sigma = p["angle_noise"]
    mass = rng.normal(2.0, 0.04, n)
    length = rng.normal(0.25, 0.002, n)
    angle = rng.normal(0.6, sigma, n)
    torque = mass * 9.81 * length * np.cos(angle)
    nominal = 2 * 9.81 * 0.25 * np.cos(0.6)
    low, high = np.quantile(torque, [0.025, 0.975])
    metrics = dict(
        mean_torque_Nm=np.mean(torque),
        std_torque_Nm=np.std(torque, ddof=1),
        lower_quantile_Nm=low,
        upper_quantile_Nm=high,
        nominal_torque_Nm=nominal,
    )
    derivation = [
        f'Draw {n} samples with generator seed {int(p["seed"])}.',
        f"Mass sigma is 0.04 kg, lever sigma is 0.002 m, angle sigma is {sigma:g} rad.",
        f"Evaluate gravity torque for each sample; nominal value is {nominal:.6f} Nm.",
        f"Sample mean is {np.mean(torque):.6f} Nm and sample standard deviation is
    {np.std(torque,ddof=1):.6f} Nm.",
        f"Central empirical 95% interval is [{low:.6f},{high:.6f}] Nm.",
    ]
    notes = [
        "These quantiles describe the assumed population, not a confidence interval for the
    sample mean.",
        "Independent Gaussian inputs are a declared teaching assumption; shared calibration
    errors can create correlations.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"seed": 17, "samples": 5000, "angle_noise": 0.02}
    parameters["angle_noise"] = [0.001, 0.005, 0.01, 0.02, 0.05, 0.1][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
```

```
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Generate uncertain quantities first, then evaluate the same torque equation vectorially. Compare the nonlinear mean with the nominal prediction. Distinguish the distribution width from the standard error of a mean estimate.

I. Expected behavior and six worked cases

The controlled variable is angle_noise. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

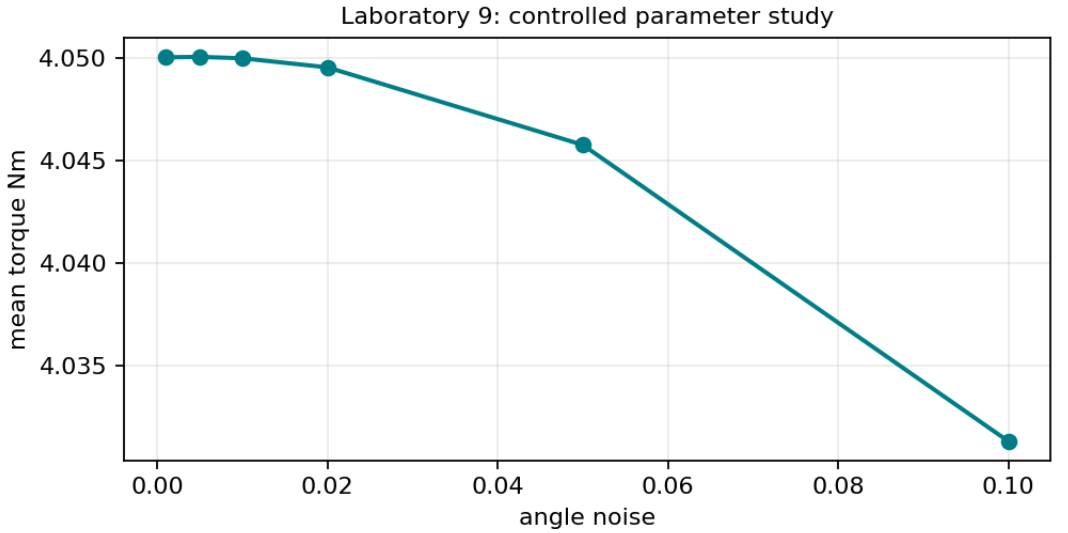


Figure L9. mean torque Nm as angle_noise changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	angle_noise	mean torque Nm
0	0.001	4.05003363
1	0.005	4.05005263

Case	angle_noise	mean torque Nm
2	0.01	4.04998425
3	0.02	4.04954036
4	0.05	4.04575343
5	0.1	4.03128607

Worked case 0: angle_noise = 0.001

This case keeps the nominal configuration except for angle_noise=0.001. The full input is {"angle_noise": 0.001, "samples": 5000, "seed": 17}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mean torque Nm	4.05003363
std torque Nm	0.08737753
lower quantile Nm	3.88050878
upper quantile Nm	4.2218778
nominal torque Nm	4.04827119

1. Draw 5000 samples with generator seed 17.
2. Mass sigma is 0.04 kg, lever sigma is 0.002 m, angle sigma is 0.001 rad.
3. Evaluate gravity torque for each sample; nominal value is 4.048271 Nm.
4. Sample mean is 4.050034 Nm and sample standard deviation is 0.087378 Nm.
5. Central empirical 95% interval is [3.880509,4.221878] Nm.

Worked case 1: angle_noise = 0.005

This case keeps the nominal configuration except for angle_noise=0.005. The full input is {"angle_noise": 0.005, "samples": 5000, "seed": 17}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mean torque Nm	4.05005263
std torque Nm	0.08844329
lower quantile Nm	3.87723807
upper quantile Nm	4.22493927
nominal torque Nm	4.04827119

1. Draw 5000 samples with generator seed 17.
2. Mass sigma is 0.04 kg, lever sigma is 0.002 m, angle sigma is 0.005 rad.
3. Evaluate gravity torque for each sample; nominal value is 4.048271 Nm.
4. Sample mean is 4.050053 Nm and sample standard deviation is 0.088443 Nm.
5. Central empirical 95% interval is [3.877238,4.224939] Nm.

Worked case 2: angle_noise = 0.01

This case keeps the nominal configuration except for angle_noise=0.01. The full input is {"angle_noise": 0.01, "samples": 5000, "seed": 17}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mean torque Nm	4.04998425
std torque Nm	0.09168126
lower quantile Nm	3.87142732
upper quantile Nm	4.22996162

Computed quantity	Numerical result
nominal torque Nm	4.04827119

1. Draw 5000 samples with generator seed 17.
2. Mass sigma is 0.04 kg, lever sigma is 0.002 m, angle sigma is 0.01 rad.
3. Evaluate gravity torque for each sample; nominal value is 4.048271 Nm.
4. Sample mean is 4.049984 Nm and sample standard deviation is 0.091681 Nm.
5. Central empirical 95% interval is [3.871427,4.229962] Nm.

Worked case 3: angle_noise = 0.02

This case keeps the nominal configuration except for angle_noise=0.02. The full input is {"angle_noise": 0.02, "samples": 5000, "seed": 17}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mean torque Nm	4.04954036
std torque Nm	0.10360821
lower quantile Nm	3.84319415
upper quantile Nm	4.24903658
nominal torque Nm	4.04827119

1. Draw 5000 samples with generator seed 17.
2. Mass sigma is 0.04 kg, lever sigma is 0.002 m, angle sigma is 0.02 rad.
3. Evaluate gravity torque for each sample; nominal value is 4.048271 Nm.
4. Sample mean is 4.049540 Nm and sample standard deviation is 0.103608 Nm.
5. Central empirical 95% interval is [3.843194,4.249037] Nm.

Worked case 4: angle_noise = 0.05

This case keeps the nominal configuration except for angle_noise=0.05. The full input is {"angle_noise": 0.05, "samples": 5000, "seed": 17}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mean torque Nm	4.04575343
std torque Nm	0.16435706
lower quantile Nm	3.70817561
upper quantile Nm	4.36225416
nominal torque Nm	4.04827119

1. Draw 5000 samples with generator seed 17.
2. Mass sigma is 0.04 kg, lever sigma is 0.002 m, angle sigma is 0.05 rad.
3. Evaluate gravity torque for each sample; nominal value is 4.048271 Nm.
4. Sample mean is 4.045753 Nm and sample standard deviation is 0.164357 Nm.
5. Central empirical 95% interval is [3.708176,4.362254] Nm.

Worked case 5: angle_noise = 0.1

This case keeps the nominal configuration except for angle_noise=0.1. The full input is {"angle_noise": 0.1, "samples": 5000, "seed": 17}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mean torque Nm	4.03128607
std torque Nm	0.29164591
lower quantile Nm	3.40336391
upper quantile Nm	4.55447884

Computed quantity	Numerical result
nominal torque Nm	4.04827119

1. Draw 5000 samples with generator seed 17.
2. Mass sigma is 0.04 kg, lever sigma is 0.002 m, angle sigma is 0.1 rad.
3. Evaluate gravity torque for each sample; nominal value is 4.048271 Nm.
4. Sample mean is 4.031286 Nm and sample standard deviation is 0.291646 Nm.
5. Central empirical 95% interval is [3.403364,4.554479] Nm.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

These quantiles describe the assumed population, not a confidence interval for the sample mean.

Independent Gaussian inputs are a declared teaching assumption; shared calibration errors can create correlations.

K. Failure modes and troubleshooting

Calling a percentile interval a guarantee; assuming independent errors without evidence; selecting a seed because it gives an attractive answer.

L. Practical exercises and extension tasks

1. Compare repeated seeds at 100 and 10,000 samples.
2. Correlate mass and lever length and inspect the output spread.
3. Use a bounded distribution for angle and compare tail behavior.

M. Connection to the Asimov research platform

Use measured calibration and manufacturing variation to justify randomization ranges rather than arbitrary broad noise.

N. Completion record

Record your chosen case, predicted trend, measured mean_torque_Nm, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 10, Sampling, aliasing and the difference between signal and display rate. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 10 - Sampling, aliasing and the difference between signal and display rate

A. The engineering question

Sampling converts a continuous-time signal into discrete observations.

B. Concepts and mathematical model

Different underlying frequencies can produce the same sampled sequence, so a plot can look smooth and still misrepresent motion. The Nyquist condition is a minimum for reconstructing a band-limited signal; real sensors require anti-alias filtering and margin. Display decimation is separate from measurement sampling and should never reduce the feedback rate accidentally.

x[n]=sin(2*pi*f*n/fs); aliased frequency = abs((f+fs/2) mod fs - fs/2).

C. Real-world meaning and limits

High-frequency encoder/IMU motion and low-rate network telemetry serve different purposes; the dashboard is not a control sensor.

D. Architecture and data flow

Configuration (sample_rate, signal_frequency, duration) -> numerical model -> nyquist_Hz, alias_Hz, fft_peak_Hz -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

sample_rate and signal_frequency are Hertz; duration is seconds. Samples begin at zero phase in this controlled example.

Parameter	Nominal value	Interpretation
sample_rate	100	Hertz
signal_frequency	35	Hertz
duration	2	seconds

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/10_signals.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only sample_rate changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/10_signals.py --case 0

.venv-book/bin/python expanded_lab/modules/10_signals.py --case 3

.venv-book/bin/python expanded_lab/run.py signals --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/10_signals.py

```
"""Sampling, aliasing and the difference between signal and display rate. Numerical teaching
     experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    fs, frequency, T = p["sample_rate"], p["signal_frequency"], p["duration"]
    n = max(4, int(fs * T))
    t = np.arange(n) / fs
    x = np.sin(2 * np.pi * frequency * t)
    alias = abs((frequency + fs / 2) % fs - fs / 2)
    spectrum = np.abs(np.fft.rfft(x))
    frequencies = np.fft.rfftfreq(n, 1 / fs)
    peak = float(frequencies[np.argmax(spectrum)])
    metrics = dict(
        nyquist_Hz=fs / 2,
        alias_Hz=alias,
        fft_peak_Hz=peak,
        samples=n,
        sampled_rms=float(np.sqrt(np.mean(x * x))),
    )
    derivation = [
        f"Sample a {frequency:g} Hz sinusoid at {fs:g} samples per second.",
        f"The sample interval is {1/fs:.6f} s and Nyquist frequency is {fs/2:.6f} Hz.",
        f"The analytical alias frequency is {alias:.6f} Hz.",
        f"An {n}-sample FFT resolves bins spaced {fs/n:.6f} Hz; peak bin is {peak:.6f} Hz.",
        f"Sampled RMS is {np.sqrt(np.mean(x*x)):.6f}; exact coincidences can produce
    misleading zeros.",
    ]
    notes = [
        (
            "Signal lies below Nyquist, but practical reconstruction still needs filtering
    assumptions."
            if frequency < fs / 2
            else "The original signal cannot be uniquely recovered from these samples
    without extra information."
        ),
        "FFT leakage depends on record duration and windowing; a peak bin alone is not a
    precision frequency estimate.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"sample_rate": 100.0, "signal_frequency": 35.0, "duration": 2.0}
    parameters["sample_rate"] = [20.0, 40.0, 60.0, 80.0, 100.0, 200.0][args.case]
```

```
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Calculate the expected alias independently before looking at the FFT. The frequency-bin calculation prevents overclaiming resolution from a short record. Consider anti-alias filtering before a physical sampling boundary.

I. Expected behavior and six worked cases

The controlled variable is sample_rate. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

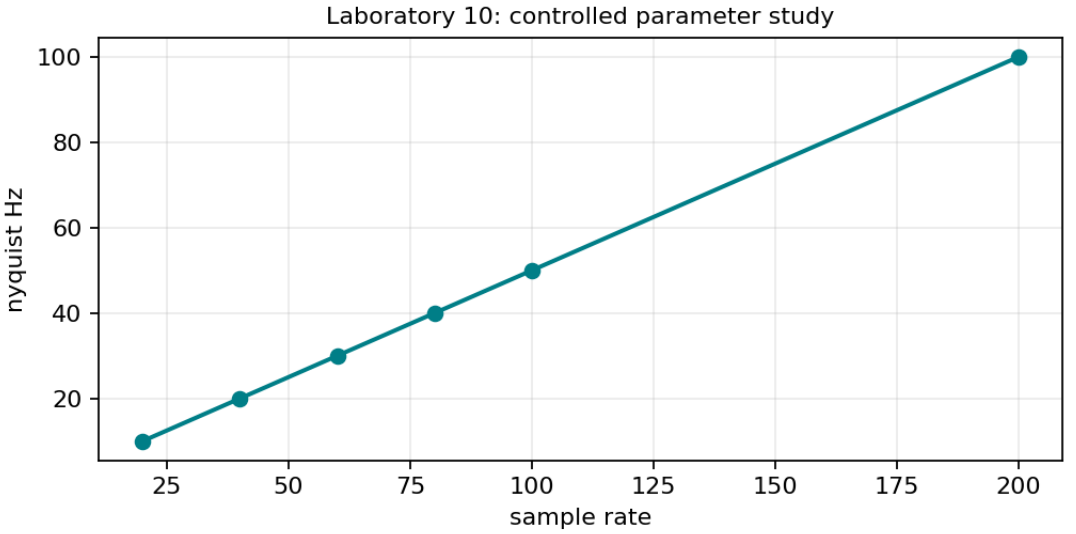


Figure L10. nyquist Hz as sample_rate changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	sample_rate	nyquist Hz
0	20	10

Case	sample_rate	nyquist Hz
1	40	20
2	60	30
3	80	40
4	100	50
5	200	100

Worked case 0: sample_rate = 20

This case keeps the nominal configuration except for sample_rate=20. The full input is {"duration": 2.0, "sample_rate": 20.0, "signal_frequency": 35.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
nyquist Hz	10
alias Hz	5
fft peak Hz	5
samples	40
sampled rms	0.70710678

1. Sample a 35 Hz sinusoid at 20 samples per second.
2. The sample interval is 0.050000 s and Nyquist frequency is 10.000000 Hz.
3. The analytical alias frequency is 5.000000 Hz.
4. An 40-sample FFT resolves bins spaced 0.500000 Hz; peak bin is 5.000000 Hz.
5. Sampled RMS is 0.707107; exact coincidences can produce misleading zeros.

Worked case 1: sample_rate = 40

This case keeps the nominal configuration except for sample_rate=40. The full input is {"duration": 2.0, "sample_rate": 40.0, "signal_frequency": 35.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
nyquist Hz	20
alias Hz	5
fft peak Hz	5
samples	80
sampled rms	0.70710678

1. Sample a 35 Hz sinusoid at 40 samples per second.
2. The sample interval is 0.025000 s and Nyquist frequency is 20.000000 Hz.
3. The analytical alias frequency is 5.000000 Hz.
4. An 80-sample FFT resolves bins spaced 0.500000 Hz; peak bin is 5.000000 Hz.
5. Sampled RMS is 0.707107; exact coincidences can produce misleading zeros.

Worked case 2: sample_rate = 60

This case keeps the nominal configuration except for sample_rate=60. The full input is {"duration": 2.0, "sample_rate": 60.0, "signal_frequency": 35.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
nyquist Hz	30
alias Hz	25
fft peak Hz	25

Computed quantity	Numerical result
samples	120
sampled rms	0.70710678

1. Sample a 35 Hz sinusoid at 60 samples per second.
2. The sample interval is 0.016667 s and Nyquist frequency is 30.000000 Hz.
3. The analytical alias frequency is 25.000000 Hz.
4. An 120-sample FFT resolves bins spaced 0.500000 Hz; peak bin is 25.000000 Hz.
5. Sampled RMS is 0.707107; exact coincidences can produce misleading zeros.

Worked case 3: sample_rate = 80

This case keeps the nominal configuration except for sample_rate=80. The full input is {"duration": 2.0, "sample_rate": 80.0, "signal_frequency": 35.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
nyquist Hz	40
alias Hz	35
fft peak Hz	35
samples	160
sampled rms	0.70710678

1. Sample a 35 Hz sinusoid at 80 samples per second.
2. The sample interval is 0.012500 s and Nyquist frequency is 40.000000 Hz.
3. The analytical alias frequency is 35.000000 Hz.
4. An 160-sample FFT resolves bins spaced 0.500000 Hz; peak bin is 35.000000 Hz.

5. Sampled RMS is 0.707107; exact coincidences can produce misleading zeros.

Signal lies below Nyquist, but practical reconstruction still needs filtering assumptions.

Worked case 4: sample_rate = 100

This case keeps the nominal configuration except for sample_rate=100. The full input is {"duration": 2.0, "sample_rate": 100.0, "signal_frequency": 35.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
nyquist Hz	50
alias Hz	35
fft peak Hz	35
samples	200
sampled rms	0.70710678

1. Sample a 35 Hz sinusoid at 100 samples per second.
2. The sample interval is 0.010000 s and Nyquist frequency is 50.000000 Hz.
3. The analytical alias frequency is 35.000000 Hz.
4. An 200-sample FFT resolves bins spaced 0.500000 Hz; peak bin is 35.000000 Hz.
5. Sampled RMS is 0.707107; exact coincidences can produce misleading zeros.

Signal lies below Nyquist, but practical reconstruction still needs filtering assumptions.

Worked case 5: sample_rate = 200

This case keeps the nominal configuration except for sample_rate=200. The full input is {"duration": 2.0, "sample_rate": 200.0, "signal_frequency": 35.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
nyquist Hz	100
alias Hz	35
fft peak Hz	35
samples	400
sampled rms	0.70710678

1. Sample a 35 Hz sinusoid at 200 samples per second.
2. The sample interval is 0.005000 s and Nyquist frequency is 100.000000 Hz.
3. The analytical alias frequency is 35.000000 Hz.
4. An 400-sample FFT resolves bins spaced 0.500000 Hz; peak bin is 35.000000 Hz.
5. Sampled RMS is 0.707107; exact coincidences can produce misleading zeros.

Signal lies below Nyquist, but practical reconstruction still needs filtering assumptions.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The original signal cannot be uniquely recovered from these samples without extra information.

FFT leakage depends on record duration and windowing; a peak bin alone is not a precision frequency estimate.

K. Failure modes and troubleshooting

Inferring physical oscillation frequency from a low-rate dashboard; reducing sample rate without filtering; trusting a line drawn between sparse points.

L. Practical exercises and extension tasks

1. Change sinusoid phase and inspect exact-coincidence cases.
2. Compare a two-second and ten-second FFT record.
3. Design a display decimator while preserving the original controller samples.

M. Connection to the Asimov research platform

High-frequency encoder/IMU motion and low-rate network telemetry serve different purposes; the dashboard is not a control sensor.

N. Completion record

Record your chosen case, predicted trend, measured nyquist_Hz, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 11, Link masses, payload shifts and center of mass. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Part VII - Mechanics, kinematics and contact

A geometric model becomes an engineering model only when its mass, inertia, load paths and contact assumptions are explicit. These laboratories move from mass aggregation through rotation and transmission effects to inverse kinematics and support feasibility. They use simple idealized bodies to isolate one phenomenon at a time. The complete humanoid model combines these phenomena; its apparent complexity is a reason to validate each component relationship first, not a reason to skip the equations.

Laboratory 11 - Link masses, payload shifts and center of mass

A. The engineering question

Center of mass is an average weighted by mass, not by link count.

B. Concepts and mathematical model

A light component placed far from the body can shift the combined center substantially. Adding a payload also changes inertia and torque requirements, but COM alone describes only the first moment of mass. This experiment adds a point payload to a simple three-link assembly and calculates both the new COM and the gravity moment about an origin. All coordinates must share one frame; averaging body-local positions would produce a physically meaningless answer.

c = sum(m_i*r_i)/sum(m_i); M_gravity = sum(r_i cross m_i*g).

C. Real-world meaning and limits

Weigh actual assemblies and measure their reference positions; CAD mass depends on assigned density and included components.

D. Architecture and data flow

Configuration (payload_mass, payload_x) -> numerical model -> total_mass_kg, com_x_m, com_z_m -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

payload_mass is kilograms; payload_x is meters in the common origin frame. The other three masses are fixed teaching links.

Parameter	Nominal value	Interpretation
payload_mass	1	kilograms
payload_x	0.3	meters

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/11_center_mass.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only payload_mass changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/11_center_mass.py --case 0

.venv-book/bin/python expanded_lab/modules/11_center_mass.py --case 3

.venv-book/bin/python expanded_lab/run.py center_mass --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/11_center_mass.py

```
"""Link masses, payload shifts and center of mass. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    masses = np.array([4.0, 2.0, 1.0, p["payload_mass"]])
    positions = np.array(
        [
            [0.0, 0.0, 0.4],
            [0.1, 0.0, 0.3],
            [-0.1, 0.0, 0.2],
            [p["payload_x"], 0.0, 0.5],
        ]
    )
    total = masses.sum()
    com = (masses[:, None] * positions).sum(axis=0) / total
    forces = masses[:, None] * np.array([0.0, 0.0, -9.81])
    moment = np.cross(positions, forces).sum(axis=0)
    expected = np.cross(com, total * np.array([0.0, 0.0, -9.81]))
    assert np.allclose(moment, expected)
    metrics = dict(
        total_mass_kg=total,
        com_x_m=com[0],
        com_z_m=com[2],
        gravity_pitch_moment_Nm=moment[1],
        weight_N=total * 9.81,
    )
    derivation = [
        f'Total mass is 4+2+1+{p["payload_mass"]:g} = {total:.6f} kg.',
        f'The payload lies at x={p["payload_x"]:g} m in the shared frame.',
        f"Sum mass times position, then divide by total: COM={com.tolist()} m.",
        f"Sum each gravity moment to obtain pitch moment {moment[1]:.6f} Nm.",
        "Applying total weight at the COM reproduces the same moment, providing an
    independent check.",
    ]
    notes = [
        "This static point-mass model does not represent distributed inertia or dynamic
    contact stability.",
        f"The horizontal COM displacement from the origin is {com[0]:.6f} m; compare it with
     a measured support margin.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"payload_mass": 1.0, "payload_x": 0.3}
```

```
parameters["payload_mass"] = [0.0, 0.25, 0.5, 1.0, 2.0, 4.0][args.case]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Broadcast masses over vector positions, form gravity forces, and compare distributed moments with the equivalent resultant at COM. This verifies the coordinate and weighting conventions together.

I. Expected behavior and six worked cases

The controlled variable is payload_mass. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

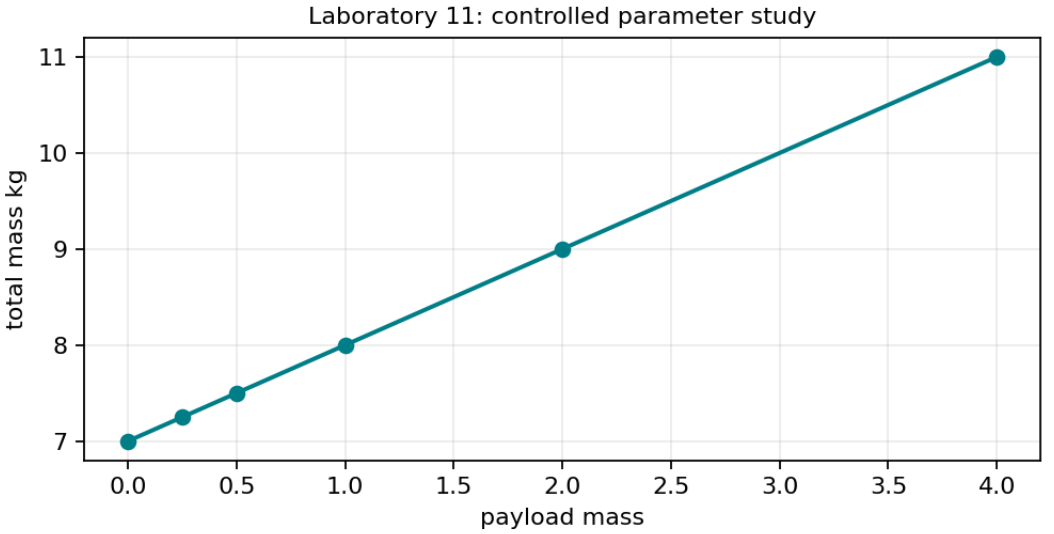


Figure L11. total mass kg as payload_mass changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	payload_mass	total mass kg
0	0	7

Case	payload_mass	total mass kg
1	0.25	7.25
2	0.5	7.5
3	1	8
4	2	9
5	4	11

Worked case 0: payload_mass = 0

This case keeps the nominal configuration except for payload_mass=0. The full input is {"payload_mass": 0.0, "payload_x": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
total mass kg	7
com x m	0.01428571
com z m	0.34285714
gravity pitch moment Nm	0.981
weight N	68.67

1. Total mass is 4+2+1+0 = 7.000000 kg.
2. The payload lies at x=0.3 m in the shared frame.
3. Sum mass times position, then divide by total: COM=[0.014285714285714287, 0.0, 0.3428571428571429] m.
4. Sum each gravity moment to obtain pitch moment 0.981000 Nm.
5. Applying total weight at the COM reproduces the same moment, providing an independent check.

Worked case 1: payload_mass = 0.25

This case keeps the nominal configuration except for payload_mass=0.25. The full input is {"payload_mass": 0.25, "payload_x": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
total mass kg	7.25
com x m	0.02413793
com z m	0.34827586
gravity pitch moment Nm	1.71675
weight N	71.1225

1. Total mass is 4+2+1+0.25 = 7.250000 kg.
2. The payload lies at x=0.3 m in the shared frame.
3. Sum mass times position, then divide by total: COM=[0.02413793103448276, 0.0, 0.34827586206896555] m.
4. Sum each gravity moment to obtain pitch moment 1.716750 Nm.
5. Applying total weight at the COM reproduces the same moment, providing an independent check.

The horizontal COM displacement from the origin is 0.024138 m; compare it with a measured support margin.

Worked case 2: payload_mass = 0.5

This case keeps the nominal configuration except for payload_mass=0.5. The full input is {"payload_mass": 0.5, "payload_x": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
total mass kg	7.5
com x m	0.03333333

Computed quantity	Numerical result
com z m	0.35333333
gravity pitch moment Nm	2.4525
weight N	73.575

1. Total mass is 4+2+1+0.5 = 7.500000 kg.
2. The payload lies at x=0.3 m in the shared frame.
3. Sum mass times position, then divide by total: COM=[0.0333333333333333, 0.0, 0.3533333333333334] m.
4. Sum each gravity moment to obtain pitch moment 2.452500 Nm.
5. Applying total weight at the COM reproduces the same moment, providing an independent check.

The horizontal COM displacement from the origin is 0.033333 m; compare it with a measured support margin.

Worked case 3: payload_mass = 1

This case keeps the nominal configuration except for payload_mass=1. The full input is {"payload_mass": 1.0, "payload_x": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
total mass kg	8
com x m	0.05
com z m	0.3625
gravity pitch moment Nm	3.924
weight N	78.48

1. Total mass is 4+2+1+1 = 8.000000 kg.
2. The payload lies at x=0.3 m in the shared frame.

3. Sum mass times position, then divide by total: COM=[0.05, 0.0, 0.36250000000000004] m.
4. Sum each gravity moment to obtain pitch moment 3.924000 Nm.
5. Applying total weight at the COM reproduces the same moment, providing an independent check.

The horizontal COM displacement from the origin is 0.050000 m; compare it with a measured support margin.

Worked case 4: payload_mass = 2

This case keeps the nominal configuration except for payload_mass=2. The full input is {"payload_mass": 2.0, "payload_x": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
total mass kg	9
com x m	0.07777778
com z m	0.37777778
gravity pitch moment Nm	6.867
weight N	88.29

1. Total mass is 4+2+1+2 = 9.000000 kg.
2. The payload lies at x=0.3 m in the shared frame.
3. Sum mass times position, then divide by total: COM=[0.07777777777777778, 0.0, 0.3777777777777778] m.
4. Sum each gravity moment to obtain pitch moment 6.867000 Nm.
5. Applying total weight at the COM reproduces the same moment, providing an independent check.

The horizontal COM displacement from the origin is 0.077778 m; compare it with a measured support margin.

Worked case 5: payload_mass = 4

This case keeps the nominal configuration except for payload_mass=4. The full input is {"payload_mass": 4.0, "payload_x": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
total mass kg	11
com x m	0.11818182
com z m	0.4
gravity pitch moment Nm	12.753
weight N	107.91

1. Total mass is 4+2+1+4 = 11.000000 kg.
2. The payload lies at x=0.3 m in the shared frame.
3. Sum mass times position, then divide by total: COM=[0.11818181818181818, 0.0, 0.4] m.
4. Sum each gravity moment to obtain pitch moment 12.753000 Nm.
5. Applying total weight at the COM reproduces the same moment, providing an independent check.

The horizontal COM displacement from the origin is 0.118182 m; compare it with a measured support margin.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

This static point-mass model does not represent distributed inertia or dynamic contact stability.

The horizontal COM displacement from the origin is 0.014286 m; compare it with a measured support margin.

K. Failure modes and troubleshooting

Averaging local link coordinates; omitting batteries or cables from mass accounting; treating COM margin as a dynamic balance proof.

L. Practical exercises and extension tasks

1. Move the payload laterally and recompute roll moment.
2. Estimate COM uncertainty for a 10 mm payload-position error.
3. Add a fifth link and verify the equivalent resultant identity.

M. Connection to the Asimov research platform

Weigh actual assemblies and measure their reference positions; CAD mass depends on assigned density and included components.

N. Completion record

Record your chosen case, predicted trend, measured total_mass_kg, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 12, Inertia tensors and the parallel-axis theorem. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 12 - Inertia tensors and the parallel-axis theorem

A. The engineering question

Mass describes resistance to translation.

B. Concepts and mathematical model

Inertia describes resistance to angular acceleration about a chosen axis and origin. Two assemblies with equal mass can require very different torque if their mass is distributed differently. The inertia tensor must be expressed in a named frame; moving the reference origin adds a parallel-axis term. Physical principal inertias are nonnegative and obey constraints that arbitrary matrix entries may violate. Here a uniform box is shifted away from an axis to make the geometric effect visible.

I_box = m/12*diag(b^2+c^2,a^2+c^2,a^2+b^2); I_origin = I_com + m*((r dot r)*I-r*r.T).

C. Real-world meaning and limits

Validate CAD inertia units, density assignments and reference frames before policy training or gravity compensation.

D. Architecture and data flow

Configuration (mass, offset, acceleration) -> numerical model -> Izz_kg_m2, minimum_principal_kg_m2, torque_z_Nm -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

mass is kilograms; offset is the x displacement of COM from the target origin in meters; acceleration is rad/s^2.

Parameter	Nominal value	Interpretation
mass	2	kilograms
offset	0.1	meters
acceleration	3	SI acceleration in the chapter equation

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/12_inertia.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only offset changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/12_inertia.py --case 0

.venv-book/bin/python expanded_lab/modules/12_inertia.py --case 3

.venv-book/bin/python expanded_lab/run.py inertia --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/12_inertia.py

```
"""Inertia tensors and the parallel-axis theorem. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    m = p["mass"]
    a, b, c = 0.2, 0.1, 0.08
    r = np.array([p["offset"], 0.02, 0.0])
    Ic = m / 12 * np.diag([b * b + c * c, a * a + c * c, a * a + b * b])
    shift = m * ((r @ r) * np.eye(3) - np.outer(r, r))
    Io = Ic + shift
    principal = np.linalg.eigvalsh(Io)
    alpha = np.array([0.0, 0.0, p["acceleration"]])
    tau = Io @ alpha
    assert np.all(principal > 0) and np.allclose(Io, Io.T)
    metrics = dict(
        Izz_kg_m2=Io[2, 2],
        minimum_principal_kg_m2=principal[0],
        torque_z_Nm=tau[2],
        offset_contribution_kg_m2=shift[2, 2],
    )
    derivation = [
        f"A uniform {m:g} kg box has COM Izz={Ic[2,2]:.8f} kg*m^2.",
        f"Shift the reference by {r.tolist()} m.",
        f"The parallel-axis contribution to Izz is {shift[2,2]:.8f} kg*m^2.",
        f"Total Izz is {Io[2,2]:.8f}; requested z acceleration is {alpha[2]:g} rad/s^2.",
        f"Ignoring other dynamics, required z torque is {tau[2]:.8f} Nm.",
    ]
    notes = [
        "This acceleration-only torque excludes gravity, Coriolis terms, friction and
    transmission effects.",
        "An inertia tensor attached to the wrong origin can make a controller consistently
    wrong even when total mass is correct.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"mass": 2.0, "offset": 0.1, "acceleration": 3.0}
    parameters["offset"] = [0.0, 0.02, 0.05, 0.1, 0.2, 0.3][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
```

```
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The outer product creates both diagonal and off-diagonal terms. Eigenvalues check positive principal inertias; the torque calculation shows how placement affects actuation demand.

I. Expected behavior and six worked cases

The controlled variable is offset. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

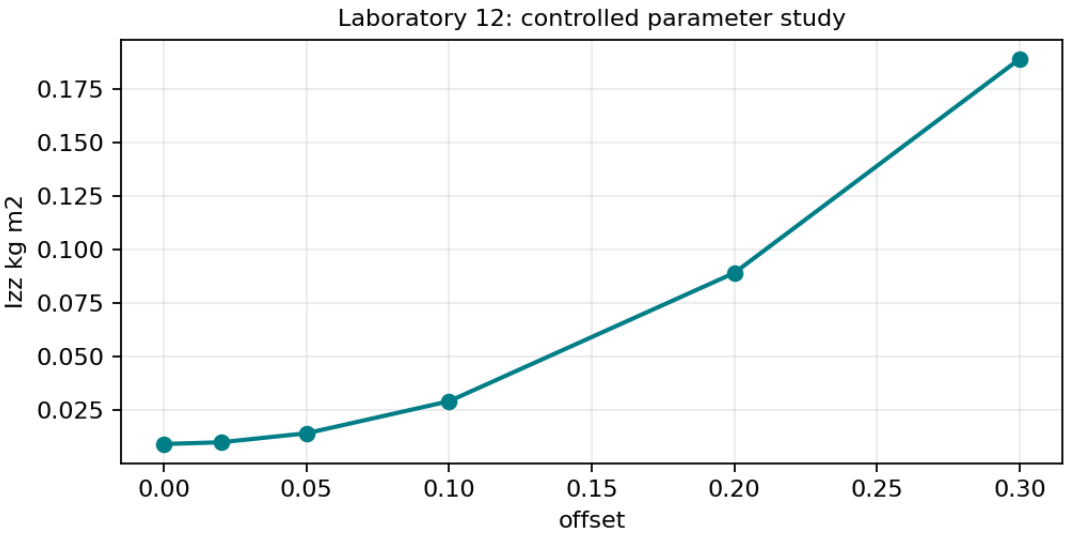


Figure L12. Izz kg m2 as offset changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	offset	Izz kg m2
0	0	0.00913333
1	0.02	0.00993333
2	0.05	0.01413333
3	0.1	0.02913333

Case	offset	Izz kg m2
4	0.2	0.08913333
5	0.3	0.18913333

Worked case 0: offset = 0

This case keeps the nominal configuration except for offset=0. The full input is {"acceleration": 3.0, "mass": 2.0, "offset": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
Izz kg m2	0.00913333
minimum principal kg m2	0.00353333
torque z Nm	0.0274
offset contribution kg m2	0.0008

1. A uniform 2 kg box has COM Izz=0.00833333 kg*m^2.
2. Shift the reference by [0.0, 0.02, 0.0] m.
3. The parallel-axis contribution to Izz is 0.00080000 kg*m^2.
4. Total Izz is 0.00913333; requested z acceleration is 3 rad/s^2.
5. Ignoring other dynamics, required z torque is 0.02740000 Nm.

Worked case 1: offset = 0.02

This case keeps the nominal configuration except for offset=0.02. The full input is {"acceleration": 3.0, "mass": 2.0, "offset": 0.02}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
Izz kg m2	0.00993333

Computed quantity	Numerical result
minimum principal kg m2	0.00340845
torque z Nm	0.0298
offset contribution kg m2	0.0016

1. A uniform 2 kg box has COM Izz=0.00833333 kg*m^2.
2. Shift the reference by [0.02, 0.02, 0.0] m.
3. The parallel-axis contribution to Izz is 0.00160000 kg*m^2.
4. Total Izz is 0.00993333; requested z acceleration is 3 rad/s^2.
5. Ignoring other dynamics, required z torque is 0.02980000 Nm.

Worked case 2: offset = 0.05

This case keeps the nominal configuration except for offset=0.05. The full input is {"acceleration": 3.0, "mass": 2.0, "offset": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
Izz kg m2	0.01413333
minimum principal kg m2	0.00311736
torque z Nm	0.0424
offset contribution kg m2	0.0058

1. A uniform 2 kg box has COM Izz=0.00833333 kg*m^2.
2. Shift the reference by [0.05, 0.02, 0.0] m.
3. The parallel-axis contribution to Izz is 0.00580000 kg*m^2.
4. Total Izz is 0.01413333; requested z acceleration is 3 rad/s^2.
5. Ignoring other dynamics, required z torque is 0.04240000 Nm.

Worked case 3: offset = 0.1

This case keeps the nominal configuration except for offset=0.1. The full input is {"acceleration": 3.0, "mass": 2.0, "offset": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
Izz kg m2	0.02913333
minimum principal kg m2	0.00288932
torque z Nm	0.0874
offset contribution kg m2	0.0208

1. A uniform 2 kg box has COM Izz=0.00833333 kg*m^2.
2. Shift the reference by [0.1, 0.02, 0.0] m.
3. The parallel-axis contribution to Izz is 0.02080000 kg*m^2.
4. Total Izz is 0.02913333; requested z acceleration is 3 rad/s^2.
5. Ignoring other dynamics, required z torque is 0.08740000 Nm.

Worked case 4: offset = 0.2

This case keeps the nominal configuration except for offset=0.2. The full input is {"acceleration": 3.0, "mass": 2.0, "offset": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
Izz kg m2	0.08913333
minimum principal kg m2	0.00277998
torque z Nm	0.2674
offset contribution kg m2	0.0808

1. A uniform 2 kg box has COM Izz=0.00833333 kg*m^2.

2. Shift the reference by [0.2, 0.02, 0.0] m.
3. The parallel-axis contribution to Izz is 0.08080000 kg*m^2.
4. Total Izz is 0.08913333; requested z acceleration is 3 rad/s^2.
5. Ignoring other dynamics, required z torque is 0.26740000 Nm.

Worked case 5: offset = 0.3

This case keeps the nominal configuration except for offset=0.3. The full input is {"acceleration": 3.0, "mass": 2.0, "offset": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
Izz kg m2	0.18913333
minimum principal kg m2	0.00275486
torque z Nm	0.5674
offset contribution kg m2	0.1808

1. A uniform 2 kg box has COM Izz=0.00833333 kg*m^2.
2. Shift the reference by [0.3, 0.02, 0.0] m.
3. The parallel-axis contribution to Izz is 0.18080000 kg*m^2.
4. Total Izz is 0.18913333; requested z acceleration is 3 rad/s^2.
5. Ignoring other dynamics, required z torque is 0.56740000 Nm.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

This acceleration-only torque excludes gravity, Coriolis terms, friction and transmission effects.

An inertia tensor attached to the wrong origin can make a controller consistently wrong even when total mass is correct.

K. Failure modes and troubleshooting

Using grams as kilograms; importing inertia in mm^2 units; moving inertia without transforming its frame; diagonalizing away real coupling.

L. Practical exercises and extension tasks

1. Rotate the tensor by R*I*R.T and compare kinetic energy in both frames.
2. Double every box dimension and derive the inertia scaling at fixed mass.
3. Compare a point mass with a distributed box.

M. Connection to the Asimov research platform

Validate CAD inertia units, density assignments and reference frames before policy training or gravity compensation.

N. Completion record

Record your chosen case, predicted trend, measured Izz_kg_m2, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 13, Angular momentum, kinetic energy and off-axis motion. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 13 - Angular momentum, kinetic energy and off-axis motion

A. The engineering question

For rotation about a fixed principal axis, angular momentum and angular velocity are parallel.

B. Concepts and mathematical model

For a general rigid body they need not be. The inertia tensor maps angular velocity to angular momentum, and rotational kinetic energy is a quadratic form. This distinction matters when arms or torso motions redistribute whole-body angular momentum. Conservation arguments require the external moment assumptions to be explicit; contact forces and a support fixture can inject moments.

L=I*omega; E_rot=0.5*omega.T*I*omega; tau=I*alpha+omega cross (I*omega).

C. Real-world meaning and limits

Whole-body controllers must account for angular momentum and contact wrench feasibility; a torso orientation target alone is incomplete.

D. Architecture and data flow

Configuration (speed) -> numerical model -> kinetic_energy_J, momentum_norm_kg_m2_s, misalignment_rad -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

speed is the x angular-velocity component in rad/s; the y and z components and inertia tensor are fixed.

Parameter	Nominal value	Interpretation
speed	1	angular speed in the chapter equation

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/13_angular_momentum.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only speed changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/13_angular_momentum.py --case 0

.venv-book/bin/python expanded_lab/modules/13_angular_momentum.py --case 3

.venv-book/bin/python expanded_lab/run.py angular_momentum --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/13_angular_momentum.py

```
"""Angular momentum, kinetic energy and off-axis motion. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    I = np.diag([0.08, 0.12, 0.2])
    omega = np.array([p["speed"], 0.7, 0.2])
    alpha = np.array([0.0, 0.3, 0.0])
    momentum = I @ omega
    energy = 0.5 * omega @ momentum
    gyroscopic = np.cross(omega, momentum)
    torque = I @ alpha + gyroscopic
    cosine = (omega @ momentum) / (
        np.linalg.norm(omega) * np.linalg.norm(momentum)
    )
    angle = np.arccos(np.clip(cosine, -1, 1))
    metrics = dict(
        kinetic_energy_J=energy,
        momentum_norm_kg_m2_s=np.linalg.norm(momentum),
        misalignment_rad=angle,
        gyroscopic_torque_Nm=np.linalg.norm(gyroscopic),
        required_torque_Nm=np.linalg.norm(torque),
    )
    derivation = [
        f"Angular velocity is {omega.tolist()} rad/s.",
        f"Multiply by diagonal inertia to obtain L={momentum.tolist()} kg*m^2/s.",
        f"The quadratic energy is {energy:.6f} J.",
        f"Compute omega cross L: {gyroscopic.tolist()} Nm.",
        f"Include the acceleration term to obtain torque {torque.tolist()} Nm.",
    ]
    notes = [
        f"L and omega differ in direction by {angle:.6f} rad in this non-spherical body.",
        "The calculation assumes a rigid body about its COM with a consistently expressed
    inertia tensor.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"speed": 1.0}
    parameters["speed"] = [0.1, 0.3, 0.7, 1.0, 2.0, 4.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
```

```
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Compute momentum before energy and gyroscopic torque so the directional coupling is visible. A spherical-inertia extension should eliminate the gyroscopic term for any omega.

I. Expected behavior and six worked cases

The controlled variable is speed. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

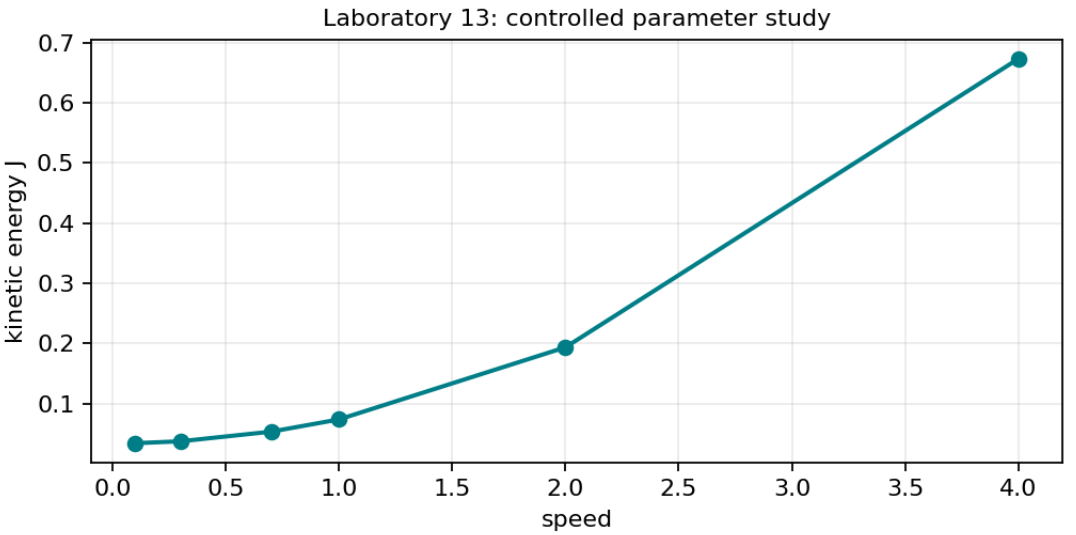


Figure L13. kinetic energy J as speed changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	speed	kinetic energy J
0	0.1	0.0338
1	0.3	0.037
2	0.7	0.053

Case	speed	kinetic energy J
3	1	0.0734
4	2	0.1934
5	4	0.6734

Worked case 0: speed = 0.1

This case keeps the nominal configuration except for speed=0.1. The full input is {"speed": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
kinetic energy J	0.0338
momentum norm kg m2 s	0.09338094
misalignment rad	0.17269333
gyroscopic torque Nm	0.01179152
required torque Nm	0.03552802

1. Angular velocity is [0.1, 0.7, 0.2] rad/s.
2. Multiply by diagonal inertia to obtain L=[0.008, 0.08399999999999999, 0.04000000000000001] kg*m^2/s.
3. The quadratic energy is 0.033800 J.
4. Compute omega cross L: [0.011200000000000005, -0.002400000000000001, 0.0027999999999999995] Nm.
5. Include the acceleration term to obtain torque [0.011200000000000005, 0.0336, 0.0027999999999999995] Nm.

Worked case 1: speed = 0.3

This case keeps the nominal configuration except for speed=0.3. The full input is {"speed": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
kinetic energy J	0.037
momentum norm kg m2 s	0.0960833
misalignment rad	0.20961727
gyroscopic torque Nm	0.01574293
required torque Nm	0.03202249

1. Angular velocity is [0.3, 0.7, 0.2] rad/s.
2. Multiply by diagonal inertia to obtain L=[0.024, 0.08399999999999999, 0.04000000000000001] kg*m^2/s.
3. The quadratic energy is 0.037000 J.
4. Compute omega cross L: [0.011200000000000005, -0.007200000000000015, 0.008399999999999998] Nm.
5. Include the acceleration term to obtain torque [0.011200000000000005, 0.028799999999999996, 0.008399999999999998] Nm.

L and omega differ in direction by 0.209617 rad in this non-spherical body.

Worked case 2: speed = 0.7

This case keeps the nominal configuration except for speed=0.7. The full input is {"speed": 0.7}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
kinetic energy J	0.053
momentum norm kg m2 s	0.10859098
misalignment rad	0.25948336
gyroscopic torque Nm	0.02813965
required torque Nm	0.02963511

1. Angular velocity is [0.7, 0.7, 0.2] rad/s.

2. Multiply by diagonal inertia to obtain L=[0.055999999999999994, 0.08399999999999999, 0.04000000000000001] kg*m^2/s.
3. The quadratic energy is 0.053000 J.
4. Compute omega cross L: [0.011200000000000005, -0.016800000000000002, 0.0196] Nm.
5. Include the acceleration term to obtain torque [0.011200000000000005, 0.019199999999999995, 0.0196] Nm.

L and omega differ in direction by 0.259483 rad in this non-spherical body.

Worked case 3: speed = 1

This case keeps the nominal configuration except for speed=1. The full input is {"speed": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
kinetic energy J	0.0734
momentum norm kg m2 s	0.12270289
misalignment rad	0.25674916
gyroscopic torque Nm	0.03854141
required torque Nm	0.03245674

1. Angular velocity is [1.0, 0.7, 0.2] rad/s.
2. Multiply by diagonal inertia to obtain L=[0.08, 0.08399999999999999, 0.04000000000000001] kg*m^2/s.
3. The quadratic energy is 0.073400 J.
4. Compute omega cross L: [0.011200000000000005, -0.024000000000000007, 0.027999999999999997] Nm.
5. Include the acceleration term to obtain torque [0.011200000000000005, 0.01199999999999999, 0.027999999999999997] Nm.

L and omega differ in direction by 0.256749 rad in this non-spherical body.

Worked case 4: speed = 2

This case keeps the nominal configuration except for speed=2. The full input is {"speed": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
kinetic energy J	0.1934
momentum norm kg m2 s	0.18508376
misalignment rad	0.19052989
gyroscopic torque Nm	0.07460188
required torque Nm	0.05835615

1. Angular velocity is [2.0, 0.7, 0.2] rad/s.
2. Multiply by diagonal inertia to obtain L=[0.16, 0.08399999999999999, 0.04000000000000001] kg*m^2/s.
3. The quadratic energy is 0.193400 J.
4. Compute omega cross L: [0.011200000000000005, -0.048000000000000015, 0.055999999999999994] Nm.
5. Include the acceleration term to obtain torque [0.011200000000000005, -0.012000000000000018, 0.055999999999999994] Nm.

L and omega differ in direction by 0.190530 rad in this non-spherical body.

Worked case 5: speed = 4

This case keeps the nominal configuration except for speed=4. The full input is {"speed": 4.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
kinetic energy J	0.6734
momentum norm kg m2 s	0.33325066
misalignment rad	0.10940494

Computed quantity	Numerical result
gyroscopic torque Nm	0.14793728
required torque Nm	0.12755172

1. Angular velocity is [4.0, 0.7, 0.2] rad/s.
2. Multiply by diagonal inertia to obtain L=[0.32, 0.08399999999999999, 0.04000000000000001] kg*m^2/s.
3. The quadratic energy is 0.673400 J.
4. Compute omega cross L: [0.011200000000000005, -0.09600000000000003, 0.11199999999999999] Nm.
5. Include the acceleration term to obtain torque [0.011200000000000005, -0.06000000000000003, 0.11199999999999999] Nm.

L and omega differ in direction by 0.109405 rad in this non-spherical body.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

L and omega differ in direction by 0.172693 rad in this non-spherical body.

The calculation assumes a rigid body about its COM with a consistently expressed inertia tensor.

K. Failure modes and troubleshooting

Treating inertia as one scalar in a three-axis problem; omitting external moments when discussing conservation.

L. Practical exercises and extension tasks

1. Set all diagonal inertias equal and verify L is parallel to omega.
2. Reverse omega and compare momentum, energy and gyroscopic torque.
3. Evaluate kinetic energy after a frame rotation.

M. Connection to the Asimov research platform

Whole-body controllers must account for angular momentum and contact wrench feasibility; a torso orientation target alone is incomplete.

N. Completion record

Record your chosen case, predicted trend, measured kinetic_energy_J, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 14, Gravity, acceleration and joint-load budgeting. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 14 - Gravity, acceleration and joint-load budgeting

A. The engineering question

A motor must support gravity and accelerate the link while overcoming friction.

B. Concepts and mathematical model

Looking only at the static gravity torque can underestimate peaks; looking only at the peak actuator rating can overestimate continuous capability. This experiment combines a point payload and a uniform rod into an explicitly simple joint-load model. The result is useful for reasoning about sensitivity, but cannot replace a released mechanism model, bearing analysis or duty-cycle study.

tau = (m_payload*g*l + m_rod*g*l/2)*cos(q) + (m_payload*l^2+m_rod*l^2/3)*q_ddot + tau_friction.

C. Real-world meaning and limits

Use release-specific mass/inertia, measured friction and continuous motor envelopes before choosing a physical load.

D. Architecture and data flow

Configuration (length, payload, rod_mass, angle_deg, acceleration) -> numerical model -> gravity_Nm, inertia_kg_m2, acceleration_Nm -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

length is meters; payload and rod_mass are kilograms; acceleration is output-axis rad/s^2; angle is measured from horizontal.

Parameter	Nominal value	Interpretation
length	0.25	meters
payload	2	Defined by the explicit chapter model and parameter contract
rod_mass	1	Defined by the explicit chapter model and parameter contract
angle_deg	20	degrees
acceleration	3	SI acceleration in the chapter equation

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/14_joint_loading.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only acceleration changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.

5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/14_joint_loading.py --case 0

.venv-book/bin/python expanded_lab/modules/14_joint_loading.py --case 3

.venv-book/bin/python expanded_lab/run.py joint_loading --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/14_joint_loading.py

```
"""Gravity, acceleration and joint-load budgeting. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    l, mp, mr = p["length"], p["payload"], p["rod_mass"]
    q = np.deg2rad(p["angle_deg"])
    alpha = p["acceleration"]
    gravity = (mp * 9.81 * l + mr * 9.81 * l / 2) * np.cos(q)
    inertia = mp * l * l + mr * l * l / 3
    dynamic = inertia * alpha
    friction = 0.3
    total = gravity + dynamic + friction
    metrics = dict(
        gravity_Nm=gravity,
        inertia_kg_m2=inertia,
        acceleration_Nm=dynamic,
        total_required_Nm=total,
        gravity_fraction=gravity / total,
    )
    derivation = [
        f"Payload mass {mp:g} kg acts at {l:g} m; rod COM acts at {l/2:g} m.",
        f'At angle {p["angle_deg"]:g} degrees, gravity torque is {gravity:.6f} Nm.',
        f"Total modeled rotational inertia is {inertia:.6f} kg*m^2.",
        f"Acceleration {alpha:g} rad/s^2 adds {dynamic:.6f} Nm.",
        f"Add 0.3 Nm illustrative friction: total demand is {total:.6f} Nm.",
    ]
    notes = [
        "The load budget separates assumptions so the dominant contribution can be
    identified.",
        "This one-axis model excludes impact, gear reflected inertia, cable forces and
    multi-link coupling.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {
        "length": 0.25,
        "payload": 2.0,
        "rod_mass": 1.0,
        "angle_deg": 20.0,
        "acceleration": 3.0,
    }
    parameters["acceleration"] = [0.0, 1.0, 2.0, 3.0, 6.0, 12.0][args.case]
    metrics, derivation, notes = experiment(parameters)
```

```
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The rod and payload have different lever arms and inertia formulas. Keeping gravity and acceleration terms separate makes the result auditable and supports a duty-cycle calculation later.

I. Expected behavior and six worked cases

The controlled variable is acceleration. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

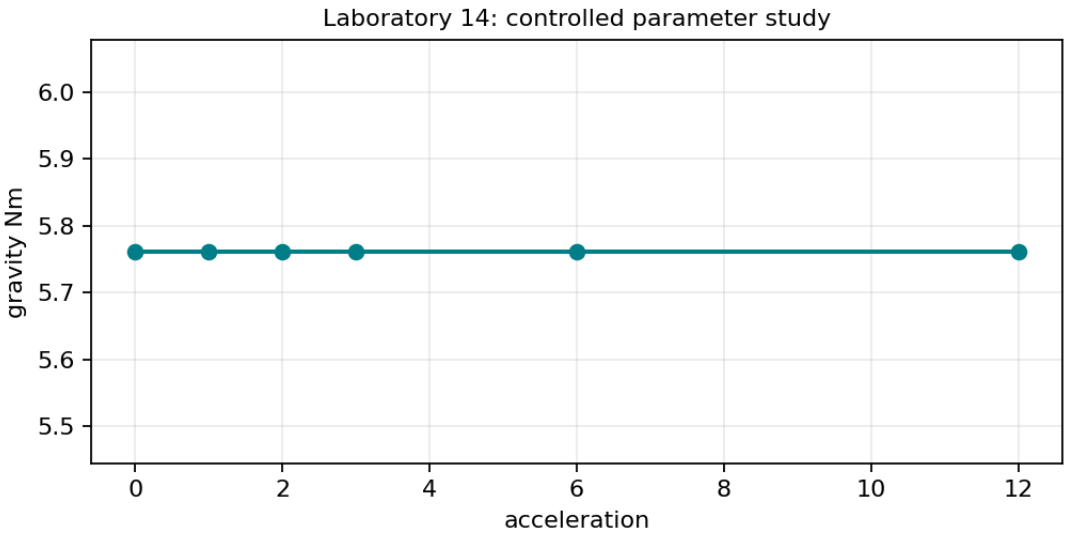


Figure L14. gravity Nm as acceleration changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	acceleration	gravity Nm
0	0	5.76149038
1	1	5.76149038

Case	acceleration	gravity Nm
2	2	5.76149038
3	3	5.76149038
4	6	5.76149038
5	12	5.76149038

Worked case 0: acceleration = 0

This case keeps the nominal configuration except for acceleration=0. The full input is {"acceleration": 0.0, "angle_deg": 20.0, "length": 0.25, "payload": 2.0, "rod_mass": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
gravity Nm	5.76149038
inertia kg m2	0.14583333
acceleration Nm	0
total required Nm	6.06149038
gravity fraction	0.95050722

1. Payload mass 2 kg acts at 0.25 m; rod COM acts at 0.125 m.
2. At angle 20 degrees, gravity torque is 5.761490 Nm.
3. Total modeled rotational inertia is 0.145833 kg*m^2.
4. Acceleration 0 rad/s^2 adds 0.000000 Nm.
5. Add 0.3 Nm illustrative friction: total demand is 6.061490 Nm.

Worked case 1: acceleration = 1

This case keeps the nominal configuration except for acceleration=1. The full input is {"acceleration": 1.0, "angle_deg": 20.0, "length": 0.25, "payload": 2.0, "rod_mass": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
gravity Nm	5.76149038
inertia kg m2	0.14583333
acceleration Nm	0.14583333
total required Nm	6.20732371
gravity fraction	0.92817624

1. Payload mass 2 kg acts at 0.25 m; rod COM acts at 0.125 m.
2. At angle 20 degrees, gravity torque is 5.761490 Nm.
3. Total modeled rotational inertia is 0.145833 kg*m^2.
4. Acceleration 1 rad/s^2 adds 0.145833 Nm.
5. Add 0.3 Nm illustrative friction: total demand is 6.207324 Nm.

Worked case 2: acceleration = 2

This case keeps the nominal configuration except for acceleration=2. The full input is {"acceleration": 2.0, "angle_deg": 20.0, "length": 0.25, "payload": 2.0, "rod_mass": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
gravity Nm	5.76149038
inertia kg m2	0.14583333
acceleration Nm	0.29166667
total required Nm	6.35315705
gravity fraction	0.90687045

1. Payload mass 2 kg acts at 0.25 m; rod COM acts at 0.125 m.
2. At angle 20 degrees, gravity torque is 5.761490 Nm.
3. Total modeled rotational inertia is 0.145833 kg*m^2.

4. Acceleration 2 rad/s^2 adds 0.291667 Nm.
5. Add 0.3 Nm illustrative friction: total demand is 6.353157 Nm.

Worked case 3: acceleration = 3

This case keeps the nominal configuration except for acceleration=3. The full input is {"acceleration": 3.0, "angle_deg": 20.0, "length": 0.25, "payload": 2.0, "rod_mass": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
gravity Nm	5.76149038
inertia kg m2	0.14583333
acceleration Nm	0.4375
total required Nm	6.49899038
gravity fraction	0.88652084

1. Payload mass 2 kg acts at 0.25 m; rod COM acts at 0.125 m.
2. At angle 20 degrees, gravity torque is 5.761490 Nm.
3. Total modeled rotational inertia is 0.145833 kg*m^2.
4. Acceleration 3 rad/s^2 adds 0.437500 Nm.
5. Add 0.3 Nm illustrative friction: total demand is 6.498990 Nm.

Worked case 4: acceleration = 6

This case keeps the nominal configuration except for acceleration=6. The full input is {"acceleration": 6.0, "angle_deg": 20.0, "length": 0.25, "payload": 2.0, "rod_mass": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
gravity Nm	5.76149038
inertia kg m2	0.14583333

Computed quantity	Numerical result
acceleration Nm	0.875
total required Nm	6.93649038
gravity fraction	0.83060598

1. Payload mass 2 kg acts at 0.25 m; rod COM acts at 0.125 m.
2. At angle 20 degrees, gravity torque is 5.761490 Nm.
3. Total modeled rotational inertia is 0.145833 kg*m^2.
4. Acceleration 6 rad/s^2 adds 0.875000 Nm.
5. Add 0.3 Nm illustrative friction: total demand is 6.936490 Nm.

Worked case 5: acceleration = 12

This case keeps the nominal configuration except for acceleration=12. The full input is {"acceleration": 12.0, "angle_deg": 20.0, "length": 0.25, "payload": 2.0, "rod_mass": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
gravity Nm	5.76149038
inertia kg m2	0.14583333
acceleration Nm	1.75
total required Nm	7.81149038
gravity fraction	0.73756609

1. Payload mass 2 kg acts at 0.25 m; rod COM acts at 0.125 m.
2. At angle 20 degrees, gravity torque is 5.761490 Nm.
3. Total modeled rotational inertia is 0.145833 kg*m^2.
4. Acceleration 12 rad/s^2 adds 1.750000 Nm.
5. Add 0.3 Nm illustrative friction: total demand is 7.811490 Nm.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The load budget separates assumptions so the dominant contribution can be identified.

This one-axis model excludes impact, gear reflected inertia, cable forces and multi-link coupling.

K. Failure modes and troubleshooting

Using motor-shaft acceleration with output-axis inertia; applying a static margin to an impact; omitting the fixture reaction load.

L. Practical exercises and extension tasks

1. Compare the horizontal and vertical configurations.
2. Double payload mass and identify linear terms.
3. Calculate a trajectory torque trace rather than one worst-looking pose.

M. Connection to the Asimov research platform

Use release-specific mass/inertia, measured friction and continuous motor envelopes before choosing a physical load.

N. Completion record

Record your chosen case, predicted trend, measured gravity_Nm, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 15, Gear ratio, reflected inertia and backlash dead zones. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 15 - Gear ratio, reflected inertia and backlash dead zones

A. The engineering question

A gearbox changes the coordinate system between a motor and an output.

B. Concepts and mathematical model

Rotor inertia reflected to the output grows with the square of the velocity ratio, which can make a high-ratio transmission dynamically significant. Backlash is a lost-motion region during reversals, not a random measurement error. This laboratory combines an inertia calculation with a simple play operator that holds the output until the input crosses the slack boundary. It is deliberately a scalar transmission model, not an identified Asimov actuator.

I_reflected = J_motor*G^2; backlash output remains inside input +/- half_gap until a boundary forces motion.

C. Real-world meaning and limits

Actual transmission behavior requires measured reversals and verified encoder placement; do not copy the teaching gap into a physical controller.

D. Architecture and data flow

Configuration (ratio, rotor_inertia, backlash) -> numerical model -> reflected_inertia_kg_m2, tracking_rmse_rad, peak_lost_motion_rad -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

ratio is motor/output speed ratio; rotor_inertia is motor-shaft kg*m^2; backlash is total output-coordinate gap in radians.

Parameter	Nominal value	Interpretation
ratio	36	dimensionless motor/output ratio
rotor_inertia	0.00002	Defined by the explicit chapter model and parameter contract
backlash	0.01	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/15_gear_backlash.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only backlash changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/15_gear_backlash.py --case 0

.venv-book/bin/python expanded_lab/modules/15_gear_backlash.py --case 3

.venv-book/bin/python expanded_lab/run.py gear_backlash --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/15_gear_backlash.py

```
"""Gear ratio, reflected inertia and backlash dead zones. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    ratio = p["ratio"]
    J = p["rotor_inertia"]
    gap = p["backlash"]
    t = np.linspace(0, 4, 1001)
    desired = 0.1 * np.sin(2 * np.pi * 0.5 * t)
    actual = np.zeros_like(desired)
    for i in range(1, len(t)):
        previous = actual[i - 1]
        actual[i] = np.clip(previous, desired[i] - gap / 2, desired[i] + gap / 2)
    error = desired - actual
    metrics = dict(
        reflected_inertia_kg_m2=J * ratio**2,
        tracking_rmse_rad=np.sqrt(np.mean(error**2)),
        peak_lost_motion_rad=np.max(np.abs(error)),
        transmitted_range_rad=np.ptp(actual),
    )
    derivation = [
        f"Rotor inertia {J:.3e} kg*m^2 is reflected through ratio {ratio:g}.",
        f"Output-equivalent inertia is {J*ratio**2:.8f} kg*m^2.",
        f"The commanded output trajectory has 0.1 rad amplitude and 0.5 Hz frequency.",
        f"Apply a total backlash gap of {gap:.6f} rad with a stateful play operator.",
        f"The resulting RMSE is {np.sqrt(np.mean(error**2)):.6f} rad; peak error is
    {np.max(np.abs(error)):.6f} rad.",
    ]
    notes = [
        "A stateful dead zone creates hysteresis; adding independent Gaussian noise would
    model a different phenomenon.",
        "The play model excludes stiffness, friction and load-dependent tooth contact.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"ratio": 36.0, "rotor_inertia": 2e-05, "backlash": 0.01}
    parameters["backlash"] = [0.0, 0.002, 0.005, 0.01, 0.02, 0.04][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
```

```
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The output carries memory through previous, so reversal behavior differs from a stateless clipping function. The reflected inertia calculation is independent of the backlash trajectory and must not be conflated with it.

I. Expected behavior and six worked cases

The controlled variable is backlash. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

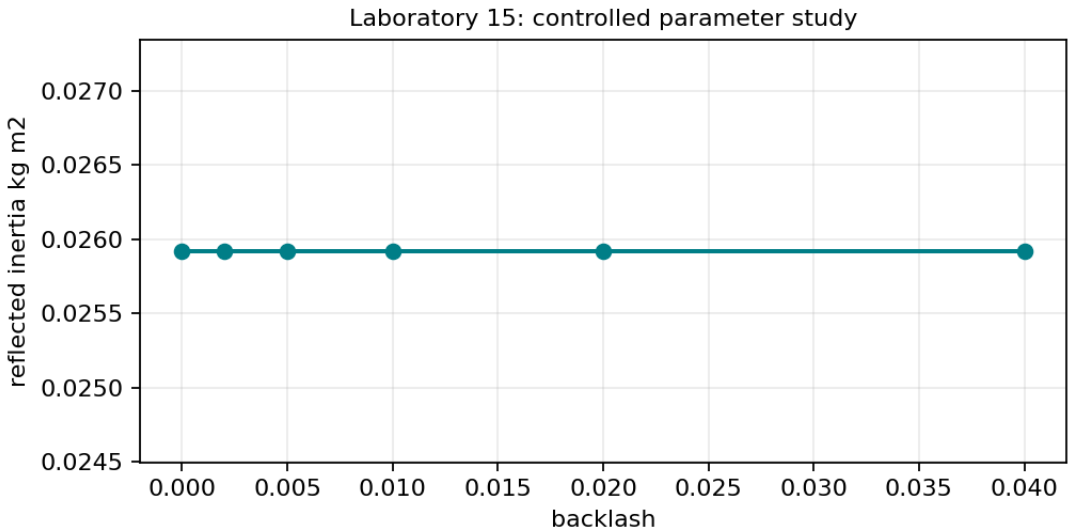


Figure L15. reflected inertia kg m2 as backlash changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	backlash	reflected inertia kg m2
0	0	0.02592
1	0.002	0.02592
2	0.005	0.02592

Case	backlash	reflected inertia kg m2
3	0.01	0.02592
4	0.02	0.02592
5	0.04	0.02592

Worked case 0: backlash = 0

This case keeps the nominal configuration except for backlash=0. The full input is {"backlash": 0.0, "ratio": 36.0, "rotor_inertia": 2e-05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reflected inertia kg m2	0.02592
tracking rmse rad	0
peak lost motion rad	0
transmitted range rad	0.2

1. Rotor inertia 2.000e-05 kg*m^2 is reflected through ratio 36.
2. Output-equivalent inertia is 0.02592000 kg*m^2.
3. The commanded output trajectory has 0.1 rad amplitude and 0.5 Hz frequency.
4. Apply a total backlash gap of 0.000000 rad with a stateful play operator.
5. The resulting RMSE is 0.000000 rad; peak error is 0.000000 rad.

Worked case 1: backlash = 0.002

This case keeps the nominal configuration except for backlash=0.002. The full input is {"backlash": 0.002, "ratio": 36.0, "rotor_inertia": 2e-05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reflected inertia kg m2	0.02592
tracking rmse rad	0.00098241
peak lost motion rad	0.001
transmitted range rad	0.198

1. Rotor inertia 2.000e-05 kg*m^2 is reflected through ratio 36.
2. Output-equivalent inertia is 0.02592000 kg*m^2.
3. The commanded output trajectory has 0.1 rad amplitude and 0.5 Hz frequency.
4. Apply a total backlash gap of 0.002000 rad with a stateful play operator.
5. The resulting RMSE is 0.000982 rad; peak error is 0.001000 rad.

Worked case 2: backlash = 0.005

This case keeps the nominal configuration except for backlash=0.005. The full input is {"backlash": 0.005, "ratio": 36.0, "rotor_inertia": 2e-05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reflected inertia kg m2	0.02592
tracking rmse rad	0.00242939
peak lost motion rad	0.0025
transmitted range rad	0.195

1. Rotor inertia 2.000e-05 kg*m^2 is reflected through ratio 36.
2. Output-equivalent inertia is 0.02592000 kg*m^2.
3. The commanded output trajectory has 0.1 rad amplitude and 0.5 Hz frequency.
4. Apply a total backlash gap of 0.005000 rad with a stateful play operator.

5. The resulting RMSE is 0.002429 rad; peak error is 0.002500 rad.

Worked case 3: backlash = 0.01

This case keeps the nominal configuration except for backlash=0.01. The full input is {"backlash": 0.01, "ratio": 36.0, "rotor_inertia": 2e-05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reflected inertia kg m2	0.02592
tracking rmse rad	0.00479646
peak lost motion rad	0.005
transmitted range rad	0.19

1. Rotor inertia 2.000e-05 kg*m^2 is reflected through ratio 36.
2. Output-equivalent inertia is 0.02592000 kg*m^2.
3. The commanded output trajectory has 0.1 rad amplitude and 0.5 Hz frequency.
4. Apply a total backlash gap of 0.010000 rad with a stateful play operator.
5. The resulting RMSE is 0.004796 rad; peak error is 0.005000 rad.

Worked case 4: backlash = 0.02

This case keeps the nominal configuration except for backlash=0.02. The full input is {"backlash": 0.02, "ratio": 36.0, "rotor_inertia": 2e-05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reflected inertia kg m2	0.02592
tracking rmse rad	0.00940506
peak lost motion rad	0.01
transmitted range rad	0.18

1. Rotor inertia 2.000e-05 kg*m^2 is reflected through ratio 36.
2. Output-equivalent inertia is 0.02592000 kg*m^2.
3. The commanded output trajectory has 0.1 rad amplitude and 0.5 Hz frequency.
4. Apply a total backlash gap of 0.020000 rad with a stateful play operator.
5. The resulting RMSE is 0.009405 rad; peak error is 0.010000 rad.

Worked case 5: backlash = 0.04

This case keeps the nominal configuration except for backlash=0.04. The full input is {"backlash": 0.04, "ratio": 36.0, "rotor_inertia": 2e-05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reflected inertia kg m2	0.02592
tracking rmse rad	0.01822161
peak lost motion rad	0.02
transmitted range rad	0.16

1. Rotor inertia 2.000e-05 kg*m^2 is reflected through ratio 36.
2. Output-equivalent inertia is 0.02592000 kg*m^2.
3. The commanded output trajectory has 0.1 rad amplitude and 0.5 Hz frequency.
4. Apply a total backlash gap of 0.040000 rad with a stateful play operator.
5. The resulting RMSE is 0.018222 rad; peak error is 0.020000 rad.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

A stateful dead zone creates hysteresis; adding independent Gaussian noise would model a different phenomenon.

The play model excludes stiffness, friction and load-dependent tooth contact.

K. Failure modes and troubleshooting

Double-applying gear ratio to an output encoder; assuming all tracking error is sensor noise; ignoring reversal direction.

L. Practical exercises and extension tasks

1. Add a load direction and compare reversal behavior.
2. Sweep gear ratio separately and show its squared inertia dependence.
3. Fit backlash from a recorded slow bidirectional experiment.

M. Connection to the Asimov research platform

Actual transmission behavior requires measured reversals and verified encoder placement; do not copy the teaching gap into a physical controller.

N. Completion record

Record your chosen case, predicted trend, measured reflected_inertia_kg_m2, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 16, Closed-form two-link inverse kinematics and branch choice. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 16 - Closed-form two-link inverse kinematics and branch choice

A. The engineering question

Inverse kinematics often has several solutions or no solution.

B. Concepts and mathematical model

A two-link planar arm makes those possibilities explicit: two elbow branches can reach the same point, while targets outside an annulus cannot be reached. Clipping an impossible cosine into [-1,1] without reporting the original violation hides reachability failure. Branch selection should preserve continuity and respect limits. This simple analytic model becomes a reference for numerical IK and for understanding why more joints do not eliminate ambiguity.

cos(q2)=(x^2+y^2-l1^2-l2^2)/(2*l1*l2);
q1=atan2(y,x)-atan2(l2*sin(q2),l1+l2*cos(q2)).

C. Real-world meaning and limits

The humanoid arm has more axes and geometry, but the same reachability, branch, limit and collision questions remain.

D. Architecture and data flow

Configuration (target_x, target_y) -> numerical model -> reachable, q1_rad, q2_rad -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

target_x and target_y are meters in a fixed planar base frame; q1 and q2 are relative joint angles.

Parameter	Nominal value	Interpretation
target_x	0.35	Defined by the explicit chapter model and parameter contract
target_y	0.1	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/16_planar_ik.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only target_x changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/16_planar_ik.py --case 0

.venv-book/bin/python expanded_lab/modules/16_planar_ik.py --case 3

.venv-book/bin/python expanded_lab/run.py planar_ik --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/16_planar_ik.py

```
"""Closed-form two-link inverse kinematics and branch choice. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    l1, l2 = 0.3, 0.2
    x, y = p["target_x"], p["target_y"]
    cosine = (x * x + y * y - l1 * l1 - l2 * l2) / (2 * l1 * l2)
    reachable = abs(cosine) <= 1
    q2 = np.arccos(np.clip(cosine, -1, 1))
    q1 = np.arctan2(y, x) - np.arctan2(l2 * np.sin(q2), l1 + l2 * np.cos(q2))
    predicted = np.array(
        [
            l1 * np.cos(q1) + l2 * np.cos(q1 + q2),
            l1 * np.sin(q1) + l2 * np.sin(q1 + q2),
        ]
    )
    residual = np.linalg.norm(predicted - [x, y])
    metrics = dict(
        reachable=float(reachable),
        q1_rad=q1,
        q2_rad=q2,
        residual_m=residual,
        cosine_raw=cosine,
    )
    derivation = [
        f"Target is ({x:g},{y:g}) m with link lengths {l1:g},{l2:g} m.",
        f"Compute the raw elbow cosine {cosine:.8f}; reachability is {reachable}.",
        f"The positive-elbow candidate is q2={q2:.8f} rad.",
        f"Compute shoulder correction to obtain q1={q1:.8f} rad.",
        f"Forward kinematics returns {predicted.tolist()} m, with residual {residual:.8f}
    m.",
    ]
    notes = [
        (
            "The target is geometrically reachable; joint limits and collisions still
    require separate checks."
            if reachable
            else "The clipped candidate is only a nearest-boundary diagnostic. It must not
    be returned as a successful IK solution."
        ),
        "Negating q2 and recomputing q1 gives the other elbow branch when the target is not
    singular.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
```

```
parser = argparse.ArgumentParser()
parser.add_argument("--case", type=int, default=0, choices=range(6))
args = parser.parse_args()
parameters = {"target_x": 0.35, "target_y": 0.1}
parameters["target_x"] = [0.05, 0.1, 0.2, 0.35, 0.48, 0.6][args.case]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Compute the reachability flag before clipping for numerical evaluation. The forward residual is an independent success metric, while branch continuity requires information from the previous pose.

I. Expected behavior and six worked cases

The controlled variable is target_x. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

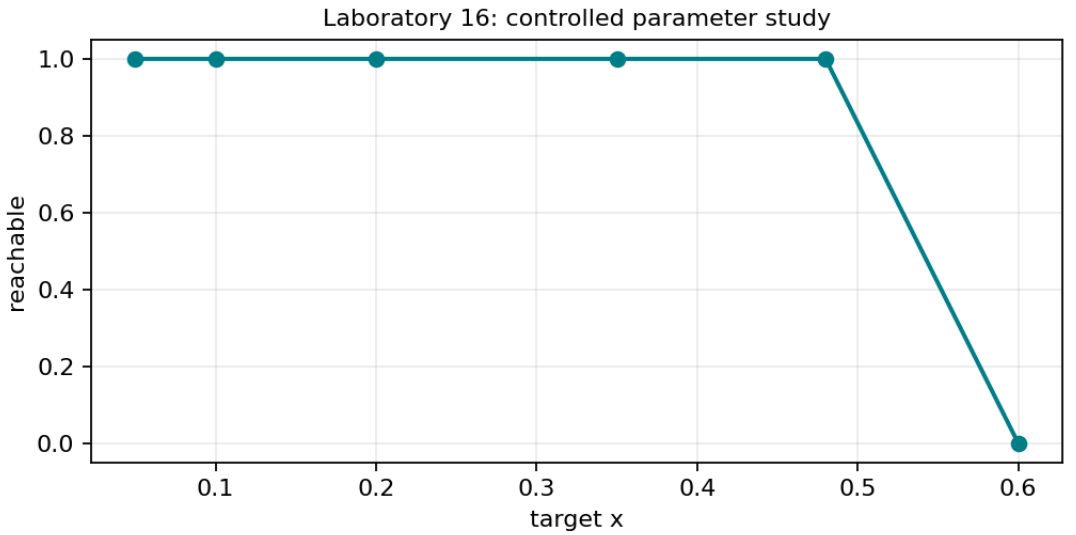


Figure L16. reachable as target_x changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	target_x	reachable
0	0.05	1
1	0.1	1
2	0.2	1
3	0.35	1
4	0.48	1
5	0.6	0

Worked case 0: target_x = 0.05

This case keeps the nominal configuration except for target_x=0.05. The full input is {"target_x": 0.05, "target_y": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reachable	1
q1 rad	0.73540395
q2 rad	2.93711245
residual m	4.3885418e-17
cosine raw	-0.97916667

1. Target is (0.05,0.1) m with link lengths 0.3,0.2 m.
2. Compute the raw elbow cosine -0.97916667; reachability is True.
3. The positive-elbow candidate is q2=2.93711245 rad.
4. Compute shoulder correction to obtain q1=0.73540395 rad.
5. Forward kinematics returns [0.04999999999999999, 0.09999999999999996] m, with residual 0.00000000 m.

Worked case 1: target_x = 0.1

This case keeps the nominal configuration except for target_x=0.1. The full input is {"target_x": 0.1, "target_y": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reachable	1
q1 rad	0.18472956
q2 rad	2.73045479
residual m	6.2063354e-17
cosine raw	-0.91666667

1. Target is (0.1,0.1) m with link lengths 0.3,0.2 m.
2. Compute the raw elbow cosine -0.91666667; reachability is True.
3. The positive-elbow candidate is q2=2.73045479 rad.
4. Compute shoulder correction to obtain q1=0.18472956 rad.
5. Forward kinematics returns [0.09999999999999995, 0.09999999999999998] m, with residual 0.00000000 m.

Worked case 2: target_x = 0.2

This case keeps the nominal configuration except for target_x=0.2. The full input is {"target_x": 0.2, "target_y": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reachable	1
q1 rad	-0.26608005
q2 rad	2.30052398
residual m	6.2063354e-17

Computed quantity	Numerical result
cosine raw	-0.66666667

1. Target is (0.2,0.1) m with link lengths 0.3,0.2 m.
2. Compute the raw elbow cosine -0.66666667; reachability is True.
3. The positive-elbow candidate is q2=2.30052398 rad.
4. Compute shoulder correction to obtain q1=-0.26608005 rad.
5. Forward kinematics returns [0.19999999999999996, 0.09999999999999998] m, with residual 0.00000000 m.

Worked case 3: target_x = 0.35

This case keeps the nominal configuration except for target_x=0.35. The full input is {"target_x": 0.35, "target_y": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reachable	1
q1 rad	-0.30325418
q2 rad	1.54996149
residual m	5.7219585e-17
cosine raw	0.02083333

1. Target is (0.35,0.1) m with link lengths 0.3,0.2 m.
2. Compute the raw elbow cosine 0.02083333; reachability is True.
3. The positive-elbow candidate is q2=1.54996149 rad.
4. Compute shoulder correction to obtain q1=-0.30325418 rad.
5. Forward kinematics returns [0.3499999999999999, 0.09999999999999999] m, with residual 0.00000000 m.

Worked case 4: target_x = 0.48

This case keeps the nominal configuration except for target_x=0.48. The full input is {"target_x": 0.48, "target_y": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reachable	1
q1 rad	0.04483964
q2 rad	0.40271584
residual m	5.7219585e-17
cosine raw	0.92

1. Target is (0.48,0.1) m with link lengths 0.3,0.2 m.
2. Compute the raw elbow cosine 0.92000000; reachability is True.
3. The positive-elbow candidate is q2=0.40271584 rad.
4. Compute shoulder correction to obtain q1=0.04483964 rad.
5. Forward kinematics returns [0.48000000000000004, 0.09999999999999999] m, with residual 0.00000000 m.

Worked case 5: target_x = 0.6

This case keeps the nominal configuration except for target_x=0.6. The full input is {"target_x": 0.6, "target_y": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reachable	0
q1 rad	0.16514868
q2 rad	0
residual m	0.10827625

Computed quantity	Numerical result
cosine raw	2

1. Target is (0.6,0.1) m with link lengths 0.3,0.2 m.
2. Compute the raw elbow cosine 2.00000000; reachability is False.
3. The positive-elbow candidate is q2=0.00000000 rad.
4. Compute shoulder correction to obtain q1=0.16514868 rad.
5. Forward kinematics returns [0.4931969619160719, 0.08219949365267865] m, with residual 0.10827625 m.

The clipped candidate is only a nearest-boundary diagnostic. It must not be returned as a successful IK solution.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The target is geometrically reachable; joint limits and collisions still require separate checks.

Negating q2 and recomputing q1 gives the other elbow branch when the target is not singular.

K. Failure modes and troubleshooting

Calling a clipped cosine a valid solution; mixing absolute and relative elbow angles; assuming endpoint reachability guarantees collision freedom.

L. Practical exercises and extension tasks

1. Compute both branches and choose the closest to a previous joint state.
2. Add asymmetric joint limits and identify rejected branches.
3. Follow a Cartesian line through a near-singular pose and inspect continuity.

M. Connection to the Asimov research platform

The humanoid arm has more axes and geometry, but the same reachability, branch, limit and collision questions remain.

N. Completion record

Record your chosen case, predicted trend, measured reachable, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 17, Singular values and damped least-squares motion. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 17 - Singular values and damped least-squares motion

A. The engineering question

Near a kinematic singularity, an inverse Jacobian can demand enormous joint changes to achieve a small endpoint motion.

B. Concepts and mathematical model

Damped least squares trades exact Cartesian tracking for bounded numerical sensitivity. Singular values reveal which directions are poorly controllable. Damping is not a safety proof: a bounded numerical update can still violate a joint limit or collision constraint. This laboratory exposes the tradeoff with a nearly stretched two-link arm.

dq = J.T*(J*J.T+lambda^2*I)^-1*error; J=U*S*V.T; directional gain=s/(s^2+lambda^2).

C. Real-world meaning and limits

Use conservative increment, velocity and collision checks around the numerical solver; hardware compliance changes the achieved endpoint.

D. Architecture and data flow

Configuration (elbow, damping) -> numerical model -> min_singular_value, joint_step_norm_rad, predicted_residual_m -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

elbow is the relative second-joint angle in radians; damping is the regularization scale in the selected Jacobian units.

Parameter	Nominal value	Interpretation
elbow	0.03	Defined by the explicit chapter model and parameter contract
damping	0.01	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/17_damped_ik.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only damping changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/17_damped_ik.py --case 0

.venv-book/bin/python expanded_lab/modules/17_damped_ik.py --case 3

.venv-book/bin/python expanded_lab/run.py damped_ik --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/17_damped_ik.py

```
"""Singular values and damped least-squares motion. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    q1 = 0.2
    q2 = p["elbow"]
    lam = p["damping"]
    J = np.array(
        [
            [-0.3 * np.sin(q1) - 0.2 * np.sin(q1 + q2), -0.2 * np.sin(q1 + q2)],
            [0.3 * np.cos(q1) + 0.2 * np.cos(q1 + q2), 0.2 * np.cos(q1 + q2)],
        ]
    )
    error = np.array([0.01, 0.0])
    singular = np.linalg.svd(J, compute_uv=False)
    dq = J.T @ np.linalg.solve(J @ J.T + lam * lam * np.eye(2), error)
    predicted = J @ dq
    metrics = dict(
        min_singular_value=singular[-1],
        joint_step_norm_rad=np.linalg.norm(dq),
        predicted_residual_m=np.linalg.norm(error - predicted),
        max_joint_step_rad=np.max(np.abs(dq)),
    )
    derivation = [
        f"Use pose q1={q1:g}, q2={q2:g} rad and Cartesian request {error.tolist()} m.",
        f"The Jacobian singular values are {singular.tolist()}.",
        f"Choose damping lambda={lam:g} and solve the regularized 2x2 system.",
        f"The joint update is {dq.tolist()} rad.",
        f"Linearized achieved displacement is {predicted.tolist()} m; residual is
    {np.linalg.norm(error-predicted):.8f} m.",
    ]
    notes = [
        "The linearized prediction is only valid for sufficiently small joint updates;
    recompute nonlinear FK after each iteration.",
        "A larger damping value can improve numerical behavior while deliberately leaving
    more task error.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"elbow": 0.03, "damping": 0.01}
    parameters["damping"] = [0.0001, 0.001, 0.005, 0.01, 0.03, 0.1][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
```

```
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Inspect singular values before choosing damping. Compare joint-step magnitude and residual together; selecting the smallest residual alone rewards unreasonable motion near singularity.

I. Expected behavior and six worked cases

The controlled variable is damping. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

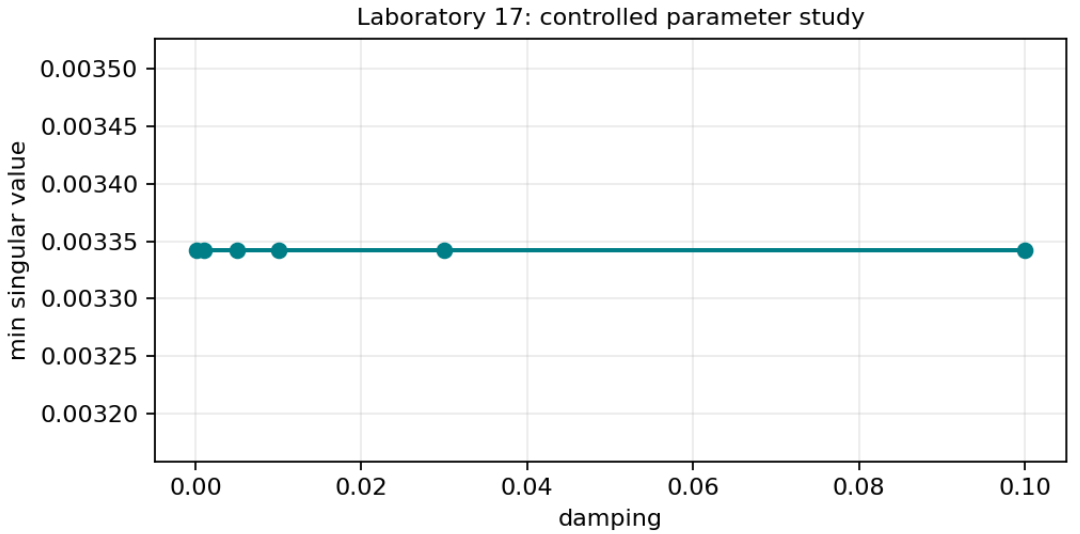


Figure L17. min singular value as damping changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	damping	min singular value
0	0.0001	0.00334239
1	0.001	0.00334239

Case	damping	min singular value
2	0.005	0.00334239
3	0.01	0.00334239
4	0.03	0.00334239
5	0.1	0.00334239

Worked case 0: damping = 0.0001

This case keeps the nominal configuration except for damping=0.0001. The full input is {"damping": 0.0001, "elbow": 0.03}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
min singular value	0.00334239
joint step norm rad	2.92070524
predicted residual m	8.7383648e-06
max joint step rad	2.71327577

1. Use pose q1=0.2, q2=0.03 rad and Cartesian request [0.01, 0.0] m.
2. The Jacobian singular values are [0.5384559707877523, 0.0033423902970501785].
3. Choose damping lambda=0.0001 and solve the regularized 2x2 system.
4. The joint update is [1.0810428867396382, -2.713275765484518] rad.
5. Linearized achieved displacement is [0.009991461844996323, -1.8598193941951011e-06] m; residual is 0.00000874 m.

Worked case 1: damping = 0.001

This case keeps the nominal configuration except for damping=0.001. The full input is {"damping": 0.001, "elbow": 0.03}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
min singular value	0.00334239
joint step norm rad	2.68314411
predicted residual m	0.00080276
max joint step rad	2.49270528

1. Use pose q1=0.2, q2=0.03 rad and Cartesian request [0.01, 0.0] m.
2. The Jacobian singular values are [0.5384559707877523, 0.0033423902970501785].
3. Choose damping lambda=0.001 and solve the regularized 2x2 system.
4. The joint update is [0.9928155294304304, -2.492705281898182] rad.
5. Linearized achieved displacement is [0.00921563121821457, -0.00017085413119924766] m; residual is 0.00080276 m.

Worked case 2: damping = 0.005

This case keeps the nominal configuration except for damping=0.005. The full input is {"damping": 0.005, "elbow": 0.03}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
min singular value	0.00334239
joint step norm rad	0.90287369
predicted residual m	0.00675314
max joint step rad	0.83975812

1. Use pose q1=0.2, q2=0.03 rad and Cartesian request [0.01, 0.0] m.
2. The Jacobian singular values are [0.5384559707877523, 0.0033423902970501785].
3. Choose damping lambda=0.005 and solve the regularized 2x2 system.
4. The joint update is [0.33164317598195225, -0.8397581249487018] rad.
5. Linearized achieved displacement is [0.0034015591968046942, -0.0014371723570492413] m; residual is 0.00675314 m.

Worked case 3: damping = 0.01

This case keeps the nominal configuration except for damping=0.01. The full input is {"damping": 0.01, "elbow": 0.03}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
min singular value	0.00334239
joint step norm rad	0.29378916
predicted residual m	0.008789
max joint step rad	0.27421959

1. Use pose q1=0.2, q2=0.03 rad and Cartesian request [0.01, 0.0] m.
2. The Jacobian singular values are [0.5384559707877523, 0.0033423902970501785].
3. Choose damping lambda=0.01 and solve the regularized 2x2 system.
4. The joint update is [0.10543096784534178, -0.2742195862584573] rad.
5. Linearized achieved displacement is [0.0014122322972766987, -0.0018699507652449 78] m; residual is 0.00878900 m.

Worked case 4: damping = 0.03

This case keeps the nominal configuration except for damping=0.03. The full input is {"damping": 0.03, "elbow": 0.03}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
min singular value	0.00334239
joint step norm rad	0.03605779
predicted residual m	0.00965107
max joint step rad	0.03474182

1. Use pose q1=0.2, q2=0.03 rad and Cartesian request [0.01, 0.0] m.
2. The Jacobian singular values are [0.5384559707877523, 0.0033423902970501785].
3. Choose damping lambda=0.03 and solve the regularized 2x2 system.
4. The joint update is [0.009652461746241459, -0.03474182456657826] rad.
5. Linearized achieved displacement is [0.0005686677258760275, -0.002047717344637136] m; residual is 0.00965107 m.

Worked case 5: damping = 0.1

This case keeps the nominal configuration except for damping=0.1. The full input is {"damping": 0.1, "elbow": 0.03}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
min singular value	0.00334239
joint step norm rad	0.00502414
predicted residual m	0.00976022
max joint step rad	0.00444793

1. Use pose q1=0.2, q2=0.03 rad and Cartesian request [0.01, 0.0] m.
2. The Jacobian singular values are [0.5384559707877523, 0.0033423902970501785].
3. Choose damping lambda=0.1 and solve the regularized 2x2 system.
4. The joint update is [-0.0023362171727600717, -0.0044479280585467615] rad.

5. Linearized achieved displacement is [0.00044856693651852995, -0.002007993356992157] m; residual is 0.00976022 m.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The linearized prediction is only valid for sufficiently small joint updates; recompute nonlinear FK after each iteration.

A larger damping value can improve numerical behavior while deliberately leaving more task error.

K. Failure modes and troubleshooting

Inverting J directly near rank loss; comparing damping values across differently scaled translation/orientation tasks without unit normalization.

L. Practical exercises and extension tasks

1. Sweep elbow angle toward zero while holding damping fixed.
2. Cap joint increments and recompute the actual nonlinear displacement.
3. Add a secondary posture task in the null space of a redundant arm.

M. Connection to the Asimov research platform

Use conservative increment, velocity and collision checks around the numerical solver; hardware compliance changes the achieved endpoint.

N. Completion record

Record your chosen case, predicted trend, measured min_singular_value, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 18, Support polygons, center of pressure and load transfer. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 18 - Support polygons, center of pressure and load transfer

A. The engineering question

A support polygon is determined by active contact geometry, not by a drawing of where the feet might be.

B. Concepts and mathematical model

Contact loads determine the center of pressure, while COM projection provides a static load-feasibility clue under restrictive assumptions. An unloaded foot should not enlarge the support region. This laboratory uses two point contact resultants to show how vertical load redistribution moves COP and how a horizontal COM shift consumes a chosen geometric margin.

COP_x=sum(Fz_i*x_i)/sum(Fz_i); signed margin=min(x-xmin,xmax-x); Fz_i must be nonnegative for unilateral contact.

C. Real-world meaning and limits

Foot contact sensing may require estimation; simulation contact truth must not be silently substituted for unavailable hardware force sensors.

D. Architecture and data flow

Configuration (left_load, mass, com_x) -> numerical model -> cop_x_m, com_margin_m, left_force_N -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

left_load is the fraction of total vertical load; mass is kilograms; com_x is the COM projection in meters.

Parameter	Nominal value	Interpretation
left_load	0.5	Defined by the explicit chapter model and parameter contract
mass	10	kilograms
com_x	0.02	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/18_support_polygon.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only left_load changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/18_support_polygon.py --case 0

.venv-book/bin/python expanded_lab/modules/18_support_polygon.py --case 3

.venv-book/bin/python expanded_lab/run.py support_polygon --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/18_support_polygon.py

```
"""Support polygons, center of pressure and load transfer. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    left = p["left_load"]
    right = 1 - left
    weight = p["mass"] * 9.81
    assert 0 <= left <= 1
    positions = np.array([-0.1, 0.1])
    forces = weight * np.array([left, right])
    cop = float(forces @ positions / forces.sum())
    com = p["com_x"]
    margin = min(com + 0.14, 0.14 - com)
    moment = weight * (com - cop)
    metrics = dict(
        cop_x_m=cop,
        com_margin_m=margin,
        left_force_N=forces[0],
        right_force_N=forces[1],
        unbalanced_pitch_moment_Nm=moment,
    )
    derivation = [
        f"Total vertical load is {weight:.6f} N.",
        f"Left fraction {left:g} gives forces {forces.tolist()} N at x=-0.1,+0.1 m.",
        f"The weighted contact location is COP x={cop:.6f} m.",
        f"COM x={com:g} m has signed margin {margin:.6f} m inside the illustrative outer
    edges.",
        f"The gravity/COP moment difference is {moment:.6f} Nm.",
    ]
    notes = [
        (
            "One contact resultant has zero load; do not treat both point contacts as
    equally active support."
            if min(left, right) == 0
            else "Both contact resultants are compressive in this static illustration."
        ),
        "The .14 m outer edges are teaching geometry; real support requires measured foot
    patches and contact state.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"left_load": 0.5, "mass": 10.0, "com_x": 0.02}
```

```
    parameters["left_load"] = [0.0, 0.1, 0.25, 0.5, 0.75, 1.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Compute COP from forces, not masses. The residual pitch moment is a useful sign check but does not model whole-body angular momentum or dynamic ground reaction.

I. Expected behavior and six worked cases

The controlled variable is left_load. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

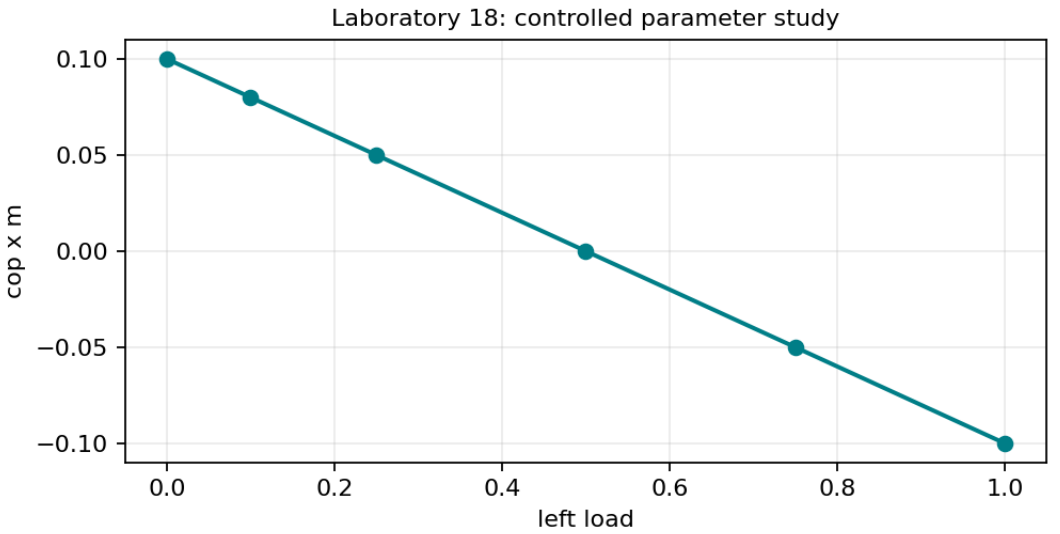


Figure L18. cop x m as left_load changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	left_load	cop x m
0	0	0.1

Case	left_load	cop x m
1	0.1	0.08
2	0.25	0.05
3	0.5	0
4	0.75	-0.05
5	1	-0.1

Worked case 0: left_load = 0

This case keeps the nominal configuration except for left_load=0. The full input is {"com_x": 0.02, "left_load": 0.0, "mass": 10.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
cop x m	0.1
com margin m	0.12
left force N	0
right force N	98.1
unbalanced pitch moment Nm	-7.848

1. Total vertical load is 98.100000 N.
2. Left fraction 0 gives forces [0.0, 98.10000000000001] N at x=-0.1,+0.1 m.
3. The weighted contact location is COP x=0.100000 m.
4. COM x=0.02 m has signed margin 0.120000 m inside the illustrative outer edges.
5. The gravity/COP moment difference is -7.848000 Nm.

Worked case 1: left_load = 0.1

This case keeps the nominal configuration except for left_load=0.1. The full input is {"com_x": 0.02, "left_load": 0.1, "mass": 10.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
cop x m	0.08
com margin m	0.12
left force N	9.81
right force N	88.29
unbalanced pitch moment Nm	-5.886

1. Total vertical load is 98.100000 N.
2. Left fraction 0.1 gives forces [9.810000000000002, 88.29] N at x=-0.1,+0.1 m.
3. The weighted contact location is COP x=0.080000 m.
4. COM x=0.02 m has signed margin 0.120000 m inside the illustrative outer edges.
5. The gravity/COP moment difference is -5.886000 Nm.

Both contact resultants are compressive in this static illustration.

Worked case 2: left_load = 0.25

This case keeps the nominal configuration except for left_load=0.25. The full input is {"com_x": 0.02, "left_load": 0.25, "mass": 10.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
cop x m	0.05
com margin m	0.12

Computed quantity	Numerical result
left force N	24.525
right force N	73.575
unbalanced pitch moment Nm	-2.943

1. Total vertical load is 98.100000 N.
2. Left fraction 0.25 gives forces [24.525000000000002, 73.575] N at x=-0.1,+0.1 m.
3. The weighted contact location is COP x=0.050000 m.
4. COM x=0.02 m has signed margin 0.120000 m inside the illustrative outer edges.
5. The gravity/COP moment difference is -2.943000 Nm.

Both contact resultants are compressive in this static illustration.

Worked case 3: left_load = 0.5

This case keeps the nominal configuration except for left_load=0.5. The full input is {"com_x": 0.02, "left_load": 0.5, "mass": 10.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
cop x m	0
com margin m	0.12
left force N	49.05
right force N	49.05
unbalanced pitch moment Nm	1.962

1. Total vertical load is 98.100000 N.
2. Left fraction 0.5 gives forces [49.050000000000004, 49.050000000000004] N at x=-0.1,+0.1 m.
3. The weighted contact location is COP x=0.000000 m.

4. COM x=0.02 m has signed margin 0.120000 m inside the illustrative outer edges.
5. The gravity/COP moment difference is 1.962000 Nm.

Both contact resultants are compressive in this static illustration.

Worked case 4: left_load = 0.75

This case keeps the nominal configuration except for left_load=0.75. The full input is {"com_x": 0.02, "left_load": 0.75, "mass": 10.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
cop x m	-0.05
com margin m	0.12
left force N	73.575
right force N	24.525
unbalanced pitch moment Nm	6.867

1. Total vertical load is 98.100000 N.
2. Left fraction 0.75 gives forces [73.575, 24.525000000000002] N at x=-0.1,+0.1 m.
3. The weighted contact location is COP x=-0.050000 m.
4. COM x=0.02 m has signed margin 0.120000 m inside the illustrative outer edges.
5. The gravity/COP moment difference is 6.867000 Nm.

Both contact resultants are compressive in this static illustration.

Worked case 5: left_load = 1

This case keeps the nominal configuration except for left_load=1. The full input is {"com_x": 0.02, "left_load": 1.0, "mass": 10.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
cop x m	-0.1
com margin m	0.12
left force N	98.1
right force N	0
unbalanced pitch moment Nm	11.772

1. Total vertical load is 98.100000 N.
2. Left fraction 1 gives forces [98.10000000000001, 0.0] N at x=-0.1,+0.1 m.
3. The weighted contact location is COP x=-0.100000 m.
4. COM x=0.02 m has signed margin 0.120000 m inside the illustrative outer edges.
5. The gravity/COP moment difference is 11.772000 Nm.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

One contact resultant has zero load; do not treat both point contacts as equally active support.

The .14 m outer edges are teaching geometry; real support requires measured foot patches and contact state.

K. Failure modes and troubleshooting

Expanding the support hull with unloaded feet; confusing COP and COM; dividing by total contact force near zero.

L. Practical exercises and extension tasks

1. Vary COM x and find the load split needed for zero pitch moment.
2. Replace point contacts with measured foot corners and compute a convex hull.

3. Simulate a lifting foot and update the active support set.

M. Connection to the Asimov research platform

Foot contact sensing may require estimation; simulation contact truth must not be silently substituted for unavailable hardware force sensors.

N. Completion record

Record your chosen case, predicted trend, measured cop_x_m, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 19, Linear inverted pendulum prediction and capture-point intuition. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 19 - Linear inverted pendulum prediction and capture-point intuition

A. The engineering question

The linear inverted pendulum model approximates horizontal COM motion at fixed height above a support plane.

B. Concepts and mathematical model

Its unstable mode explains why a small COM displacement can grow without corrective contact placement. The capture point combines position and velocity into a location associated with stopping the divergent motion under the model assumptions. It is useful intuition, but variable height, angular momentum, foot limits and impact dynamics can invalidate the simplified prediction.

omega=sqrt(g/h); x_ddot=omega^2*(x-zmp); capture point xi=x+x_dot/omega.

C. Real-world meaning and limits

Validate fixed-height and contact assumptions before using LIPM predictions in an actual humanoid controller.

D. Architecture and data flow

Configuration (height, horizon) -> numerical model -> natural_rate_per_s, final_com_x_m, final_speed_m_s -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

height is fixed COM height in meters; horizon is seconds; the model assumes a constant planar ZMP.

Parameter	Nominal value	Interpretation
height	0.6	Defined by the explicit chapter model and parameter contract
horizon	0.3	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/19_lipm.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only height changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project
```

```
.venv-book/bin/python expanded_lab/modules/19_lipm.py --case 0

.venv-book/bin/python expanded_lab/modules/19_lipm.py --case 3

.venv-book/bin/python expanded_lab/run.py lipm --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/19_lipm.py

```
"""Linear inverted pendulum prediction and capture-point intuition. Numerical teaching
    experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    h, T = p["height"], p["horizon"]
    w = np.sqrt(9.81 / h)
    x0, v0, zmp = 0.02, 0.05, 0.0
    x = zmp + (x0 - zmp) * np.cosh(w * T) + v0 / w * np.sinh(w * T)
    v = (x0 - zmp) * w * np.sinh(w * T) + v0 * np.cosh(w * T)
    capture = x0 + v0 / w
    metrics = dict(
        natural_rate_per_s=w,
        final_com_x_m=x,
        final_speed_m_s=v,
        initial_capture_point_m=capture,
        divergent_growth_factor=np.exp(w * T),
    )
    derivation = [
        f"At height {h:g} m, omega=sqrt(9.81/h)={w:.6f} per second.",
        f"Start at x0={x0:g} m and v0={v0:g} m/s with ZMP zero.",
        f"Over {T:g} s, the hyperbolic solution predicts x={x:.6f} m and v={v:.6f} m/s.",
        f"The initial capture-point coordinate is {capture:.6f} m.",
        f"The divergent mode multiplies by exp(omega*T)={np.exp(w*T):.6f}.",
    ]
    notes = [
        "A fixed support point does not recover the displaced state in this open-loop
    model.",
        "A predicted capture point outside reachable/contact-feasible support is a warning,
    not an invitation to command an impossible step.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"height": 0.6, "horizon": 0.3}
    parameters["height"] = [0.3, 0.4, 0.5, 0.6, 0.8, 1.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The exact hyperbolic solution avoids confusing integration error with the physical instability of the approximation. Compare the growth factor across heights while keeping initial state fixed.

I. Expected behavior and six worked cases

The controlled variable is height. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

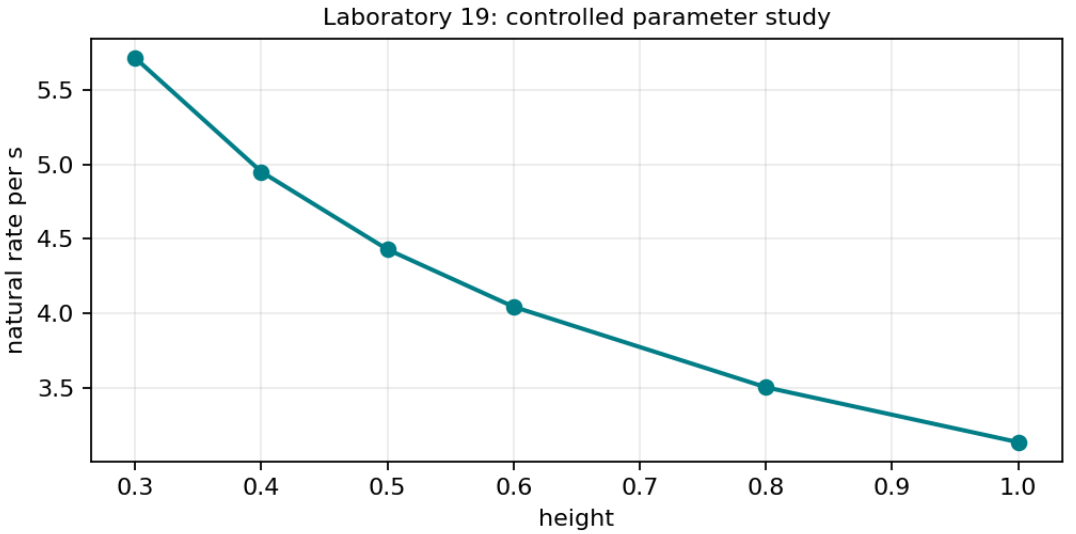


Figure L19. natural rate per s as height changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	height	natural rate per s
0	0.3	5.71839138
1	0.4	4.95227221
2	0.5	4.42944692
3	0.6	4.04351332
4	0.8	3.50178526

Case	height	natural rate per s
5	1	3.13209195

Worked case 0: height = 0.3

This case keeps the nominal configuration except for height=0.3. The full input is {"height": 0.3, "horizon": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
natural rate per s	5.71839138
final com x m	0.08091342
final speed m s	0.45111675
initial capture point m	0.02874372
divergent growth factor	5.55955136

1. At height 0.3 m, omega=sqrt(9.81/h)=5.718391 per second.
2. Start at x0=0.02 m and v0=0.05 m/s with ZMP zero.
3. Over 0.3 s, the hyperbolic solution predicts x=0.080913 m and v=0.451117 m/s.
4. The initial capture-point coordinate is 0.028744 m.
5. The divergent mode multiplies by exp(omega*T)=5.559551.

Worked case 1: height = 0.4

This case keeps the nominal configuration except for height=0.4. The full input is {"height": 0.4, "horizon": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
natural rate per s	4.95227221
final com x m	0.06760336

Computed quantity	Numerical result
final speed m s	0.32368892
initial capture point m	0.03009638
divergent growth factor	4.41797595

1. At height 0.4 m, omega=sqrt(9.81/h)=4.952272 per second.
2. Start at x0=0.02 m and v0=0.05 m/s with ZMP zero.
3. Over 0.3 s, the hyperbolic solution predicts x=0.067603 m and v=0.323689 m/s.
4. The initial capture-point coordinate is 0.030096 m.
5. The divergent mode multiplies by exp(omega*T)=4.417976.

Worked case 2: height = 0.5

This case keeps the nominal configuration except for height=0.5. The full input is {"height": 0.5, "horizon": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
natural rate per s	4.42944692
final com x m	0.06023528
final speed m s	0.25659119
initial capture point m	0.03128809
divergent growth factor	3.77663755

1. At height 0.5 m, omega=sqrt(9.81/h)=4.429447 per second.
2. Start at x0=0.02 m and v0=0.05 m/s with ZMP zero.
3. Over 0.3 s, the hyperbolic solution predicts x=0.060235 m and v=0.256591 m/s.
4. The initial capture-point coordinate is 0.031288 m.
5. The divergent mode multiplies by exp(omega*T)=3.776638.

Worked case 3: height = 0.6

This case keeps the nominal configuration except for height=0.6. The full input is {"height": 0.6, "horizon": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
natural rate per s	4.04351332
final com x m	0.05556939
final speed m s	0.21551822
initial capture point m	0.03236548
divergent growth factor	3.36374184

1. At height 0.6 m, omega=sqrt(9.81/h)=4.043513 per second.
2. Start at x0=0.02 m and v0=0.05 m/s with ZMP zero.
3. Over 0.3 s, the hyperbolic solution predicts x=0.055569 m and v=0.215518 m/s.
4. The initial capture-point coordinate is 0.032365 m.
5. The divergent mode multiplies by exp(omega*T)=3.363742.

Worked case 4: height = 0.8

This case keeps the nominal configuration except for height=0.8. The full input is {"height": 0.8, "horizon": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
natural rate per s	3.50178526
final com x m	0.0500047
final speed m s	0.16809822
initial capture point m	0.03427843
divergent growth factor	2.85918202

1. At height 0.8 m, omega=sqrt(9.81/h)=3.501785 per second.
2. Start at x0=0.02 m and v0=0.05 m/s with ZMP zero.
3. Over 0.3 s, the hyperbolic solution predicts x=0.050005 m and v=0.168098 m/s.
4. The initial capture-point coordinate is 0.034278 m.
5. The divergent mode multiplies by exp(omega*T)=2.859182.

Worked case 5: height = 1

This case keeps the nominal configuration except for height=1. The full input is {"height": 1.0, "horizon": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
natural rate per s	3.13209195
final com x m	0.04680478
final speed m s	0.14165678
initial capture point m	0.03596377
divergent growth factor	2.55902822

1. At height 1 m, omega=sqrt(9.81/h)=3.132092 per second.
2. Start at x0=0.02 m and v0=0.05 m/s with ZMP zero.
3. Over 0.3 s, the hyperbolic solution predicts x=0.046805 m and v=0.141657 m/s.
4. The initial capture-point coordinate is 0.035964 m.
5. The divergent mode multiplies by exp(omega*T)=2.559028.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

A fixed support point does not recover the displaced state in this open-loop model.

A predicted capture point outside reachable/contact-feasible support is a warning, not an invitation to command an impossible step.

K. Failure modes and troubleshooting

Treating the capture point as a complete gait planner; ignoring available foot motion and contact torque limits.

L. Practical exercises and extension tasks

1. Set initial velocity so the divergent component cancels for a chosen support point.
2. Compare exact prediction with a numerical integrator.
3. Add a bounded ZMP choice and test feasibility against support edges.

M. Connection to the Asimov research platform

Validate fixed-height and contact assumptions before using LIPM predictions in an actual humanoid controller.

N. Completion record

Record your chosen case, predicted trend, measured natural_rate_per_s, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 20, Friction cones, slip feasibility and contact margins. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 20 - Friction cones, slip feasibility and contact margins

A. The engineering question

A contact can push but usually cannot pull, and its tangential force is limited by friction and normal load.

B. Concepts and mathematical model

This simple inequality couples balance and propulsion: reducing normal load reduces available horizontal force. A friction coefficient is not a universal property independent of surfaces, wear or contamination. This laboratory evaluates a requested horizontal acceleration under a Coulomb-friction approximation and distinguishes feasibility margin from a measured slip guarantee.

F_t	<= mu*F_n; F_t_required =m*a; margin=mu*F_n-	F_t_required	.

C. Real-world meaning and limits

Real foot-ground interaction needs measurement across the intended surfaces and support states; simulation friction tuning is not certification.

D. Architecture and data flow

Configuration (mass, mu, acceleration, supported_fraction) -> numerical model -> normal_force_N, tangential_required_N, friction_capacity_N -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

mass is kilograms; mu is dimensionless assumed friction; acceleration is m/s^2; supported_fraction scales normal loading.

Parameter	Nominal value	Interpretation
mass	10	kilograms
mu	0.6	Defined by the explicit chapter model and parameter contract
acceleration	2	SI acceleration in the chapter equation
supported_fraction	1	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/20_friction_cone.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only mu changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/20_friction_cone.py --case 0

.venv-book/bin/python expanded_lab/modules/20_friction_cone.py --case 3

.venv-book/bin/python expanded_lab/run.py friction_cone --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/20_friction_cone.py

```
"""Friction cones, slip feasibility and contact margins. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    mass, mu, acc = p["mass"], p["mu"], p["acceleration"]
    normal = mass * 9.81 * p["supported_fraction"]
    required = mass * acc
    capacity = mu * normal
    margin = capacity - abs(required)
    angle = np.arctan(mu)
    utilization = abs(required) / capacity
    metrics = dict(
        normal_force_N=normal,
        tangential_required_N=required,
        friction_capacity_N=capacity,
        force_margin_N=margin,
        utilization=utilization,
        cone_half_angle_rad=angle,
    )
    derivation = [
        f"The modeled normal force is {normal:.6f} N.",
        f"Acceleration {acc:g} m/s^2 requires horizontal force {required:.6f} N.",
        f"At mu={mu:g}, Coulomb capacity is {capacity:.6f} N.",
        f"Margin is {margin:.6f} N and utilization is {utilization:.6f}.",
        f"The friction-cone half angle is atan(mu)={angle:.6f} rad.",
    ]
    notes = [
        (
            "The requested force is inside the modeled friction bound."
            if margin >= 0
            else "The requested force exceeds the modeled friction capacity; a controller
    must reduce demand or change support."
        ),
        "This static feasibility check excludes torsional friction, impact, compliance and
    changing normal force.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {
        "mass": 10.0,
        "mu": 0.6,
        "acceleration": 2.0,
```

```
    "supported_fraction": 1.0,
}
parameters["mu"] = [0.1, 0.2, 0.3, 0.4, 0.6, 0.8][args.case]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Calculate load before friction capacity and compare force magnitudes. Utilization makes conditions comparable, but remains conditional on the assumed normal force and coefficient.

I. Expected behavior and six worked cases

The controlled variable is mu. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

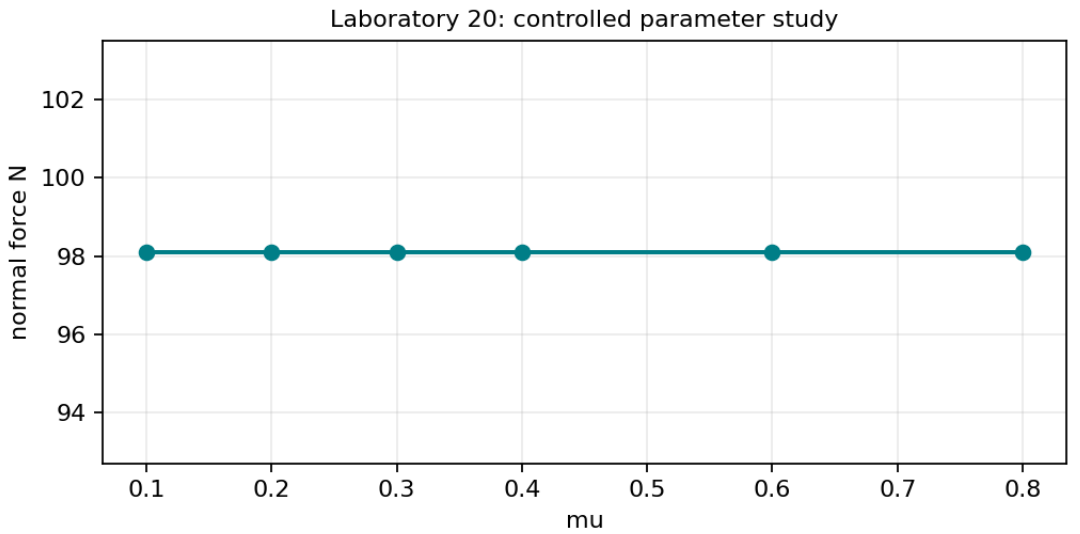


Figure L20. normal force N as mu changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	mu	normal force N
0	0.1	98.1
1	0.2	98.1
2	0.3	98.1
3	0.4	98.1
4	0.6	98.1
5	0.8	98.1

Worked case 0: mu = 0.1

This case keeps the nominal configuration except for mu=0.1. The full input is {"acceleration": 2.0, "mass": 10.0, "mu": 0.1, "supported_fraction": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
normal force N	98.1
tangential required N	20
friction capacity N	9.81
force margin N	-10.19
utilization	2.03873598
cone half angle rad	0.09966865

1. The modeled normal force is 98.100000 N.
2. Acceleration 2 m/s^2 requires horizontal force 20.000000 N.
3. At mu=0.1, Coulomb capacity is 9.810000 N.
4. Margin is -10.190000 N and utilization is 2.038736.
5. The friction-cone half angle is atan(mu)=0.099669 rad.

Worked case 1: mu = 0.2

This case keeps the nominal configuration except for mu=0.2. The full input is {"acceleration": 2.0, "mass": 10.0, "mu": 0.2, "supported_fraction": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
normal force N	98.1
tangential required N	20
friction capacity N	19.62
force margin N	-0.38
utilization	1.01936799
cone half angle rad	0.19739556

1. The modeled normal force is 98.100000 N.
2. Acceleration 2 m/s^2 requires horizontal force 20.000000 N.
3. At mu=0.2, Coulomb capacity is 19.620000 N.
4. Margin is -0.380000 N and utilization is 1.019368.
5. The friction-cone half angle is atan(mu)=0.197396 rad.

Worked case 2: mu = 0.3

This case keeps the nominal configuration except for mu=0.3. The full input is {"acceleration": 2.0, "mass": 10.0, "mu": 0.3, "supported_fraction": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
normal force N	98.1
tangential required N	20
friction capacity N	29.43

Computed quantity	Numerical result
force margin N	9.43
utilization	0.67957866
cone half angle rad	0.29145679

1. The modeled normal force is 98.100000 N.
2. Acceleration 2 m/s^2 requires horizontal force 20.000000 N.
3. At mu=0.3, Coulomb capacity is 29.430000 N.
4. Margin is 9.430000 N and utilization is 0.679579.
5. The friction-cone half angle is atan(mu)=0.291457 rad.

The requested force is inside the modeled friction bound.

Worked case 3: mu = 0.4

This case keeps the nominal configuration except for mu=0.4. The full input is {"acceleration": 2.0, "mass": 10.0, "mu": 0.4, "supported_fraction": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
normal force N	98.1
tangential required N	20
friction capacity N	39.24
force margin N	19.24
utilization	0.509684
cone half angle rad	0.38050638

1. The modeled normal force is 98.100000 N.
2. Acceleration 2 m/s^2 requires horizontal force 20.000000 N.
3. At mu=0.4, Coulomb capacity is 39.240000 N.
4. Margin is 19.240000 N and utilization is 0.509684.

5. The friction-cone half angle is atan(mu)=0.380506 rad.

The requested force is inside the modeled friction bound.

Worked case 4: mu = 0.6

This case keeps the nominal configuration except for mu=0.6. The full input is {"acceleration": 2.0, "mass": 10.0, "mu": 0.6, "supported_fraction": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
normal force N	98.1
tangential required N	20
friction capacity N	58.86
force margin N	38.86
utilization	0.33978933
cone half angle rad	0.5404195

1. The modeled normal force is 98.100000 N.
2. Acceleration 2 m/s^2 requires horizontal force 20.000000 N.
3. At mu=0.6, Coulomb capacity is 58.860000 N.
4. Margin is 38.860000 N and utilization is 0.339789.
5. The friction-cone half angle is atan(mu)=0.540420 rad.

The requested force is inside the modeled friction bound.

Worked case 5: mu = 0.8

This case keeps the nominal configuration except for mu=0.8. The full input is {"acceleration": 2.0, "mass": 10.0, "mu": 0.8, "supported_fraction": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
normal force N	98.1
tangential required N	20
friction capacity N	78.48
force margin N	58.48
utilization	0.254842
cone half angle rad	0.67474094

1. The modeled normal force is 98.100000 N.
2. Acceleration 2 m/s^2 requires horizontal force 20.000000 N.
3. At mu=0.8, Coulomb capacity is 78.480000 N.
4. Margin is 58.480000 N and utilization is 0.254842.
5. The friction-cone half angle is atan(mu)=0.674741 rad.

The requested force is inside the modeled friction bound.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The requested force exceeds the modeled friction capacity; a controller must reduce demand or change support.

This static feasibility check excludes torsional friction, impact, compliance and changing normal force.

K. Failure modes and troubleshooting

Using visual foot contact as proof of load; making friction arbitrarily large to rescue a bad gait; ignoring unloading during stepping.

L. Practical exercises and extension tasks

1. Reduce normal support fraction while keeping horizontal demand fixed.

2. Include a lateral acceleration and use the vector norm.
3. Estimate a conservative friction interval and evaluate worst-case feasibility.

M. Connection to the Asimov research platform

Real foot-ground interaction needs measurement across the intended surfaces and support states; simulation friction tuning is not certification.

N. Completion record

Record your chosen case, predicted trend, measured normal_force_N, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 21, Second-order joint dynamics and PD tuning. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Part VIII - Feedback, trajectories and actuator constraints

A command generator, feedback law and actuator are different components. A smooth trajectory can exceed torque authority; a stable ideal controller can become unstable with delay; a motor that meets a peak load can overheat over repeated cycles. The laboratories below keep these boundaries explicit and measure the tradeoffs. Treat every numerical value as an exercise input until it has been replaced by an identified or verified physical parameter.

Laboratory 21 - Second-order joint dynamics and PD tuning

A. The engineering question

Feedback gains set a closed-loop dynamic behavior, not merely a preference for stronger correction.

B. Concepts and mathematical model

For a single inertia with viscous damping, proportional feedback adds stiffness and derivative feedback adds damping. Natural frequency and damping ratio connect these gains to response speed and overshoot. The same numerical gains behave differently when inertia changes. This experiment integrates an ideal one-axis joint with a small target step and reports error and overshoot rather than judging the response by appearance.

I*q_ddot + (b+Kd)*q_dot + Kp*q = Kp*q_target; omega_n=sqrt(Kp/I); zeta=(b+Kd)/(2*sqrt(Kp*I)).

C. Real-world meaning and limits

Physical gain changes require identified inertia, sensor noise, delay and a guarded test envelope; theoretical tuning is only a starting point.

D. Architecture and data flow

Configuration (inertia, kp, kd) -> numerical model -> natural_frequency_rad_s, damping_ratio, overshoot_fraction -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

inertia is output-axis kg*m^2; kp is Nm/rad; kd is Nm*s/rad; physical viscous damping is fixed at 0.1 Nm*s/rad.

Parameter	Nominal value	Interpretation
inertia	0.08	kg*m^2
kp	20	Nm/rad
kd	2	Nm*s/rad

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/21_pd_control.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only kd changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/21_pd_control.py --case 0

.venv-book/bin/python expanded_lab/modules/21_pd_control.py --case 3

.venv-book/bin/python expanded_lab/run.py pd_control --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/21_pd_control.py

```
"""Second-order joint dynamics and PD tuning. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    I, kp, kd = p["inertia"], p["kp"], p["kd"]
    b = 0.1
    dt = 0.001
    target = 0.1
    q = 0.0
    v = 0.0
    positions = []
    for _ in range(3000):
        torque = kp * (target - q) - kd * v
        acceleration = (torque - b * v) / I
        v += dt * acceleration
        q += dt * v
        positions.append(q)
    positions = np.array(positions)
    wn = np.sqrt(kp / I)
    zeta = (b + kd) / (2 * np.sqrt(kp * I))
    overshoot = max(0.0, positions.max() - target) / target
    metrics = dict(
        natural_frequency_rad_s=wn,
        damping_ratio=zeta,
        overshoot_fraction=overshoot,
        final_error_rad=target - q,
        trajectory_rmse_rad=np.sqrt(np.mean((positions - target) ** 2)),
    )
    derivation = [
        f"Joint inertia is {I:g} kg*m^2; Kp={kp:g} Nm/rad; Kd={kd:g} Nm*s/rad.",
        f"The implied natural frequency is {wn:.6f} rad/s and damping ratio is {zeta:.6f}.",
        "Apply a 0.1 rad target and integrate semi-implicitly at 1 ms for three seconds.",
        f"Peak position is {positions.max():.6f} rad, producing overshoot fraction
    {overshoot:.6f}.",
        f'Final tracking error is {target-q:.8f} rad; trajectory RMSE is
    {metrics["trajectory_rmse_rad"]:.6f} rad.',
    ]
    notes = [
        (
            "This model is underdamped."
            if zeta < 1
            else "This model is critically damped or overdamped."
        ),
        "There is no torque saturation, delay, gravity or contact in this
    identification-friendly reference.",
    ]
    return metrics, derivation, notes
```

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"inertia": 0.08, "kp": 20.0, "kd": 2.0}
    parameters["kd"] = [0.1, 0.5, 1.0, 2.0, 4.0, 8.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The velocity update precedes the position update, giving semi-implicit integration. Compare simulated overshoot with the damping classification; then vary inertia to see why gains are model dependent.

I. Expected behavior and six worked cases

The controlled variable is kd. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

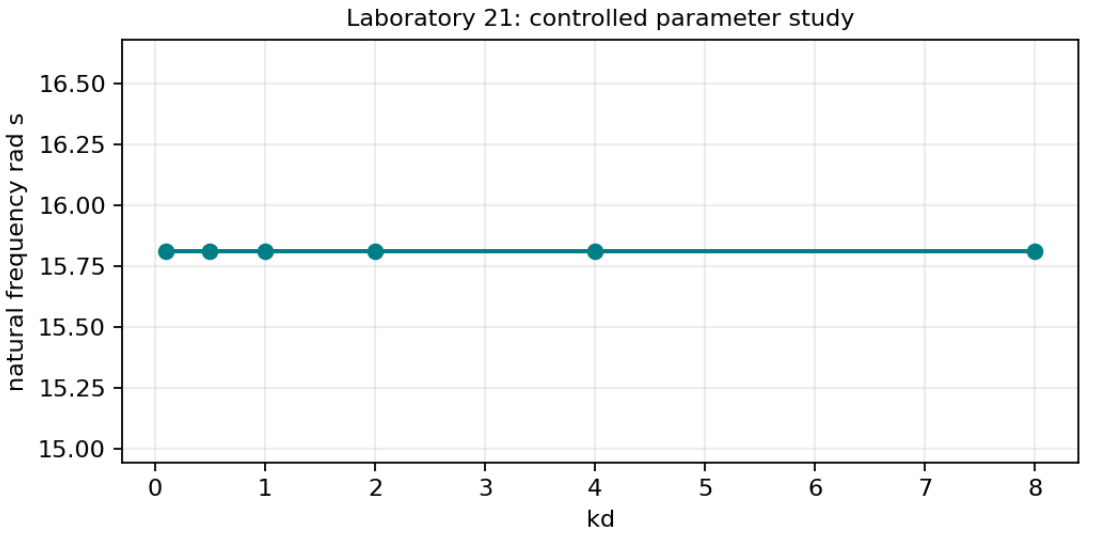


Figure L21. natural frequency rad s as kd changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	kd	natural frequency rad s
0	0.1	15.8113883
1	0.5	15.8113883
2	1	15.8113883
3	2	15.8113883
4	4	15.8113883
5	8	15.8113883

Worked case 0: kd = 0.1

This case keeps the nominal configuration except for kd=0.1. The full input is {"inertia": 0.08, "kd": 0.1, "kp": 20.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
natural frequency rad s	15.8113883
damping ratio	0.07905694
overshoot fraction	0.77936206
final error rad	-0.00232973
trajectory rmse rad	0.0260547

1. Joint inertia is 0.08 kg*m^2; Kp=20 Nm/rad; Kd=0.1 Nm*s/rad.
2. The implied natural frequency is 15.811388 rad/s and damping ratio is 0.079057.
3. Apply a 0.1 rad target and integrate semi-implicitly at 1 ms for three seconds.
4. Peak position is 0.177936 rad, producing overshoot fraction 0.779362.
5. Final tracking error is -0.00232973 rad; trajectory RMSE is 0.026055 rad.

Worked case 1: kd = 0.5

This case keeps the nominal configuration except for kd=0.5. The full input is {"inertia": 0.08, "kd": 0.5, "kp": 20.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
natural frequency rad s	15.8113883
damping ratio	0.23717082
overshoot fraction	0.46371639
final error rad	-4.7227297e-07
trajectory rmse rad	0.01637283

1. Joint inertia is 0.08 kg*m^2; Kp=20 Nm/rad; Kd=0.5 Nm*s/rad.
2. The implied natural frequency is 15.811388 rad/s and damping ratio is 0.237171.
3. Apply a 0.1 rad target and integrate semi-implicitly at 1 ms for three seconds.
4. Peak position is 0.146372 rad, producing overshoot fraction 0.463716.
5. Final tracking error is -0.00000047 rad; trajectory RMSE is 0.016373 rad.

Worked case 2: kd = 1

This case keeps the nominal configuration except for kd=1. The full input is {"inertia": 0.08, "kd": 1.0, "kp": 20.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
natural frequency rad s	15.8113883
damping ratio	0.43481318
overshoot fraction	0.21797678
final error rad	-2.8162611e-12

Computed quantity	Numerical result
trajectory rmse rad	0.01444713

1. Joint inertia is 0.08 kg*m^2; Kp=20 Nm/rad; Kd=1 Nm*s/rad.
2. The implied natural frequency is 15.811388 rad/s and damping ratio is 0.434813.
3. Apply a 0.1 rad target and integrate semi-implicitly at 1 ms for three seconds.
4. Peak position is 0.121798 rad, producing overshoot fraction 0.217977.
5. Final tracking error is -0.00000000 rad; trajectory RMSE is 0.014447 rad.

Worked case 3: kd = 2

This case keeps the nominal configuration except for kd=2. The full input is {"inertia": 0.08, "kd": 2.0, "kp": 20.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
natural frequency rad s	15.8113883
damping ratio	0.83009789
overshoot fraction	0.00840482
final error rad	-2.2204460e-16
trajectory rmse rad	0.01530782

1. Joint inertia is 0.08 kg*m^2; Kp=20 Nm/rad; Kd=2 Nm*s/rad.
2. The implied natural frequency is 15.811388 rad/s and damping ratio is 0.830098.
3. Apply a 0.1 rad target and integrate semi-implicitly at 1 ms for three seconds.
4. Peak position is 0.100840 rad, producing overshoot fraction 0.008405.
5. Final tracking error is -0.00000000 rad; trajectory RMSE is 0.015308 rad.

Worked case 4: kd = 4

This case keeps the nominal configuration except for kd=4. The full input is {"inertia": 0.08, "kd": 4.0, "kp": 20.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
natural frequency rad s	15.8113883
damping ratio	1.6206673
overshoot fraction	0
final error rad	9.1892507e-09
trajectory rmse rad	0.01923597

1. Joint inertia is 0.08 kg*m^2; Kp=20 Nm/rad; Kd=4 Nm*s/rad.
2. The implied natural frequency is 15.811388 rad/s and damping ratio is 1.620667.
3. Apply a 0.1 rad target and integrate semi-implicitly at 1 ms for three seconds.
4. Peak position is 0.100000 rad, producing overshoot fraction 0.000000.
5. Final tracking error is 0.00000001 rad; trajectory RMSE is 0.019236 rad.

This model is critically damped or overdamped.

Worked case 5: kd = 8

This case keeps the nominal configuration except for kd=8. The full input is {"inertia": 0.08, "kd": 8.0, "kp": 20.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
natural frequency rad s	15.8113883
damping ratio	3.20180613
overshoot fraction	0
final error rad	0.00005188

Computed quantity	Numerical result
trajectory rmse rad	0.02621631

1. Joint inertia is 0.08 kg*m^2; Kp=20 Nm/rad; Kd=8 Nm*s/rad.
2. The implied natural frequency is 15.811388 rad/s and damping ratio is 3.201806.
3. Apply a 0.1 rad target and integrate semi-implicitly at 1 ms for three seconds.
4. Peak position is 0.099948 rad, producing overshoot fraction 0.000000.
5. Final tracking error is 0.00005188 rad; trajectory RMSE is 0.026216 rad.

This model is critically damped or overdamped.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

This model is underdamped.

There is no torque saturation, delay, gravity or contact in this identification-friendly reference.

K. Failure modes and troubleshooting

Increasing Kp to compensate for torque saturation; differentiating a noisy setpoint instead of defining derivative action carefully.

L. Practical exercises and extension tasks

1. Double inertia and recover the original natural frequency and damping ratio.
2. Add a torque cap and compare the linear prediction to the actual response.
3. Measure settling within a predeclared error band.

M. Connection to the Asimov research platform

Physical gain changes require identified inertia, sensor noise, delay and a guarded test envelope; theoretical tuning is only a starting point.

N. Completion record

Record your chosen case, predicted trend, measured natural_frequency_rad_s, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 22, Integral action, saturation and anti-windup. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 22 - Integral action, saturation and anti-windup

A. The engineering question

Integral action accumulates persistent error and can compensate an unknown constant disturbance.

B. Concepts and mathematical model

When an actuator saturates, continued integration stores a demand that the plant cannot realize. After the target changes, this stored integral can keep the output saturated in the wrong direction. Conditional integration is a simple anti-windup method: integrate only when unsaturated or when the error would move the demand back toward the permitted range. The experiment explicitly compares the same plant with and without this logic.

u_raw=Kp*e+Ki*integral-Kd*v; u=clip(u_raw); integrate if unsaturated or e*(u_raw-u)<=0.

C. Real-world meaning and limits

Integral action can command sustained physical torque; thermal and fault behavior must be part of its design.

D. Architecture and data flow

Configuration (torque_cap) -> numerical model -> plain_final_error_rad, protected_final_error_rad, plain_integral -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

torque_cap is the symmetric teaching actuator limit in Nm; plant inertia is 0.2 kg*m^2 and timestep is 2 ms.

Parameter	Nominal value	Interpretation
torque_cap	1	Nm

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/22_antiwindup.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only torque_cap changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/22_antiwindup.py --case 0

.venv-book/bin/python expanded_lab/modules/22_antiwindup.py --case 3
```

```
.venv-book/bin/python expanded_lab/run.py antiwindup --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/22_antiwindup.py

```
"""Integral action, saturation and anti-windup. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def simulate(cap, anti):
    q = v = integral = 0.0
    dt = 0.002
    history = []
    saturated = 0
    for k in range(2000):
        target = 0.5 if k < 500 else 0.0
        error = target - q
        raw = 12 * error + 10 * integral - 2 * v
        u = np.clip(raw, -cap, cap)
        if not anti or abs(raw) <= cap or error * (raw - u) <= 0:
            integral += dt * error
        v += dt * (u - 0.2 * v) / 0.2
        q += dt * v
        history.append(q)
        saturated += abs(raw) > cap
    return np.array(history), integral, saturated / 2000


def experiment(p):
    cap = p["torque_cap"]
    plain, ip, sp = simulate(cap, False)
    protected, ia, sa = simulate(cap, True)
    metrics = dict(
        plain_final_error_rad=abs(plain[-1]),
        protected_final_error_rad=abs(protected[-1]),
        plain_integral=ip,
        protected_integral=ia,
        plain_saturation_fraction=sp,
        protected_saturation_fraction=sa,
    )
    derivation = [
        f"Use symmetric actuator limit +/-{cap:g} Nm.",
        "Request 0.5 rad for one second, then return the target to zero.",
        f"Without anti-windup, saturation fraction is {sp:.6f} and final integral is
    {ip:.6f}.",
        f"Conditional integration gives saturation fraction {sa:.6f} and integral
    {ia:.6f}.",
        f"Final absolute errors are {abs(plain[-1]):.6f} and {abs(protected[-1]):.6f} rad
    respectively.",
    ]
    notes = [
        "Anti-windup improves consistency with actuator authority but does not guarantee
    every transient metric improves.",
        "Integral state must also be handled explicitly when changing modes, resetting
```

```
    faults or switching controllers.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"torque_cap": 1.0}
    parameters["torque_cap"] = [0.1, 0.25, 0.5, 1.0, 2.0, 5.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The paired simulations share target, gains and plant. Only the integral update rule changes, making the comparison causal. Inspect the return-to-zero segment rather than only the initial rise.

I. Expected behavior and six worked cases

The controlled variable is torque_cap. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

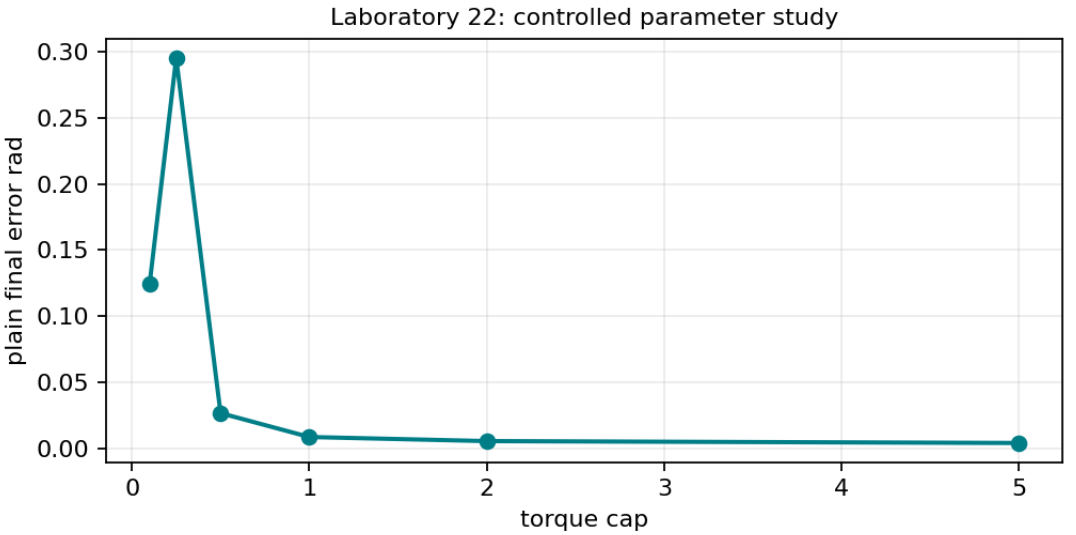


Figure L22. plain final error rad as torque_cap changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	torque_cap	plain final error rad
0	0.1	0.12410345
1	0.25	0.29555289
2	0.5	0.02649489
3	1	0.00841963
4	2	0.00530972
5	5	0.00387863

Worked case 0: torque_cap = 0.1

This case keeps the nominal configuration except for torque_cap=0.1. The full input is {"torque_cap": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
plain final error rad	0.12410345

Computed quantity	Numerical result
protected final error rad	0.00146551
plain integral	-0.03609486
protected integral	-0.00169588
plain saturation fraction	0.98
protected saturation fraction	0.77

1. Use symmetric actuator limit +/-0.1 Nm.
2. Request 0.5 rad for one second, then return the target to zero.
3. Without anti-windup, saturation fraction is 0.980000 and final integral is -0.036095.
4. Conditional integration gives saturation fraction 0.770000 and integral -0.001696.
5. Final absolute errors are 0.124103 and 0.001466 rad respectively.

Worked case 1: torque_cap = 0.25

This case keeps the nominal configuration except for torque_cap=0.25. The full input is {"torque_cap": 0.25}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
plain final error rad	0.29555289
protected final error rad	0.00011126
plain integral	-0.01474504
protected integral	0.00015196
plain saturation fraction	0.989
protected saturation fraction	0.669

1. Use symmetric actuator limit +/-0.25 Nm.

2. Request 0.5 rad for one second, then return the target to zero.
3. Without anti-windup, saturation fraction is 0.989000 and final integral is -0.014745.
4. Conditional integration gives saturation fraction 0.669000 and integral 0.000152.
5. Final absolute errors are 0.295553 and 0.000111 rad respectively.

Worked case 2: torque_cap = 0.5

This case keeps the nominal configuration except for torque_cap=0.5. The full input is {"torque_cap": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
plain final error rad	0.02649489
protected final error rad	0.00052309
plain integral	-0.02626605
protected integral	-0.00052365
plain saturation fraction	0.645
protected saturation fraction	0.345

1. Use symmetric actuator limit +/-0.5 Nm.
2. Request 0.5 rad for one second, then return the target to zero.
3. Without anti-windup, saturation fraction is 0.645000 and final integral is -0.026266.
4. Conditional integration gives saturation fraction 0.345000 and integral -0.000524.
5. Final absolute errors are 0.026495 and 0.000523 rad respectively.

Worked case 3: torque_cap = 1

This case keeps the nominal configuration except for torque_cap=1. The full input is {"torque_cap": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
plain final error rad	0.00841963
protected final error rad	0.00132785
plain integral	-0.00841603
protected integral	-0.00132748
plain saturation fraction	0.177
protected saturation fraction	0.147

1. Use symmetric actuator limit +/-1 Nm.
2. Request 0.5 rad for one second, then return the target to zero.
3. Without anti-windup, saturation fraction is 0.177000 and final integral is -0.008416.
4. Conditional integration gives saturation fraction 0.147000 and integral -0.001327.
5. Final absolute errors are 0.008420 and 0.001328 rad respectively.

Worked case 4: torque_cap = 2

This case keeps the nominal configuration except for torque_cap=2. The full input is {"torque_cap": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
plain final error rad	0.00530972
protected final error rad	0.00208745
plain integral	-0.00530725
protected integral	-0.00208652
plain saturation fraction	0.084
protected saturation fraction	0.073

1. Use symmetric actuator limit +/-2 Nm.
2. Request 0.5 rad for one second, then return the target to zero.
3. Without anti-windup, saturation fraction is 0.084000 and final integral is -0.005307.
4. Conditional integration gives saturation fraction 0.073000 and integral -0.002087.
5. Final absolute errors are 0.005310 and 0.002087 rad respectively.

Worked case 5: torque_cap = 5

This case keeps the nominal configuration except for torque_cap=5. The full input is {"torque_cap": 5.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
plain final error rad	0.00387863
protected final error rad	0.00350293
plain integral	-0.00387674
protected integral	-0.00350123
plain saturation fraction	0.0105
protected saturation fraction	0.0095

1. Use symmetric actuator limit +/-5 Nm.
2. Request 0.5 rad for one second, then return the target to zero.
3. Without anti-windup, saturation fraction is 0.010500 and final integral is -0.003877.
4. Conditional integration gives saturation fraction 0.009500 and integral -0.003501.
5. Final absolute errors are 0.003879 and 0.003503 rad respectively.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Anti-windup improves consistency with actuator authority but does not guarantee every transient metric improves.

Integral state must also be handled explicitly when changing modes, resetting faults or switching controllers.

K. Failure modes and troubleshooting

Clamping the output while leaving integral unbounded; resetting integral unexpectedly during a loaded transition; claiming one example proves all PID tuning.

L. Practical exercises and extension tasks

1. Add back-calculation anti-windup and compare with conditional integration.
2. Apply a constant load and examine steady-state error.
3. Define a bumpless transfer from PD to PID.

M. Connection to the Asimov research platform

Integral action can command sustained physical torque; thermal and fault behavior must be part of its design.

N. Completion record

Record your chosen case, predicted trend, measured plain_final_error_rad, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 23, Minimum-jerk trajectories and duration constraints. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 23 - Minimum-jerk trajectories and duration constraints

A. The engineering question

A position step implies unbounded ideal velocity and acceleration.

B. Concepts and mathematical model

A fifth-order minimum-jerk interpolation has zero endpoint velocity and acceleration, making it a useful smooth reference for a point-to-point move. Smoothness does not establish feasibility: shortening the duration increases velocity, acceleration and jerk by different powers. This laboratory computes those quantities analytically and compares them with finite sampled values so a trajectory can be rejected before being sent to a controller.

s=10*r^3-15*r^4+6*r^5; q=q0+delta*s; q_dot=delta*s_prime/T; q_ddot=delta*s_second/T^2.

C. Real-world meaning and limits

Physical trajectories need torque, collision, cable and thermal feasibility in addition to smooth kinematic references.

D. Architecture and data flow

Configuration (duration, excursion) -> numerical model -> peak_speed_rad_s, peak_acceleration_rad_s2, peak_jerk_rad_s3 -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

duration is seconds; excursion is output-joint radians. The interpolation starts from zero relative displacement.

Parameter	Nominal value	Interpretation
duration	2	seconds
excursion	0.4	radians

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/23_minimum_jerk.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only duration changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/23_minimum_jerk.py --case 0

.venv-book/bin/python expanded_lab/modules/23_minimum_jerk.py --case 3

.venv-book/bin/python expanded_lab/run.py minimum_jerk --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/23_minimum_jerk.py

```
"""Minimum-jerk trajectories and duration constraints. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    T, delta = p["duration"], p["excursion"]
    r = np.linspace(0, 1, 1001)
    s = 10 * r**3 - 15 * r**4 + 6 * r**5
    ds = 30 * r**2 - 60 * r**3 + 30 * r**4
    dds = 60 * r - 180 * r**2 + 120 * r**3
    ddds = 60 - 360 * r + 360 * r**2
    q = delta * s
    v = delta * ds / T
    a = delta * dds / T**2
    j = delta * ddds / T**3
    assert np.isclose(q[0], 0) and np.isclose(q[-1], delta)
    assert np.allclose(v[[0, -1]], 0) and np.allclose(a[[0, -1]], 0)
    metrics = dict(
        peak_speed_rad_s=np.max(abs(v)),
        peak_acceleration_rad_s2=np.max(abs(a)),
        peak_jerk_rad_s3=np.max(abs(j)),
        final_position_rad=q[-1],
        speed_limit_margin_rad_s=0.5 - np.max(abs(v)),
    )
    derivation = [
        f"Move through delta={delta:g} rad over duration T={T:g} s.",
        "Evaluate the polynomial and its first three derivatives at 1001 normalized
    positions.",
        f"Peak speed is {np.max(abs(v)):.6f} rad/s.",
        f"Peak acceleration is {np.max(abs(a)):.6f} rad/s^2; peak jerk is
    {np.max(abs(j)):.6f} rad/s^3.",
        f'Against an illustrative 0.5 rad/s speed envelope, margin is
    {metrics["speed_limit_margin_rad_s"]:.6f} rad/s.',
    ]
    notes = [
        "The speed envelope is a teaching assumption and must not be substituted for a
    released actuator limit.",
        "Zero endpoint velocity and acceleration do not ensure the controller can supply the
     required intermediate torque.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"duration": 2.0, "excursion": 0.4}
```

```
    parameters["duration"] = [0.25, 0.5, 1.0, 2.0, 3.0, 5.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Use normalized time to separate trajectory shape from duration. The endpoint assertions check boundary conditions, while derivative scaling exposes why a fast smooth move can still be infeasible.

I. Expected behavior and six worked cases

The controlled variable is duration. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

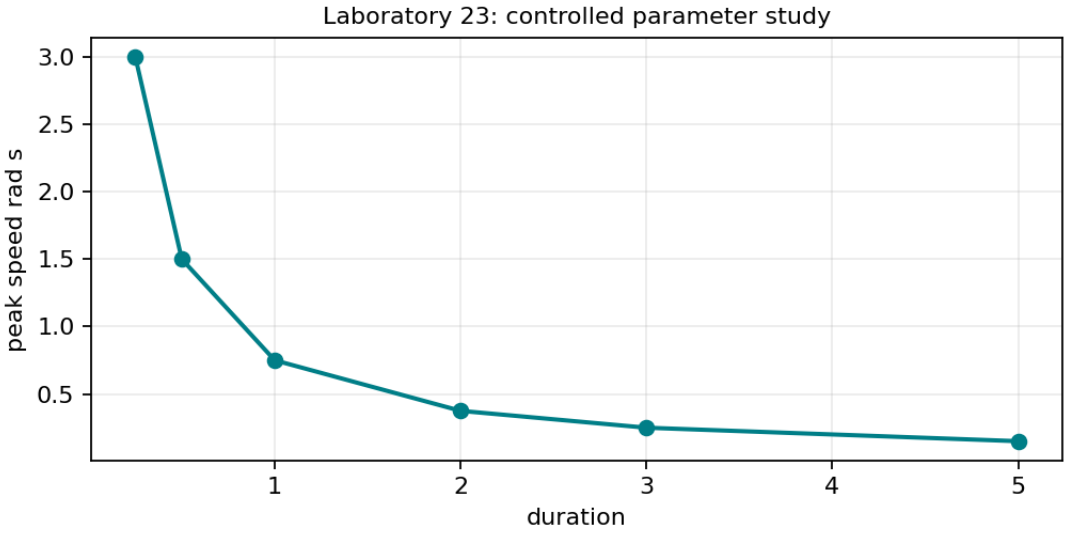


Figure L23. peak speed rad s as duration changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	duration	peak speed rad s
0	0.25	3

Case	duration	peak speed rad s
1	0.5	1.5
2	1	0.75
3	2	0.375
4	3	0.25
5	5	0.15

Worked case 0: duration = 0.25

This case keeps the nominal configuration except for duration=0.25. The full input is {"duration": 0.25, "excursion": 0.4}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
peak speed rad s	3
peak acceleration rad s2	36.95034701
peak jerk rad s3	1536
final position rad	0.4
speed limit margin rad s	-2.5

1. Move through delta=0.4 rad over duration T=0.25 s.
2. Evaluate the polynomial and its first three derivatives at 1001 normalized positions.
3. Peak speed is 3.000000 rad/s.
4. Peak acceleration is 36.950347 rad/s^2; peak jerk is 1536.000000 rad/s^3.
5. Against an illustrative 0.5 rad/s speed envelope, margin is -2.500000 rad/s.

Worked case 1: duration = 0.5

This case keeps the nominal configuration except for duration=0.5. The full input is {"duration": 0.5, "excursion": 0.4}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
peak speed rad s	1.5
peak acceleration rad s2	9.23758675
peak jerk rad s3	192
final position rad	0.4
speed limit margin rad s	-1

1. Move through delta=0.4 rad over duration T=0.5 s.
2. Evaluate the polynomial and its first three derivatives at 1001 normalized positions.
3. Peak speed is 1.500000 rad/s.
4. Peak acceleration is 9.237587 rad/s^2; peak jerk is 192.000000 rad/s^3.
5. Against an illustrative 0.5 rad/s speed envelope, margin is -1.000000 rad/s.

Worked case 2: duration = 1

This case keeps the nominal configuration except for duration=1. The full input is {"duration": 1.0, "excursion": 0.4}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
peak speed rad s	0.75
peak acceleration rad s2	2.30939669
peak jerk rad s3	24
final position rad	0.4

Computed quantity	Numerical result
speed limit margin rad s	-0.25

1. Move through delta=0.4 rad over duration T=1 s.
2. Evaluate the polynomial and its first three derivatives at 1001 normalized positions.
3. Peak speed is 0.750000 rad/s.
4. Peak acceleration is 2.309397 rad/s^2; peak jerk is 24.000000 rad/s^3.
5. Against an illustrative 0.5 rad/s speed envelope, margin is -0.250000 rad/s.

Worked case 3: duration = 2

This case keeps the nominal configuration except for duration=2. The full input is {"duration": 2.0, "excursion": 0.4}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
peak speed rad s	0.375
peak acceleration rad s2	0.57734917
peak jerk rad s3	3
final position rad	0.4
speed limit margin rad s	0.125

1. Move through delta=0.4 rad over duration T=2 s.
2. Evaluate the polynomial and its first three derivatives at 1001 normalized positions.
3. Peak speed is 0.375000 rad/s.
4. Peak acceleration is 0.577349 rad/s^2; peak jerk is 3.000000 rad/s^3.
5. Against an illustrative 0.5 rad/s speed envelope, margin is 0.125000 rad/s.

Worked case 4: duration = 3

This case keeps the nominal configuration except for duration=3. The full input is {"duration": 3.0, "excursion": 0.4}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
peak speed rad s	0.25
peak acceleration rad s2	0.25659963
peak jerk rad s3	0.88888889
final position rad	0.4
speed limit margin rad s	0.25

1. Move through delta=0.4 rad over duration T=3 s.
2. Evaluate the polynomial and its first three derivatives at 1001 normalized positions.
3. Peak speed is 0.250000 rad/s.
4. Peak acceleration is 0.256600 rad/s^2; peak jerk is 0.888889 rad/s^3.
5. Against an illustrative 0.5 rad/s speed envelope, margin is 0.250000 rad/s.

Worked case 5: duration = 5

This case keeps the nominal configuration except for duration=5. The full input is {"duration": 5.0, "excursion": 0.4}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
peak speed rad s	0.15
peak acceleration rad s2	0.09237587
peak jerk rad s3	0.192
final position rad	0.4

Computed quantity	Numerical result
speed limit margin rad s	0.35

1. Move through delta=0.4 rad over duration T=5 s.
2. Evaluate the polynomial and its first three derivatives at 1001 normalized positions.
3. Peak speed is 0.150000 rad/s.
4. Peak acceleration is 0.092376 rad/s^2; peak jerk is 0.192000 rad/s^3.
5. Against an illustrative 0.5 rad/s speed envelope, margin is 0.350000 rad/s.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The speed envelope is a teaching assumption and must not be substituted for a released actuator limit.

Zero endpoint velocity and acceleration do not ensure the controller can supply the required intermediate torque.

K. Failure modes and troubleshooting

Checking position limits but not derivatives; differentiating noisy numerical samples when analytic derivatives are available; ignoring start velocity.

L. Practical exercises and extension tasks

1. Derive a minimum duration from a chosen speed limit.
2. Check an acceleration limit and choose the stricter duration.
3. Generalize to nonzero initial velocity using a boundary-value polynomial.

M. Connection to the Asimov research platform

Physical trajectories need torque, collision, cable and thermal feasibility in addition to smooth kinematic references.

N. Completion record

Record your chosen case, predicted trend, measured peak_speed_rad_s, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 24, Position envelopes, slew limits and stopping distance. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 24 - Position envelopes, slew limits and stopping distance

A. The engineering question

A position limit alone does not protect a joint already moving toward a stop.

B. Concepts and mathematical model

The controller also needs enough distance to decelerate within its available authority. A slew limiter bounds requested changes but can introduce tracking lag, while an emergency stop requires a separate physically reviewed outcome. This experiment applies a velocity-limited target and calculates an ideal stopping-distance screen including reaction delay. It is a planning calculation, not a certified stopping model.

q_next=q_prev+clip(q_request-q_prev,+/-vmax*dt);
d_stop=v*delay+v^2/(2*a_brake).

C. Real-world meaning and limits

Physical limits require independent sensing, conservative travel margins and a stop strategy compatible with the support rig.

D. Architecture and data flow

Configuration (speed, dt, delay, deceleration) -> numerical model -> final_target_rad, ideal_stopping_distance_rad, remaining_travel_margin_rad -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

speed is rad/s; dt and delay are seconds; deceleration is a positive assumed rad/s^2 magnitude.

Parameter	Nominal value	Interpretation
speed	0.4	angular speed in the chapter equation
dt	0.01	seconds
delay	0.05	seconds
deceleration	2	rad/s^2

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/24_rate_limits.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only speed changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/24_rate_limits.py --case 0

.venv-book/bin/python expanded_lab/modules/24_rate_limits.py --case 3

.venv-book/bin/python expanded_lab/run.py rate_limits --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/24_rate_limits.py

```
"""Position envelopes, slew limits and stopping distance. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    vmax, dt, delay = p["speed"], p["dt"], p["delay"]
    brake = p["deceleration"]
    q = 0.0
    request = 0.5
    trace = []
    for _ in range(500):
        q += np.clip(request - q, -vmax * dt, vmax * dt)
        trace.append(q)
    stopping = vmax * delay + vmax * vmax / (2 * brake)
    margin = 0.12 - stopping
    metrics = dict(
        final_target_rad=q,
        ideal_stopping_distance_rad=stopping,
        remaining_travel_margin_rad=margin,
        per_update_step_rad=vmax * dt,
        time_to_target_s=request / vmax,
    )
    derivation = [
        f"At vmax={vmax:g} rad/s and dt={dt:g} s, each target increment is capped at
    {vmax*dt:.6f} rad.",
        f"Reaction delay contributes {vmax*delay:.6f} rad of travel.",
        f"Ideal constant deceleration {brake:g} rad/s^2 contributes
    {vmax*vmax/(2*brake):.6f} rad.",
        f"Total screened stopping distance is {stopping:.6f} rad.",
        f"With 0.12 rad assumed remaining travel, margin is {margin:.6f} rad.",
    ]
    notes = [
        (
            "The ideal calculation leaves positive travel margin."
            if margin >= 0
            else "The ideal calculation already exceeds available travel; reduce speed or
    increase clearance before considering uncertainty."
        ),
        "Load, control delay, brake authority and power-loss behavior must be measured for a
     physical stopping claim.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
```

```
parameters = {"speed": 0.4, "dt": 0.01, "delay": 0.05, "deceleration": 2.0}
parameters["speed"] = [0.1, 0.2, 0.4, 0.6, 0.8, 1.0][args.case]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Calculate reaction travel and braking travel separately. The target slew limiter governs requests, whereas actual velocity and stopping must be monitored from feedback.

I. Expected behavior and six worked cases

The controlled variable is speed. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

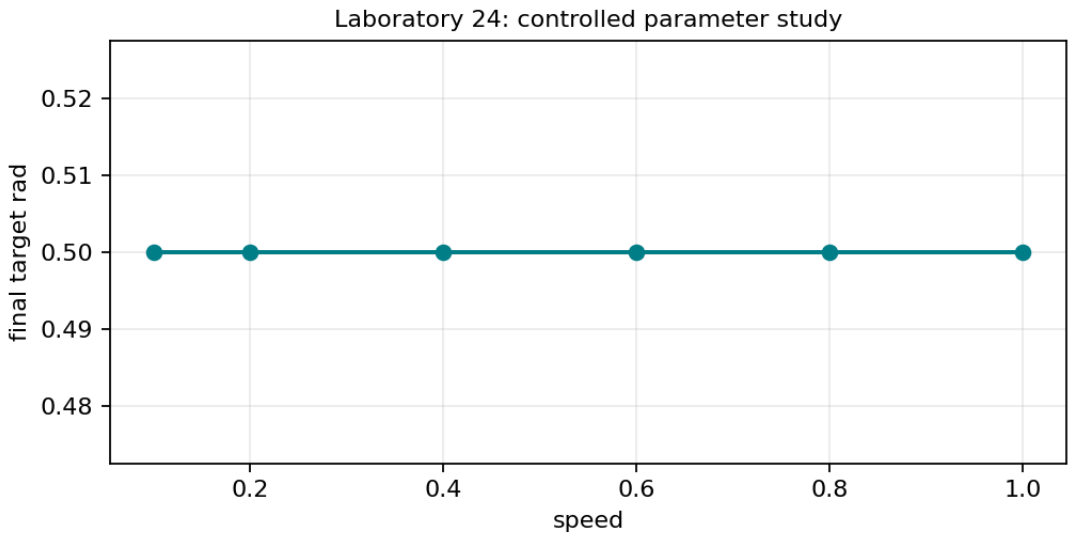


Figure L24. final target rad as speed changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	speed	final target rad
0	0.1	0.5
1	0.2	0.5
2	0.4	0.5
3	0.6	0.5
4	0.8	0.5
5	1	0.5

Worked case 0: speed = 0.1

This case keeps the nominal configuration except for speed=0.1. The full input is {"deceleration": 2.0, "delay": 0.05, "dt": 0.01, "speed": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final target rad	0.5
ideal stopping distance rad	0.0075
remaining travel margin rad	0.1125
per update step rad	0.001
time to target s	5

1. At vmax=0.1 rad/s and dt=0.01 s, each target increment is capped at 0.001000 rad.
2. Reaction delay contributes 0.005000 rad of travel.
3. Ideal constant deceleration 2 rad/s^2 contributes 0.002500 rad.
4. Total screened stopping distance is 0.007500 rad.
5. With 0.12 rad assumed remaining travel, margin is 0.112500 rad.

Worked case 1: speed = 0.2

This case keeps the nominal configuration except for speed=0.2. The full input is {"deceleration": 2.0, "delay": 0.05, "dt": 0.01, "speed": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final target rad	0.5
ideal stopping distance rad	0.02
remaining travel margin rad	0.1
per update step rad	0.002
time to target s	2.5

1. At vmax=0.2 rad/s and dt=0.01 s, each target increment is capped at 0.002000 rad.
2. Reaction delay contributes 0.010000 rad of travel.
3. Ideal constant deceleration 2 rad/s^2 contributes 0.010000 rad.
4. Total screened stopping distance is 0.020000 rad.
5. With 0.12 rad assumed remaining travel, margin is 0.100000 rad.

Worked case 2: speed = 0.4

This case keeps the nominal configuration except for speed=0.4. The full input is {"deceleration": 2.0, "delay": 0.05, "dt": 0.01, "speed": 0.4}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final target rad	0.5
ideal stopping distance rad	0.06
remaining travel margin rad	0.06
per update step rad	0.004

Computed quantity	Numerical result
time to target s	1.25

1. At vmax=0.4 rad/s and dt=0.01 s, each target increment is capped at 0.004000 rad.
2. Reaction delay contributes 0.020000 rad of travel.
3. Ideal constant deceleration 2 rad/s^2 contributes 0.040000 rad.
4. Total screened stopping distance is 0.060000 rad.
5. With 0.12 rad assumed remaining travel, margin is 0.060000 rad.

Worked case 3: speed = 0.6

This case keeps the nominal configuration except for speed=0.6. The full input is {"deceleration": 2.0, "delay": 0.05, "dt": 0.01, "speed": 0.6}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final target rad	0.5
ideal stopping distance rad	0.12
remaining travel margin rad	0
per update step rad	0.006
time to target s	0.83333333

1. At vmax=0.6 rad/s and dt=0.01 s, each target increment is capped at 0.006000 rad.
2. Reaction delay contributes 0.030000 rad of travel.
3. Ideal constant deceleration 2 rad/s^2 contributes 0.090000 rad.
4. Total screened stopping distance is 0.120000 rad.
5. With 0.12 rad assumed remaining travel, margin is 0.000000 rad.

Worked case 4: speed = 0.8

This case keeps the nominal configuration except for speed=0.8. The full input is {"deceleration": 2.0, "delay": 0.05, "dt": 0.01, "speed": 0.8}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final target rad	0.5
ideal stopping distance rad	0.2
remaining travel margin rad	-0.08
per update step rad	0.008
time to target s	0.625

1. At vmax=0.8 rad/s and dt=0.01 s, each target increment is capped at 0.008000 rad.
2. Reaction delay contributes 0.040000 rad of travel.
3. Ideal constant deceleration 2 rad/s^2 contributes 0.160000 rad.
4. Total screened stopping distance is 0.200000 rad.
5. With 0.12 rad assumed remaining travel, margin is -0.080000 rad.

The ideal calculation already exceeds available travel; reduce speed or increase clearance before considering uncertainty.

Worked case 5: speed = 1

This case keeps the nominal configuration except for speed=1. The full input is {"deceleration": 2.0, "delay": 0.05, "dt": 0.01, "speed": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final target rad	0.5
ideal stopping distance rad	0.3

Computed quantity	Numerical result
remaining travel margin rad	-0.18
per update step rad	0.01
time to target s	0.5

1. At vmax=1 rad/s and dt=0.01 s, each target increment is capped at 0.010000 rad.
2. Reaction delay contributes 0.050000 rad of travel.
3. Ideal constant deceleration 2 rad/s^2 contributes 0.250000 rad.
4. Total screened stopping distance is 0.300000 rad.
5. With 0.12 rad assumed remaining travel, margin is -0.180000 rad.

The ideal calculation already exceeds available travel; reduce speed or increase clearance before considering uncertainty.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The ideal calculation leaves positive travel margin.

Load, control delay, brake authority and power-loss behavior must be measured for a physical stopping claim.

K. Failure modes and troubleshooting

Calling a setpoint limiter a measured speed limiter; assuming torque remains available after power isolation; ignoring gravity-assisted motion.

L. Practical exercises and extension tasks

1. Propagate uncertainty in delay and deceleration to a conservative margin.
2. Add acceleration-limited setpoint generation.
3. Compare command motion and measured motion under saturation.

M. Connection to the Asimov research platform

Physical limits require independent sensing, conservative travel margins and a stop strategy compatible with the support rig.

N. Completion record

Record your chosen case, predicted trend, measured final_target_rad, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 25, Gravity compensation and model-mismatch torque. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 25 - Gravity compensation and model-mismatch torque

A. The engineering question

Gravity compensation predicts load torque from configuration and feeds it forward so feedback need not create all supporting effort through error.

B. Concepts and mathematical model

A biased mass or lever estimate creates a pose-dependent residual. This is different from a constant encoder offset or viscous friction. The experiment synthesizes static measurements over a range of angles and estimates the gravity coefficient from a cosine basis. It then quantifies the residual if the controller uses an incorrect payload mass.

gravity torque = A*cos(q); A=m*g*l; A_hat=(cos(q).T*y)/(cos(q).T*cos(q)).

C. Real-world meaning and limits

Actual gravity compensation requires verified model conventions and low-energy measurements with load support.

D. Architecture and data flow

Configuration (model_mass) -> numerical model -> feedforward_rmse_Nm, peak_residual_Nm, identified_mass_kg -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

model_mass is the controller payload estimate in kilograms; true teaching mass is 2 kg at a 0.25 m lever.

Parameter	Nominal value	Interpretation
model_mass	1.8	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/25_gravity_identification.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only model_mass changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/25_gravity_identification.py --case 0

.venv-book/bin/python expanded_lab/modules/25_gravity_identification.py --case 3

.venv-book/bin/python expanded_lab/run.py gravity_identification --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/25_gravity_identification.py

```
"""Gravity compensation and model-mismatch torque. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    true_mass = 2.0
    model_mass = p["model_mass"]
    length = 0.25
    angles = np.linspace(-1.0, 1.0, 41)
    basis = np.cos(angles)
    true = true_mass * 9.81 * length * basis
    assumed = model_mass * 9.81 * length * basis
    rng = np.random.default_rng(3)
    measured = true + rng.normal(0, 0.03, len(angles))
    coefficient = float(basis @ measured / (basis @ basis))
    fitted = coefficient * basis
    metrics = dict(
        feedforward_rmse_Nm=np.sqrt(np.mean((true - assumed) ** 2)),
        peak_residual_Nm=np.max(abs(true - assumed)),
        identified_mass_kg=coefficient / (9.81 * length),
        fit_residual_rmse_Nm=np.sqrt(np.mean((measured - fitted) ** 2)),
    )
    derivation = [
        f"The true static gravity coefficient is {true_mass*9.81*length:.6f} Nm.",
        f"The assumed model uses mass {model_mass:g} kg and coefficient
    {model_mass*9.81*length:.6f} Nm.",
        "Generate static samples over -1 to +1 rad with 0.03 Nm standard-deviation noise.",
        f"Project measurements onto cos(q), obtaining A_hat={coefficient:.6f} Nm.",
        f'The inferred mass is {metrics["identified_mass_kg"]:.6f} kg; feedforward mismatch
    RMSE is {metrics["feedforward_rmse_Nm"]:.6f} Nm.',
    ]
    notes = [
        "The cosine model assumes a known angle convention, lever and negligible
    dynamic/friction torque during the measurement.",
        "A good gravity fit does not identify inertia, backlash or transient motor limits.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"model_mass": 1.8}
    parameters["model_mass"] = [1.0, 1.5, 1.8, 2.0, 2.2, 2.5][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
```

```
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The identification uses independent synthetic measurements rather than fitting the controller prediction. Report fit residual and feedforward-model residual separately; they answer different questions.

I. Expected behavior and six worked cases

The controlled variable is model_mass. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

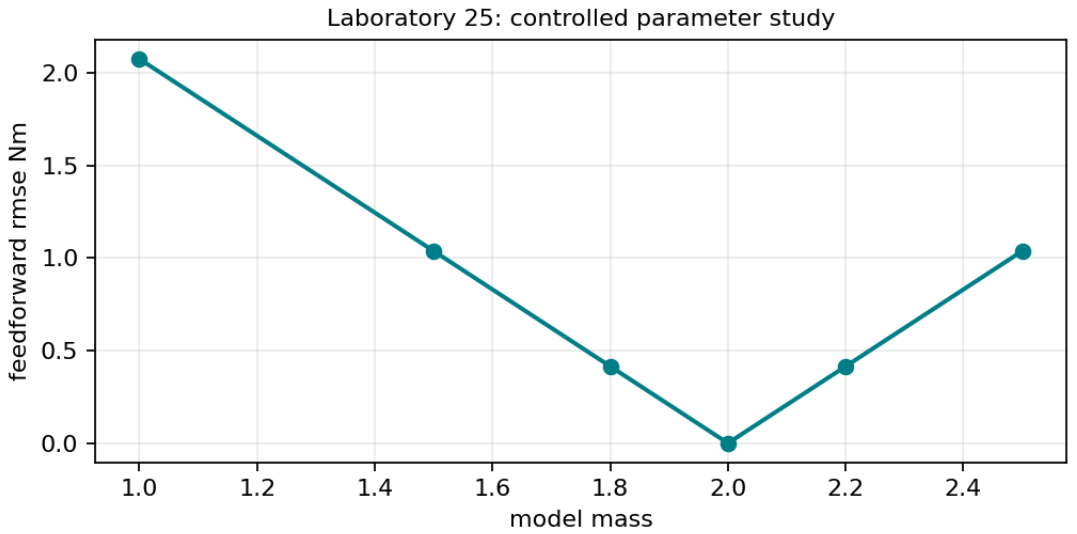


Figure L25. feedforward rmse Nm as model_mass changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	model_mass	feedforward rmse Nm
0	1	2.07598012
1	1.5	1.03799006
2	1.8	0.41519602

Case	model_mass	feedforward rmse Nm
3	2	0
4	2.2	0.41519602
5	2.5	1.03799006

Worked case 0: model_mass = 1

This case keeps the nominal configuration except for model_mass=1. The full input is {"model_mass": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
feedforward rmse Nm	2.07598012
peak residual Nm	2.4525
identified mass kg	2.00010562
fit residual rmse Nm	0.03442849

1. The true static gravity coefficient is 4.905000 Nm.
2. The assumed model uses mass 1 kg and coefficient 2.452500 Nm.
3. Generate static samples over -1 to +1 rad with 0.03 Nm standard-deviation noise.
4. Project measurements onto cos(q), obtaining A_hat=4.905259 Nm.
5. The inferred mass is 2.000106 kg; feedforward mismatch RMSE is 2.075980 Nm.

Worked case 1: model_mass = 1.5

This case keeps the nominal configuration except for model_mass=1.5. The full input is {"model_mass": 1.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
feedforward rmse Nm	1.03799006
peak residual Nm	1.22625
identified mass kg	2.00010562
fit residual rmse Nm	0.03442849

1. The true static gravity coefficient is 4.905000 Nm.
2. The assumed model uses mass 1.5 kg and coefficient 3.678750 Nm.
3. Generate static samples over -1 to +1 rad with 0.03 Nm standard-deviation noise.
4. Project measurements onto cos(q), obtaining A_hat=4.905259 Nm.
5. The inferred mass is 2.000106 kg; feedforward mismatch RMSE is 1.037990 Nm.

Worked case 2: model_mass = 1.8

This case keeps the nominal configuration except for model_mass=1.8. The full input is {"model_mass": 1.8}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
feedforward rmse Nm	0.41519602
peak residual Nm	0.4905
identified mass kg	2.00010562
fit residual rmse Nm	0.03442849

1. The true static gravity coefficient is 4.905000 Nm.
2. The assumed model uses mass 1.8 kg and coefficient 4.414500 Nm.
3. Generate static samples over -1 to +1 rad with 0.03 Nm standard-deviation noise.
4. Project measurements onto cos(q), obtaining A_hat=4.905259 Nm.

5. The inferred mass is 2.000106 kg; feedforward mismatch RMSE is 0.415196 Nm.

Worked case 3: model_mass = 2

This case keeps the nominal configuration except for model_mass=2. The full input is {"model_mass": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
feedforward rmse Nm	0
peak residual Nm	0
identified mass kg	2.00010562
fit residual rmse Nm	0.03442849

1. The true static gravity coefficient is 4.905000 Nm.
2. The assumed model uses mass 2 kg and coefficient 4.905000 Nm.
3. Generate static samples over -1 to +1 rad with 0.03 Nm standard-deviation noise.
4. Project measurements onto cos(q), obtaining A_hat=4.905259 Nm.
5. The inferred mass is 2.000106 kg; feedforward mismatch RMSE is 0.000000 Nm.

Worked case 4: model_mass = 2.2

This case keeps the nominal configuration except for model_mass=2.2. The full input is {"model_mass": 2.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
feedforward rmse Nm	0.41519602
peak residual Nm	0.4905
identified mass kg	2.00010562
fit residual rmse Nm	0.03442849

1. The true static gravity coefficient is 4.905000 Nm.
2. The assumed model uses mass 2.2 kg and coefficient 5.395500 Nm.
3. Generate static samples over -1 to +1 rad with 0.03 Nm standard-deviation noise.
4. Project measurements onto cos(q), obtaining A_hat=4.905259 Nm.
5. The inferred mass is 2.000106 kg; feedforward mismatch RMSE is 0.415196 Nm.

Worked case 5: model_mass = 2.5

This case keeps the nominal configuration except for model_mass=2.5. The full input is {"model_mass": 2.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
feedforward rmse Nm	1.03799006
peak residual Nm	1.22625
identified mass kg	2.00010562
fit residual rmse Nm	0.03442849

1. The true static gravity coefficient is 4.905000 Nm.
2. The assumed model uses mass 2.5 kg and coefficient 6.131250 Nm.
3. Generate static samples over -1 to +1 rad with 0.03 Nm standard-deviation noise.
4. Project measurements onto cos(q), obtaining A_hat=4.905259 Nm.
5. The inferred mass is 2.000106 kg; feedforward mismatch RMSE is 1.037990 Nm.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The cosine model assumes a known angle convention, lever and negligible dynamic/friction torque during the measurement.

A good gravity fit does not identify inertia, backlash or transient motor limits.

K. Failure modes and troubleshooting

Using moving samples as static gravity data; absorbing encoder-zero error into an incorrect mass estimate; ignoring multi-link coupling.

L. Practical exercises and extension tasks

1. Add a constant torque bias and fit both cosine and offset terms.
2. Introduce a small angle-zero error and inspect residual structure.
3. Compare fit and validation angle ranges.

M. Connection to the Asimov research platform

Actual gravity compensation requires verified model conventions and low-energy measurements with load support.

N. Completion record

Record your chosen case, predicted trend, measured feedforward_rmse_Nm, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 26, Torque-speed envelopes and trajectory feasibility. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 26 - Torque-speed envelopes and trajectory feasibility

A. The engineering question

An actuator does not provide its maximum torque at every speed.

B. Concepts and mathematical model

Electrical voltage, current limits, losses and thermal constraints shape a feasible envelope. A simple linear torque-speed model is useful for screening and for understanding why a trajectory can become infeasible as speed rises. It is not a substitute for a vendor curve. Here the torque demand is fixed while speed varies, making reserve and mechanical power easy to compare.

tau_available=tau_stall*max(0,1-|omega|/omega_no_load); reserve=tau_available-|tau_required|.

C. Real-world meaning and limits

Use measured or manufacturer-supplied actuator curves, firmware current limits and duty-cycle constraints for actual motors.

D. Architecture and data flow

Configuration (speed, stall_torque, no_load_speed, required_torque) -> numerical model -> available_torque_Nm, reserve_Nm, required_mechanical_power_W -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

speed is output rad/s; stall_torque and required_torque are Nm; no_load_speed is the assumed output speed at negligible torque.

Parameter	Nominal value	Interpretation
speed	2	angular speed in the chapter equation

Parameter	Nominal value	Interpretation
stall_torque	10	Defined by the explicit chapter model and parameter contract
no_load_speed	8	Defined by the explicit chapter model and parameter contract
required_torque	4	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/26_motor_envelope.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only speed changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/26_motor_envelope.py --case 0

.venv-book/bin/python expanded_lab/modules/26_motor_envelope.py --case 3

.venv-book/bin/python expanded_lab/run.py motor_envelope --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/26_motor_envelope.py

```
"""Torque-speed envelopes and trajectory feasibility. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    omega = p["speed"]
    stall = p["stall_torque"]
    free = p["no_load_speed"]
    required = p["required_torque"]
    available = stall * max(0.0, 1 - abs(omega) / free)
    reserve = available - abs(required)
    metrics = dict(
        available_torque_Nm=available,
        reserve_Nm=reserve,
        required_mechanical_power_W=required * omega,
        envelope_power_W=available * abs(omega),
        feasible=float(reserve >= 0),
    )
    derivation = [
        f"The teaching envelope has zero-speed torque {stall:g} Nm and no-load speed
    {free:g} rad/s.",
        f"At speed {omega:g} rad/s the linear speed factor is
    {max(0,1-abs(omega)/free):.6f}.",
        f"Available torque is {available:.6f} Nm while demand is {required:g} Nm.",
        f"Reserve is {reserve:.6f} Nm; feasibility flag is {reserve>=0}.",
        f"Demanded mechanical power is {required*omega:.6f} W.",
    ]
    notes = [
        "The illustrative envelope is instantaneous and excludes continuous thermal derating
     and regenerative quadrants.",
        "A positive reserve is only one constraint; position, load path, current and thermal
     checks remain separate.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {
        "speed": 2.0,
        "stall_torque": 10.0,
        "no_load_speed": 8.0,
        "required_torque": 4.0,
    }
    parameters["speed"] = [0.0, 1.0, 2.0, 4.0, 6.0, 8.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
```

```
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Separate the model of available authority from the load demand. The feasibility flag is derived from the reserve rather than from whether the numerical simulator keeps running.

I. Expected behavior and six worked cases

The controlled variable is speed. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

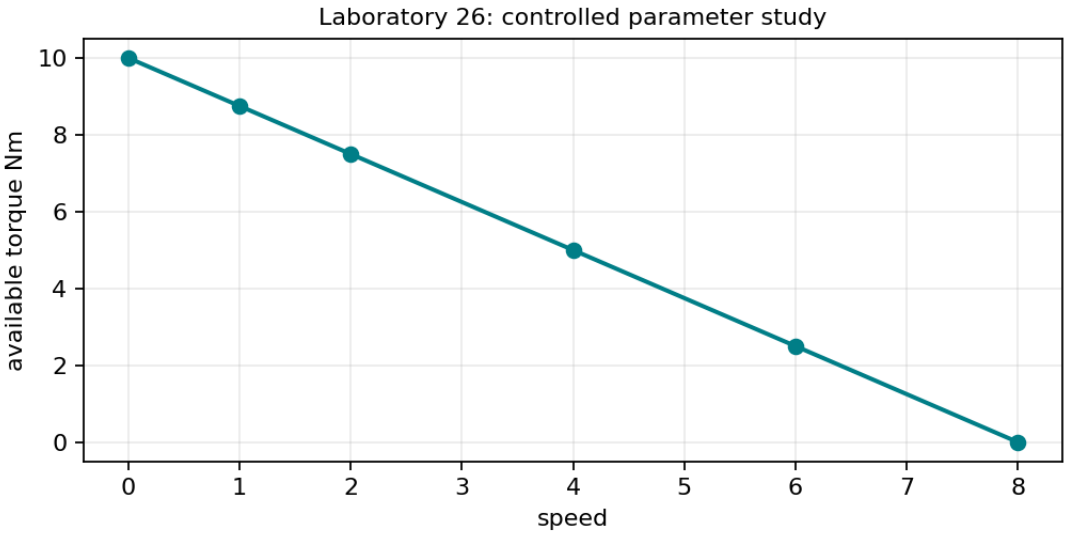


Figure L26. available torque Nm as speed changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	speed	available torque Nm
0	0	10
1	1	8.75

Case	speed	available torque Nm
2	2	7.5
3	4	5
4	6	2.5
5	8	0

Worked case 0: speed = 0

This case keeps the nominal configuration except for speed=0. The full input is {"no_load_speed": 8.0, "required_torque": 4.0, "speed": 0.0, "stall_torque": 10.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
available torque Nm	10
reserve Nm	6
required mechanical power W	0
envelope power W	0
feasible	1

1. The teaching envelope has zero-speed torque 10 Nm and no-load speed 8 rad/s.
2. At speed 0 rad/s the linear speed factor is 1.000000.
3. Available torque is 10.000000 Nm while demand is 4 Nm.
4. Reserve is 6.000000 Nm; feasibility flag is True.
5. Demanded mechanical power is 0.000000 W.

Worked case 1: speed = 1

This case keeps the nominal configuration except for speed=1. The full input is {"no_load_speed": 8.0, "required_torque": 4.0, "speed": 1.0, "stall_torque": 10.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
available torque Nm	8.75
reserve Nm	4.75
required mechanical power W	4
envelope power W	8.75
feasible	1

1. The teaching envelope has zero-speed torque 10 Nm and no-load speed 8 rad/s.
2. At speed 1 rad/s the linear speed factor is 0.875000.
3. Available torque is 8.750000 Nm while demand is 4 Nm.
4. Reserve is 4.750000 Nm; feasibility flag is True.
5. Demanded mechanical power is 4.000000 W.

Worked case 2: speed = 2

This case keeps the nominal configuration except for speed=2. The full input is {"no_load_speed": 8.0, "required_torque": 4.0, "speed": 2.0, "stall_torque": 10.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
available torque Nm	7.5
reserve Nm	3.5
required mechanical power W	8
envelope power W	15

Computed quantity	Numerical result
feasible	1

1. The teaching envelope has zero-speed torque 10 Nm and no-load speed 8 rad/s.
2. At speed 2 rad/s the linear speed factor is 0.750000.
3. Available torque is 7.500000 Nm while demand is 4 Nm.
4. Reserve is 3.500000 Nm; feasibility flag is True.
5. Demanded mechanical power is 8.000000 W.

Worked case 3: speed = 4

This case keeps the nominal configuration except for speed=4. The full input is {"no_load_speed": 8.0, "required_torque": 4.0, "speed": 4.0, "stall_torque": 10.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
available torque Nm	5
reserve Nm	1
required mechanical power W	16
envelope power W	20
feasible	1

1. The teaching envelope has zero-speed torque 10 Nm and no-load speed 8 rad/s.
2. At speed 4 rad/s the linear speed factor is 0.500000.
3. Available torque is 5.000000 Nm while demand is 4 Nm.
4. Reserve is 1.000000 Nm; feasibility flag is True.
5. Demanded mechanical power is 16.000000 W.

Worked case 4: speed = 6

This case keeps the nominal configuration except for speed=6. The full input is {"no_load_speed": 8.0, "required_torque": 4.0, "speed": 6.0, "stall_torque": 10.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
available torque Nm	2.5
reserve Nm	-1.5
required mechanical power W	24
envelope power W	15
feasible	0

1. The teaching envelope has zero-speed torque 10 Nm and no-load speed 8 rad/s.
2. At speed 6 rad/s the linear speed factor is 0.250000.
3. Available torque is 2.500000 Nm while demand is 4 Nm.
4. Reserve is -1.500000 Nm; feasibility flag is False.
5. Demanded mechanical power is 24.000000 W.

Worked case 5: speed = 8

This case keeps the nominal configuration except for speed=8. The full input is {"no_load_speed": 8.0, "required_torque": 4.0, "speed": 8.0, "stall_torque": 10.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
available torque Nm	0
reserve Nm	-4
required mechanical power W	32
envelope power W	0

Computed quantity	Numerical result
feasible	0

1. The teaching envelope has zero-speed torque 10 Nm and no-load speed 8 rad/s.
2. At speed 8 rad/s the linear speed factor is 0.000000.
3. Available torque is 0.000000 Nm while demand is 4 Nm.
4. Reserve is -4.000000 Nm; feasibility flag is False.
5. Demanded mechanical power is 32.000000 W.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The illustrative envelope is instantaneous and excludes continuous thermal derating and regenerative quadrants.

A positive reserve is only one constraint; position, load path, current and thermal checks remain separate.

K. Failure modes and troubleshooting

Treating stall torque as continuous; mixing motor-shaft and output-shaft speed; extrapolating a linear toy curve beyond the data range.

L. Practical exercises and extension tasks

1. Evaluate an entire minimum-jerk torque/speed trajectory against the envelope.
2. Add a voltage-dependent no-load speed.
3. Separate motoring and regenerative operation.

M. Connection to the Asimov research platform

Use measured or manufacturer-supplied actuator curves, firmware current limits and duty-cycle constraints for actual motors.

N. Completion record

Record your chosen case, predicted trend, measured available_torque_Nm, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 27, Feedback delay, sample-and-hold and tracking degradation. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 27 - Feedback delay, sample-and-hold and tracking degradation

A. The engineering question

A controller acts on information from the past.

B. Concepts and mathematical model

Even if each packet is delivered reliably, delayed position feedback changes the phase of the loop and can create oscillation. Sample-and-hold output adds another timing effect. This experiment stores a queue of previous states and makes the feedback delay explicit in samples. The plant and gains remain fixed, so changes in error can be attributed to the delay model rather than a simultaneous retuning.

u[k]=Kp*(reference-q[k-d])-Kd*v[k-d]; delay_seconds=d*dt.

C. Real-world meaning and limits

Physical timing measurements must include sensing, transport, scheduling and actuator application, not only network round-trip time.

D. Architecture and data flow

Configuration (delay) -> numerical model -> actual_delay_s, tracking_rmse_rad, peak_error_rad -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

delay is seconds of feedback age; plant integration is 1 ms and torque limit is an illustrative 3 Nm.

Parameter	Nominal value	Interpretation
delay	0.01	seconds

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/27_delay_control.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only delay changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/27_delay_control.py --case 0

.venv-book/bin/python expanded_lab/modules/27_delay_control.py --case 3

.venv-book/bin/python expanded_lab/run.py delay_control --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/27_delay_control.py

```
"""Feedback delay, sample-and-hold and tracking degradation. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    dt = 0.001
    delay = p["delay"]
    d = max(0, round(delay / dt))
    q = v = 0.0
    states = [(0.0, 0.0)] * (d + 1)
    errors = []
    torques = []
    for k in range(3000):
        t = k * dt
        target = 0.1 * np.sin(2 * np.pi * 0.5 * t)
        oldq, oldv = states[max(0, len(states) - 1 - d)]
        u = np.clip(20 * (target - oldq) - 2 * oldv, -3, 3)
        v += dt * (u - 0.1 * v) / 0.08
        q += dt * v
        states.append((q, v))
        errors.append(target - q)
        torques.append(u)
    errors = np.array(errors)
    torques = np.array(torques)
    metrics = dict(
        actual_delay_s=d * dt,
        tracking_rmse_rad=np.sqrt(np.mean(errors**2)),
        peak_error_rad=np.max(abs(errors)),
        saturation_fraction=np.mean(abs(torques) >= 2.999),
    )
    derivation = [
        f"Request delay {delay:g} s; rounding to 1 ms samples gives {d} delayed samples.",
        "The reference is a 0.1 rad, 0.5 Hz sinusoid; gains remain Kp=20 and Kd=2.",
        "Use delayed position and velocity but the current reference to calculate torque.",
        f"After three seconds, RMSE is {np.sqrt(np.mean(errors**2)):.6f} rad.",
        f'Peak error is {np.max(abs(errors)):.6f} rad and saturation fraction is
    {metrics["saturation_fraction"]:.6f}.',
    ]
    notes = [
        "The chosen delay is deterministic; real transport also has jitter, dropouts and
    timestamp uncertainty.",
        "A simulator with immediate state access can substantially overestimate the usable
    gain margin.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
```

```
parser = argparse.ArgumentParser()
parser.add_argument("--case", type=int, default=0, choices=range(6))
args = parser.parse_args()
parameters = {"delay": 0.01}
parameters["delay"] = [0.0, 0.002, 0.005, 0.01, 0.02, 0.05][args.case]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The state history implements delay without altering the integration timestep. Inspect the actual rounded delay because requested values may not be exactly representable in samples.

I. Expected behavior and six worked cases

The controlled variable is delay. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

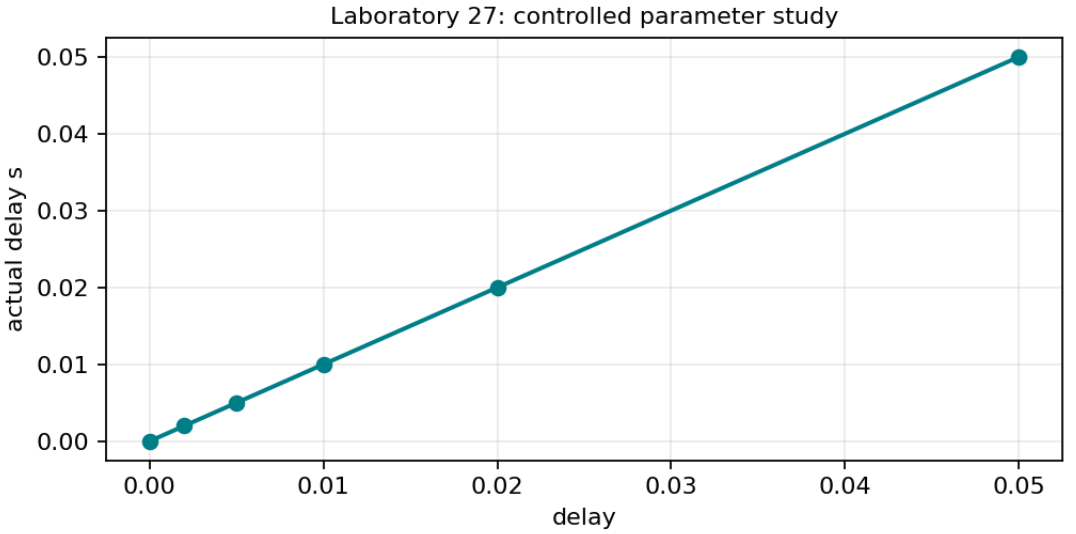


Figure L27. actual delay s as delay changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	delay	actual delay s
0	0	0
1	0.002	0.002
2	0.005	0.005
3	0.01	0.01
4	0.02	0.02
5	0.05	0.05

Worked case 0: delay = 0

This case keeps the nominal configuration except for delay=0. The full input is {"delay": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
actual delay s	0
tracking rmse rad	0.02216252
peak error rad	0.03238331
saturation fraction	0

1. Request delay 0 s; rounding to 1 ms samples gives 0 delayed samples.
2. The reference is a 0.1 rad, 0.5 Hz sinusoid; gains remain Kp=20 and Kd=2.
3. Use delayed position and velocity but the current reference to calculate torque.
4. After three seconds, RMSE is 0.022163 rad.
5. Peak error is 0.032383 rad and saturation fraction is 0.000000.

Worked case 1: delay = 0.002

This case keeps the nominal configuration except for delay=0.002. The full input is {"delay": 0.002}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
actual delay s	0.002
tracking rmse rad	0.02173876
peak error rad	0.03175124
saturation fraction	0

1. Request delay 0.002 s; rounding to 1 ms samples gives 2 delayed samples.
2. The reference is a 0.1 rad, 0.5 Hz sinusoid; gains remain Kp=20 and Kd=2.
3. Use delayed position and velocity but the current reference to calculate torque.
4. After three seconds, RMSE is 0.021739 rad.
5. Peak error is 0.031751 rad and saturation fraction is 0.000000.

Worked case 2: delay = 0.005

This case keeps the nominal configuration except for delay=0.005. The full input is {"delay": 0.005}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
actual delay s	0.005
tracking rmse rad	0.02110144
peak error rad	0.0308021
saturation fraction	0

1. Request delay 0.005 s; rounding to 1 ms samples gives 5 delayed samples.

2. The reference is a 0.1 rad, 0.5 Hz sinusoid; gains remain Kp=20 and Kd=2.
3. Use delayed position and velocity but the current reference to calculate torque.
4. After three seconds, RMSE is 0.021101 rad.
5. Peak error is 0.030802 rad and saturation fraction is 0.000000.

Worked case 3: delay = 0.01

This case keeps the nominal configuration except for delay=0.01. The full input is {"delay": 0.01}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
actual delay s	0.01
tracking rmse rad	0.02003481
peak error rad	0.02921747
saturation fraction	0

1. Request delay 0.01 s; rounding to 1 ms samples gives 10 delayed samples.
2. The reference is a 0.1 rad, 0.5 Hz sinusoid; gains remain Kp=20 and Kd=2.
3. Use delayed position and velocity but the current reference to calculate torque.
4. After three seconds, RMSE is 0.020035 rad.
5. Peak error is 0.029217 rad and saturation fraction is 0.000000.

Worked case 4: delay = 0.02

This case keeps the nominal configuration except for delay=0.02. The full input is {"delay": 0.02}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
actual delay s	0.02
tracking rmse rad	0.01788516
peak error rad	0.02603855
saturation fraction	0

1. Request delay 0.02 s; rounding to 1 ms samples gives 20 delayed samples.
2. The reference is a 0.1 rad, 0.5 Hz sinusoid; gains remain Kp=20 and Kd=2.
3. Use delayed position and velocity but the current reference to calculate torque.
4. After three seconds, RMSE is 0.017885 rad.
5. Peak error is 0.026039 rad and saturation fraction is 0.000000.

Worked case 5: delay = 0.05

This case keeps the nominal configuration except for delay=0.05. The full input is {"delay": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
actual delay s	0.05
tracking rmse rad	0.03399182
peak error rad	0.08598412
saturation fraction	0.12

1. Request delay 0.05 s; rounding to 1 ms samples gives 50 delayed samples.
2. The reference is a 0.1 rad, 0.5 Hz sinusoid; gains remain Kp=20 and Kd=2.
3. Use delayed position and velocity but the current reference to calculate torque.
4. After three seconds, RMSE is 0.033992 rad.

5. Peak error is 0.085984 rad and saturation fraction is 0.120000.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The chosen delay is deterministic; real transport also has jitter, dropouts and timestamp uncertainty.

A simulator with immediate state access can substantially overestimate the usable gain margin.

K. Failure modes and troubleshooting

Ignoring packet age because delivery is reliable; compensating delay with unvalidated gain increases; comparing different reference timing conventions.

L. Practical exercises and extension tasks

1. Add bounded seeded jitter to the feedback index.
2. Drop a burst of feedback samples and define a watchdog response.
3. Compare delayed feedback with predicted current state.

M. Connection to the Asimov research platform

Physical timing measurements must include sensing, transport, scheduling and actuator application, not only network round-trip time.

N. Completion record

Record your chosen case, predicted trend, measured actual_delay_s, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 28, First-order thermal models and duty-cycle limits. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 28 - First-order thermal models and duty-cycle limits

A. The engineering question

Temperature accumulates the history of losses.

B. Concepts and mathematical model

A motor can satisfy an instantaneous torque requirement while exceeding its thermal capacity over a longer duty cycle. A first-order resistance-capacitance model represents thermal storage and heat flow to ambient. Its time constant explains why short successful tests cannot prove continuous capability. The parameters below are illustrative and deliberately separate electrical loss from the thermal model.

C_th*T_dot=P_loss-(T-T_ambient)/R_th; steady rise=P_loss*R_th; time_constant=R_th*C_th.

C. Real-world meaning and limits

Use manufacturer temperature limits and identified thermal behavior to design real derating and stop thresholds.

D. Architecture and data flow

Configuration (loss_power, thermal_resistance, thermal_capacity, duration) -> numerical model -> final_temperature_C, steady_temperature_C, time_constant_s -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

loss_power is Watts dissipated as heat; thermal_resistance is K/W; thermal_capacity is J/K; duration is seconds.

Parameter	Nominal value	Interpretation
loss_power	20	Watts
thermal_resistance	2	K/W
thermal_capacity	100	J/K
duration	300	seconds

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/28_thermal.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only loss_power changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/28_thermal.py --case 0

.venv-book/bin/python expanded_lab/modules/28_thermal.py --case 3

.venv-book/bin/python expanded_lab/run.py thermal --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/28_thermal.py

```
"""First-order thermal models and duty-cycle limits. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    power, R, C = p["loss_power"], p["thermal_resistance"], p["thermal_capacity"]
    duration, ambient = p["duration"], 25.0
    rise = power * R * (1 - np.exp(-duration / (R * C)))
    final = ambient + rise
    threshold = 70.0
    steady = ambient + power * R
    if steady > threshold:
        crossing = -R * C * np.log(1 - (threshold - ambient) / (power * R))
    else:
        crossing = -1.0
    metrics = dict(
        final_temperature_C=final,
        steady_temperature_C=steady,
        time_constant_s=R * C,
        screening_margin_C=threshold - final,
        threshold_crossing_s=crossing,
    )
    derivation = [
        f"Assume loss power {power:g} W, Rth={R:g} K/W and Cth={C:g} J/K.",
        f"The thermal time constant is {R*C:.6f} s.",
        f"After {duration:g} s, temperature rise is {rise:.6f} K above 25 C ambient.",
        f"Predicted final temperature is {final:.6f} C and steady temperature is
    {steady:.6f} C.",
        f"The 70 C teaching threshold crossing time is {crossing:.6f} s; -1 means no
    crossing in the ideal constant-loss model.",
    ]
    notes = [
        "The 70 C threshold is an exercise parameter, not a motor or material rating.",
        "Actual winding, housing, drive and gearbox temperatures have different dynamics and
     measurement locations.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {
        "loss_power": 20.0,
        "thermal_resistance": 2.0,
        "thermal_capacity": 100.0,
        "duration": 300.0,
    }
```

```
parameters["loss_power"] = [5.0, 10.0, 15.0, 20.0, 30.0, 40.0][args.case]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The analytical solution gives both transient and steady behavior. An unreachable threshold is encoded explicitly rather than confusing infinity with a safe physical guarantee.

I. Expected behavior and six worked cases

The controlled variable is loss_power. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

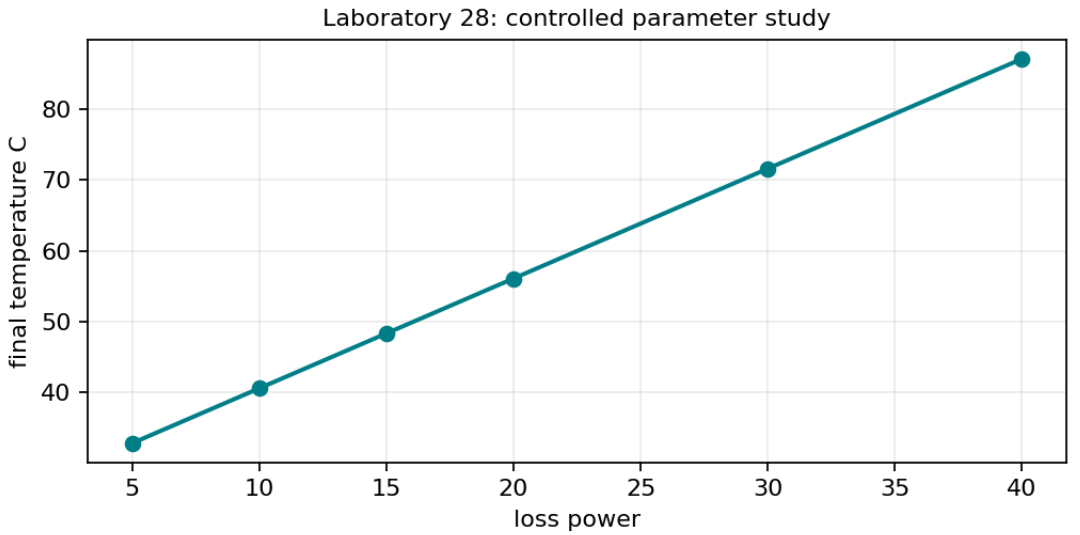


Figure L28. final temperature C as loss_power changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	loss_power	final temperature C
0	5	32.7686984

Case	loss_power	final temperature C
1	10	40.5373968
2	15	48.3060952
3	20	56.07479359
4	30	71.61219039
5	40	87.14958719

Worked case 0: loss_power = 5

This case keeps the nominal configuration except for loss_power=5. The full input is {"duration": 300.0, "loss_power": 5.0, "thermal_capacity": 100.0, "thermal_resistance": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final temperature C	32.7686984
steady temperature C	35
time constant s	200
screening margin C	37.2313016
threshold crossing s	-1

1. Assume loss power 5 W, Rth=2 K/W and Cth=100 J/K.
2. The thermal time constant is 200.000000 s.
3. After 300 s, temperature rise is 7.768698 K above 25 C ambient.
4. Predicted final temperature is 32.768698 C and steady temperature is 35.000000 C.
5. The 70 C teaching threshold crossing time is -1.000000 s; -1 means no crossing in the ideal constant-loss model.

Worked case 1: loss_power = 10

This case keeps the nominal configuration except for loss_power=10. The full input is {"duration": 300.0, "loss_power": 10.0, "thermal_capacity": 100.0, "thermal_resistance": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final temperature C	40.5373968
steady temperature C	45
time constant s	200
screening margin C	29.4626032
threshold crossing s	-1

1. Assume loss power 10 W, Rth=2 K/W and Cth=100 J/K.
2. The thermal time constant is 200.000000 s.
3. After 300 s, temperature rise is 15.537397 K above 25 C ambient.
4. Predicted final temperature is 40.537397 C and steady temperature is 45.000000 C.
5. The 70 C teaching threshold crossing time is -1.000000 s; -1 means no crossing in the ideal constant-loss model.

Worked case 2: loss_power = 15

This case keeps the nominal configuration except for loss_power=15. The full input is {"duration": 300.0, "loss_power": 15.0, "thermal_capacity": 100.0, "thermal_resistance": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final temperature C	48.3060952
steady temperature C	55
time constant s	200

Computed quantity	Numerical result
screening margin C	21.6939048
threshold crossing s	-1

1. Assume loss power 15 W, Rth=2 K/W and Cth=100 J/K.
2. The thermal time constant is 200.000000 s.
3. After 300 s, temperature rise is 23.306095 K above 25 C ambient.
4. Predicted final temperature is 48.306095 C and steady temperature is 55.000000 C.
5. The 70 C teaching threshold crossing time is -1.000000 s; -1 means no crossing in the ideal constant-loss model.

Worked case 3: loss_power = 20

This case keeps the nominal configuration except for loss_power=20. The full input is {"duration": 300.0, "loss_power": 20.0, "thermal_capacity": 100.0, "thermal_resistance": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final temperature C	56.07479359
steady temperature C	65
time constant s	200
screening margin C	13.92520641
threshold crossing s	-1

1. Assume loss power 20 W, Rth=2 K/W and Cth=100 J/K.
2. The thermal time constant is 200.000000 s.
3. After 300 s, temperature rise is 31.074794 K above 25 C ambient.
4. Predicted final temperature is 56.074794 C and steady temperature is 65.000000 C.

5. The 70 C teaching threshold crossing time is -1.000000 s; -1 means no crossing in the ideal constant-loss model.

Worked case 4: loss_power = 30

This case keeps the nominal configuration except for loss_power=30. The full input is {"duration": 300.0, "loss_power": 30.0, "thermal_capacity": 100.0, "thermal_resistance": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final temperature C	71.61219039
steady temperature C	85
time constant s	200
screening margin C	-1.61219039
threshold crossing s	277.25887222

1. Assume loss power 30 W, Rth=2 K/W and Cth=100 J/K.
2. The thermal time constant is 200.000000 s.
3. After 300 s, temperature rise is 46.612190 K above 25 C ambient.
4. Predicted final temperature is 71.612190 C and steady temperature is 85.000000 C.
5. The 70 C teaching threshold crossing time is 277.258872 s; -1 means no crossing in the ideal constant-loss model.

Worked case 5: loss_power = 40

This case keeps the nominal configuration except for loss_power=40. The full input is {"duration": 300.0, "loss_power": 40.0, "thermal_capacity": 100.0, "thermal_resistance": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final temperature C	87.14958719

Computed quantity	Numerical result
steady temperature C	105
time constant s	200
screening margin C	-17.14958719
threshold crossing s	165.33571464

1. Assume loss power 40 W, Rth=2 K/W and Cth=100 J/K.
2. The thermal time constant is 200.000000 s.
3. After 300 s, temperature rise is 62.149587 K above 25 C ambient.
4. Predicted final temperature is 87.149587 C and steady temperature is 105.000000 C.
5. The 70 C teaching threshold crossing time is 165.335715 s; -1 means no crossing in the ideal constant-loss model.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The 70 C threshold is an exercise parameter, not a motor or material rating.

Actual winding, housing, drive and gearbox temperatures have different dynamics and measurement locations.

K. Failure modes and troubleshooting

Using mechanical output power as heat loss; trusting a housing sensor to represent winding temperature instantly; ignoring ambient and airflow.

L. Practical exercises and extension tasks

1. Integrate a square-wave duty cycle numerically and compare average-loss approximations.
2. Fit Rth and Cth from a safe measured heating/cooling trace.
3. Repeat at a higher ambient temperature.

M. Connection to the Asimov research platform

Use manufacturer temperature limits and identified thermal behavior to design real derating and stop thresholds.

N. Completion record

Record your chosen case, predicted trend, measured final_temperature_C, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 29, Mechanical energy, electrical demand and regeneration. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 29 - Mechanical energy, electrical demand and regeneration

A. The engineering question

Electrical supply design depends on energy flow as well as peak current.

B. Concepts and mathematical model

Motoring consumes electrical energy; decelerating or lowering a load may return energy to the bus. A bench supply that sources current may not safely sink it. Separating positive mechanical work, negative mechanical work, conversion losses and regenerated energy avoids the misleading assumption that average motor power describes every electrical condition.

P_mech=tau*omega; P_elec=P_mech/eta for motoring, eta_regen*P_mech for regenerative operation; E=integral(P dt).

C. Real-world meaning and limits

Regeneration handling must be approved for the actual supply, battery, drive and isolation scheme before a loaded actuator test.

D. Architecture and data flow

Configuration (efficiency, regen_efficiency) -> numerical model -> energy_drawn_J, energy_returned_J, net_energy_J -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

efficiency is motoring mechanical/electrical power ratio; regen_efficiency is the fraction of negative mechanical energy returned electrically.

Parameter	Nominal value	Interpretation
efficiency	0.8	fraction
regen_efficiency	0.5	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/29_energy_regen.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only regen_efficiency changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/29_energy_regen.py --case 0

.venv-book/bin/python expanded_lab/modules/29_energy_regen.py --case 3

.venv-book/bin/python expanded_lab/run.py energy_regen --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/29_energy_regen.py

```
"""Mechanical energy, electrical demand and regeneration. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    eta = p["efficiency"]
    regen = p["regen_efficiency"]
    T = 4.0
    t = np.linspace(0, T, 2001)
    speed = 2 * np.sin(2 * np.pi * t / T)
    torque = 1.5 * np.cos(2 * np.pi * t / T) + 0.5
    mechanical = torque * speed
    electrical = np.where(mechanical >= 0, mechanical / eta, mechanical * regen)
    positive = np.trapezoid(np.maximum(electrical, 0), t)
    returned = -np.trapezoid(np.minimum(electrical, 0), t)
    metrics = dict(
        energy_drawn_J=positive,
        energy_returned_J=returned,
        net_energy_J=positive - returned,
        peak_draw_W=max(electrical),
        peak_return_W=-min(electrical),
    )
    derivation = [
        "Generate a four-second signed torque and speed cycle.",
        f"Motoring efficiency is {eta:g}; regenerative efficiency is {regen:g}.",
        f"Convert positive and negative mechanical power using their respective
    efficiencies.",
        f"Integrate positive electrical power: {positive:.6f} J drawn.",
        f"Integrate negative electrical power: {returned:.6f} J returned; net is
    {positive-returned:.6f} J.",
    ]
    notes = [
        "Negative mechanical power represents energy leaving the mechanical system; the
    electrical destination depends on the actual drive and power topology.",
        "A low net-energy cycle can still impose large instantaneous source and sink
    requirements.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"efficiency": 0.8, "regen_efficiency": 0.5}
    parameters["regen_efficiency"] = [0.0, 0.2, 0.4, 0.6, 0.8, 1.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
```

```
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Use signed power before integrating and keep motoring and regeneration totals separate. Clipping all negative power to zero would conceal a supply design requirement.

I. Expected behavior and six worked cases

The controlled variable is regen_efficiency. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

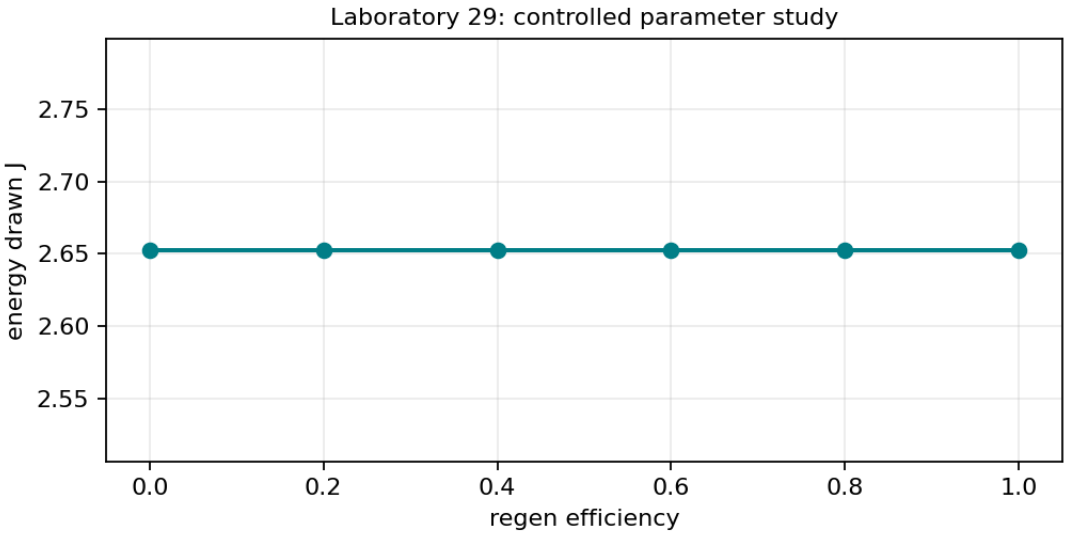


Figure L29. energy drawn J as regen_efficiency changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	regen_efficiency	energy drawn J
0	0	2.65257797
1	0.2	2.65257797

Case	regen_efficiency	energy drawn J
2	0.4	2.65257797
3	0.6	2.65257797
4	0.8	2.65257797
5	1	2.65257797

Worked case 0: regen_efficiency = 0

This case keeps the nominal configuration except for regen_efficiency=0. The full input is {"efficiency": 0.8, "regen_efficiency": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
energy drawn J	2.65257797
energy returned J	0
net energy J	2.65257797
peak draw W	2.80545561
peak return W	0

1. Generate a four-second signed torque and speed cycle.
2. Motoring efficiency is 0.8; regenerative efficiency is 0.
3. Convert positive and negative mechanical power using their respective efficiencies.
4. Integrate positive electrical power: 2.652578 J drawn.
5. Integrate negative electrical power: -0.000000 J returned; net is 2.652578 J.

Worked case 1: regen_efficiency = 0.2

This case keeps the nominal configuration except for regen_efficiency=0.2. The full input is {"efficiency": 0.8, "regen_efficiency": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
energy drawn J	2.65257797
energy returned J	0.42441247
net energy J	2.22816549
peak draw W	2.80545561
peak return W	0.4488729

1. Generate a four-second signed torque and speed cycle.
2. Motoring efficiency is 0.8; regenerative efficiency is 0.2.
3. Convert positive and negative mechanical power using their respective efficiencies.
4. Integrate positive electrical power: 2.652578 J drawn.
5. Integrate negative electrical power: 0.424412 J returned; net is 2.228165 J.

Worked case 2: regen_efficiency = 0.4

This case keeps the nominal configuration except for regen_efficiency=0.4. The full input is {"efficiency": 0.8, "regen_efficiency": 0.4}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
energy drawn J	2.65257797
energy returned J	0.84882495
net energy J	1.80375302

Computed quantity	Numerical result
peak draw W	2.80545561
peak return W	0.89774579

1. Generate a four-second signed torque and speed cycle.
2. Motoring efficiency is 0.8; regenerative efficiency is 0.4.
3. Convert positive and negative mechanical power using their respective efficiencies.
4. Integrate positive electrical power: 2.652578 J drawn.
5. Integrate negative electrical power: 0.848825 J returned; net is 1.803753 J.

Worked case 3: regen_efficiency = 0.6

This case keeps the nominal configuration except for regen_efficiency=0.6. The full input is {"efficiency": 0.8, "regen_efficiency": 0.6}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
energy drawn J	2.65257797
energy returned J	1.27323742
net energy J	1.37934054
peak draw W	2.80545561
peak return W	1.34661869

1. Generate a four-second signed torque and speed cycle.
2. Motoring efficiency is 0.8; regenerative efficiency is 0.6.
3. Convert positive and negative mechanical power using their respective efficiencies.
4. Integrate positive electrical power: 2.652578 J drawn.

5. Integrate negative electrical power: 1.273237 J returned; net is 1.379341 J.

Worked case 4: regen_efficiency = 0.8

This case keeps the nominal configuration except for regen_efficiency=0.8. The full input is {"efficiency": 0.8, "regen_efficiency": 0.8}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
energy drawn J	2.65257797
energy returned J	1.6976499
net energy J	0.95492807
peak draw W	2.80545561
peak return W	1.79549159

1. Generate a four-second signed torque and speed cycle.
2. Motoring efficiency is 0.8; regenerative efficiency is 0.8.
3. Convert positive and negative mechanical power using their respective efficiencies.
4. Integrate positive electrical power: 2.652578 J drawn.
5. Integrate negative electrical power: 1.697650 J returned; net is 0.954928 J.

Worked case 5: regen_efficiency = 1

This case keeps the nominal configuration except for regen_efficiency=1. The full input is {"efficiency": 0.8, "regen_efficiency": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
energy drawn J	2.65257797

Computed quantity	Numerical result
energy returned J	2.12206237
net energy J	0.53051559
peak draw W	2.80545561
peak return W	2.24436449

1. Generate a four-second signed torque and speed cycle.
2. Motoring efficiency is 0.8; regenerative efficiency is 1.
3. Convert positive and negative mechanical power using their respective efficiencies.
4. Integrate positive electrical power: 2.652578 J drawn.
5. Integrate negative electrical power: 2.122062 J returned; net is 0.530516 J.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Negative mechanical power represents energy leaving the mechanical system; the electrical destination depends on the actual drive and power topology.

A low net-energy cycle can still impose large instantaneous source and sink requirements.

K. Failure modes and troubleshooting

Calculating battery current from average mechanical power alone; assuming every supply can absorb regeneration; ignoring driver protection behavior.

L. Practical exercises and extension tasks

1. Estimate bus-capacitor voltage rise from returned energy.
2. Compare peak and average demand for the same cycle.

3. Add constant electronics power and recompute net energy.

M. Connection to the Asimov research platform

Regeneration handling must be approved for the actual supply, battery, drive and isolation scheme before a loaded actuator test.

N. Completion record

Record your chosen case, predicted trend, measured energy_drawn_J, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 30, Control deadlines, jitter and tail-latency budgets. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 30 - Control deadlines, jitter and tail-latency budgets

A. The engineering question

Average runtime is insufficient for a feedback deadline.

B. Concepts and mathematical model

Rare scheduling delays or long inference calls can age a command enough to matter physically. A timing budget should account for sensing, transport, estimation, control, inference and application. This experiment uses a seeded synthetic workload with occasional long delays to compare average and high-percentile completion times. It demonstrates a measurement method, not the timing performance of your laptop or a real-time operating system.

completion = sensing+transport+compute+scheduler_jitter; deadline_miss = completion>period.

C. Real-world meaning and limits

Use local embedded deadlines and independently tested stop authority where the physical control design requires them.

D. Architecture and data flow

Configuration (period_ms, jitter_mean_ms) -> numerical model -> mean_completion_ms, p99_completion_ms, max_completion_ms -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

period_ms is the target cycle deadline in milliseconds; jitter_mean_ms controls a synthetic scheduler-delay distribution.

Parameter	Nominal value	Interpretation
period_ms	2	Defined by the explicit chapter model and parameter contract
jitter_mean_ms	0.1	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/30_scheduler.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only jitter_mean_ms changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/30_scheduler.py --case 0

.venv-book/bin/python expanded_lab/modules/30_scheduler.py --case 3

.venv-book/bin/python expanded_lab/run.py scheduler --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/30_scheduler.py

```
"""Control deadlines, jitter and tail-latency budgets. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(19)
    n = 10000
    period = p["period_ms"]
    sensing = rng.uniform(0.1, 0.3, n)
    transport = rng.uniform(0.1, 0.4, n)
    compute = np.maximum(0.05, rng.normal(0.7, 0.12, n))
    jitter = rng.exponential(p["jitter_mean_ms"], n)
    rare = rng.random(n) < 0.005
    jitter[rare] += 3.0
    elapsed = sensing + transport + compute + jitter
    metrics = dict(
        mean_completion_ms=np.mean(elapsed),
        p99_completion_ms=np.quantile(elapsed, 0.99),
        max_completion_ms=np.max(elapsed),
        deadline_miss_fraction=np.mean(elapsed > period),
        minimum_observed_margin_ms=period - np.max(elapsed),
    )
    derivation = [
        f"Generate {n} synthetic cycles with period {period:g} ms.",
        f'Exponential jitter mean is {p["jitter_mean_ms"]:g} ms, plus a 0.5% long-delay
    mechanism.',
        f"Mean completion is {np.mean(elapsed):.6f} ms.",
        f"The 99th percentile is {np.quantile(elapsed,.99):.6f} ms and worst observed is
    {np.max(elapsed):.6f} ms.",
        f"Missed-deadline fraction is {np.mean(elapsed>period):.6f}.",
    ]
    notes = [
        "Worst observed latency is not a proven worst-case bound; finite sampling can miss
    rarer events.",
        "The random workload is pedagogical. Real timing claims require instrumented
    measurements under declared stress conditions.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"period_ms": 2.0, "jitter_mean_ms": 0.1}
    parameters["jitter_mean_ms"] = [0.01, 0.05, 0.1, 0.2, 0.5, 1.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
```

```
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Retain the entire latency distribution long enough to calculate misses and tails. The rare-delay mechanism explains why a reassuring mean can coexist with unacceptable behavior.

I. Expected behavior and six worked cases

The controlled variable is jitter_mean_ms. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

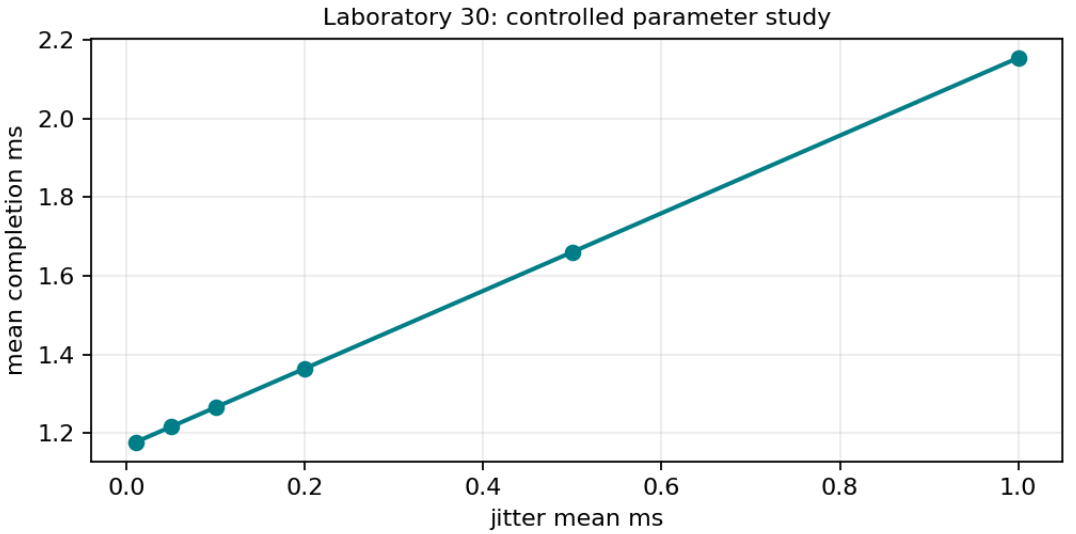


Figure L30. mean completion ms as jitter_mean_ms changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	jitter_mean_ms	mean completion ms
0	0.01	1.17679553
1	0.05	1.21628458

Case	jitter_mean_ms	mean completion ms
2	0.1	1.26564589
3	0.2	1.3643685
4	0.5	1.66053634
5	1	2.1541494

Worked case 0: jitter_mean_ms = 0.01

This case keeps the nominal configuration except for jitter_mean_ms=0.01. The full input is {"jitter_mean_ms": 0.01, "period_ms": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mean completion ms	1.17679553
p99 completion ms	1.55870418
max completion ms	4.60076518
deadline miss fraction	0.0048
minimum observed margin ms	-2.60076518

1. Generate 10000 synthetic cycles with period 2 ms.
2. Exponential jitter mean is 0.01 ms, plus a 0.5% long-delay mechanism.
3. Mean completion is 1.176796 ms.
4. The 99th percentile is 1.558704 ms and worst observed is 4.600765 ms.
5. Missed-deadline fraction is 0.004800.

Worked case 1: jitter_mean_ms = 0.05

This case keeps the nominal configuration except for jitter_mean_ms=0.05. The full input is {"jitter_mean_ms": 0.05, "period_ms": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mean completion ms	1.21628458
p99 completion ms	1.62069344
max completion ms	4.62485526
deadline miss fraction	0.0048
minimum observed margin ms	-2.62485526

1. Generate 10000 synthetic cycles with period 2 ms.
2. Exponential jitter mean is 0.05 ms, plus a 0.5% long-delay mechanism.
3. Mean completion is 1.216285 ms.
4. The 99th percentile is 1.620693 ms and worst observed is 4.624855 ms.
5. Missed-deadline fraction is 0.004800.

Worked case 2: jitter_mean_ms = 0.1

This case keeps the nominal configuration except for jitter_mean_ms=0.1. The full input is {"jitter_mean_ms": 0.1, "period_ms": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mean completion ms	1.26564589
p99 completion ms	1.78166362
max completion ms	4.72837516
deadline miss fraction	0.0054
minimum observed margin ms	-2.72837516

1. Generate 10000 synthetic cycles with period 2 ms.
2. Exponential jitter mean is 0.1 ms, plus a 0.5% long-delay mechanism.
3. Mean completion is 1.265646 ms.

4. The 99th percentile is 1.781664 ms and worst observed is 4.728375 ms.
5. Missed-deadline fraction is 0.005400.

Worked case 3: jitter_mean_ms = 0.2

This case keeps the nominal configuration except for jitter_mean_ms=0.2. The full input is {"jitter_mean_ms": 0.2, "period_ms": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mean completion ms	1.3643685
p99 completion ms	2.24949344
max completion ms	5.04997158
deadline miss fraction	0.0224
minimum observed margin ms	-3.04997158

1. Generate 10000 synthetic cycles with period 2 ms.
2. Exponential jitter mean is 0.2 ms, plus a 0.5% long-delay mechanism.
3. Mean completion is 1.364368 ms.
4. The 99th percentile is 2.249493 ms and worst observed is 5.049972 ms.
5. Missed-deadline fraction is 0.022400.

Worked case 4: jitter_mean_ms = 0.5

This case keeps the nominal configuration except for jitter_mean_ms=0.5. The full input is {"jitter_mean_ms": 0.5, "period_ms": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mean completion ms	1.66053634
p99 completion ms	3.77698821

Computed quantity	Numerical result
max completion ms	6.01476083
deadline miss fraction	0.1902
minimum observed margin ms	-4.01476083

1. Generate 10000 synthetic cycles with period 2 ms.
2. Exponential jitter mean is 0.5 ms, plus a 0.5% long-delay mechanism.
3. Mean completion is 1.660536 ms.
4. The 99th percentile is 3.776988 ms and worst observed is 6.014761 ms.
5. Missed-deadline fraction is 0.190200.

Worked case 5: jitter_mean_ms = 1

This case keeps the nominal configuration except for jitter_mean_ms=1. The full input is {"jitter_mean_ms": 1.0, "period_ms": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mean completion ms	2.1541494
p99 completion ms	5.77391455
max completion ms	9.67822931
deadline miss fraction	0.4325
minimum observed margin ms	-7.67822931

1. Generate 10000 synthetic cycles with period 2 ms.
2. Exponential jitter mean is 1 ms, plus a 0.5% long-delay mechanism.
3. Mean completion is 2.154149 ms.
4. The 99th percentile is 5.773915 ms and worst observed is 9.678229 ms.
5. Missed-deadline fraction is 0.432500.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Worst observed latency is not a proven worst-case bound; finite sampling can miss rarer events.

The random workload is pedagogical. Real timing claims require instrumented measurements under declared stress conditions.

K. Failure modes and troubleshooting

Calling Python timer intervals hard-real-time guarantees; measuring only warm unloaded inference; ignoring clock overhead and missed samples.

L. Practical exercises and extension tasks

1. Add a separate inference task every tenth cycle and calculate the shared budget.
2. Compare a deadline watchdog with a packet-count watchdog.
3. Measure actual workload latency on the current workstation and replace the synthetic distribution.

M. Connection to the Asimov research platform

Use local embedded deadlines and independently tested stop authority where the physical control design requires them.

N. Completion record

Record your chosen case, predicted trend, measured mean_completion_ms, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 31, Encoder counts, quantization, wrapping and velocity estimates. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Part IX - Sensors, geometry and state estimation

State estimation is a disciplined explanation of what noisy, delayed measurements say about the robot. It requires units, coordinate frames, clock models and validity conditions. These laboratories make quantization, bias, acceleration contamination, measurement dropout, calibration geometry and slipping-contact assumptions visible. Each estimator is compared with known synthetic truth, allowing a precise diagnosis before the same method is tested against physical reference measurements.

Laboratory 31 - Encoder counts, quantization, wrapping and velocity estimates

A. The engineering question

An encoder converts position into a finite representation.

B. Concepts and mathematical model

Quantization error may be small for position but large after numerical differentiation, especially when the sample period is short. Wrapped single-turn angles also create artificial jumps at the boundary unless unwrapped consistently. This experiment produces a smooth known trajectory, quantizes it to counts, reconstructs angle and estimates velocity. Ground truth allows position and derivative error to be separated.

count=round(angle/(2*pi)*resolution); angle_hat=count*2*pi/resolution; velocity_hat=delta(angle_unwrapped)/dt.

C. Real-world meaning and limits

Confirm encoder placement, count scaling, wrap convention and absolute-reference persistence for the exact actuator firmware.

D. Architecture and data flow

Configuration (counts) -> numerical model -> quantization_step_rad, position_rmse_rad, velocity_rmse_rad_s -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

counts is the assumed effective output counts per revolution; the teaching sample interval is 2 ms.

Parameter	Nominal value	Interpretation
counts	4096	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/31_encoder.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only counts changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/31_encoder.py --case 0

.venv-book/bin/python expanded_lab/modules/31_encoder.py --case 3

.venv-book/bin/python expanded_lab/run.py encoder --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/31_encoder.py

```
"""Encoder counts, quantization, wrapping and velocity estimates. Numerical teaching
    experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    resolution = int(p["counts"])
    dt = 0.002
    t = np.arange(0, 2, dt)
    truth = 3.1 + 0.4 * np.sin(2 * np.pi * 0.5 * t)
    wrapped = (truth + np.pi) % (2 * np.pi) - np.pi
    counts = np.round(wrapped / (2 * np.pi) * resolution)
    measured = counts * 2 * np.pi / resolution
    unwrapped = np.unwrap(measured)
    unwrapped += truth[0] - unwrapped[0]
    velocity = np.gradient(unwrapped, dt)
    exact = 0.4 * np.pi * np.cos(np.pi * t)
    metrics = dict(
        quantization_step_rad=2 * np.pi / resolution,
        position_rmse_rad=np.sqrt(np.mean((unwrapped - truth) ** 2)),
        velocity_rmse_rad_s=np.sqrt(np.mean((velocity - exact) ** 2)),
        raw_wrap_jump_rad=np.max(abs(np.diff(measured))),
    )
    derivation = [
        f"Encoder resolution is {resolution} counts per revolution.",
        f"One count represents {2*np.pi/resolution:.8f} rad.",
        "Generate a trajectory crossing the +/-pi wrapping boundary and quantize it.",
        f'After unwrap and initial alignment, position RMSE is
    {metrics["position_rmse_rad"]:.8f} rad.',
        f'Differentiating at 2 ms gives velocity RMSE {metrics["velocity_rmse_rad_s"]:.6f}
    rad/s.',
    ]
    notes = [
        "The initial alignment removes an arbitrary integer-turn offset but is not an
    absolute-zero calibration.",
        "Higher numerical differentiation rate does not create additional encoder position
    information.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"counts": 4096}
    parameters["counts"] = [256, 512, 1024, 4096, 16384, 65536][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
```

```
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Quantize the wrapped measurement before unwrapping, then compare position and differentiated velocity separately. The raw jump metric makes a wrapping bug visible even when the angle trace looks periodic.

I. Expected behavior and six worked cases

The controlled variable is counts. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

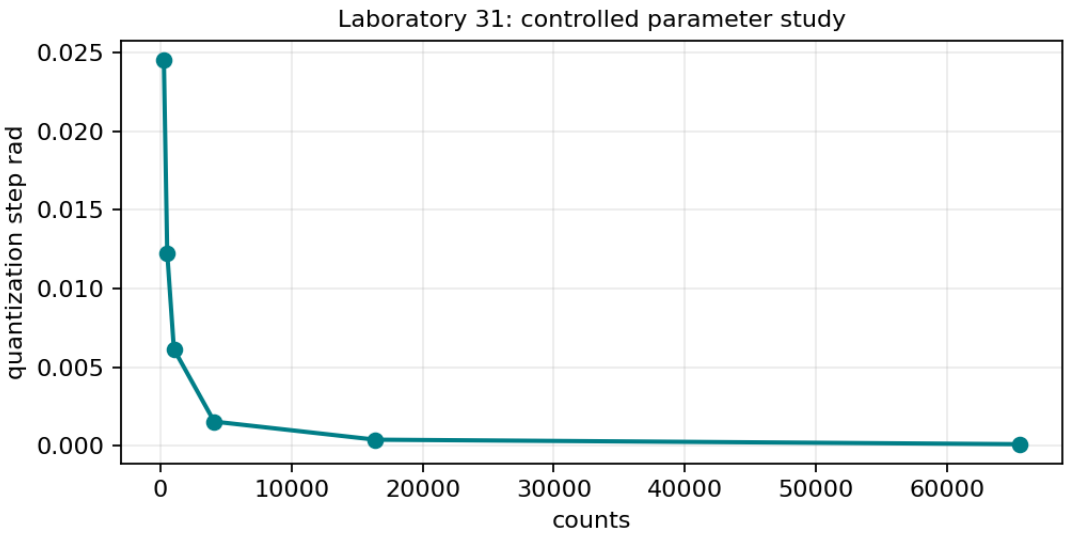


Figure L31. quantization step rad as counts changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	counts	quantization step rad
0	256	0.02454369
1	512	0.01227185

Case	counts	quantization step rad
2	1024	0.00613592
3	4096	0.00153398
4	16384	0.0003835
5	65536	0.00009587

Worked case 0: counts = 256

This case keeps the nominal configuration except for counts=256. The full input is {"counts": 256}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
quantization step rad	0.02454369
position rmse rad	0.01018916
velocity rmse rad s	2.0439869
raw wrap jump rad	6.28318531

1. Encoder resolution is 256 counts per revolution.
2. One count represents 0.02454369 rad.
3. Generate a trajectory crossing the +/-pi wrapping boundary and quantize it.
4. After unwrap and initial alignment, position RMSE is 0.01018916 rad.
5. Differentiating at 2 ms gives velocity RMSE 2.043987 rad/s.

Worked case 1: counts = 512

This case keeps the nominal configuration except for counts=512. The full input is {"counts": 512}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
quantization step rad	0.01227185
position rmse rad	0.00595705
velocity rmse rad s	1.28600645
raw wrap jump rad	6.28318531

1. Encoder resolution is 512 counts per revolution.
2. One count represents 0.01227185 rad.
3. Generate a trajectory crossing the +/-pi wrapping boundary and quantize it.
4. After unwrap and initial alignment, position RMSE is 0.00595705 rad.
5. Differentiating at 2 ms gives velocity RMSE 1.286006 rad/s.

Worked case 2: counts = 1024

This case keeps the nominal configuration except for counts=1024. The full input is {"counts": 1024}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
quantization step rad	0.00613592
position rmse rad	0.00215223
velocity rmse rad s	0.66395253
raw wrap jump rad	6.28318531

1. Encoder resolution is 1024 counts per revolution.
2. One count represents 0.00613592 rad.
3. Generate a trajectory crossing the +/-pi wrapping boundary and quantize it.
4. After unwrap and initial alignment, position RMSE is 0.00215223 rad.
5. Differentiating at 2 ms gives velocity RMSE 0.663953 rad/s.

Worked case 3: counts = 4096

This case keeps the nominal configuration except for counts=4096. The full input is {"counts": 4096}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
quantization step rad	0.00153398
position rmse rad	0.00047604
velocity rmse rad s	0.15363803
raw wrap jump rad	6.28011735

1. Encoder resolution is 4096 counts per revolution.
2. One count represents 0.00153398 rad.
3. Generate a trajectory crossing the +/-pi wrapping boundary and quantize it.
4. After unwrap and initial alignment, position RMSE is 0.00047604 rad.
5. Differentiating at 2 ms gives velocity RMSE 0.153638 rad/s.

Worked case 4: counts = 16384

This case keeps the nominal configuration except for counts=16384. The full input is {"counts": 16384}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
quantization step rad	0.0003835
position rmse rad	0.00020665
velocity rmse rad s	0.03905732
raw wrap jump rad	6.28050084

1. Encoder resolution is 16384 counts per revolution.
2. One count represents 0.00038350 rad.

3. Generate a trajectory crossing the +/-pi wrapping boundary and quantize it.
4. After unwrap and initial alignment, position RMSE is 0.00020665 rad.
5. Differentiating at 2 ms gives velocity RMSE 0.039057 rad/s.

Worked case 5: counts = 65536

This case keeps the nominal configuration except for counts=65536. The full input is {"counts": 65536}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
quantization step rad	0.00009587
position rmse rad	0.00003237
velocity rmse rad s	0.00966549
raw wrap jump rad	6.28069259

1. Encoder resolution is 65536 counts per revolution.
2. One count represents 0.00009587 rad.
3. Generate a trajectory crossing the +/-pi wrapping boundary and quantize it.
4. After unwrap and initial alignment, position RMSE is 0.00003237 rad.
5. Differentiating at 2 ms gives velocity RMSE 0.009665 rad/s.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The initial alignment removes an arbitrary integer-turn offset but is not an absolute-zero calibration.

Higher numerical differentiation rate does not create additional encoder position information.

K. Failure modes and troubleshooting

Applying unwrap after differentiation; assuming nominal encoder bit depth equals effective output accuracy; losing multi-turn reference across reboot.

L. Practical exercises and extension tasks

1. Add a missed-sample gap and use actual timestamps in differentiation.
2. Compare a short-window slope fit with a two-point difference.
3. Model a motor-side encoder through a gearbox and add backlash.

M. Connection to the Asimov research platform

Confirm encoder placement, count scaling, wrap convention and absolute-reference persistence for the exact actuator firmware.

N. Completion record

Record your chosen case, predicted trend, measured quantization_step_rad, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 32, Gyroscope bias calibration and attitude drift. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 32 - Gyroscope bias calibration and attitude drift

A. The engineering question

A small gyro bias integrates into a large orientation error.

B. Concepts and mathematical model

Averaging a stationary segment can estimate a constant offset, but temperature drift or motion during calibration violates that assumption. Measurement noise and bias uncertainty affect integrated orientation differently. The experiment estimates bias from a known stationary period, then compares raw and corrected integration against a known angular motion. It separates the calibration samples from the evaluation interval.

omega_measured=omega_true+bias+noise; bias_hat=mean(stationary samples); angle=sum((omega_measured-bias_hat)*dt).

C. Real-world meaning and limits

Physical IMU calibration requires a known stationary fixture and documented temperature/mounting conditions.

D. Architecture and data flow

Configuration (calibration_seconds) -> numerical model -> estimated_bias_rad_s, bias_error_rad_s, raw_final_angle_error_rad -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

calibration_seconds is the stationary averaging duration; injected gyro bias and noise are in radians per second.

Parameter	Nominal value	Interpretation
calibration_seconds	2	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/32_gyro_bias.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only calibration_seconds changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/32_gyro_bias.py --case 0

.venv-book/bin/python expanded_lab/modules/32_gyro_bias.py --case 3

.venv-book/bin/python expanded_lab/run.py gyro_bias --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/32_gyro_bias.py

```
"""Gyroscope bias calibration and attitude drift. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(23)
    dt = 0.01
    T = 10.0
    n = int(T / dt)
    calibration = int(p["calibration_seconds"] / dt)
    calibration = max(2, calibration)
    bias = 0.015
    sigma = 0.01
    stationary = bias + rng.normal(0, sigma, calibration)
    estimated = stationary.mean()
    truth = 0.2 * np.ones(n)
    measured = truth + bias + rng.normal(0, sigma, n)
    raw = np.cumsum(measured) * dt
    corrected = np.cumsum(measured - estimated) * dt
    expected = np.cumsum(truth) * dt
    metrics = dict(
        estimated_bias_rad_s=estimated,
        bias_error_rad_s=estimated - bias,
        raw_final_angle_error_rad=raw[-1] - expected[-1],
        corrected_final_angle_error_rad=corrected[-1] - expected[-1],
        corrected_angle_rmse_rad=np.sqrt(np.mean((corrected - expected) ** 2)),
    )
    derivation = [
        f"Collect {calibration} stationary samples at 100 Hz.",
        f"True injected bias is {bias:g} rad/s; estimated bias is {estimated:.8f} rad/s.",
        f"Integrate a 0.2 rad/s motion for {T:g} s.",
        f"Raw final attitude error is {raw[-1]-expected[-1]:.8f} rad.",
        f"After bias correction, final error is {corrected[-1]-expected[-1]:.8f} rad.",
    ]
    notes = [
        "This calibration assumes zero true angular velocity and constant bias during both
    intervals.",
        "Longer averaging reduces random estimation error but cannot remove a
    temperature-dependent bias change.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"calibration_seconds": 2.0}
    parameters["calibration_seconds"] = [0.1, 0.25, 0.5, 1.0, 2.0, 5.0][args.case]
```

```
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Use separate random samples for calibration and motion. Integrating the same noisy calibration record as evaluation would conceal the actual generalization problem.

I. Expected behavior and six worked cases

The controlled variable is calibration_seconds. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

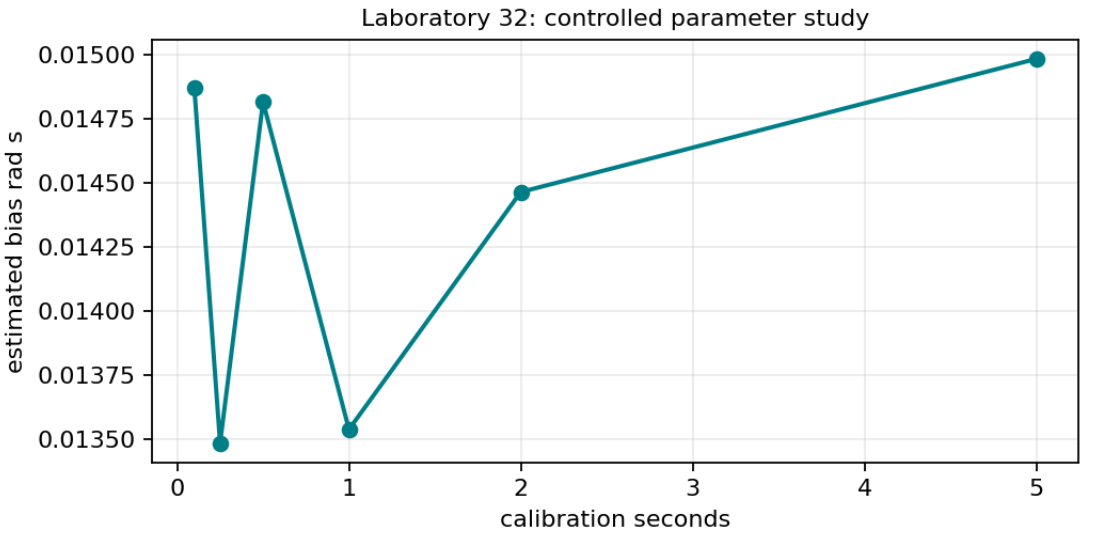


Figure L32. estimated bias rad s as calibration_seconds changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	calibration_seconds	estimated bias rad s
0	0.1	0.01487279

Case	calibration_seconds	estimated bias rad s
1	0.25	0.01348609
2	0.5	0.01481712
3	1	0.01353986
4	2	0.01446547
5	5	0.01498463

Worked case 0: calibration_seconds = 0.1

This case keeps the nominal configuration except for calibration_seconds=0.1. The full input is {"calibration_seconds": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
estimated bias rad s	0.01487279
bias error rad s	-0.00012721
raw final angle error rad	0.14615633
corrected final angle error rad	-0.00257158
corrected angle rmse rad	0.00090744

1. Collect 10 stationary samples at 100 Hz.
2. True injected bias is 0.015 rad/s; estimated bias is 0.01487279 rad/s.
3. Integrate a 0.2 rad/s motion for 10 s.
4. Raw final attitude error is 0.14615633 rad.
5. After bias correction, final error is -0.00257158 rad.

Worked case 1: calibration_seconds = 0.25

This case keeps the nominal configuration except for calibration_seconds=0.25. The full input is {"calibration_seconds": 0.25}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
estimated bias rad s	0.01348609
bias error rad s	-0.00151391
raw final angle error rad	0.14639323
corrected final angle error rad	0.01153231
corrected angle rmse rad	0.00823944

1. Collect 25 stationary samples at 100 Hz.
2. True injected bias is 0.015 rad/s; estimated bias is 0.01348609 rad/s.
3. Integrate a 0.2 rad/s motion for 10 s.
4. Raw final attitude error is 0.14639323 rad.
5. After bias correction, final error is 0.01153231 rad.

Worked case 2: calibration_seconds = 0.5

This case keeps the nominal configuration except for calibration_seconds=0.5. The full input is {"calibration_seconds": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
estimated bias rad s	0.01481712
bias error rad s	-0.00018288
raw final angle error rad	0.14579471
corrected final angle error rad	-0.00237649

Computed quantity	Numerical result
corrected angle rmse rad	0.00103681

1. Collect 50 stationary samples at 100 Hz.
2. True injected bias is 0.015 rad/s; estimated bias is 0.01481712 rad/s.
3. Integrate a 0.2 rad/s motion for 10 s.
4. Raw final attitude error is 0.14579471 rad.
5. After bias correction, final error is -0.00237649 rad.

Worked case 3: calibration_seconds = 1

This case keeps the nominal configuration except for calibration_seconds=1. The full input is {"calibration_seconds": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
estimated bias rad s	0.01353986
bias error rad s	-0.00146014
raw final angle error rad	0.1477647
corrected final angle error rad	0.01236609
corrected angle rmse rad	0.00850044

1. Collect 100 stationary samples at 100 Hz.
2. True injected bias is 0.015 rad/s; estimated bias is 0.01353986 rad/s.
3. Integrate a 0.2 rad/s motion for 10 s.
4. Raw final attitude error is 0.14776470 rad.
5. After bias correction, final error is 0.01236609 rad.

Worked case 4: calibration_seconds = 2

This case keeps the nominal configuration except for calibration_seconds=2. The full input is {"calibration_seconds": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
estimated bias rad s	0.01446547
bias error rad s	-0.00053453
raw final angle error rad	0.14646226
corrected final angle error rad	0.00180752
corrected angle rmse rad	0.00263157

1. Collect 200 stationary samples at 100 Hz.
2. True injected bias is 0.015 rad/s; estimated bias is 0.01446547 rad/s.
3. Integrate a 0.2 rad/s motion for 10 s.
4. Raw final attitude error is 0.14646226 rad.
5. After bias correction, final error is 0.00180752 rad.

Worked case 5: calibration_seconds = 5

This case keeps the nominal configuration except for calibration_seconds=5. The full input is {"calibration_seconds": 5.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
estimated bias rad s	0.01498463
bias error rad s	-0.00001537
raw final angle error rad	0.14713983
corrected final angle error rad	-0.00270643

Computed quantity	Numerical result
corrected angle rmse rad	0.0028349

1. Collect 500 stationary samples at 100 Hz.
2. True injected bias is 0.015 rad/s; estimated bias is 0.01498463 rad/s.
3. Integrate a 0.2 rad/s motion for 10 s.
4. Raw final attitude error is 0.14713983 rad.
5. After bias correction, final error is -0.00270643 rad.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

This calibration assumes zero true angular velocity and constant bias during both intervals.

Longer averaging reduces random estimation error but cannot remove a temperature-dependent bias change.

K. Failure modes and troubleshooting

Calibrating while the robot vibrates or moves; forgetting sensor-to-body rotation; subtracting degrees-per-second bias from radian data.

L. Practical exercises and extension tasks

1. Introduce a slow temperature-dependent bias ramp.
2. Repeat across several seeds and quantify calibration variability.
3. Reject calibration intervals with excessive measured variance.

M. Connection to the Asimov research platform

Physical IMU calibration requires a known stationary fixture and documented temperature/mounting conditions.

N. Completion record

Record your chosen case, predicted trend, measured estimated_bias_rad_s, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 33, Complementary attitude filtering and acceleration contamination. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 33 - Complementary attitude filtering and acceleration contamination

A. The engineering question

A gyroscope tracks rapid angular changes but drifts.

B. Concepts and mathematical model

An accelerometer can provide a gravity-direction reference when other acceleration is small, but cannot distinguish gravity from arbitrary linear acceleration by itself. A complementary filter blends integrated gyro attitude with a slower accelerometer estimate. The blend controls the response to bias and contamination. This scalar roll example makes those competing effects visible before a three-dimensional estimator is introduced.

roll_pred=roll_prev+gyro*dt; roll_next=alpha*roll_pred+(1-alpha)*roll_acc; alpha=tau/(tau+dt).

C. Real-world meaning and limits

IMU correction assumptions must be tested in supported stationary, moving and contact-transition conditions.

D. Architecture and data flow

Configuration (time_constant) -> numerical model -> alpha, total_rmse_rad, clean_rmse_rad -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

time_constant is seconds; alpha is derived from it and the fixed 10 ms sample interval.

Parameter	Nominal value	Interpretation
time_constant	0.5	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/33_complementary.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only time_constant changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/33_complementary.py --case 0

.venv-book/bin/python expanded_lab/modules/33_complementary.py --case 3

.venv-book/bin/python expanded_lab/run.py complementary --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/33_complementary.py

```
"""Complementary attitude filtering and acceleration contamination. Numerical teaching
    experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(29)
    dt = 0.01
    t = np.arange(0, 8, dt)
    truth = 0.15 * np.sin(2 * np.pi * 0.3 * t)
    rate = 0.15 * 2 * np.pi * 0.3 * np.cos(2 * np.pi * 0.3 * t)
    gyro = rate + 0.01 + rng.normal(0, 0.005, len(t))
    accel_angle = truth + rng.normal(0, 0.02, len(t))
    accel_angle[(t > 3) & (t < 4)] += 0.15
    tau = p["time_constant"]
    alpha = tau / (tau + dt)
    estimate = 0.0
    history = []
    for g, a in zip(gyro, accel_angle):
        estimate = alpha * (estimate + g * dt) + (1 - alpha) * a
        history.append(estimate)
    error = np.array(history) - truth
    clean = (t < 3) | (t > 5)
    metrics = dict(
        alpha=alpha,
        total_rmse_rad=np.sqrt(np.mean(error**2)),
        clean_rmse_rad=np.sqrt(np.mean(error[clean] ** 2)),
        contamination_peak_error_rad=np.max(abs(error[(t > 3) & (t < 4)])),
        final_error_rad=error[-1],
    )
    derivation = [
        f"Time constant {tau:g} s with dt=0.01 s gives alpha={alpha:.8f}.",
        "True roll is a 0.15 rad, 0.3 Hz sinusoid; gyro bias is 0.01 rad/s.",
        "Contaminate the accelerometer roll proxy by +0.15 rad between three and four
    seconds.",
        f'Whole-record RMSE is {metrics["total_rmse_rad"]:.6f} rad; clean-segment RMSE is
    {metrics["clean_rmse_rad"]:.6f} rad.',
        f'Peak error during contamination is {metrics["contamination_peak_error_rad"]:.6f}
    rad.',
    ]
    notes = [
        "The synthetic accelerometer-angle disturbance represents a limitation of
    gravity-based attitude correction during motion.",
        "A production estimator needs measurement validity checks and a consistent
    three-dimensional orientation representation.",
    ]
    return metrics, derivation, notes

if __name__ == "__main__":
```

```
parser = argparse.ArgumentParser()
parser.add_argument("--case", type=int, default=0, choices=range(6))
args = parser.parse_args()
parameters = {"time_constant": 0.5}
parameters["time_constant"] = [0.05, 0.1, 0.25, 0.5, 1.0, 2.0][args.case]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Compare clean and contaminated segments separately instead of rewarding a filter for one aggregate metric. Fast accelerometer correction reduces bias drift but follows false gravity cues more strongly.

I. Expected behavior and six worked cases

The controlled variable is time_constant. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

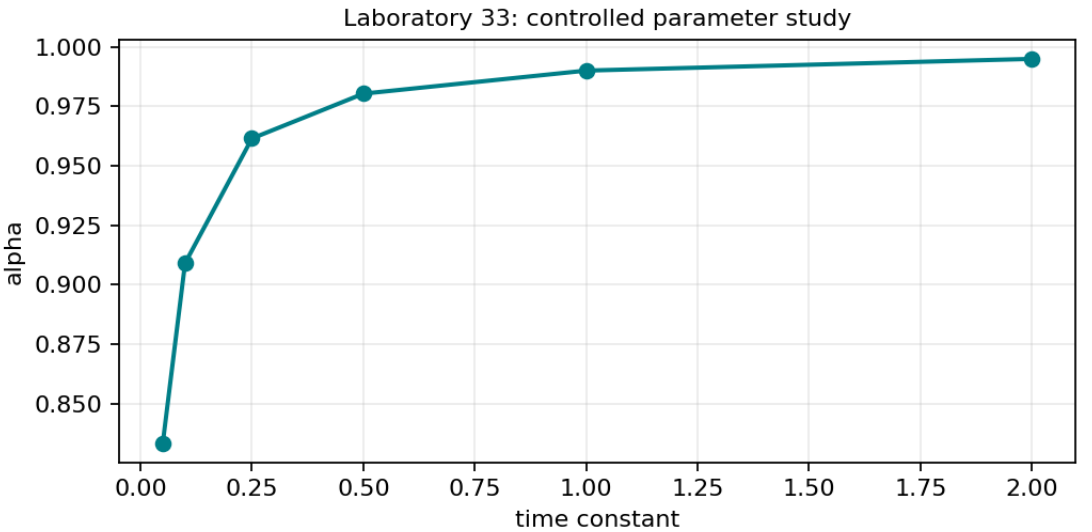


Figure L33. alpha as time_constant changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	time_constant	alpha
0	0.05	0.83333333
1	0.1	0.90909091
2	0.25	0.96153846
3	0.5	0.98039216
4	1	0.99009901
5	2	0.99502488

Worked case 0: time_constant = 0.05

This case keeps the nominal configuration except for time_constant=0.05. The full input is {"time_constant": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
alpha	0.83333333
total rmse rad	0.05270032
clean rmse rad	0.00632525
contamination peak error rad	0.16137189
final error rad	-0.00094408

1. Time constant 0.05 s with dt=0.01 s gives alpha=0.83333333.
2. True roll is a 0.15 rad, 0.3 Hz sinusoid; gyro bias is 0.01 rad/s.
3. Contaminate the accelerometer roll proxy by +0.15 rad between three and four seconds.
4. Whole-record RMSE is 0.052700 rad; clean-segment RMSE is 0.006325 rad.
5. Peak error during contamination is 0.161372 rad.

Worked case 1: time_constant = 0.1

This case keeps the nominal configuration except for time_constant=0.1. The full input is {"time_constant": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
alpha	0.90909091
total rmse rad	0.05129666
clean rmse rad	0.0047827
contamination peak error rad	0.15794639
final error rad	-0.0008678

1. Time constant 0.1 s with dt=0.01 s gives alpha=0.90909091.
2. True roll is a 0.15 rad, 0.3 Hz sinusoid; gyro bias is 0.01 rad/s.
3. Contaminate the accelerometer roll proxy by +0.15 rad between three and four seconds.
4. Whole-record RMSE is 0.051297 rad; clean-segment RMSE is 0.004783 rad.
5. Peak error during contamination is 0.157946 rad.

Worked case 2: time_constant = 0.25

This case keeps the nominal configuration except for time_constant=0.25. The full input is {"time_constant": 0.25}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
alpha	0.96153846
total rmse rad	0.04755824
clean rmse rad	0.00412304
contamination peak error rad	0.15254579

Computed quantity	Numerical result
final error rad	-0.00051307

1. Time constant 0.25 s with dt=0.01 s gives alpha=0.96153846.
2. True roll is a 0.15 rad, 0.3 Hz sinusoid; gyro bias is 0.01 rad/s.
3. Contaminate the accelerometer roll proxy by +0.15 rad between three and four seconds.
4. Whole-record RMSE is 0.047558 rad; clean-segment RMSE is 0.004123 rad.
5. Peak error during contamination is 0.152546 rad.

Worked case 3: time_constant = 0.5

This case keeps the nominal configuration except for time_constant=0.5. The full input is {"time_constant": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
alpha	0.98039216
total rmse rad	0.0429261
clean rmse rad	0.00790984
contamination peak error rad	0.13563467
final error rad	0.00201579

1. Time constant 0.5 s with dt=0.01 s gives alpha=0.98039216.
2. True roll is a 0.15 rad, 0.3 Hz sinusoid; gyro bias is 0.01 rad/s.
3. Contaminate the accelerometer roll proxy by +0.15 rad between three and four seconds.
4. Whole-record RMSE is 0.042926 rad; clean-segment RMSE is 0.007910 rad.
5. Peak error during contamination is 0.135635 rad.

Worked case 4: time_constant = 1

This case keeps the nominal configuration except for time_constant=1. The full input is {"time_constant": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
alpha	0.99009901
total rmse rad	0.03899949
clean rmse rad	0.01760907
contamination peak error rad	0.10528095
final error rad	0.00907646

1. Time constant 1 s with dt=0.01 s gives alpha=0.99009901.
2. True roll is a 0.15 rad, 0.3 Hz sinusoid; gyro bias is 0.01 rad/s.
3. Contaminate the accelerometer roll proxy by +0.15 rad between three and four seconds.
4. Whole-record RMSE is 0.038999 rad; clean-segment RMSE is 0.017609 rad.
5. Peak error during contamination is 0.105281 rad.

Worked case 5: time_constant = 2

This case keeps the nominal configuration except for time_constant=2. The full input is {"time_constant": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
alpha	0.99502488
total rmse rad	0.03870177
clean rmse rad	0.02874565
contamination peak error rad	0.07719958

Computed quantity	Numerical result
final error rad	0.02542635

1. Time constant 2 s with dt=0.01 s gives alpha=0.99502488.
2. True roll is a 0.15 rad, 0.3 Hz sinusoid; gyro bias is 0.01 rad/s.
3. Contaminate the accelerometer roll proxy by +0.15 rad between three and four seconds.
4. Whole-record RMSE is 0.038702 rad; clean-segment RMSE is 0.028746 rad.
5. Peak error during contamination is 0.077200 rad.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The synthetic accelerometer-angle disturbance represents a limitation of gravity-based attitude correction during motion.

A production estimator needs measurement validity checks and a consistent three-dimensional orientation representation.

K. Failure modes and troubleshooting

Treating accelerometer-derived roll as ground truth while accelerating; tuning alpha without regard to sampling period.

L. Practical exercises and extension tasks

1. Gate accelerometer correction when acceleration norm deviates from the stationary reference.
2. Estimate gyro bias as an additional state.
3. Compare a fixed alpha with a fixed time constant under changed sampling rate.

M. Connection to the Asimov research platform

IMU correction assumptions must be tested in supported stationary, moving and contact-transition conditions.

N. Completion record

Record your chosen case, predicted trend, measured alpha, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 34, Scalar Kalman filtering and covariance interpretation. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 34 - Scalar Kalman filtering and covariance interpretation

A. The engineering question

A Kalman filter combines a prediction with a measurement according to their modeled uncertainty.

B. Concepts and mathematical model

The gain is not an arbitrary smoothing coefficient: it emerges from predicted state variance and measurement variance. If those variances are wrong, the numerical covariance can look precise while the estimate is biased or overconfident. This scalar random-walk model provides a complete update cycle and a direct comparison between empirical error and reported uncertainty.

P_pred=P+Q; K=P_pred/(P_pred+R); x=x_pred+K*(z-x_pred); P=(1-K)^2*P_pred+K^2*R.

C. Real-world meaning and limits

Robot estimators require a physically defensible process model and measurement covariances based on actual sensing conditions.

D. Architecture and data flow

Configuration (assumed_measurement_variance) -> numerical model -> estimate_rmse, reported_std, final_gain -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

assumed_measurement_variance is squared measurement units; process variance and actual measurement noise stay fixed.

Parameter	Nominal value	Interpretation
assumed_measurement_variance	0.01	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/34_kalman_scalar.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only assumed_measurement_variance changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/34_kalman_scalar.py --case 0

.venv-book/bin/python expanded_lab/modules/34_kalman_scalar.py --case 3

.venv-book/bin/python expanded_lab/run.py kalman_scalar --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/34_kalman_scalar.py

```
"""Scalar Kalman filtering and covariance interpretation. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(31)
    n = 500
    Q = 0.0001
    true_R = 0.01
    truth = np.cumsum(rng.normal(0, np.sqrt(Q), n))
    measurements = truth + rng.normal(0, np.sqrt(true_R), n)
    x = 0.0
    P = 1.0
    errors = []
    variances = []
    gains = []
    assumed_R = p["assumed_measurement_variance"]
    for z, actual in zip(measurements, truth):
        prediction = P + Q
        K = prediction / (prediction + assumed_R)
        x += K * (z - x)
        P = (1 - K) ** 2 * prediction + K * K * assumed_R
        errors.append(x - actual)
        variances.append(P)
        gains.append(K)
    errors = np.array(errors)
    variances = np.array(variances)
    metrics = dict(
        estimate_rmse=np.sqrt(np.mean(errors**2)),
        reported_std=np.sqrt(np.mean(variances)),
        final_gain=gains[-1],
        empirical_two_sigma_coverage=np.mean(
            abs(errors) <= 2 * np.sqrt(variances)
        ),
    )
    derivation = [
        f"True process variance per step is {Q:g}; true measurement variance is
    {true_R:g}.",
        f"The filter assumes measurement variance {assumed_R:g}.",
        "Predict variance, calculate gain, correct state, then use Joseph-form scalar
    covariance update.",
        f'Empirical RMSE is {metrics["estimate_rmse"]:.6f}; reported RMS standard deviation
    is {metrics["reported_std"]:.6f}.',
        f'Two-sigma empirical coverage is {metrics["empirical_two_sigma_coverage"]:.6f}.',
    ]
    notes = [
        "Coverage is a diagnostic under this synthetic model, not a universal confidence
    guarantee.",
```

```
        "Underestimating measurement variance can increase trust in noisy data and make the
    reported covariance overconfident.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"assumed_measurement_variance": 0.01}
    parameters["assumed_measurement_variance"] = [
        0.0001,
        0.001,
        0.005,
        0.01,
        0.05,
        0.1,
    ][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The Joseph covariance form retains nonnegativity more robustly than a careless subtractive implementation. Compare uncertainty calibration as well as state error.

I. Expected behavior and six worked cases

The controlled variable is assumed_measurement_variance. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

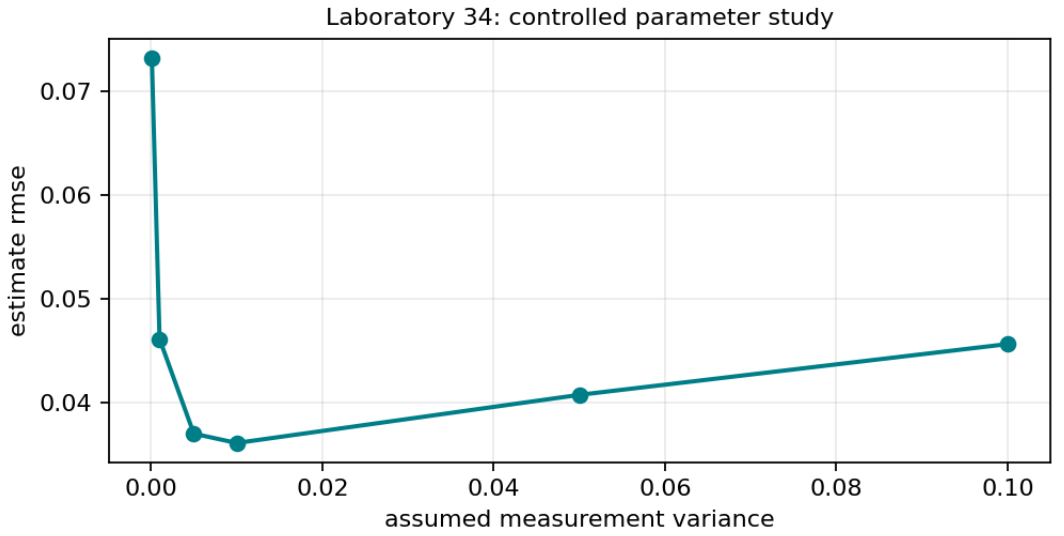


Figure L34. estimate rmse as assumed_measurement_variance changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	assumed_measurement_variance	estimate rmse
0	0.0001	0.07318392
1	0.001	0.04609931
2	0.005	0.03697675
3	0.01	0.03607727
4	0.05	0.0407165
5	0.1	0.04561209

Worked case 0: assumed_measurement_variance = 0.0001

This case keeps the nominal configuration except for assumed_measurement_variance=0.0001. The full input is {"assumed_measurement_variance": 0.0001}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
estimate rmse	0.07318392
reported std	0.00786709
final gain	0.61803399
empirical two sigma coverage	0.172

1. True process variance per step is 0.0001; true measurement variance is 0.01.
2. The filter assumes measurement variance 0.0001.
3. Predict variance, calculate gain, correct state, then use Joseph-form scalar covariance update.
4. Empirical RMSE is 0.073184; reported RMS standard deviation is 0.007867.
5. Two-sigma empirical coverage is 0.172000.

Worked case 1: assumed_measurement_variance = 0.001

This case keeps the nominal configuration except for assumed_measurement_variance=0.001. The full input is {"assumed_measurement_variance": 0.001}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
estimate rmse	0.04609931
reported std	0.01651004
final gain	0.27015621
empirical two sigma coverage	0.486

1. True process variance per step is 0.0001; true measurement variance is 0.01.
2. The filter assumes measurement variance 0.001.

3. Predict variance, calculate gain, correct state, then use Joseph-form scalar covariance update.
4. Empirical RMSE is 0.046099; reported RMS standard deviation is 0.016510.
5. Two-sigma empirical coverage is 0.486000.

Worked case 2: assumed_measurement_variance = 0.005

This case keeps the nominal configuration except for assumed_measurement_variance=0.005. The full input is {"assumed_measurement_variance": 0.005}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
estimate rmse	0.03697675
reported std	0.02603787
final gain	0.13177447
empirical two sigma coverage	0.844

1. True process variance per step is 0.0001; true measurement variance is 0.01.
2. The filter assumes measurement variance 0.005.
3. Predict variance, calculate gain, correct state, then use Joseph-form scalar covariance update.
4. Empirical RMSE is 0.036977; reported RMS standard deviation is 0.026038.
5. Two-sigma empirical coverage is 0.844000.

Worked case 3: assumed_measurement_variance = 0.01

This case keeps the nominal configuration except for assumed_measurement_variance=0.01. The full input is {"assumed_measurement_variance": 0.01}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
estimate rmse	0.03607727
reported std	0.03155553
final gain	0.09512492
empirical two sigma coverage	0.926

1. True process variance per step is 0.0001; true measurement variance is 0.01.
2. The filter assumes measurement variance 0.01.
3. Predict variance, calculate gain, correct state, then use Joseph-form scalar covariance update.
4. Empirical RMSE is 0.036077; reported RMS standard deviation is 0.031556.
5. Two-sigma empirical coverage is 0.926000.

Worked case 4: assumed_measurement_variance = 0.05

This case keeps the nominal configuration except for assumed_measurement_variance=0.05. The full input is {"assumed_measurement_variance": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
estimate rmse	0.0407165
reported std	0.04980345

Computed quantity	Numerical result
final gain	0.04373254
empirical two sigma coverage	0.978

1. True process variance per step is 0.0001; true measurement variance is 0.01.
2. The filter assumes measurement variance 0.05.
3. Predict variance, calculate gain, correct state, then use Joseph-form scalar covariance update.
4. Empirical RMSE is 0.040717; reported RMS standard deviation is 0.049803.
5. Two-sigma empirical coverage is 0.978000.

Worked case 5: assumed_measurement_variance = 0.1

This case keeps the nominal configuration except for assumed_measurement_variance=0.1. The full input is {"assumed_measurement_variance": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
estimate rmse	0.04561209
reported std	0.06126555
final gain	0.03112673
empirical two sigma coverage	0.984

1. True process variance per step is 0.0001; true measurement variance is 0.01.
2. The filter assumes measurement variance 0.1.
3. Predict variance, calculate gain, correct state, then use Joseph-form scalar covariance update.

4. Empirical RMSE is 0.045612; reported RMS standard deviation is 0.061266.
5. Two-sigma empirical coverage is 0.984000.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Coverage is a diagnostic under this synthetic model, not a universal confidence guarantee.

Underestimating measurement variance can increase trust in noisy data and make the reported covariance overconfident.

K. Failure modes and troubleshooting

Using standard deviation where variance is required; confusing measurement residual with state error; assuming diagonal covariance means physical independence.

L. Practical exercises and extension tasks

1. Change actual noise without changing filter assumptions.
2. Add a constant measurement bias and inspect covariance failure.
3. Compute normalized innovations and test their scale.

M. Connection to the Asimov research platform

Robot estimators require a physically defensible process model and measurement covariances based on actual sensing conditions.

N. Completion record

Record your chosen case, predicted trend, measured estimate_rmse, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 35, Position and velocity estimation with intermittent measurements. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 35 - Position and velocity estimation with intermittent measurements

A. The engineering question

Velocity is often not measured directly or is too noisy to use after simple differentiation.

B. Concepts and mathematical model

A constant-velocity Kalman model estimates position and speed together while carrying uncertainty through missing observations. Prediction during a dropout should increase uncertainty; inventing a repeated fresh measurement would hide the lack of information. This laboratory removes a block of position measurements and measures estimation error during that interval.

F=[[1,dt],[0,1]]; H=[1,0]; P_pred=F*P*F.T+Q;
K=P_pred*H.T/(H*P_pred*H.T+R).

C. Real-world meaning and limits

Contact-based velocity updates must be disabled or downweighted when the assumed foot slips or loses load.

D. Architecture and data flow

Configuration (dropout_seconds) -> numerical model -> velocity_rmse_m_s, dropout_velocity_rmse_m_s, final_velocity_m_s -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

dropout_seconds is the duration of missing position observations; measurement standard deviation is 0.01 m.

Parameter	Nominal value	Interpretation
dropout_seconds	0.5	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/35_kalman_velocity.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only dropout_seconds changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/35_kalman_velocity.py --case 0

.venv-book/bin/python expanded_lab/modules/35_kalman_velocity.py --case 3

.venv-book/bin/python expanded_lab/run.py kalman_velocity --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/35_kalman_velocity.py

```
"""Position and velocity estimation with intermittent measurements. Numerical teaching
    experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(37)
    dt = 0.02
    t = np.arange(0, 6, dt)
    true_position = 0.2 * t + 0.02 * np.sin(t)
    true_velocity = 0.2 + 0.02 * np.cos(t)
    measurements = true_position + rng.normal(0, 0.01, len(t))
    F = np.array([[1.0, dt], [0.0, 1.0]])
    H = np.array([[1.0, 0.0]])
    Q = 0.01 * np.array([[dt**4 / 4, dt**3 / 2], [dt**3 / 2, dt**2]])
    x = np.zeros(2)
    P = np.eye(2)
    errors = []
    drop_errors = []
    dropout = p["dropout_seconds"]
    end = 2 + dropout
    for i, z in enumerate(measurements):
        x = F @ x
        P = F @ P @ F.T + Q
        missing = 2 <= t[i] < end
        if not missing:
            S = float((H @ P @ H.T)[0, 0] + 0.0001)
            K = P @ H.T / S
            x += K[:, 0] * (z - float((H @ x)[0]))
            A = np.eye(2) - K @ H
            P = A @ P @ A.T + K * 0.0001 @ K.T
        error = x[1] - true_velocity[i]
        errors.append(error)
        if missing:
            drop_errors.append(error)
        assert np.linalg.eigvalsh(P).min() > -1e-10
    metrics = dict(
        velocity_rmse_m_s=np.sqrt(np.mean(np.array(errors) ** 2)),
        dropout_velocity_rmse_m_s=(
            np.sqrt(np.mean(np.array(drop_errors) ** 2)) if drop_errors else 0.0
        ),
        final_velocity_m_s=x[1],
        final_position_std_m=np.sqrt(P[0, 0]),
        final_velocity_std_m_s=np.sqrt(P[1, 1]),
    )
    derivation = [
        f"Use a two-state position/velocity model with dt={dt:g} s.",
        f"Drop measurements from t=2 s to t={end:g} s.",
        "Predict through missing samples without applying fabricated corrections.",
```

```
        f'Whole-run speed RMSE is {metrics["velocity_rmse_m_s"]:.6f} m/s.',
        f'Dropout speed RMSE is {metrics["dropout_velocity_rmse_m_s"]:.6f} m/s; final speed
    estimate is {x[1]:.6f} m/s.',
    ]
    notes = [
        "The true motion contains a small acceleration that is not exactly constant
    velocity.",
        "The filter state and covariance are expressed in one dimension; full robot base
    estimation requires orientation and contact models.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"dropout_seconds": 0.5}
    parameters["dropout_seconds"] = [0.0, 0.1, 0.25, 0.5, 1.0, 2.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Prediction and correction are separate branches. Inspect covariance positive-semidefiniteness and avoid accidentally updating on stale copies during the dropout.

I. Expected behavior and six worked cases

The controlled variable is dropout_seconds. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

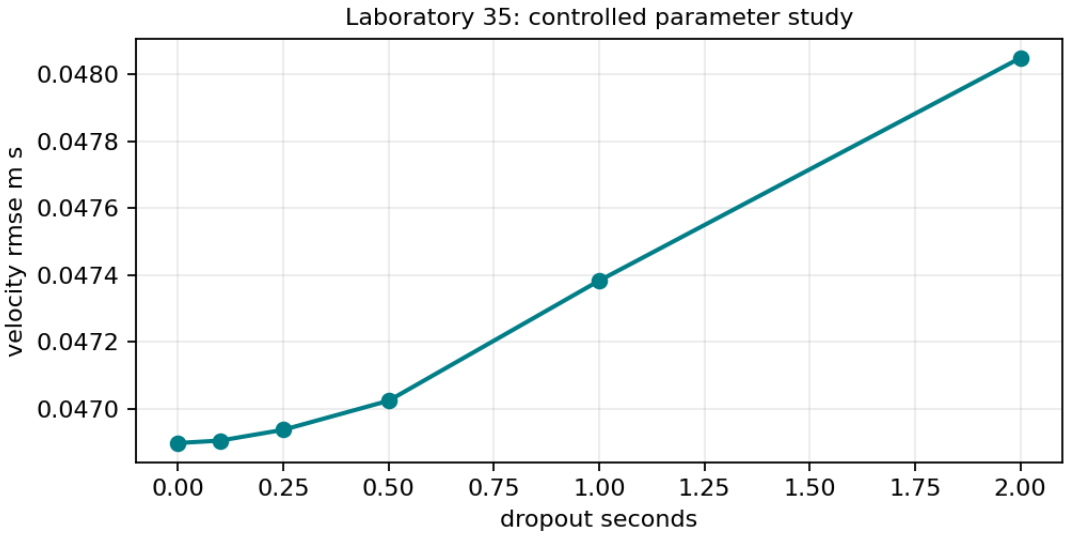


Figure L35. velocity rmse m s as dropout_seconds changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	dropout_seconds	velocity rmse m s
0	0	0.04689866
1	0.1	0.04690584
2	0.25	0.04693812
3	0.5	0.04702495
4	1	0.04738355
5	2	0.04804862

Worked case 0: dropout_seconds = 0

This case keeps the nominal configuration except for dropout_seconds=0. The full input is {"dropout_seconds": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
velocity rmse m s	0.04689866

Computed quantity	Numerical result
dropout velocity rmse m s	0
final velocity m s	0.20728961
final position std m	0.00292494
final velocity std m s	0.00935228

1. Use a two-state position/velocity model with dt=0.02 s.
2. Drop measurements from t=2 s to t=2 s.
3. Predict through missing samples without applying fabricated corrections.
4. Whole-run speed RMSE is 0.046899 m/s.
5. Dropout speed RMSE is 0.000000 m/s; final speed estimate is 0.207290 m/s.

Worked case 1: dropout_seconds = 0.1

This case keeps the nominal configuration except for dropout_seconds=0.1. The full input is {"dropout_seconds": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
velocity rmse m s	0.04690584
dropout velocity rmse m s	0.01198
final velocity m s	0.20729009
final position std m	0.00292494
final velocity std m s	0.00935228

1. Use a two-state position/velocity model with dt=0.02 s.
2. Drop measurements from t=2 s to t=2.1 s.
3. Predict through missing samples without applying fabricated corrections.
4. Whole-run speed RMSE is 0.046906 m/s.

5. Dropout speed RMSE is 0.011980 m/s; final speed estimate is 0.207290 m/s.

Worked case 2: dropout_seconds = 0.25

This case keeps the nominal configuration except for dropout_seconds=0.25. The full input is {"dropout_seconds": 0.25}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
velocity rmse m s	0.04693812
dropout velocity rmse m s	0.01340119
final velocity m s	0.20729138
final position std m	0.00292494
final velocity std m s	0.00935228

1. Use a two-state position/velocity model with dt=0.02 s.
2. Drop measurements from t=2 s to t=2.25 s.
3. Predict through missing samples without applying fabricated corrections.
4. Whole-run speed RMSE is 0.046938 m/s.
5. Dropout speed RMSE is 0.013401 m/s; final speed estimate is 0.207291 m/s.

Worked case 3: dropout_seconds = 0.5

This case keeps the nominal configuration except for dropout_seconds=0.5. The full input is {"dropout_seconds": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
velocity rmse m s	0.04702495
dropout velocity rmse m s	0.01537378

Computed quantity	Numerical result
final velocity m s	0.20729021
final position std m	0.00292494
final velocity std m s	0.00935228

1. Use a two-state position/velocity model with dt=0.02 s.
2. Drop measurements from t=2 s to t=2.5 s.
3. Predict through missing samples without applying fabricated corrections.
4. Whole-run speed RMSE is 0.047025 m/s.
5. Dropout speed RMSE is 0.015374 m/s; final speed estimate is 0.207290 m/s.

Worked case 4: dropout_seconds = 1

This case keeps the nominal configuration except for dropout_seconds=1. The full input is {"dropout_seconds": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
velocity rmse m s	0.04738355
dropout velocity rmse m s	0.01850473
final velocity m s	0.2072862
final position std m	0.00292494
final velocity std m s	0.00935228

1. Use a two-state position/velocity model with dt=0.02 s.
2. Drop measurements from t=2 s to t=3 s.
3. Predict through missing samples without applying fabricated corrections.
4. Whole-run speed RMSE is 0.047384 m/s.

5. Dropout speed RMSE is 0.018505 m/s; final speed estimate is 0.207286 m/s.

Worked case 5: dropout_seconds = 2

This case keeps the nominal configuration except for dropout_seconds=2. The full input is {"dropout_seconds": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
velocity rmse m s	0.04804862
dropout velocity rmse m s	0.01982028
final velocity m s	0.20735177
final position std m	0.0029252
final velocity std m s	0.00935443

1. Use a two-state position/velocity model with dt=0.02 s.
2. Drop measurements from t=2 s to t=4 s.
3. Predict through missing samples without applying fabricated corrections.
4. Whole-run speed RMSE is 0.048049 m/s.
5. Dropout speed RMSE is 0.019820 m/s; final speed estimate is 0.207352 m/s.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The true motion contains a small acceleration that is not exactly constant velocity.

The filter state and covariance are expressed in one dimension; full robot base estimation requires orientation and contact models.

K. Failure modes and troubleshooting

Labeling a held last measurement as newly sampled; measuring velocity truth in a different frame; omitting process noise during unmodeled acceleration.

L. Practical exercises and extension tasks

1. Add timestamp jitter and rebuild F from actual intervals.
2. Inject a sensor jump and add an innovation gate.
3. Compare with differentiated noisy position.

M. Connection to the Asimov research platform

Contact-based velocity updates must be disabled or downweighted when the assumed foot slips or loses load.

N. Completion record

Record your chosen case, predicted trend, measured velocity_rmse_m_s, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 36, Camera intrinsics, projection and pixel-error sensitivity. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 36 - Camera intrinsics, projection and pixel-error sensitivity

A. The engineering question

A camera converts spatial geometry into pixel measurements.

B. Concepts and mathematical model

Focal lengths are measured in pixels after calibration; principal point is the pixel location of the optical axis. Projection divides by depth, making near-zero or negative depth invalid for a simple visible-point model. Pixel error maps back into metric lateral error in proportion to depth. This laboratory computes a projection and an inverse reconstruction with known depth, making the missing-depth assumption explicit.

u=fx*X/Z+cx; v=fy*Y/Z+cy; X=(u-cx)*Z/fx; lateral uncertainty approx Z*sigma_pixel/fx.

C. Real-world meaning and limits

The documented Asimov monocular camera does not by itself provide the known depth used in this numerical exercise.

D. Architecture and data flow

Configuration (depth, pixel_noise) -> numerical model -> pixel_u, pixel_v, lateral_uncertainty_m -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

depth is positive optical-axis meters; pixel_noise is assumed image-coordinate standard deviation in pixels.

Parameter	Nominal value	Interpretation
depth	1	Defined by the explicit chapter model and parameter contract

Parameter	Nominal value	Interpretation
pixel_noise	1	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/36_pinhole.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only depth changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/36_pinhole.py --case 0

.venv-book/bin/python expanded_lab/modules/36_pinhole.py --case 3

.venv-book/bin/python expanded_lab/run.py pinhole --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/36_pinhole.py

```
"""Camera intrinsics, projection and pixel-error sensitivity. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    Z = p["depth"]
    fx, fy, cx, cy = 400.0, 410.0, 320.0, 240.0
    X, Y = 0.1, -0.05
    sigma = p["pixel_noise"]
    assert Z > 0
    u = fx * X / Z + cx
    v = fy * Y / Z + cy
    reconstructed = np.array([(u - cx) * Z / fx, (v - cy) * Z / fy, Z])
    error = np.linalg.norm(reconstructed - [X, Y, Z])
    lateral = Z * sigma / fx
    metrics = dict(
        pixel_u=u,
        pixel_v=v,
        lateral_uncertainty_m=lateral,
        roundtrip_error_m=error,
        projection_scale_px_per_m=fx / Z,
    )
    derivation = [
        f"Use fx={fx:g}, fy={fy:g}, cx={cx:g}, cy={cy:g} pixels.",
        f"Project point (0.1,-0.05,{Z:g}) m.",
        f"Pixel result is u={u:.6f}, v={v:.6f}.",
        f"Assuming exact known depth, inverse projection returns {reconstructed.tolist()}
    m.",
        f"Pixel noise sigma={sigma:g} px maps to lateral sigma approximately {lateral:.6f}
    m.",
    ]
    notes = [
        "The inverse result requires known depth; a monocular pixel alone does not identify
    a metric 3-D point.",
        "Distortion, rolling shutter, calibration uncertainty and camera-to-body extrinsics
    are excluded here.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"depth": 1.0, "pixel_noise": 1.0}
    parameters["depth"] = [0.2, 0.5, 1.0, 2.0, 3.0, 5.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
```

```
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Perform a projection/inverse round trip before adding noise. The uncertainty scaling then isolates geometric depth sensitivity from calibration and detector accuracy.

I. Expected behavior and six worked cases

The controlled variable is depth. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

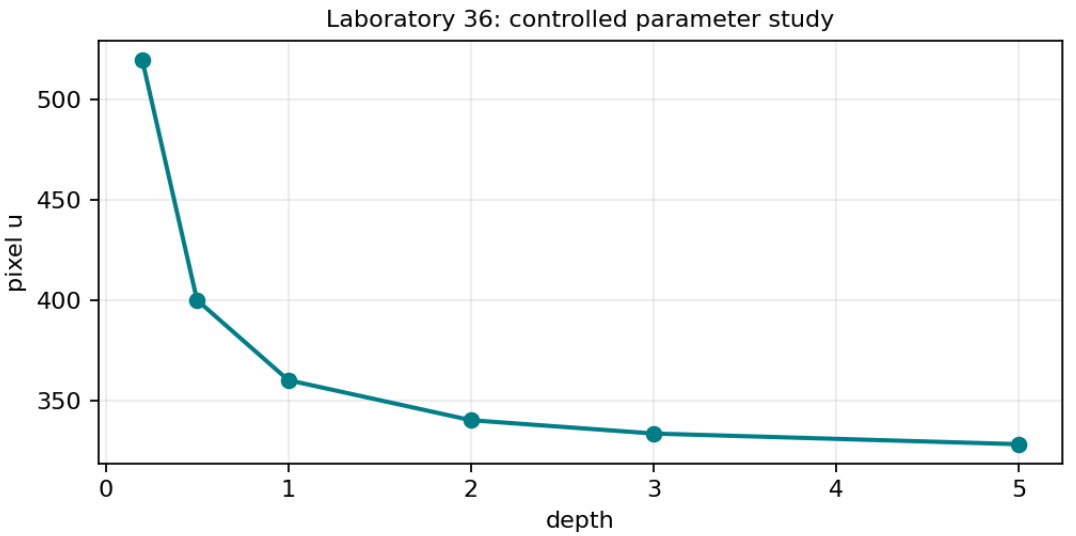


Figure L36. pixel u as depth changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	depth	pixel u
0	0.2	520
1	0.5	400

Case	depth	pixel u
2	1	360
3	2	340
4	3	333.33333333
5	5	328

Worked case 0: depth = 0.2

This case keeps the nominal configuration except for depth=0.2. The full input is {"depth": 0.2, "pixel_noise": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
pixel u	520
pixel v	137.5
lateral uncertainty m	0.0005
roundtrip error m	0
projection scale px per m	2000

1. Use fx=400, fy=410, cx=320, cy=240 pixels.
2. Project point (0.1,-0.05,0.2) m.
3. Pixel result is u=520.000000, v=137.500000.
4. Assuming exact known depth, inverse projection returns [0.1, -0.05, 0.2] m.
5. Pixel noise sigma=1 px maps to lateral sigma approximately 0.000500 m.

Worked case 1: depth = 0.5

This case keeps the nominal configuration except for depth=0.5. The full input is {"depth": 0.5, "pixel_noise": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
pixel u	400
pixel v	199
lateral uncertainty m	0.00125
roundtrip error m	0
projection scale px per m	800

1. Use fx=400, fy=410, cx=320, cy=240 pixels.
2. Project point (0.1,-0.05,0.5) m.
3. Pixel result is u=400.000000, v=199.000000.
4. Assuming exact known depth, inverse projection returns [0.1, -0.05, 0.5] m.
5. Pixel noise sigma=1 px maps to lateral sigma approximately 0.001250 m.

Worked case 2: depth = 1

This case keeps the nominal configuration except for depth=1. The full input is {"depth": 1.0, "pixel_noise": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
pixel u	360
pixel v	219.5
lateral uncertainty m	0.0025
roundtrip error m	0

Computed quantity	Numerical result
projection scale px per m	400

1. Use fx=400, fy=410, cx=320, cy=240 pixels.
2. Project point (0.1,-0.05,1) m.
3. Pixel result is u=360.000000, v=219.500000.
4. Assuming exact known depth, inverse projection returns [0.1, -0.05, 1.0] m.
5. Pixel noise sigma=1 px maps to lateral sigma approximately 0.002500 m.

Worked case 3: depth = 2

This case keeps the nominal configuration except for depth=2. The full input is {"depth": 2.0, "pixel_noise": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
pixel u	340
pixel v	229.75
lateral uncertainty m	0.005
roundtrip error m	0
projection scale px per m	200

1. Use fx=400, fy=410, cx=320, cy=240 pixels.
2. Project point (0.1,-0.05,2) m.
3. Pixel result is u=340.000000, v=229.750000.
4. Assuming exact known depth, inverse projection returns [0.1, -0.05, 2.0] m.
5. Pixel noise sigma=1 px maps to lateral sigma approximately 0.005000 m.

Worked case 4: depth = 3

This case keeps the nominal configuration except for depth=3. The full input is {"depth": 3.0, "pixel_noise": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
pixel u	333.33333333
pixel v	233.16666667
lateral uncertainty m	0.0075
roundtrip error m	1.6768595e-16
projection scale px per m	133.33333333

1. Use fx=400, fy=410, cx=320, cy=240 pixels.
2. Project point (0.1,-0.05,3) m.
3. Pixel result is u=333.333333, v=233.166667.
4. Assuming exact known depth, inverse projection returns [0.09999999999999985, -0.05000000000000007, 3.0] m.
5. Pixel noise sigma=1 px maps to lateral sigma approximately 0.007500 m.

Worked case 5: depth = 5

This case keeps the nominal configuration except for depth=5. The full input is {"depth": 5.0, "pixel_noise": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
pixel u	328
pixel v	235.9
lateral uncertainty m	0.0125
roundtrip error m	6.9388939e-17
projection scale px per m	80

1. Use fx=400, fy=410, cx=320, cy=240 pixels.
2. Project point (0.1,-0.05,5) m.
3. Pixel result is u=328.000000, v=235.900000.
4. Assuming exact known depth, inverse projection returns [0.1, -0.04999999999999993, 5.0] m.
5. Pixel noise sigma=1 px maps to lateral sigma approximately 0.012500 m.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The inverse result requires known depth; a monocular pixel alone does not identify a metric 3-D point.

Distortion, rolling shutter, calibration uncertainty and camera-to-body extrinsics are excluded here.

K. Failure modes and troubleshooting

Using image width as focal length without calibration; assuming Euclidean range equals optical-axis depth; mixing RGB camera and depth camera frames.

L. Practical exercises and extension tasks

1. Propagate focal-length uncertainty in addition to pixel noise.
2. Add a calibrated camera-to-base transform.
3. Reject points behind the camera and near the projection singularity.

M. Connection to the Asimov research platform

The documented Asimov monocular camera does not by itself provide the known depth used in this numerical exercise.

N. Completion record

Record your chosen case, predicted trend, measured pixel_u, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 37, Stereo depth, disparity uncertainty and range limits. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 37 - Stereo depth, disparity uncertainty and range limits

A. The engineering question

Stereo geometry can estimate depth from a calibrated baseline and image disparity.

B. Concepts and mathematical model

Small disparity at long range makes depth highly sensitive to pixel error. Thus a detector may identify an object reliably while its metric position is too uncertain for reaching. The first-order uncertainty calculation is informative near a well-behaved measurement, but poor disparity and outliers need explicit rejection. This is a generic optional sensor study, not a claim that the base Asimov includes stereo depth.

Z=f*B/d; sigma_Z approx f*B*sigma_d/d^2; relative depth uncertainty=sigma_d/d.

C. Real-world meaning and limits

Adding a depth sensor changes mounting, compute, power and calibration budgets and requires explicit compatibility verification.

D. Architecture and data flow

Configuration (true_depth, disparity_noise) -> numerical model -> disparity_px, depth_std_m, relative_depth_std -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

true_depth is meters; disparity_noise is pixels; baseline and focal length are fixed calibrated teaching values.

Parameter	Nominal value	Interpretation
true_depth	1	Defined by the explicit chapter model and parameter contract
disparity_noise	0.5	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/37_stereo_depth.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only true_depth changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/37_stereo_depth.py --case 0

.venv-book/bin/python expanded_lab/modules/37_stereo_depth.py --case 3

.venv-book/bin/python expanded_lab/run.py stereo_depth --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/37_stereo_depth.py

```
"""Stereo depth, disparity uncertainty and range limits. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    Z = p["true_depth"]
    f, B = 400.0, 0.06
    sigma = p["disparity_noise"]
    disparity = f * B / Z
    std = f * B * sigma / disparity**2
    noisy_depth = f * B / (disparity + sigma)
    metrics = dict(
        disparity_px=disparity,
        depth_std_m=std,
        relative_depth_std=std / Z,
        one_sigma_positive_disparity_depth_m=noisy_depth,
        linearization_error_m=abs((Z - std) - noisy_depth),
    )
    derivation = [
        f"Use focal length {f:g} px and calibrated baseline {B:g} m.",
        f"At true depth {Z:g} m, expected disparity is {disparity:.6f} px.",
        f"Disparity sigma={sigma:g} px propagates to sigma_Z={std:.6f} m.",
        f"A +1 sigma disparity perturbation gives exact depth {noisy_depth:.6f} m.",
        f'Compare the first-order prediction Z-sigma_Z; linearization discrepancy is
    {metrics["linearization_error_m"]:.6f} m.',
    ]
    notes = [
        "Depth uncertainty grows quadratically with range in this fixed-camera
    approximation.",
        "When disparity approaches its uncertainty scale, Gaussian linearization and naive
    depth estimates become unreliable.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"true_depth": 1.0, "disparity_noise": 0.5}
    parameters["true_depth"] = [0.3, 0.5, 1.0, 2.0, 3.0, 5.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
```

```
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Compare exact perturbed geometry with a derivative-based approximation. This reveals where uncertainty propagation itself needs a nonlinear or robust treatment.

I. Expected behavior and six worked cases

The controlled variable is true_depth. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

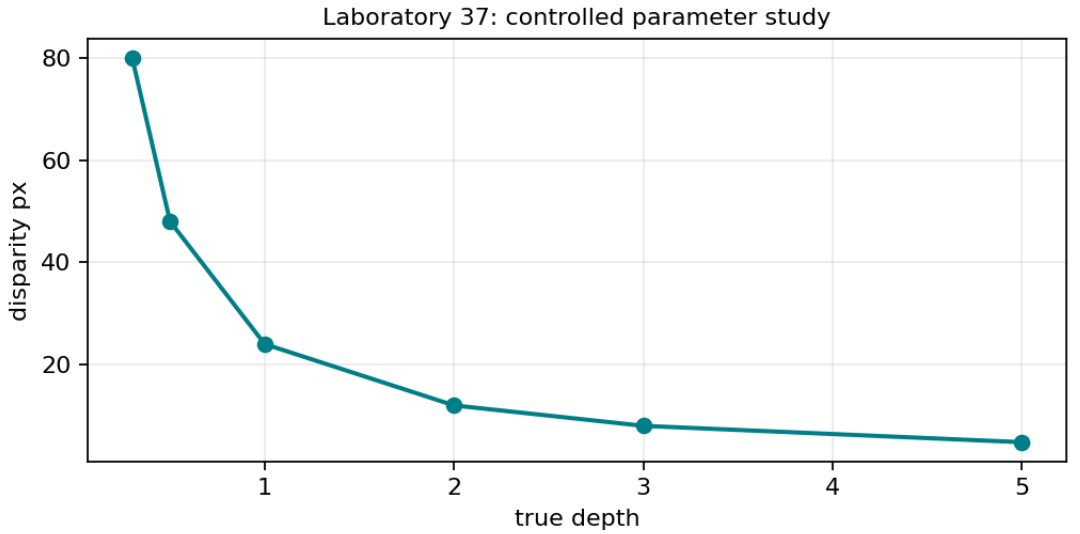


Figure L37. disparity px as true_depth changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	true_depth	disparity px
0	0.3	80
1	0.5	48
2	1	24
3	2	12

Case	true_depth	disparity px
4	3	8
5	5	4.8

Worked case 0: true_depth = 0.3

This case keeps the nominal configuration except for true_depth=0.3. The full input is {"disparity_noise": 0.5, "true_depth": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
disparity px	80
depth std m	0.001875
relative depth std	0.00625
one sigma positive disparity depth m	0.29813665
linearization error m	0.00001165

1. Use focal length 400 px and calibrated baseline 0.06 m.
2. At true depth 0.3 m, expected disparity is 80.000000 px.
3. Disparity sigma=0.5 px propagates to sigma_Z=0.001875 m.
4. A +1 sigma disparity perturbation gives exact depth 0.298137 m.
5. Compare the first-order prediction Z-sigma_Z; linearization discrepancy is 0.000012 m.

Worked case 1: true_depth = 0.5

This case keeps the nominal configuration except for true_depth=0.5. The full input is {"disparity_noise": 0.5, "true_depth": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
disparity px	48
depth std m	0.00520833
relative depth std	0.01041667
one sigma positive disparity depth m	0.49484536
linearization error m	0.00005369

1. Use focal length 400 px and calibrated baseline 0.06 m.
2. At true depth 0.5 m, expected disparity is 48.000000 px.
3. Disparity sigma=0.5 px propagates to sigma_Z=0.005208 m.
4. A +1 sigma disparity perturbation gives exact depth 0.494845 m.
5. Compare the first-order prediction Z-sigma_Z; linearization discrepancy is 0.000054 m.

Worked case 2: true_depth = 1

This case keeps the nominal configuration except for true_depth=1. The full input is {"disparity_noise": 0.5, "true_depth": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
disparity px	24
depth std m	0.02083333
relative depth std	0.02083333
one sigma positive disparity depth m	0.97959184
linearization error m	0.00042517

1. Use focal length 400 px and calibrated baseline 0.06 m.
2. At true depth 1 m, expected disparity is 24.000000 px.
3. Disparity sigma=0.5 px propagates to sigma_Z=0.020833 m.

4. A +1 sigma disparity perturbation gives exact depth 0.979592 m.
5. Compare the first-order prediction Z-sigma_Z; linearization discrepancy is 0.000425 m.

Worked case 3: true_depth = 2

This case keeps the nominal configuration except for true_depth=2. The full input is {"disparity_noise": 0.5, "true_depth": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
disparity px	12
depth std m	0.08333333
relative depth std	0.04166667
one sigma positive disparity depth m	1.92
linearization error m	0.00333333

1. Use focal length 400 px and calibrated baseline 0.06 m.
2. At true depth 2 m, expected disparity is 12.000000 px.
3. Disparity sigma=0.5 px propagates to sigma_Z=0.083333 m.
4. A +1 sigma disparity perturbation gives exact depth 1.920000 m.
5. Compare the first-order prediction Z-sigma_Z; linearization discrepancy is 0.003333 m.

Worked case 4: true_depth = 3

This case keeps the nominal configuration except for true_depth=3. The full input is {"disparity_noise": 0.5, "true_depth": 3.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
disparity px	8

Computed quantity	Numerical result
depth std m	0.1875
relative depth std	0.0625
one sigma positive disparity depth m	2.82352941
linearization error m	0.01102941

1. Use focal length 400 px and calibrated baseline 0.06 m.
2. At true depth 3 m, expected disparity is 8.000000 px.
3. Disparity sigma=0.5 px propagates to sigma_Z=0.187500 m.
4. A +1 sigma disparity perturbation gives exact depth 2.823529 m.
5. Compare the first-order prediction Z-sigma_Z; linearization discrepancy is 0.011029 m.

Worked case 5: true_depth = 5

This case keeps the nominal configuration except for true_depth=5. The full input is {"disparity_noise": 0.5, "true_depth": 5.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
disparity px	4.8
depth std m	0.52083333
relative depth std	0.10416667
one sigma positive disparity depth m	4.52830189
linearization error m	0.04913522

1. Use focal length 400 px and calibrated baseline 0.06 m.
2. At true depth 5 m, expected disparity is 4.800000 px.
3. Disparity sigma=0.5 px propagates to sigma_Z=0.520833 m.
4. A +1 sigma disparity perturbation gives exact depth 4.528302 m.

5. Compare the first-order prediction Z-sigma_Z; linearization discrepancy is 0.049135 m.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Depth uncertainty grows quadratically with range in this fixed-camera approximation.

When disparity approaches its uncertainty scale, Gaussian linearization and naive depth estimates become unreliable.

K. Failure modes and troubleshooting

Assuming a precise-looking depth number has low uncertainty; ignoring rectification and synchronization; treating disparity outliers as ordinary noise.

L. Practical exercises and extension tasks

1. Define a maximum permitted relative-depth uncertainty and calculate usable range.
2. Monte Carlo disparity noise and inspect the depth distribution.
3. Combine depth and camera-extrinsic uncertainty in base coordinates.

M. Connection to the Asimov research platform

Adding a depth sensor changes mounting, compute, power and calibration budgets and requires explicit compatibility verification.

N. Completion record

Record your chosen case, predicted trend, measured disparity_px, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 38, Timestamps, interpolation and asynchronous sensor streams. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 38 - Timestamps, interpolation and asynchronous sensor streams

A. The engineering question

Sensors rarely produce measurements at identical times.

B. Concepts and mathematical model

Comparing an encoder sample at one instant with an IMU sample at another creates a synchronization error even if both values are individually accurate. Interpolation can align streams for analysis, but it needs timestamp validity and does not recover arbitrarily fast unseen motion. This laboratory samples a known signal with a clock offset and compares naive alignment with timestamp-aware interpolation.

y_aligned(t_ref)=interpolate((t_sensor+offset,y_sensor),t_ref); error depends on signal slope, offset and sampling interval.

C. Real-world meaning and limits

Time alignment affects state estimation, camera localization and replay; document the clock source of every recorded field.

D. Architecture and data flow

Configuration (offset, sample_rate) -> numerical model -> naive_alignment_rmse, corrected_alignment_rmse, nominal_phase_error_rad -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

offset is seconds added to the actual sampling instant; sample_rate is Hertz; reference timestamps remain independent.

Parameter	Nominal value	Interpretation
offset	0.01	meters
sample_rate	100	Hertz

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/38_resampling.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only offset changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/38_resampling.py --case 0

.venv-book/bin/python expanded_lab/modules/38_resampling.py --case 3

.venv-book/bin/python expanded_lab/run.py resampling --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/38_resampling.py

```
"""Timestamps, interpolation and asynchronous sensor streams. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    offset = p["offset"]
    fs = p["sample_rate"]
    frequency = 2.0
    reference_t = np.arange(0.2, 2.8, 0.005)
    sensor_t = np.arange(0, 3, 1 / fs)
    measured = np.sin(2 * np.pi * frequency * (sensor_t + offset))
    truth = np.sin(2 * np.pi * frequency * reference_t)
    naive = np.interp(reference_t, sensor_t, measured)
    corrected = np.interp(reference_t, sensor_t + offset, measured)
    metrics = dict(
        naive_alignment_rmse=np.sqrt(np.mean((naive - truth) ** 2)),
        corrected_alignment_rmse=np.sqrt(np.mean((corrected - truth) ** 2)),
        nominal_phase_error_rad=2 * np.pi * frequency * offset,
        sensor_interval_s=1 / fs,
    )
    derivation = [
        f"The signal is a 2 Hz sinusoid sampled at {fs:g} Hz.",
        f"Sensor timestamp offset is {offset:g} s, equivalent to phase
    {2*np.pi*frequency*offset:.6f} rad.",
        "Naive interpolation treats nominal sensor times as correct.",
        f'Naive RMSE is {metrics["naive_alignment_rmse"]:.6f}.',
        f'Adding the known offset before interpolation reduces RMSE to
    {metrics["corrected_alignment_rmse"]:.6f}.',
    ]
    notes = [
        "Residual after offset correction includes interpolation error from finite
    sampling.",
        "An unknown or drifting clock offset requires estimation and validity limits;
    arrival time is not a substitute for sample time.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"offset": 0.01, "sample_rate": 100.0}
    parameters["offset"] = [0.0, 0.001, 0.005, 0.01, 0.02, 0.05][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
```

```
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The synthetic signal is evaluated at actual sensor instants so the offset has a precise meaning. Correcting timestamps rather than shifting values preserves the measurement model.

I. Expected behavior and six worked cases

The controlled variable is offset. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

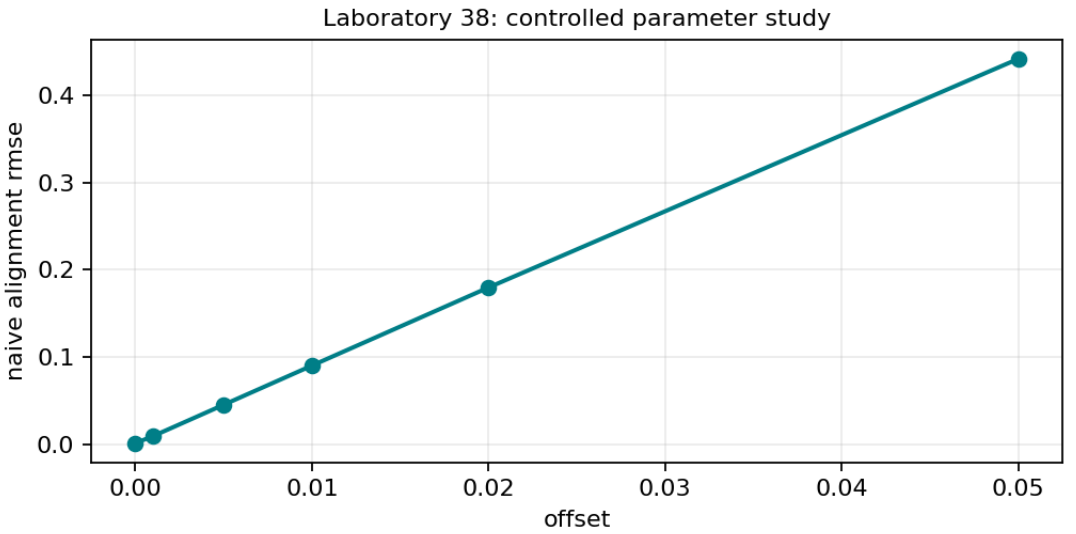


Figure L38. naive alignment rmse as offset changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	offset	naive alignment rmse
0	0	0.00097213
1	0.001	0.00906162
2	0.005	0.04505083

Case	offset	naive alignment rmse
3	0.01	0.09003897
4	0.02	0.17967479
5	0.05	0.44212903

Worked case 0: offset = 0

This case keeps the nominal configuration except for offset=0. The full input is {"offset": 0.0, "sample_rate": 100.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
naive alignment rmse	0.00097213
corrected alignment rmse	0.00097213
nominal phase error rad	0
sensor interval s	0.01

1. The signal is a 2 Hz sinusoid sampled at 100 Hz.
2. Sensor timestamp offset is 0 s, equivalent to phase 0.000000 rad.
3. Naive interpolation treats nominal sensor times as correct.
4. Naive RMSE is 0.000972.
5. Adding the known offset before interpolation reduces RMSE to 0.000972.

Worked case 1: offset = 0.001

This case keeps the nominal configuration except for offset=0.001. The full input is {"offset": 0.001, "sample_rate": 100.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
naive alignment rmse	0.00906162

Computed quantity	Numerical result
corrected alignment rmse	0.00099673
nominal phase error rad	0.01256637
sensor interval s	0.01

1. The signal is a 2 Hz sinusoid sampled at 100 Hz.
2. Sensor timestamp offset is 0.001 s, equivalent to phase 0.012566 rad.
3. Naive interpolation treats nominal sensor times as correct.
4. Naive RMSE is 0.009062.
5. Adding the known offset before interpolation reduces RMSE to 0.000997.

Worked case 2: offset = 0.005

This case keeps the nominal configuration except for offset=0.005. The full input is {"offset": 0.005, "sample_rate": 100.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
naive alignment rmse	0.04505083
corrected alignment rmse	0.00097225
nominal phase error rad	0.06283185
sensor interval s	0.01

1. The signal is a 2 Hz sinusoid sampled at 100 Hz.
2. Sensor timestamp offset is 0.005 s, equivalent to phase 0.062832 rad.
3. Naive interpolation treats nominal sensor times as correct.
4. Naive RMSE is 0.045051.
5. Adding the known offset before interpolation reduces RMSE to 0.000972.

Worked case 3: offset = 0.01

This case keeps the nominal configuration except for offset=0.01. The full input is {"offset": 0.01, "sample_rate": 100.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
naive alignment rmse	0.09003897
corrected alignment rmse	0.00097213
nominal phase error rad	0.12566371
sensor interval s	0.01

1. The signal is a 2 Hz sinusoid sampled at 100 Hz.
2. Sensor timestamp offset is 0.01 s, equivalent to phase 0.125664 rad.
3. Naive interpolation treats nominal sensor times as correct.
4. Naive RMSE is 0.090039.
5. Adding the known offset before interpolation reduces RMSE to 0.000972.

Worked case 4: offset = 0.02

This case keeps the nominal configuration except for offset=0.02. The full input is {"offset": 0.02, "sample_rate": 100.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
naive alignment rmse	0.17967479
corrected alignment rmse	0.00097213
nominal phase error rad	0.25132741
sensor interval s	0.01

1. The signal is a 2 Hz sinusoid sampled at 100 Hz.

2. Sensor timestamp offset is 0.02 s, equivalent to phase 0.251327 rad.
3. Naive interpolation treats nominal sensor times as correct.
4. Naive RMSE is 0.179675.
5. Adding the known offset before interpolation reduces RMSE to 0.000972.

Worked case 5: offset = 0.05

This case keeps the nominal configuration except for offset=0.05. The full input is {"offset": 0.05, "sample_rate": 100.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
naive alignment rmse	0.44212903
corrected alignment rmse	0.00097213
nominal phase error rad	0.62831853
sensor interval s	0.01

1. The signal is a 2 Hz sinusoid sampled at 100 Hz.
2. Sensor timestamp offset is 0.05 s, equivalent to phase 0.628319 rad.
3. Naive interpolation treats nominal sensor times as correct.
4. Naive RMSE is 0.442129.
5. Adding the known offset before interpolation reduces RMSE to 0.000972.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Residual after offset correction includes interpolation error from finite sampling.

An unknown or drifting clock offset requires estimation and validity limits; arrival time is not a substitute for sample time.

K. Failure modes and troubleshooting

Interpolating across long dropouts; subtracting unsynchronized host clocks; using future samples in a real-time algorithm without accounting for latency.

L. Practical exercises and extension tasks

1. Add linear clock drift and estimate it from matched events.
2. Mark interpolation across large gaps invalid.
3. Compare causal hold, linear interpolation and delayed smoothing.

M. Connection to the Asimov research platform

Time alignment affects state estimation, camera localization and replay; document the clock source of every recorded field.

N. Completion record

Record your chosen case, predicted trend, measured naive_alignment_rmse, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 39, Contact detection, thresholds and hysteresis. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 39 - Contact detection, thresholds and hysteresis

A. The engineering question

A noisy force-like signal can chatter around a single contact threshold, repeatedly changing the controller's assumed support set.

B. Concepts and mathematical model

Hysteresis uses a higher entry threshold and lower exit threshold, creating memory that reduces switching. It also delays transitions and can miss weak contacts. Evaluate detection against labeled conditions, not merely a smoother-looking plot. This experiment uses synthetic contact loads with noise and reports false positives, false negatives and transition count.

if not_contact and load>=enter: contact=True; if contact and load<=exit: contact=False; exit<enter.

C. Real-world meaning and limits

If physical force sensors are absent, contact estimation needs a separately validated model and uncertainty treatment.

D. Architecture and data flow

Configuration (noise_std) -> numerical model -> false_positive_fraction, false_negative_fraction, transition_count -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

noise_std is Newtons of synthetic zero-mean noise; the true supported load is 30 N during the labeled interval.

Parameter	Nominal value	Interpretation
noise_std	5	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/39_contact_hysteresis.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only noise_std changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/39_contact_hysteresis.py --case 0

.venv-book/bin/python expanded_lab/modules/39_contact_hysteresis.py --case 3

.venv-book/bin/python expanded_lab/run.py contact_hysteresis --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/39_contact_hysteresis.py

```
"""Contact detection, thresholds and hysteresis. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(41)
    t = np.arange(0, 4, 0.01)
    truth = (t >= 1) & (t < 3)
    signal = np.where(truth, 30.0, 0.0)
    signal = signal + rng.normal(0, p["noise_std"], len(t))
    enter, exit = 15.0, 5.0
    state = False
    predicted = []
    for value in signal:
        if not state and value >= enter:
            state = True
        elif state and value <= exit:
            state = False
        predicted.append(state)
    predicted = np.array(predicted)
    false_positive = np.mean(predicted[~truth])
    false_negative = np.mean(~predicted[truth])
    switches = np.count_nonzero(predicted[1:] != predicted[:-1])
    metrics = dict(
        false_positive_fraction=false_positive,
        false_negative_fraction=false_negative,
        transition_count=switches,
        accuracy=np.mean(predicted == truth),
        signal_noise_std_N=p["noise_std"],
    )
    derivation = [
        "The labeled contact interval is one to three seconds with mean load 30 N.",
        f'Add Gaussian noise with sigma={p["noise_std"]:g} N.',
        "Enter contact at 15 N and leave at 5 N using a stateful threshold.",
        f"False-positive fraction is {false_positive:.6f}; false-negative fraction is
    {false_negative:.6f}.",
        f"The detector made {switches} state transitions instead of the ideal two.",
    ]
    notes = [
        "Thresholds are teaching values; a real detector must be calibrated for sensor
    range, preload, bias and contact dynamics.",
        "A stable contact flag does not prove no slip or a known contact wrench.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
```

```
args = parser.parse_args()
parameters = {"noise_std": 5.0}
parameters["noise_std"] = [0.5, 2.0, 5.0, 8.0, 12.0, 20.0][args.case]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The detector carries state between samples. Calculate errors separately inside and outside contact; an always-contact classifier can look acceptable if the dataset is imbalanced.

I. Expected behavior and six worked cases

The controlled variable is noise_std. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

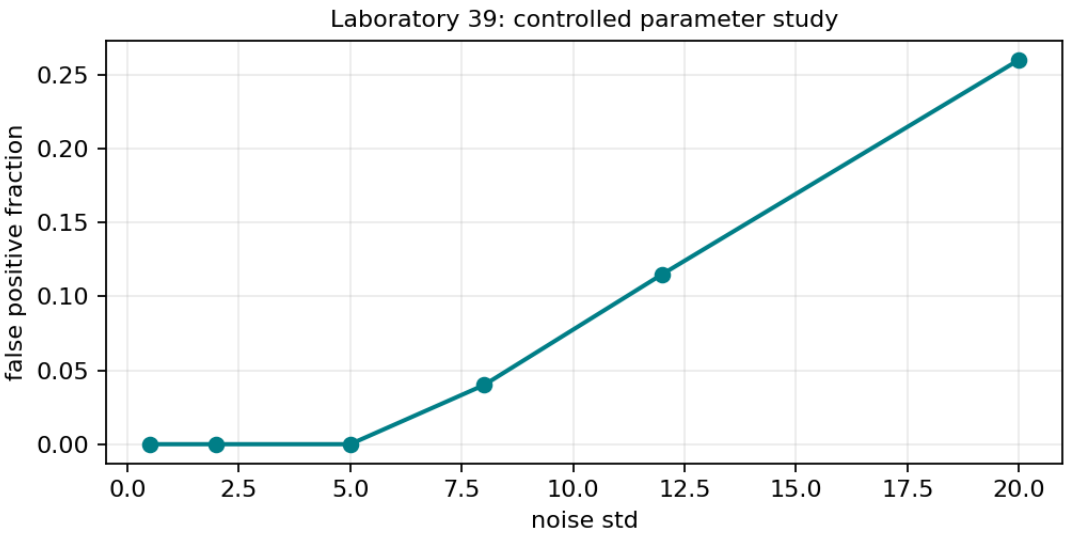


Figure L39. false positive fraction as noise_std changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	noise_std	false positive fraction
0	0.5	0
1	2	0
2	5	0
3	8	0.04
4	12	0.115
5	20	0.26

Worked case 0: noise_std = 0.5

This case keeps the nominal configuration except for noise_std=0.5. The full input is {"noise_std": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
false positive fraction	0
false negative fraction	0
transition count	2
accuracy	1
signal noise std N	0.5

1. The labeled contact interval is one to three seconds with mean load 30 N.
2. Add Gaussian noise with sigma=0.5 N.
3. Enter contact at 15 N and leave at 5 N using a stateful threshold.
4. False-positive fraction is 0.000000; false-negative fraction is 0.000000.
5. The detector made 2 state transitions instead of the ideal two.

Worked case 1: noise_std = 2

This case keeps the nominal configuration except for noise_std=2. The full input is {"noise_std": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
false positive fraction	0
false negative fraction	0
transition count	2
accuracy	1
signal noise std N	2

1. The labeled contact interval is one to three seconds with mean load 30 N.
2. Add Gaussian noise with sigma=2 N.
3. Enter contact at 15 N and leave at 5 N using a stateful threshold.
4. False-positive fraction is 0.000000; false-negative fraction is 0.000000.
5. The detector made 2 state transitions instead of the ideal two.

Worked case 2: noise_std = 5

This case keeps the nominal configuration except for noise_std=5. The full input is {"noise_std": 5.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
false positive fraction	0
false negative fraction	0
transition count	2
accuracy	1
signal noise std N	5

1. The labeled contact interval is one to three seconds with mean load 30 N.

2. Add Gaussian noise with sigma=5 N.
3. Enter contact at 15 N and leave at 5 N using a stateful threshold.
4. False-positive fraction is 0.000000; false-negative fraction is 0.000000.
5. The detector made 2 state transitions instead of the ideal two.

Worked case 3: noise_std = 8

This case keeps the nominal configuration except for noise_std=8. The full input is {"noise_std": 8.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
false positive fraction	0.04
false negative fraction	0.005
transition count	16
accuracy	0.9775
signal noise std N	8

1. The labeled contact interval is one to three seconds with mean load 30 N.
2. Add Gaussian noise with sigma=8 N.
3. Enter contact at 15 N and leave at 5 N using a stateful threshold.
4. False-positive fraction is 0.040000; false-negative fraction is 0.005000.
5. The detector made 16 state transitions instead of the ideal two.

Worked case 4: noise_std = 12

This case keeps the nominal configuration except for noise_std=12. The full input is {"noise_std": 12.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
false positive fraction	0.115
false negative fraction	0.03

Computed quantity	Numerical result
transition count	46
accuracy	0.9275
signal noise std N	12

1. The labeled contact interval is one to three seconds with mean load 30 N.
2. Add Gaussian noise with sigma=12 N.
3. Enter contact at 15 N and leave at 5 N using a stateful threshold.
4. False-positive fraction is 0.115000; false-negative fraction is 0.030000.
5. The detector made 46 state transitions instead of the ideal two.

Worked case 5: noise_std = 20

This case keeps the nominal configuration except for noise_std=20. The full input is {"noise_std": 20.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
false positive fraction	0.26
false negative fraction	0.11
transition count	92
accuracy	0.815
signal noise std N	20

1. The labeled contact interval is one to three seconds with mean load 30 N.
2. Add Gaussian noise with sigma=20 N.
3. Enter contact at 15 N and leave at 5 N using a stateful threshold.
4. False-positive fraction is 0.260000; false-negative fraction is 0.110000.
5. The detector made 92 state transitions instead of the ideal two.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Thresholds are teaching values; a real detector must be calibrated for sensor range, preload, bias and contact dynamics.

A stable contact flag does not prove no slip or a known contact wrench.

K. Failure modes and troubleshooting

Treating a threshold flag as load measurement; choosing thresholds on the same trials used for final evaluation; ignoring sensor bias.

L. Practical exercises and extension tasks

1. Compare with a single threshold and count chatter.
2. Add a minimum-duration debounce and measure latency.
3. Introduce slowly increasing preload and test failure behavior.

M. Connection to the Asimov research platform

If physical force sensors are absent, contact estimation needs a separately validated model and uncertainty treatment.

N. Completion record

Record your chosen case, predicted trend, measured false_positive_fraction, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 40, Zero-velocity updates, foot constraints and slip bias. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 40 - Zero-velocity updates, foot constraints and slip bias

A. The engineering question

A stationary contacting foot supplies a useful constraint on body motion.

B. Concepts and mathematical model

If the foot actually slips, enforcing zero velocity injects a biased pseudo-measurement into the estimator. This illustrates a broader sensor-fusion principle: a model-based constraint is only a valid measurement while its assumptions hold. The laboratory combines integrated acceleration with occasional zero-velocity corrections and then deliberately violates the stationary-contact assumption.

v_pred=v_prev+a*dt; contact correction
v_next=(1-K)*v_pred+K*v_contact_assumed.

C. Real-world meaning and limits

Physical base-velocity estimators must tolerate contact transitions and slipping surfaces before learned policies depend on them.

D. Architecture and data flow

Configuration (slip_speed) -> numerical model -> constrained_rmse_m_s, unconstrained_rmse_m_s, slip_interval_bias_m_s -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

slip_speed is meters per second during the deliberately invalid contact-constraint interval.

Parameter	Nominal value	Interpretation
slip_speed	0.05	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/40_zero_velocity.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only slip_speed changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/40_zero_velocity.py --case 0

.venv-book/bin/python expanded_lab/modules/40_zero_velocity.py --case 3

.venv-book/bin/python expanded_lab/run.py zero_velocity --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/40_zero_velocity.py

```
"""Zero-velocity updates, foot constraints and slip bias. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(43)
    dt = 0.01
    t = np.arange(0, 6, dt)
    slip = p["slip_speed"]
    truth = np.where(t < 2, 0.0, np.where(t < 4, slip, 0.0))
    acceleration = np.gradient(truth, dt)
    measured = acceleration + 0.02 + rng.normal(0, 0.01, len(t))
    estimate = 0.0
    raw = 0.0
    errors = []
    raw_errors = []
    for i, a in enumerate(measured):
        raw += a * dt
        estimate += a * dt
        assumed_contact = (i % 20) == 0
        if assumed_contact:
            estimate = 0.2 * estimate
        errors.append(estimate - truth[i])
        raw_errors.append(raw - truth[i])
    metrics = dict(
        constrained_rmse_m_s=np.sqrt(np.mean(np.array(errors) ** 2)),
        unconstrained_rmse_m_s=np.sqrt(np.mean(np.array(raw_errors) ** 2)),
        slip_interval_bias_m_s=np.mean(np.array(errors)[(t >= 2) & (t < 4)]),
        final_constrained_error_m_s=errors[-1],
    )
    derivation = [
        f"The synthetic foot/body velocity is {slip:g} m/s during a two-second slip
    interval.",
        "Acceleration measurement includes +0.02 m/s^2 bias and 0.01 m/s^2 noise.",
        "Every 0.2 s, impose a strong zero-velocity pseudo-measurement regardless of actual
    slip.",
        f'Constrained RMSE is {metrics["constrained_rmse_m_s"]:.6f} m/s; unconstrained RMSE
    is {metrics["unconstrained_rmse_m_s"]:.6f} m/s.',
        f'Mean bias during slip is {metrics["slip_interval_bias_m_s"]:.6f} m/s.',
    ]
    notes = [
        "Zero-velocity constraints can reduce drift in valid stationary intervals and create
     systematic error during slip.",
        "Use contact confidence, slip indicators and innovation checks rather than
    permanently enforcing the constraint.",
    ]
    return metrics, derivation, notes
```

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"slip_speed": 0.05}
    parameters["slip_speed"] = [0.0, 0.01, 0.03, 0.05, 0.1, 0.2][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Run an unconstrained integrator beside the constrained estimate to expose the tradeoff. The contact assumption is intentionally wrong during slip, allowing a controlled failure analysis.

I. Expected behavior and six worked cases

The controlled variable is slip_speed. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

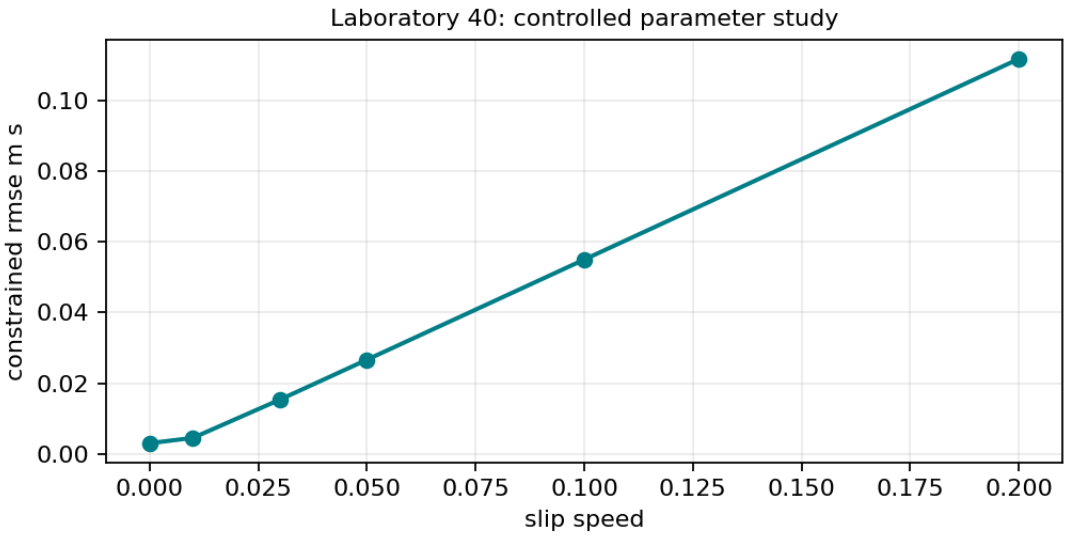


Figure L40. constrained rmse m s as slip_speed changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	slip_speed	constrained rmse m s
0	0	0.00308652
1	0.01	0.00459835
2	0.03	0.01541259
3	0.05	0.02667408
4	0.1	0.05498371
5	0.2	0.11169503

Worked case 0: slip_speed = 0

This case keeps the nominal configuration except for slip_speed=0. The full input is {"slip_speed": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
constrained rmse m s	0.00308652
unconstrained rmse m s	0.06902091
slip interval bias m s	0.0030906
final constrained error m s	0.00509715

1. The synthetic foot/body velocity is 0 m/s during a two-second slip interval.
2. Acceleration measurement includes +0.02 m/s^2 bias and 0.01 m/s^2 noise.
3. Every 0.2 s, impose a strong zero-velocity pseudo-measurement regardless of actual slip.
4. Constrained RMSE is 0.003087 m/s; unconstrained RMSE is 0.069021 m/s.
5. Mean bias during slip is 0.003091 m/s.

Worked case 1: slip_speed = 0.01

This case keeps the nominal configuration except for slip_speed=0.01. The full input is {"slip_speed": 0.01}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
constrained rmse m s	0.00459835
unconstrained rmse m s	0.06901645
slip interval bias m s	-0.0066844
final constrained error m s	0.00509715

1. The synthetic foot/body velocity is 0.01 m/s during a two-second slip interval.
2. Acceleration measurement includes +0.02 m/s^2 bias and 0.01 m/s^2 noise.
3. Every 0.2 s, impose a strong zero-velocity pseudo-measurement regardless of actual slip.
4. Constrained RMSE is 0.004598 m/s; unconstrained RMSE is 0.069016 m/s.
5. Mean bias during slip is -0.006684 m/s.

Worked case 2: slip_speed = 0.03

This case keeps the nominal configuration except for slip_speed=0.03. The full input is {"slip_speed": 0.03}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
constrained rmse m s	0.01541259
unconstrained rmse m s	0.06901115
slip interval bias m s	-0.0262344
final constrained error m s	0.00509715

1. The synthetic foot/body velocity is 0.03 m/s during a two-second slip interval.
2. Acceleration measurement includes +0.02 m/s^2 bias and 0.01 m/s^2 noise.
3. Every 0.2 s, impose a strong zero-velocity pseudo-measurement regardless of actual slip.
4. Constrained RMSE is 0.015413 m/s; unconstrained RMSE is 0.069011 m/s.
5. Mean bias during slip is -0.026234 m/s.

Worked case 3: slip_speed = 0.05

This case keeps the nominal configuration except for slip_speed=0.05. The full input is {"slip_speed": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
constrained rmse m s	0.02667408
unconstrained rmse m s	0.06901067
slip interval bias m s	-0.0457844
final constrained error m s	0.00509714

1. The synthetic foot/body velocity is 0.05 m/s during a two-second slip interval.
2. Acceleration measurement includes +0.02 m/s^2 bias and 0.01 m/s^2 noise.
3. Every 0.2 s, impose a strong zero-velocity pseudo-measurement regardless of actual slip.
4. Constrained RMSE is 0.026674 m/s; unconstrained RMSE is 0.069011 m/s.
5. Mean bias during slip is -0.045784 m/s.

Worked case 4: slip_speed = 0.1

This case keeps the nominal configuration except for slip_speed=0.1. The full input is {"slip_speed": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
constrained rmse m s	0.05498371
unconstrained rmse m s	0.06903062
slip interval bias m s	-0.0946594
final constrained error m s	0.00509714

1. The synthetic foot/body velocity is 0.1 m/s during a two-second slip interval.
2. Acceleration measurement includes +0.02 m/s^2 bias and 0.01 m/s^2 noise.
3. Every 0.2 s, impose a strong zero-velocity pseudo-measurement regardless of actual slip.
4. Constrained RMSE is 0.054984 m/s; unconstrained RMSE is 0.069031 m/s.
5. Mean bias during slip is -0.094659 m/s.

Worked case 5: slip_speed = 0.2

This case keeps the nominal configuration except for slip_speed=0.2. The full input is {"slip_speed": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
constrained rmse m s	0.11169503
unconstrained rmse m s	0.06916093
slip interval bias m s	-0.1924094
final constrained error m s	0.00509713

1. The synthetic foot/body velocity is 0.2 m/s during a two-second slip interval.
2. Acceleration measurement includes +0.02 m/s^2 bias and 0.01 m/s^2 noise.

3. Every 0.2 s, impose a strong zero-velocity pseudo-measurement regardless of actual slip.
4. Constrained RMSE is 0.111695 m/s; unconstrained RMSE is 0.069161 m/s.
5. Mean bias during slip is -0.192409 m/s.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Zero-velocity constraints can reduce drift in valid stationary intervals and create systematic error during slip.

Use contact confidence, slip indicators and innovation checks rather than permanently enforcing the constraint.

K. Failure modes and troubleshooting

Using foot contact as proof of zero world velocity; confusing a kinematic constraint with an independent sensor; hiding bias in a low covariance.

L. Practical exercises and extension tasks

1. Gate corrections using an independent slip detector.
2. Add a bias state and compare observability in stationary and moving intervals.
3. Evaluate both drift reduction and slip-induced bias.

M. Connection to the Asimov research platform

Physical base-velocity estimators must tolerate contact transitions and slipping surfaces before learned policies depend on them.

N. Completion record

Record your chosen case, predicted trend, measured constrained_rmse_m_s, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 41, Typed joint-command contracts and rejection accounting. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Part X - Software interfaces, timing and supervisory control

Your software-engineering experience is directly useful here, but physical control gives ordinary interface bugs greater consequences. Message ordering, queue age, activation state, packet interpretation and preprocessing versions must be explicit. These laboratories test contracts with deliberate faults and expose the limits of software-only evidence. The fictional wire format is never a motor protocol; no chapter opens a CAN bus or enables a physical actuator.

Laboratory 41 - Typed joint-command contracts and rejection accounting

A. The engineering question

A valid JSON object is not necessarily a valid robot command.

B. Concepts and mathematical model

A command contract must constrain joint names and order, vector length, numeric finiteness, units, range, sequence and age. Validation should occur before a command enters the controller's active state. Rejection counters make faults observable without turning malformed requests into motor motion. This experiment defines a small typed-like schema and injects different kinds of failures into a deterministic message stream.

accept = schema_ok and finite and limits_ok and sequence_new and 0<=age<=max_age.

C. Real-world meaning and limits

Physical command validation must use the commissioned joint/motor mapping and remain subordinate to local watchdogs and isolation.

D. Architecture and data flow

Configuration (invalid_fraction) -> numerical model -> accepted_count, rejected_count, rejection_fraction -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

invalid_fraction is the probability of an injected synthetic schema/age/range fault, not a measured network error rate.

Parameter	Nominal value	Interpretation
invalid_fraction	0.1	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/41_message_contracts.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only invalid_fraction changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/41_message_contracts.py --case 0

.venv-book/bin/python expanded_lab/modules/41_message_contracts.py --case 3

.venv-book/bin/python expanded_lab/run.py message_contracts --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/41_message_contracts.py

```
"""Typed joint-command contracts and rejection accounting. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def valid(message, now, last_sequence):
    names = ["joint_a", "joint_b"]
    if message.get("names") != names:
        return False, "names"
    q = np.asarray(message.get("position", []), dtype=float)
    if q.shape != (2,):
        return False, "shape"
    if not np.isfinite(q).all():
        return False, "nonfinite"
    if np.any(abs(q) > 0.5):
        return False, "range"
    if message["sequence"] <= last_sequence:
        return False, "sequence"
    age = now - message["sent"]
    if not 0 <= age <= 0.1:
        return False, "age"
    return True, "accepted"


def experiment(p):
    rng = np.random.default_rng(47)
    n = 1000
    last = -1
    counts = {}
    corrupt = p["invalid_fraction"]
    for i in range(n):
        now = i * 0.02
        msg = dict(
            names=["joint_a", "joint_b"],
            position=[0.1, -0.1],
            sequence=i,
            sent=now - 0.01,
        )
        if rng.random() < corrupt:
            kind = i % 4
            if kind == 0:
                msg["names"] = ["joint_b", "joint_a"]
            if kind == 1:
                msg["position"] = [float("nan"), 0.0]
            if kind == 2:
                msg["sent"] = now - 0.2
            if kind == 3:
                msg["position"] = [2.0, 0.0]
        ok, reason = valid(msg, now, last)
```

```
        counts[reason] = counts.get(reason, 0) + 1
        if ok:
            last = i
    accepted = counts.get("accepted", 0)
    metrics = dict(
        accepted_count=accepted,
        rejected_count=n - accepted,
        rejection_fraction=(n - accepted) / n,
        last_accepted_sequence=last,
    )
    derivation = [
        f"Generate {n} commands at 50 Hz with corruption probability {corrupt:g}.",
        "Corrupt name order, numeric finiteness, age or range in alternating injected
    cases.",
        "Validate the entire candidate before updating last accepted sequence.",
        f"Accepted {accepted} messages; rejected {n-accepted}.",
        f"Rejection reasons are {counts}.",
    ]
    notes = [
        "Rejected messages do not update the accepted sequence or active target in this
    contract example.",
        "A production parser must also handle missing keys, malicious payload sizes, schema
    versions and transport authentication.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"invalid_fraction": 0.1}
    parameters["invalid_fraction"] = [0.0, 0.01, 0.05, 0.1, 0.25, 0.5][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Validation is side-effect free until success is known. The experiment records rejected categories and accepted sequence progression, illustrating observability around a strict boundary.

I. Expected behavior and six worked cases

The controlled variable is invalid_fraction. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

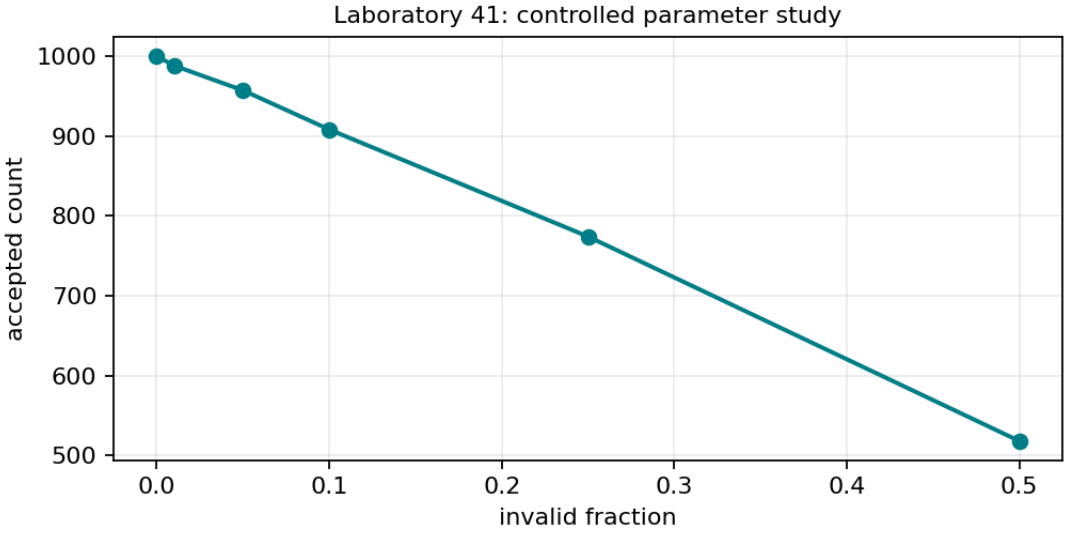


Figure L41. accepted count as invalid_fraction changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	invalid_fraction	accepted count
0	0	1000
1	0.01	988
2	0.05	957
3	0.1	908
4	0.25	774
5	0.5	518

Worked case 0: invalid_fraction = 0

This case keeps the nominal configuration except for invalid_fraction=0. The full input is {"invalid_fraction": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
accepted count	1000
rejected count	0
rejection fraction	0
last accepted sequence	999

1. Generate 1000 commands at 50 Hz with corruption probability 0.
2. Corrupt name order, numeric finiteness, age or range in alternating injected cases.
3. Validate the entire candidate before updating last accepted sequence.
4. Accepted 1000 messages; rejected 0.
5. Rejection reasons are {'accepted': 1000}.

Worked case 1: invalid_fraction = 0.01

This case keeps the nominal configuration except for invalid_fraction=0.01. The full input is {"invalid_fraction": 0.01}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
accepted count	988
rejected count	12
rejection fraction	0.012
last accepted sequence	999

1. Generate 1000 commands at 50 Hz with corruption probability 0.01.

2. Corrupt name order, numeric finiteness, age or range in alternating injected cases.
3. Validate the entire candidate before updating last accepted sequence.
4. Accepted 988 messages; rejected 12.
5. Rejection reasons are {'accepted': 988, 'age': 4, 'names': 4, 'nonfinite': 2, 'range': 2}.

Worked case 2: invalid_fraction = 0.05

This case keeps the nominal configuration except for invalid_fraction=0.05. The full input is {"invalid_fraction": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
accepted count	957
rejected count	43
rejection fraction	0.043
last accepted sequence	999

1. Generate 1000 commands at 50 Hz with corruption probability 0.05.
2. Corrupt name order, numeric finiteness, age or range in alternating injected cases.
3. Validate the entire candidate before updating last accepted sequence.
4. Accepted 957 messages; rejected 43.
5. Rejection reasons are {'accepted': 957, 'age': 12, 'nonfinite': 15, 'range': 8, 'names': 8}.

Worked case 3: invalid_fraction = 0.1

This case keeps the nominal configuration except for invalid_fraction=0.1. The full input is {"invalid_fraction": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
accepted count	908
rejected count	92
rejection fraction	0.092
last accepted sequence	999

1. Generate 1000 commands at 50 Hz with corruption probability 0.1.
2. Corrupt name order, numeric finiteness, age or range in alternating injected cases.
3. Validate the entire candidate before updating last accepted sequence.
4. Accepted 908 messages; rejected 92.
5. Rejection reasons are {'accepted': 908, 'nonfinite': 26, 'age': 26, 'range': 18, 'names': 22}.

Worked case 4: invalid_fraction = 0.25

This case keeps the nominal configuration except for invalid_fraction=0.25. The full input is {"invalid_fraction": 0.25}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
accepted count	774
rejected count	226
rejection fraction	0.226
last accepted sequence	999

1. Generate 1000 commands at 50 Hz with corruption probability 0.25.
2. Corrupt name order, numeric finiteness, age or range in alternating injected cases.
3. Validate the entire candidate before updating last accepted sequence.
4. Accepted 774 messages; rejected 226.

5. Rejection reasons are {'accepted': 774, 'range': 57, 'age': 57, 'nonfinite': 59, 'names': 53}.

Worked case 5: invalid_fraction = 0.5

This case keeps the nominal configuration except for invalid_fraction=0.5. The full input is {"invalid_fraction": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
accepted count	518
rejected count	482
rejection fraction	0.482
last accepted sequence	997

1. Generate 1000 commands at 50 Hz with corruption probability 0.5.
2. Corrupt name order, numeric finiteness, age or range in alternating injected cases.
3. Validate the entire candidate before updating last accepted sequence.
4. Accepted 518 messages; rejected 482.
5. Rejection reasons are {'accepted': 518, 'age': 115, 'range': 113, 'nonfinite': 126, 'names': 128}.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Rejected messages do not update the accepted sequence or active target in this contract example.

A production parser must also handle missing keys, malicious payload sizes, schema versions and transport authentication.

K. Failure modes and troubleshooting

Coercing missing targets to zero; partially applying a vector before all fields pass; accepting reordered names because lengths match.

L. Practical exercises and extension tasks

1. Add explicit schema version and unit fields.
2. Test duplicate and out-of-order sequences.
3. Apply an input-size limit before allocating a vector.

M. Connection to the Asimov research platform

Physical command validation must use the commissioned joint/motor mapping and remain subordinate to local watchdogs and isolation.

N. Completion record

Record your chosen case, predicted trend, measured accepted_count, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 42, Message queues, delivery policy and stale-command risk. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 42 - Message queues, delivery policy and stale-command risk

A. The engineering question

A reliable queue can preserve every message while delivering commands too late to be useful.

B. Concepts and mathematical model

A latest-value stream favors freshness but intentionally discards intermediate samples. Those are different tradeoffs for telemetry, trajectories and events. This experiment simulates a producer faster than its consumer and compares FIFO delivery with replacing pending state by the newest sample. It illustrates queue-age behavior without pretending to implement all DDS QoS semantics.

age_at_use = consumer_time - production_time; FIFO preserves history; latest-value keeps at most one pending state.

C. Real-world meaning and limits

ROS QoS configuration must be paired with application-level age checks and control-specific command semantics.

D. Architecture and data flow

Configuration (consumer_period) -> numerical model -> fifo_mean_age_s, fifo_max_age_s, latest_mean_age_s -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

consumer_period is seconds; producer period is fixed at 10 ms and experiment duration is five seconds.

Parameter	Nominal value	Interpretation
consumer_period	0.02	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/42_queue_age.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only consumer_period changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/42_queue_age.py --case 0

.venv-book/bin/python expanded_lab/modules/42_queue_age.py --case 3

.venv-book/bin/python expanded_lab/run.py queue_age --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/42_queue_age.py

```
"""Message queues, delivery policy and stale-command risk. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    producer_dt = 0.01
    consumer_dt = p["consumer_period"]
    production = np.arange(0, 5, producer_dt)
    consumption = np.arange(0, 5, consumer_dt)
    fifo = []
    latest = []
    queue = []
    index = 0
    dropped = 0
    for now in consumption:
        arrivals = []
        while index < len(production) and production[index] <= now + 1e-12:
            arrivals.append(production[index])
            index += 1
        queue.extend(arrivals)
        if queue:
            fifo.append(now - queue.pop(0))
        available = production[production <= now + 1e-12]
        if len(available):
            latest.append(now - available[-1])
        dropped += max(0, len(arrivals) - 1)
    metrics = dict(
        fifo_mean_age_s=np.mean(fifo),
        fifo_max_age_s=np.max(fifo),
        latest_mean_age_s=np.mean(latest),
        latest_max_age_s=np.max(latest),
        fifo_backlog_count=len(queue),
        replaced_arrivals_count=dropped,
    )
    derivation = [
        f"The producer emits state every {producer_dt:g} s.",
        f"The consumer processes one item every {consumer_dt:g} s.",
        f"FIFO retains {len(queue)} pending messages after the five-second run.",
        f"FIFO maximum age is {np.max(fifo):.6f} s; latest-value maximum age is
    {np.max(latest):.6f} s.",
        f"Latest-value replacement intentionally skips at least {dropped} intermediate
    arrivals in this schedule.",
    ]
    notes = [
        "A trajectory is not always replaceable by its last point; choose semantics
    appropriate to the command type.",
        "The simulation concerns queue policy only and does not reproduce packet loss, DDS
    discovery or actual ROS middleware timing.",
```

```
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"consumer_period": 0.02}
    parameters["consumer_period"] = [0.005, 0.01, 0.015, 0.02, 0.05, 0.1][
        args.case
    ]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Compute age at consumption, not at arrival. The FIFO backlog exposes overload that can be hidden by successful delivery acknowledgments.

I. Expected behavior and six worked cases

The controlled variable is consumer_period. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

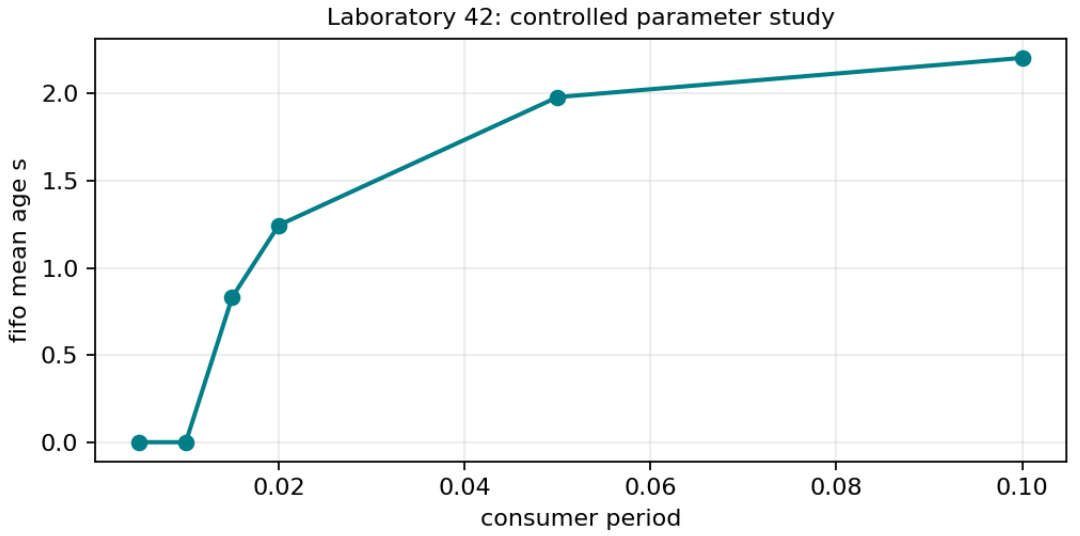


Figure L42. fifo mean age s as consumer_period changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	consumer_period	fifo mean age s
0	0.005	0
1	0.01	0
2	0.015	0.8325
3	0.02	1.245
4	0.05	1.98
5	0.1	2.205

Worked case 0: consumer_period = 0.005

This case keeps the nominal configuration except for consumer_period=0.005. The full input is {"consumer_period": 0.005}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
fifo mean age s	0
fifo max age s	0
latest mean age s	0.0025
latest max age s	0.005
fifo backlog count	0
replaced arrivals count	0

1. The producer emits state every 0.01 s.
2. The consumer processes one item every 0.005 s.
3. FIFO retains 0 pending messages after the five-second run.

4. FIFO maximum age is 0.000000 s; latest-value maximum age is 0.005000 s.
5. Latest-value replacement intentionally skips at least 0 intermediate arrivals in this schedule.

Worked case 1: consumer_period = 0.01

This case keeps the nominal configuration except for consumer_period=0.01. The full input is {"consumer_period": 0.01}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
fifo mean age s	0
fifo max age s	0
latest mean age s	0
latest max age s	0
fifo backlog count	0
replaced arrivals count	0

1. The producer emits state every 0.01 s.
2. The consumer processes one item every 0.01 s.
3. FIFO retains 0 pending messages after the five-second run.
4. FIFO maximum age is 0.000000 s; latest-value maximum age is 0.000000 s.
5. Latest-value replacement intentionally skips at least 0 intermediate arrivals in this schedule.

Worked case 2: consumer_period = 0.015

This case keeps the nominal configuration except for consumer_period=0.015. The full input is {"consumer_period": 0.015}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
fifo mean age s	0.8325
fifo max age s	1.665
latest mean age s	0.0025
latest max age s	0.005
fifo backlog count	166
replaced arrivals count	166

1. The producer emits state every 0.01 s.
2. The consumer processes one item every 0.015 s.
3. FIFO retains 166 pending messages after the five-second run.
4. FIFO maximum age is 1.665000 s; latest-value maximum age is 0.005000 s.
5. Latest-value replacement intentionally skips at least 166 intermediate arrivals in this schedule.

Worked case 3: consumer_period = 0.02

This case keeps the nominal configuration except for consumer_period=0.02. The full input is {"consumer_period": 0.02}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
fifo mean age s	1.245
fifo max age s	2.49
latest mean age s	0
latest max age s	0
fifo backlog count	249
replaced arrivals count	249

1. The producer emits state every 0.01 s.
2. The consumer processes one item every 0.02 s.
3. FIFO retains 249 pending messages after the five-second run.
4. FIFO maximum age is 2.490000 s; latest-value maximum age is 0.000000 s.
5. Latest-value replacement intentionally skips at least 249 intermediate arrivals in this schedule.

Worked case 4: consumer_period = 0.05

This case keeps the nominal configuration except for consumer_period=0.05. The full input is {"consumer_period": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
fifo mean age s	1.98
fifo max age s	3.96
latest mean age s	6.5225603e-17
latest max age s	8.8817842e-16
fifo backlog count	396
replaced arrivals count	396

1. The producer emits state every 0.01 s.
2. The consumer processes one item every 0.05 s.
3. FIFO retains 396 pending messages after the five-second run.
4. FIFO maximum age is 3.960000 s; latest-value maximum age is 0.000000 s.
5. Latest-value replacement intentionally skips at least 396 intermediate arrivals in this schedule.

Worked case 5: consumer_period = 0.1

This case keeps the nominal configuration except for consumer_period=0.1. The full input is {"consumer_period": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
fifo mean age s	2.205
fifo max age s	4.41
latest mean age s	8.3266727e-17
latest max age s	8.8817842e-16
fifo backlog count	441
replaced arrivals count	441

1. The producer emits state every 0.01 s.
2. The consumer processes one item every 0.1 s.
3. FIFO retains 441 pending messages after the five-second run.
4. FIFO maximum age is 4.410000 s; latest-value maximum age is 0.000000 s.
5. Latest-value replacement intentionally skips at least 441 intermediate arrivals in this schedule.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

A trajectory is not always replaceable by its last point; choose semantics appropriate to the command type.

The simulation concerns queue policy only and does not reproduce packet loss, DDS discovery or actual ROS middleware timing.

K. Failure modes and troubleshooting

Choosing reliable delivery as a substitute for deadlines; using deep queues for current-state commands; dropping events that require ordered processing.

L. Practical exercises and extension tasks

1. Add a bounded FIFO depth and count dropped old messages.
2. Reject messages older than a declared deadline.
3. Model a buffered trajectory separately from a replaceable velocity target.

M. Connection to the Asimov research platform

ROS QoS configuration must be paired with application-level age checks and control-specific command semantics.

N. Completion record

Record your chosen case, predicted trend, measured fifo_mean_age_s, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 43, Lifecycle activation, preconditions and fault latching. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 43 - Lifecycle activation, preconditions and fault latching

A. The engineering question

A robot process should not become motion-capable merely because it started.

B. Concepts and mathematical model

Configuration, inactive readiness, active control, fault handling and shutdown are distinct states. A fault should normally require a deliberate recovery path rather than disappear when a healthy packet arrives. This laboratory implements a small supervisor state machine and injects failed preconditions, making activation authority explicit and testable.

configure: unconfigured->inactive; activate requires all readiness flags; fault: active->fault; recovery never directly resumes active motion.

C. Real-world meaning and limits

Commissioning records, local embedded protection and independent stop behavior must support the activation decision.

D. Architecture and data flow

Configuration (failed_readiness_probability) -> numerical model -> activation_count, blocked_count, fault_count -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

failed_readiness_probability controls independent simulated readiness flags; it is not an estimated physical reliability rate.

Parameter	Nominal value	Interpretation
failed_readiness_probability	0.1	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/43_lifecycle.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only failed_readiness_probability changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/43_lifecycle.py --case 0

.venv-book/bin/python expanded_lab/modules/43_lifecycle.py --case 3

.venv-book/bin/python expanded_lab/run.py lifecycle --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/43_lifecycle.py

```
"""Lifecycle activation, preconditions and fault latching. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def transition(state, event, ready):
    if event == "shutdown":
        return "finalized"
    if event == "fault" and state == "active":
        return "fault"
    if state == "unconfigured" and event == "configure":
        return "inactive"
    if state == "inactive" and event == "activate" and all(ready.values()):
        return "active"
    if state == "active" and event == "deactivate":
        return "inactive"
    if state == "fault" and event == "reset":
        return "inactive"
    return state


def experiment(p):
    rng = np.random.default_rng(53)
    trials = 200
    activations = 0
    blocked = 0
    faults = 0
    unsafe_resume = 0
    for _ in range(trials):
        ready = {
            k: rng.random() >= p["failed_readiness_probability"]
            for k in ["calibration", "limits", "feedback", "support"]
        }
        state = transition("unconfigured", "configure", ready)
        state = transition(state, "activate", ready)
        if state == "active":
            activations += 1
            state = transition(state, "fault", ready)
            faults += 1
            state = transition(state, "healthy_packet", ready)
            unsafe_resume += state == "active"
        else:
            blocked += 1
    assert unsafe_resume == 0
    metrics = dict(
        activation_count=activations,
        blocked_count=blocked,
        fault_count=faults,
        automatic_resume_count=unsafe_resume,
```

```
        activation_fraction=activations / trials,
    )
    derivation = [
        f'Run {trials} readiness trials with per-flag failure probability
    {p["failed_readiness_probability"]:g}.',
        "Configuration moves the supervisor to inactive; activation checks all four
    readiness flags.",
        f"Activate {activations} trials and block {blocked}.",
        f"Inject a fault into every active trial, then deliver a healthy packet.",
        f"Automatic active resumption count is {unsafe_resume}; the assertion requires
    zero.",
    ]
    notes = [
        "Readiness flags represent evidence that must exist outside this synthetic state
    machine.",
        "A reset returns to inactive; a new reviewed activation is required before motion
    can resume.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"failed_readiness_probability": 0.1}
    parameters["failed_readiness_probability"] = [0.0, 0.01, 0.05, 0.1, 0.2, 0.5][
        args.case
    ]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Keep transition logic separate from side effects such as enabling a drive. Test forbidden transitions as carefully as the happy path.

I. Expected behavior and six worked cases

The controlled variable is failed_readiness_probability. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

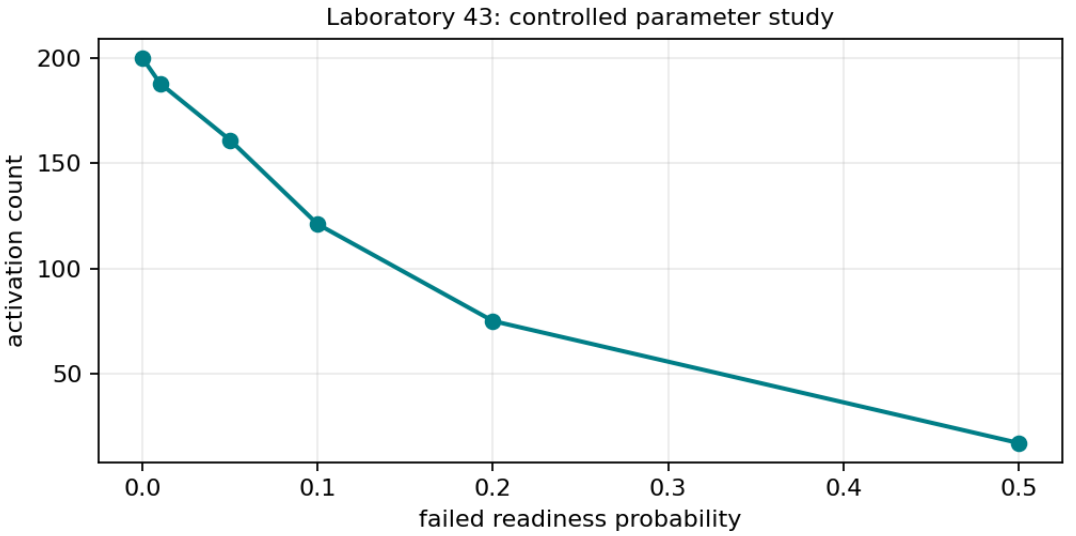


Figure L43. activation count as failed_readiness_probability changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	failed_readiness_probability	activation count
0	0	200
1	0.01	188
2	0.05	161
3	0.1	121
4	0.2	75
5	0.5	17

Worked case 0: failed_readiness_probability = 0

This case keeps the nominal configuration except for failed_readiness_probability=0. The full input is {"failed_readiness_probability": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
activation count	200

Computed quantity	Numerical result
blocked count	0
fault count	200
automatic resume count	0
activation fraction	1

1. Run 200 readiness trials with per-flag failure probability 0.
2. Configuration moves the supervisor to inactive; activation checks all four readiness flags.
3. Activate 200 trials and block 0.
4. Inject a fault into every active trial, then deliver a healthy packet.
5. Automatic active resumption count is 0; the assertion requires zero.

Worked case 1: failed_readiness_probability = 0.01

This case keeps the nominal configuration except for failed_readiness_probability=0.01. The full input is {"failed_readiness_probability": 0.01}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
activation count	188
blocked count	12
fault count	188
automatic resume count	0
activation fraction	0.94

1. Run 200 readiness trials with per-flag failure probability 0.01.
2. Configuration moves the supervisor to inactive; activation checks all four readiness flags.
3. Activate 188 trials and block 12.

4. Inject a fault into every active trial, then deliver a healthy packet.
5. Automatic active resumption count is 0; the assertion requires zero.

Worked case 2: failed_readiness_probability = 0.05

This case keeps the nominal configuration except for failed_readiness_probability=0.05. The full input is {"failed_readiness_probability": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
activation count	161
blocked count	39
fault count	161
automatic resume count	0
activation fraction	0.805

1. Run 200 readiness trials with per-flag failure probability 0.05.
2. Configuration moves the supervisor to inactive; activation checks all four readiness flags.
3. Activate 161 trials and block 39.
4. Inject a fault into every active trial, then deliver a healthy packet.
5. Automatic active resumption count is 0; the assertion requires zero.

Worked case 3: failed_readiness_probability = 0.1

This case keeps the nominal configuration except for failed_readiness_probability=0.1. The full input is {"failed_readiness_probability": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
activation count	121
blocked count	79

Computed quantity	Numerical result
fault count	121
automatic resume count	0
activation fraction	0.605

1. Run 200 readiness trials with per-flag failure probability 0.1.
2. Configuration moves the supervisor to inactive; activation checks all four readiness flags.
3. Activate 121 trials and block 79.
4. Inject a fault into every active trial, then deliver a healthy packet.
5. Automatic active resumption count is 0; the assertion requires zero.

Worked case 4: failed_readiness_probability = 0.2

This case keeps the nominal configuration except for failed_readiness_probability=0.2. The full input is {"failed_readiness_probability": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
activation count	75
blocked count	125
fault count	75
automatic resume count	0
activation fraction	0.375

1. Run 200 readiness trials with per-flag failure probability 0.2.
2. Configuration moves the supervisor to inactive; activation checks all four readiness flags.
3. Activate 75 trials and block 125.
4. Inject a fault into every active trial, then deliver a healthy packet.

5. Automatic active resumption count is 0; the assertion requires zero.

Worked case 5: failed_readiness_probability = 0.5

This case keeps the nominal configuration except for failed_readiness_probability=0.5. The full input is {"failed_readiness_probability": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
activation count	17
blocked count	183
fault count	17
automatic resume count	0
activation fraction	0.085

1. Run 200 readiness trials with per-flag failure probability 0.5.
2. Configuration moves the supervisor to inactive; activation checks all four readiness flags.
3. Activate 17 trials and block 183.
4. Inject a fault into every active trial, then deliver a healthy packet.
5. Automatic active resumption count is 0; the assertion requires zero.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Readiness flags represent evidence that must exist outside this synthetic state machine.

A reset returns to inactive; a new reviewed activation is required before motion can resume.

K. Failure modes and troubleshooting

Treating ROS node startup as hardware readiness; clearing a latched fault on fresh data; assuming a software state verifies physical isolation.

L. Practical exercises and extension tasks

1. Add explicit evidence IDs to each readiness flag.
2. Enumerate every state/event pair and test allowed transitions.
3. Model emergency isolation separately from software inactive state.

M. Connection to the Asimov research platform

Commissioning records, local embedded protection and independent stop behavior must support the activation decision.

N. Completion record

Record your chosen case, predicted trend, measured activation_count, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 44, Multi-rate controllers, policy decimation and sample age. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 44 - Multi-rate controllers, policy decimation and sample age

A. The engineering question

Robotics stacks use several update rates.

B. Concepts and mathematical model

Fast actuator feedback, slower policy inference and still slower telemetry can coexist if their interfaces specify sample-and-hold behavior and allowed age. Confusing a decimated display rate with the feedback rate can degrade control silently. This laboratory schedules deterministic physics, control, policy and logging events from one integer tick counter, avoiding floating-point modulo tests on accumulated time.

event at tick k when k mod decimation=0; period=base_dt*decimation; held-action age=(k-last_policy_tick)*base_dt.

C. Real-world meaning and limits

The physical controller must distinguish simulation clock, device sample clock and monotonic deadline clock.

D. Architecture and data flow

Configuration (policy_decimation) -> numerical model -> policy_rate_Hz, maximum_held_action_age_s, mean_held_action_age_s -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

policy_decimation is an integer base-tick count; base step is 1 ms, control step 2 ms and telemetry step 100 ms.

Parameter	Nominal value	Interpretation
policy_decimation	20	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/44_timing_schedule.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only policy_decimation changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/44_timing_schedule.py --case 0

.venv-book/bin/python expanded_lab/modules/44_timing_schedule.py --case 3

.venv-book/bin/python expanded_lab/run.py timing_schedule --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/44_timing_schedule.py

```
"""Multi-rate controllers, policy decimation and sample age. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    dt = 0.001
    n = 5000
    decimation = int(p["policy_decimation"])
    last_policy = 0
    ages = []
    counts = dict(physics=0, control=0, policy=0, telemetry=0)
    for tick in range(n):
        counts["physics"] += 1
        if tick % 2 == 0:
            counts["control"] += 1
        if tick % decimation == 0:
            last_policy = tick
            counts["policy"] += 1
        if tick % 100 == 0:
            counts["telemetry"] += 1
        ages.append((tick - last_policy) * dt)
    metrics = dict(
        policy_rate_Hz=1 / (dt * decimation),
        maximum_held_action_age_s=max(ages),
        mean_held_action_age_s=np.mean(ages),
        control_updates=counts["control"],
        policy_updates=counts["policy"],
        telemetry_updates=counts["telemetry"],
    )
    derivation = [
        f"Use {n} base ticks at {dt:g} s per tick.",
        f"Control runs every two ticks; policy runs every {decimation} ticks; telemetry
    every 100.",
        f"The policy rate is {1/(dt*decimation):.6f} Hz.",
        f"The oldest held policy action reaches {max(ages):.6f} s before replacement.",
        f"Event counts are {counts}.",
    ]
    notes = [
        "This schedule is logical simulation time and assumes every computation completes
    before its deadline.",
        "Wall-time execution still needs measured compute/transport budgets and an overrun
    response.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
```

```
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"policy_decimation": 20}
    parameters["policy_decimation"] = [2, 5, 10, 20, 50, 100][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Integer scheduling makes rate relationships reproducible. Track action age explicitly so the consumer can enforce a contract even when the producer rate changes.

I. Expected behavior and six worked cases

The controlled variable is policy_decimation. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

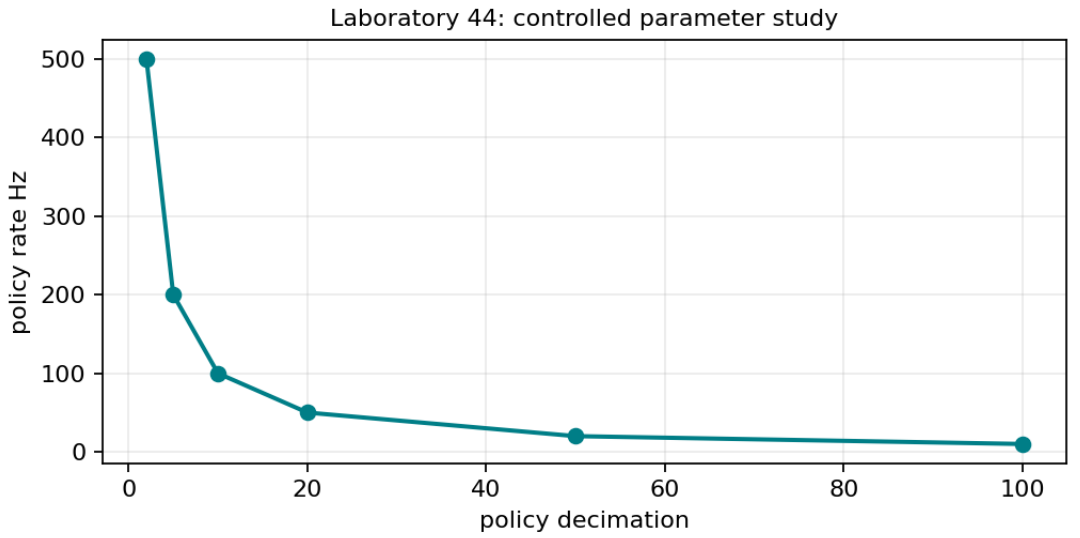


Figure L44. policy rate Hz as policy_decimation changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	policy_decimation	policy rate Hz
0	2	500
1	5	200
2	10	100
3	20	50
4	50	20
5	100	10

Worked case 0: policy_decimation = 2

This case keeps the nominal configuration except for policy_decimation=2. The full input is {"policy_decimation": 2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
policy rate Hz	500
maximum held action age s	0.001
mean held action age s	0.0005
control updates	2500
policy updates	2500
telemetry updates	50

1. Use 5000 base ticks at 0.001 s per tick.
2. Control runs every two ticks; policy runs every 2 ticks; telemetry every 100.
3. The policy rate is 500.000000 Hz.
4. The oldest held policy action reaches 0.001000 s before replacement.

5. Event counts are {'physics': 5000, 'control': 2500, 'policy': 2500, 'telemetry': 50}.

Worked case 1: policy_decimation = 5

This case keeps the nominal configuration except for policy_decimation=5. The full input is {"policy_decimation": 5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
policy rate Hz	200
maximum held action age s	0.004
mean held action age s	0.002
control updates	2500
policy updates	1000
telemetry updates	50

1. Use 5000 base ticks at 0.001 s per tick.
2. Control runs every two ticks; policy runs every 5 ticks; telemetry every 100.
3. The policy rate is 200.000000 Hz.
4. The oldest held policy action reaches 0.004000 s before replacement.
5. Event counts are {'physics': 5000, 'control': 2500, 'policy': 1000, 'telemetry': 50}.

Worked case 2: policy_decimation = 10

This case keeps the nominal configuration except for policy_decimation=10. The full input is {"policy_decimation": 10}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
policy rate Hz	100
maximum held action age s	0.009
mean held action age s	0.0045
control updates	2500
policy updates	500
telemetry updates	50

1. Use 5000 base ticks at 0.001 s per tick.
2. Control runs every two ticks; policy runs every 10 ticks; telemetry every 100.
3. The policy rate is 100.000000 Hz.
4. The oldest held policy action reaches 0.009000 s before replacement.
5. Event counts are {'physics': 5000, 'control': 2500, 'policy': 500, 'telemetry': 50}.

Worked case 3: policy_decimation = 20

This case keeps the nominal configuration except for policy_decimation=20. The full input is {"policy_decimation": 20}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
policy rate Hz	50
maximum held action age s	0.019
mean held action age s	0.0095
control updates	2500
policy updates	250
telemetry updates	50

1. Use 5000 base ticks at 0.001 s per tick.
2. Control runs every two ticks; policy runs every 20 ticks; telemetry every 100.
3. The policy rate is 50.000000 Hz.
4. The oldest held policy action reaches 0.019000 s before replacement.
5. Event counts are {'physics': 5000, 'control': 2500, 'policy': 250, 'telemetry': 50}.

Worked case 4: policy_decimation = 50

This case keeps the nominal configuration except for policy_decimation=50. The full input is {"policy_decimation": 50}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
policy rate Hz	20
maximum held action age s	0.049
mean held action age s	0.0245
control updates	2500
policy updates	100
telemetry updates	50

1. Use 5000 base ticks at 0.001 s per tick.
2. Control runs every two ticks; policy runs every 50 ticks; telemetry every 100.
3. The policy rate is 20.000000 Hz.
4. The oldest held policy action reaches 0.049000 s before replacement.
5. Event counts are {'physics': 5000, 'control': 2500, 'policy': 100, 'telemetry': 50}.

Worked case 5: policy_decimation = 100

This case keeps the nominal configuration except for policy_decimation=100. The full input is {"policy_decimation": 100}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
policy rate Hz	10
maximum held action age s	0.099
mean held action age s	0.0495
control updates	2500
policy updates	50
telemetry updates	50

1. Use 5000 base ticks at 0.001 s per tick.
2. Control runs every two ticks; policy runs every 100 ticks; telemetry every 100.
3. The policy rate is 10.000000 Hz.
4. The oldest held policy action reaches 0.099000 s before replacement.
5. Event counts are {'physics': 5000, 'control': 2500, 'policy': 50, 'telemetry': 50}.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

This schedule is logical simulation time and assumes every computation completes before its deadline.

Wall-time execution still needs measured compute/transport budgets and an overrun response.

K. Failure modes and troubleshooting

Using accumulated float time modulo a period; assuming nominal loop rates equal measured wall rates; forgetting action holds during missed inference.

L. Practical exercises and extension tasks

1. Add an inference-overrun event and specify the held-action timeout.
2. Reschedule a 60 Hz camera on a 1 kHz tick using a fractional accumulator.
3. Record every producer/consumer sample age in a replay trace.

M. Connection to the Asimov research platform

The physical controller must distinguish simulation clock, device sample clock and monotonic deadline clock.

N. Completion record

Record your chosen case, predicted trend, measured policy_rate_Hz, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 45, CAN bandwidth screening without invented motor commands. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 45 - CAN bandwidth screening without invented motor commands

A. The engineering question

A communication link has finite capacity, and payload bits are not the entire frame cost.

B. Concepts and mathematical model

Arbitration, framing, acknowledgment, inter-frame spacing, bit stuffing and retransmission consume time. This laboratory uses an explicit assumed worst-case frame length to screen a bus budget. It does not define motor packet semantics or resolve the Asimov bus assignment. The purpose is to reject implausible update-rate plans before implementing a transport.

bus utilization = joints*frames_per_joint_per_cycle*cycle_rate*assumed_frame_bits/bitrate.

C. Real-world meaning and limits

Obtain the exact actuator protocol and release-specific Asimov bus mapping before choosing bitrates or connecting a populated bus.

D. Architecture and data flow

Configuration (joints, cycle_rate, frame_bits, bitrate, frames_per_joint) -> numerical model -> frames_per_second, required_bits_per_second, utilization_fraction -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

joints is actuator count on one hypothetical segment; frame_bits is an explicit total-wire-length assumption; bitrate is bits/s.

Parameter	Nominal value	Interpretation
joints	6	Defined by the explicit chapter model and parameter contract
cycle_rate	500	Defined by the explicit chapter model and parameter contract
frame_bits	135	Defined by the explicit chapter model and parameter contract
bitrate	1.0000000e+06	Defined by the explicit chapter model and parameter contract
frames_per_joint	2	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/45_can_budget.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only cycle_rate changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/45_can_budget.py --case 0

.venv-book/bin/python expanded_lab/modules/45_can_budget.py --case 3

.venv-book/bin/python expanded_lab/run.py can_budget --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/45_can_budget.py

```
"""CAN bandwidth screening without invented motor commands. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    joints = int(p["joints"])
    rate = p["cycle_rate"]
    bits = p["frame_bits"]
    baud = p["bitrate"]
    frames = joints * p["frames_per_joint"] * rate
    demand = frames * bits
    utilization = demand / baud
    frame_time = bits / baud
    cycle_wire = joints * p["frames_per_joint"] * frame_time
    metrics = dict(
        frames_per_second=frames,
        required_bits_per_second=demand,
        utilization_fraction=utilization,
        one_frame_wire_time_us=frame_time * 1e6,
        minimum_cycle_wire_time_ms=cycle_wire * 1000,
    )
    derivation = [
        f'Plan {joints} joints at {rate:g} cycles/s with {p["frames_per_joint"]:g} frames
    per joint per cycle.',
        f"Frame demand is {frames:.6f} per second.",
        f"Assume {bits:g} total wire bits per frame at {baud:g} bit/s.",
        f"Required wire throughput is {demand:.6f} bit/s and utilization is
    {utilization:.6f}.",
        f"The minimum cycle wire time is {cycle_wire*1000:.6f} ms before additional
    contention or error handling.",
    ]
    notes = [
        (
            "The budget exceeds nominal bus capacity and is infeasible under these
    assumptions."
            if utilization > 1
            else "Nominal capacity is not exceeded, but margin and worst-case latency still
    need analysis."
        ),
        "The assumed frame length is a teaching budget input; confirm the actual bus
    generation, protocol and traffic pattern.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
```

```
    args = parser.parse_args()
    parameters = {
        "joints": 6,
        "cycle_rate": 500.0,
        "frame_bits": 135.0,
        "bitrate": 1000000.0,
        "frames_per_joint": 2.0,
    }
    parameters["cycle_rate"] = [50.0, 100.0, 200.0, 300.0, 500.0, 1000.0][
        args.case
    ]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Start from traffic semantics: one command plus one response gives a different budget from broadcast commands and asynchronous telemetry. Treat utilization above one as a mathematical impossibility, not a tuning issue.

I. Expected behavior and six worked cases

The controlled variable is cycle_rate. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

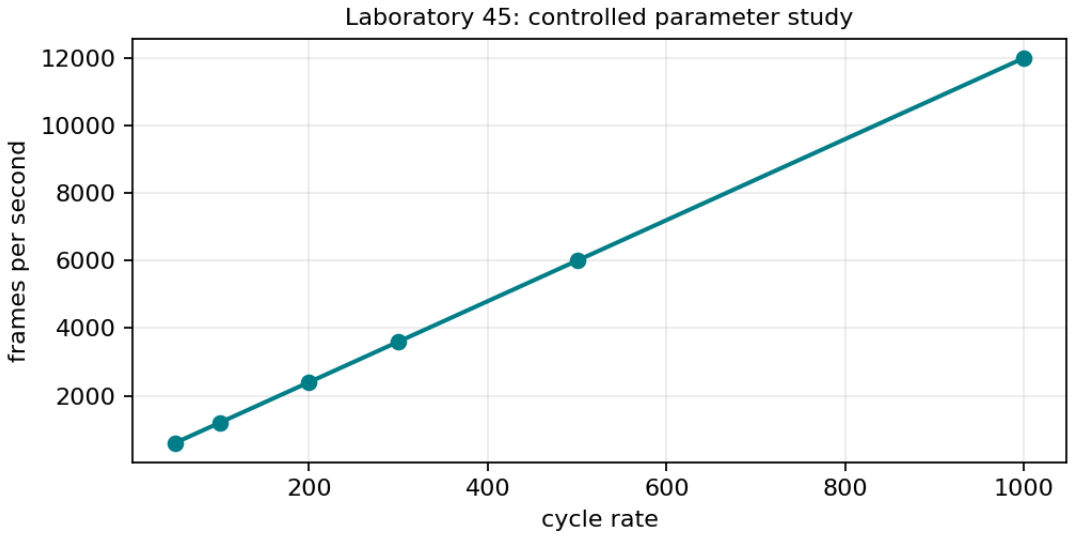


Figure L45. frames per second as cycle_rate changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	cycle_rate	frames per second
0	50	600
1	100	1200
2	200	2400
3	300	3600
4	500	6000
5	1000	12000

Worked case 0: cycle_rate = 50

This case keeps the nominal configuration except for cycle_rate=50. The full input is {"bitrate": 1000000.0, "cycle_rate": 50.0, "frame_bits": 135.0, "frames_per_joint": 2.0, "joints": 6}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
frames per second	600

Computed quantity	Numerical result
required bits per second	81000
utilization fraction	0.081
one frame wire time us	135
minimum cycle wire time ms	1.62

1. Plan 6 joints at 50 cycles/s with 2 frames per joint per cycle.
2. Frame demand is 600.000000 per second.
3. Assume 135 total wire bits per frame at 1e+06 bit/s.
4. Required wire throughput is 81000.000000 bit/s and utilization is 0.081000.
5. The minimum cycle wire time is 1.620000 ms before additional contention or error handling.

Worked case 1: cycle_rate = 100

This case keeps the nominal configuration except for cycle_rate=100. The full input is {"bitrate": 1000000.0, "cycle_rate": 100.0, "frame_bits": 135.0, "frames_per_joint": 2.0, "joints": 6}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
frames per second	1200
required bits per second	1.6200000e+05
utilization fraction	0.162
one frame wire time us	135
minimum cycle wire time ms	1.62

1. Plan 6 joints at 100 cycles/s with 2 frames per joint per cycle.
2. Frame demand is 1200.000000 per second.
3. Assume 135 total wire bits per frame at 1e+06 bit/s.

4. Required wire throughput is 162000.000000 bit/s and utilization is 0.162000.
5. The minimum cycle wire time is 1.620000 ms before additional contention or error handling.

Worked case 2: cycle_rate = 200

This case keeps the nominal configuration except for cycle_rate=200. The full input is {"bitrate": 1000000.0, "cycle_rate": 200.0, "frame_bits": 135.0, "frames_per_joint": 2.0, "joints": 6}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
frames per second	2400
required bits per second	3.2400000e+05
utilization fraction	0.324
one frame wire time us	135
minimum cycle wire time ms	1.62

1. Plan 6 joints at 200 cycles/s with 2 frames per joint per cycle.
2. Frame demand is 2400.000000 per second.
3. Assume 135 total wire bits per frame at 1e+06 bit/s.
4. Required wire throughput is 324000.000000 bit/s and utilization is 0.324000.
5. The minimum cycle wire time is 1.620000 ms before additional contention or error handling.

Worked case 3: cycle_rate = 300

This case keeps the nominal configuration except for cycle_rate=300. The full input is {"bitrate": 1000000.0, "cycle_rate": 300.0, "frame_bits": 135.0, "frames_per_joint": 2.0, "joints": 6}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
frames per second	3600
required bits per second	4.8600000e+05
utilization fraction	0.486
one frame wire time us	135
minimum cycle wire time ms	1.62

1. Plan 6 joints at 300 cycles/s with 2 frames per joint per cycle.
2. Frame demand is 3600.000000 per second.
3. Assume 135 total wire bits per frame at 1e+06 bit/s.
4. Required wire throughput is 486000.000000 bit/s and utilization is 0.486000.
5. The minimum cycle wire time is 1.620000 ms before additional contention or error handling.

Worked case 4: cycle_rate = 500

This case keeps the nominal configuration except for cycle_rate=500. The full input is {"bitrate": 1000000.0, "cycle_rate": 500.0, "frame_bits": 135.0, "frames_per_joint": 2.0, "joints": 6}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
frames per second	6000
required bits per second	8.1000000e+05
utilization fraction	0.81
one frame wire time us	135
minimum cycle wire time ms	1.62

1. Plan 6 joints at 500 cycles/s with 2 frames per joint per cycle.
2. Frame demand is 6000.000000 per second.

3. Assume 135 total wire bits per frame at 1e+06 bit/s.
4. Required wire throughput is 810000.000000 bit/s and utilization is 0.810000.
5. The minimum cycle wire time is 1.620000 ms before additional contention or error handling.

Worked case 5: cycle_rate = 1000

This case keeps the nominal configuration except for cycle_rate=1000. The full input is {"bitrate": 1000000.0, "cycle_rate": 1000.0, "frame_bits": 135.0, "frames_per_joint": 2.0, "joints": 6}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
frames per second	12000
required bits per second	1.6200000e+06
utilization fraction	1.62
one frame wire time us	135
minimum cycle wire time ms	1.62

1. Plan 6 joints at 1000 cycles/s with 2 frames per joint per cycle.
2. Frame demand is 12000.000000 per second.
3. Assume 135 total wire bits per frame at 1e+06 bit/s.
4. Required wire throughput is 1620000.000000 bit/s and utilization is 1.620000.
5. The minimum cycle wire time is 1.620000 ms before additional contention or error handling.

The budget exceeds nominal bus capacity and is infeasible under these assumptions.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Nominal capacity is not exceeded, but margin and worst-case latency still need analysis.

The assumed frame length is a teaching budget input; confirm the actual bus generation, protocol and traffic pattern.

K. Failure modes and troubleshooting

Counting only the eight-byte payload; ignoring response traffic and retransmissions; assuming all published interfaces share one bus.

L. Practical exercises and extension tasks

1. Allocate traffic across two confirmed independent buses and compare.
2. Add periodic diagnostic frames and reserved fault traffic.
3. Analyze worst-case response age under arbitration priorities.

M. Connection to the Asimov research platform

Obtain the exact actuator protocol and release-specific Asimov bus mapping before choosing bitrates or connecting a populated bus.

N. Completion record

Record your chosen case, predicted trend, measured frames_per_second, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 46, A synthetic packet codec and corruption tests. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 46 - A synthetic packet codec and corruption tests

A. The engineering question

Binary protocols need an explicit wire contract: field order, byte order, numeric representation, version, length and integrity checks.

B. Concepts and mathematical model

A successful unpack operation does not prove the values are safe. This laboratory defines a fictional teaching packet carrying one target and sequence number, then corrupts random bytes to test CRC-based rejection. It is deliberately incompatible with real actuators and must never be transmitted to them.

packet = magic + version + sequence + float target + CRC16(payload); validate length, magic, CRC, finiteness and range.

C. Real-world meaning and limits

Implement real motor encoding only from confirmed manufacturer protocol documentation, with inhibited-output tests and reviewed configuration.

D. Architecture and data flow

Configuration (target) -> numerical model -> packet_bytes, target_roundtrip_error_rad, single_bit_faults_rejected -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

target is a fictional joint angle in radians; sequence, magic, version and CRC format are fixed by this exercise only.

Parameter	Nominal value	Interpretation
target	0.1	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/46_packet_codec.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only target changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/46_packet_codec.py --case 0

.venv-book/bin/python expanded_lab/modules/46_packet_codec.py --case 3

.venv-book/bin/python expanded_lab/run.py packet_codec --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/46_packet_codec.py

```
"""A synthetic packet codec and corruption tests. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np

import struct


def crc16(data):
    crc = 0xFFFF
    for byte in data:
        crc ^= byte << 8
        for _ in range(8):
            crc = (
                ((crc << 1) ^ 0x1021) & 0xFFFF
                if crc & 0x8000
                else (crc << 1) & 0xFFFF
            )
    return crc


def encode(sequence, target):
    payload = struct.pack("<2sBIf", b"ED", 1, sequence, target)
    return payload + struct.pack("<H", crc16(payload))


def decode(packet):
    if len(packet) != 13:
        raise ValueError("length")
    payload, check = packet[:-2], struct.unpack("<H", packet[-2:])[0]
    if crc16(payload) != check:
        raise ValueError("crc")
    magic, version, sequence, target = struct.unpack("<2sBIf", payload)
    if magic != b"ED" or version != 1:
        raise ValueError("schema")
    if not np.isfinite(target) or abs(target) > 0.5:
        raise ValueError("target")
    return sequence, target


def experiment(p):
    rng = np.random.default_rng(59)
    rejected = 0
    n = 500
    target = p["target"]
    base = encode(7, target)
    seq, recovered = decode(base)
    for _ in range(n):
        damaged = bytearray(base)
        index = int(rng.integers(0, len(base)))
        damaged[index] ^= 1 << int(rng.integers(0, 8))
```

```
        try:
            decode(damaged)
        except ValueError:
            rejected += 1
    assert seq == 7 and rejected == n
    metrics = dict(
        packet_bytes=len(base),
        target_roundtrip_error_rad=abs(recovered - target),
        single_bit_faults_rejected=rejected,
        rejection_fraction=rejected / n,
    )
    derivation = [
        f"Encode sequence 7 and target {target:g} rad with explicit little-endian fields.",
        f"The complete fictional packet occupies {len(base)} bytes.",
        f"Float32 round-trip error is {abs(recovered-target):.9e} rad.",
        f"Inject {n} random single-bit corruptions across payload and checksum.",
        f"Reject {rejected} corrupted packets; no physical transport exists.",
    ]
    notes = [
        "CRC detects accidental corruption; it is not authentication or a defense against a
    deliberate sender.",
        "This CRC and packet layout are teaching choices, not the CAN wire format or Asimov
    motor protocol.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"target": 0.1}
    parameters["target"] = [-0.4, -0.1, 0.0, 0.1, 0.25, 0.4][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Separate codec integrity from application validity. Length and CRC are checked before interpreting fields; numerical range and schema checks occur after unpacking.

I. Expected behavior and six worked cases

The controlled variable is target. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

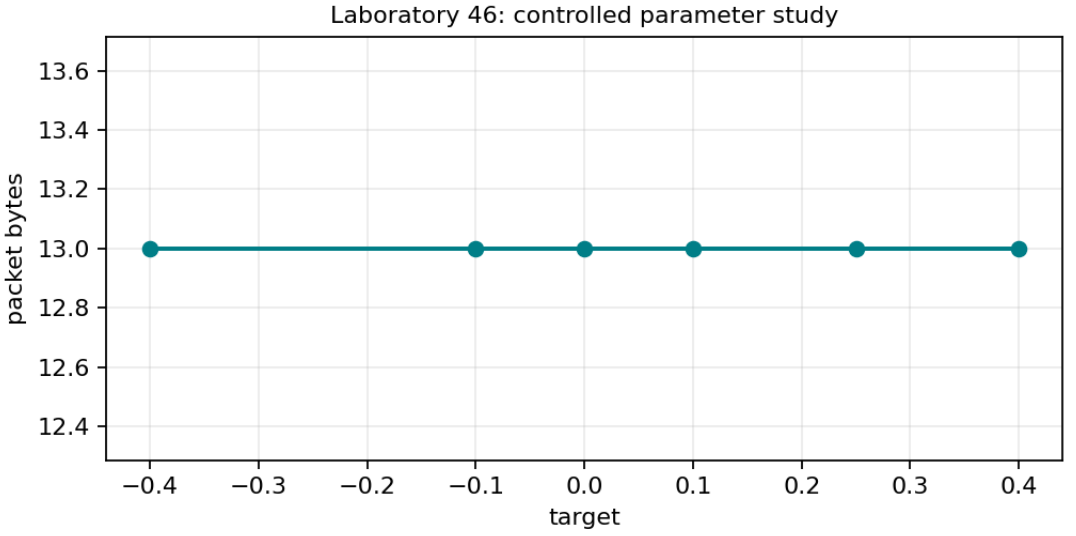


Figure L46. packet bytes as target changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	target	packet bytes
0	-0.4	13
1	-0.1	13
2	0	13
3	0.1	13
4	0.25	13
5	0.4	13

Worked case 0: target = -0.4

This case keeps the nominal configuration except for target=-0.4. The full input is {"target": -0.4}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
packet bytes	13
target roundtrip error rad	5.9604645e-09
single bit faults rejected	500
rejection fraction	1

1. Encode sequence 7 and target -0.4 rad with explicit little-endian fields.
2. The complete fictional packet occupies 13 bytes.
3. Float32 round-trip error is 5.960464455e-09 rad.
4. Inject 500 random single-bit corruptions across payload and checksum.
5. Reject 500 corrupted packets; no physical transport exists.

Worked case 1: target = -0.1

This case keeps the nominal configuration except for target=-0.1. The full input is {"target": -0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
packet bytes	13
target roundtrip error rad	1.4901161e-09
single bit faults rejected	500
rejection fraction	1

1. Encode sequence 7 and target -0.1 rad with explicit little-endian fields.
2. The complete fictional packet occupies 13 bytes.
3. Float32 round-trip error is 1.490116114e-09 rad.

4. Inject 500 random single-bit corruptions across payload and checksum.
5. Reject 500 corrupted packets; no physical transport exists.

Worked case 2: target = 0

This case keeps the nominal configuration except for target=0. The full input is {"target": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
packet bytes	13
target roundtrip error rad	0
single bit faults rejected	500
rejection fraction	1

1. Encode sequence 7 and target 0 rad with explicit little-endian fields.
2. The complete fictional packet occupies 13 bytes.
3. Float32 round-trip error is 0.000000000e+00 rad.
4. Inject 500 random single-bit corruptions across payload and checksum.
5. Reject 500 corrupted packets; no physical transport exists.

Worked case 3: target = 0.1

This case keeps the nominal configuration except for target=0.1. The full input is {"target": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
packet bytes	13
target roundtrip error rad	1.4901161e-09
single bit faults rejected	500
rejection fraction	1

1. Encode sequence 7 and target 0.1 rad with explicit little-endian fields.
2. The complete fictional packet occupies 13 bytes.
3. Float32 round-trip error is 1.490116114e-09 rad.
4. Inject 500 random single-bit corruptions across payload and checksum.
5. Reject 500 corrupted packets; no physical transport exists.

Worked case 4: target = 0.25

This case keeps the nominal configuration except for target=0.25. The full input is {"target": 0.25}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
packet bytes	13
target roundtrip error rad	0
single bit faults rejected	500
rejection fraction	1

1. Encode sequence 7 and target 0.25 rad with explicit little-endian fields.
2. The complete fictional packet occupies 13 bytes.
3. Float32 round-trip error is 0.000000000e+00 rad.
4. Inject 500 random single-bit corruptions across payload and checksum.
5. Reject 500 corrupted packets; no physical transport exists.

Worked case 5: target = 0.4

This case keeps the nominal configuration except for target=0.4. The full input is {"target": 0.4}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
packet bytes	13
target roundtrip error rad	5.9604645e-09

Computed quantity	Numerical result
single bit faults rejected	500
rejection fraction	1

1. Encode sequence 7 and target 0.4 rad with explicit little-endian fields.
2. The complete fictional packet occupies 13 bytes.
3. Float32 round-trip error is 5.960464455e-09 rad.
4. Inject 500 random single-bit corruptions across payload and checksum.
5. Reject 500 corrupted packets; no physical transport exists.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

CRC detects accidental corruption; it is not authentication or a defense against a deliberate sender.

This CRC and packet layout are teaching choices, not the CAN wire format or Asimov motor protocol.

K. Failure modes and troubleshooting

Assuming a checksum authenticates a sender; relying on native byte order; sending an educational packet to real hardware.

L. Practical exercises and extension tasks

1. Add a unit field and reject a mismatched unit.
2. Test truncated, appended and invalid-version packets.
3. Design replay protection independently of CRC.

M. Connection to the Asimov research platform

Implement real motor encoding only from confirmed manufacturer protocol documentation, with inhibited-output tests and reviewed configuration.

N. Completion record

Record your chosen case, predicted trend, measured packet_bytes, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 47, Observation ordering, normalization and schema hashes. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 47 - Observation ordering, normalization and schema hashes

A. The engineering question

A learned policy sees an ordered vector, not a collection of semantically labeled Python variables.

B. Concepts and mathematical model

Reordering two fields can produce plausible finite numbers and completely wrong actions. Normalization is also part of the policy interface and must be frozen for deployment. A schema hash can identify the ordered names and units, but only if the entire preprocessing contract is included. This laboratory demonstrates a silent ordering error and validates a canonical schema representation.

z_i=(x_i-mean_i)/std_i; policy_input order is part of the model; hash=SHA256(canonical schema bytes).

C. Real-world meaning and limits

Hardware estimators must supply the same units, frames, order, timing and normalization used in policy training.

D. Architecture and data flow

Configuration (gyro_offset) -> numerical model -> reference_action, correct_action, reordered_action -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

gyro_offset is an added angular-rate offset in rad/s; all normalized fields follow the explicitly named ordered schema.

Parameter	Nominal value	Interpretation
gyro_offset	0	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/47_observation_schema.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only gyro_offset changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/47_observation_schema.py --case 0

.venv-book/bin/python expanded_lab/modules/47_observation_schema.py --case 3

.venv-book/bin/python expanded_lab/run.py observation_schema --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/47_observation_schema.py

```
"""Observation ordering, normalization and schema hashes. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np

import hashlib
import json


def experiment(p):
    raw = np.array([0.2, -0.1, 0.5])
    mean = np.array([0.0, 0.0, 0.1])
    std = np.array([0.5, 0.5, 0.2])
    names = ["elbow_rad", "knee_rad", "gyro_rad_s"]
    normalized = (raw - mean) / std
    weights = np.array([0.4, -0.2, 0.1])
    baseline = float(weights @ normalized)
    shifted = raw.copy()
    shifted[2] += p["gyro_offset"]
    correct = float(weights @ ((shifted - mean) / std))
    wrong = float(weights @ ((shifted[[2, 1, 0]] - mean) / std))
    schema = dict(
        names=names, mean=mean.tolist(), std=std.tolist(), dtype="float32"
    )
    encoded = json.dumps(schema, sort_keys=True, separators=(",", ":")).encode()
    digest = hashlib.sha256(encoded).hexdigest()
    metrics = dict(
        reference_action=baseline,
        correct_action=correct,
        reordered_action=wrong,
        reordering_action_error=abs(correct - wrong),
        normalized_gyro=(shifted[2] - mean[2]) / std[2],
    )
    derivation = [
        f"Ordered fields are {names}.",
        f"Means are {mean.tolist()} and standard deviations are {std.tolist()}.",
        f'Apply gyro offset {p["gyro_offset"]:g} rad/s and normalize in the declared
    order.',
        f"Correct action is {correct:.6f}; a silent field swap produces {wrong:.6f}.",
        f"Canonical schema SHA256 is {digest}.",
    ]
    notes = [
        "A schema hash detects version mismatch only when both producer and consumer
    actually enforce it.",
        "Include clipping, frame conventions, history stacking and action scaling in a real
    deployment contract.",
    ]
    return metrics, derivation, notes

if __name__ == "__main__":
```

```
parser = argparse.ArgumentParser()
parser.add_argument("--case", type=int, default=0, choices=range(6))
args = parser.parse_args()
parameters = {"gyro_offset": 0.0}
parameters["gyro_offset"] = [-0.2, -0.1, 0.0, 0.1, 0.2, 0.5][args.case]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The reference linear actor is deliberately simple so the action difference can be traced exactly to preprocessing. Canonical serialization avoids hashes changing with dictionary formatting alone.

I. Expected behavior and six worked cases

The controlled variable is gyro_offset. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

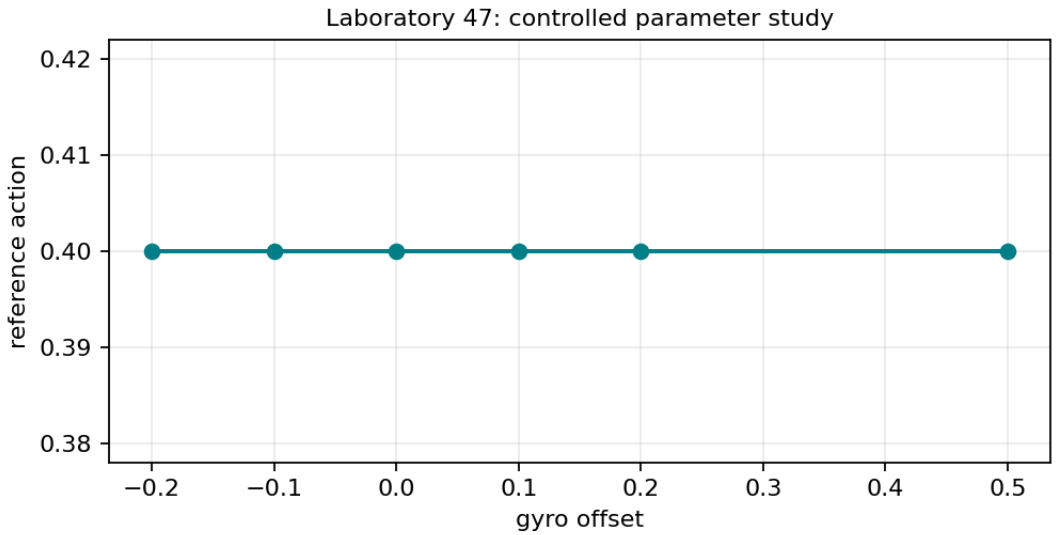


Figure L47. reference action as gyro_offset changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	gyro_offset	reference action
0	-0.2	0.4
1	-0.1	0.4
2	0	0.4
3	0.1	0.4
4	0.2	0.4
5	0.5	0.4

Worked case 0: gyro_offset = -0.2

This case keeps the nominal configuration except for gyro_offset=-0.2. The full input is {"gyro_offset": -0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reference action	0.4
correct action	0.3
reordered action	0.33
reordering action error	0.03
normalized gyro	1

1. Ordered fields are ['elbow_rad', 'knee_rad', 'gyro_rad_s'].
2. Means are [0.0, 0.0, 0.1] and standard deviations are [0.5, 0.5, 0.2].
3. Apply gyro offset -0.2 rad/s and normalize in the declared order.
4. Correct action is 0.300000; a silent field swap produces 0.330000.
5. Canonical schema SHA256 is `170b1c90844d3b85720078bbb063de561268d6bed4cf43601e2269735f9c48f8`.

Worked case 1: gyro_offset = -0.1

This case keeps the nominal configuration except for gyro_offset=-0.1. The full input is {"gyro_offset": -0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reference action	0.4
correct action	0.35
reordered action	0.41
reordering action error	0.06
normalized gyro	1.5

1. Ordered fields are ['elbow_rad', 'knee_rad', 'gyro_rad_s'].
2. Means are [0.0, 0.0, 0.1] and standard deviations are [0.5, 0.5, 0.2].
3. Apply gyro offset -0.1 rad/s and normalize in the declared order.
4. Correct action is 0.350000; a silent field swap produces 0.410000.
5. Canonical schema SHA256 is `170b1c90844d3b85720078bbb063de561268d6bed4cf43601e2269735f9c48f8`.

Worked case 2: gyro_offset = 0

This case keeps the nominal configuration except for gyro_offset=0. The full input is {"gyro_offset": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reference action	0.4
correct action	0.4
reordered action	0.49
reordering action error	0.09
normalized gyro	2

1. Ordered fields are ['elbow_rad', 'knee_rad', 'gyro_rad_s'].
2. Means are [0.0, 0.0, 0.1] and standard deviations are [0.5, 0.5, 0.2].
3. Apply gyro offset 0 rad/s and normalize in the declared order.
4. Correct action is 0.400000; a silent field swap produces 0.490000.
5. Canonical schema SHA256 is `170b1c90844d3b85720078bbb063de561268d6bed4cf43601e2269735f9c48f8`.

Worked case 3: gyro_offset = 0.1

This case keeps the nominal configuration except for gyro_offset=0.1. The full input is {"gyro_offset": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reference action	0.4
correct action	0.45
reordered action	0.57
reordering action error	0.12
normalized gyro	2.5

1. Ordered fields are ['elbow_rad', 'knee_rad', 'gyro_rad_s'].
2. Means are [0.0, 0.0, 0.1] and standard deviations are [0.5, 0.5, 0.2].
3. Apply gyro offset 0.1 rad/s and normalize in the declared order.
4. Correct action is 0.450000; a silent field swap produces 0.570000.
5. Canonical schema SHA256 is `170b1c90844d3b85720078bbb063de561268d6bed4cf43601e2269735f9c48f8`.

Worked case 4: gyro_offset = 0.2

This case keeps the nominal configuration except for gyro_offset=0.2. The full input is {"gyro_offset": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reference action	0.4
correct action	0.5
reordered action	0.65
reordering action error	0.15
normalized gyro	3

1. Ordered fields are ['elbow_rad', 'knee_rad', 'gyro_rad_s'].
2. Means are [0.0, 0.0, 0.1] and standard deviations are [0.5, 0.5, 0.2].
3. Apply gyro offset 0.2 rad/s and normalize in the declared order.
4. Correct action is 0.500000; a silent field swap produces 0.650000.
5. Canonical schema SHA256 is `170b1c90844d3b85720078bbb063de561268d6bed4cf43601e2269735f9c48f8`.

Worked case 5: gyro_offset = 0.5

This case keeps the nominal configuration except for gyro_offset=0.5. The full input is {"gyro_offset": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
reference action	0.4
correct action	0.65
reordered action	0.89
reordering action error	0.24
normalized gyro	4.5

1. Ordered fields are ['elbow_rad', 'knee_rad', 'gyro_rad_s'].
2. Means are [0.0, 0.0, 0.1] and standard deviations are [0.5, 0.5, 0.2].
3. Apply gyro offset 0.5 rad/s and normalize in the declared order.

4. Correct action is 0.650000; a silent field swap produces 0.890000.
5. Canonical schema SHA256 is `170b1c90844d3b85720078bbb063de561268d6bed4cf43601e2269735f9c48f8`.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

A schema hash detects version mismatch only when both producer and consumer actually enforce it.

Include clipping, frame conventions, history stacking and action scaling in a real deployment contract.

K. Failure modes and troubleshooting

Hashing unordered names without normalization; updating running statistics during evaluation; interpreting finite actions as evidence of correct semantics.

L. Practical exercises and extension tasks

1. Add action scales and clipping to the schema hash.
2. Create a migration test for an added observation field.
3. Compare batched and single-observation preprocessing.

M. Connection to the Asimov research platform

Hardware estimators must supply the same units, frames, order, timing and normalization used in policy training.

N. Completion record

Record your chosen case, predicted trend, measured reference_action, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 48, Offline replay, action parity and fault localization. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 48 - Offline replay, action parity and fault localization

A. The engineering question

Replay separates policy inference from the physical plant so interface changes can be investigated without motion.

B. Concepts and mathematical model

A useful replay artifact includes ordered observations, timestamps, policy version, preprocessing state and expected actions. Comparing only a single zero input misses errors that appear in the operating distribution. This experiment uses a deterministic nonlinear actor and injects a preprocessing scale mismatch, then reports action disagreement over a dataset rather than one sample.

action=tanh(W*observation+b); parity metrics include maximum absolute error, RMSE and threshold-exceedance fraction.

C. Real-world meaning and limits

Replay is a required intermediate step, but still must be followed by inhibited-output and supported physical validation.

D. Architecture and data flow

Configuration (scale_error_factor) -> numerical model -> action_rmse, maximum_absolute_action_error, mismatch_fraction -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

scale_error_factor multiplies one candidate observation channel; one is the intentionally matching baseline.

Parameter	Nominal value	Interpretation
scale_error_factor	1	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/48_replay.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only scale_error_factor changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/48_replay.py --case 0

.venv-book/bin/python expanded_lab/modules/48_replay.py --case 3

.venv-book/bin/python expanded_lab/run.py replay --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/48_replay.py

```
"""Offline replay, action parity and fault localization. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(61)
    obs = rng.normal(0, 1, (1000, 4))
    W = np.array([[0.2, -0.1, 0.3, 0.4], [-0.3, 0.2, 0.1, -0.2]])
    bias = np.array([0.05, -0.02])
    reference = np.tanh(obs @ W.T + bias)
    migrated = obs.copy()
    migrated[:, 2] *= p["scale_error_factor"]
    candidate = np.tanh(migrated @ W.T + bias)
    difference = candidate - reference
    per_sample = np.max(abs(difference), axis=1)
    metrics = dict(
        action_rmse=np.sqrt(np.mean(difference**2)),
        maximum_absolute_action_error=np.max(abs(difference)),
        mismatch_fraction=np.mean(per_sample > 1e-5),
        mean_action_magnitude=np.mean(abs(reference)),
    )
    derivation = [
        "Generate 1000 four-dimensional observations from a fixed seeded distribution.",
        f'The candidate preprocessing multiplies field 2 by {p["scale_error_factor"]:g}.',
        "Run the same tanh actor for reference and candidate pipelines.",
        f'Action RMSE is {metrics["action_rmse"]:.8f}; maximum disagreement is
    {metrics["maximum_absolute_action_error"]:.8f}.',
        f'Fraction exceeding 1e-5 action error is {metrics["mismatch_fraction"]:.6f}.',
    ]
    notes = [
        "The replay distribution is synthetic; physical logs and edge cases must be included
     before deployment.",
        "Exact inference parity does not validate the policy behavior or the physical
    actuator mapping.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"scale_error_factor": 1.0}
    parameters["scale_error_factor"] = [0.5, 0.9, 0.99, 1.0, 1.01, 1.1][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
```

```
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Keep the actor identical while changing only preprocessing. Dataset-level metrics locate the regression before any controller or hardware backend is involved.

I. Expected behavior and six worked cases

The controlled variable is scale_error_factor. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

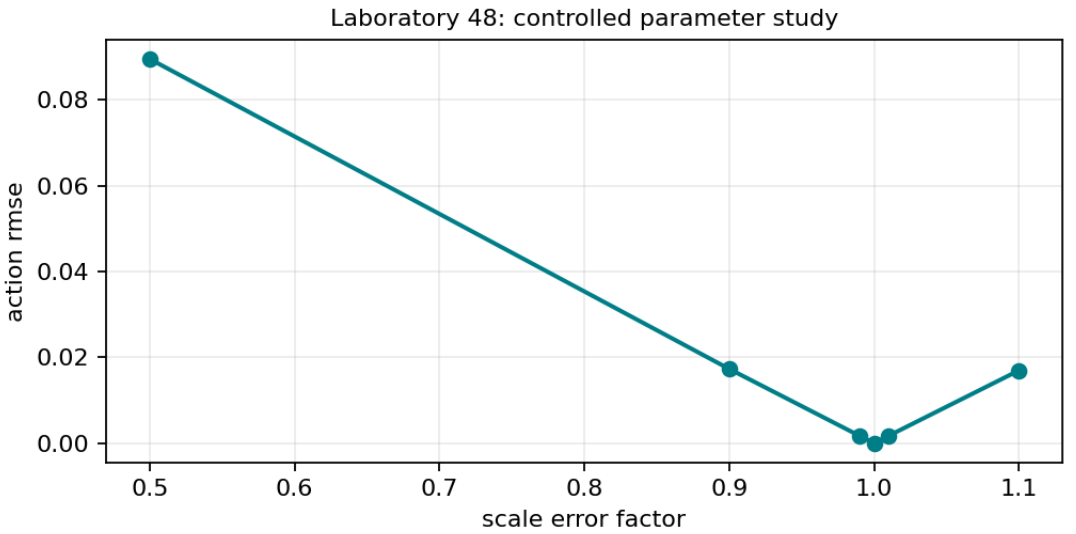


Figure L48. action rmse as scale_error_factor changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	scale_error_factor	action rmse
0	0.5	0.08955793
1	0.9	0.01731354
2	0.99	0.00171569

Case	scale_error_factor	action rmse
3	1	0
4	1.01	0.00171212
5	1.1	0.01695659

Worked case 0: scale_error_factor = 0.5

This case keeps the nominal configuration except for scale_error_factor=0.5. The full input is {"scale_error_factor": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
action rmse	0.08955793
maximum absolute action error	0.45855058
mismatch fraction	1
mean action magnitude	0.34551997

1. Generate 1000 four-dimensional observations from a fixed seeded distribution.
2. The candidate preprocessing multiplies field 2 by 0.5.
3. Run the same tanh actor for reference and candidate pipelines.
4. Action RMSE is 0.08955793; maximum disagreement is 0.45855058.
5. Fraction exceeding 1e-5 action error is 1.000000.

Worked case 1: scale_error_factor = 0.9

This case keeps the nominal configuration except for scale_error_factor=0.9. The full input is {"scale_error_factor": 0.9}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
action rmse	0.01731354
maximum absolute action error	0.09223477
mismatch fraction	1
mean action magnitude	0.34551997

1. Generate 1000 four-dimensional observations from a fixed seeded distribution.
2. The candidate preprocessing multiplies field 2 by 0.9.
3. Run the same tanh actor for reference and candidate pipelines.
4. Action RMSE is 0.01731354; maximum disagreement is 0.09223477.
5. Fraction exceeding 1e-5 action error is 1.000000.

Worked case 2: scale_error_factor = 0.99

This case keeps the nominal configuration except for scale_error_factor=0.99. The full input is {"scale_error_factor": 0.99}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
action rmse	0.00171569
maximum absolute action error	0.00920524
mismatch fraction	0.994
mean action magnitude	0.34551997

1. Generate 1000 four-dimensional observations from a fixed seeded distribution.
2. The candidate preprocessing multiplies field 2 by 0.99.
3. Run the same tanh actor for reference and candidate pipelines.
4. Action RMSE is 0.00171569; maximum disagreement is 0.00920524.

5. Fraction exceeding 1e-5 action error is 0.994000.

Worked case 3: scale_error_factor = 1

This case keeps the nominal configuration except for scale_error_factor=1. The full input is {"scale_error_factor": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
action rmse	0
maximum absolute action error	0
mismatch fraction	0
mean action magnitude	0.34551997

1. Generate 1000 four-dimensional observations from a fixed seeded distribution.
2. The candidate preprocessing multiplies field 2 by 1.
3. Run the same tanh actor for reference and candidate pipelines.
4. Action RMSE is 0.00000000; maximum disagreement is 0.00000000.
5. Fraction exceeding 1e-5 action error is 0.000000.

Worked case 4: scale_error_factor = 1.01

This case keeps the nominal configuration except for scale_error_factor=1.01. The full input is {"scale_error_factor": 1.01}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
action rmse	0.00171212
maximum absolute action error	0.00919545
mismatch fraction	0.994
mean action magnitude	0.34551997

1. Generate 1000 four-dimensional observations from a fixed seeded distribution.
2. The candidate preprocessing multiplies field 2 by 1.01.
3. Run the same tanh actor for reference and candidate pipelines.
4. Action RMSE is 0.00171212; maximum disagreement is 0.00919545.
5. Fraction exceeding 1e-5 action error is 0.994000.

Worked case 5: scale_error_factor = 1.1

This case keeps the nominal configuration except for scale_error_factor=1.1. The full input is {"scale_error_factor": 1.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
action rmse	0.01695659
maximum absolute action error	0.09126159
mismatch fraction	1
mean action magnitude	0.34551997

1. Generate 1000 four-dimensional observations from a fixed seeded distribution.
2. The candidate preprocessing multiplies field 2 by 1.1.
3. Run the same tanh actor for reference and candidate pipelines.
4. Action RMSE is 0.01695659; maximum disagreement is 0.09126159.
5. Fraction exceeding 1e-5 action error is 1.000000.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The replay distribution is synthetic; physical logs and edge cases must be included before deployment.

Exact inference parity does not validate the policy behavior or the physical actuator mapping.

K. Failure modes and troubleshooting

Testing only zero observations; comparing unclipped exported actions with clipped framework outputs; using unrelated normalization checkpoints.

L. Practical exercises and extension tasks

1. Add timestamp-based observation history and replay it exactly.
2. Compare float32 and float64 inference tolerances.
3. Inject a joint-name permutation and identify the affected action channels.

M. Connection to the Asimov research platform

Replay is a required intermediate step, but still must be followed by inhibited-output and supported physical validation.

N. Completion record

Record your chosen case, predicted trend, measured action_rmse, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 49, Ring buffers, bounded memory and time-window diagnostics. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 49 - Ring buffers, bounded memory and time-window diagnostics

A. The engineering question

Long-running robot processes cannot retain every high-rate sample in an unbounded Python list.

B. Concepts and mathematical model

A ring buffer bounds memory by overwriting old data, but that choice defines the available diagnostic horizon. Logging and visualization should consume copied snapshots without blocking the control loop. This laboratory implements a fixed-size numerical buffer and verifies wraparound, chronology and horizon as capacity changes.

write_index=total_samples mod capacity; chronological snapshot starts at the oldest retained index after wrap.

C. Real-world meaning and limits

Telemetry transport and rendering must remain outside the timing-critical actuator path and expose dropped-sample counts.

D. Architecture and data flow

Configuration (capacity) -> numerical model -> retained_samples, horizon_s, allocated_bytes -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

capacity is samples; three float64 columns store timestamp and two diagnostic signals at 500 Hz.

Parameter	Nominal value	Interpretation
capacity	1000	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/49_telemetry_buffer.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only capacity changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/49_telemetry_buffer.py --case 0

.venv-book/bin/python expanded_lab/modules/49_telemetry_buffer.py --case 3

.venv-book/bin/python expanded_lab/run.py telemetry_buffer --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/49_telemetry_buffer.py

```
"""Ring buffers, bounded memory and time-window diagnostics. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    capacity = int(p["capacity"])
    rate = 500
    n = 5000
    buffer = np.zeros((capacity, 3))
    count = 0
    for i in range(n):
        t = i / rate
        buffer[count % capacity] = [t, np.sin(t), np.cos(t)]
        count += 1
    retained = min(count, capacity)
    if count <= capacity:
        snapshot = buffer[:retained].copy()
    else:
        start = count % capacity
        snapshot = np.concatenate([buffer[start:], buffer[:start]]).copy()
    assert np.all(np.diff(snapshot[:, 0]) > 0)
    expected_last = (n - 1) / rate
    assert np.isclose(snapshot[-1, 0], expected_last)
    horizon = snapshot[-1, 0] - snapshot[0, 0]
    metrics = dict(
        retained_samples=retained,
        horizon_s=horizon,
        allocated_bytes=buffer.nbytes,
        oldest_timestamp_s=snapshot[0, 0],
        latest_timestamp_s=snapshot[-1, 0],
    )
    derivation = [
        f"Write {n} samples at {rate} Hz into capacity {capacity}.",
        f"The allocated numerical storage is {buffer.nbytes} bytes.",
        f"After wrapping, retain {retained} most recent samples.",
        f"Chronological snapshot runs from {snapshot[0,0]:.6f} to {snapshot[-1,0]:.6f} s.",
        f"The retained diagnostic horizon is {horizon:.6f} s.",
    ]
    notes = [
        "Memory accounting covers the array payload, not interpreter, transport or rendering
     overhead.",
        "The example is single-threaded; concurrent producers/consumers require a reviewed
    synchronization or lock-free design.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
```

```
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"capacity": 1000}
    parameters["capacity"] = [100, 250, 500, 1000, 2000, 5000][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The wrap index and chronological view are separate concerns. Copying the returned snapshot prevents a plot from reading values while the producer overwrites them.

I. Expected behavior and six worked cases

The controlled variable is capacity. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

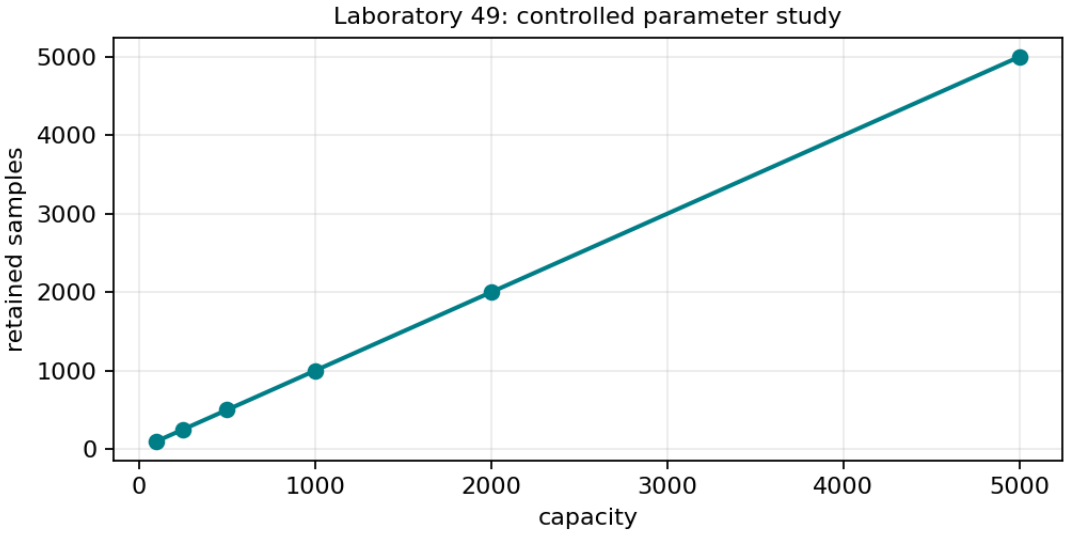


Figure L49. retained samples as capacity changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	capacity	retained samples
0	100	100
1	250	250
2	500	500
3	1000	1000
4	2000	2000
5	5000	5000

Worked case 0: capacity = 100

This case keeps the nominal configuration except for capacity=100. The full input is {"capacity": 100}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
retained samples	100
horizon s	0.198
allocated bytes	2400
oldest timestamp s	9.8
latest timestamp s	9.998

1. Write 5000 samples at 500 Hz into capacity 100.
2. The allocated numerical storage is 2400 bytes.
3. After wrapping, retain 100 most recent samples.
4. Chronological snapshot runs from 9.800000 to 9.998000 s.
5. The retained diagnostic horizon is 0.198000 s.

Worked case 1: capacity = 250

This case keeps the nominal configuration except for capacity=250. The full input is {"capacity": 250}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
retained samples	250
horizon s	0.498
allocated bytes	6000
oldest timestamp s	9.5
latest timestamp s	9.998

1. Write 5000 samples at 500 Hz into capacity 250.
2. The allocated numerical storage is 6000 bytes.
3. After wrapping, retain 250 most recent samples.
4. Chronological snapshot runs from 9.500000 to 9.998000 s.
5. The retained diagnostic horizon is 0.498000 s.

Worked case 2: capacity = 500

This case keeps the nominal configuration except for capacity=500. The full input is {"capacity": 500}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
retained samples	500
horizon s	0.998
allocated bytes	12000
oldest timestamp s	9
latest timestamp s	9.998

1. Write 5000 samples at 500 Hz into capacity 500.

2. The allocated numerical storage is 12000 bytes.
3. After wrapping, retain 500 most recent samples.
4. Chronological snapshot runs from 9.000000 to 9.998000 s.
5. The retained diagnostic horizon is 0.998000 s.

Worked case 3: capacity = 1000

This case keeps the nominal configuration except for capacity=1000. The full input is {"capacity": 1000}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
retained samples	1000
horizon s	1.998
allocated bytes	24000
oldest timestamp s	8
latest timestamp s	9.998

1. Write 5000 samples at 500 Hz into capacity 1000.
2. The allocated numerical storage is 24000 bytes.
3. After wrapping, retain 1000 most recent samples.
4. Chronological snapshot runs from 8.000000 to 9.998000 s.
5. The retained diagnostic horizon is 1.998000 s.

Worked case 4: capacity = 2000

This case keeps the nominal configuration except for capacity=2000. The full input is {"capacity": 2000}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
retained samples	2000
horizon s	3.998

Computed quantity	Numerical result
allocated bytes	48000
oldest timestamp s	6
latest timestamp s	9.998

1. Write 5000 samples at 500 Hz into capacity 2000.
2. The allocated numerical storage is 48000 bytes.
3. After wrapping, retain 2000 most recent samples.
4. Chronological snapshot runs from 6.000000 to 9.998000 s.
5. The retained diagnostic horizon is 3.998000 s.

Worked case 5: capacity = 5000

This case keeps the nominal configuration except for capacity=5000. The full input is {"capacity": 5000}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
retained samples	5000
horizon s	9.998
allocated bytes	1.2000000e+05
oldest timestamp s	0
latest timestamp s	9.998

1. Write 5000 samples at 500 Hz into capacity 5000.
2. The allocated numerical storage is 120000 bytes.
3. After wrapping, retain 5000 most recent samples.
4. Chronological snapshot runs from 0.000000 to 9.998000 s.
5. The retained diagnostic horizon is 9.998000 s.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Memory accounting covers the array payload, not interpreter, transport or rendering overhead.

The example is single-threaded; concurrent producers/consumers require a reviewed synchronization or lock-free design.

K. Failure modes and troubleshooting

Unbounded logging in a control process; plotting mutable array views; assuming bounded storage guarantees nonblocking disk writes.

L. Practical exercises and extension tasks

1. Add a consumer that decimates by timestamp rather than index.
2. Define a disk-writer queue with a bounded drop policy.
3. Measure control-loop impact of snapshot copies.

M. Connection to the Asimov research platform

Telemetry transport and rendering must remain outside the timing-critical actuator path and expose dropped-sample counts.

N. Completion record

Record your chosen case, predicted trend, measured retained_samples, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 50, Fault persistence, debounce and explicit re-arming. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 50 - Fault persistence, debounce and explicit re-arming

A. The engineering question

A single transient threshold crossing may be noise, while a persistent violation may require a stop.

B. Concepts and mathematical model

Debounce trades false alarms against detection delay. After a real fault is latched, a return to normal measurements should not automatically re-enable motion. This laboratory injects both short spikes and a sustained thermal-like violation, then measures detection latency under different debounce durations. It distinguishes detection policy from the physical action that a real supervisor must execute.

consecutive_bad+=1 while violation; latch when consecutive_bad>=N; latched state persists until an explicit validated reset.

C. Real-world meaning and limits

Physical fault responses depend on load support, stored energy and independent isolation; this numerical detector is not the protective system.

D. Architecture and data flow

Configuration (debounce_seconds) -> numerical model -> required_bad_samples, first_latch_time_s, detection_latency_from_sustained_fault_s -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

debounce_seconds is the required persistence duration at a fixed 10 ms measurement interval.

Parameter	Nominal value	Interpretation
debounce_seconds	0.05	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/50_fault_supervisor.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only debounce_seconds changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/50_fault_supervisor.py --case 0

.venv-book/bin/python expanded_lab/modules/50_fault_supervisor.py --case 3

.venv-book/bin/python expanded_lab/run.py fault_supervisor --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/50_fault_supervisor.py

```
"""Fault persistence, debounce and explicit re-arming. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    dt = 0.01
    t = np.arange(0, 5, dt)
    signal = np.ones(len(t)) * 30.0
    signal[50] = 90.0
    signal[100:102] = 90.0
    signal[t >= 3] = 90.0
    required = max(1, round(p["debounce_seconds"] / dt))
    count = 0
    latched = False
    first = None
    false_before = 0
    for i, value in enumerate(signal):
        count = count + 1 if value > 70 else 0
        if count >= required and not latched:
            latched = True
            first = t[i]
            false_before += t[i] < 3
    latency = -1.0 if first is None else first - 3.0
    metrics = dict(
        required_bad_samples=required,
        first_latch_time_s=-1 if first is None else first,
        detection_latency_from_sustained_fault_s=latency,
        false_latch_before_fault=false_before,
        latched_final=float(latched),
    )
    derivation = [
        f'At 100 Hz, debounce duration {p["debounce_seconds"]:g} s becomes {required}
    consecutive samples.',
        "Inject a one-sample spike at 0.5 s and a two-sample spike at 1 s.",
        "At 3 s, begin a sustained threshold violation.",
        f"First latched fault occurs at {first} s; early false-latch count is
    {false_before}.",
        f"Relative to sustained-fault onset, reported latency is {latency:.6f} s; a negative
     value means an earlier spike already latched.",
    ]
    notes = [
        "The temperature values and threshold are synthetic and do not specify a physical
    motor limit.",
        "Detection delay must be included in the reviewed response envelope; debouncing
    every fault category identically is inappropriate.",
    ]
    return metrics, derivation, notes
```

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"debounce_seconds": 0.05}
    parameters["debounce_seconds"] = [0.01, 0.02, 0.03, 0.05, 0.1, 0.2][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The fault latch changes only once, while the persistence counter can reset on healthy measurements. Early spikes deliberately expose the difference between nuisance rejection and delayed true detection.

I. Expected behavior and six worked cases

The controlled variable is debounce_seconds. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

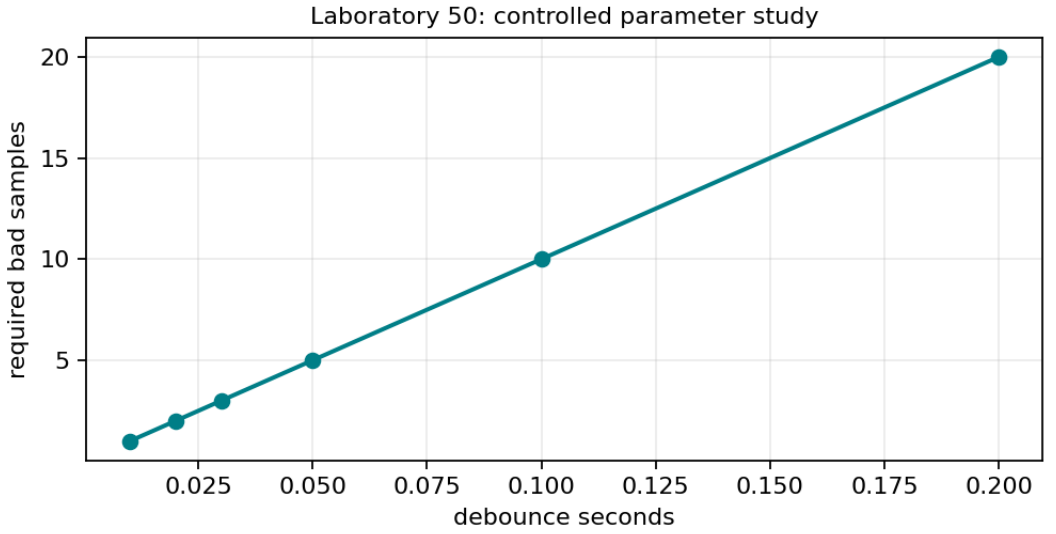


Figure L50. required bad samples as debounce_seconds changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	debounce_seconds	required bad samples
0	0.01	1
1	0.02	2
2	0.03	3
3	0.05	5
4	0.1	10
5	0.2	20

Worked case 0: debounce_seconds = 0.01

This case keeps the nominal configuration except for debounce_seconds=0.01. The full input is {"debounce_seconds": 0.01}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
required bad samples	1
first latch time s	0.5
detection latency from sustained fault s	-2.5
false latch before fault	1
latched final	1

1. At 100 Hz, debounce duration 0.01 s becomes 1 consecutive samples.
2. Inject a one-sample spike at 0.5 s and a two-sample spike at 1 s.
3. At 3 s, begin a sustained threshold violation.
4. First latched fault occurs at 0.5 s; early false-latch count is 1.
5. Relative to sustained-fault onset, reported latency is -2.500000 s; a negative value means an earlier spike already latched.

Worked case 1: debounce_seconds = 0.02

This case keeps the nominal configuration except for debounce_seconds=0.02. The full input is {"debounce_seconds": 0.02}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
required bad samples	2
first latch time s	1.01
detection latency from sustained fault s	-1.99
false latch before fault	1
latched final	1

1. At 100 Hz, debounce duration 0.02 s becomes 2 consecutive samples.
2. Inject a one-sample spike at 0.5 s and a two-sample spike at 1 s.
3. At 3 s, begin a sustained threshold violation.
4. First latched fault occurs at 1.01 s; early false-latch count is 1.
5. Relative to sustained-fault onset, reported latency is -1.990000 s; a negative value means an earlier spike already latched.

Worked case 2: debounce_seconds = 0.03

This case keeps the nominal configuration except for debounce_seconds=0.03. The full input is {"debounce_seconds": 0.03}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
required bad samples	3
first latch time s	3.02
detection latency from sustained fault s	0.02
false latch before fault	0

Computed quantity	Numerical result
latched final	1

1. At 100 Hz, debounce duration 0.03 s becomes 3 consecutive samples.
2. Inject a one-sample spike at 0.5 s and a two-sample spike at 1 s.
3. At 3 s, begin a sustained threshold violation.
4. First latched fault occurs at 3.02 s; early false-latch count is 0.
5. Relative to sustained-fault onset, reported latency is 0.020000 s; a negative value means an earlier spike already latched.

Worked case 3: debounce_seconds = 0.05

This case keeps the nominal configuration except for debounce_seconds=0.05. The full input is {"debounce_seconds": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
required bad samples	5
first latch time s	3.04
detection latency from sustained fault s	0.04
false latch before fault	0
latched final	1

1. At 100 Hz, debounce duration 0.05 s becomes 5 consecutive samples.
2. Inject a one-sample spike at 0.5 s and a two-sample spike at 1 s.
3. At 3 s, begin a sustained threshold violation.
4. First latched fault occurs at 3.04 s; early false-latch count is 0.
5. Relative to sustained-fault onset, reported latency is 0.040000 s; a negative value means an earlier spike already latched.

Worked case 4: debounce_seconds = 0.1

This case keeps the nominal configuration except for debounce_seconds=0.1. The full input is {"debounce_seconds": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
required bad samples	10
first latch time s	3.09
detection latency from sustained fault s	0.09
false latch before fault	0
latched final	1

1. At 100 Hz, debounce duration 0.1 s becomes 10 consecutive samples.
2. Inject a one-sample spike at 0.5 s and a two-sample spike at 1 s.
3. At 3 s, begin a sustained threshold violation.
4. First latched fault occurs at 3.09 s; early false-latch count is 0.
5. Relative to sustained-fault onset, reported latency is 0.090000 s; a negative value means an earlier spike already latched.

Worked case 5: debounce_seconds = 0.2

This case keeps the nominal configuration except for debounce_seconds=0.2. The full input is {"debounce_seconds": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
required bad samples	20
first latch time s	3.19
detection latency from sustained fault s	0.19
false latch before fault	0

Computed quantity	Numerical result
latched final	1

1. At 100 Hz, debounce duration 0.2 s becomes 20 consecutive samples.
2. Inject a one-sample spike at 0.5 s and a two-sample spike at 1 s.
3. At 3 s, begin a sustained threshold violation.
4. First latched fault occurs at 3.19 s; early false-latch count is 0.
5. Relative to sustained-fault onset, reported latency is 0.190000 s; a negative value means an earlier spike already latched.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The temperature values and threshold are synthetic and do not specify a physical motor limit.

Detection delay must be included in the reviewed response envelope; debouncing every fault category identically is inappropriate.

K. Failure modes and troubleshooting

Automatically clearing a fault on healthy data; debouncing an emergency input without design justification; ignoring total reaction travel during debounce.

L. Practical exercises and extension tasks

1. Use different policies for stale feedback, overcurrent and thermal drift.
2. Add an explicit inactive reset state and prohibit direct active resumption.
3. Log the first triggering sample and pre-fault buffer.

M. Connection to the Asimov research platform

Physical fault responses depend on load support, stored energy and independent isolation; this numerical detector is not the protective system.

N. Completion record

Record your chosen case, predicted trend, measured required_bad_samples, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 51, Markov decision processes and discounted reward. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Part XI - Learning algorithms, evaluation and embodied skills

Before training a difficult humanoid behavior, verify the mathematics of rewards, returns, policy updates, action processing and evaluation. Small deterministic examples are faster to diagnose than a million-step rollout. The learning laboratories connect algorithmic definitions with the deployed observation/action interface, then connect perception evidence to an allowlisted skill. A reward curve, detector confidence or successful API call cannot substitute for measured task completion.

Laboratory 51 - Markov decision processes and discounted reward

A. The engineering question

A reward function expresses what training should favor, but its meaning depends on episode duration, timestep and discounting.

B. Concepts and mathematical model

Reward per step can change when policy rate changes even if physical behavior stays identical. Discounted return weights near-term and distant outcomes differently. This laboratory compares a short immediate reward with a delayed larger reward and calculates the effective discount horizon. It is a reasoning tool for reward design, not an argument that one discount factor is universally correct.

G_t=sum(gamma^k*r[t+k]); continuous-time discount gamma=exp(-dt/tau); horizon scale approx 1/(1-gamma) steps.

C. Real-world meaning and limits

Physical task success must be evaluated directly; training return alone does not establish useful robot behavior.

D. Architecture and data flow

Configuration (gamma) -> numerical model -> immediate_return, delayed_return, delayed_preference_margin -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

gamma is a dimensionless per-policy-step discount strictly between zero and one; the teaching policy step is 20 ms.

Parameter	Nominal value	Interpretation
gamma	0.99	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/51_discounted_return.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only gamma changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/51_discounted_return.py --case 0

.venv-book/bin/python expanded_lab/modules/51_discounted_return.py --case 3

.venv-book/bin/python expanded_lab/run.py discounted_return --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/51_discounted_return.py

```
"""Markov decision processes and discounted reward. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    gamma = p["gamma"]
    dt = 0.02
    immediate = np.zeros(100)
    delayed = np.zeros(100)
    immediate[0] = 1.0
    delayed[50] = 3.0
    weights = gamma ** np.arange(100)
    a = float(weights @ immediate)
    b = float(weights @ delayed)
    horizon = 1 / (1 - gamma)
    tau = -dt / np.log(gamma)
    metrics = dict(
        immediate_return=a,
        delayed_return=b,
        delayed_preference_margin=b - a,
        effective_horizon_steps=horizon,
        continuous_time_scale_s=tau,
    )
    derivation = [
        f"Choose gamma={gamma:g} at control period {dt:g} s.",
        "One candidate receives reward 1 now; the other receives reward 3 after 50 steps.",
        f"Immediate return is {a:.6f}; delayed return is 3*gamma^50={b:.6f}.",
        f"The geometric horizon scale is {horizon:.6f} steps.",
        f"The matching exponential time constant is {tau:.6f} s.",
    ]
    notes = [
        (
            "The delayed outcome is preferred under this discount."
            if b > a
            else "The immediate outcome is preferred despite the larger undiscounted delayed
     reward."
        ),
        "Episode termination, reward scaling and physical-time interpretation remain part of
     the task specification.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"gamma": 0.99}
    parameters["gamma"] = [0.9, 0.95, 0.97, 0.98, 0.99, 0.995][args.case]
```

```
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The sparse reward sequences isolate discount preference from action dynamics. Mapping gamma to seconds prevents accidental task changes when control frequency changes.

I. Expected behavior and six worked cases

The controlled variable is gamma. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

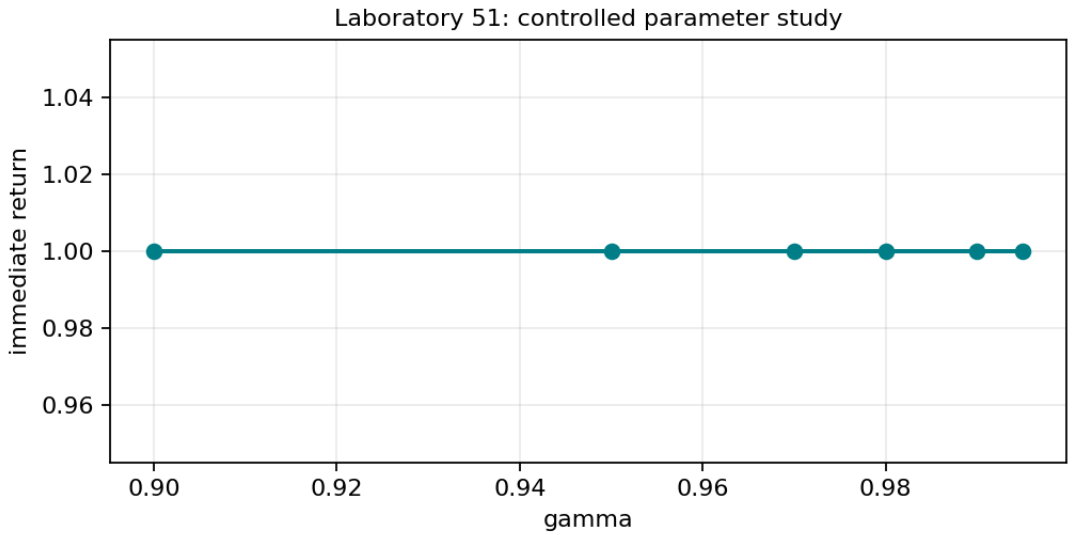


Figure L51. immediate return as gamma changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	gamma	immediate return
0	0.9	1

Case	gamma	immediate return
1	0.95	1
2	0.97	1
3	0.98	1
4	0.99	1
5	0.995	1

Worked case 0: gamma = 0.9

This case keeps the nominal configuration except for gamma=0.9. The full input is {"gamma": 0.9}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
immediate return	1
delayed return	0.01546133
delayed preference margin	-0.98453867
effective horizon steps	10
continuous time scale s	0.18982443

1. Choose gamma=0.9 at control period 0.02 s.
2. One candidate receives reward 1 now; the other receives reward 3 after 50 steps.
3. Immediate return is 1.000000; delayed return is 3*gamma^50=0.015461.
4. The geometric horizon scale is 10.000000 steps.
5. The matching exponential time constant is 0.189824 s.

Worked case 1: gamma = 0.95

This case keeps the nominal configuration except for gamma=0.95. The full input is {"gamma": 0.95}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
immediate return	1
delayed return	0.23083493
delayed preference margin	-0.76916507
effective horizon steps	20
continuous time scale s	0.38991451

1. Choose gamma=0.95 at control period 0.02 s.
2. One candidate receives reward 1 now; the other receives reward 3 after 50 steps.
3. Immediate return is 1.000000; delayed return is 3*gamma^50=0.230835.
4. The geometric horizon scale is 20.000000 steps.
5. The matching exponential time constant is 0.389915 s.

Worked case 2: gamma = 0.97

This case keeps the nominal configuration except for gamma=0.97. The full input is {"gamma": 0.97}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
immediate return	1
delayed return	0.65419613
delayed preference margin	-0.34580387
effective horizon steps	33.33333333
continuous time scale s	0.6566159

1. Choose gamma=0.97 at control period 0.02 s.
2. One candidate receives reward 1 now; the other receives reward 3 after 50 steps.
3. Immediate return is 1.000000; delayed return is 3*gamma^50=0.654196.
4. The geometric horizon scale is 33.333333 steps.
5. The matching exponential time constant is 0.656616 s.

Worked case 3: gamma = 0.98

This case keeps the nominal configuration except for gamma=0.98. The full input is {"gamma": 0.98}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
immediate return	1
delayed return	1.09250904
delayed preference margin	0.09250904
effective horizon steps	50
continuous time scale s	0.98996633

1. Choose gamma=0.98 at control period 0.02 s.
2. One candidate receives reward 1 now; the other receives reward 3 after 50 steps.
3. Immediate return is 1.000000; delayed return is 3*gamma^50=1.092509.
4. The geometric horizon scale is 50.000000 steps.
5. The matching exponential time constant is 0.989966 s.

The delayed outcome is preferred under this discount.

Worked case 4: gamma = 0.99

This case keeps the nominal configuration except for gamma=0.99. The full input is {"gamma": 0.99}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
immediate return	1
delayed return	1.8150182
delayed preference margin	0.8150182
effective horizon steps	100
continuous time scale s	1.98998325

1. Choose gamma=0.99 at control period 0.02 s.
2. One candidate receives reward 1 now; the other receives reward 3 after 50 steps.
3. Immediate return is 1.000000; delayed return is 3*gamma^50=1.815018.
4. The geometric horizon scale is 100.000000 steps.
5. The matching exponential time constant is 1.989983 s.

The delayed outcome is preferred under this discount.

Worked case 5: gamma = 0.995

This case keeps the nominal configuration except for gamma=0.995. The full input is {"gamma": 0.995}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
immediate return	1
delayed return	2.33493767
delayed preference margin	1.33493767
effective horizon steps	200
continuous time scale s	3.98999165

1. Choose gamma=0.995 at control period 0.02 s.
2. One candidate receives reward 1 now; the other receives reward 3 after 50 steps.

3. Immediate return is 1.000000; delayed return is 3*gamma^50=2.334938.
4. The geometric horizon scale is 200.000000 steps.
5. The matching exponential time constant is 3.989992 s.

The delayed outcome is preferred under this discount.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The immediate outcome is preferred despite the larger undiscounted delayed reward.

Episode termination, reward scaling and physical-time interpretation remain part of the task specification.

K. Failure modes and troubleshooting

Comparing returns across different timesteps without rescaling; rewarding survival while ignoring command tracking; hiding terminal rewards in long episodes.

L. Practical exercises and extension tasks

1. Keep a fixed discount time constant while changing policy period.
2. Add per-second energy cost and discretize it consistently.
3. Construct a reward-hacking example with a safe-looking but unproductive action.

M. Connection to the Asimov research platform

Physical task success must be evaluated directly; training return alone does not establish useful robot behavior.

N. Completion record

Record your chosen case, predicted trend, measured immediate_return, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 52, Temporal-difference residuals, GAE and episode boundaries. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 52 - Temporal-difference residuals, GAE and episode boundaries

A. The engineering question

An advantage estimate compares an action's outcome with the value baseline for its state.

B. Concepts and mathematical model

Generalized Advantage Estimation combines temporal-difference residuals over multiple horizons, trading variance against sensitivity to value errors. Episode boundaries matter: a true terminal state differs from an externally imposed time limit that still has continuation value. This laboratory computes the backward recurrence explicitly and exposes the effect of the lambda parameter while holding rewards and values fixed.

delta_t=r_t+gamma*V_next-V_t; A_t=delta_t+gamma*lambda*A_next, with valid terminal/continuation masks.

C. Real-world meaning and limits

Correct learning plumbing is essential before expensive humanoid rollouts; masking bugs can invalidate apparently successful training.

D. Architecture and data flow

Configuration (gae_lambda) -> numerical model -> initial_advantage, final_advantage, advantage_std -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

gae_lambda is the dimensionless trace parameter; rewards, value predictions and one terminal boundary are fixed.

Parameter	Nominal value	Interpretation
gae_lambda	0.95	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/52_gae.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only gae_lambda changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/52_gae.py --case 0

.venv-book/bin/python expanded_lab/modules/52_gae.py --case 3

.venv-book/bin/python expanded_lab/run.py gae --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/52_gae.py

```
"""Temporal-difference residuals, GAE and episode boundaries. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    gamma = 0.99
    lam = p["gae_lambda"]
    rewards = np.array([0.2, 0.1, -0.1, 0.3, 1.0])
    values = np.array([0.4, 0.45, 0.3, 0.5, 0.6, 0.0])
    terminal = np.array([False, False, False, False, True])
    advantage = np.zeros(5)
    carry = 0.0
    residual = []
    for t in reversed(range(5)):
        continuation = 0.0 if terminal[t] else 1.0
        delta = rewards[t] + gamma * values[t + 1] * continuation - values[t]
        carry = delta + gamma * lam * continuation * carry
        advantage[t] = carry
        residual.append(delta)
    returns = advantage + values[:-1]
    metrics = dict(
        initial_advantage=advantage[0],
        final_advantage=advantage[-1],
        advantage_std=np.std(advantage),
        initial_value_target=returns[0],
        mean_advantage=np.mean(advantage),
    )
    derivation = [
        f"Use rewards {rewards.tolist()} and value predictions {values.tolist()}.",
        f"The last transition is truly terminal; gamma={gamma:g}, lambda={lam:g}.",
        f"Backward TD residuals in chronological order are {list(reversed(residual))}.",
        f"The resulting advantage vector is {advantage.tolist()}.",
        f"Value targets A+V are {returns.tolist()}.",
    ]
    notes = [
        "A time-limit truncation may retain a bootstrap value while still preventing
    recurrence into the next reset episode.",
        "Correct masking needs separate termination and rollout-boundary semantics; one
    generic done flag can hide a bug.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"gae_lambda": 0.95}
```

```
    parameters["gae_lambda"] = [0.0, 0.25, 0.5, 0.75, 0.95, 1.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Run the recurrence backward and retain intermediate deltas so every term can be checked by hand. Lambda zero reduces to one-step TD residuals; lambda one includes the longest available trace.

I. Expected behavior and six worked cases

The controlled variable is gae_lambda. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

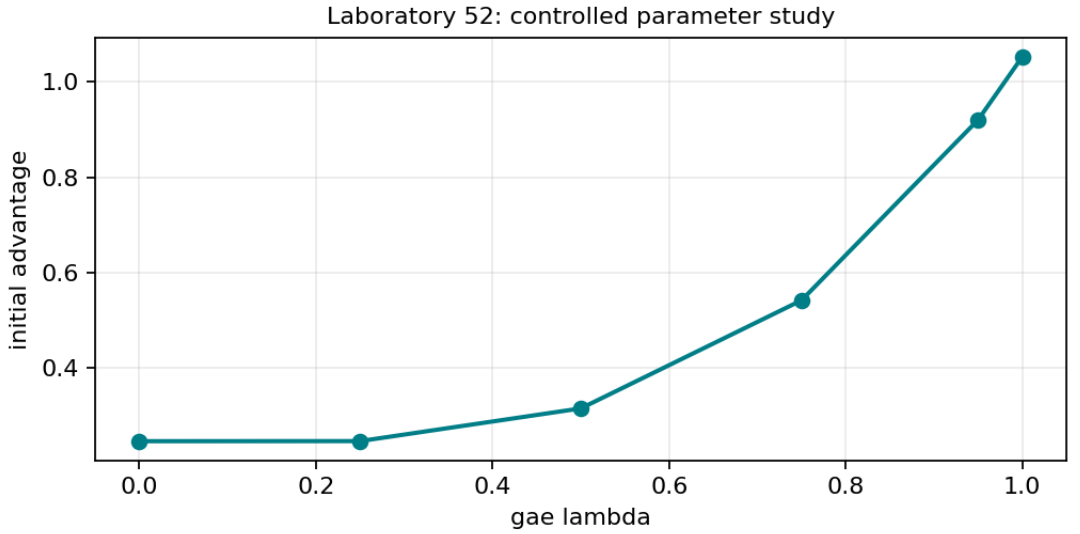


Figure L52. initial advantage as gae_lambda changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	gae_lambda	initial advantage
0	0	0.2455

Case	gae_lambda	initial advantage
1	0.25	0.24567618
2	0.5	0.3143445
3	0.75	0.54137891
4	0.95	0.92042199
5	1	1.05267571

Worked case 0: gae_lambda = 0

This case keeps the nominal configuration except for gae_lambda=0. The full input is {"gae_lambda": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
initial advantage	0.2455
final advantage	0.4
advantage std	0.17516381
initial value target	0.6455
mean advantage	0.2163

1. Use rewards [0.2, 0.1, -0.1, 0.3, 1.0] and value predictions [0.4, 0.45, 0.3, 0.5, 0.6, 0.0].
2. The last transition is truly terminal; gamma=0.99, lambda=0.
3. Backward TD residuals in chronological order are [np.float64(0.24549999999999994), np.float64(-0.05299999999999999), np.float64(0.09500000000000003), np.float64(0.3939999999999999), np.float64(0.4)].
4. The resulting advantage vector is [0.24549999999999994, -0.05299999999999999, 0.09500000000000003, 0.3939999999999999, 0.4].

5. Value targets A+V are [0.6455, 0.397, 0.395, 0.8939999999999999, 1.0].

Worked case 1: gae_lambda = 0.25

This case keeps the nominal configuration except for gae_lambda=0.25. The full input is {"gae_lambda": 0.25}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
initial advantage	0.24567618
final advantage	0.4
advantage std	0.16884058
initial value target	0.64567618
mean advantage	0.2712811

1. Use rewards [0.2, 0.1, -0.1, 0.3, 1.0] and value predictions [0.4, 0.45, 0.3, 0.5, 0.6, 0.0].
2. The last transition is truly terminal; gamma=0.99, lambda=0.25.
3. Backward TD residuals in chronological order are [np.float64(0.24549999999999994), np.float64(-0.05299999999999999), np.float64(0.09500000000000003), np.float64(0.3939999999999999), np.float64(0.4)].
4. The resulting advantage vector is [0.24567617823437493, 0.0007118312500000099, 0.2170175, 0.4929999999999999, 0.4].
5. Value targets A+V are [0.645676178234375, 0.45071183125000003, 0.5170175, 0.9929999999999999, 1.0].

Worked case 2: gae_lambda = 0.5

This case keeps the nominal configuration except for gae_lambda=0.5. The full input is {"gae_lambda": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
initial advantage	0.3143445

Computed quantity	Numerical result
final advantage	0.4
advantage std	0.14620289
initial value target	0.7143445
mean advantage	0.36669286

1. Use rewards [0.2, 0.1, -0.1, 0.3, 1.0] and value predictions [0.4, 0.45, 0.3, 0.5, 0.6, 0.0].
2. The last transition is truly terminal; gamma=0.99, lambda=0.5.
3. Backward TD residuals in chronological order are [np.float64(0.24549999999999994), np.float64(-0.05299999999999999), np.float64(0.09500000000000003), np.float64(0.3939999999999999), np.float64(0.4)].
4. The resulting advantage vector is [0.31434450099999994, 0.13907979999999998, 0.38803999999999994, 0.5919999999999999, 0.4].
5. Value targets A+V are [0.714344501, 0.5890797999999999, 0.68804, 1.0919999999999999, 1.0].

Worked case 3: gae_lambda = 0.75

This case keeps the nominal configuration except for gae_lambda=0.75. The full input is {"gae_lambda": 0.75}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
initial advantage	0.54137891
final advantage	0.4
advantage std	0.11516543
initial value target	0.94137891
mean advantage	0.52778731

1. Use rewards [0.2, 0.1, -0.1, 0.3, 1.0] and value predictions [0.4, 0.45, 0.3, 0.5, 0.6, 0.0].
2. The last transition is truly terminal; gamma=0.99, lambda=0.75.
3. Backward TD residuals in chronological order are [np.float64(0.24549999999999994), np.float64(-0.05299999999999999), np.float64(0.09500000000000003), np.float64(0.3939999999999999), np.float64(0.4)].
4. The resulting advantage vector is [0.5413789131718747, 0.3984901187499998, 0.6080674999999998, 0.6909999999999998, 0.4].
5. Value targets A+V are [0.9413789131718747, 0.8484901187499998, 0.9080674999999998, 1.1909999999999998, 1.0].

Worked case 4: gae_lambda = 0.95

This case keeps the nominal configuration except for gae_lambda=0.95. The full input is {"gae_lambda": 0.95}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
initial advantage	0.92042199
final advantage	0.4
advantage std	0.17596429
initial value target	1.32042199
mean advantage	0.7255231

1. Use rewards [0.2, 0.1, -0.1, 0.3, 1.0] and value predictions [0.4, 0.45, 0.3, 0.5, 0.6, 0.0].
2. The last transition is truly terminal; gamma=0.99, lambda=0.95.
3. Backward TD residuals in chronological order are [np.float64(0.24549999999999994), np.float64(-0.05299999999999999), np.float64(0.09500000000000003), np.float64(0.3939999999999999), np.float64(0.4)].
4. The resulting advantage vector is [0.9204219867172749, 0.71762040055, 0.8193731, 0.7702, 0.4].

5. Value targets A+V are [1.320421986717275, 1.16762040055, 1.1193731, 1.2702, 1.0].

Worked case 5: gae_lambda = 1

This case keeps the nominal configuration except for gae_lambda=1. The full input is {"gae_lambda": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
initial advantage	1.05267571
final advantage	0.4
advantage std	0.21414113
initial value target	1.45267571
mean advantage	0.78702094

1. Use rewards [0.2, 0.1, -0.1, 0.3, 1.0] and value predictions [0.4, 0.45, 0.3, 0.5, 0.6, 0.0].
2. The last transition is truly terminal; gamma=0.99, lambda=1.
3. Backward TD residuals in chronological order are [np.float64(0.24549999999999994), np.float64(-0.05299999999999999), np.float64(0.09500000000000003), np.float64(0.3939999999999999), np.float64(0.4)].
4. The resulting advantage vector is [1.05267571, 0.815329, 0.8771, 0.7899999999999999, 0.4].
5. Value targets A+V are [1.4526757099999998, 1.265329, 1.1771, 1.29, 1.0].

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

A time-limit truncation may retain a bootstrap value while still preventing recurrence into the next reset episode.

Correct masking needs separate termination and rollout-boundary semantics; one generic done flag can hide a bug.

K. Failure modes and troubleshooting

Bootstrapping a true terminal as if it continued; propagating advantages across reset episodes; normalizing advantages without checking their construction.

L. Practical exercises and extension tasks

1. Replace the final terminal with a time-limit truncation and nonzero final value.
2. Calculate the lambda-zero and lambda-one results manually.
3. Inject a value bias and compare its effect across lambda values.

M. Connection to the Asimov research platform

Correct learning plumbing is essential before expensive humanoid rollouts; masking bugs can invalidate apparently successful training.

N. Completion record

Record your chosen case, predicted trend, measured initial_advantage, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 53, PPO clipping, advantage signs and surrogate objectives. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 53 - PPO clipping, advantage signs and surrogate objectives

A. The engineering question

PPO's clipped surrogate is a constrained-update heuristic, not a direct clipping of the motor command.

B. Concepts and mathematical model

The probability ratio measures how the new policy changes likelihood of sampled actions. Whether clipping limits the objective depends on both the ratio and the advantage sign. A common implementation mistake is to apply the same intuitive rule to positive and negative advantages without evaluating the minimum. This laboratory computes both branches on a fixed batch so the behavior is inspectable.

L=min(ratio*A,clip(ratio,1-epsilon,1+epsilon)*A); policy loss=-mean(L).

C. Real-world meaning and limits

Policy optimization constraints do not replace bounded action scaling, runtime supervision or physical commissioning.

D. Architecture and data flow

Configuration (clip_epsilon) -> numerical model -> policy_loss, unclipped_loss, objective_changed_fraction -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

clip_epsilon is the dimensionless probability-ratio interval half-width; the sampled ratios and advantages are fixed.

Parameter	Nominal value	Interpretation
clip_epsilon	0.2	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/53_ppo_clipping.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only clip_epsilon changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/53_ppo_clipping.py --case 0

.venv-book/bin/python expanded_lab/modules/53_ppo_clipping.py --case 3

.venv-book/bin/python expanded_lab/run.py ppo_clipping --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/53_ppo_clipping.py

```
"""PPO clipping, advantage signs and surrogate objectives. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    epsilon = p["clip_epsilon"]
    ratios = np.array([0.5, 0.8, 0.95, 1.0, 1.05, 1.2, 1.5])
    advantage = np.array([-1.0, -0.5, 1.0, -0.2, 0.5, 1.0, -1.0])
    raw = ratios * advantage
    clipped = np.clip(ratios, 1 - epsilon, 1 + epsilon) * advantage
    objective = np.minimum(raw, clipped)
    active = np.abs(raw - objective) > 1e-12
    metrics = dict(
        policy_loss=-np.mean(objective),
        unclipped_loss=-np.mean(raw),
        objective_changed_fraction=np.mean(active),
        ratio_outside_clip_fraction=np.mean(
            (ratios < 1 - epsilon) | (ratios > 1 + epsilon)
        ),
        minimum_sample_objective=np.min(objective),
    )
    derivation = [
        f"Probability ratios are {ratios.tolist()}.",
        f"Advantages are {advantage.tolist()}; clipping epsilon is {epsilon:g}.",
        f"Unclipped sample objectives are {raw.tolist()}.",
        f"Apply the minimum with the clipped branch to obtain {objective.tolist()}.",
        f'Negative mean objective gives policy loss {metrics["policy_loss"]:.6f}.',
    ]
    notes = [
        "Ratio outside the clip interval is not identical to a changed surrogate
    contribution; advantage sign matters.",
        "This calculation omits value loss, entropy bonus, optimizer behavior and rollout
    collection.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"clip_epsilon": 0.2}
    parameters["clip_epsilon"] = [0.05, 0.1, 0.15, 0.2, 0.3, 0.4][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
```

```
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Calculate raw and clipped products before taking their elementwise minimum. Inspect one positive-advantage and one negative-advantage sample by hand to verify the sign logic.

I. Expected behavior and six worked cases

The controlled variable is clip_epsilon. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

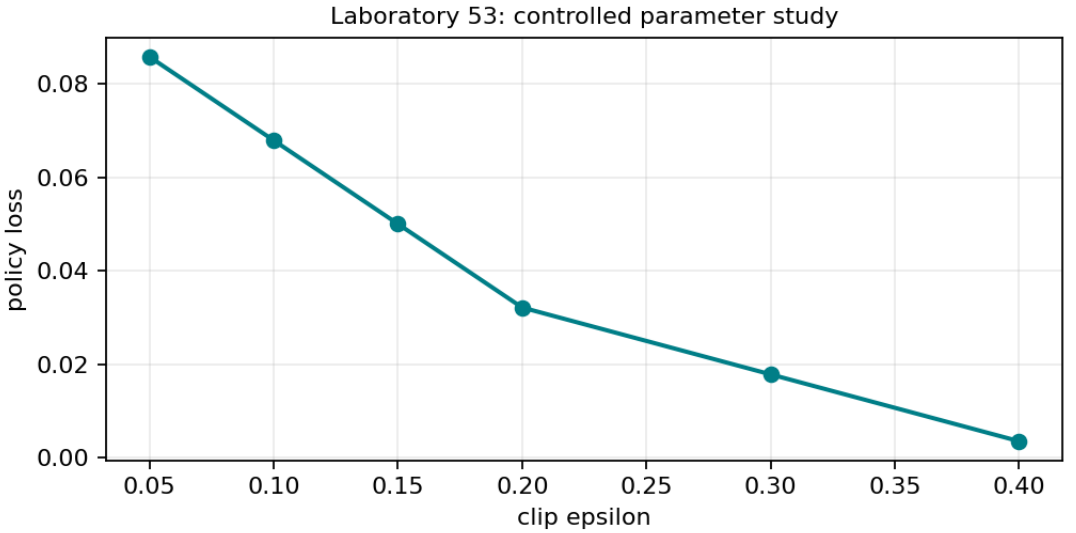


Figure L53. policy loss as clip_epsilon changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	clip_epsilon	policy loss
0	0.05	0.08571429
1	0.1	0.06785714
2	0.15	0.05

Case	clip_epsilon	policy loss
3	0.2	0.03214286
4	0.3	0.01785714
5	0.4	0.00357143

Worked case 0: clip_epsilon = 0.05

This case keeps the nominal configuration except for clip_epsilon=0.05. The full input is {"clip_epsilon": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
policy loss	0.08571429
unclipped loss	-0.01071429
objective changed fraction	0.42857143
ratio outside clip fraction	0.57142857
minimum sample objective	-1.5

1. Probability ratios are [0.5, 0.8, 0.95, 1.0, 1.05, 1.2, 1.5].
2. Advantages are [-1.0, -0.5, 1.0, -0.2, 0.5, 1.0, -1.0]; clipping epsilon is 0.05.
3. Unclipped sample objectives are [-0.5, -0.4, 0.95, -0.2, 0.525, 1.2, -1.5].
4. Apply the minimum with the clipped branch to obtain [-0.95, -0.475, 0.95, -0.2, 0.525, 1.05, -1.5].
5. Negative mean objective gives policy loss 0.085714.

Worked case 1: clip_epsilon = 0.1

This case keeps the nominal configuration except for clip_epsilon=0.1. The full input is {"clip_epsilon": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
policy loss	0.06785714
unclipped loss	-0.01071429
objective changed fraction	0.42857143
ratio outside clip fraction	0.57142857
minimum sample objective	-1.5

1. Probability ratios are [0.5, 0.8, 0.95, 1.0, 1.05, 1.2, 1.5].
2. Advantages are [-1.0, -0.5, 1.0, -0.2, 0.5, 1.0, -1.0]; clipping epsilon is 0.1.
3. Unclipped sample objectives are [-0.5, -0.4, 0.95, -0.2, 0.525, 1.2, -1.5].
4. Apply the minimum with the clipped branch to obtain [-0.9, -0.45, 0.95, -0.2, 0.525, 1.1, -1.5].
5. Negative mean objective gives policy loss 0.067857.

Worked case 2: clip_epsilon = 0.15

This case keeps the nominal configuration except for clip_epsilon=0.15. The full input is {"clip_epsilon": 0.15}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
policy loss	0.05
unclipped loss	-0.01071429
objective changed fraction	0.42857143
ratio outside clip fraction	0.57142857
minimum sample objective	-1.5

1. Probability ratios are [0.5, 0.8, 0.95, 1.0, 1.05, 1.2, 1.5].
2. Advantages are [-1.0, -0.5, 1.0, -0.2, 0.5, 1.0, -1.0]; clipping epsilon is 0.15.
3. Unclipped sample objectives are [-0.5, -0.4, 0.95, -0.2, 0.525, 1.2, -1.5].

4. Apply the minimum with the clipped branch to obtain [-0.85, -0.425, 0.95, -0.2, 0.525, 1.15, -1.5].
5. Negative mean objective gives policy loss 0.050000.

Worked case 3: clip_epsilon = 0.2

This case keeps the nominal configuration except for clip_epsilon=0.2. The full input is {"clip_epsilon": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
policy loss	0.03214286
unclipped loss	-0.01071429
objective changed fraction	0.14285714
ratio outside clip fraction	0.28571429
minimum sample objective	-1.5

1. Probability ratios are [0.5, 0.8, 0.95, 1.0, 1.05, 1.2, 1.5].
2. Advantages are [-1.0, -0.5, 1.0, -0.2, 0.5, 1.0, -1.0]; clipping epsilon is 0.2.
3. Unclipped sample objectives are [-0.5, -0.4, 0.95, -0.2, 0.525, 1.2, -1.5].
4. Apply the minimum with the clipped branch to obtain [-0.8, -0.4, 0.95, -0.2, 0.525, 1.2, -1.5].
5. Negative mean objective gives policy loss 0.032143.

Worked case 4: clip_epsilon = 0.3

This case keeps the nominal configuration except for clip_epsilon=0.3. The full input is {"clip_epsilon": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
policy loss	0.01785714
unclipped loss	-0.01071429

Computed quantity	Numerical result
objective changed fraction	0.14285714
ratio outside clip fraction	0.28571429
minimum sample objective	-1.5

1. Probability ratios are [0.5, 0.8, 0.95, 1.0, 1.05, 1.2, 1.5].
2. Advantages are [-1.0, -0.5, 1.0, -0.2, 0.5, 1.0, -1.0]; clipping epsilon is 0.3.
3. Unclipped sample objectives are [-0.5, -0.4, 0.95, -0.2, 0.525, 1.2, -1.5].
4. Apply the minimum with the clipped branch to obtain [-0.7, -0.4, 0.95, -0.2, 0.525, 1.2, -1.5].
5. Negative mean objective gives policy loss 0.017857.

Worked case 5: clip_epsilon = 0.4

This case keeps the nominal configuration except for clip_epsilon=0.4. The full input is {"clip_epsilon": 0.4}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
policy loss	0.00357143
unclipped loss	-0.01071429
objective changed fraction	0.14285714
ratio outside clip fraction	0.28571429
minimum sample objective	-1.5

1. Probability ratios are [0.5, 0.8, 0.95, 1.0, 1.05, 1.2, 1.5].
2. Advantages are [-1.0, -0.5, 1.0, -0.2, 0.5, 1.0, -1.0]; clipping epsilon is 0.4.
3. Unclipped sample objectives are [-0.5, -0.4, 0.95, -0.2, 0.525, 1.2, -1.5].
4. Apply the minimum with the clipped branch to obtain [-0.6, -0.4, 0.95, -0.2, 0.525, 1.2, -1.5].
5. Negative mean objective gives policy loss 0.003571.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Ratio outside the clip interval is not identical to a changed surrogate contribution; advantage sign matters.

This calculation omits value loss, entropy bonus, optimizer behavior and rollout collection.

K. Failure modes and troubleshooting

Confusing policy-ratio clipping with action limits; maximizing a loss written for minimization; interpreting clipping as a physical safety constraint.

L. Practical exercises and extension tasks

1. Plot surrogate objective versus ratio for both advantage signs.
2. Add an entropy term and inspect its separate contribution.
3. Compare approximate KL diagnostics with the ratio distribution.

M. Connection to the Asimov research platform

Policy optimization constraints do not replace bounded action scaling, runtime supervision or physical commissioning.

N. Completion record

Record your chosen case, predicted trend, measured policy_loss, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 54, Gaussian policies, action limits and saturation probability. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 54 - Gaussian policies, action limits and saturation probability

A. The engineering question

A stochastic continuous policy often samples from a Gaussian distribution before applying environment bounds.

B. Concepts and mathematical model

Large standard deviation can create many clipped actions, changing the action distribution seen by the plant. A bounded tanh transformation has different density mathematics from simple clipping. This laboratory quantifies clipping frequency and variance reduction for a one-dimensional Gaussian policy. It also calculates Gaussian entropy so exploration magnitude has an interpretable scale.

a_raw~Normal(mu,sigma^2); a=clip(a_raw,-1,1);
entropy=0.5*log(2*pi*e*sigma^2).

C. Real-world meaning and limits

Physical inference generally uses deterministic policy evaluation plus explicit bounded action processing validated against training semantics.

D. Architecture and data flow

Configuration (mean, std) -> numerical model -> clipped_fraction, raw_mean, bounded_mean -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

mean and std are normalized action units before the environment action scale and joint limits.

Parameter	Nominal value	Interpretation
mean	0.2	Defined by the explicit chapter model and parameter contract
std	0.5	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/54_gaussian_policy.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only std changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/54_gaussian_policy.py --case 0

.venv-book/bin/python expanded_lab/modules/54_gaussian_policy.py --case 3

.venv-book/bin/python expanded_lab/run.py gaussian_policy --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/54_gaussian_policy.py

```
"""Gaussian policies, action limits and saturation probability. Numerical teaching
    experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(67)
    mu = p["mean"]
    sigma = p["std"]
    raw = rng.normal(mu, sigma, 20000)
    bounded = np.clip(raw, -1, 1)
    entropy = 0.5 * np.log(2 * np.pi * np.e * sigma * sigma)
    metrics = dict(
        clipped_fraction=np.mean(abs(raw) > 1),
        raw_mean=np.mean(raw),
        bounded_mean=np.mean(bounded),
        raw_std=np.std(raw),
        bounded_std=np.std(bounded),
        gaussian_entropy_nats=entropy,
    )
    derivation = [
        f"Sample 20,000 actions from mean {mu:g} and standard deviation {sigma:g}.",
        f"Unbounded Gaussian entropy is {entropy:.6f} nats.",
        f'Clip to [-1,1]; clipped fraction is {metrics["clipped_fraction"]:.6f}.',
        f"Raw sample standard deviation is {np.std(raw):.6f}; bounded standard deviation is
    {np.std(bounded):.6f}.",
        f"Bounded mean is {np.mean(bounded):.6f}, which can shift when clipping is
    asymmetric around the policy mean.",
    ]
    notes = [
        "The entropy formula describes the unbounded Gaussian, not the clipped mixed
    distribution.",
        "A tanh-squashed policy requires a change-of-variables correction when computing log
     probabilities.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"mean": 0.2, "std": 0.5}
    parameters["std"] = [0.05, 0.1, 0.2, 0.5, 1.0, 2.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
```

```
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Compare raw and applied distributions rather than logging only the sampled Gaussian parameters. A high clipped fraction can make exploration look numerically broad while applied actions concentrate at limits.

I. Expected behavior and six worked cases

The controlled variable is std. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

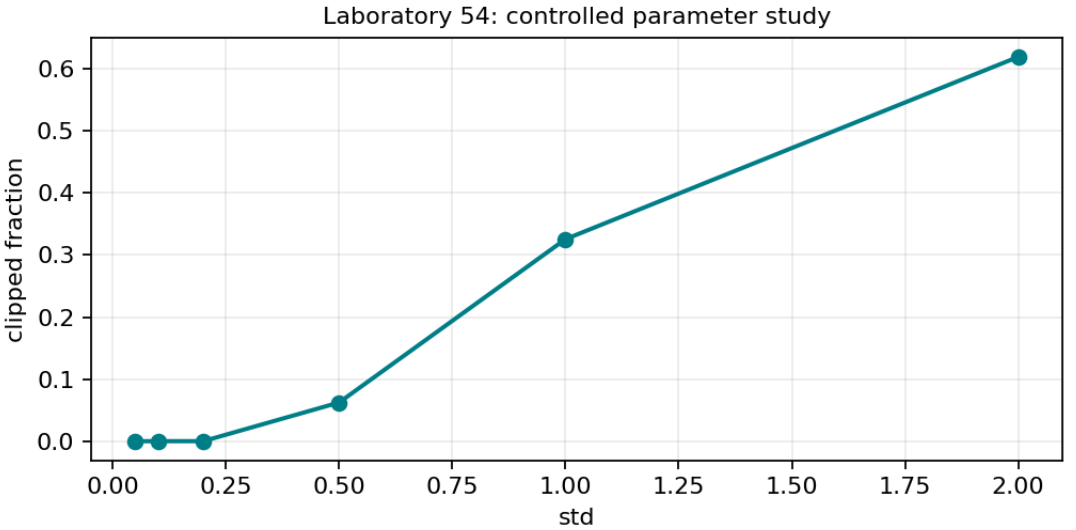


Figure L54. clipped fraction as std changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	std	clipped fraction
0	0.05	0
1	0.1	0
2	0.2	0.00005

Case	std	clipped fraction
3	0.5	0.0623
4	1	0.3254
5	2	0.61905

Worked case 0: std = 0.05

This case keeps the nominal configuration except for std=0.05. The full input is {"mean": 0.2, "std": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
clipped fraction	0
raw mean	0.2001035
bounded mean	0.2001035
raw std	0.04973828
bounded std	0.04973828
gaussian entropy nats	-1.57679374

1. Sample 20,000 actions from mean 0.2 and standard deviation 0.05.
2. Unbounded Gaussian entropy is -1.576794 nats.
3. Clip to [-1,1]; clipped fraction is 0.000000.
4. Raw sample standard deviation is 0.049738; bounded standard deviation is 0.049738.
5. Bounded mean is 0.200104, which can shift when clipping is asymmetric around the policy mean.

Worked case 1: std = 0.1

This case keeps the nominal configuration except for std=0.1. The full input is {"mean": 0.2, "std": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
clipped fraction	0
raw mean	0.200207
bounded mean	0.200207
raw std	0.09947656
bounded std	0.09947656
gaussian entropy nats	-0.88364656

1. Sample 20,000 actions from mean 0.2 and standard deviation 0.1.
2. Unbounded Gaussian entropy is -0.883647 nats.
3. Clip to [-1,1]; clipped fraction is 0.000000.
4. Raw sample standard deviation is 0.099477; bounded standard deviation is 0.099477.
5. Bounded mean is 0.200207, which can shift when clipping is asymmetric around the policy mean.

Worked case 2: std = 0.2

This case keeps the nominal configuration except for std=0.2. The full input is {"mean": 0.2, "std": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
clipped fraction	0.00005
raw mean	0.200414
bounded mean	0.20041272
raw std	0.19895312
bounded std	0.19894788
gaussian entropy nats	-0.19049938

1. Sample 20,000 actions from mean 0.2 and standard deviation 0.2.
2. Unbounded Gaussian entropy is -0.190499 nats.
3. Clip to [-1,1]; clipped fraction is 0.000050.
4. Raw sample standard deviation is 0.198953; bounded standard deviation is 0.198948.
5. Bounded mean is 0.200413, which can shift when clipping is asymmetric around the policy mean.

Worked case 3: std = 0.5

This case keeps the nominal configuration except for std=0.5. The full input is {"mean": 0.2, "std": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
clipped fraction	0.0623
raw mean	0.201035
bounded mean	0.19079612
raw std	0.4973828
bounded std	0.47122932
gaussian entropy nats	0.72579135

1. Sample 20,000 actions from mean 0.2 and standard deviation 0.5.
2. Unbounded Gaussian entropy is 0.725791 nats.
3. Clip to [-1,1]; clipped fraction is 0.062300.
4. Raw sample standard deviation is 0.497383; bounded standard deviation is 0.471229.
5. Bounded mean is 0.190796, which can shift when clipping is asymmetric around the policy mean.

Worked case 4: std = 1

This case keeps the nominal configuration except for std=1. The full input is {"mean": 0.2, "std": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
clipped fraction	0.3254
raw mean	0.20207
bounded mean	0.13974087
raw std	0.9947656
bounded std	0.70848934
gaussian entropy nats	1.41893853

1. Sample 20,000 actions from mean 0.2 and standard deviation 1.
2. Unbounded Gaussian entropy is 1.418939 nats.
3. Clip to [-1,1]; clipped fraction is 0.325400.
4. Raw sample standard deviation is 0.994766; bounded standard deviation is 0.708489.
5. Bounded mean is 0.139741, which can shift when clipping is asymmetric around the policy mean.

Worked case 5: std = 2

This case keeps the nominal configuration except for std=2. The full input is {"mean": 0.2, "std": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
clipped fraction	0.61905
raw mean	0.20414
bounded mean	0.0792849

Computed quantity	Numerical result
raw std	1.98953121
bounded std	0.85736778
gaussian entropy nats	2.11208571

1. Sample 20,000 actions from mean 0.2 and standard deviation 2.
2. Unbounded Gaussian entropy is 2.112086 nats.
3. Clip to [-1,1]; clipped fraction is 0.619050.
4. Raw sample standard deviation is 1.989531; bounded standard deviation is 0.857368.
5. Bounded mean is 0.079285, which can shift when clipping is asymmetric around the policy mean.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The entropy formula describes the unbounded Gaussian, not the clipped mixed distribution.

A tanh-squashed policy requires a change-of-variables correction when computing log probabilities.

K. Failure modes and troubleshooting

Using raw Gaussian entropy as the applied distribution entropy; omitting export-time clipping; changing action scaling without retraining/evaluation.

L. Practical exercises and extension tasks

1. Repeat with policy mean near an action boundary.
2. Compare clipping with tanh transformation histograms.
3. Propagate normalized actions into an actual simulated torque envelope.

M. Connection to the Asimov research platform

Physical inference generally uses deterministic policy evaluation plus explicit bounded action processing validated against training semantics.

N. Completion record

Record your chosen case, predicted trend, measured clipped_fraction, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 55, Streaming normalization, numerical stability and evaluation leakage. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 55 - Streaming normalization, numerical stability and evaluation leakage

A. The engineering question

Policies often require normalized observations.

B. Concepts and mathematical model

Streaming mean and variance can be estimated without retaining every sample, but evaluation must normally freeze the training statistics. Updating them on test observations changes the policy input distribution and leaks information across the evaluation boundary. This laboratory uses Welford's stable recurrence and compares a frozen training normalizer with a normalizer incorrectly refitted after a distribution shift.

n+=1; delta=x-mean; mean+=delta/n; M2+=delta*(x-mean); variance=M2/(n-1).

C. Real-world meaning and limits

Hardware observation normalization must match the validated training/replay pipeline, including clipping and missing-data treatment.

D. Architecture and data flow

Configuration (test_shift) -> numerical model -> training_mean, training_variance, frozen_test_mean -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

test_shift is the mean offset of the held-out synthetic observation distribution in raw channel units.

Parameter	Nominal value	Interpretation
test_shift	1	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/55_running_statistics.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only test_shift changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/55_running_statistics.py --case 0

.venv-book/bin/python expanded_lab/modules/55_running_statistics.py --case 3

.venv-book/bin/python expanded_lab/run.py running_statistics --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/55_running_statistics.py

```
"""Streaming normalization, numerical stability and evaluation leakage. Numerical teaching
    experiment only."""

import math
import json
import argparse
import numpy as np


def welford(samples):
    n = 0
    mean = 0.0
    M2 = 0.0
    for value in samples:
        n += 1
        delta = value - mean
        mean += delta / n
        M2 += delta * (value - mean)
    return mean, M2 / (n - 1)


def experiment(p):
    rng = np.random.default_rng(71)
    training = rng.normal(0, 1, 5000)
    testing = rng.normal(p["test_shift"], 1, 1000)
    mean, var = welford(training)
    frozen = (testing - mean) / np.sqrt(var + 1e-8)
    leaked_mean, leaked_var = welford(np.concatenate([training, testing]))
    leaked = (testing - leaked_mean) / np.sqrt(leaked_var + 1e-8)
    assert np.isclose(mean, training.mean())
    assert np.isclose(var, np.var(training, ddof=1))
    metrics = dict(
        training_mean=mean,
        training_variance=var,
        frozen_test_mean=np.mean(frozen),
        leaked_test_mean=np.mean(leaked),
        mean_normalized_disagreement=np.mean(abs(frozen - leaked)),
    )
    derivation = [
        "Estimate training statistics with 5,000 seeded samples using Welford updates.",
        f'The held-out distribution has mean shift {p["test_shift"]:g}.',
        f"Frozen normalizer produces test mean {np.mean(frozen):.6f}.",
        f"Updating statistics with evaluation data changes the test mean to
    {np.mean(leaked):.6f}.",
        f"Mean per-sample normalized disagreement is {np.mean(abs(frozen-leaked)):.6f}.",
    ]
    notes = [
        "A shifted test mean can be useful evidence of distribution shift; normalizing it
    away may hide the problem.",
        "Explicit online adaptation can be legitimate, but it is a different deployed
    algorithm requiring separate validation.",
    ]
    return metrics, derivation, notes
```

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"test_shift": 1.0}
    parameters["test_shift"] = [0.0, 0.2, 0.5, 1.0, 2.0, 4.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Verify the streaming calculation against a batch reference, then freeze it. Separate numerical correctness of the estimator from the experimental policy of when it is allowed to update.

I. Expected behavior and six worked cases

The controlled variable is test_shift. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

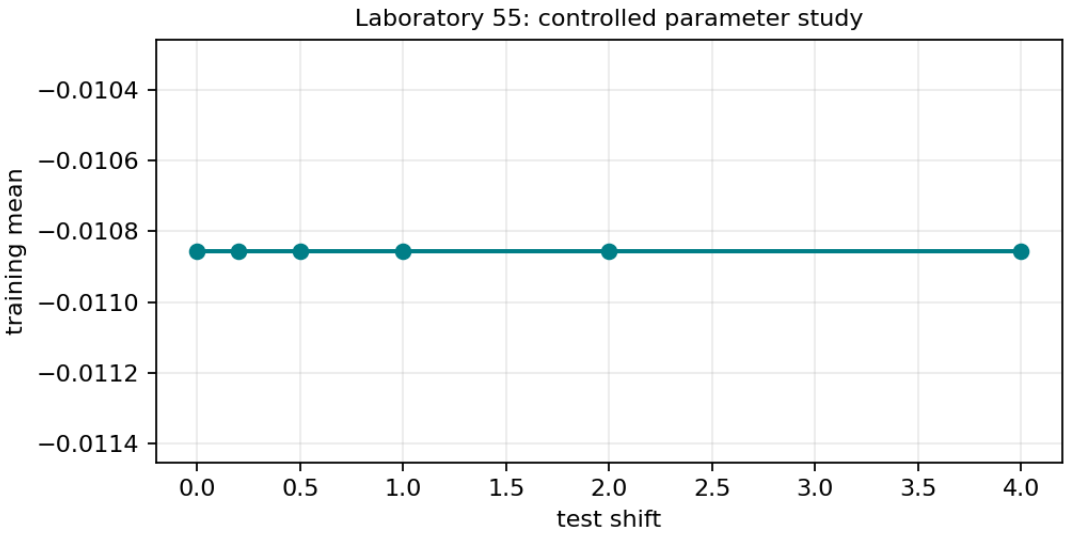


Figure L55. training mean as test_shift changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	test_shift	training mean
0	0	-0.01085555
1	0.2	-0.01085555
2	0.5	-0.01085555
3	1	-0.01085555
4	2	-0.01085555
5	4	-0.01085555

Worked case 0: test_shift = 0

This case keeps the nominal configuration except for test_shift=0. The full input is {"test_shift": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
training mean	-0.01085555
training variance	0.94154387
frozen test mean	0.01337223
leaked test mean	0.01108065
mean normalized disagreement	0.00501517

1. Estimate training statistics with 5,000 seeded samples using Welford updates.
2. The held-out distribution has mean shift 0.
3. Frozen normalizer produces test mean 0.013372.
4. Updating statistics with evaluation data changes the test mean to 0.011081.
5. Mean per-sample normalized disagreement is 0.005015.

Worked case 1: test_shift = 0.2

This case keeps the nominal configuration except for test_shift=0.2. The full input is {"test_shift": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
training mean	-0.01085555
training variance	0.94154387
frozen test mean	0.21948728
leaked test mean	0.18127759
mean normalized disagreement	0.0382097

1. Estimate training statistics with 5,000 seeded samples using Welford updates.
2. The held-out distribution has mean shift 0.2.
3. Frozen normalizer produces test mean 0.219487.
4. Updating statistics with evaluation data changes the test mean to 0.181278.
5. Mean per-sample normalized disagreement is 0.038210.

Worked case 2: test_shift = 0.5

This case keeps the nominal configuration except for test_shift=0.5. The full input is {"test_shift": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
training mean	-0.01085555
training variance	0.94154387
frozen test mean	0.52865986
leaked test mean	0.42989609

Computed quantity	Numerical result
mean normalized disagreement	0.09876378

1. Estimate training statistics with 5,000 seeded samples using Welford updates.
2. The held-out distribution has mean shift 0.5.
3. Frozen normalizer produces test mean 0.528660.
4. Updating statistics with evaluation data changes the test mean to 0.429896.
5. Mean per-sample normalized disagreement is 0.098764.

Worked case 3: test_shift = 1

This case keeps the nominal configuration except for test_shift=1. The full input is {"test_shift": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
training mean	-0.01085555
training variance	0.94154387
frozen test mean	1.0439475
leaked test mean	0.80677996
mean normalized disagreement	0.23716754

1. Estimate training statistics with 5,000 seeded samples using Welford updates.
2. The held-out distribution has mean shift 1.
3. Frozen normalizer produces test mean 1.043947.
4. Updating statistics with evaluation data changes the test mean to 0.806780.
5. Mean per-sample normalized disagreement is 0.237168.

Worked case 4: test_shift = 2

This case keeps the nominal configuration except for test_shift=2. The full input is {"test_shift": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
training mean	-0.01085555
training variance	0.94154387
frozen test mean	2.07452277
leaked test mean	1.36280645
mean normalized disagreement	0.71171632

1. Estimate training statistics with 5,000 seeded samples using Welford updates.
2. The held-out distribution has mean shift 2.
3. Frozen normalizer produces test mean 2.074523.
4. Updating statistics with evaluation data changes the test mean to 1.362806.
5. Mean per-sample normalized disagreement is 0.711716.

Worked case 5: test_shift = 4

This case keeps the nominal configuration except for test_shift=4. The full input is {"test_shift": 4.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
training mean	-0.01085555
training variance	0.94154387
frozen test mean	4.13567331
leaked test mean	1.87257638

Computed quantity	Numerical result
mean normalized disagreement	2.26309693

1. Estimate training statistics with 5,000 seeded samples using Welford updates.
2. The held-out distribution has mean shift 4.
3. Frozen normalizer produces test mean 4.135673.
4. Updating statistics with evaluation data changes the test mean to 1.872576.
5. Mean per-sample normalized disagreement is 2.263097.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

A shifted test mean can be useful evidence of distribution shift; normalizing it away may hide the problem.

Explicit online adaptation can be legitimate, but it is a different deployed algorithm requiring separate validation.

K. Failure modes and troubleshooting

Updating statistics during final evaluation; resetting normalizers when loading a policy; confusing population and sample variance conventions.

L. Practical exercises and extension tasks

1. Extend Welford to vector observations.
2. Merge statistics from two independent training workers.
3. Store normalizer parameters and schema hash with a policy checkpoint.

M. Connection to the Asimov research platform

Hardware observation normalization must match the validated training/replay pipeline, including clipping and missing-data treatment.

N. Completion record

Record your chosen case, predicted trend, measured training_mean, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 56, Domain randomization, correlations and feasible parameter sets. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 56 - Domain randomization, correlations and feasible parameter sets

A. The engineering question

Randomization should represent plausible physical variation.

B. Concepts and mathematical model

Sampling every parameter independently can produce impossible combinations, such as mass and inertia that no real assembly could have. Correlated randomization can preserve relationships while exploring uncertainty. This laboratory samples a density scale and length scale, deriving mass and rod inertia consistently, then compares it with independent inertia sampling. It illustrates why wider randomization is not automatically better.

For fixed cross-section with length scale sL and density scale srho: m=m0*srho*sL; I=m*(L0*sL)^2/3.

C. Real-world meaning and limits

Sim-to-real ranges should reflect measured actuator strength, delays, contact surfaces, calibration error and expected operating conditions.

D. Architecture and data flow

Configuration (range_fraction) -> numerical model -> mass_std_kg, consistent_inertia_std_kg_m2, mass_inertia_correlation -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

range_fraction is the symmetric fractional range of density and length scales; nominal rod mass is 2 kg and length 0.3 m.

Parameter	Nominal value	Interpretation
range_fraction	0.1	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/56_domain_randomization.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only range_fraction changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/56_domain_randomization.py --case 0

.venv-book/bin/python expanded_lab/modules/56_domain_randomization.py --case 3

.venv-book/bin/python expanded_lab/run.py domain_randomization --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/56_domain_randomization.py

```
"""Domain randomization, correlations and feasible parameter sets. Numerical teaching
    experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(73)
    n = 5000
    width = p["range_fraction"]
    density = rng.uniform(1 - width, 1 + width, n)
    length_scale = rng.uniform(1 - width, 1 + width, n)
    mass = 2 * density * length_scale
    length = 0.3 * length_scale
    consistent = mass * length**2 / 3
    independent = 0.06 * rng.uniform(1 - width, 1 + width, n)
    implied_length = np.sqrt(3 * independent / mass)
    metrics = dict(
        mass_std_kg=np.std(mass),
        consistent_inertia_std_kg_m2=np.std(consistent),
        mass_inertia_correlation=np.corrcoef(mass, consistent)[0, 1],
        independent_implied_length_rmse_m=np.sqrt(
            np.mean((implied_length - length) ** 2)
        ),
        maximum_consistent_inertia_kg_m2=np.max(consistent),
    )
    derivation = [
        f"Sample density and length scales within +/-{width*100:g} percent.",
        "Derive mass and inertia from the same physical dimensions rather than unrelated
    ranges.",
        f"Mass standard deviation is {np.std(mass):.6f} kg and inertia standard deviation is
     {np.std(consistent):.6f} kg*m^2.",
        f'Mass/inertia sample correlation is {metrics["mass_inertia_correlation"]:.6f}.',
        f'Independent inertia sampling implies a rod-length RMSE of
    {metrics["independent_implied_length_rmse_m"]:.6f} m relative to the actual sampled
    length.',
    ]
    notes = [
        "The chosen manufacturing model is still simplified; real uncertainty may arise from
     payload, assembly and calibration rather than density changes.",
        "Randomization ranges need identification evidence and held-out evaluation, not just
     an intuition that more variation is safer.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
```

```
parameters = {"range_fraction": 0.1}
parameters["range_fraction"] = [0.01, 0.025, 0.05, 0.1, 0.2, 0.3][args.case]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Generate latent physical variations first, then derive dependent parameters. Compare statistical coverage with physical consistency rather than judging samples only by marginal ranges.

I. Expected behavior and six worked cases

The controlled variable is range_fraction. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

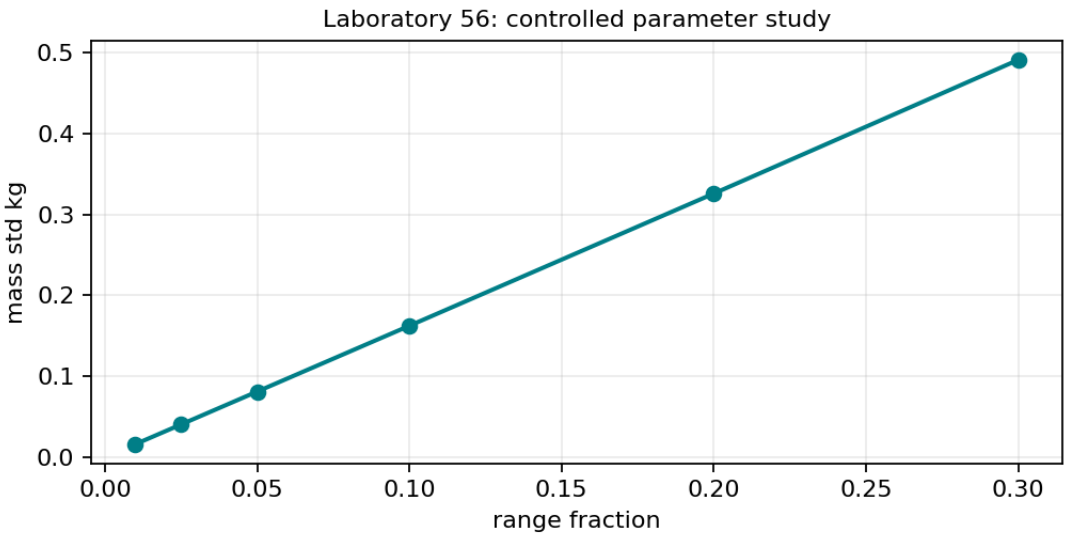


Figure L56. mass std kg as range_fraction changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	range_fraction	mass std kg
0	0.01	0.01623354
1	0.025	0.04058687
2	0.05	0.08119077
3	0.1	0.16250174
4	0.2	0.32589935
5	0.3	0.49101698

Worked case 0: range_fraction = 0.01

This case keeps the nominal configuration except for range_fraction=0.01. The full input is {"range_fraction": 0.01}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mass std kg	0.01623354
consistent inertia std kg m2	0.001094
mass inertia correlation	0.8919605
independent implied length rmse m	0.00286993
maximum consistent inertia kg m2	0.06242462

1. Sample density and length scales within +/-1 percent.
2. Derive mass and inertia from the same physical dimensions rather than unrelated ranges.
3. Mass standard deviation is 0.016234 kg and inertia standard deviation is 0.001094 kg*m^2.
4. Mass/inertia sample correlation is 0.891960.
5. Independent inertia sampling implies a rod-length RMSE of 0.002870 m relative to the actual sampled length.

Worked case 1: range_fraction = 0.025

This case keeps the nominal configuration except for range_fraction=0.025. The full input is {"range_fraction": 0.025}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mass std kg	0.04058687
consistent inertia std kg m2	0.00273593
mass inertia correlation	0.89198415
independent implied length rmse m	0.00717536
maximum consistent inertia kg m2	0.06619842

1. Sample density and length scales within +/-2.5 percent.
2. Derive mass and inertia from the same physical dimensions rather than unrelated ranges.
3. Mass standard deviation is 0.040587 kg and inertia standard deviation is 0.002736 kg*m^2.
4. Mass/inertia sample correlation is 0.891984.
5. Independent inertia sampling implies a rod-length RMSE of 0.007175 m relative to the actual sampled length.

Worked case 2: range_fraction = 0.05

This case keeps the nominal configuration except for range_fraction=0.05. The full input is {"range_fraction": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mass std kg	0.08119077
consistent inertia std kg m2	0.00547789
mass inertia correlation	0.89204348

Computed quantity	Numerical result
independent implied length rmse m	0.01435604
maximum consistent inertia kg m2	0.07286513

1. Sample density and length scales within +/-5 percent.
2. Derive mass and inertia from the same physical dimensions rather than unrelated ranges.
3. Mass standard deviation is 0.081191 kg and inertia standard deviation is 0.005478 kg*m^2.
4. Mass/inertia sample correlation is 0.892043.
5. Independent inertia sampling implies a rod-length RMSE of 0.014356 m relative to the actual sampled length.

Worked case 3: range_fraction = 0.1

This case keeps the nominal configuration except for range_fraction=0.1. The full input is {"range_fraction": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mass std kg	0.16250174
consistent inertia std kg m2	0.01100211
mass inertia correlation	0.89223425
independent implied length rmse m	0.02875988
maximum consistent inertia kg m2	0.08769602

1. Sample density and length scales within +/-10 percent.
2. Derive mass and inertia from the same physical dimensions rather than unrelated ranges.
3. Mass standard deviation is 0.162502 kg and inertia standard deviation is 0.011002 kg*m^2.
4. Mass/inertia sample correlation is 0.892234.

5. Independent inertia sampling implies a rod-length RMSE of 0.028760 m relative to the actual sampled length.

Worked case 4: range_fraction = 0.2

This case keeps the nominal configuration except for range_fraction=0.2. The full input is {"range_fraction": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mass std kg	0.32589935
consistent inertia std kg m2	0.02236775
mass inertia correlation	0.89287222
independent implied length rmse m	0.05793277
maximum consistent inertia kg m2	0.12402677

1. Sample density and length scales within +/-20 percent.
2. Derive mass and inertia from the same physical dimensions rather than unrelated ranges.
3. Mass standard deviation is 0.325899 kg and inertia standard deviation is 0.022368 kg*m^2.
4. Mass/inertia sample correlation is 0.892872.
5. Independent inertia sampling implies a rod-length RMSE of 0.057933 m relative to the actual sampled length.

Worked case 5: range_fraction = 0.3

This case keeps the nominal configuration except for range_fraction=0.3. The full input is {"range_fraction": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
mass std kg	0.49101698
consistent inertia std kg m2	0.03445635

Computed quantity	Numerical result
mass inertia correlation	0.89376348
independent implied length rmse m	0.08801483
maximum consistent inertia kg m2	0.17062402

1. Sample density and length scales within +/-30 percent.
2. Derive mass and inertia from the same physical dimensions rather than unrelated ranges.
3. Mass standard deviation is 0.491017 kg and inertia standard deviation is 0.034456 kg*m^2.
4. Mass/inertia sample correlation is 0.893763.
5. Independent inertia sampling implies a rod-length RMSE of 0.088015 m relative to the actual sampled length.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The chosen manufacturing model is still simplified; real uncertainty may arise from payload, assembly and calibration rather than density changes.

Randomization ranges need identification evidence and held-out evaluation, not just an intuition that more variation is safer.

K. Failure modes and troubleshooting

Sampling nonphysical inertias; randomizing unobservable ground truth instead of measurements; choosing ranges after looking at final test performance.

L. Practical exercises and extension tasks

1. Add a discrete payload mode instead of scaling every link.
2. Correlate friction with temperature in a measured way.
3. Separate training randomization and held-out stress conditions.

M. Connection to the Asimov research platform

Sim-to-real ranges should reflect measured actuator strength, delays, contact surfaces, calibration error and expected operating conditions.

N. Completion record

Record your chosen case, predicted trend, measured mass_std_kg, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 57, Fall rates, confidence intervals and finite evidence. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 57 - Fall rates, confidence intervals and finite evidence

A. The engineering question

A controller that does not fall in a small number of trials has not demonstrated zero failure probability.

B. Concepts and mathematical model

Report the number of trials, condition distribution, training seeds and observed failures. A confidence interval quantifies sampling uncertainty under assumptions about independent comparable trials; it does not cover untested terrain or correlated hardware faults. This laboratory calculates a Wilson binomial interval and an exact one-sided upper bound for the special zero-failure case.

Wilson center=(p_hat+z^2/(2n))/(1+z^2/n); zero-failure one-sided upper bound=1-alpha^(1/n).

C. Real-world meaning and limits

Physical risk assessment requires broader evidence than binomial simulation fall counts, including consequences and independent protection.

D. Architecture and data flow

Configuration (trials, observed_fraction) -> numerical model -> observed_failure_fraction, wilson_lower, wilson_upper -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

trials is the independent comparable evaluation count; observed_fraction is converted to an integer failure count and reported exactly.

Parameter	Nominal value	Interpretation
trials	100	Defined by the explicit chapter model and parameter contract
observed_fraction	0.02	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/57_evaluation_intervals.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only trials changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/57_evaluation_intervals.py --case 0

.venv-book/bin/python expanded_lab/modules/57_evaluation_intervals.py --case 3
```

```
.venv-book/bin/python expanded_lab/run.py evaluation_intervals --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/57_evaluation_intervals.py

```
"""Fall rates, confidence intervals and finite evidence. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    n = int(p["trials"])
    failures = int(round(n * p["observed_fraction"]))
    phat = failures / n
    z = 1.959963984540054
    denom = 1 + z * z / n
    center = (phat + z * z / (2 * n)) / denom
    half = z * np.sqrt(phat * (1 - phat) / n + z * z / (4 * n * n)) / denom
    zero_upper = 1 - 0.05 ** (1 / n)
    metrics = dict(
        observed_failure_fraction=phat,
        wilson_lower=max(0, center - half),
        wilson_upper=min(1, center + half),
        zero_failure_upper_if_applicable=zero_upper,
        observed_failures=failures,
    )
    derivation = [
        f"Use {n} trials and {failures} observed failures.",
        f"Observed fraction is {phat:.8f}.",
        f"The 95% Wilson interval is [{max(0,center-half):.8f},{min(1,center+half):.8f}].",
        f"If all {n} trials had zero failures, a one-sided 95% upper bound would be
    {zero_upper:.8f}.",
        "The zero-failure formula solves (1-p)^n=0.05 under the binomial independent-trial
    assumption.",
    ]
    notes = [
        "Changing the terrain, disturbance distribution or controller release changes the
    population to which these intervals apply.",
        "Repeated trials with shared initialization or correlated faults may provide less
    information than the independent model assumes.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"trials": 100, "observed_fraction": 0.02}
    parameters["trials"] = [10, 20, 50, 100, 500, 1000][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
```

```
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Retain integer failures and trials before computing a fraction. The interval width and zero-failure bound make the limits of a small demonstration explicit.

I. Expected behavior and six worked cases

The controlled variable is trials. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

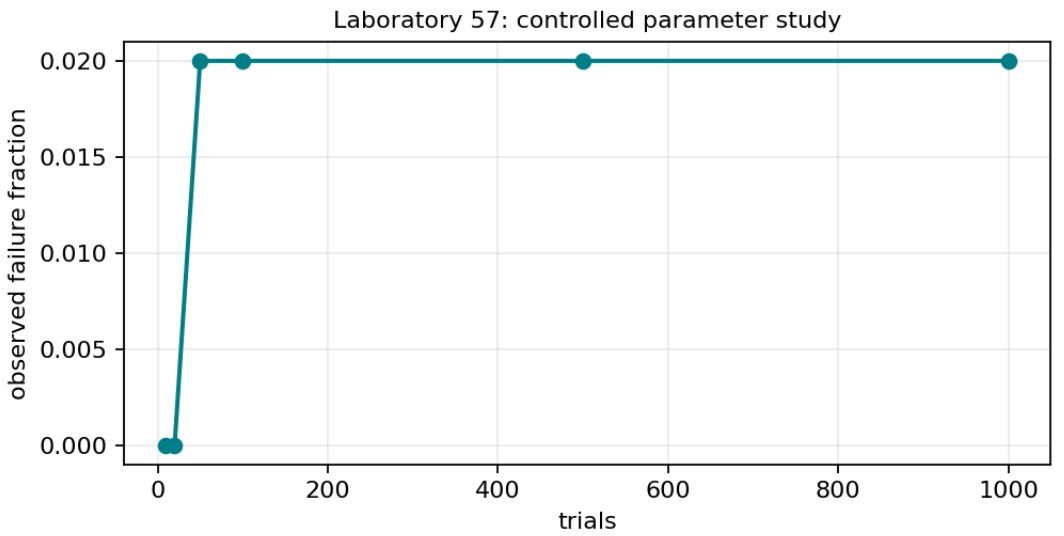


Figure L57. observed failure fraction as trials changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	trials	observed failure fraction
0	10	0
1	20	0
2	50	0.02

Case	trials	observed failure fraction
3	100	0.02
4	500	0.02
5	1000	0.02

Worked case 0: trials = 10

This case keeps the nominal configuration except for trials=10. The full input is {"observed_fraction": 0.02, "trials": 10}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
observed failure fraction	0
wilson lower	0
wilson upper	0.2775328
zero failure upper if applicable	0.25886555
observed failures	0

1. Use 10 trials and 0 observed failures.
2. Observed fraction is 0.00000000.
3. The 95% Wilson interval is [0.00000000,0.27753280].
4. If all 10 trials had zero failures, a one-sided 95% upper bound would be 0.25886555.
5. The zero-failure formula solves (1-p)^n=0.05 under the binomial independent-trial assumption.

Worked case 1: trials = 20

This case keeps the nominal configuration except for trials=20. The full input is {"observed_fraction": 0.02, "trials": 20}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
observed failure fraction	0
wilson lower	0
wilson upper	0.16112516
zero failure upper if applicable	0.13910834
observed failures	0

1. Use 20 trials and 0 observed failures.
2. Observed fraction is 0.00000000.
3. The 95% Wilson interval is [0.00000000,0.16112516].
4. If all 20 trials had zero failures, a one-sided 95% upper bound would be 0.13910834.
5. The zero-failure formula solves (1-p)^n=0.05 under the binomial independent-trial assumption.

Worked case 2: trials = 50

This case keeps the nominal configuration except for trials=50. The full input is {"observed_fraction": 0.02, "trials": 50}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
observed failure fraction	0.02
wilson lower	0.00353926
wilson upper	0.10495444

Computed quantity	Numerical result
zero failure upper if applicable	0.05815508
observed failures	1

1. Use 50 trials and 1 observed failures.
2. Observed fraction is 0.02000000.
3. The 95% Wilson interval is [0.00353926,0.10495444].
4. If all 50 trials had zero failures, a one-sided 95% upper bound would be 0.05815508.
5. The zero-failure formula solves (1-p)^n=0.05 under the binomial independent-trial assumption.

Worked case 3: trials = 100

This case keeps the nominal configuration except for trials=100. The full input is {"observed_fraction": 0.02, "trials": 100}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
observed failure fraction	0.02
wilson lower	0.00550197
wilson upper	0.07001179
zero failure upper if applicable	0.02951305
observed failures	2

1. Use 100 trials and 2 observed failures.
2. Observed fraction is 0.02000000.
3. The 95% Wilson interval is [0.00550197,0.07001179].
4. If all 100 trials had zero failures, a one-sided 95% upper bound would be 0.02951305.

5. The zero-failure formula solves (1-p)^n=0.05 under the binomial independent-trial assumption.

Worked case 4: trials = 500

This case keeps the nominal configuration except for trials=500. The full input is {"observed_fraction": 0.02, "trials": 500}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
observed failure fraction	0.02
wilson lower	0.01089918
wilson upper	0.03642018
zero failure upper if applicable	0.00597355
observed failures	10

1. Use 500 trials and 10 observed failures.
2. Observed fraction is 0.02000000.
3. The 95% Wilson interval is [0.01089918,0.03642018].
4. If all 500 trials had zero failures, a one-sided 95% upper bound would be 0.00597355.
5. The zero-failure formula solves (1-p)^n=0.05 under the binomial independent-trial assumption.

Worked case 5: trials = 1000

This case keeps the nominal configuration except for trials=1000. The full input is {"observed_fraction": 0.02, "trials": 1000}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
observed failure fraction	0.02

Computed quantity	Numerical result
wilson lower	0.01298368
wilson upper	0.03069001
zero failure upper if applicable	0.00299125
observed failures	20

1. Use 1000 trials and 20 observed failures.
2. Observed fraction is 0.02000000.
3. The 95% Wilson interval is [0.01298368,0.03069001].
4. If all 1000 trials had zero failures, a one-sided 95% upper bound would be 0.00299125.
5. The zero-failure formula solves (1-p)^n=0.05 under the binomial independent-trial assumption.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Changing the terrain, disturbance distribution or controller release changes the population to which these intervals apply.

Repeated trials with shared initialization or correlated faults may provide less information than the independent model assumes.

K. Failure modes and troubleshooting

Reporting only percentage without denominator; combining unlike test conditions into one reassuring score; treating simulated trials as physical reliability evidence.

L. Practical exercises and extension tasks

1. Calculate trials required to bound zero-failure probability below a chosen level.
2. Report intervals separately by terrain and training seed.

3. Model correlated failures and compare effective evidence.

M. Connection to the Asimov research platform

Physical risk assessment requires broader evidence than binomial simulation fall counts, including consequences and independent protection.

N. Completion record

Record your chosen case, predicted trend, measured observed_failure_fraction, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 58, Gait phase metrics, contact slip and energy accounting. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 58 - Gait phase metrics, contact slip and energy accounting

A. The engineering question

A walking evaluation needs more than distance traveled.

B. Concepts and mathematical model

A falling robot can move far, a dragging foot can track velocity, and a low net-energy number can hide large power peaks. Contact-conditioned slip should exclude swing motion. This laboratory synthesizes a walking-like trace with known command, stance masks and controlled slip, then computes tracking, path distance, positive mechanical work and contact slip. It demonstrates metric definitions without claiming a trained gait.

distance=sum(||delta xy||); tracking_RMSE=sqrt(mean((v-v_cmd)^2)); slip_RMS uses stance samples only; positive work=integral(max(P,0) dt).

C. Real-world meaning and limits

Use measured/estimated world-frame foot motion and contact load confidence when evaluating a real gait.

D. Architecture and data flow

Configuration (stance_slip) -> numerical model -> distance_m, velocity_rmse_m_s, stance_slip_rms_m_s -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

stance_slip is synthetic loaded-foot world speed in m/s; stance mask is known truth for the metric test.

Parameter	Nominal value	Interpretation
stance_slip	0.01	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/58_gait_evaluation.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only stance_slip changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project
```

```
.venv-book/bin/python expanded_lab/modules/58_gait_evaluation.py --case 0
```

```
.venv-book/bin/python expanded_lab/modules/58_gait_evaluation.py --case 3

.venv-book/bin/python expanded_lab/run.py gait_evaluation --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/58_gait_evaluation.py

```
"""Gait phase metrics, contact slip and energy accounting. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    t = np.arange(0, 10, 0.01)
    dt = 0.01
    command = 0.2
    velocity = command + 0.02 * np.sin(2 * np.pi * 0.7 * t)
    position = np.cumsum(velocity) * dt
    phase = (t % 1.0) < 0.6
    foot_speed = np.where(phase, p["stance_slip"], 0.3 * np.sin(2 * np.pi * t))
    power = 12 + 8 * np.sin(2 * np.pi * t)
    distance = float(np.sum(abs(np.diff(position))))
    slip = np.sqrt(np.mean(foot_speed[phase] ** 2))
    metrics = dict(
        distance_m=distance,
        velocity_rmse_m_s=np.sqrt(np.mean((velocity - command) ** 2)),
        stance_slip_rms_m_s=slip,
        all_phase_foot_speed_rms_m_s=np.sqrt(np.mean(foot_speed**2)),
        positive_mechanical_work_J=np.trapezoid(np.maximum(power, 0), t),
        stance_fraction=np.mean(phase),
    )
    derivation = [
        "Create a ten-second synthetic trajectory commanded at 0.2 m/s.",
        f'Loaded-stance foot speed is {p["stance_slip"]:g} m/s; swing motion is excluded
    from the slip metric.',
        f'Distance is {distance:.6f} m and velocity RMSE is
    {metrics["velocity_rmse_m_s"]:.6f} m/s.',
        f'Stance-only slip RMS is {slip:.6f} m/s while all-phase RMS is
    {metrics["all_phase_foot_speed_rms_m_s"]:.6f} m/s.',
        f'Positive mechanical work is {metrics["positive_mechanical_work_J"]:.6f} J.',
    ]
    notes = [
        "These traces are synthetic metric fixtures, not walking results from the Asimov
    model.",
        "Mechanical work is not battery energy; physical electrical measurements require
    voltage/current timing and regeneration accounting.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"stance_slip": 0.01}
    parameters["stance_slip"] = [0.0, 0.002, 0.005, 0.01, 0.03, 0.1][args.case]
```

```
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The all-phase foot-speed metric is deliberately included as a counterexample: it mainly measures swing, not unwanted slip. Contact confidence should accompany real metric reports.

I. Expected behavior and six worked cases

The controlled variable is stance_slip. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

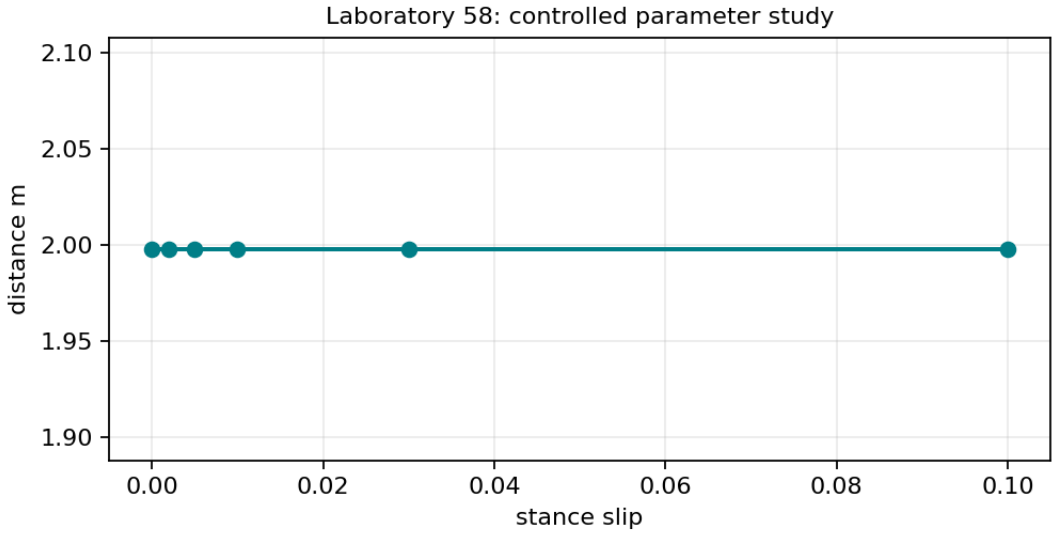


Figure L58. distance m as stance_slip changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	stance_slip	distance m
0	0	1.998

Case	stance_slip	distance m
1	0.002	1.998
2	0.005	1.998
3	0.01	1.998
4	0.03	1.998
5	0.1	1.998

Worked case 0: stance_slip = 0

This case keeps the nominal configuration except for stance_slip=0. The full input is {"stance_slip": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
distance m	1.998
velocity rmse m s	0.01414214
stance slip rms m s	0
all phase foot speed rms m s	0.1466101
positive mechanical work J	119.88251162
stance fraction	0.602

1. Create a ten-second synthetic trajectory commanded at 0.2 m/s.
2. Loaded-stance foot speed is 0 m/s; swing motion is excluded from the slip metric.
3. Distance is 1.998000 m and velocity RMSE is 0.014142 m/s.
4. Stance-only slip RMS is 0.000000 m/s while all-phase RMS is 0.146610 m/s.
5. Positive mechanical work is 119.882512 J.

Worked case 1: stance_slip = 0.002

This case keeps the nominal configuration except for stance_slip=0.002. The full input is {"stance_slip": 0.002}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
distance m	1.998
velocity rmse m s	0.01414214
stance slip rms m s	0.002
all phase foot speed rms m s	0.14661831
positive mechanical work J	119.88251162
stance fraction	0.602

1. Create a ten-second synthetic trajectory commanded at 0.2 m/s.
2. Loaded-stance foot speed is 0.002 m/s; swing motion is excluded from the slip metric.
3. Distance is 1.998000 m and velocity RMSE is 0.014142 m/s.
4. Stance-only slip RMS is 0.002000 m/s while all-phase RMS is 0.146618 m/s.
5. Positive mechanical work is 119.882512 J.

Worked case 2: stance_slip = 0.005

This case keeps the nominal configuration except for stance_slip=0.005. The full input is {"stance_slip": 0.005}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
distance m	1.998
velocity rmse m s	0.01414214
stance slip rms m s	0.005

Computed quantity	Numerical result
all phase foot speed rms m s	0.14666141
positive mechanical work J	119.88251162
stance fraction	0.602

1. Create a ten-second synthetic trajectory commanded at 0.2 m/s.
2. Loaded-stance foot speed is 0.005 m/s; swing motion is excluded from the slip metric.
3. Distance is 1.998000 m and velocity RMSE is 0.014142 m/s.
4. Stance-only slip RMS is 0.005000 m/s while all-phase RMS is 0.146661 m/s.
5. Positive mechanical work is 119.882512 J.

Worked case 3: stance_slip = 0.01

This case keeps the nominal configuration except for stance_slip=0.01. The full input is {"stance_slip": 0.01}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
distance m	1.998
velocity rmse m s	0.01414214
stance slip rms m s	0.01
all phase foot speed rms m s	0.14681526
positive mechanical work J	119.88251162
stance fraction	0.602

1. Create a ten-second synthetic trajectory commanded at 0.2 m/s.
2. Loaded-stance foot speed is 0.01 m/s; swing motion is excluded from the slip metric.
3. Distance is 1.998000 m and velocity RMSE is 0.014142 m/s.

4. Stance-only slip RMS is 0.010000 m/s while all-phase RMS is 0.146815 m/s.
5. Positive mechanical work is 119.882512 J.

Worked case 4: stance_slip = 0.03

This case keeps the nominal configuration except for stance_slip=0.03. The full input is {"stance_slip": 0.03}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
distance m	1.998
velocity rmse m s	0.01414214
stance slip rms m s	0.03
all phase foot speed rms m s	0.14844635
positive mechanical work J	119.88251162
stance fraction	0.602

1. Create a ten-second synthetic trajectory commanded at 0.2 m/s.
2. Loaded-stance foot speed is 0.03 m/s; swing motion is excluded from the slip metric.
3. Distance is 1.998000 m and velocity RMSE is 0.014142 m/s.
4. Stance-only slip RMS is 0.030000 m/s while all-phase RMS is 0.148446 m/s.
5. Positive mechanical work is 119.882512 J.

Worked case 5: stance_slip = 0.1

This case keeps the nominal configuration except for stance_slip=0.1. The full input is {"stance_slip": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
distance m	1.998

Computed quantity	Numerical result
velocity rmse m s	0.01414214
stance slip rms m s	0.1
all phase foot speed rms m s	0.16587501
positive mechanical work J	119.88251162
stance fraction	0.602

1. Create a ten-second synthetic trajectory commanded at 0.2 m/s.
2. Loaded-stance foot speed is 0.1 m/s; swing motion is excluded from the slip metric.
3. Distance is 1.998000 m and velocity RMSE is 0.014142 m/s.
4. Stance-only slip RMS is 0.100000 m/s while all-phase RMS is 0.165875 m/s.
5. Positive mechanical work is 119.882512 J.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

These traces are synthetic metric fixtures, not walking results from the Asimov model.

Mechanical work is not battery energy; physical electrical measurements require voltage/current timing and regeneration accounting.

K. Failure modes and troubleshooting

Including swing speed in foot slip; integrating commanded velocity instead of measured motion; omitting falls from a distance score.

L. Practical exercises and extension tasks

1. Add a fall event and report survival time separately from distance.
2. Include commanded yaw-rate tracking.

3. Use uncertain contact labels and quantify metric sensitivity.

M. Connection to the Asimov research platform

Use measured/estimated world-frame foot motion and contact load confidence when evaluating a real gait.

N. Completion record

Record your chosen case, predicted trend, measured distance_m, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 59, Color segmentation, false positives and image-space evidence. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 59 - Color segmentation, false positives and image-space evidence

A. The engineering question

A controlled color target is useful for testing the perception-to-behavior path before adding a learned detector.

B. Concepts and mathematical model

Thresholding is sensitive to illumination, distractors, sensor noise and object size. Pixel area is not physical area unless camera geometry and depth are known. This experiment creates a labeled synthetic RGB-like image, adds noise and measures precision, recall and centroid error for a simple green-excess threshold. It provides a ground-truth test fixture, not a benchmark of real camera performance.

mask=(G-R>threshold) and (G-B>threshold); precision=TP/(TP+FP); recall=TP/(TP+FN).

C. Real-world meaning and limits

Physical perception must carry timestamps, calibration identity, uncertainty and an explicit no-detection result into the skill supervisor.

D. Architecture and data flow

Configuration (noise_std) -> numerical model -> precision, recall, centroid_error_px -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

noise_std is synthetic per-channel intensity noise on a 0-255 scale; images are teaching arrays, not captured hardware frames.

Parameter	Nominal value	Interpretation
noise_std	10	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/59_image_segmentation.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only noise_std changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project
```

```
.venv-book/bin/python expanded_lab/modules/59_image_segmentation.py --case 0
```

```
.venv-book/bin/python expanded_lab/modules/59_image_segmentation.py --case 3

.venv-book/bin/python expanded_lab/run.py image_segmentation --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/59_image_segmentation.py

```
"""Color segmentation, false positives and image-space evidence. Numerical teaching
    experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(79)
    h, w = 120, 160
    image = np.ones((h, w, 3)) * 40
    truth = np.zeros((h, w), bool)
    truth[40:70, 90:120] = True
    image[truth] = [30, 180, 30]
    image[10:30, 10:30] = [40, 100, 40]
    noisy = np.clip(image + rng.normal(0, p["noise_std"], image.shape), 0, 255)
    mask = (noisy[:, :, 1] - noisy[:, :, 0] > 70) & (
        noisy[:, :, 1] - noisy[:, :, 2] > 70
    )
    tp = np.count_nonzero(mask & truth)
    fp = np.count_nonzero(mask & ~truth)
    fn = np.count_nonzero(~mask & truth)
    ys, xs = np.where(mask)
    centroid_error = (
        np.hypot(xs.mean() - 104.5, ys.mean() - 54.5) if len(xs) else 1000.0
    )
    metrics = dict(
        precision=tp / max(1, tp + fp),
        recall=tp / max(1, tp + fn),
        centroid_error_px=centroid_error,
        predicted_area_px=mask.sum(),
        false_positive_pixels=fp,
    )
    derivation = [
        "Place a 30x30 pixel bright-green target and a dimmer green distractor in a 160x120
    image.",
        f'Add pixel-channel noise sigma={p["noise_std"]:g}.',
        "Threshold green excess over red and blue by 70 intensity units.",
        f'Count TP={tp}, FP={fp}, FN={fn}; precision={metrics["precision"]:.6f},
    recall={metrics["recall"]:.6f}.',
        f"The predicted-mask centroid differs from the target center by {centroid_error:.6f}
     pixels.",
    ]
    notes = [
        "All foreground pixels are pooled, so distractors can shift the centroid;
    connected-component selection is an important extension.",
        "No depth or metric reach target is inferred from this mask.",
    ]
    return metrics, derivation, notes

if __name__ == "__main__":
```

```
parser = argparse.ArgumentParser()
parser.add_argument("--case", type=int, default=0, choices=range(6))
args = parser.parse_args()
parameters = {"noise_std": 10.0}
parameters["noise_std"] = [0.0, 5.0, 10.0, 20.0, 30.0, 50.0][args.case]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Ground-truth masks make precision and recall calculable. Evaluate centroid localization separately from pixel classification because a small false-positive region far away can shift the target estimate.

I. Expected behavior and six worked cases

The controlled variable is noise_std. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

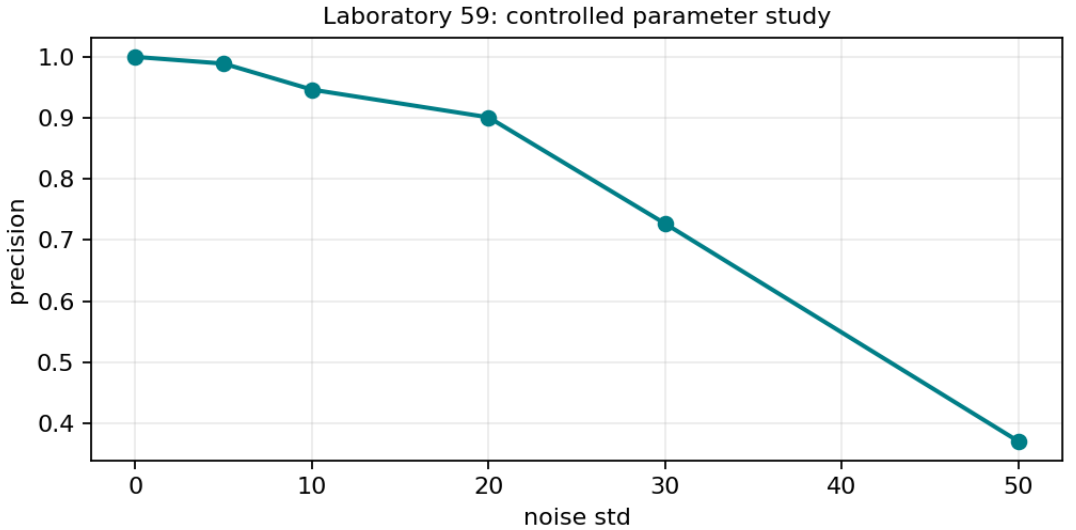


Figure L59. precision as noise_std changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	noise_std	precision
0	0	1
1	5	0.98901099
2	10	0.94637224
3	20	0.9010101
4	30	0.72687986
5	50	0.37038988

Worked case 0: noise_std = 0

This case keeps the nominal configuration except for noise_std=0. The full input is {"noise_std": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
precision	1
recall	1
centroid error px	0
predicted area px	900
false positive pixels	0

1. Place a 30x30 pixel bright-green target and a dimmer green distractor in a 160x120 image.
2. Add pixel-channel noise sigma=0.
3. Threshold green excess over red and blue by 70 intensity units.
4. Count TP=900, FP=0, FN=0; precision=1.000000, recall=1.000000.
5. The predicted-mask centroid differs from the target center by 0.000000 pixels.

Worked case 1: noise_std = 5

This case keeps the nominal configuration except for noise_std=5. The full input is {"noise_std": 5.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
precision	0.98901099
recall	1
centroid error px	1.01671693
predicted area px	910
false positive pixels	10

1. Place a 30x30 pixel bright-green target and a dimmer green distractor in a 160x120 image.
2. Add pixel-channel noise sigma=5.
3. Threshold green excess over red and blue by 70 intensity units.
4. Count TP=900, FP=10, FN=0; precision=0.989011, recall=1.000000.
5. The predicted-mask centroid differs from the target center by 1.016717 pixels.

Worked case 2: noise_std = 10

This case keeps the nominal configuration except for noise_std=10. The full input is {"noise_std": 10.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
precision	0.94637224
recall	1
centroid error px	4.96058189
predicted area px	951

Computed quantity	Numerical result
false positive pixels	51

1. Place a 30x30 pixel bright-green target and a dimmer green distractor in a 160x120 image.
2. Add pixel-channel noise sigma=10.
3. Threshold green excess over red and blue by 70 intensity units.
4. Count TP=900, FP=51, FN=0; precision=0.946372, recall=1.000000.
5. The predicted-mask centroid differs from the target center by 4.960582 pixels.

Worked case 3: noise_std = 20

This case keeps the nominal configuration except for noise_std=20. The full input is {"noise_std": 20.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
precision	0.9010101
recall	0.99111111
centroid error px	8.2329377
predicted area px	990
false positive pixels	98

1. Place a 30x30 pixel bright-green target and a dimmer green distractor in a 160x120 image.
2. Add pixel-channel noise sigma=20.
3. Threshold green excess over red and blue by 70 intensity units.
4. Count TP=892, FP=98, FN=8; precision=0.901010, recall=0.991111.
5. The predicted-mask centroid differs from the target center by 8.232938 pixels.

Worked case 4: noise_std = 30

This case keeps the nominal configuration except for noise_std=30. The full input is {"noise_std": 30.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
precision	0.72687986
recall	0.93444444
centroid error px	12.78969139
predicted area px	1157
false positive pixels	316

1. Place a 30x30 pixel bright-green target and a dimmer green distractor in a 160x120 image.
2. Add pixel-channel noise sigma=30.
3. Threshold green excess over red and blue by 70 intensity units.
4. Count TP=841, FP=316, FN=59; precision=0.726880, recall=0.934444.
5. The predicted-mask centroid differs from the target center by 12.789691 pixels.

Worked case 5: noise_std = 50

This case keeps the nominal configuration except for noise_std=50. The full input is {"noise_std": 50.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
precision	0.37038988
recall	0.78111111
centroid error px	19.47774569
predicted area px	1898

Computed quantity	Numerical result
false positive pixels	1195

1. Place a 30x30 pixel bright-green target and a dimmer green distractor in a 160x120 image.
2. Add pixel-channel noise sigma=50.
3. Threshold green excess over red and blue by 70 intensity units.
4. Count TP=703, FP=1195, FN=197; precision=0.370390, recall=0.781111.
5. The predicted-mask centroid differs from the target center by 19.477746 pixels.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

All foreground pixels are pooled, so distractors can shift the centroid; connected-component selection is an important extension.

No depth or metric reach target is inferred from this mask.

K. Failure modes and troubleshooting

Calling color classification object recognition; ignoring distractors; converting pixel coordinates directly into joint targets without calibration.

L. Practical exercises and extension tasks

1. Add connected-component filtering and compare centroid error.
2. Vary illumination independently of sensor noise.
3. Build a real-image held-out dataset under declared lighting.

M. Connection to the Asimov research platform

Physical perception must carry timestamps, calibration identity, uncertainty and an explicit no-detection result into the skill supervisor.

N. Completion record

Record your chosen case, predicted trend, measured precision, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 60, Allowlisted skills, bounded parameters and behavior outcomes. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 60 - Allowlisted skills, bounded parameters and behavior outcomes

A. The engineering question

High-level instruction following should select a validated capability rather than generate arbitrary low-level actuation.

B. Concepts and mathematical model

A skill request needs a fixed name, a bounded parameter schema, preconditions and a timeout. Invalid requests should produce a clear rejection without partially applying motion. This experiment defines a small supported-gesture library and evaluates malformed, stale and over-limit planner requests. It isolates the task-selection boundary from both language-model implementation and physical transport.

approved_skill = allowlist[name] and parameters_in_bounds and readiness and fresh_request; execution requires feedback-based completion.

C. Real-world meaning and limits

A physical skill library must expose only commissioned capabilities and keep local limits, watchdogs and independent stop authority in control.

D. Architecture and data flow

Configuration (amplitude) -> numerical model -> approved_requests, rejected_requests, requested_amplitude_rad -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

amplitude is a requested supported-joint excursion in radians; the 0.1 rad limit is an educational schema choice.

Parameter	Nominal value	Interpretation
amplitude	0.05	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/60_skill_planner.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only amplitude changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/60_skill_planner.py --case 0

.venv-book/bin/python expanded_lab/modules/60_skill_planner.py --case 3

.venv-book/bin/python expanded_lab/run.py skill_planner --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/60_skill_planner.py

```
"""Allowlisted skills, bounded parameters and behavior outcomes. Numerical teaching
    experiment only."""

import math
import json
import argparse
import numpy as np


def approve(request, ready, now):
    if not ready:
        return False, "not_ready"
    if request.get("skill") not in ("observe", "supported_wave"):
        return False, "unknown_skill"
    if not 0 <= now - request.get("sent", -100) <= 0.5:
        return False, "stale"
    if request["skill"] == "supported_wave":
        amplitude = request.get("amplitude")
        if not isinstance(amplitude, (int, float)) or not np.isfinite(amplitude):
            return False, "type"
        if not 0 <= amplitude <= 0.1:
            return False, "amplitude"
    return True, "approved"


def experiment(p):
    amplitude = p["amplitude"]
    now = 1.0
    requests = [
        dict(skill="supported_wave", amplitude=amplitude, sent=0.9),
        dict(skill="arbitrary_motor_code", amplitude=0.01, sent=0.9),
        dict(skill="supported_wave", amplitude=0.05, sent=0.0),
        dict(skill="observe", sent=0.9),
    ]
    decisions = [approve(r, True, now) for r in requests]
    approved = sum(x[0] for x in decisions)
    not_ready = approve(requests[0], False, now)
    assert not not_ready[0]
    metrics = dict(
        approved_requests=approved,
        rejected_requests=len(requests) - approved,
        requested_amplitude_rad=amplitude,
        primary_request_approved=float(decisions[0][0]),
        inactive_request_approved=float(not_ready[0]),
    )
    derivation = [
        f"The planner requests supported_wave amplitude {amplitude:g} rad.",
        "The allowlist contains observe and supported_wave only.",
        f"Decisions for normal, unknown, stale and observation requests are {decisions}.",
        f"Approved {approved} requests and rejected {len(requests)-approved}.",
        f"The same request with readiness false is rejected as {not_ready[1]}.",
    ]
    notes = [
```

```
        "Approval is permission to start a bounded skill; it is not proof the physical task
    completed.",
        "The executor still needs cancellation, tracking checks, timeout and a reviewed stop
     response.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"amplitude": 0.05}
    parameters["amplitude"] = [-0.05, 0.0, 0.025, 0.05, 0.1, 0.2][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Separate request approval from execution and completion. The unknown-skill test proves the planner cannot introduce a new actuation capability by naming it.

I. Expected behavior and six worked cases

The controlled variable is amplitude. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

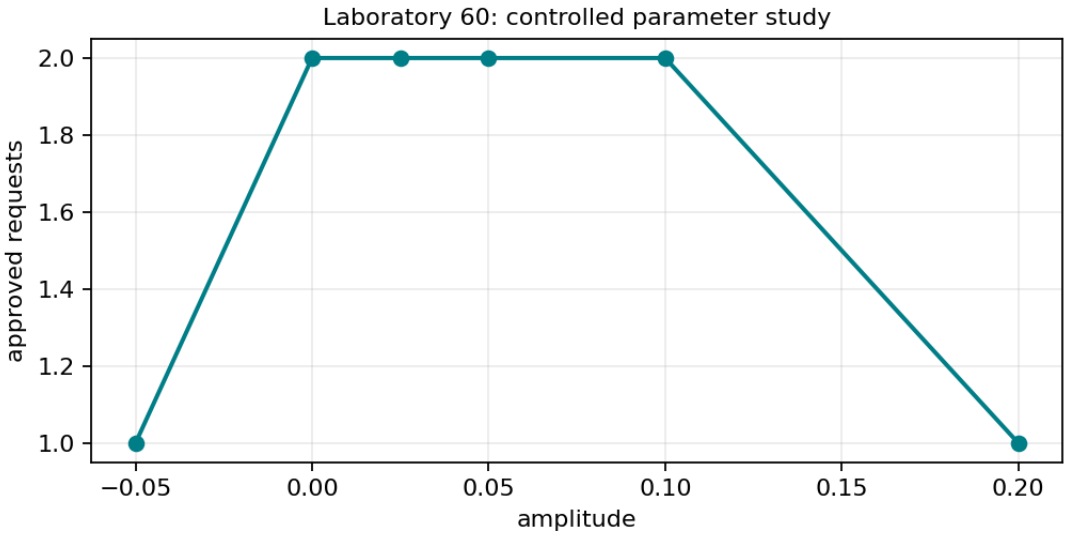


Figure L60. approved requests as amplitude changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	amplitude	approved requests
0	-0.05	1
1	0	2
2	0.025	2
3	0.05	2
4	0.1	2
5	0.2	1

Worked case 0: amplitude = -0.05

This case keeps the nominal configuration except for amplitude=-0.05. The full input is {"amplitude": -0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
approved requests	1

Computed quantity	Numerical result
rejected requests	3
requested amplitude rad	-0.05
primary request approved	0
inactive request approved	0

1. The planner requests supported_wave amplitude -0.05 rad.
2. The allowlist contains observe and supported_wave only.
3. Decisions for normal, unknown, stale and observation requests are [(False, 'amplitude'), (False, 'unknown_skill'), (False, 'stale'), (True, 'approved')].
4. Approved 1 requests and rejected 3.
5. The same request with readiness false is rejected as not_ready.

Worked case 1: amplitude = 0

This case keeps the nominal configuration except for amplitude=0. The full input is {"amplitude": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
approved requests	2
rejected requests	2
requested amplitude rad	0
primary request approved	1
inactive request approved	0

1. The planner requests supported_wave amplitude 0 rad.
2. The allowlist contains observe and supported_wave only.
3. Decisions for normal, unknown, stale and observation requests are [(True, 'approved'), (False, 'unknown_skill'), (False, 'stale'), (True, 'approved')].
4. Approved 2 requests and rejected 2.

5. The same request with readiness false is rejected as not_ready.

Worked case 2: amplitude = 0.025

This case keeps the nominal configuration except for amplitude=0.025. The full input is {"amplitude": 0.025}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
approved requests	2
rejected requests	2
requested amplitude rad	0.025
primary request approved	1
inactive request approved	0

1. The planner requests supported_wave amplitude 0.025 rad.
2. The allowlist contains observe and supported_wave only.
3. Decisions for normal, unknown, stale and observation requests are [(True, 'approved'), (False, 'unknown_skill'), (False, 'stale'), (True, 'approved')].
4. Approved 2 requests and rejected 2.
5. The same request with readiness false is rejected as not_ready.

Worked case 3: amplitude = 0.05

This case keeps the nominal configuration except for amplitude=0.05. The full input is {"amplitude": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
approved requests	2
rejected requests	2
requested amplitude rad	0.05
primary request approved	1

Computed quantity	Numerical result
inactive request approved	0

1. The planner requests supported_wave amplitude 0.05 rad.
2. The allowlist contains observe and supported_wave only.
3. Decisions for normal, unknown, stale and observation requests are [(True, 'approved'), (False, 'unknown_skill'), (False, 'stale'), (True, 'approved')].
4. Approved 2 requests and rejected 2.
5. The same request with readiness false is rejected as not_ready.

Worked case 4: amplitude = 0.1

This case keeps the nominal configuration except for amplitude=0.1. The full input is {"amplitude": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
approved requests	2
rejected requests	2
requested amplitude rad	0.1
primary request approved	1
inactive request approved	0

1. The planner requests supported_wave amplitude 0.1 rad.
2. The allowlist contains observe and supported_wave only.
3. Decisions for normal, unknown, stale and observation requests are [(True, 'approved'), (False, 'unknown_skill'), (False, 'stale'), (True, 'approved')].
4. Approved 2 requests and rejected 2.
5. The same request with readiness false is rejected as not_ready.

Worked case 5: amplitude = 0.2

This case keeps the nominal configuration except for amplitude=0.2. The full input is {"amplitude": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
approved requests	1
rejected requests	3
requested amplitude rad	0.2
primary request approved	0
inactive request approved	0

1. The planner requests supported_wave amplitude 0.2 rad.
2. The allowlist contains observe and supported_wave only.
3. Decisions for normal, unknown, stale and observation requests are [(False, 'amplitude'), (False, 'unknown_skill'), (False, 'stale'), (True, 'approved')].
4. Approved 1 requests and rejected 3.
5. The same request with readiness false is rejected as not_ready.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Approval is permission to start a bounded skill; it is not proof the physical task completed.

The executor still needs cancellation, tracking checks, timeout and a reviewed stop response.

K. Failure modes and troubleshooting

Accepting arbitrary code from a planner; confusing language confidence with readiness; considering a successful API call a completed physical task.

L. Practical exercises and extension tasks

1. Add strict unknown-field rejection and a schema version.
2. Define feedback-based success for the supported gesture.
3. Inject observation disappearance between approval and execution.

M. Connection to the Asimov research platform

A physical skill library must expose only commissioned capabilities and keep local limits, watchdogs and independent stop authority in control.

N. Completion record

Record your chosen case, predicted trend, measured approved_requests, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 61, Landed cost, VAT, contingency and scope decisions. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Part XII - Physical engineering, evidence and integration

These laboratories develop procurement arithmetic, fit variation, load screening, electrical energy reasoning, bidirectional calibration and evidence management. Their values are fictional teaching inputs unless the earlier source register explicitly identifies a verified fact. They support better questions and better experiments; they do not approve a fabricated structure, power system or complete robot. The final synthetic capstone integrates the same authority boundaries used by the practical Asimov simulation.

Laboratory 61 - Landed cost, VAT, contingency and scope decisions

A. The engineering question

A feasible project budget is a release-matched bill of materials with quantities, tax basis, shipping, fabrication, tools, protection and contingency.

B. Concepts and mathematical model

A partially priced list cannot prove affordability. This laboratory uses an explicitly fictional procurement scenario to teach reconciliation and sensitivity. It separates VAT-inclusive EU items from net-priced imports, then reserves contingency before deciding whether the stage fits the robot budget. The invented values are not supplier prices and must never be presented as a verified Asimov BOM.

landed = EU_gross + (import_net+shipping)*(1+tax_fraction) + fabrication + protection; total=landed*(1+contingency).

C. Real-world meaning and limits

The earlier verified-source procurement ledger remains authoritative about what was and was not priced; this scenario provides no new live quotes.

D. Architecture and data flow

Configuration (motor_count, motor_allowance) -> numerical model -> import_landed_EUR, subtotal_EUR, reserve_EUR -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

motor_count is the hypothetical stage count; motor_allowance is an invented net EUR budget amount, not a quotation.

Parameter	Nominal value	Interpretation
motor_count	3	Defined by the explicit chapter model and parameter contract
motor_allowance	400	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/61_landed_budget.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only motor_count changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/61_landed_budget.py --case 0

.venv-book/bin/python expanded_lab/modules/61_landed_budget.py --case 3

.venv-book/bin/python expanded_lab/run.py landed_budget --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/61_landed_budget.py

```
"""Landed cost, VAT, contingency and scope decisions. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    motors = int(p["motor_count"])
    unit = p["motor_allowance"]
    eu_gross = 350.0
    import_net = motors * unit
    shipping = 120.0
    tax = 0.19
    fabrication = 600.0
    protection = 700.0
    contingency = 0.15
    import_landed = (import_net + shipping) * (1 + tax)
    subtotal = eu_gross + import_landed + fabrication + protection
    reserve = subtotal * contingency
    total = subtotal + reserve
    metrics = dict(
        import_landed_EUR=import_landed,
        subtotal_EUR=subtotal,
        reserve_EUR=reserve,
        total_stage_EUR=total,
        budget_margin_EUR=6000 - total,
    )
    derivation = [
        f"This hypothetical stage contains {motors} motors at an internal allowance of EUR
    {unit:g} each.",
        f"Net import allowance EUR {import_net:g} plus EUR 120 shipping is taxed at an
    assumed 19%.",
        f"Import landed cost is EUR {import_landed:.2f}; EU gross, fabrication and
    protection give subtotal EUR {subtotal:.2f}.",
        f"Reserve 15% contingency: EUR {reserve:.2f}.",
        f"Total is EUR {total:.2f}; margin against EUR 6,000 is EUR {6000-total:.2f}.",
    ]
    notes = [
        "Every amount here is a teaching allowance. Actual VAT jurisdiction, customs
    treatment, supplier terms and compatibility must be checked.",
        "A negative margin means reduce scope or increase funding; essential protection is
    not the balancing item.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"motor_count": 3, "motor_allowance": 400.0}
```

```
    parameters["motor_count"] = [1, 2, 3, 4, 6, 12][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The tax basis is explicit and VAT-inclusive EU items are not taxed a second time. The reserve remains a distinct line so it cannot be mistaken for component spending.

I. Expected behavior and six worked cases

The controlled variable is motor_count. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

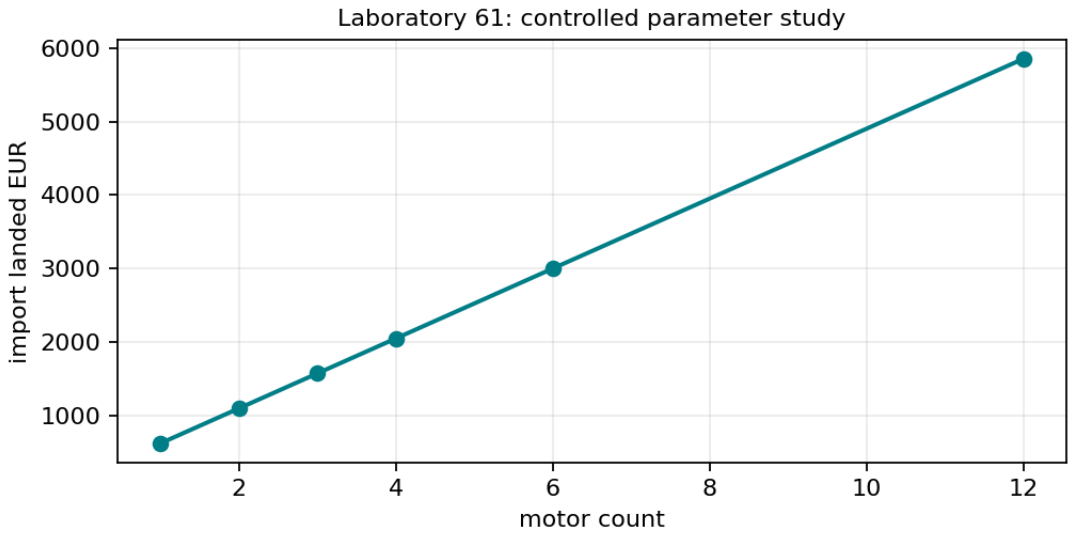


Figure L61. import landed EUR as motor_count changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	motor_count	import landed EUR
0	1	618.8

Case	motor_count	import landed EUR
1	2	1094.8
2	3	1570.8
3	4	2046.8
4	6	2998.8
5	12	5854.8

Worked case 0: motor_count = 1

This case keeps the nominal configuration except for motor_count=1. The full input is {"motor_allowance": 400.0, "motor_count": 1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
import landed EUR	618.8
subtotal EUR	2268.8
reserve EUR	340.32
total stage EUR	2609.12
budget margin EUR	3390.88

1. This hypothetical stage contains 1 motors at an internal allowance of EUR 400 each.
2. Net import allowance EUR 400 plus EUR 120 shipping is taxed at an assumed 19%.
3. Import landed cost is EUR 618.80; EU gross, fabrication and protection give subtotal EUR 2268.80.
4. Reserve 15% contingency: EUR 340.32.
5. Total is EUR 2609.12; margin against EUR 6,000 is EUR 3390.88.

Worked case 1: motor_count = 2

This case keeps the nominal configuration except for motor_count=2. The full input is {"motor_allowance": 400.0, "motor_count": 2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
import landed EUR	1094.8
subtotal EUR	2744.8
reserve EUR	411.72
total stage EUR	3156.52
budget margin EUR	2843.48

1. This hypothetical stage contains 2 motors at an internal allowance of EUR 400 each.
2. Net import allowance EUR 800 plus EUR 120 shipping is taxed at an assumed 19%.
3. Import landed cost is EUR 1094.80; EU gross, fabrication and protection give subtotal EUR 2744.80.
4. Reserve 15% contingency: EUR 411.72.
5. Total is EUR 3156.52; margin against EUR 6,000 is EUR 2843.48.

Worked case 2: motor_count = 3

This case keeps the nominal configuration except for motor_count=3. The full input is {"motor_allowance": 400.0, "motor_count": 3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
import landed EUR	1570.8
subtotal EUR	3220.8
reserve EUR	483.12

Computed quantity	Numerical result
total stage EUR	3703.92
budget margin EUR	2296.08

1. This hypothetical stage contains 3 motors at an internal allowance of EUR 400 each.
2. Net import allowance EUR 1200 plus EUR 120 shipping is taxed at an assumed 19%.
3. Import landed cost is EUR 1570.80; EU gross, fabrication and protection give subtotal EUR 3220.80.
4. Reserve 15% contingency: EUR 483.12.
5. Total is EUR 3703.92; margin against EUR 6,000 is EUR 2296.08.

Worked case 3: motor_count = 4

This case keeps the nominal configuration except for motor_count=4. The full input is {"motor_allowance": 400.0, "motor_count": 4}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
import landed EUR	2046.8
subtotal EUR	3696.8
reserve EUR	554.52
total stage EUR	4251.32
budget margin EUR	1748.68

1. This hypothetical stage contains 4 motors at an internal allowance of EUR 400 each.
2. Net import allowance EUR 1600 plus EUR 120 shipping is taxed at an assumed 19%.
3. Import landed cost is EUR 2046.80; EU gross, fabrication and protection give subtotal EUR 3696.80.

4. Reserve 15% contingency: EUR 554.52.
5. Total is EUR 4251.32; margin against EUR 6,000 is EUR 1748.68.

Worked case 4: motor_count = 6

This case keeps the nominal configuration except for motor_count=6. The full input is {"motor_allowance": 400.0, "motor_count": 6}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
import landed EUR	2998.8
subtotal EUR	4648.8
reserve EUR	697.32
total stage EUR	5346.12
budget margin EUR	653.88

1. This hypothetical stage contains 6 motors at an internal allowance of EUR 400 each.
2. Net import allowance EUR 2400 plus EUR 120 shipping is taxed at an assumed 19%.
3. Import landed cost is EUR 2998.80; EU gross, fabrication and protection give subtotal EUR 4648.80.
4. Reserve 15% contingency: EUR 697.32.
5. Total is EUR 5346.12; margin against EUR 6,000 is EUR 653.88.

Worked case 5: motor_count = 12

This case keeps the nominal configuration except for motor_count=12. The full input is {"motor_allowance": 400.0, "motor_count": 12}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
import landed EUR	5854.8
subtotal EUR	7504.8
reserve EUR	1125.72
total stage EUR	8630.52
budget margin EUR	-2630.52

1. This hypothetical stage contains 12 motors at an internal allowance of EUR 400 each.
2. Net import allowance EUR 4800 plus EUR 120 shipping is taxed at an assumed 19%.
3. Import landed cost is EUR 5854.80; EU gross, fabrication and protection give subtotal EUR 7504.80.
4. Reserve 15% contingency: EUR 1125.72.
5. Total is EUR 8630.52; margin against EUR 6,000 is EUR -2630.52.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Every amount here is a teaching allowance. Actual VAT jurisdiction, customs treatment, supplier terms and compatibility must be checked.

A negative margin means reduce scope or increase funding; essential protection is not the balancing item.

K. Failure modes and troubleshooting

Counting unquoted parts as zero; double-adding VAT; omitting tools/support; treating a compatible torque number as a compatible actuator.

L. Practical exercises and extension tasks

1. Replace allowances with dated exact supplier quotes and recompute.

2. Separate optional laptop spending from the robot budget.
3. Calculate the maximum motor allowance compatible with the remaining stage budget.

M. Connection to the Asimov research platform

The earlier verified-source procurement ledger remains authoritative about what was and was not priced; this scenario provides no new live quotes.

N. Completion record

Record your chosen case, predicted trend, measured import_landed_EUR, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 62, Dimensional tolerances, stack-ups and manufacturing variation. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 62 - Dimensional tolerances, stack-ups and manufacturing variation

A. The engineering question

An assembly can fail to fit even when each part looks close to nominal.

B. Concepts and mathematical model

Tolerance stacks accumulate dimensional variation along a functional path. Worst-case addition provides a deterministic bound under bounded independent specifications, while root-sum-square estimates rely on statistical assumptions. Manufacturing bias and correlation can invalidate the reassuring statistical result. This laboratory compares both calculations with seeded manufactured samples and makes the probability model explicit.

worst_case=sum(abs(t_i)); RSS=sqrt(sum(sigma_i^2)); assembly_error=sum(component_errors).

C. Real-world meaning and limits

Critical bearing, shaft and fastener fits must follow approved drawings and inspection procedures; the example window is not an Asimov tolerance.

D. Architecture and data flow

Configuration (scale, correlation) -> numerical model -> independent_RSS_std_mm, empirical_std_mm, three_sigma_worst_case_mm -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

scale multiplies component dimensional standard deviations; correlation is the nonnegative common-factor fraction.

Parameter	Nominal value	Interpretation
scale	1	Defined by the explicit chapter model and parameter contract
correlation	0	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/62_tolerance_stack.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only correlation changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/62_tolerance_stack.py --case 0

.venv-book/bin/python expanded_lab/modules/62_tolerance_stack.py --case 3

.venv-book/bin/python expanded_lab/run.py tolerance_stack --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/62_tolerance_stack.py

```
"""Dimensional tolerances, stack-ups and manufacturing variation. Numerical teaching
    experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(83)
    n = 20000
    sigma = np.array([0.02, 0.03, 0.015]) * p["scale"]
    correlation = p["correlation"]
    common = rng.normal(0, 1, (n, 1))
    independent = rng.normal(0, 1, (n, 3))
    errors = sigma * (
        np.sqrt(correlation) * common + np.sqrt(1 - correlation) * independent
    )
    total = errors.sum(axis=1)
    assumed_independent = np.sqrt(np.sum(sigma**2))
    worst = 3 * sigma.sum()
    metrics = dict(
        independent_RSS_std_mm=assumed_independent,
        empirical_std_mm=np.std(total),
        three_sigma_worst_case_mm=worst,
        fit_violation_fraction=np.mean(abs(total) > 0.15),
        empirical_95_percentile_abs_mm=np.quantile(abs(total), 0.95),
    )
    derivation = [
        f"Component standard deviations are {sigma.tolist()} mm.",
        f"Use pairwise correlation generated by a common factor with rho={correlation:g}.",
        f"Independent RSS prediction is {assumed_independent:.6f} mm.",
        f"Empirical stack standard deviation is {np.std(total):.6f} mm.",
        f'Against an illustrative +/-0.15 mm assembly window, violation fraction is
    {metrics["fit_violation_fraction"]:.6f}.',
    ]
    notes = [
        "The three-sigma sum is a bounded-specification illustration, not an absolute bound
    for unbounded Gaussian samples.",
        "Actual drawing tolerances, process capability, datum strategy and measurement
    uncertainty determine the proper analysis.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"scale": 1.0, "correlation": 0.0}
    parameters["correlation"] = [0.0, 0.1, 0.25, 0.5, 0.75, 1.0][args.case]
    metrics, derivation, notes = experiment(parameters)
```

```
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Correlated errors are constructed from shared and independent Gaussian factors. Compare empirical variability with the RSS assumption before using the statistical result for fit decisions.

I. Expected behavior and six worked cases

The controlled variable is correlation. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

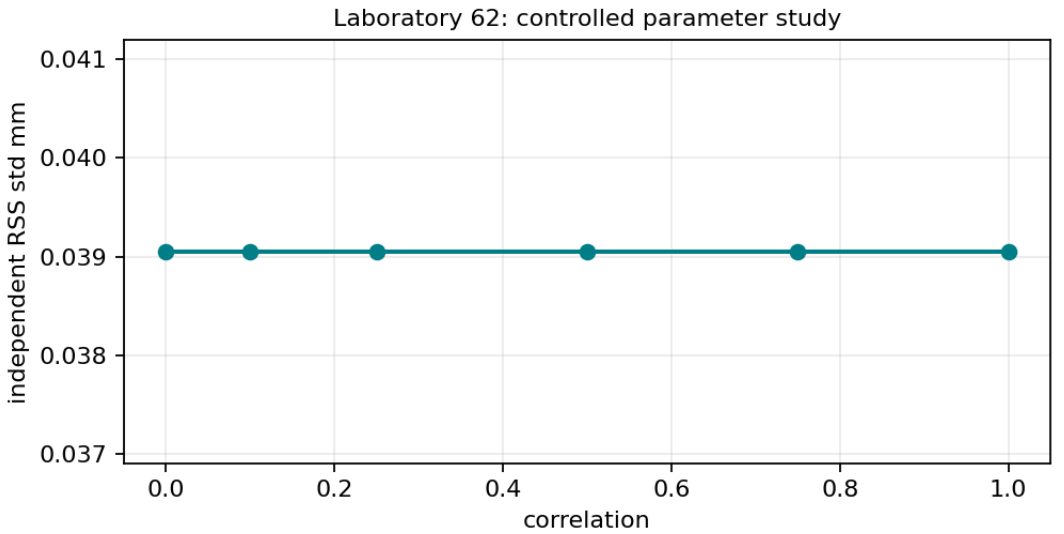


Figure L62. independent RSS std mm as correlation changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	correlation	independent RSS std mm
0	0	0.03905125
1	0.1	0.03905125

Case	correlation	independent RSS std mm
2	0.25	0.03905125
3	0.5	0.03905125
4	0.75	0.03905125
5	1	0.03905125

Worked case 0: correlation = 0

This case keeps the nominal configuration except for correlation=0. The full input is {"correlation": 0.0, "scale": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
independent RSS std mm	0.03905125
empirical std mm	0.03924028
three sigma worst case mm	0.195
fit violation fraction	0.00005
empirical 95 percentile abs mm	0.07765769

1. Component standard deviations are [0.02, 0.03, 0.015] mm.
2. Use pairwise correlation generated by a common factor with rho=0.
3. Independent RSS prediction is 0.039051 mm.
4. Empirical stack standard deviation is 0.039240 mm.
5. Against an illustrative +/-0.15 mm assembly window, violation fraction is 0.000050.

Worked case 1: correlation = 0.1

This case keeps the nominal configuration except for correlation=0.1. The full input is {"correlation": 0.1, "scale": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
independent RSS std mm	0.03905125
empirical std mm	0.04263978
three sigma worst case mm	0.195
fit violation fraction	0.0006
empirical 95 percentile abs mm	0.08334629

1. Component standard deviations are [0.02, 0.03, 0.015] mm.
2. Use pairwise correlation generated by a common factor with rho=0.1.
3. Independent RSS prediction is 0.039051 mm.
4. Empirical stack standard deviation is 0.042640 mm.
5. Against an illustrative +/-0.15 mm assembly window, violation fraction is 0.000600.

Worked case 2: correlation = 0.25

This case keeps the nominal configuration except for correlation=0.25. The full input is {"correlation": 0.25, "scale": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
independent RSS std mm	0.03905125
empirical std mm	0.04715121
three sigma worst case mm	0.195
fit violation fraction	0.00165
empirical 95 percentile abs mm	0.09233163

1. Component standard deviations are [0.02, 0.03, 0.015] mm.
2. Use pairwise correlation generated by a common factor with rho=0.25.
3. Independent RSS prediction is 0.039051 mm.

4. Empirical stack standard deviation is 0.047151 mm.
5. Against an illustrative +/-0.15 mm assembly window, violation fraction is 0.001650.

Worked case 3: correlation = 0.5

This case keeps the nominal configuration except for correlation=0.5. The full input is {"correlation": 0.5, "scale": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
independent RSS std mm	0.03905125
empirical std mm	0.05378017
three sigma worst case mm	0.195
fit violation fraction	0.005
empirical 95 percentile abs mm	0.10534384

1. Component standard deviations are [0.02, 0.03, 0.015] mm.
2. Use pairwise correlation generated by a common factor with rho=0.5.
3. Independent RSS prediction is 0.039051 mm.
4. Empirical stack standard deviation is 0.053780 mm.
5. Against an illustrative +/-0.15 mm assembly window, violation fraction is 0.005000.

Worked case 4: correlation = 0.75

This case keeps the nominal configuration except for correlation=0.75. The full input is {"correlation": 0.75, "scale": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
independent RSS std mm	0.03905125
empirical std mm	0.0596332

Computed quantity	Numerical result
three sigma worst case mm	0.195
fit violation fraction	0.0113
empirical 95 percentile abs mm	0.11682198

1. Component standard deviations are [0.02, 0.03, 0.015] mm.
2. Use pairwise correlation generated by a common factor with rho=0.75.
3. Independent RSS prediction is 0.039051 mm.
4. Empirical stack standard deviation is 0.059633 mm.
5. Against an illustrative +/-0.15 mm assembly window, violation fraction is 0.011300.

Worked case 5: correlation = 1

This case keeps the nominal configuration except for correlation=1. The full input is {"correlation": 1.0, "scale": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
independent RSS std mm	0.03905125
empirical std mm	0.06485032
three sigma worst case mm	0.195
fit violation fraction	0.0214
empirical 95 percentile abs mm	0.12672294

1. Component standard deviations are [0.02, 0.03, 0.015] mm.
2. Use pairwise correlation generated by a common factor with rho=1.
3. Independent RSS prediction is 0.039051 mm.
4. Empirical stack standard deviation is 0.064850 mm.
5. Against an illustrative +/-0.15 mm assembly window, violation fraction is 0.021400.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The three-sigma sum is a bounded-specification illustration, not an absolute bound for unbounded Gaussian samples.

Actual drawing tolerances, process capability, datum strategy and measurement uncertainty determine the proper analysis.

K. Failure modes and troubleshooting

Treating drawing limits as Gaussian sigma; ignoring common fixture bias; mixing dimensions from inconsistent datums.

L. Practical exercises and extension tasks

1. Add a systematic machining offset to all parts.
2. Replace Gaussian errors with bounded uniform errors.
3. Include instrument uncertainty in acceptance decisions.

M. Connection to the Asimov research platform

Critical bearing, shaft and fastener fits must follow approved drawings and inspection procedures; the example window is not an Asimov tolerance.

N. Completion record

Record your chosen case, predicted trend, measured independent_RSS_std_mm, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 63, Load paths, bolt-group screening and nominal stress. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 63 - Load paths, bolt-group screening and nominal stress

A. The engineering question

Fastener calculations begin with the load path and joint geometry.

B. Concepts and mathematical model

Dividing a total force equally among bolts can be wrong when an applied moment creates nonuniform load. Preload, joint stiffness, friction, prying, fatigue and thread engagement further complicate real design. This laboratory screens an ideal symmetric bolt circle under direct axial force plus a moment, reporting the largest modeled bolt load and nominal tensile stress. It intentionally stops short of approving a joint.

F_i=F_total/n + M*y_i/sum(y_j^2); nominal stress=F_max/assumed_tensile_area.

C. Real-world meaning and limits

Load-bearing robot and support-rig joints require competent mechanical design review, released fastener specifications and documented assembly.

D. Architecture and data flow

Configuration (radius, axial_force, moment) -> numerical model -> maximum_bolt_load_N, minimum_bolt_load_N, nominal_max_tensile_stress_MPa -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

radius is meters; axial_force is Newtons; moment is Nm; the assumed tensile area is only a numerical teaching input.

Parameter	Nominal value	Interpretation
radius	0.025	Defined by the explicit chapter model and parameter contract
axial_force	400	Defined by the explicit chapter model and parameter contract
moment	5	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/63_fastener_screen.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only radius changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/63_fastener_screen.py --case 0

.venv-book/bin/python expanded_lab/modules/63_fastener_screen.py --case 3

.venv-book/bin/python expanded_lab/run.py fastener_screen --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/63_fastener_screen.py

```
"""Load paths, bolt-group screening and nominal stress. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    n = 4
    radius = p["radius"]
    F = p["axial_force"]
    moment = p["moment"]
    angles = np.arange(n) * 2 * np.pi / n
    y = radius * np.sin(angles)
    loads = F / n + moment * y / np.sum(y * y)
    area = 8e-6
    stress = loads.max() / area / 1e6
    metrics = dict(
        maximum_bolt_load_N=loads.max(),
        minimum_bolt_load_N=loads.min(),
        nominal_max_tensile_stress_MPa=stress,
        bolt_circle_radius_m=radius,
        recovered_total_force_N=loads.sum(),
    )
    assert np.isclose(loads.sum(), F)
    derivation = [
        f"Four ideal bolts lie on radius {radius:g} m with y coordinates {y.tolist()} m.",
        f"Apply direct axial force {F:g} N and bending moment {moment:g} Nm.",
        f"The simplified bolt loads are {loads.tolist()} N.",
        f"Maximum load {loads.max():.6f} N divided by assumed area 8 mm^2 gives {stress:.6f}
     MPa.",
        f"Summed axial force recovers {loads.sum():.6f} N as a consistency check.",
    ]
    notes = [
        "Negative calculated load indicates the simple tension-only interpretation needs
    joint contact/separation analysis.",
        "No allowable stress, bolt grade, preload or tightening torque is specified; this is
     not a structural approval.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"radius": 0.025, "axial_force": 400.0, "moment": 5.0}
    parameters["radius"] = [0.015, 0.02, 0.025, 0.03, 0.04, 0.05][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
```

```
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The moment contribution depends on bolt position, while its sum is zero for the symmetric group. Check recovered force before trusting peak stress.

I. Expected behavior and six worked cases

The controlled variable is radius. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

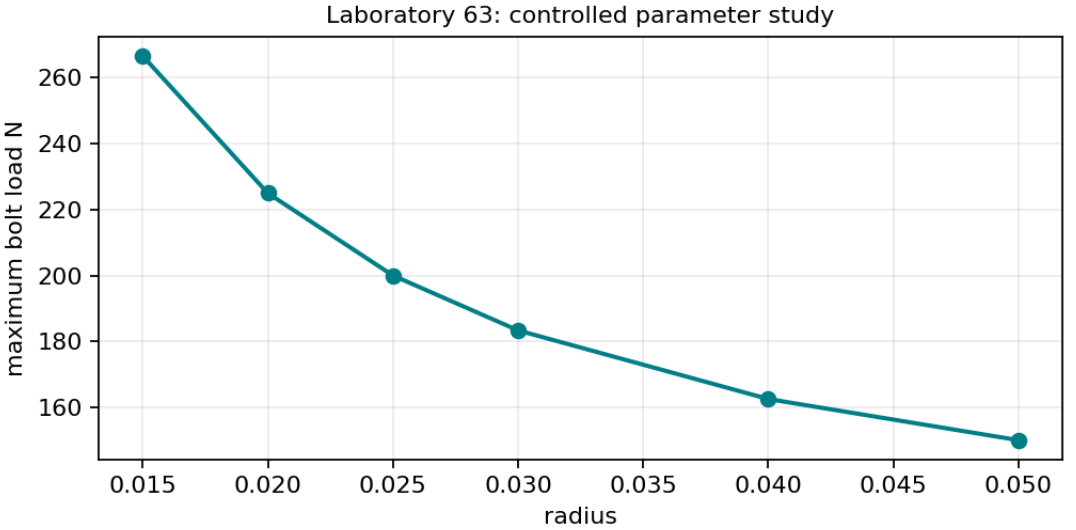


Figure L63. maximum bolt load N as radius changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	radius	maximum bolt load N
0	0.015	266.66666667
1	0.02	225
2	0.025	200

Case	radius	maximum bolt load N
3	0.03	183.33333333
4	0.04	162.5
5	0.05	150

Worked case 0: radius = 0.015

This case keeps the nominal configuration except for radius=0.015. The full input is {"axial_force": 400.0, "moment": 5.0, "radius": 0.015}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
maximum bolt load N	266.66666667
minimum bolt load N	-66.66666667
nominal max tensile stress MPa	33.33333333
bolt circle radius m	0.015
recovered total force N	400

1. Four ideal bolts lie on radius 0.015 m with y coordinates [0.0, 0.015, 1.8369701987210296e-18, -0.015] m.
2. Apply direct axial force 400 N and bending moment 5 Nm.
3. The simplified bolt loads are [100.0, 266.66666666666663, 100.00000000000001, -66.66666666666666] N.
4. Maximum load 266.666667 N divided by assumed area 8 mm^2 gives 33.333333 MPa.
5. Summed axial force recovers 400.000000 N as a consistency check.

Worked case 1: radius = 0.02

This case keeps the nominal configuration except for radius=0.02. The full input is {"axial_force": 400.0, "moment": 5.0, "radius": 0.02}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
maximum bolt load N	225
minimum bolt load N	-25
nominal max tensile stress MPa	28.125
bolt circle radius m	0.02
recovered total force N	400

1. Four ideal bolts lie on radius 0.02 m with y coordinates [0.0, 0.02, 2.4492935982947064e-18, -0.02] m.
2. Apply direct axial force 400 N and bending moment 5 Nm.
3. The simplified bolt loads are [100.0, 225.0, 100.00000000000001, -25.0] N.
4. Maximum load 225.000000 N divided by assumed area 8 mm^2 gives 28.125000 MPa.
5. Summed axial force recovers 400.000000 N as a consistency check.

Worked case 2: radius = 0.025

This case keeps the nominal configuration except for radius=0.025. The full input is {"axial_force": 400.0, "moment": 5.0, "radius": 0.025}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
maximum bolt load N	200
minimum bolt load N	1.4210855e-14
nominal max tensile stress MPa	25

Computed quantity	Numerical result
bolt circle radius m	0.025
recovered total force N	400

1. Four ideal bolts lie on radius 0.025 m with y coordinates [0.0, 0.025, 3.061616997868383e-18, -0.025] m.
2. Apply direct axial force 400 N and bending moment 5 Nm.
3. The simplified bolt loads are [100.0, 200.0, 100.00000000000001, 1.4210854715202004e-14] N.
4. Maximum load 200.000000 N divided by assumed area 8 mm^2 gives 25.000000 MPa.
5. Summed axial force recovers 400.000000 N as a consistency check.

Worked case 3: radius = 0.03

This case keeps the nominal configuration except for radius=0.03. The full input is {"axial_force": 400.0, "moment": 5.0, "radius": 0.03}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
maximum bolt load N	183.33333333
minimum bolt load N	16.66666667
nominal max tensile stress MPa	22.91666667
bolt circle radius m	0.03
recovered total force N	400

1. Four ideal bolts lie on radius 0.03 m with y coordinates [0.0, 0.03, 3.673940397442059e-18, -0.03] m.
2. Apply direct axial force 400 N and bending moment 5 Nm.
3. The simplified bolt loads are [100.0, 183.33333333333331, 100.00000000000001, 16.66666666666667] N.

4. Maximum load 183.333333 N divided by assumed area 8 mm^2 gives 22.916667 MPa.
5. Summed axial force recovers 400.000000 N as a consistency check.

Worked case 4: radius = 0.04

This case keeps the nominal configuration except for radius=0.04. The full input is {"axial_force": 400.0, "moment": 5.0, "radius": 0.04}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
maximum bolt load N	162.5
minimum bolt load N	37.5
nominal max tensile stress MPa	20.3125
bolt circle radius m	0.04
recovered total force N	400

1. Four ideal bolts lie on radius 0.04 m with y coordinates [0.0, 0.04, 4.898587196589413e-18, -0.04] m.
2. Apply direct axial force 400 N and bending moment 5 Nm.
3. The simplified bolt loads are [100.0, 162.5, 100.00000000000001, 37.5] N.
4. Maximum load 162.500000 N divided by assumed area 8 mm^2 gives 20.312500 MPa.
5. Summed axial force recovers 400.000000 N as a consistency check.

Worked case 5: radius = 0.05

This case keeps the nominal configuration except for radius=0.05. The full input is {"axial_force": 400.0, "moment": 5.0, "radius": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
maximum bolt load N	150
minimum bolt load N	50
nominal max tensile stress MPa	18.75
bolt circle radius m	0.05
recovered total force N	400

1. Four ideal bolts lie on radius 0.05 m with y coordinates [0.0, 0.05, 6.123233995736766e-18, -0.05] m.
2. Apply direct axial force 400 N and bending moment 5 Nm.
3. The simplified bolt loads are [100.0, 150.0, 100.0, 50.00000000000001] N.
4. Maximum load 150.000000 N divided by assumed area 8 mm^2 gives 18.750000 MPa.
5. Summed axial force recovers 400.000000 N as a consistency check.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Negative calculated load indicates the simple tension-only interpretation needs joint contact/separation analysis.

No allowable stress, bolt grade, preload or tightening torque is specified; this is not a structural approval.

K. Failure modes and troubleshooting

Assuming equal bolt loading under moment; using nominal shank area instead of the appropriate tensile area; selecting torque without joint/preload analysis.

L. Practical exercises and extension tasks

1. Change the bolt-group geometry and recompute the moment distribution.

2. Add a conservative dynamic load case.
3. List the missing evidence needed for a real design approval.

M. Connection to the Asimov research platform

Load-bearing robot and support-rig joints require competent mechanical design review, released fastener specifications and documented assembly.

N. Completion record

Record your chosen case, predicted trend, measured maximum_bolt_load_N, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 64, Cable resistance, voltage drop and heat screening. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 64 - Cable resistance, voltage drop and heat screening

A. The engineering question

A conductor's resistance creates both voltage drop and heat.

B. Concepts and mathematical model

The return path contributes to loop resistance and must be included. A short cable can still heat at high current, while a longer cable can cause a controller brownout before a thermal problem becomes obvious. This calculation screens resistance effects with an assumed copper resistivity; it does not choose a safe wire gauge, fuse or connector rating. Installation method, insulation, bundling, temperature and fault protection require separate review.

R_loop=rho*(2*length)/area; voltage_drop=I*R_loop; heating=I^2*R_loop.

C. Real-world meaning and limits

Actual conductor, connector, fuse and insulation selection requires verified hardware specifications and installation-specific electrical review.

D. Architecture and data flow

Configuration (one_way_length, area_mm2, current) -> numerical model -> loop_resistance_ohm, voltage_drop_V, conductor_heat_W -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

one_way_length is meters; area_mm2 is conductor cross-section in mm^2; current is Amperes.

Parameter	Nominal value	Interpretation
one_way_length	1	Defined by the explicit chapter model and parameter contract
area_mm2	1.5	Defined by the explicit chapter model and parameter contract
current	5	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/64_cable_loss.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only current changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.

5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/64_cable_loss.py --case 0

.venv-book/bin/python expanded_lab/modules/64_cable_loss.py --case 3

.venv-book/bin/python expanded_lab/run.py cable_loss --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/64_cable_loss.py

```
"""Cable resistance, voltage drop and heat screening. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rho = 1.72e-8
    length = p["one_way_length"]
    area = p["area_mm2"] * 1e-6
    current = p["current"]
    R = rho * 2 * length / area
    drop = current * R
    heat = current * current * R
    metrics = dict(
        loop_resistance_ohm=R,
        voltage_drop_V=drop,
        conductor_heat_W=heat,
        drop_fraction_of_48V=drop / 48.0,
        heat_per_one_way_meter_W_m=heat / length,
    )
    derivation = [
        f'Assume rho={rho:.3e} ohm*m, one-way length {length:g} m and area {p["area_mm2"]:g}
     mm^2.',
        f"Include outgoing and return conductors: loop resistance is {R:.8f} ohm.",
        f"At current {current:g} A, voltage drop is {drop:.6f} V.",
        f"Joule heating is I^2*R={heat:.6f} W across the loop.",
        f"Drop is {drop/48*100:.6f}% of an illustrative 48 V bus.",
    ]
    notes = [
        "The 48 V reference and resistivity are declared arithmetic assumptions, not an
    approved operating voltage or wire rating.",
        "Connector contact resistance, flex life, thermal environment and protective-device
    coordination are excluded.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"one_way_length": 1.0, "area_mm2": 1.5, "current": 5.0}
    parameters["current"] = [1.0, 2.0, 5.0, 10.0, 15.0, 20.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
```

```
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Convert square millimeters into square meters before applying resistivity. Voltage drop scales linearly with current while heating scales quadratically, so one nominal-voltage check cannot establish a thermal margin.

I. Expected behavior and six worked cases

The controlled variable is current. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

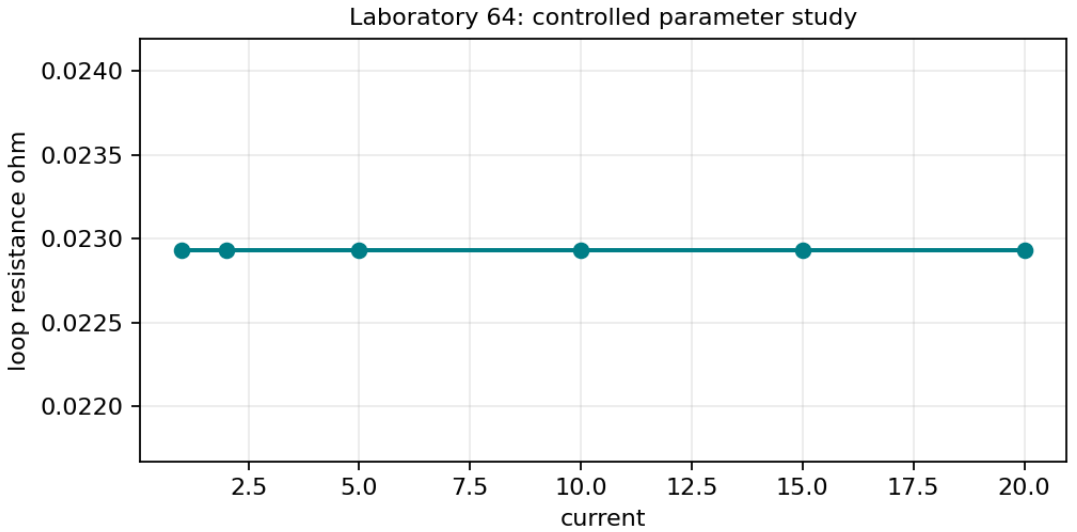


Figure L64. loop resistance ohm as current changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	current	loop resistance ohm
0	1	0.02293333
1	2	0.02293333
2	5	0.02293333
3	10	0.02293333

Case	current	loop resistance ohm
4	15	0.02293333
5	20	0.02293333

Worked case 0: current = 1

This case keeps the nominal configuration except for current=1. The full input is {"area_mm2": 1.5, "current": 1.0, "one_way_length": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
loop resistance ohm	0.02293333
voltage drop V	0.02293333
conductor heat W	0.02293333
drop fraction of 48V	0.00047778
heat per one way meter W m	0.02293333

1. Assume rho=1.720e-08 ohm*m, one-way length 1 m and area 1.5 mm^2.
2. Include outgoing and return conductors: loop resistance is 0.02293333 ohm.
3. At current 1 A, voltage drop is 0.022933 V.
4. Joule heating is I^2*R=0.022933 W across the loop.
5. Drop is 0.047778% of an illustrative 48 V bus.

Worked case 1: current = 2

This case keeps the nominal configuration except for current=2. The full input is {"area_mm2": 1.5, "current": 2.0, "one_way_length": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
loop resistance ohm	0.02293333
voltage drop V	0.04586667
conductor heat W	0.09173333
drop fraction of 48V	0.00095556
heat per one way meter W m	0.09173333

1. Assume rho=1.720e-08 ohm*m, one-way length 1 m and area 1.5 mm^2.
2. Include outgoing and return conductors: loop resistance is 0.02293333 ohm.
3. At current 2 A, voltage drop is 0.045867 V.
4. Joule heating is I^2*R=0.091733 W across the loop.
5. Drop is 0.095556% of an illustrative 48 V bus.

Worked case 2: current = 5

This case keeps the nominal configuration except for current=5. The full input is {"area_mm2": 1.5, "current": 5.0, "one_way_length": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
loop resistance ohm	0.02293333
voltage drop V	0.11466667
conductor heat W	0.57333333
drop fraction of 48V	0.00238889
heat per one way meter W m	0.57333333

1. Assume rho=1.720e-08 ohm*m, one-way length 1 m and area 1.5 mm^2.
2. Include outgoing and return conductors: loop resistance is 0.02293333 ohm.

3. At current 5 A, voltage drop is 0.114667 V.
4. Joule heating is I^2*R=0.573333 W across the loop.
5. Drop is 0.238889% of an illustrative 48 V bus.

Worked case 3: current = 10

This case keeps the nominal configuration except for current=10. The full input is {"area_mm2": 1.5, "current": 10.0, "one_way_length": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
loop resistance ohm	0.02293333
voltage drop V	0.22933333
conductor heat W	2.29333333
drop fraction of 48V	0.00477778
heat per one way meter W m	2.29333333

1. Assume rho=1.720e-08 ohm*m, one-way length 1 m and area 1.5 mm^2.
2. Include outgoing and return conductors: loop resistance is 0.02293333 ohm.
3. At current 10 A, voltage drop is 0.229333 V.
4. Joule heating is I^2*R=2.293333 W across the loop.
5. Drop is 0.477778% of an illustrative 48 V bus.

Worked case 4: current = 15

This case keeps the nominal configuration except for current=15. The full input is {"area_mm2": 1.5, "current": 15.0, "one_way_length": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
loop resistance ohm	0.02293333

Computed quantity	Numerical result
voltage drop V	0.344
conductor heat W	5.16
drop fraction of 48V	0.00716667
heat per one way meter W m	5.16

1. Assume rho=1.720e-08 ohm*m, one-way length 1 m and area 1.5 mm^2.
2. Include outgoing and return conductors: loop resistance is 0.02293333 ohm.
3. At current 15 A, voltage drop is 0.344000 V.
4. Joule heating is I^2*R=5.160000 W across the loop.
5. Drop is 0.716667% of an illustrative 48 V bus.

Worked case 5: current = 20

This case keeps the nominal configuration except for current=20. The full input is {"area_mm2": 1.5, "current": 20.0, "one_way_length": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
loop resistance ohm	0.02293333
voltage drop V	0.45866667
conductor heat W	9.17333333
drop fraction of 48V	0.00955556
heat per one way meter W m	9.17333333

1. Assume rho=1.720e-08 ohm*m, one-way length 1 m and area 1.5 mm^2.
2. Include outgoing and return conductors: loop resistance is 0.02293333 ohm.
3. At current 20 A, voltage drop is 0.458667 V.

4. Joule heating is I^2*R=9.173333 W across the loop.
5. Drop is 0.955556% of an illustrative 48 V bus.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The 48 V reference and resistivity are declared arithmetic assumptions, not an approved operating voltage or wire rating.

Connector contact resistance, flex life, thermal environment and protective-device coordination are excluded.

K. Failure modes and troubleshooting

Forgetting the return conductor; using current averages to assess peak drop; assuming a connector shell name establishes current capacity.

L. Practical exercises and extension tasks

1. Add measured connector contact resistance.
2. Evaluate duty-cycle RMS current for heating.
3. Propagate a warm-conductor resistance increase and compare drop.

M. Connection to the Asimov research platform

Actual conductor, connector, fuse and insulation selection requires verified hardware specifications and installation-specific electrical review.

N. Completion record

Record your chosen case, predicted trend, measured loop_resistance_ohm, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 65, Capacitor charging, stored energy and inrush screening. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 65 - Capacitor charging, stored energy and inrush screening

A. The engineering question

Power electronics often contain input capacitance.

B. Concepts and mathematical model

Connecting an uncharged capacitor to a voltage source can create a large transient limited by source and circuit impedance. A resistor-capacitor model explains initial current, charging time and stored energy, but real switching devices, supplies and precharge sequences require a reviewed design. This exercise does not prescribe a physical precharge circuit or safe component values; it makes the energy and timing quantities explicit.

I0=V/R; Vc(t)=V*(1-exp(-t/(R*C))); E_stored=0.5*C*V^2; t_95=-R*C*ln(0.05).

C. Real-world meaning and limits

Use the released power topology and competent electrical review for actual actuator supply, battery, protection and discharge design.

D. Architecture and data flow

Configuration (voltage, resistance, capacitance) -> numerical model -> initial_current_A, time_constant_s, charge_95_percent_time_s -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

voltage is Volts; resistance is ohms; capacitance is Farads. Values are educational, not a build prescription.

Parameter	Nominal value	Interpretation
voltage	24	Defined by the explicit chapter model and parameter contract
resistance	10	Defined by the explicit chapter model and parameter contract
capacitance	0.002	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/65_inrush.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only resistance changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/65_inrush.py --case 0

.venv-book/bin/python expanded_lab/modules/65_inrush.py --case 3

.venv-book/bin/python expanded_lab/run.py inrush --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/65_inrush.py

```
"""Capacitor charging, stored energy and inrush screening. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    V, R, C = p["voltage"], p["resistance"], p["capacitance"]
    tau = R * C
    initial = V / R
    energy = 0.5 * C * V * V
    t95 = -tau * np.log(0.05)
    power0 = V * V / R
    metrics = dict(
        initial_current_A=initial,
        time_constant_s=tau,
        charge_95_percent_time_s=t95,
        stored_energy_J=energy,
        initial_resistor_power_W=power0,
    )
    derivation = [
        f"Use illustrative source V={V:g} V, R={R:g} ohm and C={C:g} F.",
        f"Initial ideal charging current is {initial:.6f} A.",
        f"The RC time constant is {tau:.6f} s and 95% charge time is {t95:.6f} s.",
        f"Capacitor stored energy at full voltage is {energy:.6f} J.",
        f"Initial resistor power is {power0:.6f} W and decays with charging.",
    ]
    notes = [
        "Stored energy can remain after source disconnection; a physical discharge/isolation
     procedure must verify it.",
        "Resistor pulse energy, switching ratings, failure modes and supply behavior are not
     established by the ideal RC equation.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"voltage": 24.0, "resistance": 10.0, "capacitance": 0.002}
    parameters["resistance"] = [1.0, 2.0, 5.0, 10.0, 20.0, 50.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Analyze current, energy and charging time together. A large resistance reduces initial current but lengthens the charging interval; a nominal power rating alone does not specify pulse capability.

I. Expected behavior and six worked cases

The controlled variable is resistance. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.



Figure L65. initial current A as resistance changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	resistance	initial current A
0	1	24
1	2	12
2	5	4.8
3	10	2.4
4	20	1.2

Case	resistance	initial current A
5	50	0.48

Worked case 0: resistance = 1

This case keeps the nominal configuration except for resistance=1. The full input is {"capacitance": 0.002, "resistance": 1.0, "voltage": 24.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
initial current A	24
time constant s	0.002
charge 95 percent time s	0.00599146
stored energy J	0.576
initial resistor power W	576

1. Use illustrative source V=24 V, R=1 ohm and C=0.002 F.
2. Initial ideal charging current is 24.000000 A.
3. The RC time constant is 0.002000 s and 95% charge time is 0.005991 s.
4. Capacitor stored energy at full voltage is 0.576000 J.
5. Initial resistor power is 576.000000 W and decays with charging.

Worked case 1: resistance = 2

This case keeps the nominal configuration except for resistance=2. The full input is {"capacitance": 0.002, "resistance": 2.0, "voltage": 24.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
initial current A	12

Computed quantity	Numerical result
time constant s	0.004
charge 95 percent time s	0.01198293
stored energy J	0.576
initial resistor power W	288

1. Use illustrative source V=24 V, R=2 ohm and C=0.002 F.
2. Initial ideal charging current is 12.000000 A.
3. The RC time constant is 0.004000 s and 95% charge time is 0.011983 s.
4. Capacitor stored energy at full voltage is 0.576000 J.
5. Initial resistor power is 288.000000 W and decays with charging.

Worked case 2: resistance = 5

This case keeps the nominal configuration except for resistance=5. The full input is {"capacitance": 0.002, "resistance": 5.0, "voltage": 24.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
initial current A	4.8
time constant s	0.01
charge 95 percent time s	0.02995732
stored energy J	0.576
initial resistor power W	115.2

1. Use illustrative source V=24 V, R=5 ohm and C=0.002 F.
2. Initial ideal charging current is 4.800000 A.
3. The RC time constant is 0.010000 s and 95% charge time is 0.029957 s.
4. Capacitor stored energy at full voltage is 0.576000 J.
5. Initial resistor power is 115.200000 W and decays with charging.

Worked case 3: resistance = 10

This case keeps the nominal configuration except for resistance=10. The full input is {"capacitance": 0.002, "resistance": 10.0, "voltage": 24.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
initial current A	2.4
time constant s	0.02
charge 95 percent time s	0.05991465
stored energy J	0.576
initial resistor power W	57.6

1. Use illustrative source V=24 V, R=10 ohm and C=0.002 F.
2. Initial ideal charging current is 2.400000 A.
3. The RC time constant is 0.020000 s and 95% charge time is 0.059915 s.
4. Capacitor stored energy at full voltage is 0.576000 J.
5. Initial resistor power is 57.600000 W and decays with charging.

Worked case 4: resistance = 20

This case keeps the nominal configuration except for resistance=20. The full input is {"capacitance": 0.002, "resistance": 20.0, "voltage": 24.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
initial current A	1.2
time constant s	0.04
charge 95 percent time s	0.11982929
stored energy J	0.576

Computed quantity	Numerical result
initial resistor power W	28.8

1. Use illustrative source V=24 V, R=20 ohm and C=0.002 F.
2. Initial ideal charging current is 1.200000 A.
3. The RC time constant is 0.040000 s and 95% charge time is 0.119829 s.
4. Capacitor stored energy at full voltage is 0.576000 J.
5. Initial resistor power is 28.800000 W and decays with charging.

Worked case 5: resistance = 50

This case keeps the nominal configuration except for resistance=50. The full input is {"capacitance": 0.002, "resistance": 50.0, "voltage": 24.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
initial current A	0.48
time constant s	0.1
charge 95 percent time s	0.29957323
stored energy J	0.576
initial resistor power W	11.52

1. Use illustrative source V=24 V, R=50 ohm and C=0.002 F.
2. Initial ideal charging current is 0.480000 A.
3. The RC time constant is 0.100000 s and 95% charge time is 0.299573 s.
4. Capacitor stored energy at full voltage is 0.576000 J.
5. Initial resistor power is 11.520000 W and decays with charging.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Stored energy can remain after source disconnection; a physical discharge/isolation procedure must verify it.

Resistor pulse energy, switching ratings, failure modes and supply behavior are not established by the ideal RC equation.

K. Failure modes and troubleshooting

Ignoring stored energy after disconnect; assuming a supply current limit substitutes for all inrush protection; choosing components from one scalar calculation.

L. Practical exercises and extension tasks

1. Calculate energy dissipated in the ideal resistor over the charge.
2. Model a partial charge before reconnection.
3. List fault cases a real precharge controller would have to detect.

M. Connection to the Asimov research platform

Use the released power topology and competent electrical review for actual actuator supply, battery, protection and discharge design.

N. Completion record

Record your chosen case, predicted trend, measured initial_current_A, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 66, Bidirectional calibration and hidden hysteresis. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 66 - Bidirectional calibration and hidden hysteresis

A. The engineering question

A sensor calibration fitted in one direction may fail on reversal.

B. Concepts and mathematical model

Backlash, friction and structural compliance can create direction-dependent observations that a single scale-and-offset model cannot represent. The correct response is to expose the residual structure rather than silently widen a tolerance. This experiment generates forward and reverse measurements with a known hysteresis term, fits a shared linear calibration and quantifies the remaining directional bias.

measurement=a*reference+b+direction*hysteresis+noise; a shared fit cannot remove opposite signed direction offsets.

C. Real-world meaning and limits

Joint and ankle calibration should include reversals, load conditions and temperature ranges relevant to the intended physical skill.

D. Architecture and data flow

Configuration (hysteresis) -> numerical model -> fitted_scale, fitted_offset_rad, pooled_residual_rmse_rad -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

hysteresis is the signed measurement offset magnitude in radians for forward versus reverse motion.

Parameter	Nominal value	Interpretation
hysteresis	0.005	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/66_hysteresis_calibration.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only hysteresis changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/66_hysteresis_calibration.py --case 0

.venv-book/bin/python expanded_lab/modules/66_hysteresis_calibration.py --case 3

.venv-book/bin/python expanded_lab/run.py hysteresis_calibration --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/66_hysteresis_calibration.py

```
"""Bidirectional calibration and hidden hysteresis. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(89)
    x = np.linspace(-0.5, 0.5, 100)
    h = p["hysteresis"]
    a, b = 1.02, 0.01
    forward = a * x + b + h + rng.normal(0, 0.001, len(x))
    reverse = a * x + b - h + rng.normal(0, 0.001, len(x))
    X = np.column_stack([np.tile(x, 2), np.ones(200)])
    y = np.concatenate([forward, reverse])
    theta = np.linalg.lstsq(X, y, rcond=None)[0]
    residual = y - X @ theta
    f_bias = residual[:100].mean()
    r_bias = residual[100:].mean()
    metrics = dict(
        fitted_scale=theta[0],
        fitted_offset_rad=theta[1],
        pooled_residual_rmse_rad=np.sqrt(np.mean(residual**2)),
        forward_bias_rad=f_bias,
        reverse_bias_rad=r_bias,
        directional_separation_rad=f_bias - r_bias,
    )
    derivation = [
        f"True scale is {a:g} and offset {b:g} rad.",
        f"Add directional offset +/-{h:g} rad and 0.001 rad noise.",
        f"Fit one shared scale/offset to both directions: {theta.tolist()}.",
        f"Forward residual mean is {f_bias:.6f} rad; reverse mean is {r_bias:.6f} rad.",
        f"Directional separation is {f_bias-r_bias:.6f} rad, revealing structure hidden by a
     pooled fit.",
    ]
    notes = [
        "A low aggregate fit error can still conceal a consistent direction-dependent error
    relevant to reaching or balance.",
        "A calibration model should match the phenomenon and be validated on independent
    bidirectional trajectories.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"hysteresis": 0.005}
    parameters["hysteresis"] = [0.0, 0.001, 0.002, 0.005, 0.01, 0.02][args.case]
    metrics, derivation, notes = experiment(parameters)
```

```
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Preserve direction labels during data collection. A residual-versus-direction comparison distinguishes hysteresis from ordinary zero-mean measurement noise.

I. Expected behavior and six worked cases

The controlled variable is hysteresis. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

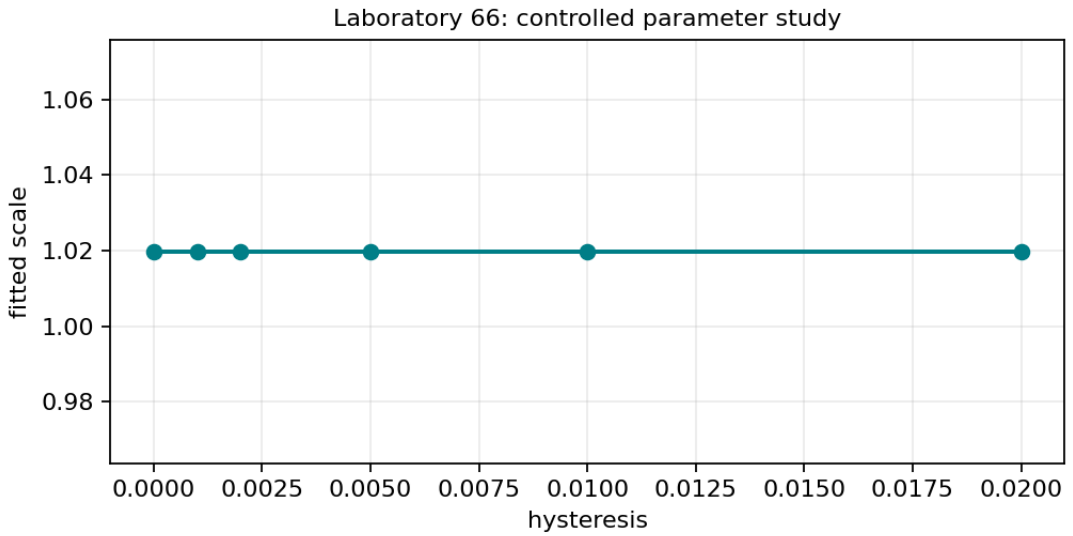


Figure L66. fitted scale as hysteresis changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	hysteresis	fitted scale
0	0	1.0197408
1	0.001	1.0197408

Case	hysteresis	fitted scale
2	0.002	1.0197408
3	0.005	1.0197408
4	0.01	1.0197408
5	0.02	1.0197408

Worked case 0: hysteresis = 0

This case keeps the nominal configuration except for hysteresis=0. The full input is {"hysteresis": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
fitted scale	1.0197408
fitted offset rad	0.00998084
pooled residual rmse rad	0.00094312
forward bias rad	-0.00005539
reverse bias rad	0.00005539
directional separation rad	-0.00011078

1. True scale is 1.02 and offset 0.01 rad.
2. Add directional offset +/-0 rad and 0.001 rad noise.
3. Fit one shared scale/offset to both directions: [1.0197407959775369, 0.00998083550852672].
4. Forward residual mean is -0.000055 rad; reverse mean is 0.000055 rad.
5. Directional separation is -0.000111 rad, revealing structure hidden by a pooled fit.

Worked case 1: hysteresis = 0.001

This case keeps the nominal configuration except for hysteresis=0.001. The full input is {"hysteresis": 0.001}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
fitted scale	1.0197408
fitted offset rad	0.00998084
pooled residual rmse rad	0.00133368
forward bias rad	0.00094461
reverse bias rad	-0.00094461
directional separation rad	0.00188922

1. True scale is 1.02 and offset 0.01 rad.
2. Add directional offset +/-0.001 rad and 0.001 rad noise.
3. Fit one shared scale/offset to both directions: [1.0197407959775364, 0.0099808355085267 28].
4. Forward residual mean is 0.000945 rad; reverse mean is -0.000945 rad.
5. Directional separation is 0.001889 rad, revealing structure hidden by a pooled fit.

Worked case 2: hysteresis = 0.002

This case keeps the nominal configuration except for hysteresis=0.002. The full input is {"hysteresis": 0.002}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
fitted scale	1.0197408
fitted offset rad	0.00998084
pooled residual rmse rad	0.00216054

Computed quantity	Numerical result
forward bias rad	0.00194461
reverse bias rad	-0.00194461
directional separation rad	0.00388922

1. True scale is 1.02 and offset 0.01 rad.
2. Add directional offset +/-0.002 rad and 0.001 rad noise.
3. Fit one shared scale/offset to both directions: [1.0197407959775364, 0.0099980835508526721].
4. Forward residual mean is 0.001945 rad; reverse mean is -0.001945 rad.
5. Directional separation is 0.003889 rad, revealing structure hidden by a pooled fit.

Worked case 3: hysteresis = 0.005

This case keeps the nominal configuration except for hysteresis=0.005. The full input is {"hysteresis": 0.005}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
fitted scale	1.0197408
fitted offset rad	0.00998084
pooled residual rmse rad	0.00503345
forward bias rad	0.00494461
reverse bias rad	-0.00494461
directional separation rad	0.00988922

1. True scale is 1.02 and offset 0.01 rad.
2. Add directional offset +/-0.005 rad and 0.001 rad noise.
3. Fit one shared scale/offset to both directions: [1.0197407959775364, 0.0099980835508526726].

4. Forward residual mean is 0.004945 rad; reverse mean is -0.004945 rad.
5. Directional separation is 0.009889 rad, revealing structure hidden by a pooled fit.

Worked case 4: hysteresis = 0.01

This case keeps the nominal configuration except for hysteresis=0.01. The full input is {"hysteresis": 0.01}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
fitted scale	1.0197408
fitted offset rad	0.00998084
pooled residual rmse rad	0.00998908
forward bias rad	0.00994461
reverse bias rad	-0.00994461
directional separation rad	0.01988922

1. True scale is 1.02 and offset 0.01 rad.
2. Add directional offset +/-0.01 rad and 0.001 rad noise.
3. Fit one shared scale/offset to both directions: [1.0197407959775364, 0.0099980835508526726].
4. Forward residual mean is 0.009945 rad; reverse mean is -0.009945 rad.
5. Directional separation is 0.019889 rad, revealing structure hidden by a pooled fit.

Worked case 5: hysteresis = 0.02

This case keeps the nominal configuration except for hysteresis=0.02. The full input is {"hysteresis": 0.02}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
fitted scale	1.0197408

Computed quantity	Numerical result
fitted offset rad	0.00998084
pooled residual rmse rad	0.01996682
forward bias rad	0.01994461
reverse bias rad	-0.01994461
directional separation rad	0.03988922

1. True scale is 1.02 and offset 0.01 rad.
2. Add directional offset +/-0.02 rad and 0.001 rad noise.
3. Fit one shared scale/offset to both directions: [1.0197407959775366, 0.0099980835508526723].
4. Forward residual mean is 0.019945 rad; reverse mean is -0.019945 rad.
5. Directional separation is 0.039889 rad, revealing structure hidden by a pooled fit.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

A low aggregate fit error can still conceal a consistent direction-dependent error relevant to reaching or balance.

A calibration model should match the phenomenon and be validated on independent bidirectional trajectories.

K. Failure modes and troubleshooting

Fitting only one sweep direction; treating backlash as a fixed zero offset; using the same calibration trajectories as final validation.

L. Practical exercises and extension tasks

1. Fit an explicit direction-dependent term and compare held-out residuals.
2. Add temperature dependence and test a two-factor model.

3. Detect reversals from velocity with a deadband.

M. Connection to the Asimov research platform

Joint and ankle calibration should include reversals, load conditions and temperature ranges relevant to the intended physical skill.

N. Completion record

Record your chosen case, predicted trend, measured fitted_scale, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 67, Commissioning evidence as a dependency graph. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 67 - Commissioning evidence as a dependency graph

A. The engineering question

Readiness is not a single checklist checkbox.

B. Concepts and mathematical model

A higher-level test depends on lower-level evidence, and an updated hardware or software revision can invalidate earlier results. A dependency graph makes that structure explicit. This laboratory evaluates synthetic evidence records and calculates which capabilities are blocked when one prerequisite becomes stale. The software audit cannot independently verify a physical test; it can only enforce completeness and revision consistency of recorded evidence.

gate passes iff its own record is PASS and revision-matched and every prerequisite recursively passes.

C. Real-world meaning and limits

Physical readiness remains no-go until actual tests and reviewed evidence replace synthetic fixtures and all required protections pass.

D. Architecture and data flow

Configuration (stale_gate_index) -> numerical model -> passing_gate_count, blocked_gate_count, supported_limb_ready -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

stale_gate_index selects a synthetic record with an old revision; -1 leaves every artificial record current.

Parameter	Nominal value	Interpretation
stale_gate_index	2	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/67_commissioning_evidence.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only stale_gate_index changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/67_commissioning_evidence.py --case 0
```

```
.venv-book/bin/python expanded_lab/modules/67_commissioning_evidence.py --case 3

.venv-book/bin/python expanded_lab/run.py commissioning_evidence --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/67_commissioning_evidence.py

```
"""Commissioning evidence as a dependency graph. Numerical teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    graph = {
        "inspection": [],
        "isolation": ["inspection"],
        "single_joint": ["isolation"],
        "calibration": ["single_joint"],
        "supported_limb": ["calibration"],
        "physical_skill": ["supported_limb"],
        "walking": ["physical_skill"],
    }
    stale_index = int(p["stale_gate_index"])
    keys = list(graph)
    records = {k: dict(status="PASS", revision="r1") for k in keys}
    if stale_index >= 0:
        records[keys[stale_index]]["revision"] = "r0"
    memo = {}

    def passes(name):
        if name in memo:
            return memo[name]
        own = (
            records[name]["status"] == "PASS"
            and records[name]["revision"] == "r1"
        )
        memo[name] = own and all(passes(dep) for dep in graph[name])
        return memo[name]

    results = {k: passes(k) for k in keys}
    metrics = dict(
        passing_gate_count=sum(results.values()),
        blocked_gate_count=sum(not v for v in results.values()),
        supported_limb_ready=float(results["supported_limb"]),
        physical_skill_ready=float(results["physical_skill"]),
        walking_ready=float(results["walking"]),
    )
    derivation = [
        f"The synthetic gate order is {keys}.",
        f"Stale gate index is {stale_index}; records must match revision r1.",
        "Evaluate each gate recursively, requiring all prerequisite evidence.",
        f"Results are {results}.",
        f"Total passing count is {sum(results.values())}; blocked count is {sum(not v for v
    in results.values())}.",
    ]
    notes = [
        "All PASS records here are artificial test fixtures, not physical commissioning
```

```
    evidence.",
        "A real record should include procedure, threshold rationale, instrument, raw
    artifact, date, reviewer and exact configuration.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"stale_gate_index": 2}
    parameters["stale_gate_index"] = [-1, 0, 1, 2, 4, 6][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Recursive evaluation propagates a lower-level invalidation upward. Memoization avoids repeated work; a production graph should also reject dependency cycles.

I. Expected behavior and six worked cases

The controlled variable is stale_gate_index. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

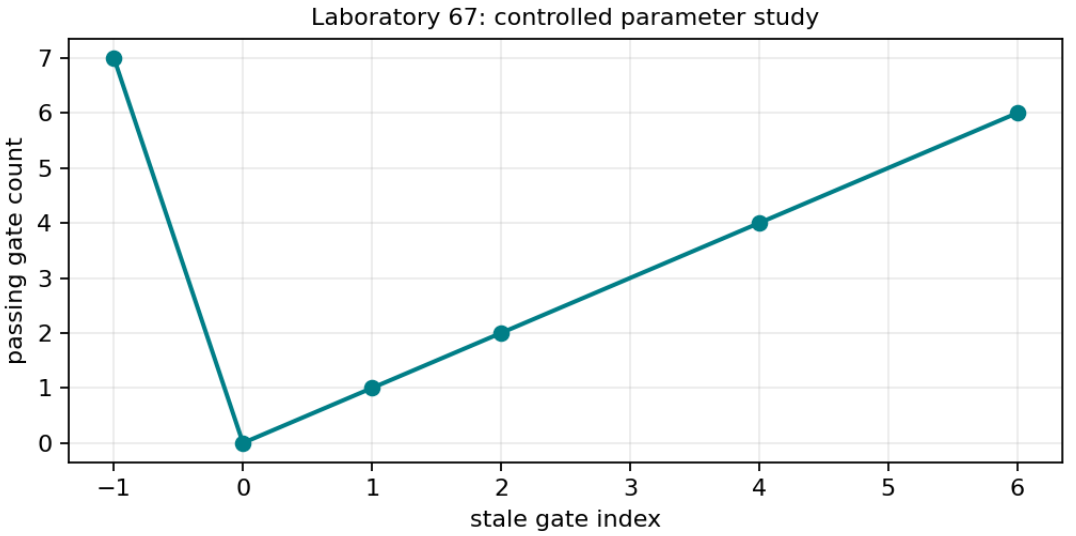


Figure L67. passing gate count as stale_gate_index changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	stale_gate_index	passing gate count
0	-1	7
1	0	0
2	1	1
3	2	2
4	4	4
5	6	6

Worked case 0: stale_gate_index = -1

This case keeps the nominal configuration except for stale_gate_index=-1. The full input is {"stale_gate_index": -1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
passing gate count	7

Computed quantity	Numerical result
blocked gate count	0
supported limb ready	1
physical skill ready	1
walking ready	1

1. The synthetic gate order is ['inspection', 'isolation', 'single_joint', 'calibration', 'supported_limb', 'physical_skill', 'walking'].
2. Stale gate index is -1; records must match revision r1.
3. Evaluate each gate recursively, requiring all prerequisite evidence.
4. Results are {'inspection': True, 'isolation': True, 'single_joint': True, 'calibration': True, 'supported_limb': True, 'physical_skill': True, 'walking': True}.
5. Total passing count is 7; blocked count is 0.

Worked case 1: stale_gate_index = 0

This case keeps the nominal configuration except for stale_gate_index=0. The full input is {"stale_gate_index": 0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
passing gate count	0
blocked gate count	7
supported limb ready	0
physical skill ready	0
walking ready	0

1. The synthetic gate order is ['inspection', 'isolation', 'single_joint', 'calibration', 'supported_limb', 'physical_skill', 'walking'].
2. Stale gate index is 0; records must match revision r1.

3. Evaluate each gate recursively, requiring all prerequisite evidence.
4. Results are {'inspection': False, 'isolation': False, 'single_joint': False, 'calibration': False, 'supported_limb': False, 'physical_skill': False, 'walking': False}.
5. Total passing count is 0; blocked count is 7.

Worked case 2: stale_gate_index = 1

This case keeps the nominal configuration except for stale_gate_index=1. The full input is {"stale_gate_index": 1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
passing gate count	1
blocked gate count	6
supported limb ready	0
physical skill ready	0
walking ready	0

1. The synthetic gate order is ['inspection', 'isolation', 'single_joint', 'calibration', 'supported_limb', 'physical_skill', 'walking'].
2. Stale gate index is 1; records must match revision r1.
3. Evaluate each gate recursively, requiring all prerequisite evidence.
4. Results are {'inspection': True, 'isolation': False, 'single_joint': False, 'calibration': False, 'supported_limb': False, 'physical_skill': False, 'walking': False}.
5. Total passing count is 1; blocked count is 6.

Worked case 3: stale_gate_index = 2

This case keeps the nominal configuration except for stale_gate_index=2. The full input is {"stale_gate_index": 2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
passing gate count	2
blocked gate count	5
supported limb ready	0
physical skill ready	0
walking ready	0

1. The synthetic gate order is ['inspection', 'isolation', 'single_joint', 'calibration', 'supported_limb', 'physical_skill', 'walking'].
2. Stale gate index is 2; records must match revision r1.
3. Evaluate each gate recursively, requiring all prerequisite evidence.
4. Results are {'inspection': True, 'isolation': True, 'single_joint': False, 'calibration': False, 'supported_limb': False, 'physical_skill': False, 'walking': False}.
5. Total passing count is 2; blocked count is 5.

Worked case 4: stale_gate_index = 4

This case keeps the nominal configuration except for stale_gate_index=4. The full input is {"stale_gate_index": 4}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
passing gate count	4
blocked gate count	3
supported limb ready	0
physical skill ready	0
walking ready	0

1. The synthetic gate order is ['inspection', 'isolation', 'single_joint', 'calibration', 'supported_limb', 'physical_skill', 'walking'].

2. Stale gate index is 4; records must match revision r1.
3. Evaluate each gate recursively, requiring all prerequisite evidence.
4. Results are {'inspection': True, 'isolation': True, 'single_joint': True, 'calibration': True, 'supported_limb': False, 'physical_skill': False, 'walking': False}.
5. Total passing count is 4; blocked count is 3.

Worked case 5: stale_gate_index = 6

This case keeps the nominal configuration except for stale_gate_index=6. The full input is {"stale_gate_index": 6}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
passing gate count	6
blocked gate count	1
supported limb ready	1
physical skill ready	1
walking ready	0

1. The synthetic gate order is ['inspection', 'isolation', 'single_joint', 'calibration', 'supported_limb', 'physical_skill', 'walking'].
2. Stale gate index is 6; records must match revision r1.
3. Evaluate each gate recursively, requiring all prerequisite evidence.
4. Results are {'inspection': True, 'isolation': True, 'single_joint': True, 'calibration': True, 'supported_limb': True, 'physical_skill': True, 'walking': False}.
5. Total passing count is 6; blocked count is 1.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

All PASS records here are artificial test fixtures, not physical commissioning evidence.

A real record should include procedure, threshold rationale, instrument, raw artifact, date, reviewer and exact configuration.

K. Failure modes and troubleshooting

Treating a completed checkbox as permanent across revisions; skipping independent evidence verification; inferring physical readiness from a software result.

L. Practical exercises and extension tasks

1. Add model, firmware and calibration hashes to each record.
2. Detect dependency cycles and missing gate definitions.
3. Generate a minimal retest plan after one configuration change.

M. Connection to the Asimov research platform

Physical readiness remains no-go until actual tests and reviewed evidence replace synthetic fixtures and all required protections pass.

N. Completion record

Record your chosen case, predicted trend, measured passing_gate_count, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 68, Immutable experiment manifests and configuration identity. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 68 - Immutable experiment manifests and configuration identity

A. The engineering question

Reproducibility depends on more than a Git commit.

B. Concepts and mathematical model

Model files, calibration, policy weights, normalizer parameters, environment packages and test conditions can change behavior independently. A manifest should identify every relevant artifact and distinguish content identity from a filename. Canonical serialization and cryptographic hashes help detect changes, but they do not establish that an artifact is correct or trusted. This laboratory changes one configuration field and traces the resulting manifest identity.

artifact_hash=SHA256(bytes); manifest_hash=SHA256(canonical JSON containing artifact hashes and experiment parameters).

C. Real-world meaning and limits

Physical deployment records need exact model, firmware, calibration and policy identities plus a documented rollback bundle.

D. Architecture and data flow

Configuration (action_scale) -> numerical model -> content_identity_changed, changed_field_count, action_scale -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

action_scale is the normalized-to-joint action multiplier in this manifest example; it is not a physical command recommendation.

Parameter	Nominal value	Interpretation
action_scale	0.15	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/68_manifest_hashes.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only action_scale changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/68_manifest_hashes.py --case 0

.venv-book/bin/python expanded_lab/modules/68_manifest_hashes.py --case 3

.venv-book/bin/python expanded_lab/run.py manifest_hashes --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/68_manifest_hashes.py

```
"""Immutable experiment manifests and configuration identity. Numerical teaching experiment
    only."""

import math
import json
import argparse
import numpy as np

import hashlib
import json


def digest(value):
    encoded = json.dumps(value, sort_keys=True, separators=(",", ":")).encode()
    return hashlib.sha256(encoded).hexdigest()


def experiment(p):
    base = dict(
        model="sha256:model_example",
        policy="sha256:policy_example",
        calibration="calibration_r1",
        seed=7,
        control_period=0.02,
        action_scale=0.15,
    )
    modified = dict(base)
    modified["action_scale"] = p["action_scale"]
    h0 = digest(base)
    h1 = digest(modified)
    reordered = {k: modified[k] for k in reversed(list(modified))}
    assert digest(reordered) == h1
    changed = [k for k in base if base[k] != modified[k]]
    metrics = dict(
        content_identity_changed=float(h0 != h1),
        changed_field_count=len(changed),
        action_scale=p["action_scale"],
        scale_ratio_to_baseline=p["action_scale"] / 0.15,
    )
    derivation = [
        f"Baseline fields are {base}.",
        f'Change action_scale to {p["action_scale"]:g}; changed fields are {changed}.',
        f"Baseline canonical manifest hash is {h0}.",
        f"Candidate canonical manifest hash is {h1}.",
        "Reordering dictionary insertion order preserves the canonical hash, verified by
    assertion.",
    ]
    notes = [
        "The model/policy hash strings are explicit fictional labels in this unit exercise;
    a real manifest must hash actual file bytes.",
        "A hash detects content changes but does not prove provenance, authenticity,
    compatibility or safety.",
    ]
```

```
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"action_scale": 0.15}
    parameters["action_scale"] = [0.05, 0.1, 0.15, 0.2, 0.25, 0.3][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Canonical serialization avoids irrelevant formatting differences. The changed-field list explains the semantic change rather than leaving reviewers with only two opaque hashes.

I. Expected behavior and six worked cases

The controlled variable is action_scale. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

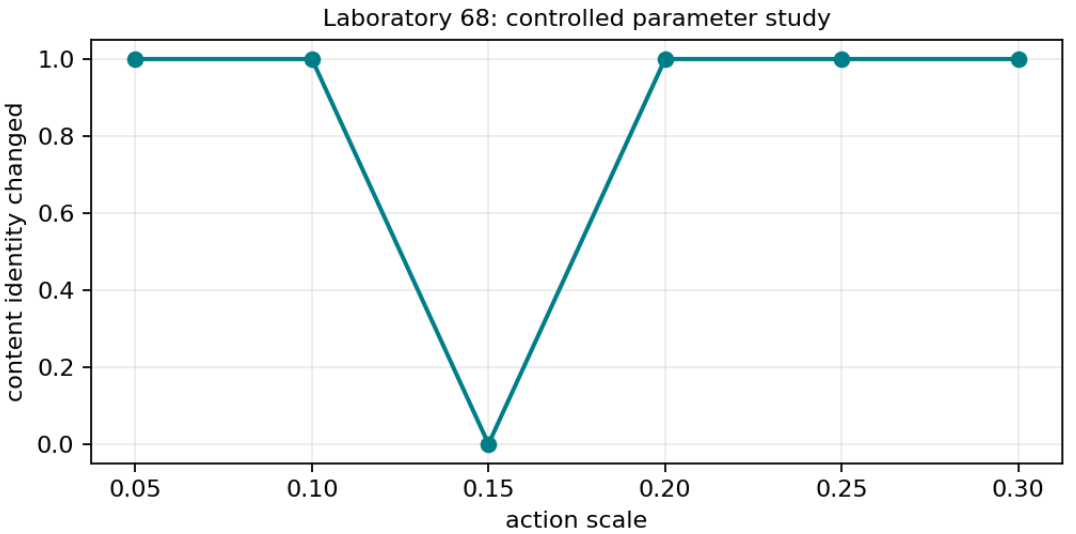


Figure L68. content identity changed as action_scale changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	action_scale	content identity changed
0	0.05	1
1	0.1	1
2	0.15	0
3	0.2	1
4	0.25	1
5	0.3	1

Worked case 0: action_scale = 0.05

This case keeps the nominal configuration except for action_scale=0.05. The full input is {"action_scale": 0.05}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
content identity changed	1
changed field count	1
action scale	0.05
scale ratio to baseline	0.33333333

1. Baseline fields are {'model': 'sha256:model_example', 'policy': 'sha256:policy_example', 'calibration': 'calibration_r1', 'seed': 7, 'control_period': 0.02, 'action_scale': 0.15}.
2. Change action_scale to 0.05; changed fields are ['action_scale'].
3. Baseline canonical manifest hash is `b3ffecbc531968289a1458991e696fb02a5890cfcec06fe38f7b3c688db3d93d`.
4. Candidate canonical manifest hash is `d92cea4610c05499c458db26ae8e692d0c46c79f06d10c2530d1698ff1414c35`.

5. Reordering dictionary insertion order preserves the canonical hash, verified by assertion.

Worked case 1: action_scale = 0.1

This case keeps the nominal configuration except for action_scale=0.1. The full input is {"action_scale": 0.1}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
content identity changed	1
changed field count	1
action scale	0.1
scale ratio to baseline	0.66666667

1. Baseline fields are {'model': 'sha256:model_example', 'policy': 'sha256:policy_example', 'calibration': 'calibration_r1', 'seed': 7, 'control_period': 0.02, 'action_scale': 0.15}.
2. Change action_scale to 0.1; changed fields are ['action_scale'].
3. Baseline canonical manifest hash is `b3ffecbc531968289a1458991e696fb02a5890cfcec06fe38f7b3c688db3d93d`.
4. Candidate canonical manifest hash is `e7250d2ab441055d5893d940a042266adda063850cc944a393a2d74dd54abf8c`.
5. Reordering dictionary insertion order preserves the canonical hash, verified by assertion.

Worked case 2: action_scale = 0.15

This case keeps the nominal configuration except for action_scale=0.15. The full input is {"action_scale": 0.15}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
content identity changed	0
changed field count	0

Computed quantity	Numerical result
action scale	0.15
scale ratio to baseline	1

1. Baseline fields are {'model': 'sha256:model_example', 'policy': 'sha256:policy_example', 'calibration': 'calibration_r1', 'seed': 7, 'control_period': 0.02, 'action_scale': 0.15}.
2. Change action_scale to 0.15; changed fields are [].
3. Baseline canonical manifest hash is `b3ffecbc531968289a1458991e696fb02a5890cfcec06fe38f7b3c688db3d93d`.
4. Candidate canonical manifest hash is `b3ffecbc531968289a1458991e696fb02a5890cfcec06fe38f7b3c688db3d93d`.
5. Reordering dictionary insertion order preserves the canonical hash, verified by assertion.

Worked case 3: action_scale = 0.2

This case keeps the nominal configuration except for action_scale=0.2. The full input is {"action_scale": 0.2}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
content identity changed	1
changed field count	1
action scale	0.2
scale ratio to baseline	1.33333333

1. Baseline fields are {'model': 'sha256:model_example', 'policy': 'sha256:policy_example', 'calibration': 'calibration_r1', 'seed': 7, 'control_period': 0.02, 'action_scale': 0.15}.
2. Change action_scale to 0.2; changed fields are ['action_scale'].
3. Baseline canonical manifest hash is `b3ffecbc531968289a1458991e696fb02a5890cfcec06fe38f7b3c688db3d93d`.

4. Candidate canonical manifest hash is `0a5cd509dcf3fae033998f9f3586297bf17339f56f376b1905b93239ee9f1de0`.
5. Reordering dictionary insertion order preserves the canonical hash, verified by assertion.

Worked case 4: action_scale = 0.25

This case keeps the nominal configuration except for action_scale=0.25. The full input is {"action_scale": 0.25}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
content identity changed	1
changed field count	1
action scale	0.25
scale ratio to baseline	1.66666667

1. Baseline fields are {'model': 'sha256:model_example', 'policy': 'sha256:policy_example', 'calibration': 'calibration_r1', 'seed': 7, 'control_period': 0.02, 'action_scale': 0.15}.
2. Change action_scale to 0.25; changed fields are ['action_scale'].
3. Baseline canonical manifest hash is `b3ffecbc531968289a1458991e696fb02a5890cfcec06fe38f7b3c688db3d93d`.
4. Candidate canonical manifest hash is `db7d39a86bf237b483c39e3f66dae19308764904e79becb4947b4ed888db5f38`.
5. Reordering dictionary insertion order preserves the canonical hash, verified by assertion.

Worked case 5: action_scale = 0.3

This case keeps the nominal configuration except for action_scale=0.3. The full input is {"action_scale": 0.3}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
content identity changed	1
changed field count	1
action scale	0.3
scale ratio to baseline	2

1. Baseline fields are {'model': 'sha256:model_example', 'policy': 'sha256:policy_example', 'calibration': 'calibration_r1', 'seed': 7, 'control_period': 0.02, 'action_scale': 0.15}.
2. Change action_scale to 0.3; changed fields are ['action_scale'].
3. Baseline canonical manifest hash is `b3ffecbc531968289a1458991e696fb02a5890cfcec06fe38f7b3c688db3d93d`.
4. Candidate canonical manifest hash is `1ed29eb5869766785dd5c359085a7b513fea7defddeda163fcaf5c090e8516a4`.
5. Reordering dictionary insertion order preserves the canonical hash, verified by assertion.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The model/policy hash strings are explicit fictional labels in this unit exercise; a real manifest must hash actual file bytes.

A hash detects content changes but does not prove provenance, authenticity, compatibility or safety.

K. Failure modes and troubleshooting

Hashing a path instead of file bytes; forgetting calibration/normalizer artifacts; accepting an unchanged source commit as proof of unchanged behavior.

L. Practical exercises and extension tasks

1. Build a manifest from actual supplied lab files.
2. Include the resolved package lock and model commit.
3. Verify a release bundle before allowing offline replay or activation.

M. Connection to the Asimov research platform

Physical deployment records need exact model, firmware, calibration and policy identities plus a documented rollback bundle.

N. Completion record

Record your chosen case, predicted trend, measured content_identity_changed, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 69, Fault injection, diagnostic coverage and remaining blind spots. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 69 - Fault injection, diagnostic coverage and remaining blind spots

A. The engineering question

A protection claim should name the faults considered and the evidence for detecting and responding to them.

B. Concepts and mathematical model

Injecting a convenient subset of faults can produce a misleading coverage percentage. Some failures are independent, some share causes, and some defeat both the controller and monitor. This laboratory builds a synthetic fault campaign with known detection probabilities to show the difference between observed detection, fault prevalence weighting and untested categories. The numbers are teaching probabilities, not measured reliability data.

observed coverage=detected/injected; weighted residual probability=sum(fault_weight*(1-detection_probability)) under stated assumptions.

C. Real-world meaning and limits

Actual protective systems need hazard-specific design and independent evidence; this campaign is a method exercise only.

D. Architecture and data flow

Configuration (sensor_fault_detection) -> numerical model -> observed_detection_fraction, expected_weighted_detection, expected_undetected_fraction -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

sensor_fault_detection is an assumed probability for a synthetic sensor-bias fault; other campaign parameters remain fixed.

Parameter	Nominal value	Interpretation
sensor_fault_detection	0.5	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.

2. Open /data/Robotics_Project/expanded_lab/modules/69_failure_modes.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only sensor_fault_detection changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project

.venv-book/bin/python expanded_lab/modules/69_failure_modes.py --case 0

.venv-book/bin/python expanded_lab/modules/69_failure_modes.py --case 3

.venv-book/bin/python expanded_lab/run.py failure_modes --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/69_failure_modes.py

```
"""Fault injection, diagnostic coverage and remaining blind spots. Numerical teaching
    experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    rng = np.random.default_rng(97)
    n = 2000
    coverage = p["sensor_fault_detection"]
    categories = ["stale", "range", "sensor_bias", "power_loss"]
    weights = np.array([0.3, 0.2, 0.3, 0.2])
    detection = np.array([0.99, 0.99, coverage, 0.4])
    chosen = rng.choice(4, n, p=weights)
    detected = rng.random(n) < detection[chosen]
    observed = detected.mean()
    expected = float(weights @ detection)
    counts = {
        name: int(np.count_nonzero(chosen == i))
        for i, name in enumerate(categories)
    }
    metrics = dict(
        observed_detection_fraction=observed,
        expected_weighted_detection=expected,
        expected_undetected_fraction=1 - expected,
        undetected_fault_count=np.count_nonzero(~detected),
        sensor_bias_detection_assumption=coverage,
    )
    derivation = [
        f"Inject {n} synthetic faults with category counts {counts}.",
        f"Assumed category detection probabilities are {detection.tolist()}.",
        f"Weighted expected detection is {expected:.6f}.",
        f"The seeded campaign observes detection fraction {observed:.6f}.",
        f"Expected residual undetected fraction is {1-expected:.6f} under the chosen
    artificial mixture.",
    ]
    notes = [
        "The category mixture and detection probabilities are invented experiment inputs and
     cannot establish a physical safety integrity level.",
        "Detection alone is incomplete: the protective response, common-cause failures and
    consequences must also be evaluated.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"sensor_fault_detection": 0.5}
```

```
parameters["sensor_fault_detection"] = [0.0, 0.25, 0.5, 0.75, 0.9, 0.99][
    args.case
]
metrics, derivation, notes = experiment(parameters)
result = dict(
    parameters=parameters,
    metrics={k: float(v) for k, v in metrics.items()},
    derivation=derivation,
    notes=notes,
)
print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

Report per-category injections as well as aggregate coverage. A frequent easy-to-detect fault can dominate the aggregate while a serious blind spot remains.

I. Expected behavior and six worked cases

The controlled variable is sensor_fault_detection. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

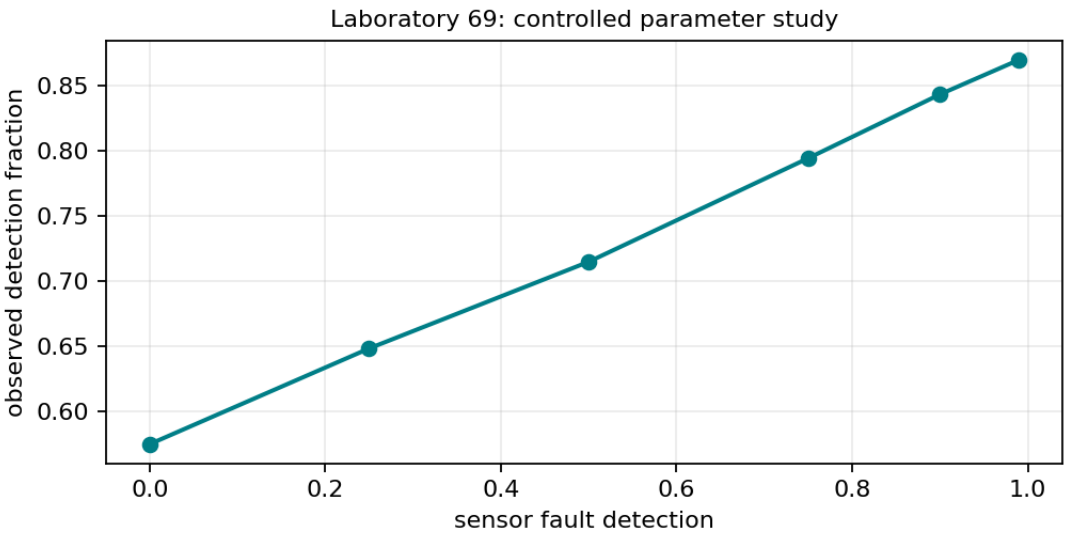


Figure L69. observed detection fraction as sensor_fault_detection changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	sensor_fault_detection	observed detection fraction
0	0	0.575
1	0.25	0.6485
2	0.5	0.715
3	0.75	0.7945
4	0.9	0.8435
5	0.99	0.87

Worked case 0: sensor_fault_detection = 0

This case keeps the nominal configuration except for sensor_fault_detection=0. The full input is {"sensor_fault_detection": 0.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
observed detection fraction	0.575
expected weighted detection	0.575
expected undetected fraction	0.425
undetected fault count	850
sensor bias detection assumption	0

1. Inject 2000 synthetic faults with category counts {'stale': 568, 'range': 410, 'sensor_bias': 598, 'power_loss': 424}.
2. Assumed category detection probabilities are [0.99, 0.99, 0.0, 0.4].
3. Weighted expected detection is 0.575000.
4. The seeded campaign observes detection fraction 0.575000.
5. Expected residual undetected fraction is 0.425000 under the chosen artificial mixture.

Worked case 1: sensor_fault_detection = 0.25

This case keeps the nominal configuration except for sensor_fault_detection=0.25. The full input is {"sensor_fault_detection": 0.25}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
observed detection fraction	0.6485
expected weighted detection	0.65
expected undetected fraction	0.35
undetected fault count	703
sensor bias detection assumption	0.25

1. Inject 2000 synthetic faults with category counts {'stale': 568, 'range': 410, 'sensor_bias': 598, 'power_loss': 424}.
2. Assumed category detection probabilities are [0.99, 0.99, 0.25, 0.4].
3. Weighted expected detection is 0.650000.
4. The seeded campaign observes detection fraction 0.648500.
5. Expected residual undetected fraction is 0.350000 under the chosen artificial mixture.

Worked case 2: sensor_fault_detection = 0.5

This case keeps the nominal configuration except for sensor_fault_detection=0.5. The full input is {"sensor_fault_detection": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
observed detection fraction	0.715
expected weighted detection	0.725
expected undetected fraction	0.275

Computed quantity	Numerical result
undetected fault count	570
sensor bias detection assumption	0.5

1. Inject 2000 synthetic faults with category counts {'stale': 568, 'range': 410, 'sensor_bias': 598, 'power_loss': 424}.
2. Assumed category detection probabilities are [0.99, 0.99, 0.5, 0.4].
3. Weighted expected detection is 0.725000.
4. The seeded campaign observes detection fraction 0.715000.
5. Expected residual undetected fraction is 0.275000 under the chosen artificial mixture.

Worked case 3: sensor_fault_detection = 0.75

This case keeps the nominal configuration except for sensor_fault_detection=0.75. The full input is {"sensor_fault_detection": 0.75}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
observed detection fraction	0.7945
expected weighted detection	0.8
expected undetected fraction	0.2
undetected fault count	411
sensor bias detection assumption	0.75

1. Inject 2000 synthetic faults with category counts {'stale': 568, 'range': 410, 'sensor_bias': 598, 'power_loss': 424}.
2. Assumed category detection probabilities are [0.99, 0.99, 0.75, 0.4].
3. Weighted expected detection is 0.800000.
4. The seeded campaign observes detection fraction 0.794500.

5. Expected residual undetected fraction is 0.200000 under the chosen artificial mixture.

Worked case 4: sensor_fault_detection = 0.9

This case keeps the nominal configuration except for sensor_fault_detection=0.9. The full input is {"sensor_fault_detection": 0.9}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
observed detection fraction	0.8435
expected weighted detection	0.845
expected undetected fraction	0.155
undetected fault count	313
sensor bias detection assumption	0.9

1. Inject 2000 synthetic faults with category counts {'stale': 568, 'range': 410, 'sensor_bias': 598, 'power_loss': 424}.
2. Assumed category detection probabilities are [0.99, 0.99, 0.9, 0.4].
3. Weighted expected detection is 0.845000.
4. The seeded campaign observes detection fraction 0.843500.
5. Expected residual undetected fraction is 0.155000 under the chosen artificial mixture.

Worked case 5: sensor_fault_detection = 0.99

This case keeps the nominal configuration except for sensor_fault_detection=0.99. The full input is {"sensor_fault_detection": 0.99}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
observed detection fraction	0.87

Computed quantity	Numerical result
expected weighted detection	0.872
expected undetected fraction	0.128
undetected fault count	260
sensor bias detection assumption	0.99

1. Inject 2000 synthetic faults with category counts {'stale': 568, 'range': 410, 'sensor_bias': 598, 'power_loss': 424}.
2. Assumed category detection probabilities are [0.99, 0.99, 0.99, 0.4].
3. Weighted expected detection is 0.872000.
4. The seeded campaign observes detection fraction 0.870000.
5. Expected residual undetected fraction is 0.128000 under the chosen artificial mixture.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

The category mixture and detection probabilities are invented experiment inputs and cannot establish a physical safety integrity level.

Detection alone is incomplete: the protective response, common-cause failures and consequences must also be evaluated.

K. Failure modes and troubleshooting

Equating detection with prevention; claiming reliability from invented fault weights; ignoring a common power or software failure affecting both monitor and controller.

L. Practical exercises and extension tasks

1. Calculate coverage per fault category with confidence intervals.
2. Inject a common-cause monitor failure.

3. Add response-time and physical-outcome checks to the campaign definition.

M. Connection to the Asimov research platform

Actual protective systems need hazard-specific design and independent evidence; this campaign is a method exercise only.

N. Completion record

Record your chosen case, predicted trend, measured observed_detection_fraction, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Continue with Laboratory 70, An integrated synthetic joint with perception, gating and fault recovery. Keep the present result as a reference fixture for diagnosing changes in the larger stack.

Laboratory 70 - An integrated synthetic joint with perception, gating and fault recovery

A. The engineering question

The final numerical capstone combines a simple observation, a bounded skill request, a feedback controller, telemetry and a latched fault.

B. Concepts and mathematical model

It remains a fictional one-axis plant, so every equation and state transition is inspectable. The point is to test the complete chain and demonstrate that a fault blocks further motion authority. A physical backend is intentionally absent. Transfer to the supported Asimov simulation is the next integration exercise; transfer to hardware requires its separate commissioning evidence.

plant: I*q_ddot=u-b*q_dot; skill target is bounded; stale command age latches fault; latched state disables new actuation in the synthetic plant.

C. Real-world meaning and limits

Reuse the architecture and tests, not the synthetic plant parameters or zero-torque stop assumption, when building a physical demonstration.

D. Architecture and data flow

Configuration (dropout_start) -> numerical model -> final_position_rad, peak_applied_torque_Nm, tracking_rmse_rad -> assertions and experiment record.

E. Prerequisites and parameter contract

Use the Python environment from Lesson 0. This laboratory needs NumPy and the standard library; it has no simulator, ROS or physical-transport dependency. The complete module is printed below and saved at the specified path. A fixed seed is part of the experiment where randomness is present.

dropout_start is the time in seconds when synthetic command refresh ends; five seconds means no dropout during the four-second run.

Parameter	Nominal value	Interpretation
dropout_start	2	Defined by the explicit chapter model and parameter contract

F. Implementation and experiment protocol

1. Read the model equation and identify which assumptions separate it from the physical robot.
2. Open /data/Robotics_Project/expanded_lab/modules/70_capstone.py and locate the experiment function before running it.
3. Run case 0, then case 3 and case 5. Only dropout_start changes in the published study.
4. Predict the direction of change in the first metric before comparing with the worked solutions.
5. Re-run the complete six-case study and preserve results with the source and environment identity.

```
cd /data/Robotics_Project
```

```
.venv-book/bin/python expanded_lab/modules/70_capstone.py --case 0
```

```
.venv-book/bin/python expanded_lab/modules/70_capstone.py --case 3

.venv-book/bin/python expanded_lab/run.py capstone --case 5
```

The supplied .venv-book was used to prepare the numerical results. On another machine, use your own Lesson 0 environment instead of assuming that directory has been copied or is portable.

G. Complete executable implementation

/data/Robotics_Project/expanded_lab/modules/70_capstone.py

```
"""An integrated synthetic joint with perception, gating and fault recovery. Numerical
    teaching experiment only."""

import math
import json
import argparse
import numpy as np


def experiment(p):
    dt = 0.002
    duration = 4.0
    n = round(duration / dt)
    timeout = 0.1
    q = v = 0.0
    state = "OBSERVE"
    last_sent = 0.0
    fault_time = None
    errors = []
    max_torque = 0.0
    object_visible = True
    drop_at = p["dropout_start"]
    for k in range(n):
        t = k * dt
        if state == "OBSERVE" and object_visible:
            state = "EXECUTE"
        if t < drop_at:
            last_sent = t
        if t - last_sent > timeout and state == "EXECUTE":
            state = "FAULT"
            fault_time = t
        target = 0.05 * (1 - np.cos(2 * np.pi * min(t, 3.0) / 3.0))
        torque = (
            np.clip(8 * (target - q) - 1.2 * v, -0.5, 0.5)
            if state == "EXECUTE"
            else 0.0
        )
        v += dt * (torque - 0.2 * v) / 0.05
        q += dt * v
        errors.append(target - q)
        max_torque = max(max_torque, abs(torque))
    metrics = dict(
        final_position_rad=q,
        peak_applied_torque_Nm=max_torque,
        tracking_rmse_rad=np.sqrt(np.mean(np.array(errors) ** 2)),
        fault_latched=float(state == "FAULT"),
        fault_time_s=-1 if fault_time is None else fault_time,
    )
    derivation = [
        "The synthetic object observation enables only a predefined 0.1 rad peak
    supported-style gesture.",
        f"Command refresh stops at t={drop_at:g} s; timeout is {timeout:g} s.",
        "A stale command latches FAULT and removes torque authority in this unloaded
```

```
    numerical plant.",
        f"Final state is {state}; fault time is {fault_time}.",
        f'Peak torque is {max_torque:.6f} Nm; final position is {q:.6f} rad and RMSE is
    {metrics["tracking_rmse_rad"]:.6f} rad.',
    ]
    notes = [
        "Zero torque after a fault is suitable only for this explicitly unloaded damped
    synthetic plant; a loaded physical robot needs a reviewed support/stop strategy.",
        "No automatic healthy-message transition leaves the fault state, and no physical bus
     is opened.",
    ]
    return metrics, derivation, notes


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--case", type=int, default=0, choices=range(6))
    args = parser.parse_args()
    parameters = {"dropout_start": 2.0}
    parameters["dropout_start"] = [0.5, 1.0, 1.5, 2.0, 3.0, 5.0][args.case]
    metrics, derivation, notes = experiment(parameters)
    result = dict(
        parameters=parameters,
        metrics={k: float(v) for k, v in metrics.items()},
        derivation=derivation,
        notes=notes,
    )
    print(json.dumps(result, indent=2))
```

H. Walkthrough and design decisions

The state machine owns actuation permission, while the controller computes a limited command only in EXECUTE. The plant continues evolving after a fault, exposing why disabling output is not the same as freezing mechanics.

I. Expected behavior and six worked cases

The controlled variable is dropout_start. All other parameters stay at their declared nominal values. The following graph and table summarize the executed cases. Values are computed, not hand-entered estimates. Read the derivation for each case to distinguish a useful result from a numerically valid but physically inapplicable one.

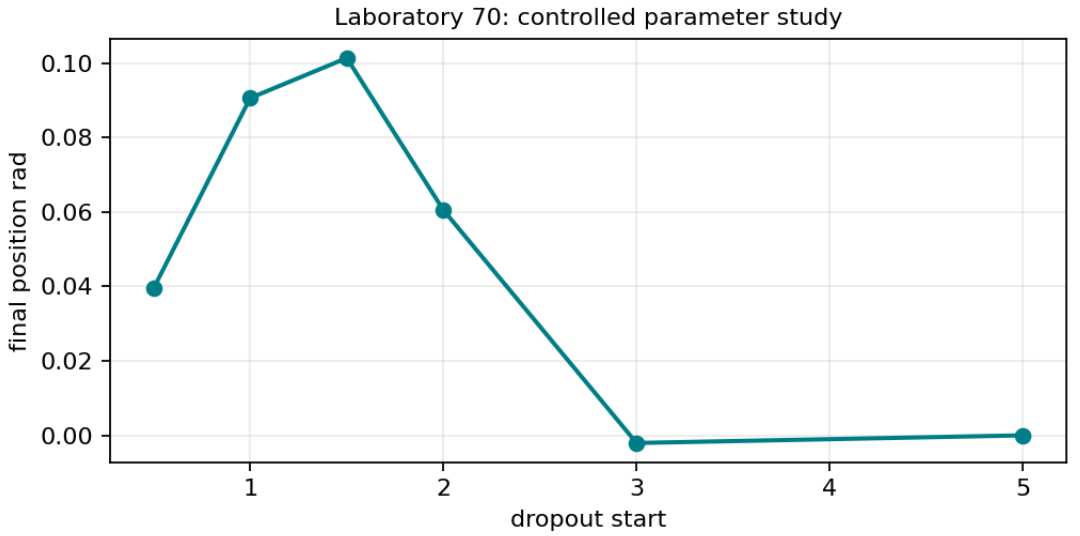


Figure L70. final position rad as dropout_start changes. Axes use the quantities and units defined in this laboratory, not undocumented hardware ratings.

Case	dropout_start	final position rad
0	0.5	0.03950397
1	1	0.09063787
2	1.5	0.10140148
3	2	0.06055133
4	3	-0.00205178
5	5	2.0539015e-06

Worked case 0: dropout_start = 0.5

This case keeps the nominal configuration except for dropout_start=0.5. The full input is {"dropout_start": 0.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final position rad	0.03950397

Computed quantity	Numerical result
peak applied torque Nm	0.02219542
tracking rmse rad	0.0366836
fault latched	1
fault time s	0.6

1. The synthetic object observation enables only a predefined 0.1 rad peak supported-style gesture.
2. Command refresh stops at t=0.5 s; timeout is 0.1 s.
3. A stale command latches FAULT and removes torque authority in this unloaded numerical plant.
4. Final state is FAULT; fault time is 0.6.
5. Peak torque is 0.022195 Nm; final position is 0.039504 rad and RMSE is 0.036684 rad.

Worked case 1: dropout_start = 1

This case keeps the nominal configuration except for dropout_start=1. The full input is {"dropout_start": 1.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final position rad	0.09063787
peak applied torque Nm	0.0226765
tracking rmse rad	0.05663015
fault latched	1
fault time s	1.098

1. The synthetic object observation enables only a predefined 0.1 rad peak supported-style gesture.
2. Command refresh stops at t=1 s; timeout is 0.1 s.

3. A stale command latches FAULT and removes torque authority in this unloaded numerical plant.
4. Final state is FAULT; fault time is 1.098.
5. Peak torque is 0.022677 Nm; final position is 0.090638 rad and RMSE is 0.056630 rad.

Worked case 2: dropout_start = 1.5

This case keeps the nominal configuration except for dropout_start=1.5. The full input is {"dropout_start": 1.5}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final position rad	0.10140148
peak applied torque Nm	0.0226765
tracking rmse rad	0.06385488
fault latched	1
fault time s	1.598

1. The synthetic object observation enables only a predefined 0.1 rad peak supported-style gesture.
2. Command refresh stops at t=1.5 s; timeout is 0.1 s.
3. A stale command latches FAULT and removes torque authority in this unloaded numerical plant.
4. Final state is FAULT; fault time is 1.598.
5. Peak torque is 0.022677 Nm; final position is 0.101401 rad and RMSE is 0.063855 rad.

Worked case 3: dropout_start = 2

This case keeps the nominal configuration except for dropout_start=2. The full input is {"dropout_start": 2.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final position rad	0.06055133
peak applied torque Nm	0.0226765
tracking rmse rad	0.03767868
fault latched	1
fault time s	2.1

1. The synthetic object observation enables only a predefined 0.1 rad peak supported-style gesture.
2. Command refresh stops at t=2 s; timeout is 0.1 s.
3. A stale command latches FAULT and removes torque authority in this unloaded numerical plant.
4. Final state is FAULT; fault time is 2.1.
5. Peak torque is 0.022677 Nm; final position is 0.060551 rad and RMSE is 0.037679 rad.

Worked case 4: dropout_start = 3

This case keeps the nominal configuration except for dropout_start=3. The full input is {"dropout_start": 3.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final position rad	-0.00205178
peak applied torque Nm	0.02271077
tracking rmse rad	0.01074721
fault latched	1
fault time s	3.1

1. The synthetic object observation enables only a predefined 0.1 rad peak supported-style gesture.

2. Command refresh stops at t=3 s; timeout is 0.1 s.
3. A stale command latches FAULT and removes torque authority in this unloaded numerical plant.
4. Final state is FAULT; fault time is 3.1.
5. Peak torque is 0.022711 Nm; final position is -0.002052 rad and RMSE is 0.010747 rad.

Worked case 5: dropout_start = 5

This case keeps the nominal configuration except for dropout_start=5. The full input is {"dropout_start": 5.0}. Follow the calculation in order, then compare all reported quantities rather than selecting one favorable metric.

Computed quantity	Numerical result
final position rad	2.0539015e-06
peak applied torque Nm	0.02271077
tracking rmse rad	0.01072771
fault latched	0
fault time s	-1

1. The synthetic object observation enables only a predefined 0.1 rad peak supported-style gesture.
2. Command refresh stops at t=5 s; timeout is 0.1 s.
3. A stale command latches FAULT and removes torque authority in this unloaded numerical plant.
4. Final state is EXECUTE; fault time is None.
5. Peak torque is 0.022711 Nm; final position is 0.000002 rad and RMSE is 0.010728 rad.

J. Validation and independent reasoning

All six published configurations were executed with finite numerical results. Assertions in the module check the stated identities or state-machine invariants where applicable. Finiteness is only a basic execution check; the independent reasoning below is the substantive part of validation.

Zero torque after a fault is suitable only for this explicitly unloaded damped synthetic plant; a loaded physical robot needs a reviewed support/stop strategy.

No automatic healthy-message transition leaves the fault state, and no physical bus is opened.

K. Failure modes and troubleshooting

Assuming a stopped software loop stops physical motion; evaluating only the no-fault case; declaring completion without measured feedback.

L. Practical exercises and extension tasks

1. Add feedback-based completion and a separate CANCEL state.
2. Replay the same target into the supported Asimov simulator.
3. Inject nonfinite observations and reversed encoder direction, then verify rejection and diagnostic evidence.

M. Connection to the Asimov research platform

Reuse the architecture and tests, not the synthetic plant parameters or zero-torque stop assumption, when building a physical demonstration.

N. Completion record

Record your chosen case, predicted trend, measured final_position_rad, the largest discrepancy from the worked result, and the explanation for that discrepancy. Preserve the code revision, environment identity and raw JSON. For a hardware connection, name the measurement or verified document that would replace each teaching assumption before use.

O. Next laboratory connection

Return to the supported Asimov perception-to-gesture project with the quantitative checks from these laboratories. Release only the capabilities supported by actual simulation and commissioning evidence.

Appendices – Reproducibility and evidence

Appendix A. Complete numerical and fault tests

The regression suite intentionally tests meaningful model and interface properties rather than mirroring every line of implementation. Run it from humanoid_lab with the pinned environment. These tests do not prove physical safety, successful ROS integration, learned walking or real camera performance.

File: /data/Robotics_Project/humanoid_lab/tests/test_core.py

/data/Robotics_Project/humanoid_lab/tests/test_core.py

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import numpy as np
import pytest
import mujoco as mj
from model import load, command, addresses
from kinematics import planar, solve
from balance import margin_rectangle

def test_asimov_contract():
    m, d = load()
    assert (m.nq, m.nv, m.nu) == (25, 25, 25)
    assert np.isclose(m.opt.timestep, .002)
    assert np.all(m.body_mass[1:] > 0)
    assert np.isfinite(d.qpos).all()

def test_bounded_feedback_and_rejection():
    m, d = load()
    with pytest.raises(ValueError):
        command(m, d, np.full(m.nu, np.nan))
    ids, qi, vi = addresses(m)
    for _ in range(500):
        command(m, d, m.qpos0[qi]); mj.mj_step(m, d)
    assert np.isfinite(d.qpos).all()
    assert np.max(np.abs(d.qvel)) < 50
    assert np.all(np.abs(d.ctrl) <= np.max(np.abs(m.actuator_ctrlrange), axis=1)*.25+1e-8)

def test_planar_geometry_and_ik():
    assert np.allclose(planar([0, 0]), [.5, 0])
    m, d = load()
    target = d.xpos[m.body('left_wrist_yaw_link').id].copy()+[.02, 0, .01]
    ok, error = solve(m, d, target)
    assert ok and error < .002

def test_static_margin_sign():
    assert margin_rectangle([0, 0], -.1, .1, -.1, .1) == .1
    assert margin_rectangle([.2, 0], -.1, .1, -.1, .1) < 0
```

File: /data/Robotics_Project/humanoid_lab/tests/test_contracts.py

/data/Robotics_Project/humanoid_lab/tests/test_contracts.py

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import numpy as np
import pytest
from safety import Gate
from perception import detect
from learning import HumanoidTask

def test_fault_latches():
    g = Gate(np.array([-1.]), np.array([1.]))
    with pytest.raises(RuntimeError, match='stale'):
        g.validate(1., 0., 1, [0.], [0.], .01)
    with pytest.raises(RuntimeError):
        g.validate(1., 1., 2, [0.], [0.], .01)

def test_detection_does_not_invent_depth():
    im = np.zeros((100, 100, 3), np.uint8)
    assert detect(im) is None
    im[20:40, 30:60, 1] = 255
    assert detect(im)['depth_m'] is None

def test_seeded_reset_and_action_contract():
    env = HumanoidTask()
    a, _ = env.reset(seed=3); b, _ = env.reset(seed=3)
    assert np.array_equal(a, b)
    obs, reward, done, truncated, info = env.step(np.zeros(1))
    assert env.observation_space.contains(obs)
    assert np.isfinite(reward) and not done
    with pytest.raises(ValueError):
        env.step(np.array([np.nan]))
```

Appendix B. Measured preparation results

The following results were observed while preparing this edition, using the pinned checkout and CPU teaching environment. They are evidence about the included software, not a performance guarantee for every workstation. The initial full test suite passed seven tests. The final preparation suite and artifact checks are recorded in validation_report.json beside the manuscript.

Experiment	Measured outcome	Interpretation
Supported model load	nq=25, nv=25, nu=25	Adapter and actual asset agree on scalar coordinates

Experiment	Measured outcome	Interpretation
Ten-second joint run	5,000 samples, 2 ms physics step	Headless stepping/logging executes
Elbow tracking	RMSE about .01213 rad	Below the .1 rad exercise threshold
Elbow speed	Peak about .15508 rad/s	Measured simulation result, not a physical limit
Elbow torque command	Peak about 1.22718 Nm	Applied teaching command, not measured physical torque
Nearby wrist position IK	Residual about .00006504 m	Numerical position-only solve converges
Synthetic perception	Centroid near (210,110), unknown depth retained	Correct positive test and explicit uncertainty
Supported object gesture	1,500 physics steps, COMPLETE_SIMULATION	Perception-to-control software chain executes
Passive unsupported trials	Five of five fell around .50-.52 s	No standing capability demonstrated by passive model
PPO smoke training	2,048 timesteps, one training seed	Training pipeline check only
PPO held-out reach evaluation	Mean final error about .03744 rad over 20 seeds	Does not pass proposed .03 rad gate
Policy export	Traced actor saved; sample and 100 rollout comparisons pass	External action clipping retained; hardware replay still required
Procurement audit	EUR 189 known subtotal; many unknowns	Full-robot feasibility remains unestablished
Commissioning audit	No-go with NOT_TESTED defaults	No physical commissioning claimed
ROS integration	Source supplied and syntax inspected	Runtime integration requires reader-side Jazzy testing

The policy evaluation's zero fall fraction is trivial for supported reach. It is not a standing metric. The environment's observations and inner-loop model are idealized. Do not deploy the smoke-trained actor on hardware.

Rendered model and telemetry

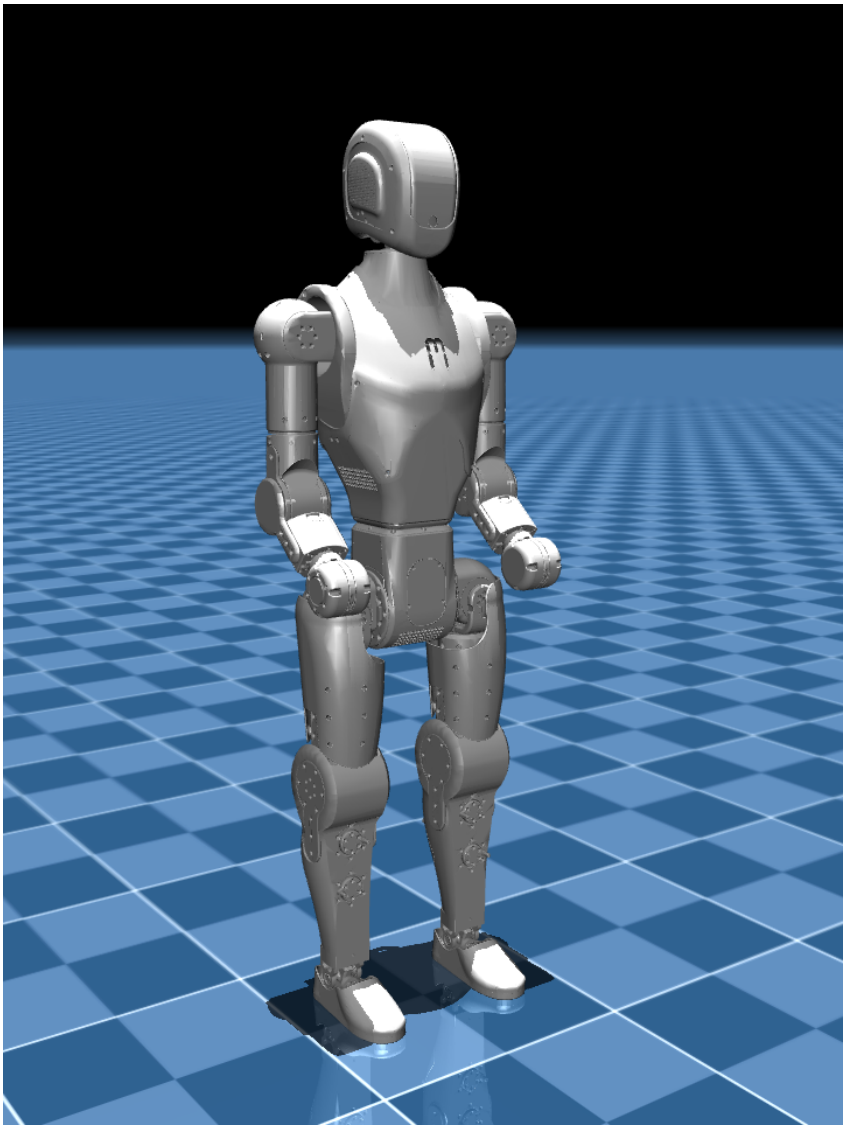


Figure B1. Actual pinned Asimov geometry rendered in the teaching fixture. Geometry comes from the upstream source under its published hardware license; this is a simulation view, not a photograph of an assembled robot.

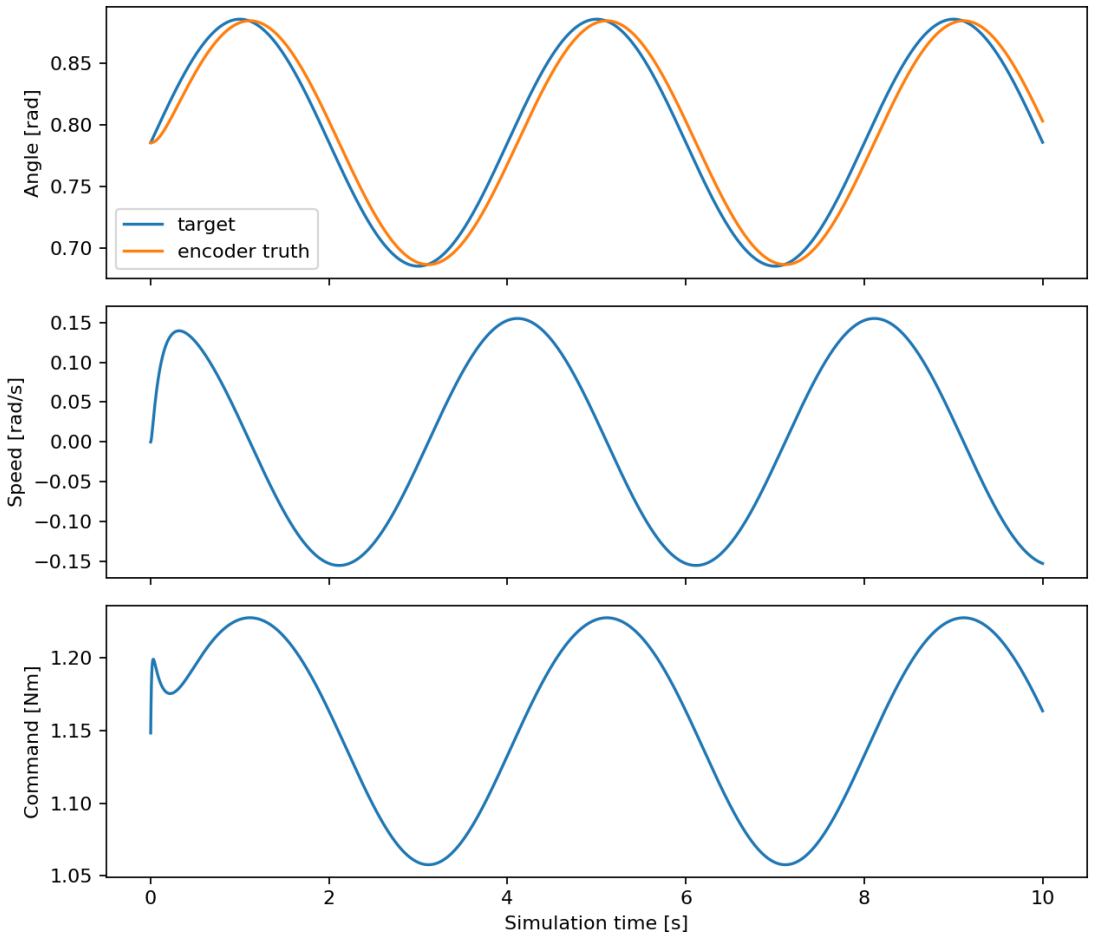


Figure B2. Ten simulated seconds of supported elbow tracking, speed and applied teaching torque. The plots are generated from the supplied CSV.

Appendix C. Live telemetry and model-rendering utilities

For a simple live dashboard, start python simulate.py --seconds 60 in one terminal and python live_dashboard.py in a second terminal from the same environment and directory. The simulator flushes its CSV periodically; the dashboard reads a recent window at 4 Hz. It is an observability path and must never become part of the actuator feedback deadline. This live GUI utility is supplied for desktop use; the preparation run verified the saved plot and offscreen model render, not an interactive desktop session.

File: /data/Robotics_Project/humanoid_lab/live_dashboard.py

/data/Robotics_Project/humanoid_lab/live_dashboard.py

```
"""Read the simulation CSV while simulate.py writes it; run in another terminal."""
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from model import ROOT
fig, axes = plt.subplots(2, 1, sharex=True)

def update(_):
    path = ROOT/'logs/joints.csv'
    if not path.exists(): return
    try:
        data = np.genfromtxt(path, delimiter=',', names=True, invalid_raise=False)
        if data.size < 2: return
        data = data[-2500:]
        axes[0].clear(); axes[1].clear()
        axes[0].plot(data['t'], data['target'], label='target')
        axes[0].plot(data['t'], data['q'], label='feedback')
        axes[0].set_ylabel('Joint angle [rad]'); axes[0].legend()
        axes[1].plot(data['t'], data['command_Nm'])
        axes[1].set_ylabel('Command [Nm]'); axes[1].set_xlabel('Simulation time [s]')
    except (ValueError, OSError, IndexError):
        return  # Writer may be between records; try next display interval.

animation = FuncAnimation(fig, update, interval=250, cache_frame_data=False)
plt.show()
```

For an offscreen geometry check on a machine with EGL support, run MUJOCO_GL=egl python render_model.py. Set the graphics backend before MuJoCo imports. If EGL fails, check the host graphics configuration; headless physics remains available. The rendered geometry was successfully produced during preparation.

File: /data/Robotics_Project/humanoid_lab/render_model.py

/data/Robotics_Project/humanoid_lab/render_model.py

```
"""Offscreen inspection image; set MUJOCO_GL before importing MuJoCo."""
import mujoco as mj
import matplotlib.pyplot as plt
from model import ROOT, load
m, d = load()
with mj.Renderer(m, height=960, width=720) as renderer:
    camera = mj.MjvCamera()
    mj.mjv_defaultCamera(camera)
    camera.lookat[:] = [0, 0, .65]
    camera.distance = 1.9
    camera.azimuth = 145
    camera.elevation = -10
    options = mj.MjvOption()
    options.geomgroup[3] = 0
    renderer.update_scene(d, camera=camera, scene_option=options)
    plt.imsave(ROOT/'logs/asimov_model.png', renderer.render())
print(ROOT/'logs/asimov_model.png')
```

Appendix D. Mermaid architecture sources

The diagrams in the book are rendered for PDF legibility. These Mermaid sources provide editable text for your project documentation.

```
flowchart TB
  Human[Human instruction] --> Planner[Task planner]
  Planner --> Skills[Validated skills]
  Skills --> Safety[Safety supervisor]
  Safety --> Controller[Joint or whole-body controller]
  Policy[Policy inference] --> Safety
  Sim[MuJoCo simulation] <--> Bridge[ROS 2 interfaces]
  Bridge <--> Controller
  Controller --> Hardware[Verified hardware abstraction]
  Hardware --> Embedded[Embedded watchdogs and drivers]
  Embedded --> Actuators[Physical actuators]
  Sensors[Sensors and estimator] --> Safety
  Sensors --> Planner
  Actuators --> Sensors
  Isolation[Independent power isolation and support] --> Actuators
  Bridge --> Logs[Telemetry and experiment records]

sequenceDiagram
  participant Human
  participant Planner
  participant Gate
  participant Controller
  participant Plant
  Human->>Planner: Request permitted skill
  Planner->>Gate: Skill name, bounded parameters, timestamp
  Gate->>Gate: Verify readiness, age, state and limits
```

```
  Gate->>Controller: Authorized trajectory
  Controller->>Plant: Bounded repeated commands
  Plant-->>Controller: Timestamped feedback
  Controller-->>Gate: Tracking and health
  Gate-->>Planner: Completed or latched fault
  Planner-->>Human: Measured result

flowchart LR
  Sensors[Encoder IMU camera contact] --> Samples[Timestamp and frame]
  Samples --> Estimator[State estimator]
  Estimator --> Control[Local feedback controller]
  Samples --> Decimator[Telemetry decimation]
  Decimator --> Dashboard[Dashboard and logs]
  Control --> Driver[Bounded actuator command]
```

Appendix E. Units, equations and glossary

Term	Meaning	Practical check
rad	Radian angle	pi rad is 180 degrees
Nm	Torque	10 N through .1 m gives 1 Nm
Hz	Updates per second	500 Hz corresponds to 2 ms
Frame	Named origin and axes	State source and destination on every transform
Quaternion	Four-number unit rotation representation	Normalize; verify component order
COM	Center of mass	Mass-weighted position; not a physical sensor
COP	Center of pressure	Requires contact-load distribution
ZMP	Zero Moment Point	Depends on model/contact assumptions
URDF	Unified Robot Description Format	Geometry/joint tree does not ensure actuator semantics
MJCF	MuJoCo model format	Contains simulator-specific dynamics/settings
QoS	Quality of service	Delivery choice does not remove message age
PD/PID	Feedback on error, rate, and optionally integral	Units and saturation must be explicit
PPO	Proximal Policy Optimization	Algorithm choice does not validate reward design
HIL	Hardware in the loop	Specify which outputs are physically inhibited
Watchdog	Deadline/freshness monitor	Test the actual fault response
E-stop	Emergency stop function	Requires reviewed physical outcome and isolation design

Term	Meaning	Practical check
Sim-to-real	Transfer of tested behavior across model/hardware differences	Identification, calibration, replay and protected tests

Useful relationships: angular speed in rad/s is RPM*2*pi/60; linear lever torque is force*perpendicular distance; mechanical power is tau*dq; electrical power is V*I; center of mass is sum(m_i*c_i)/sum(m_i); rigid transform composition follows frame order. A unit conversion error is a control bug even when the program runs without exceptions.

Appendix F. Source register and update policy

Primary sources were checked for this edition on 25 October 2026. URLs may redirect or change; the local commit snapshot is the authority for file paths used by the code. Sources support specific platform or software facts, while lesson explanations, exercise parameters and proposed gates are original instructional design. Dated price observations must be rechecked before spending.

[S1] Menlo Research Asimov 1 official repository and README. Platform overview, public kit asking price, licenses and public roadmap. https://github.com/menloresearch/asimov-1/

[S2] Pinned simulation README, commit 96a6807c163e4e6cabdc124eefcc45b0ae2cb57a. Variant paths and upstream MuJoCo validation version. https://raw.githubusercontent.com/menloresearch/asimov-1/96a6807c163e4e6cabdc124eefcc45b0ae2cb57a/sim-model/README.md

[S3] Menlo mechanical system tour. Joint layout, parallel ankle distinction and document revision caveats. https://docs.menlo.ai/asimov/1/overview/system-tour/mechanical

[S4] Menlo software system tour. Software roles, release-dependent capabilities, control modes, and DAMP distinction. https://docs.menlo.ai/asimov/1/overview/system-tour/software

[S5] Menlo electrical system tour. Compute/power architecture and unresolved public CAN mapping note. https://docs.menlo.ai/asimov/1/overview/system-tour/electrical

[S6] Official ROS 2 documentation release table, raw source. Support dates including Jazzy and Lyrical. https://raw.githubusercontent.com/ros2/ros2_documentation/rolling/source/Releases.rst

[S7] Official ROS 2 Jazzy Ubuntu installation and repository configuration source. https://raw.githubusercontent.com/ros2/ros2_documentation/jazzy/source/Installation/Ubuntu-Install-Debs.rst Repository bootstrap details: https://raw.githubusercontent.com/ros2/ros2_documentation/jazzy/source/Installation/_Apt-Repositories.rst

[S8] Gazebo Harmonic documentation, recommended ROS/Gazebo pairings. https://gazebosim.org/docs/harmonic/ros_installation/

[S9] NVIDIA Isaac Sim official requirements. Read the selected release's compatibility matrix before GPU purchasing or installing Isaac Lab. https://docs.isaacsim.omniverse.nvidia.com/latest/installation/requirements.html

[S10] MuJoCo official Python documentation. Model/state APIs, stepping, viewer modes and macOS viewer requirement. https://mujoco.readthedocs.io/en/stable/python.html

[S11] Stable-Baselines3 official custom-environment documentation. Interface expectations and environment checker. https://stable-baselines3.readthedocs.io/en/master/guide/custom_env.html

[S12] Gymnasium official environment API. Reset, step, seeded randomness, termination and truncation. https://gymnasium.farama.org/api/env/

[S13] Menlo part-organization and actuator checklist. Actuator-family counts used in procurement ledger. https://docs.menlo.ai/asimov/1/build/assembly-preparations

[S14] BerryBase direct Raspberry Pi 5 8 GB product listing. EUR 189.00 VAT-inclusive observed on the preparation date, shipping extra; this is not a complete-robot quotation. https://www.berrybase.de/raspberry-pi-5-8gb-ram

[S15] Menlo official actuator-ID assignment procedure. Confirm applicability to the purchased hardware release. https://docs.menlo.ai/asimov/1/build/assembly-preparations/assign-actuator-ids

[S16] Direct upstream GitHub tree snapshot and local checkout used for the asset audit. https://api.github.com/repos/menloresearch/asimov-1/git/trees/96a6807c163e4e6cabdc124eefcc45b0ae2cb57a?recursive=1

[S17] Original 25-DOF MJCF source at the pinned revision; authoritative for this lab's modeled limits and names. https://github.com/menloresearch/asimov-1/blob/96a6807c163e4e6cabdc124eefcc45b0ae2cb57a/sim-model/asimov_1_25dof/xml/asimov_1_25dof.xml

When updating, preserve this edition and its logs. Record the new commit and diff model joints, limits, masses, collision geometry, sensors and actuator semantics. Recreate the environment, rerun regression tests, then revisit any physical calibration or safety assumption affected by the change. A newer version is an experiment to validate before it becomes a release.

Appendix G. Expanded-edition numerical validation

The extended numerical laboratory comprises 70 distinct standalone modules and 420 published parameter cases. All cases were executed during preparation with finite reported metrics. The combined pytest suite passed 101 tests, including the original humanoid numerical tests, module-to-catalog parity for every six-case study, and independent reference or cross-case checks for conservation, frame round trips, singularity behavior, timing, uncertainty, budgets, message integrity and fault handling.

These results establish execution and selected numerical properties of the stated models. They do not certify physical structures, electrical designs, protective systems, real sensors or locomotion. The synthetic capstone, packet format, fault probabilities, costs and thresholds remain explicitly educational. Actual hardware commissioning status is unchanged: no physical actuator was connected by this work.

The full semantic test file is printed here so the reader can distinguish a meaningful physical or numerical invariant from a simple assertion that a program returned a finite number. The 70 parametrized tests compare each formatted standalone module against its catalog reference over all six declared configurations. The additional tests check independent properties. Passing these tests is a prerequisite for accepting edited numerical fixtures, not a substitute for validation against real measurements.

/data/Robotics_Project/expanded_lab/test_textbook.py

```
"""Semantic numerical checks and published-module parity for the textbook."""
from pathlib import Path
import runpy
import sys
sys.path.insert(0,str(Path(__file__).parent))
import numpy as np
import pytest
from catalog import load_catalog,evaluate

SPECS={s['key']:s for s in load_catalog()}

def result(key,**changes):
    s=SPECS[key];p=dict(s['params']);p.update(changes)
    return evaluate(s,p)['metrics']

@pytest.mark.parametrize('key',list(SPECS))
def test_standalone_module_matches_catalog_in_all_six_cases(key):
    s=SPECS[key]
    path=next((Path(__file__).parent/'modules').glob('*_'+key+'.py'))
    function=runpy.run_path(str(path))['experiment']
    for v in s['values']:
        p=dict(s['params']);p[s['vary']]=v
        reference=evaluate(s,p)['metrics'];actual,_,_=function(p)
        assert actual.keys()==reference.keys()
        assert all(np.isclose(actual[k],reference[k],rtol=1e-10,atol=1e-12)
                   for k in reference)

def test_transmission_conserves_ideal_power():
    x=result('units',efficiency=1.)
    assert np.isclose(x['input_power_W'],x['output_power_W'])

def test_force_parallel_to_lever_has_zero_torque():
    assert abs(result('vectors',angle_deg=0.)['axis_torque_Nm'])<1e-12

def test_rotation_and_transform_roundtrips():
    assert result('rotations')['roundtrip_error_m']<1e-12
    assert result('transforms')['roundtrip_error_m']<1e-12

def test_quaternion_sign_equivalence():
    assert result('quaternions')['sign_equivalence_error_rad']<1e-6

def test_derivative_accuracy_reference():
    assert result('jacobian',epsilon=1e-5)['derivative_error']<1e-8

def test_implicit_decay_stays_positive_where_explicit_is_unstable():
    x=result('integration',dt=.25)
    assert x['explicit_amplification']>1 and x['implicit_final']>0

def test_payload_shifts_com_forward():
    assert result('center_mass',payload_mass=4.)['com_x_m']>result('center_mass',payload_mas
    s=0.)['com_x_m']

def test_parallel_axis_increases_inertia():
    assert result('inertia',offset=.2)['Izz_kg_m2']>result('inertia',offset=0.)['Izz_kg_m2']
```

```
def test_unreachable_ik_is_not_reported_success():
    assert result('planar_ik',target_x=.6)['reachable']==0
    assert result('planar_ik',target_x=.35)['residual_m']<1e-10

def test_damping_trades_joint_step_for_residual():
    low=result('damped_ik',damping=.001);high=result('damped_ik',damping=.1)
    assert high['joint_step_norm_rad']<low['joint_step_norm_rad']
    assert high['predicted_residual_m']>low['predicted_residual_m']

def test_friction_impossibility_is_visible():
    assert result('friction_cone',mu=.1)['force_margin_N']<0

def test_minimum_jerk_duration_scaling():
    a=result('minimum_jerk',duration=1.);b=result('minimum_jerk',duration=2.)
    assert np.isclose(a['peak_speed_rad_s'],2*b['peak_speed_rad_s'])
    assert np.isclose(a['peak_acceleration_rad_s2'],4*b['peak_acceleration_rad_s2'])

def test_larger_speed_consumes_stopping_margin():
    assert result('rate_limits',speed=.8)['remaining_travel_margin_rad']<result('rate_limits
    ',speed=.2)['remaining_travel_margin_rad']

def test_increased_loss_raises_temperature():
    assert result('thermal',loss_power=40.)['final_temperature_C']>result('thermal',loss_pow
    er=5.)['final_temperature_C']

def test_projection_roundtrip_and_depth_uncertainty():
    assert result('pinhole')['roundtrip_error_m']<1e-12
    assert result('stereo_depth',true_depth=2.)['depth_std_m']>result('stereo_depth',true_de
    pth=1.)['depth_std_m']

def test_queue_freshness_under_overload():
    x=result('queue_age',consumer_period=.05)
    assert x['fifo_max_age_s']>x['latest_max_age_s']+.5

def test_synthetic_codec_detects_single_bit_faults():
    assert result('packet_codec')['rejection_fraction']==1.

def test_faults_do_not_auto_resume():
    assert result('lifecycle')['automatic_resume_count']==0
    assert result('skill_planner',amplitude=.2)['primary_request_approved']==0

def test_matching_replay_has_no_disagreement():
    assert result('replay',scale_error_factor=1.)['maximum_absolute_action_error']==0

def test_budget_counts_growth_and_contingency():
    x=result('landed_budget',motor_count=12)
    assert x['budget_margin_EUR']<0 and x['reserve_EUR']>0

def test_cable_heat_is_quadratic_in_current():
    a=result('cable_loss',current=5.);b=result('cable_loss',current=10.)
    assert np.isclose(b['conductor_heat_W'],4*a['conductor_heat_W'])

def test_stale_evidence_blocks_downstream_gate():
    assert result('commissioning_evidence',stale_gate_index=0)['walking_ready']==0

def test_manifest_order_does_not_hide_semantic_change():
    assert result('manifest_hashes',action_scale=.15)['content_identity_changed']==0
```

```
    assert result('manifest_hashes',action_scale=.2)['content_identity_changed']==1

def test_capstone_detects_dropout_but_not_uninterrupted_stream():
    assert result('capstone',dropout_start=1.)['fault_latched']==1
    assert result('capstone',dropout_start=5.)['fault_latched']==0
```

Run the complete numerical validation and the original humanoid checks with:

```
cd /data/Robotics_Project
.venv-book/bin/python -m pytest -q expanded_lab/test_textbook.py humanoid_lab/tests
.venv-book/bin/python expanded_lab/run.py test_all --output
    expanded_lab/results/recheck.json
```

For reproducibility, retain expanded_lab/results/textbook_cases.json, the standalone module sources, the environment lock from the practical project and the book's validation manifest. An experiment's graphs are derived from those numerical cases. Do not edit a graph's data independently of its source experiment.

Appendix H. Additional primary learning resources

The extended equations and code are original teaching implementations. The following primary resources support deeper study and alternative reference implementations. They do not establish hardware compatibility for the Asimov build.

[T1] Kevin M. Lynch and Frank C. Park, Modern Robotics, Northwestern University video supplement on homogeneous transformations. Use it to compare frame notation, composition and inversion with the transform laboratory. https://modernrobotics.northwestern.edu/nu-gm-book-resource/3-3-1-homogeneous-transformation-matrices/

[T2] Modern Robotics, Northwestern University video supplement on the space Jacobian. The spatial-twist Jacobian discussed there is distinct from the two-dimensional endpoint-position Jacobian used in the small numerical laboratory. Verify the represented velocity, frame and column convention before comparing implementations. https://modernrobotics.northwestern.edu/nu-gm-book-resource/5-1-1-space-jacobian/

[T3] John Schulman and colleagues, Proximal Policy Optimization Algorithms, 2017. The paper supplies the algorithmic context for the clipped-surrogate

exercise; that exercise implements only the stated batch objective, not the complete training algorithm. https://arxiv.org/abs/1707.06347

[T4] SciPy official Rotation API reference. This provides a library implementation for rotations and highlights component-order conventions. The extended NumPy laboratories deliberately implement small formulas directly for instruction and do not add SciPy as a dependency. https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.transform.Rotation.html

Use established numerical and robotics libraries when building a larger system after you understand their assumptions. A hand-written educational routine is useful for learning and regression fixtures, but does not automatically become the best production implementation.